[野火]LwIP 应用开发实战指南
—基于野火 STM32

EmbedFire
野火电子

2021 年 01 月 20 日

Contents

关于本项目 **1**
关于野火 **2**
开源共享，共同进步 2
联系方式 2
TODO 和悬赏任务 **3**
第 1 章 网络协议简介 **4**
1.1 常用网络协议 4
1.2 网络协议的分层模型 6
1.3 协议层报文间的封装与拆封 7
第 2 章 LwIP 简介 **9**
2.1 LwIP 的优缺点 9
2.2 LwIP 的文件说明 11
2.2.1 如何获取 LwIP 源码文件 11
2.2.2 LwIP 文件说明 14
2.3 查看 LwIP 的说明文档 19
2.4 使用 vscode 查看源码 26
2.4.1 查看文件中的符号列表（函数列表） 26
2.4.2 函数定义跳转 28
2.5 LwIP 源码里的 example 29
2.6 LwIP 的三种编程接口 31
2.6.1 RAW/Callback API 31
2.6.2 NETCONN API 32
2.6.3 SOCKET API 33
第 3 章 开发平台介绍 **35**
3.1 以太网简介 35
3.1.1 PHY 层 35
3.1.2 MAC 子层 37

3.2 STM32 的 ETH 外设 . . . 39
3.3 MII 和 RMII 接口 . . . 40
3.4 PHY：LAN8720A . . . 44
3.5 硬件设计 . . . 47
3.6 软件设计 . . . 48
3.6.1 获取 STM32 的裸机工程模板 . . . 48
3.6.2 添加 bsp_eth.c 与 bsp_eth.h . . . 48
3.6.3 修改 stm32f4xx_hal_conf.h 文件 . . . 58
第 4 章 LwIP 的网络接口管理 . . . 62
4.1 netif 结构体 . . . 62
4.2 netif 使用 . . . 68
4.3 与 netif 相关的底层函数 . . . 75
4.4 ethernetif.c 文件内容 . . . 77
4.4.1 ethernetif 数据结构 . . . 77
4.4.2 ethernetif_init() . . . 77
4.4.3 low_level_init() . . . 79
第 5 章 LwIP 的内存管理 . . . 82
5.1 几种内存分配策略 . . . 82
5.1.1 固定大小的内存块 . . . 82
5.1.2 可变长度分配 . . . 83
5.2 动态内存池（POOL） . . . 84
5.2.1 内存池的预处理 . . . 84
5.2.2 内存池的初始化 . . . 92
5.2.3 内存分配 . . . 94
5.2.4 内存释放 . . . 96
5.3 动态内存堆 . . . 98
5.3.1 内存堆的组织结构 . . . 98
5.3.2 内存堆初始化 . . . 100
5.3.3 内存分配 . . . 104
5.3.4 内存释放 . . . 110
5.4 使用 C 库的 malloc 和 free 来管理内存 . . . 115

5.5 LwIP 中的配置 116
第 6 章 网络数据包 119
6.1 TCP/IP 协议的分层思想 119
6.2 LwIP 的线程模型 121
6.3 pbuf 结构体说明 122
6.4 pbuf 的类型 123
6.4.1 PBUF_RAM 类型的 pbuf 124
6.4.2 PBUF_POOL 类型的 pbuf 127
6.4.3 PBUF_ROM 和 PBUF_REF 类型 pbuf 128
6.5 pbuf_alloc() 130
6.6 pbuf_free() 136
6.7 其它 pbuf 操作函数 141
6.7.1 pbuf_realloc() 141
6.7.2 pbuf_header() 141
6.7.3 pbuf_take() 141
6.8 网卡中使用的 pbuf 142
6.8.1 low_level_output() 142
6.8.2 low_level_input() 146
6.8.3 ethernetif_input() 150
第 7 章 无操作系统移植 LwIP 151
7.1 将 LwIP 添加到裸机工程 151
7.2 移植头文件 158
7.3 移植网卡驱动 168
7.4 LwIP 时基 182
7.5 协议栈初始化 183
7.6 获取数据包 185
7.6.1 查询方式 186
7.6.2 ping 命令详解 188
7.6.3 中断方式 191
第 8 章 有操作系统移植 LwIP 194
8.1 LwIP 中添加操作系统 194

8.1.1 拷贝 FreeRTOS 源码到工程文件夹 . . . 194
8.1.2 添加 FreeRTOS 源码到工程组文件夹 . . . 195
8.1.3 指定 FreeRTOS 头文件的路径 . . . 197
8.1.4 修改 stm32f4xx_it.c . . . 199
8.2 lwipopts.h 文件需要加入的配置 . . . 203
8.3 sys_arch.c/h 文件的编写 . . . 210
8.4 网卡底层的编写 . . . 225
8.5 协议栈初始化 . . . 229
8.6 移植后使用 ping 测试基本响应 . . . 233
第 9 章 LwIP 一探究竟 . . . 234
9.1 网卡接收数据的流程 . . . 234
9.2 内核超时处理 . . . 235
9.2.1 sys_timeo 结构体与超时链表 . . . 236
9.2.2 注册超时事件 . . . 237
9.2.3 超时检查 . . . 242
9.3 tcpip_thread 线程 . . . 244
9.4 LwIP 中的消息 . . . 247
9.4.1 消息结构 . . . 247
9.4.2 数据包消息 . . . 250
9.4.3 API 消息 . . . 252
9.5 揭开 LwIP 神秘的面纱 . . . 258
第 10 章 ARP 协议 . . . 259
10.1 链路层概述 . . . 259
10.2 MAC 地址的基本概念 . . . 259
10.3 初识 ARP . . . 260
10.4 以太网帧结构 . . . 260
10.5 IP 地址映射为物理地址 . . . 262
10.6 ARP 缓存表 . . . 262
10.7 ARP 缓存表的超时处理 . . . 268
10.8 ARP 报文 . . . 270
10.9 发送 ARP 请求包 . . . 275

10.10 数据包接收流程 279
10.10.1 以太网之数据包接收 279
10.10.2 ARP 数据包处理 284
10.10.3 更新 ARP 缓存表 289
10.11 数据包发送流程 294
10.11.1 etharp_output() 函数 294
10.11.2 etharp_output_to_arp_index() 函数 298
10.11.3 etharp_query() 函数 300
第 11 章 IP 协议 310
11.1 IP 地址 310
11.1.1 概述 310
11.1.2 IP 地址编址 311
11.1.3 特殊 IP 地址 313
11.2 局域网和广域网的概念 314
11.2.1 什么是局域网 314
11.2.2 什么是广域网 315
11.3 网络地址转换（NAT） 316
11.4 IP 数据报 317
11.5 IP 数据报的数据结构 322
11.6 IP 数据报分片 325
11.7 IP 数据报发送 332
11.8 IP 数据报接收 342
第 12 章 网际控制报文协议 ICMP 354
12.1 ICMP 功能简介 354
12.2 ICMP 报文结构 355
12.3 ICMP 报文类型 356
12.3.1 ICMP 差错报告报文 357
12.3.2 ICMP 查询报文 360
12.4 LwIP 中的 ICMP 实现 362
12.4.1 ICMP 报文数据结构 362
12.4.2 发送 ICMP 差错报文 365

12.4.3 处理 ICMP 报文 368
第 13 章 TCP 协议 373
13.1 TCP 服务简介 373
13.2 TCP 的特性 374
13.2.1 连接机制 374
13.2.2 确认与重传 374
13.2.3 缓冲机制 374
13.2.4 全双工通信 375
13.2.5 流量控制 375
13.2.6 差错控制 376
13.2.7 拥塞控制 376
13.3 端口号的概念 376
13.4 TCP 报文段结构 379
13.4.1 TCP 报文段的封装 379
13.4.2 TCP 报文段格式 380
13.5 TCP 连接 385
13.5.1 “三次握手”建立连接 386
13.5.2 “四次挥手”终止连接 391
13.6 TCP 状态 394
13.6.1 LwIP 中定义的 TCP 状态 394
13.6.2 TCP 状态转移 395
13.7 TCP 中的数据结构 398
13.8 窗口的概念 403
13.8.1 接收窗口 404
13.8.2 发送窗口 404
13.9 TCP 报文段处理 406
13.9.1 报文段缓冲队列 406
13.9.2 TCP 报文段发送 409
13.9.3 TCP 报文段接收 414
第 14 章 UDP 协议 422
14.1 UDP 协议简介 422

14.2 UDP 常用端口号 423
14.3 UDP 报文 423
14.4 UDP 报文的数据结构 425
14.4.1 UDP 报文首部结构体 425
14.4.2 UDP 控制块 425
14.5 UDP 报文发送 429
14.6 UDP 报文接收 432
第 15 章 使用 NETCONN 接口编程 440
15.1 netbuf 结构体 440
15.2 netbuf 相关函数说明 444
15.2.1 netbuf_new() 444
15.2.2 netbuf_delete() 445
15.2.3 netbuf_alloc() 445
15.2.4 netbuf_free() 446
15.2.5 netbuf_ref() 447
15.2.6 netbuf_chain() 448
15.2.7 netbuf_data() 449
15.2.8 netbuf_next() 与 netbuf_first() 449
15.2.9 netbuf_copy() 451
15.2.10 netbuf_take() 452
15.2.11 其他操作 netbuf 的宏定义 454
15.3 netconn 结构体 454
15.4 netconn 函数接口说明 457
15.4.1 netconn_new() 457
15.4.2 netconn_delete() 459
15.4.3 netconn_getaddr() 461
15.4.4 netconn_bind() 462
15.4.5 netconn_connect() 463
15.4.6 netconn_disconnect() 465
15.4.7 netconn_listen() 465
15.4.8 netconn_accept() 467

15.4.9 netconn_recv() . . . 469
15.4.10 netconn_send() . . . 472
15.4.11 netconn_sendto() . . . 473
15.4.12 netconn_write() . . . 474
15.4.13 netconn_close() . . . 479
15.5 实验 . . . 480
15.5.1 TCP Client . . . 480
15.5.2 TCP Client 实验现象 . . . 489
15.5.3 TCP Server . . . 491
15.5.4 TCP Server 实验现象 . . . 499
15.5.5 UDP . . . 501
15.5.6 UDP 实验现象 . . . 508
第 16 章 使用 Socket 接口编程 . . . 511
16.1 什么是 Socket . . . 511
16.2 LwIP 中的 Socket . . . 511
16.3 Socket API . . . 513
16.3.1 socket() . . . 513
16.3.2 bind() . . . 514
16.3.3 connect() . . . 516
16.3.4 listen() . . . 517
16.3.5 accept() . . . 517
16.3.6 read()、recv()、recvfrom() . . . 518
16.3.7 sendto() . . . 519
16.3.8 send() . . . 519
16.3.9 write() . . . 520
16.3.10 close() . . . 520
16.3.11 ioctl()、ioctlsocket() . . . 521
16.3.12 setsockopt() . . . 522
16.3.13 getsockopt() . . . 523
16.4 实验 . . . 523
16.4.1 TCP Client . . . 523

16.4.2 TCP Server . . . 526
16.4.3 UDP . . . 529
第 17 章 使用 RAW API 接口编程 . . . 533
17.1 RAW API 的 UDP 编程 . . . 533
17.1.1 新建控制块 udp_new() . . . 533
17.1.2 绑定控制块 udp_bind() . . . 534
17.1.3 建立会话 udp_connect() . . . 536
17.1.4 断开会话 udp_disconnect() . . . 538
17.1.5 接收数据 udp_recv() . . . 539
17.1.6 发送数据 udp_send() 与 udp_sendto() . . . 540
17.1.7 删除 UDP 控制块 udp_remove() . . . 542
17.2 RAW API 的 TCP 编程 . . . 543
17.2.1 新建控制块 tcp_new() . . . 543
17.2.2 绑定控制块 tcp_bind() . . . 544
17.2.3 控制块监听 tcp_listen() . . . 546
17.2.4 处理连接 tcp_accept() . . . 549
17.2.5 建立连接 tcp_connect() . . . 549
17.2.6 终止连接 tcp_close() . . . 553
17.2.7 接收数据 tcp_recv() . . . 556
17.2.8 发送数据 tcp_sent() . . . 557
17.2.9 异常处理 tcp_err() . . . 557
17.2.10 周期性回调 tcp_poll() . . . 558
17.2.11 构建报文段 tcp_write() . . . 559
17.2.12 更新接收窗口 tcp_recved() . . . 560
17.3 实验 . . . 561
17.3.1 TCP Client . . . 561
17.3.2 TCP Server . . . 569
17.3.3 UDP . . . 572
第 18 章 使用 JPerf 工具测试网速 . . . 576
18.1 iPerf 与 JPerf . . . 576
18.2 测试网络速度 . . . 576

18.2.1 获取 JPerf 网络测速工具 . . . 576
18.2.2 测试开发板接收速度（NETCONN API） . . . 578
18.2.3 测试开发板接收速度（Socket API） . . . 582
18.2.4 测试开发板发送速度（NETCONN API） . . . 588
18.2.5 测试开发板发送速度（Socket API） . . . 592
18.3 提高 LwIP 网络传输的速度 . . . 596
第 19 章 HTTP 协议 . . . 599
19.1 什么是 HTTP 协议? . . . 599
19.2 URL 与资源 . . . 600
19.3 HTTP 报文 . . . 602
19.4 使用 Postman 获取论坛数据 . . . 604
19.5 使用开发板获取论坛数据 . . . 606
第 20 章 HTTP 服务器 . . . 612
20.1 Hello World 网页 demo . . . 612
20.2 提供网页控制 LED 灯开关的功能 . . . 617
第 21 章 MQTT 协议 . . . 625
21.1 MQTT 协议简介 . . . 625
21.2 MQTT 通信模型 . . . 625
21.3 消息主题与服务质量 . . . 626
21.4 MQTT 控制报文 . . . 627
21.4.1 固定报头 . . . 627
21.4.2 可变报头 . . . 630
21.4.3 有效载荷 . . . 636
21.5 移植 MQTT 协议 . . . 636
21.6 cJSON 移植 . . . 646
第 22 章 连接到百度天工物接入 . . . 651
22.1 物接入简介 . . . 651
22.2 使用 IoT Hub . . . 651
22.2.1 创建计费套餐 . . . 651
22.2.2 创建项目 . . . 652
22.2.3 创建策略 . . . 654

22.2.4 创建身份与创建用户 654
22.2.5 MQTT 软件测试连接 658
22.3 开发板连接 IoT Hub 660
22.4 IoT Hub 的规则引擎 668
22.4.1 什么是规则引擎 668
22.4.2 使用规则引擎 669
22.5 数据可视化 672
22.5.1 IoT Hub 的时序数据库 672
22.5.2 IoT Hub 的物可视 675
第 23 章 连接到阿里云物联 681
23.1 使用阿里云物联 681
23.2 MQTT 软件测试连接 684
23.3 开发板连接阿里云物联 688
23.4 阿里云物联的规则引擎 690
第 24 章 连接到 OneNET 696
24.1 使用 OneNET 696
24.2 测试连接 698
24.3 开发板连接 OneNET 700
24.4 添加数据流 703
24.5 系统主题的发布格式 705
24.6 使用开发板发布数据点 707
24.7 数据可视化 713
联系我们 716
如何参与项目 717
项目源码 717
文档在线阅读与 pdf 717
常见问题 718
版权说明 719

关于本项目

本项目通过 git 开源：

- github 仓库地址：https://github.com/Embedfire-lwip/ebf_lwip_tutorial
- gitee 仓库地址：待添加

点击右侧链接可在线阅读本项目文档：《野火 LwIP 应用开发实战指南：基于 STM32》

本书围绕 LwIP 2.1.2 版本源码进行讲解，讲解 TCP/IP 网络协议栈的基本知识，带领读者走入网络的世界。无论你是学生、嵌入式开发者亦或者是物联网开发者，都可以从本书中学习到网络的知识，了解网络协议栈的处理的思想，本书将深入分析网络协议栈的原理与实现过程，如 ARP、IP、ICMP、TCP、UDP、HTTP、MQTT 等协议。本书还将深入讲解 LwIP 中内存管理、pbuf 数据包、网卡接口管理的原理与实现，并详细介绍 LwIP 的移植过程，读者可以将其移植到无操作系统/有操作系统中的环境中使用。

除此之外，本书还通过理论和实践相结合，使用 LwIP 接入当前最大的云平台，如阿里云、百度云、OneNET，最终达到熟练掌握。在本书中，读者可以熟练掌握 LwIP 中 NETCONN API 与 Socket API 的使用方式，并且掌握多种网络调试工具，如 Wireshark、Postman 以及 MQTT.fx 等。

全书内容循序渐进，不断迭代，在学习的时候建议做到两点：一是不能一味地看书，要把代码和理论结合起来学习，一边看书，一边调试代码。通过执行每一条程序，看看程序的执行流程和执行效果与自己想的是不是一样；二是在每学完一章之后，必须将配套的例程重写一遍（抄也可以，切记不要复制，哪怕是一个符号），举一反三，确保真正理解。推荐阅读

关于野火

开源共享，共同进步

野火在发布第一块 STM32 开发板之初，就喊出 **开源共享，共同进步**的口号，把代码和文档教程都免费提供给用户下载，而我们也一直把这个理念贯穿至今。

目前我们的产品已经包括 *STM32*、*i.MX RT 系列*、*GD32V*、*FPGA*、*Linux*、*emXGUI*、操作系统、网络、下载器等分支，覆盖电子工程应用领域的各种常用技术，其中教学类产品的代码和文档一直保持着开源的姿态发布到网络上，为电子工程师排忧解难，让嵌入式没有难用的技术是我们最大的愿望。

联系方式

- 官网：http://www.embedfire.com
- 论坛：http://www.firebbs.cn
- github 主页：https://github.com/Embedfire
- gitee 主页：https://gitee.com/wildfireteam
- 淘宝：https://yehuosm.tmall.com
- 邮箱：embedfire@embedfire.com
- 电话：0769-33894118

TODO 和悬赏任务

本页记录待改进的事项，以后会在此页发布任务清单，完成任务可获得 **奖励**，敬请期待！

如果您想参与到项目，可按 如何参与野火开源项目 的说明操作。

- **格式问题**
  - 代码改成使用 rst 的格式
  - 图片的引用需要修改
  - 把图片改成居中形式，去掉缩放
- **主体内容**
  - i2c 章节
  - 其余部分章节的内容转 rst
- 悬赏任务

若想领取悬赏任务，请发送要参与的任务和预计时间到邮箱：embedfire@embedfire.com。

任务名称	奖励	具体内容
i2c-oled	待定	参考 i2c 章节编写驱动 oled 的示例代码和笔记

第 1 章　网络协议简介

1.1 常用网络协议

互联网对人类社会产生的巨大变革，大家是有目共睹的，它几乎改变了人类生活的方方面面。互联网通信的本质是数字通信，任何数字通信都离不开通信协议的制定，通信设备只有按照约定的、统一的方式去封装和解析信息，才能实现通信。互联网通信所要遵守的众多协议，被统称为 TCP/IP。

TCP/IP 是一个协议族，包含众多的协议。但对于网络应用开发人员，可能听到更多的是其中的应用层协议，比如 HTTP、FTP、MQTT 等。

HTTP 协议是 Hyper Text Transfer Protocol（超文本传输协议）的缩写，HTTP 的应用最为广泛。比如大家日常使用电脑时的一个常规操作：打开电脑，打开浏览器，输入网址，最后按下回车，这一刻你就开启了 HTTP 通信。HTTP 协议工作于 < 客户端-服务端 > 架构之上，（服务端也称作为服务器端，除非特别说明，否则本书出现的“服务端”即为“服务器端”），浏览器作为 HTTP 客户端通过 URL 向 HTTP 服务端即 WEB 服务器发送所有请求。Web 服务器根据接收到的请求后，向客户端发送响应信息。借助这种浏览器和服务器之间的 HTTP 通信，我们能够足不出户地获得来自世界各个角落的信息。另外，网页不仅仅是大型服务器的专利，在物联网风潮盛行的今天，许多随处可见的小型设备（空调、冰箱、插座、路由器等），都内嵌网页，在物理链路畅通的情况下，用户可以用手机、平板电脑上的浏览器随时随地监控这些设备。

FTP（File Transfer Protocol）是文件传输协议的简称。FTP 是工作在应用层的网络协议。FTP 使得主机间可以共享文件，用于在两台设备之间传输文件（双向传输）。它也是一个客户端-服务端框架系统。用户可以通过一个支持 FTP 协议的客户端程序，连接到在远程主机上的 FTP 服务端程序，通过客户端程序向服务端程序发出命令，服务端程序执行用户所发出的命令，并将执行的结果返回到客户机。FTP 除了基本的文件上传/下载功能外，还有目录操作、权限设置、身份验证机制，许多网盘的文件传输功能都是基于 FTP 实现的。

在物联网发展的处期，物联网场景中的设备使用何种应用层协议进行通信一直是备受争议的话题。很多开发人员习惯了网页的开发模式，于是经常选择 HTTP 作为通信方式。使用 HTTP 有以

下不利因素：HTTP 是一种同步协议，设备需要等待服务器的响应才可以进行下一步的工作，然而在设备数量多、网络不可靠的场景下，实现同步通信很困难；HTTP 是单向的，设备只能主动向服务器发出数据，无法被动的接收来自网络的数据，这不适用于实时控制的场合；HTTP 是有许多帧头和规则的重量级协议，实现在设备中需要耗费大量的系统资源。基于上述的形势，MQTT 和 COAP 等轻量级、异步的通信协议便得到了物联网设备开发商的宠爱，尤其是 MQTT。MQTT（消息队列遥测传输）是 IBM 公司于 1990 年设计并推出的一款通信协议，于 2014 年正式成为了一个 OASIS 开放标准。近年来，MQTT 的应用呈现出爆炸性的增长势头，大有一统物联网的趋势。另外，MQTT 在物联网以外的其他领域也得到了广泛的应用，比如许多公司在制作手机 APP 时，会使用 MQTT 来实现消息推送、即时聊天等功能。

嵌入式设备接入互联网的需求越来越大，有以下几点原因：

（1）近些年，各种带网络接入功能的 MCU、SoC 层出不穷，开源轻量的 TCP/IP 协议栈日趋成熟和完善，云平台的市场越来越繁荣，这些因素大大降低了嵌入式设备的入网成本，也为许多资源受限的低端设备接入互联网提供了可能。

（2）“物联网 +”的风潮日渐盛行，设备能够被远程监控，这一点已经成为许多产品的技术要求。

（3）人们对于设备“智能性”的追求越来越高，当今热门的大数据、图像处理、语音识别、机器学习等功能都可以被集成在云端，成为云平台能提供的服务。终端设备大多是计算、存储能力有限的设备，这些设备如果想要获取“智能”，最便捷的办法就是接入云平台，利用各项云服务。

互联网的基础就是 TCP/IP。TCP/IP 是一个非常复杂的协议族，即便我们能把它的设计思想和实现原理都解释得清清楚楚，你也不见得有时间和精力去学习它，所以本书的写作重点不在于对 TCP/IP 的解读，而在于对它的应用。另外，TCP/IP 的复杂性也决定了它并不是那么简单就能用好的东西，即便我们只关注应用开发，也依然需要对它的许多概念和设计思想有所了解，才能编写出正确、高效、健壮性好的应用程序。

希望能借此书，让嵌入式开发工程师们以浓厚的兴趣和清晰的视野，搭上物联网发展的快车。

1.2 网络协议的分层模型

TCP/IP 是一个庞大的协议族，它是众多网络协议的集合，包括：ARP、IP、ICMP、UDP、TCP、DNS、DHCP、HTTP、FTP、MQTT 等等。这些协议按照功能，可以被划分为几个不同的层次，如图 *1_1* 所示。我们在上一节中介绍的 HTTP、FTP、MQTT，它们隶属于应用层。那么 TCP/IP 为什么需要分层，分层又是依靠什么依据呢?

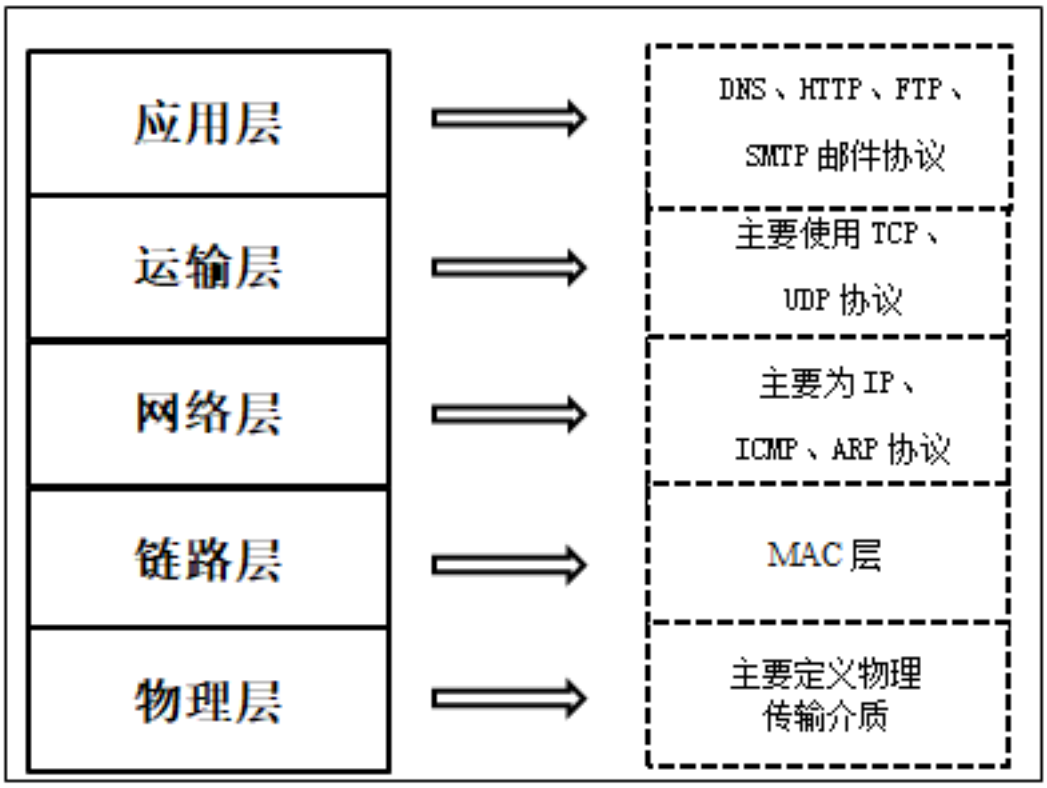


图 1-1 TCP/IP 协议的分层

图 1-1 TCP/IP 协议的分层

TCP/IP 协议栈中不同协议所完成的功能是不一样的，某些协议的实现要依赖于其它协议，依据这种依赖关系，可以将协议栈分层。在图 *1_1* 中，低层协议为相邻的上层协议提供服务，是上层协议得以实现的基础。

其中，物理层（PHY）规定了传输信号所需要的物理电平、介质特征；链路层（MAC）规定了数

据帧能被网卡接收的条件，最常见的方式是利用网卡的 MAC 地址，发送方会在欲发送的数据帧的首部加上接收方网卡的 MAC 地址信息，接收方只有监听到属于自己的 MAC 地址信息后，才会去接收并处理该数据；每台网络设备都应该有自己的网络地址，网络层规定了主机的网络地址该如何定义，以及如何在网络地址和 MAC 地址之间进行映射，即 ARP 协议；网络层实现了数据包在主机之间的传递，而一台主机内部可能运行着多个网络程序，传输层可以区分数据包是属于哪一个应用程序的，可以说传输层实现了数据包端到端的传递。另外，数据包在传输过程中可能会出现丢包、乱序和重复的现象，网络层并没有提供应对这些错误的机制，而传输层可以解决这些问题，如 TCP 协议；应用层以下的工作完成了数据的传递工作，应用层则决定了你如何应用和处理这些数据，之所以会有许多的应用层协议，是因为互联网中传递的数据种类很多、差异很大、应用场景十分多样。

1.3 协议层报文间的封装与拆封

本书的后面章节会对 TCP/IP 协议栈中的每层协议进行分析和讲解。在这里，我们以图 *1_2* 简单解释一下在数据的发送和接收过程中，TCP/IP 都做了哪些事儿。

当用户发送数据时，将数据向下交给传输层，这是处于应用层的操作，应用层可以通过调用传输层的接口来编写特定的应用程序。而 TCP/IP 协议一般也会包含一些简单的应用程序如 Telnet 远程登录、FTP 文件传输、SMTP 邮件传输协议等。传输层会在数据前面加上传输层首部（此处以 TCP 协议为例，图 *1_2* 的传输层首部为 TCP 首部，也可以是 UDP 首部），然后向下交给网络层。同样地，网络层会在数据前面加上网络层首部（IP 首部），然后将数据向下交给链路层，链路层会对数据进行最后一次封装，即在数据前面加上链路层首部（此处使用以太网接口为例），然后将数据交给网卡。最后，网卡将数据转换成物理链路上的电平信号，数据就这样被发送到了网络中。数据的发送过程，可以概括为 TCP/IP 的各层协议对数据进行封装的过程，如图 *1_2* 所示。

当设备的网卡接收到某个数据包后，它会将其放置在网卡的接收缓存中，并告知 TCP/IP 内核。然后 TCP/IP 内核就开始工作了，它会将数据包从接收缓存中取出，并逐层解析数据包中的协议首部信息，并最终将数据交给某个应用程序。数据的接收过程与发送过程正好相反，可以概括为 TCP/IP 的各层协议对数据进行解析的过程。

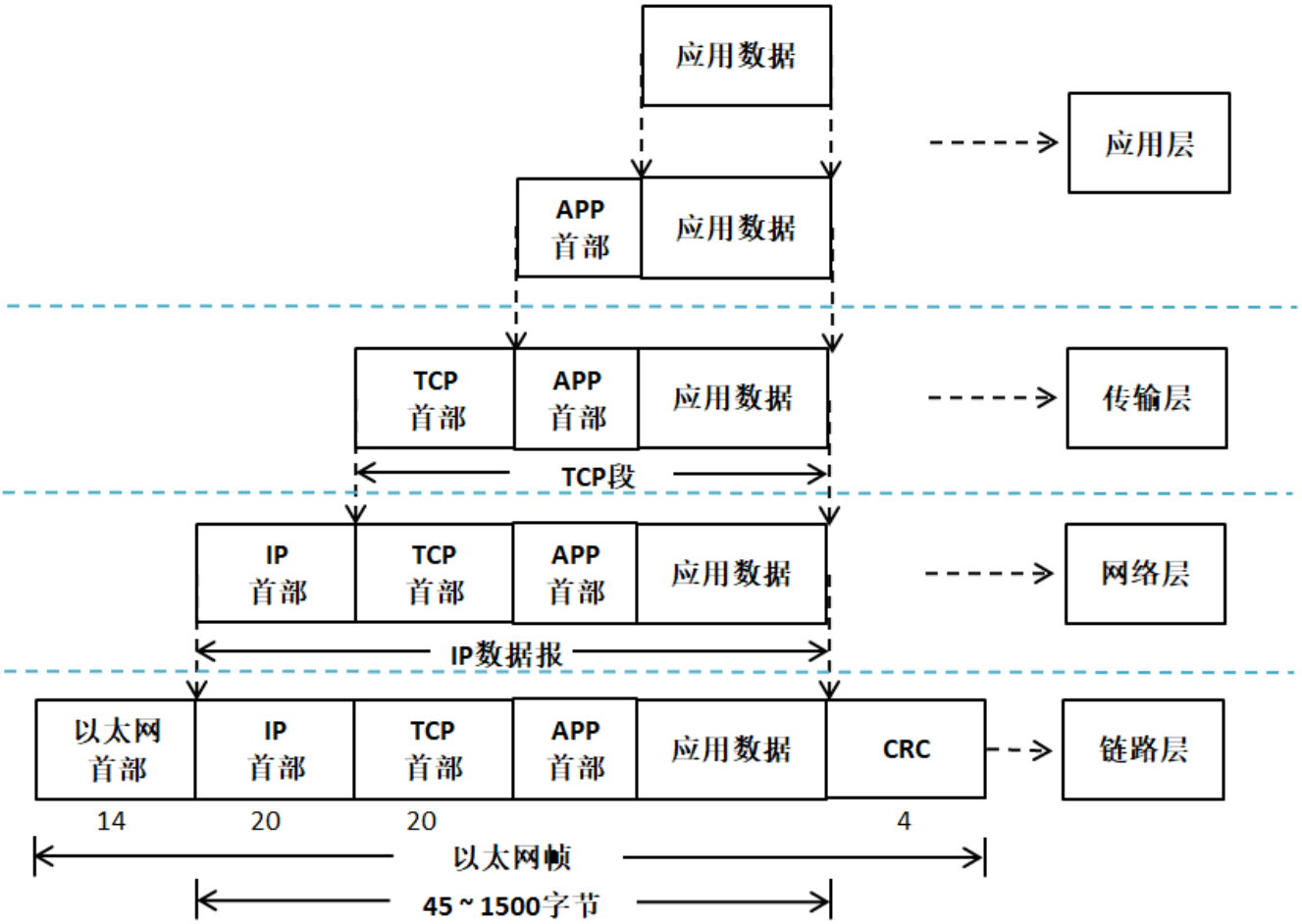


图 1-2 TCP/IP 协议栈各层的报文封装与拆封

第 2 章　LwIP 简介

本书使用最新的 LwIP 2.1.2 版本，官方下载链接：http://savannah.nongnu.org/projects/lwip/。

2.1 LwIP 的优缺点

本书以 LwIP 2. 1.2 为主要对象进行讲解，后续中出现的 LwIP 如果没有特殊声明，均指 2.1.2 版本。此时的 LwIP 2. 1.2 为最新版本，可能当这本书写完的时候，LwIP 又被更新了，对于学习而言，大家其实不必纠结于是否必须用最新的版本，因为 2.1.2 版本和它后面的版本在移植和应用方法上并没有什么区别。

LwIP 全名：Light weight IP，意思是轻量化的 TCP/IP 协议，是瑞典计算机科学院 (SICS) 的 Adam Dunkels 开发的一个小型开源的 TCP/IP 协议栈。LwIP 的设计初衷是：用少量的资源消耗实现一个较为完整的 TCP/IP 协议栈，其中“完整”主要指的是 TCP 协议的完整性，实现的重点是在保持 TCP 协议主要功能的基础上减少对 RAM 的占用。此外 LwIP 既可以移植到操作系统上运行，也可以在无操作系统的情况下独立运行。

LwIP 具有主要特性：

1. 支持 ARP 协议（以太网地址解析协议）。
2. 支持 ICMP 协议（控制报文协议），用于网络的调试与维护。
3. 支持 IGMP 协议（互联网组管理协议），可以实现多播数据的接收。
4. 支持 UDP 协议 (用户数据报协议)。
5. 支持 TCP 协议 (传输控制协议)，包括阻塞控制、RTT 估算、快速恢复和快速转发。
6. 支持 PPP 协议（点对点通信协议），支持 PPPoE。
7. 支持 DNS（域名解析）。
8. 支持 DHCP 协议，动态分配 IP 地址。

9. 支持 IP 协议，包括 IPv4、IPv6 协议，支持 IP 分片与重装功能，多网络接口下的数据包转发。
10. 支持 SNMP 协议（简单网络管理协议）。
11. 支持 AUTOIP，自动 IP 地址配置。
12. 提供专门的内部回调接口 (Raw API)，用于提高应用程序性能。
13. 提供可选择的 Socket API、NETCONN API (在多线程情况下使用) 。

LwIP 在嵌入式中使用有以下优点：

1. 资源开销低，即轻量化。LwIP 内核有自己的内存管理策略和数据包管理策略，使得内核处理数据包的效率很高。另外，LwIP 高度可剪裁，一切不需要的功能都可以通过宏编译选项去掉。LwIP 的流畅运行需要 40KB 的代码 ROM 和几十 KB 的 RAM，这让它非常适合用在内存资源受限的嵌入式设备中。
2. 支持的协议较为完整。几乎支持 TCP/IP 中所有常见的协议，这在嵌入式设备中早已够用。
3. 实现了一些常见的应用程序：DHCP 客户端、DNS 客户端、HTTP 服务器、MQTT 客户端、TFTP 服务器、SNTP 客户端等等。
4. 同时提供了三种编程接口：RAW API、NETCONN API（注：NETCONN API 即为 Sequential API，为了统一，下文均采用 NETCONN API）和 Socket API。这三种 API 的执行效率、易用性、可移植性以及时空间的开销各不相同，用户可以根据实际需要，平衡利弊，选择合适的 API 进行网络应用程序的开发。
5. 高度可移植。其源代码全部用 C 实现，用户可以很方便地实现跨处理器、跨编译器的移植。另外，它对内核中会使用到操作系统功能的地方进行了抽象，使用了一套自定义的 API，用户可以通过自己实现这些 API，从而实现跨操作系统的移植工作。
6. 开源、免费，用户可以不用承担任何商业风险地使用它。
7. 相比于嵌入式领域其它的 TCP/IP 协议栈，比如 uC-TCP/IP、FreeRTOS-TCP 等，LwIP 的发展历史要更悠久一些，得到了更多的验证和测试。LwIP 被广泛用在嵌入式网络设备中，国内一些物联网公司推出的物联网操作系统，其 TCP/IP 核心就是 LwIP；物联网知名的 WiFi 模块 ESP8266，其 TCP/IP 固件，使用的就是 LwIP。

LwIP 尽管有如此多的优点，但它毕竟是为嵌入式而生，所以并没有很完整地实现 TCP/IP 协议栈。相比于 Linux 和 Windows 系统自带的 TCP/IP 协议栈，LwIP 的功能不算完整和强大。但对于大多数物联网领域的网络应用程序，LwIP 已经足够了。

2.2 LwIP 的文件说明

2.2.1 如何获取 LwIP 源码文件

LwIP 的代码已经交给 Savannah 托管，LwIP 的项目主页是：http://savannah.nongnu.org/projects/lwip/。这个主页简单地介绍了一下 LwIP，然后给出了许多链接，你可以通过这些链接去挖掘更多关于 LwIP 的信息。在这里，我们只关注两个地方，如图 *2_1* 中的方框所示。

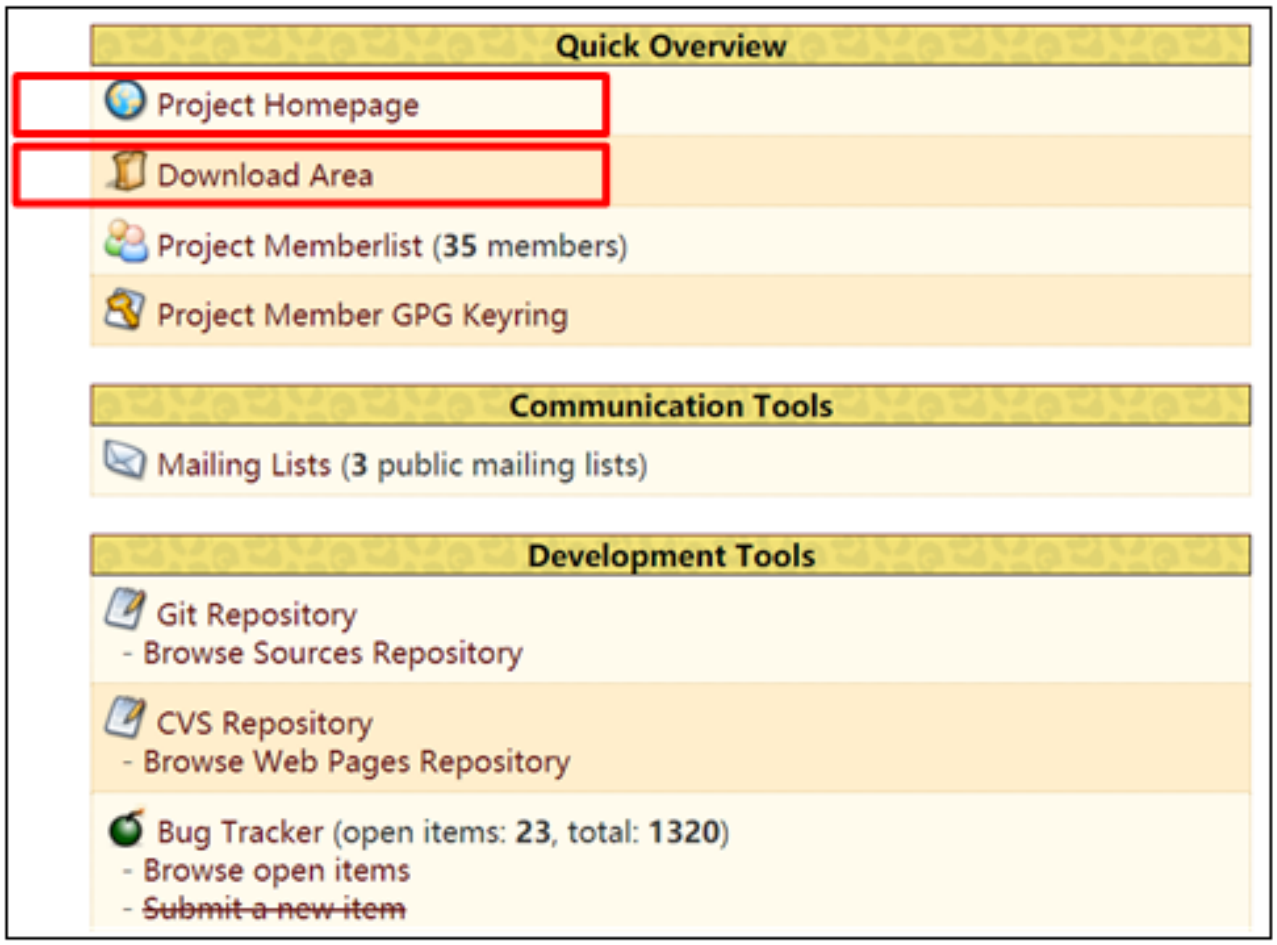


图 2-1 LwIP 项目主页截图

图 2-1 LwIP 项目主页截图

点击“Project Homepage”，会得到一个网页，如图 *2_2* 所示。这个网页可以看成是 LwIP 的官方说明文档。我们可以通过这个网页获得关于 LwIP 的很多信息，包括 LwIP 的使用注意、数据的拷贝、系统初始化流程、多线程中要注意的问题、优化方法、内核模块的分类介绍、内核数据结构、内核重要全局变量、内核源码文件等。这些内容专业性比较强，不建议初学时在它上面花费精力，并且里面的很多内容在我们教材的后续章节中中会有所讲解。在这里，读者只要知道有这么个东西就行了。

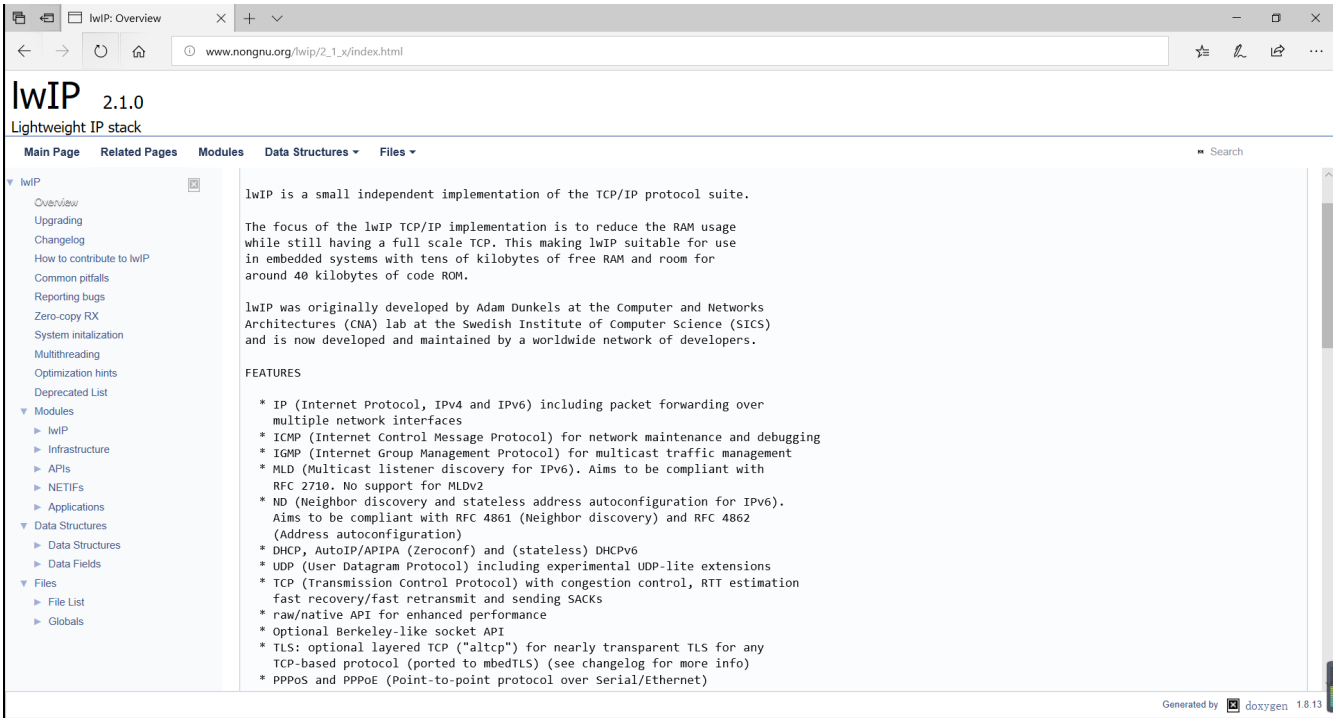


图 2-2 LwIP 官方说明文档（HTML 网页）

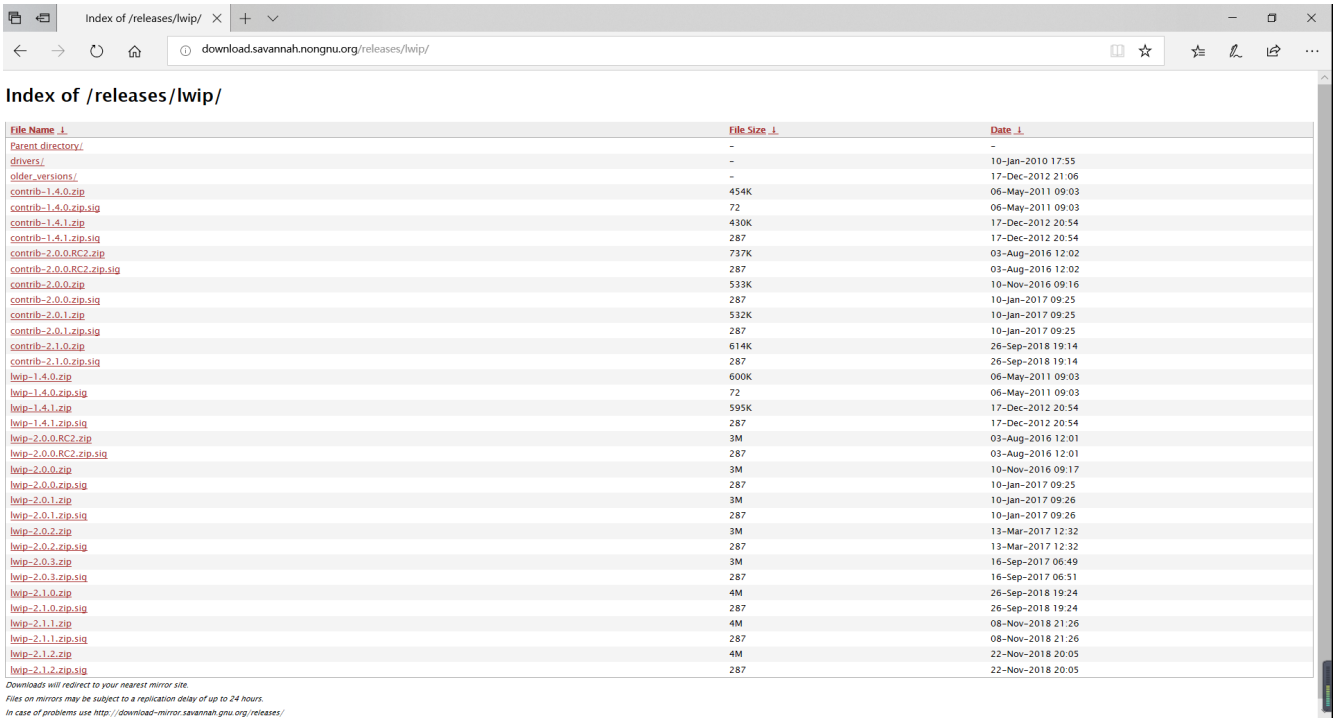


图 2-3 LwIP 的资源下载

点击“Download Area”，会得到一个网页，如图 *2_3* 所示。通过这个网页，我们可以下载到 LwIP 所有版本的源代码包和 contrib 包。你每点击一个红色字体的资源链接，浏览器就会开启一个 ftp 连接，帮助你下载想要的文件到电脑中。但是这个页面提供的下载链接，在国内一般是没有响应的。这个网页最下方的黑字内容推荐我们使用另外一个下载页面：http://download-mirror.savannah.gnu.org/releases/。在这个页面下，用户可以下载到所有在 Savannah 托管的开源软件，但我们只关心 LwIP。利用浏览器的搜索功能，快捷键 Ctrl+F，可以快速找到 lwip 目录。在这里为了方便读者，我们直接给出最终的下载链接：http://download-mirror.savannah.gnu.org/releases/lwip/。

可能有读者会问，什么是 contrib 包，它与源代码包有什么不同？源代码包里面装的主要是 LwIP 内核的源码文件，而 contrib 包里面装的是移植和应用 LwIP 的一些 demo，即应用示例。contrib 包不属于 LwIP 内核的一部分，里面的很多内容来自开源社区的贡献，因此 contrib 包的版本管理不像内核源码那样严格和规范，但也是很有参考价值的。按理说，LwIP 源码面世越久，开源社区对它的贡献就越大，所以越高版本的 contrib 包，提供的应用示例就越丰富，越有参考价值。在大版本区别不大的情况下，建议大家下载最新的 contrib 包。后续我们会对 contrib 包里面提供的应用示例进行讲解。另外，还有些“.sig”后缀的文件，这是数字签名，大家忽略就好。

2.2.2 LwIP 文件说明

按照上一小节的介绍，我们下载两个包：lwip-2.1.2.zip（源码包）和 contrib-2.1.0.zip（contrib 包）。解压以后会得到两个文件夹，如图 *2_4* 所示。

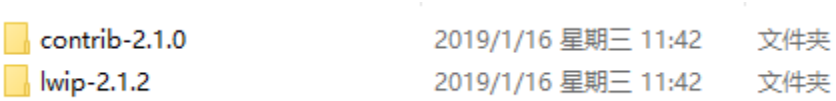


图 2-4 下载解压后得到的源码包和 contrib 包

我们先打开“lwip-2.1.2”文件夹，如图 *2_5* 所示。



图 2-5 源码包的目录

该目录的内容为：

（1）CHANGELOG 文件记录了 LwIP 在版本升级过程中源代码发生的变化。

（2）COPYING 文件记录了 LwIP 这个开源软件的 license。一个软件开源，不代表你能无限制地使用它，你需要在使用它的过程中遵守一定的规则，这些规则就是 license。大家可以用记事本打开这个 COPYING 文件看看它的内容。开源软件的 license 有很多种，LwIP 的属于 BSD License。LwIP 的开源程度是很高的，你几乎可以无限制地使用它。

（3）FILES 文件用于介绍当前目录下的目录信息。

（4）README 文件对 LwIP 进行了一个简单的介绍。

（5）UPGRADING 文件记录了 LwIP 每个大版本的更新，会对用户使用和移植 LwIP 造成的影响。所谓大版本更新指的是：1.3.x - 1.4.x –2.0.x –2.1.x。小版本更新，比如 2.0.1 –2.0.2 –2.0.3，这个过程只是一些 bug 的修复和性能的改善，不会对用户的使用造成影响。用户只要将原有工程的目录中与 LwIP 相关的旧版本文件替换成新版本的文件，重新编译，就能直接使用。

（6）doc 文件夹里面是关于 LwIP 的一些文档，可以看成是应用和移植 LwIP 的指南。但是这些文档比较零散，不成体系，而且纯文本阅读起来很费劲，阅读意义不是很大。

（7）test 文件夹里面是测试 LwIP 内核性能的源码，将它们和 LwIP 源码加入到工程中一起编译，调用它们提供的函数，可以获得许多与 LwIP 内核性能有关的指标。这种内核性能测试功能，只

有非常专业的人士才用的到。

（8）src 文件夹里面就是我们最关心的 LwIP 源码文件，下面会详细讲解。

打开 src 文件夹，如图 *2_6* 所示。



图 2-6 src 目录（LwIP 源码文件所在的目录）

api 文件夹里面装的是 NETCONN API 和 Socket API 相关的源文件，只有在操作系统的环境中，才能被编译。

apps 文件夹里面装的是应用程序的源文件，包括常见的应用程序，如 httpd、mqtt、tftp、sntp、snmp 等。

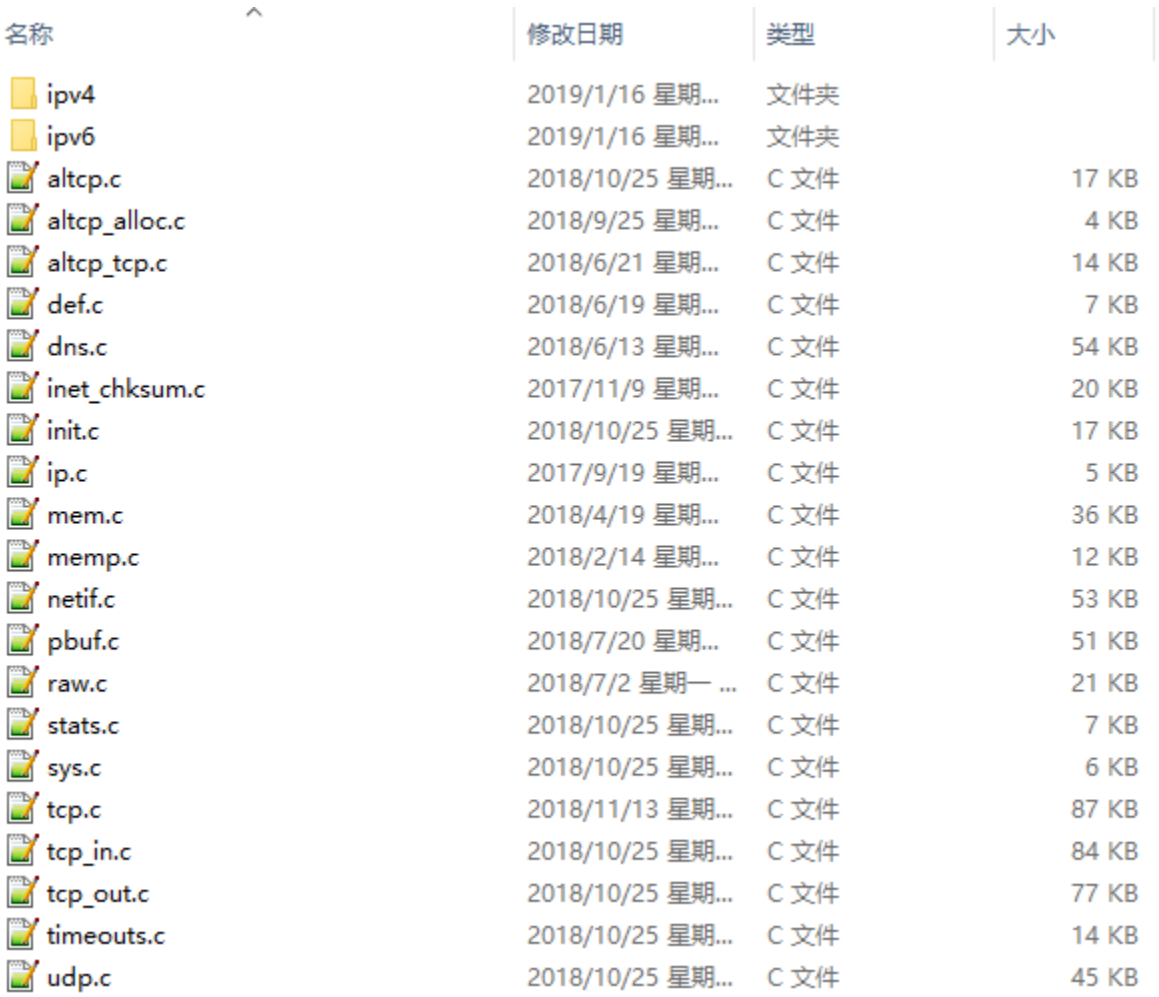
core 文件夹里面是 LwIP 的内核源文件，后续会详细讲解。

include 文件夹里面是 LwIP 所有模块对应的头文件。

netif 文件夹里面是与网卡移植有关的文件，这些文件为我们移植网卡提供了模板，我们可以直接使用。

LwIP 内核是由一系列模块组合而成的，这些模块包括：TCP/IP 协议栈的各种协议、内存管理模块、数据包管理模块、网卡管理模块、网卡接口模块、基础功能类模块、API 模块。每个模块是由相关的几个源文件和头文件组成的，通过头文件对外声明一些函数、宏、数据类型，使得其它模块可以方便地调用此模块的功能。而构成每个模块的头文件都被组织在了 include 目录中，而源文件则根据类型被分散地组织在 api、apps、core、netif 目录中。

接下来，我们介绍一下 core 文件夹，如图 *2_7* 所示。



名称	修改日期	类型	大小
ipv4	2019/1/16 星期...	文件夹	
ipv6	2019/1/16 星期...	文件夹	
altcp.c	2018/10/25 星期...	C 文件	17 KB
altcp_alloc.c	2018/9/25 星期...	C 文件	4 KB
altcp_tcp.c	2018/6/21 星期...	C 文件	14 KB
def.c	2018/6/19 星期...	C 文件	7 KB
dns.c	2018/6/13 星期...	C 文件	54 KB
inet_chksum.c	2017/11/9 星期...	C 文件	20 KB
init.c	2018/10/25 星期...	C 文件	17 KB
ip.c	2017/9/19 星期...	C 文件	5 KB
mem.c	2018/4/19 星期...	C 文件	36 KB
memp.c	2018/2/14 星期...	C 文件	12 KB
netif.c	2018/10/25 星期...	C 文件	53 KB
pbuf.c	2018/7/20 星期...	C 文件	51 KB
raw.c	2018/7/2 星期一 ...	C 文件	21 KB
stats.c	2018/10/25 星期...	C 文件	7 KB
sys.c	2018/10/25 星期...	C 文件	6 KB
tcp.c	2018/11/13 星期...	C 文件	87 KB
tcp_in.c	2018/10/25 星期...	C 文件	84 KB
tcp_out.c	2018/10/25 星期...	C 文件	77 KB
timeouts.c	2018/10/25 星期...	C 文件	14 KB
udp.c	2018/10/25 星期...	C 文件	45 KB

图 2-7 core 目录

我们逐一介绍一下这些源文件的功能。

ipv4 文件夹里面是与 IPv4 模块相关的源文件，它们实现了 IPv4 协议规定的对数据包的各种操作。ipv4 文件夹中还包括一些并非属于 IP 协议，但会受 IP 协议影响的协议源文件，包括 DHCP、ARP、ICMP、IGMP。

ipv6 文件夹里面是与 IPv6 模块相关的源文件，它们实现了 IPv6 协议规定的对数据包的各种操作。ipv6 文件夹中还包括一些并非属于 IP 协议，但会受 IP 协议影响的协议源文件，包括 DHCP、ARP、ICMP、IGMP。

altcp.c、altcp_alloc.c、altcp_tcp.c 等文件是应用程序分层 TCP 连接 API，从 TCPIP 线程使用，是一

个抽象层，可以模拟应用程序的 tcp 回调 API，同时防止直接链接，这样，应用程序可以使用其他应用程序层协议在 TCP 之上而不知道细节（例如 TLS，代理连接），此类接口我们并没有怎么使用，或者如果选择使用安全的加密传输的话，可以配合 mbed TLS 使用。

def.c 文件定义了一些基础类函数，比如主机序和网络序的转换、字符串的查找和比较、整数转换成字符串等，这些函数会被 LwIP 内核的很多模块所调用。在 include 目录里面的 def.h 文件对外声明了 def.c 所实现的函数，同时定义了许多宏，能实现一些基础操作，比如取最大值、取最小值、计算数组长度等，这些宏同样也被内核的许多模块所调用。我们经常可以看到某个内核的源文件在开始的地方 #include “def.h”。

dns.c 文件实现了域名解析的功能，有了它，用户就可以在知道服务器域名的情况下，获得该服务器的 IP 地址。很多时候我们只记得服务器域名而不记得服务器 IP 地址，例如“www.baidu.com”就是一个域名，通过 dns 功能，我们就可以得到与服务器域名对应的 IP 地址，这给用户使用带来很大的方便。

inet_chksum.c 文件提供了 LwIP 所需的校验和功能，在 IP、UDP、TCP 协议的实现中，需要计算校验和。

init.c 文件对 LwIP 的用户宏配置进行了检查，会将配置错误和不合理的地方，通过编译器的 #error 和 #warning 功能表示出来。另外，init.c 定义了 lwip_init 初始化函数，这个函数会依次对 LwIP 的各个模块进行初始化。

ip.c 文件实现了 IP 协议相关的函数，但只是封装了 ipv4 和 ipv6 文件夹中的函数。

mem.c 文件实现了动态内存池管理机制，使得 LwIP 内核的各个模块可以灵活地申请和释放内存。

memp.c 文件实现了静态内存堆管理机制，使得 LwIP 内核的各个模块可以快速地申请和释放内存。

netif.c 文件实现了网卡的操作，比如注册/删除网卡、使能/禁能网卡、设置网卡 IP 地址等等。netif.c 与 include 目录中的 netif.h 文件共同构成了 LwIP 的 netif 模块，它对网卡进行了抽象，使得 LwIP 内核可以方便地管理多个特性各异的物理网卡。

pbuf.c 文件实现了 LwIP 对网络数据包的各种操作。网络数据包在 LwIP 内核中以 pbuf 结构体的形式存在，这提高了 LwIP 内核对数据包处理效率，以及提高了数据包在各层之间递交的效率。pbuf 结构体也是我们使用 RAW/Callback API 进行网络应用程序开发的关键，后续我们会详细讲解。

raw.c 文件实现了一个传输层协议的框架，我们可以在它的基础上修改和添加代码，实现自定义的传输层协议，与 UDP/TCP 一样，它可以与 IP 层直接进行交互。这类似 RAW Socket。在实际的应用中，我们常用 UDP 和 TCP 作为传输层协议。但有时，底层网络开发人员会嫌 UDP 的可靠性太差，或者 TCP 虽然可靠性强，但是很耗费时间和内存，他们需要根据实际需求，平衡利弊，定义自己的传输层协议。LwIP 的 raw 模块可以满足他们的需求。

stat.c 文件实现了 LwIP 内核的统计功能，使用户可以实时地查看 LwIP 内核对网络数据包的处理情况。

sys.c 文件和 sys.h 文件构成了 LwIP 的 sys 模块，它提供了与临界区相关的操作。

tcp.c、tcp_in.c 和 tcp_out.c 文件实现了 TCP 协议，包括对 TCP 连接的操作、对 TCP 数据包的输入输出操作和 TCP 定时器，它们和 include 目录中名称带 tcp 的头文件共同构成了 LwIP 的 TCP 模块。TCP 模块的实现是 LwIP 的最大特点，它以很小的资源开销几乎实现了 TCP 协议中规定的全部内容。TCP 协议是非常复杂的协议，这几个与 TCP 模块相关的文件占据了 LwIP 内核的绝大部分。

timeouts.c 定义了 LwIP 内核的超时处理机制。LwIP 内核中多个模块的实现需要借助超时处理机制，包括 ARP 表项的时间统计、IP 分片报文的重装、TCP 的各种定时器、实现各种应用层协议需要的超时处理。

udp.c 文件实现了 UDP 协议，包括对 UDP 连接的操作和 UDP 数据包的操作。

2.3 查看 LwIP 的说明文档

关于 LwIP 的官方说明文档：http://www.nongnu.org/lwip/2_1_x/index.html，我就简单带大家浏览一下。打开连接，我们可以看到 LwIP 的 Overview（概述），这里就简单看看即可，我们可以点击左侧的“Common pitfalls”，查看一下 LwIP 常见的陷阱，可能在使用中会遇到，到时候注意一下即可，在前面的章节中，我们也提到过，LwIP 可以工作在无操作系统环境也可以工作在有操作系统的环境中，Common pitfalls 中提到 Mainloop Mode（主函数轮询模式）与 OS Mode（操作系统模式）需要注意的一些事情，具体见图 *2_8*。

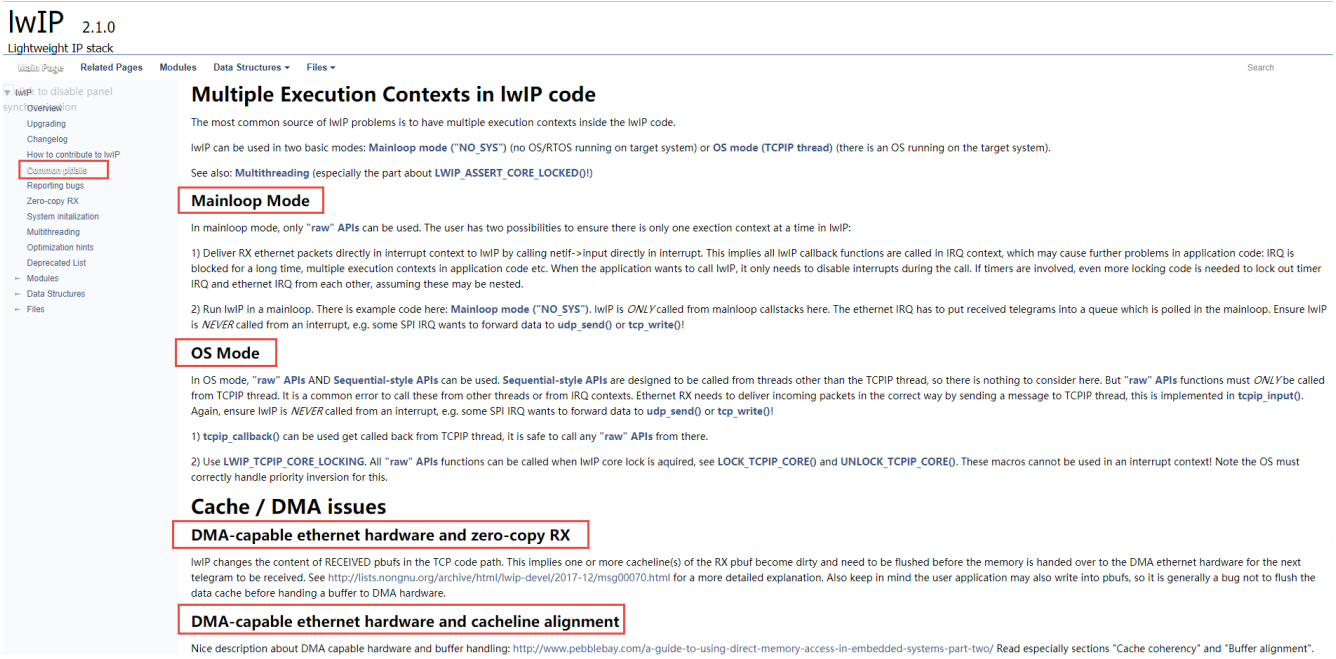


图 2-8 Common pitfalls

此外，我们还可以点击左侧的“Modules”，查看一些模块相关的说明，以及例子，比如有无操作系统相关的，如，还有基础配置，如 LwIP 的内存管理模块，数据包缓冲区等会是在“Modules –>Infrastructure”页面中，具体见图 *2_9*。

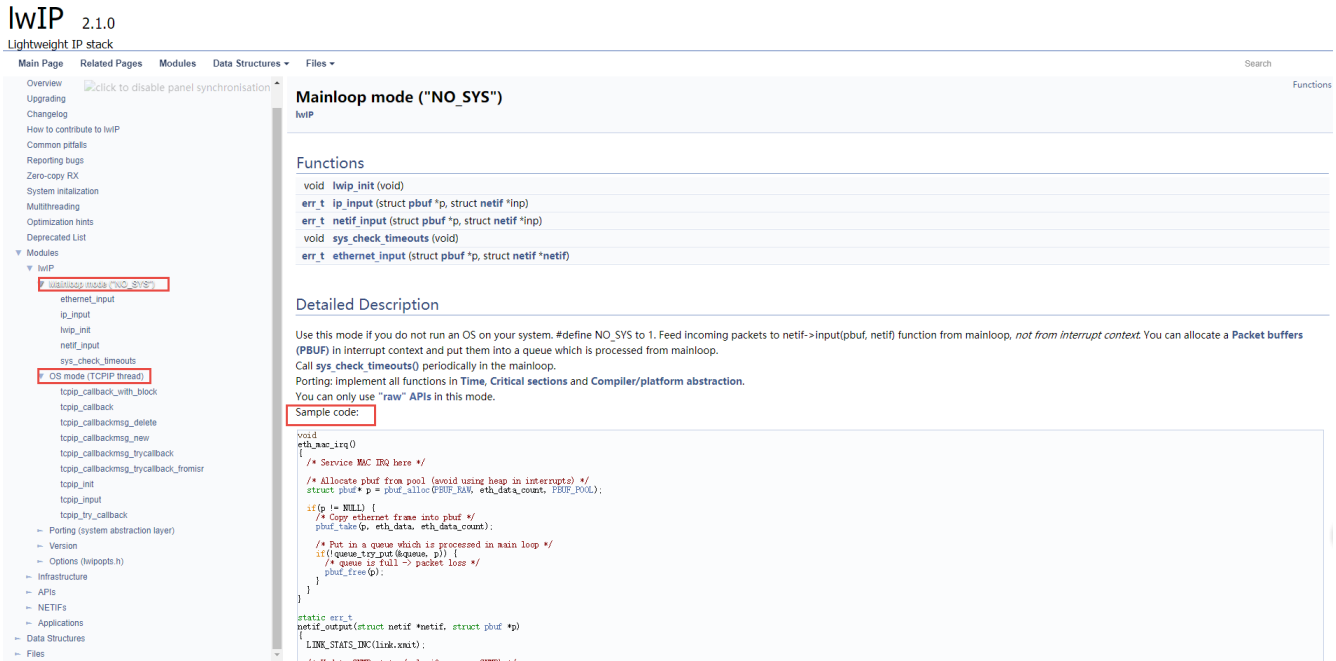


图 2-9 Modules

当然，还有很重要的一些用户常用的 API 函数，也是在“Modules”中可以找到，例如 Raw API，NETCONN API 和 Socket API 等，具体见图 *2_10*。

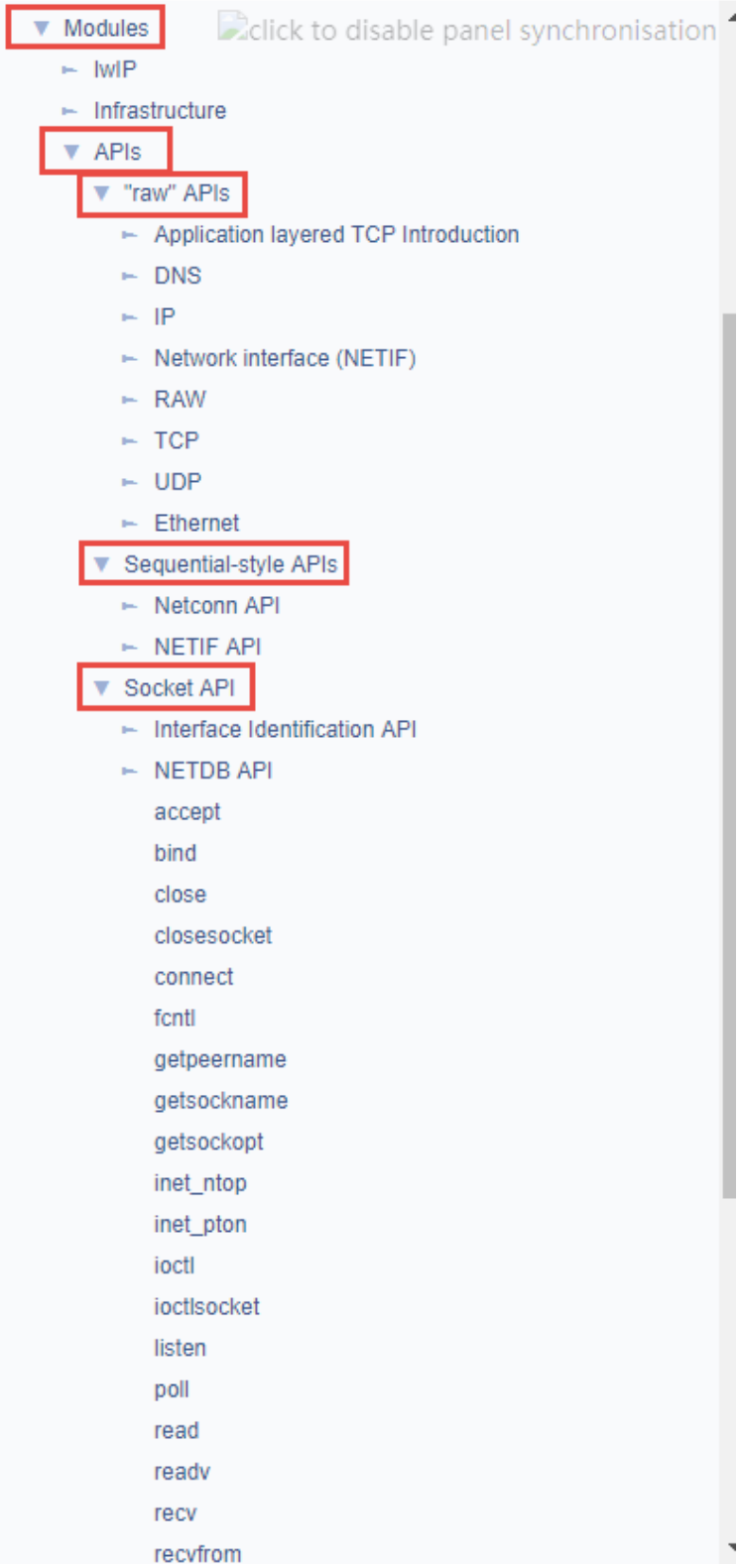

Modules
click to disable panel synchronisation
lwIP
Infrastructure
APIs
"raw" APIs
Application layered TCP Introduction
DNS
IP
Network interface (NETIF)
RAW
TCP
UDP
Ethernet
Sequential-style APIs
Netconn API
NETIF API
Socket API
Interface Identification API
NETDB API
accept
bind
close
closesocket
connect
fcntl
getpeername
getsockname
getsockopt
inet_ntop
inet_pton
ioctl
ioctlsocket
listen
poll
read
readv
recv
recvfrom

图 2-10 Modules->APIs

此外还有一些“Applications”应用层相关的说明，如 HTTP、MQTT、TFTP 等，具体见图 *2_11*。



图 2-11 Modules->Applications

还有一些数据结构相关的说明，当我们在程序中看到哪个数据结构不懂的，都可以在这里找到对应的说明，也是比较重要的，LwIP 本质就是对数据的处理，其中也使用了大量的数据结构，有空可以多研究研究它，具体见图 *2_12*。

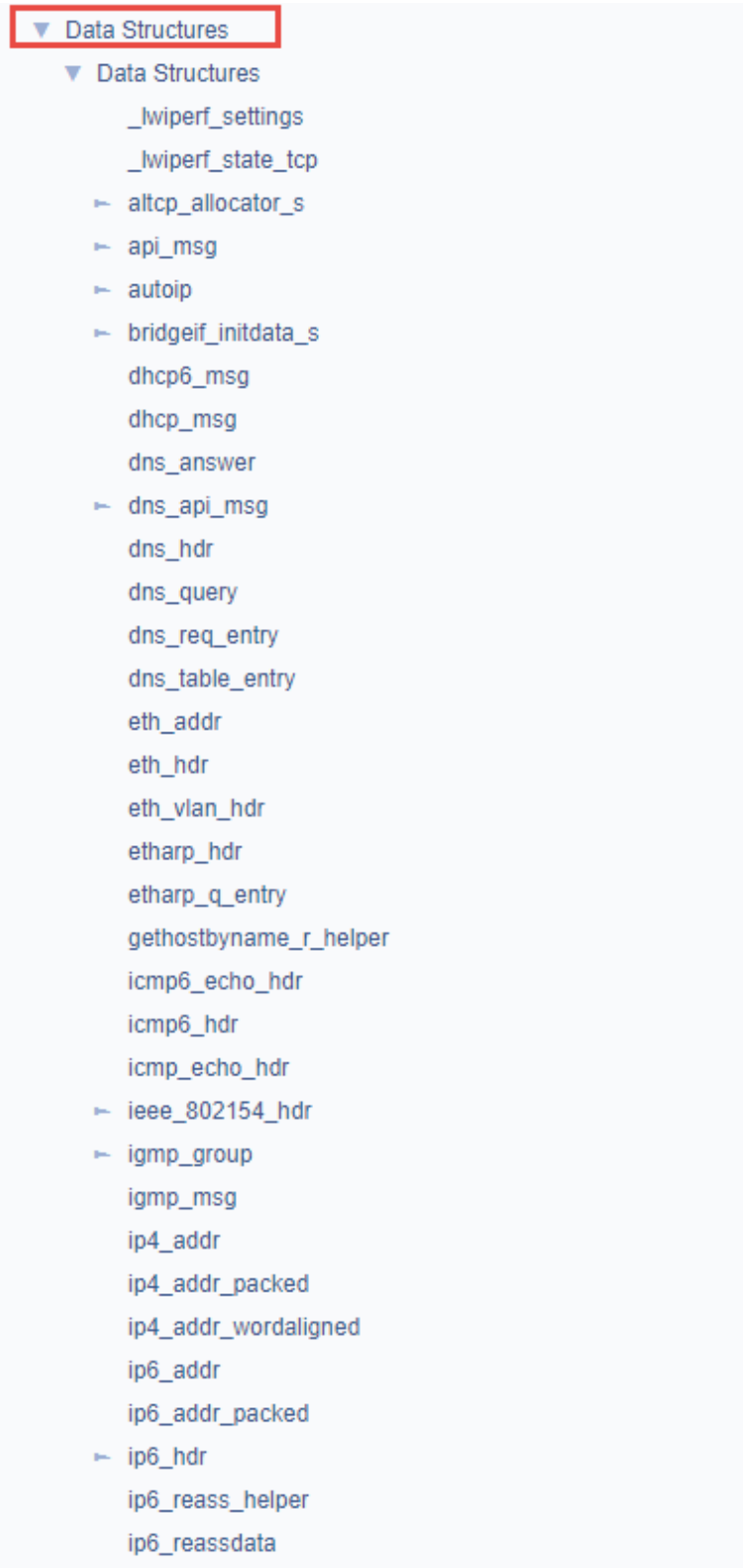

Data Structures
Data Structures
_lwiperf_settings
_lwiperf_state_tcp
altcp_allocator_s
api_msg
autoip
bridgeif_initdata_s
dhcp6_msg
dhcp_msg
dns_answer
dns_api_msg
dns_hdr
dns_query
dns_req_entry
dns_table_entry
eth_addr
eth_hdr
eth_vlan_hdr
etharp_hdr
etharp_q_entry
gethostbyname_r_helper
icmp6_echo_hdr
icmp6_hdr
icmp_echo_hdr
ieee_802154_hdr
igmp_group
igmp_msg
ip4_addr
ip4_addr_packed
ip4_addr_wordaligned
ip6_addr
ip6_addr_packed
ip6_hdr
ip6_reass_helper
ip6_reassdata

图 2-12 Data Structures

当然，我们也能通过函数名字的首字母来查找函数的作用，具体见图 *2_13*。

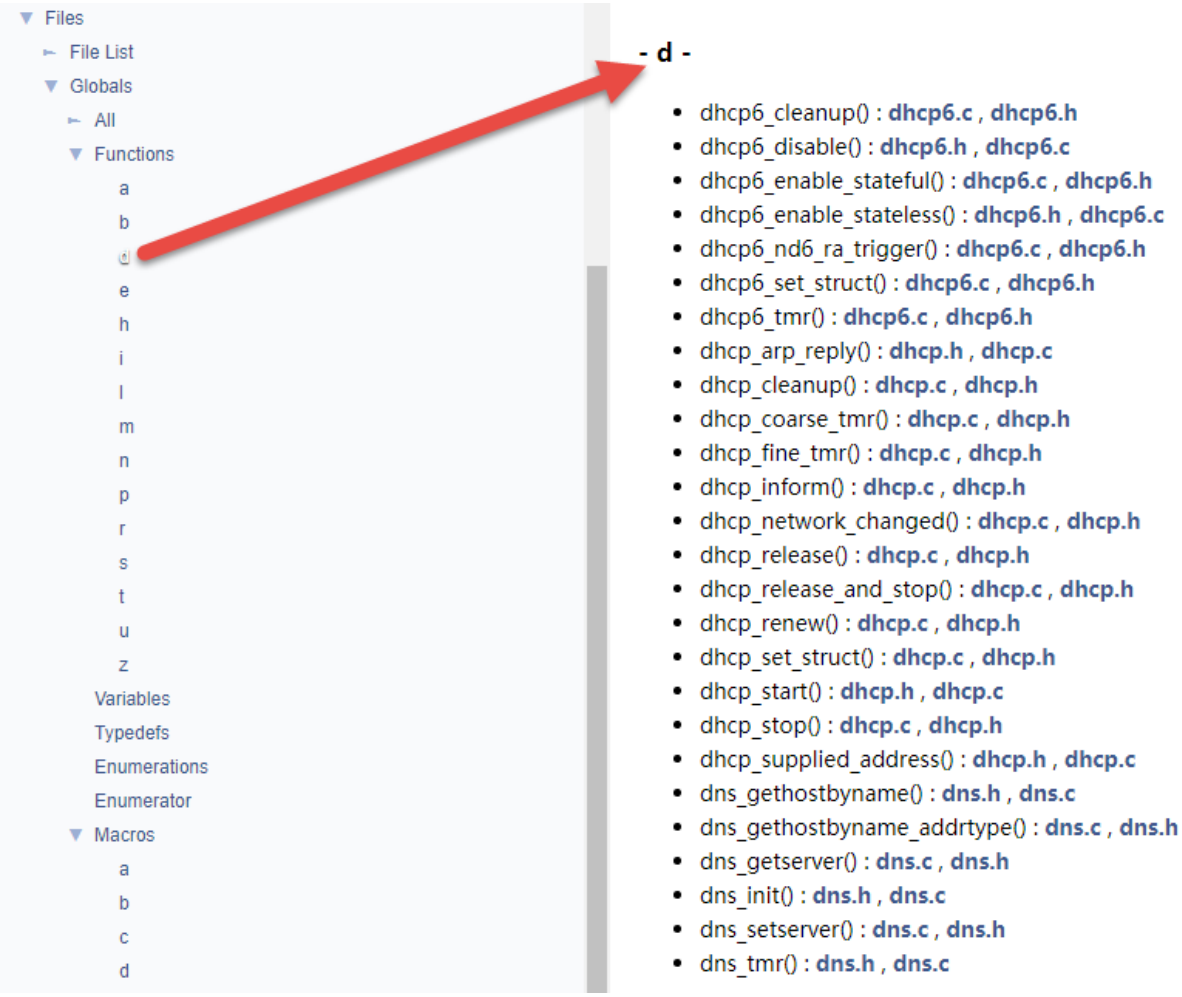


图 2-13 Function

2.4 使用 vscode 查看源码

2.4.1 查看文件中的符号列表（函数列表）

LwIP 的源码很庞大，我们使用微软的开源软件——vs code 查看源码，并且快速找到源码的函数与定义，首先我们先安装 vs code，我们可以在 https://code.visualstudio.com/download 中下载时候自己电脑的 vs code 版本，然后安装即可。

然后打开我们的源码文件夹，右键，选择 Open with Code，这样子就能直接在 vs code 打开我们整个文件夹的源码了，具体见图 *2_14*。

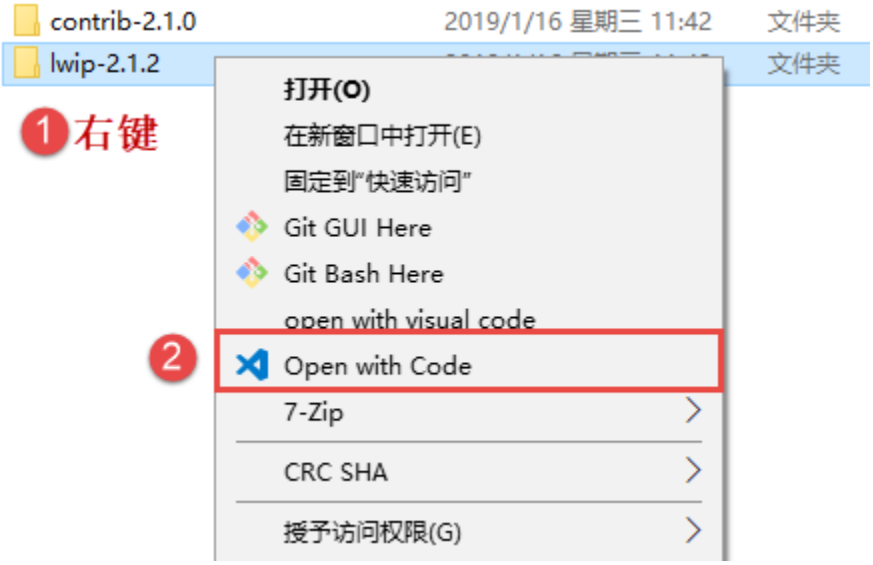


图 2-14 Open with Code

在 vs code 中，就显示了我们打开的源码，LwIP 那么多文件，我们怎么去快速找到源码文件中的某个函数呢？很简单，比如我们知道某个函数的名字的话，可以直接搜索的，这点就不必我多说，但是有时候，我们不记得某个函数的名字，只知道它在哪个文件，或者只知道在好几个文件中的某一个，那么我们就需要一个个去查找这个函数了，vs code 提供很强大的功能，就是可以快速查文件中的符号列表和函数列表，我们首先打开一个源码文件，比如 tcp.c，然后我们通过快捷键“Shift+Ctrl+O”即可打开对应源码文件的符号列表和函数列表，通过查看这些列表，就能知道该源码文件中是否有我们需要的函数或者宏定义等，具体见图 *2_15* 与图 *2_16*。

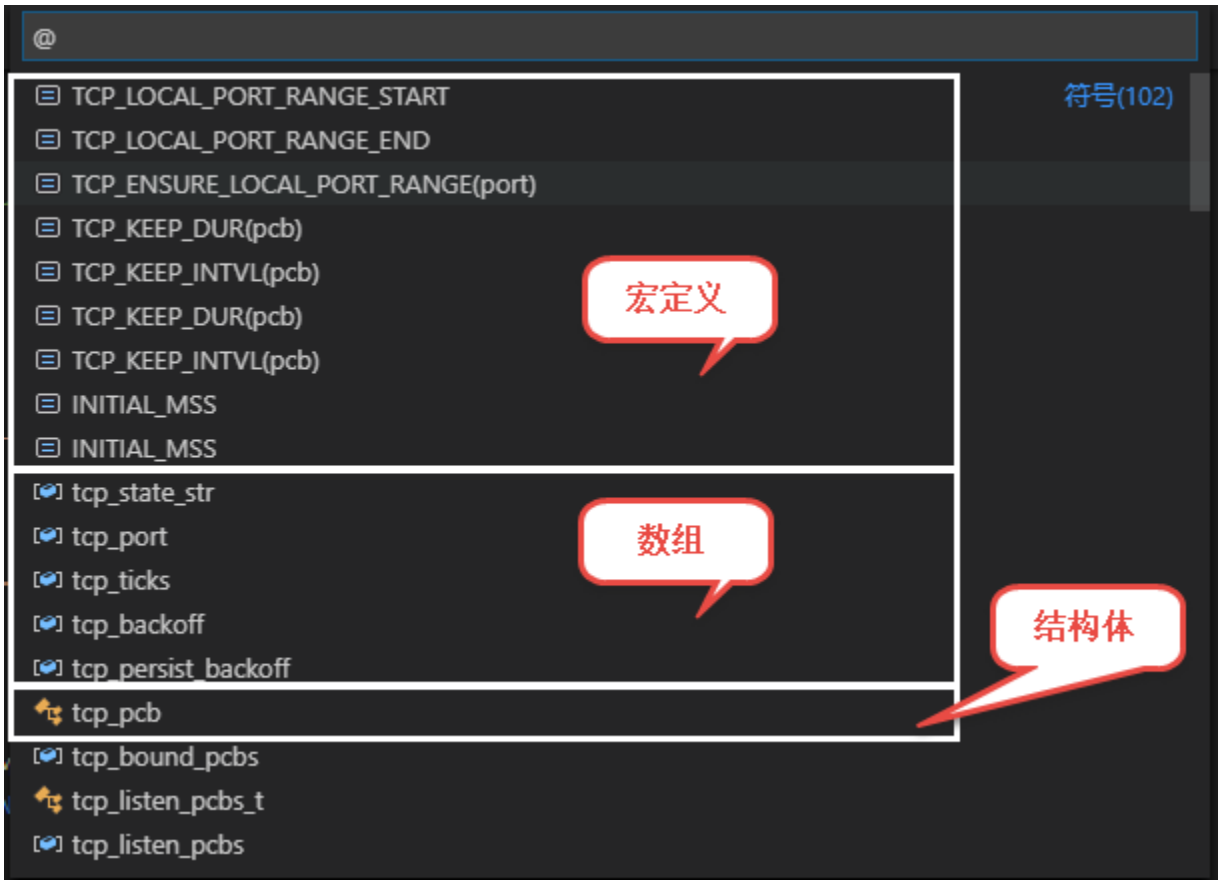

@
符号(102)
TCP_LOCAL_PORT_RANGE_START
TCP_LOCAL_PORT_RANGE_END
TCP_ENSURE_LOCAL_PORT_RANGE(port)
TCP_KEEP_DUR(pcb)
TCP_KEEP_INTVL(pcb)
TCP_KEEP_DUR(pcb)
TCP_KEEP_INTVL(pcb)
INITIAL_MSS
INITIAL_MSS
宏定义
tcp_state_str
tcp_port
tcp_ticks
tcp_backoff
tcp_persist_backoff
数组
结构体
tcp_pcb
tcp_bound_pcbs
tcp_listen_pcbs_t
tcp_listen_pcbs

图 2-15 符号列表

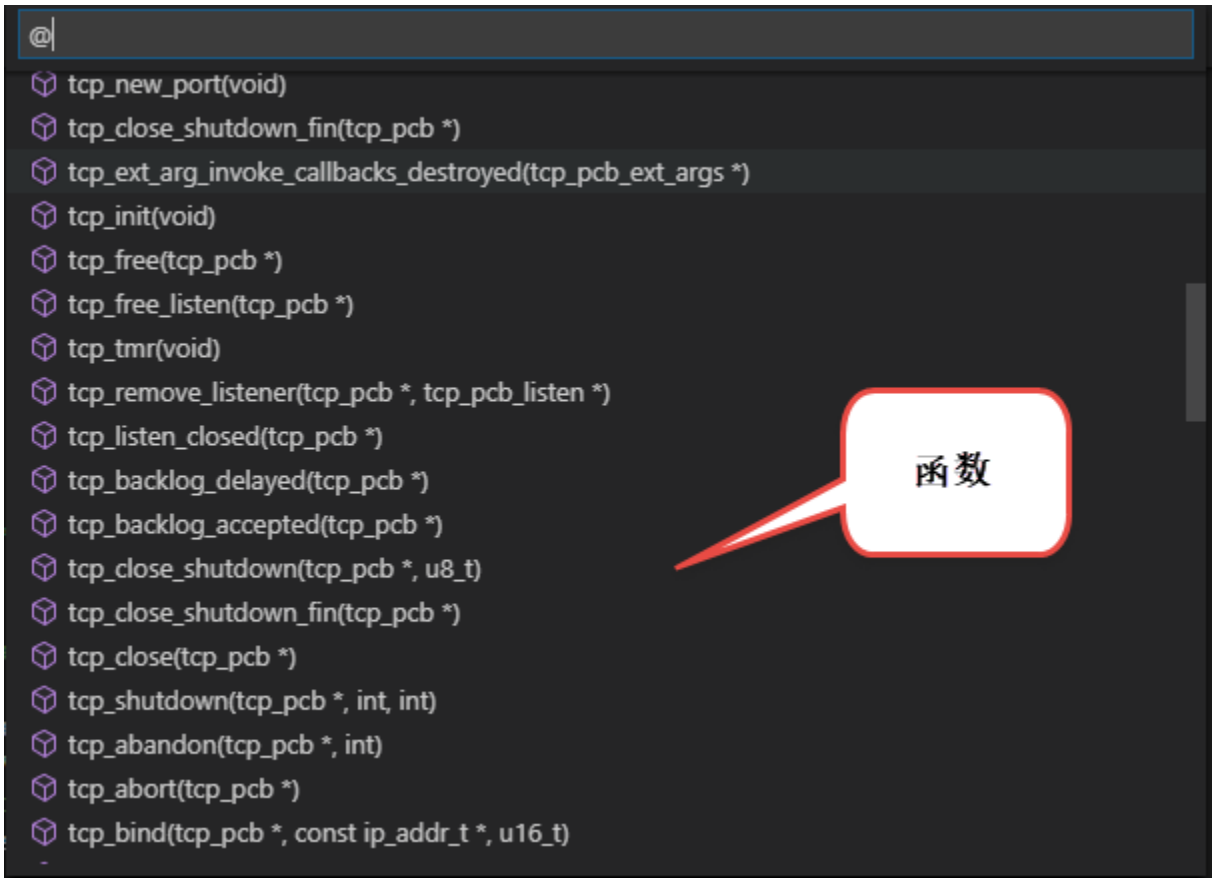


图 2-16 函数列表

2.4.2 函数定义跳转

vs code 看源码是非常方便的，比如，我们可以通过 F12 按键进行跳转到定义，通过“Alt+F12”速览定义，或者通过快捷键“Ctrl+F12”进行 Go to Declaration，这些操作还是很方便的，当然啦，我们也能通过鼠标右键，进行选择，具体见图 *2_17*。如果在查看函数之后，想返回跳转前的位置，只需要通过快捷键“Alt+ 键盘的 ←（前后左右的左按键）”跳回即可。

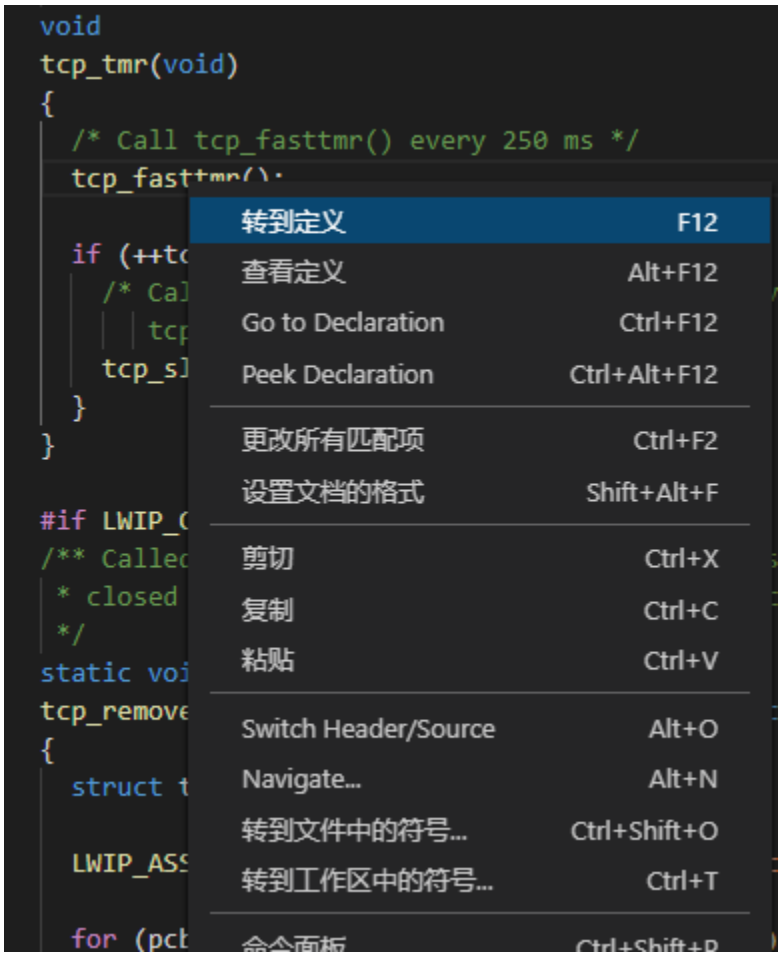


图 2-17 函数定义跳转

2.5 LwIP 源码里的 example

（后面 LwIP 的基础例程主要直接使用或参考源码里的 example 即可）

我们打开之前下载好的 contrib-2.1.0 文件夹，如图 *2_18* 所示。



图 2-18 contrib 包的目录

我们先讲解一下这个目录：

（1）addons 目录。LwIP 中很多模块的实现，都是可以由用户干预的，比如校验和、TCP 初始序列号。LwIP 的内核代码，通过宏编译选项的设置，可以将内核中某些模块的实现方法配置成 LwIP 默认的方法，或者用户自定义的方法。用户自定义的方法通常需要用户在钩子函数中实现。在实际应用中，我们采用内核默认的方法就足够了，只有在非常特定的场合下，为了性能、资源开销等因素的考虑，我们可能会需要自己实现相关的模块，或者说编写相应的钩子函数。那么这时该怎么办呢？addons 目录下的内容就为我们提供了参考。对于初学者，没必要关心这个目录。

（2）apps 目录里实现了很多应用层协议。LwIP 源码包中也有 apps 目录，但源码包中 apps 目录下的应用程序全部用 RAW/Callback API 实现，属于内核代码的一部分。而此 apps 目录里的应用程序可以是由三种 API 中的任何一种实现的。读者可以把它看成是内核源码所提供的应用程序的一个补充。

（3）examples 目录里是一些 LwIP 的应用示例。在使用 LwIP 开发应用程序时会出现的典型问题，比如如何移植网卡、如何使用 LwIP 的 API、如何使用源码中提供的应用程序，对于这些问题，这个目录为我们提供了参考。我们在后续的章节中，会使用这个目录中的例子来讲解 LwIP 的应用程序。

（4）ports 目录里是一些移植文件，它可以帮助我们将 LwIP 移植到某个具体的操作系统中。目前这个目录所提供的移植文件，只支持 FreeRTOS、UNIX、Win32。我们会在后续的章节中讲解如何移植 LwIP。

2.6 LwIP 的三种编程接口

LwIP 提供了三种编程接口，分别为 RAW/Callback API、NETCONN API、SOCKET API。它们的易用性从左到右依次提高，而执行效率从左到右依次降低，用户可以根据实际情况，平衡利弊，选择合适的 API 进行网络应用程序的开发。以下内容将分别介绍这三种 API。

2.6.1 RAW/Callback API

RAW/Callback API 是指内核回调型的 API，这在许多通信协议的 C 语言实现中都有所应用。对于从来没有接触过回调式编程的人来说，可能理解起来会比较困难，我们在后面的章节中会详细介绍它。

RAW/Callback API 是 LwIP 的一大特色，在没有操作系统支持的裸机环境中，只能使用这种 API 进行开发，同时这种 API 也可以用在操作系统环境中。这里先简要说明一下“回调”的概念。你新建了一个 TCP 或者 UDP 的连接，你想等它接收到数据以后去处理它们，这时你需要把处理该数据的操作封装成一个函数，然后将这个函数的指针注册到 LwIP 内核中。LwIP 内核会在需要的时候去检测该连接是否收到数据，如果收到了数据，内核会在第一时间调用注册的函数，这个过程被称为“回调”，这个注册函数被称为“回调函数”。这个回调函数中装着你想要的业务逻辑，在这个函数中，你可以自由地处理接收到的数据，也可以发送任何数据，也就是说，这个回调函数就是你的应用程序。到这里，我们可以发现，在回调编程中，LwIP 内核把数据交给应用程序的过程就只是一次简单的函数调用，这是非常节省时间和空间资源的。每一个回调函数实际上只是一个普通的 C 函数，这个函数在 TCP/IP 内核中被调用。每一个回调函数都作为一个参数传递给当前 TCP 或 UDP 连接。而且，为了能够保存程序的特定状态，可以向回调函数传递一个指定的状态，并且这个指定的状态是独立于 TCP/IP 协议栈的。。

在有操作系统的环境中，如果使用 RAW/Callback API，用户的应用程序就以回调函数的形式成为了内核代码的一部分，用户应用程序和内核程序会处于同一个线程之中，这就省去了任务间通信和切换任务的开销了。

简单来说，RAW/Callback API 的优点有两个：

（1）可以在没有操作系统的环境中使用。

（2）在有操作系统的环境中使用它，对比另外两种 API，可以提高应用程序的效率、节省内存开销。

RAW/Callback API 的优点是显著的，但缺点也是显著的：

（1）基于回调函数开发应用程序时的思维过程比较复杂。在后面与 RAW/Callback API 相关的章节中可以看到，利用回调函数去实现复杂的业务逻辑时，会很麻烦，而且代码的可读性较差。

（2）在操作系统环境中，应用程序代码与内核代码处于同一个线程，虽然能够节省任务间通信和切换任务的开销，但是相应地，应用程序的执行会制约内核程序的执行，不同的应用程序之间也会互相制约。在应用程序执行的过程中，内核程序将不可能得到运行，这会影响网络数据包的处理效率。如果应用程序占用的时间过长，而且碰巧这时又有大量的数据包到达，由于内核代码长期得不到执行，网卡接收缓存里的数据包就持续积累，到最后很可能因为满载而丢弃一些数据包，从而造成丢包的现象。

2.6.2 NETCONN API

在操作系统环境中，可以使用 NETCONN API 或者 Socket API 进行网络应用程序的开发。NETCONN API 是基于操作系统的 IPC 机制（即信号量和邮箱机制）实现的，它的设计将 LwIP 内核代码和网络应用程序分离成了独立的线程。如此一来，LwIP 内核线程就只负责数据包的 TCP/IP 封装和拆封，而不用进行数据的应用层处理，大大提高了系统对网络数据包的处理效率。

前面提到，使用 RAW/Callback API 会造成内核程序和网络应用程序、不同网络应用程序之间的相互制约，如果使用 NETCONN API 或者 Socket API，这种制约将不复存在。

在操作系统环境中，LwIP 内核会被实现为一个独立的线程，名为 tcpip_thread，使用 NETCONN API 或者 Socket API 的应用程序处在不同的线程中，我们可以根据任务的重要性，分配不同的优先级给这些线程，从而保证重要任务的时效性，分配优先级的原则具体见表格 2-1。

表格 2-1 线程优先级分配原则

线程	优先级
LwIP 内核线程 tcpip_thread	很高
重要的网络应用程序	高
不太重要而且处理数据比较耗时的网络应用程序	低

NETCONN API 使用了操作系统的 IPC 机制，对网络连接进行了抽象，用户可以像操作文件一样操作网络连接（打开/关闭、读/写数据）。但是 NETCONN API 并不如操作文件的 API 那样简单易用。举个例子，调用 f_read 函数读文件时，读到的数据会被放在一个用户指定的数组中，用户操作起来很方便，而 NETCONN API 的读数据 API，就没有那么人性化了。用户获得的不是一个数组，而是一个特殊的数据结构 netbuf，用户如果想使用好它，就需要对内核的 pbuf 和 netbuf 结构体有所了解，我们会在后续的章节中对它们进行讲解。NETCONN API 之所以采取这种不人性的设计，是为了避免数据包在内核程序和应用程序之间发生拷贝，从而降低程序运行效率。当然，用户如果不在意数据递交时的效率问题，也可以把 netbuf 中的数据取出来拷贝到一个数组中，然后去处理这个数组。

简单来说，NETCONN API 的优缺点是:

（1）相较于 RAW/Callback API，NETCONN API 简化了编程工作，使用户可以按照操作文件的方式来操作网络连接。但是，内核程序和网络应用程序之间的数据包传递，需要依靠操作系统的信号量和邮箱机制完成，这需要耗费更多的时间和内存，另外还要加上任务切换的时间开销，效率较低。

（2）相较于 Socket API，NETCONN API 避免了内核程序和网络应用程序之间的数据拷贝，提高了数据递交的效率。但是，NETCONN API 的易用性不如 Socket API 好，它需要用户对 LwIP 内核所使用数据结构有一定的了解。

2.6.3 SOCKET API

Socket，即套接字，它对网络连接进行了高级的抽象，使得用户可以像操作文件一样操作网络连接。它十分易用，许多网络开发人员最早接触的就是 Socket 编程，Socket 已经成为了网络编程的标准。在不同的系统中，运行着不同的 TCP/IP 协议，但是只要它实现了 Socket 的接口，那么用 Socket 编写的网络应用程序就能在其中运行。可见用 Socket 编写的网络应用程序具有很好的可移植性。

不同的系统有自己的一套 Socket 接口。Windows 系统中支持的是 WinSock，UNIX/Linux 系统中支持的是 BSD Socket，它们虽然风格不一致，但大同小异。LwIP 中的 Socket API 是 BSD Socket。但是 LwIP 并没有也没办法实现全部的 BSD Socket，如果开发人员想要移植 UNIX/Linux 系统中的网络应用程序到使用 LwIP 的系统中，就要注意这一点。

相较于 NETCONN API，Socket API 具有更好的易用性。使用 Socket API 编写的程序可读性好，便于维护，也便于移植到其它的系统中。Socket API 在内核程序和应用程序之间存在数据的拷贝，这会降低数据递交的效率。另外，LwIP 的 Socket API 是基于 NETCONN API 实现的，所以效率上相较前者要打个折扣。

第 3 章　开发平台介绍

本章主要讲解如何使用 STM32 驱动其 ETH 章节的讲解，讲解好了基础外设，以便后面章节讲解 LwIP 移植时对接底层接口。在学习本章的过程中，可以参考我们出版的书籍《STM32 库开发实战指南》中的以太网章节。

3.1 以太网简介

以太网 (Ethernet) 是互联网技术的一种，由于它是在组网技术中占的比例最高，很多人直接把以太网理解为互联网。

以太网是指遵守 IEEE 802.3 标准组成的局域网，由 IEEE 802.3 标准规定的主要是位于参考模型的物理层 (PHY) 和数据链路层中的介质访问控制子层 (MAC)。在家庭、企业和学校所组建的 PC 局域网形式一般也是以太网，其标志是使用水晶头网线来连接 (当然还有其它形式)。IEEE 还有其它局域网标准，如 IEEE 802.11 是无线局域网，俗称 Wi-Fi。IEEE802.15 是个人域网，即蓝牙技术，其中的 802.15.4 标准则是 ZigBee 技术。

现阶段，工业控制、环境监测、智能家居的嵌入式设备产生了接入互联网的需求，利用以太网技术，嵌入式设备可以非常容易地接入到现有的计算机网络中。

3.1.1 PHY 层

在物理层，由 IEEE 802.3 标准规定了以太网使用的传输介质、传输速度、数据编码方式和冲突检测机制，物理层一般是通过一个 PHY 芯片实现其功能的，我们使用的是野火 STM32F429 挑战者开发板，板载的 PHY 芯片是 LAN8720A。

3.1.1.1 传输介质

传输介质包括同轴电缆、双绞线(水晶头网线是一种双绞线)、光纤。根据不同的传输速度和距离要求，基于这三类介质的信号线又衍生出很多不同的种类。最常用的是“五类线”适用于 100BASE-T 和 10BASE-T 的网络，它们的网络速率分别为 100Mbps 和 10Mbps。

3.1.1.2 编码

为了让接收方在没有外部时钟参考的情况也能确定每一位的起始、结束和中间位置，在传输信号时不直接采用二进制编码。在 10BASE-T 的传输方式中采用曼彻斯特编码，在 100BASE-T 中则采用 4B/5B 编码。

曼彻斯特编码把每一个二进制位的周期分为两个间隔，在表示“1”时，以前半个周期为高电平，后半个周期为低电平。表示“0”时则相反，具体见图 *3_1*。

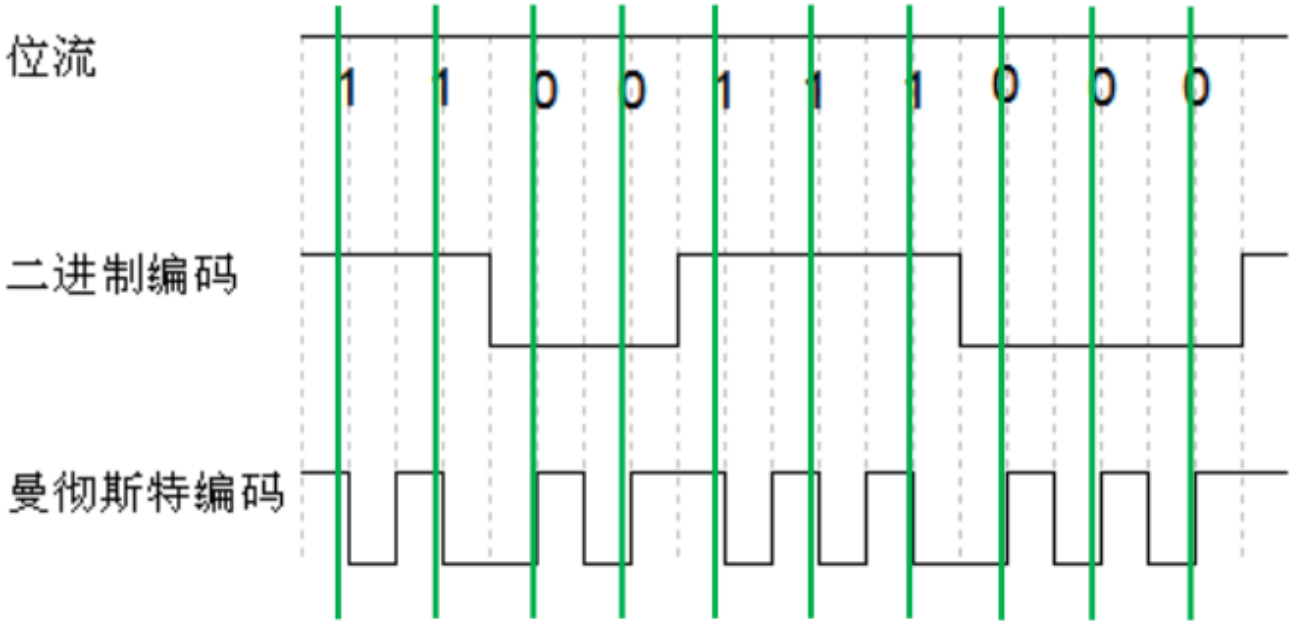


图 3-1 曼彻斯特编码

采用曼彻斯特码在每个位周期都有电压变化，便于同步。但这样的编码方式效率太低，只有 50%。

在 100BASE-T 采用的 4B/5B 编码是把待发送数据位流的每 4 位分为一组，以特定的 5 位编码来

表示，这些特定的 5 位编码能使数据流有足够多的跳变，达到同步的目的，而且效率也从曼彻斯特编码的 50% 提高到了 80%。

3.1.1.3 CSMA/CD 冲突检测

早期的以太网大多是多个节点连接到同一条网络总线上 (总线型网络)，存在信道竞争问题，因而每个连接到以太网上的节点都必须具备冲突检测功能。以太网具备 CSMA/CD 冲突检测机制，如果多个节点同时利用同一条总线发送数据，则会产生冲突，总线上的节点可通过接收到的信号与原始发送的信号的比较检测是否存在冲突，若存在冲突则停止发送数据，随机等待一段时间再重传。

现在大多数局域网组建的时候很少采用总线型网络，大多是一个设备接入到一个独立的路由或交换机接口，组成星型网络，不会产生冲突。但为了兼容，新出的产品还是带有冲突检测机制。

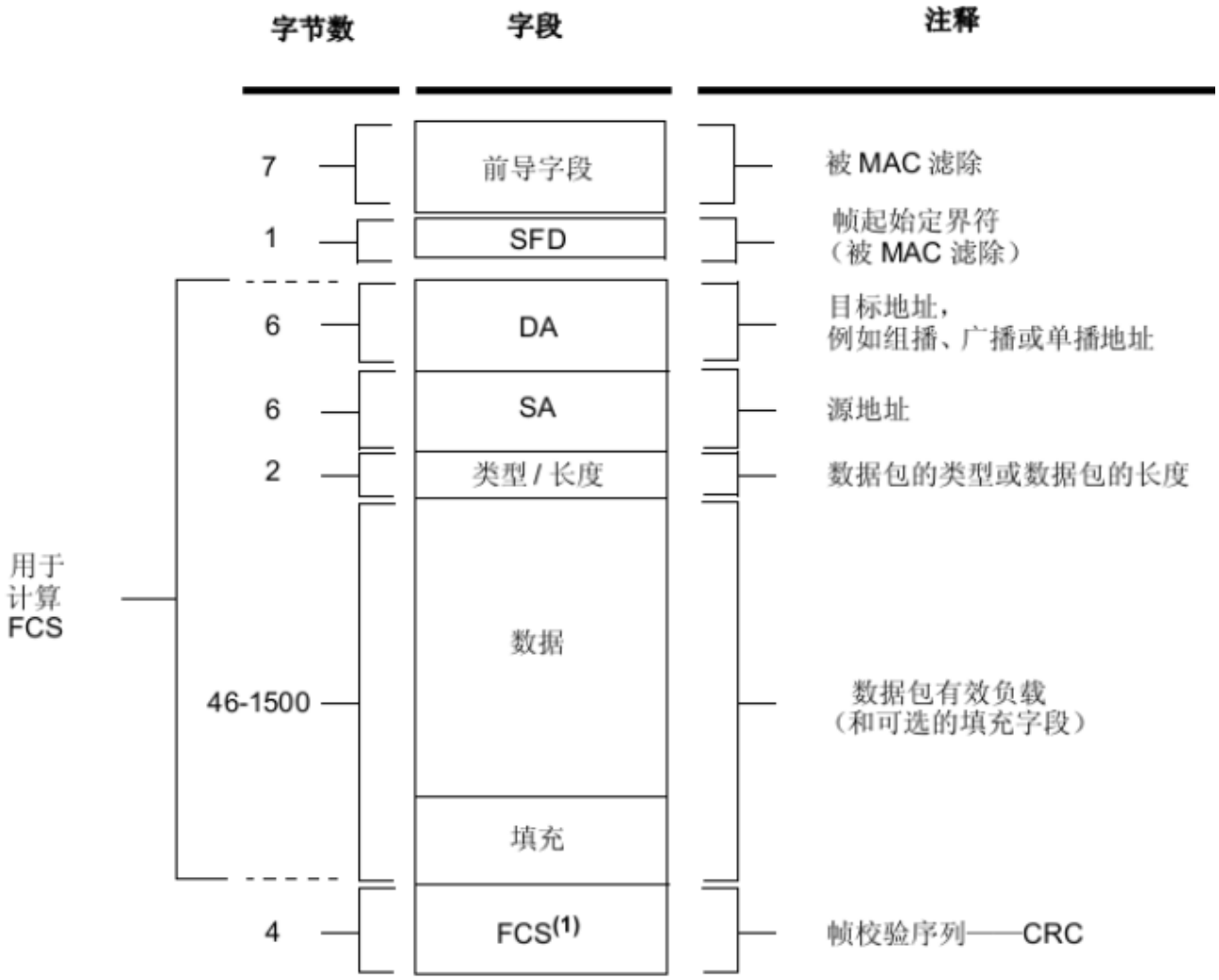
3.1.2 MAC 子层

3.1.2.1 MAC 的功能

MAC 子层是属于数据链路层的下半部分，它主要负责与物理层进行数据交接，如是否可以发送数据，发送的数据是否正确，对数据流进行控制等。它自动对来自上层的数据包加上一些控制信号，交给物理层。接收方得到正常数据时，自动去除 MAC 控制信号，把该数据包交给上层。

3.1.2.2 MAC 数据包

IEEE 对以太网上传输的数据包格式也进行了统一规定，见图 *3_2*。该数据包被称为 MAC 数据包。



注　1：发送 FCS 时，首先发送 bit 31 最后发送 bit 0。

图 3-2 MAC 数据包格式

MAC 数据包由前导字段、帧起始定界符、目标地址、源地址、数据包类型、数据域、填充域、校验和域组成。

- 前导字段，也称报头，这是一段方波，用于使收发节点的时钟同步。内容为连续 7 个字节

的 0x55。字段和帧起始定界符在 MAC 收到数据包后会自动过滤掉。

- 帧起始定界符 (SFD)：用于区分前导段与数据段的，内容为 0xD5。
- MAC 地址：MAC 地址由 48 位数字组成，它是网卡的物理地址，在以太网传输的最底层，就是根据 MAC 地址来收发数据的。部分 MAC 地址用于广播和多播，在同一个网络里不能有两个相同的 MAC 地址。PC 的网卡在出厂时已经设置好了 MAC 地址，但也可以通过一些软件来进行修改，在嵌入式的以太网控制器中可由程序进行配置。数据包中的 DA 是目标地址，SA 是源地址。
- 数据包类型：本区域可以用来描述本 MAC 数据包是属于 TCP/IP 协议层的 IP 包、ARP 包还是 SNMP 包，也可以用来描述本 MAC 数据包数据段的长度。如果该值被设置大于 0x0600，不用于长度描述，而是用于类型描述功能，表示与以太网帧相关的 MAC 客户端协议的种类。
- 数据段：数据段是 MAC 包的核心内容，它包含的数据来自 MAC 的上层。其长度可以从 0~1500 字节间变化。
- 填充域：由于协议要求整个 MAC 数据包的长度至少为 64 字节 (接收到的数据包如果少于 64 字节会被认为发生冲突，数据包被自动丢弃)，当数据段的字节少于 46 字节时，在填充域会自动填上无效数据，以使数据包符合长度要求。
- 校验和域：MAC 数据包的尾部是校验和域，它保存了 CRC 校验序列，用于检错。

以上是标准的 MAC 数据包，IEEE 802.3 同时还规定了扩展的 MAC 数据包，它是在标准的 MAC 数据包的 SA 和数据包类型之间添加 4 个字节的 QTag 前缀字段，用于获取标志的 MAC 帧。前 2 个字节固定为 0x8100，用于识别 QTag 前缀的存在；后两个字节内容分别为 3 个位的用户优先级、1 个位的标准格式指示符 (CFI) 和一个 12 位的 VLAN 标识符。

3.2 STM32 的 ETH 外设

STM32F42x 系列控制器内部集成了一个以太网外设，它实际是一个通过 DMA 控制器进行介质访问控制 (MAC)，它的功能就是实现 MAC 层的任务。借助以太网外设，STM32F42x 控制器可以通过 ETH 外设按照 IEEE 802.3-2002 标准发送和接收 MAC 数据包。

ETH 内部自带专用的 DMA 控制器用于 MAC，ETH 支持两个工业标准接口介质独立接口 (MII) 和简化介质独立接口 (RMII) 用于与外部 PHY 芯片连接。MII 和 RMII 接口用于 MAC 数据包传输，ETH 还集成了站管理接口 (SMI) 接口专门用于与外部 PHY 通信，用于访问 PHY 芯片寄存器。

物理层定义了以太网使用的传输介质、传输速度、数据编码方式和冲突检测机制，PHY 芯片是物理层功能实现的实体，生活中常用水晶头网线 + 水晶头插座 +PHY 组合构成了物理层。

ETH 有专用的 DMA 控制器，它通过 AHB 主从接口与内核和存储器相连，AHB 主接口用于控制数据传输，而 AHB 从接口用于访问 “控制与状态寄存器” (CSR) 空间。在进行数据发送是，先将数据有存储器以 DMA 传输到发送 TX FIFO 进行缓冲，然后由 MAC 内核发送；接收数据时，RX FIFO 先接收以太网数据帧，再由 DMA 传输至存储器。ETH 系统功能框图见图 *3_3*。

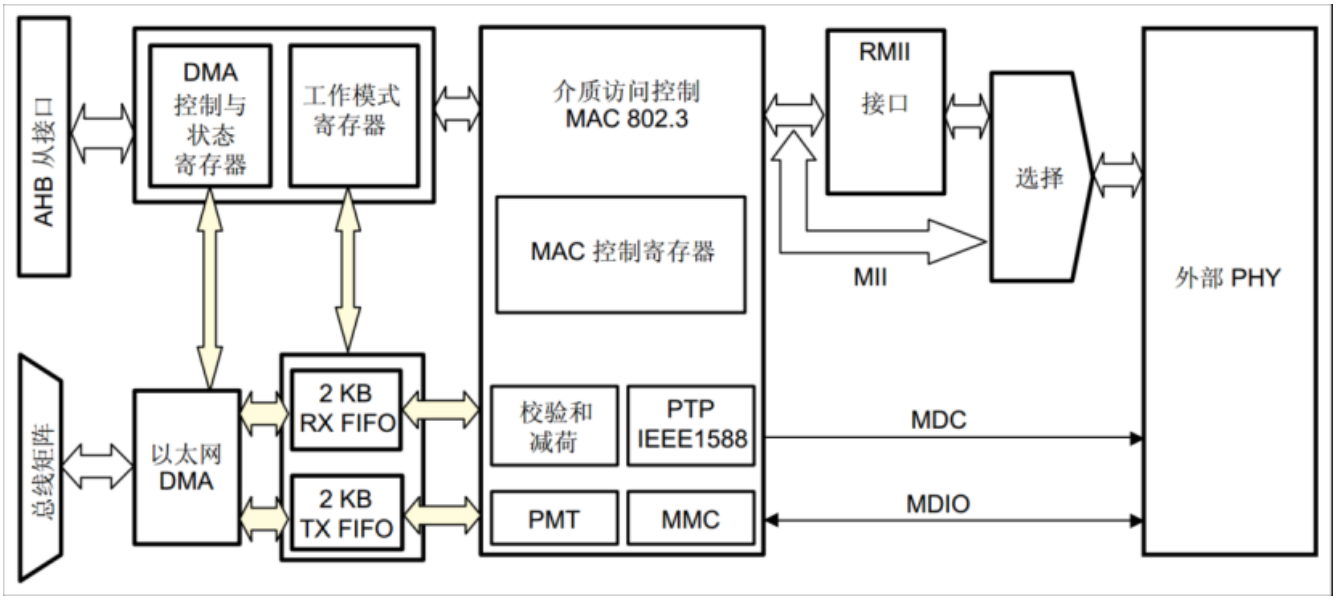


图 3-3 ETH 功能框图

3.3 MII 和 RMII 接口

介质独立接口 (MII) 用于理解 MAC 控制器和 PHY 芯片，提供数据传输路径。RMII 接口是 MII 接口的简化版本，MII 需要 16 根通信线，RMII 只需 7 根通信，在功能上是相同的。图 *3_4* 为 MII 接口连接示意图，图 *3_5* 为 RMII 接口连接示意图。

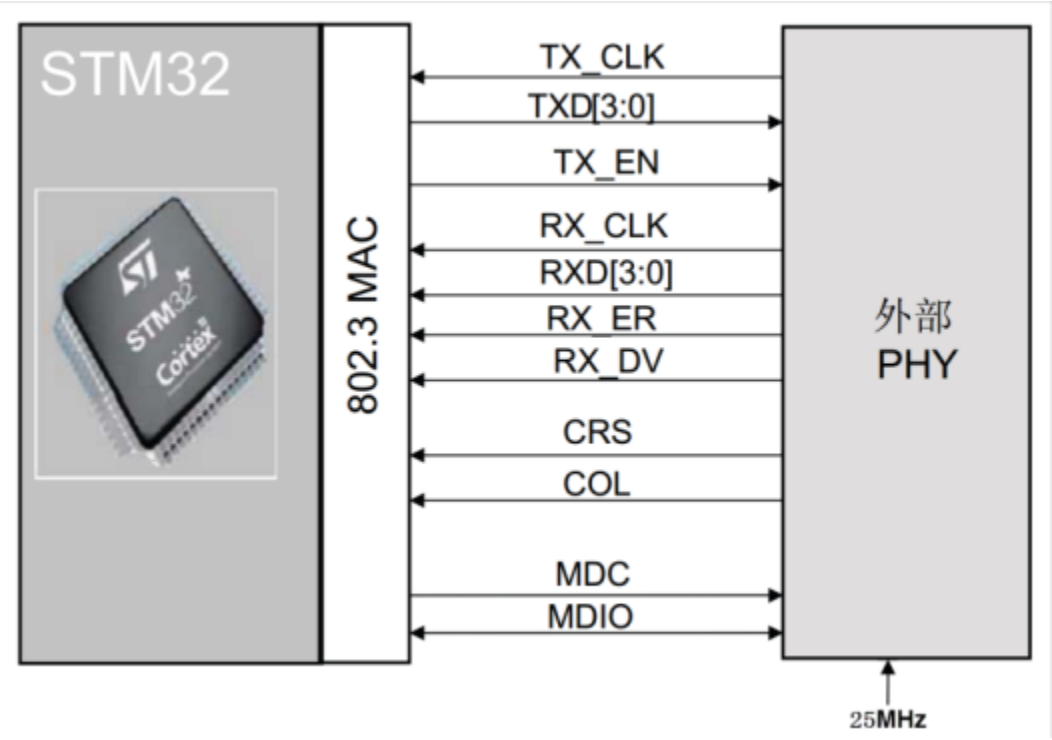


图 3-4 MII 接口连接示意图

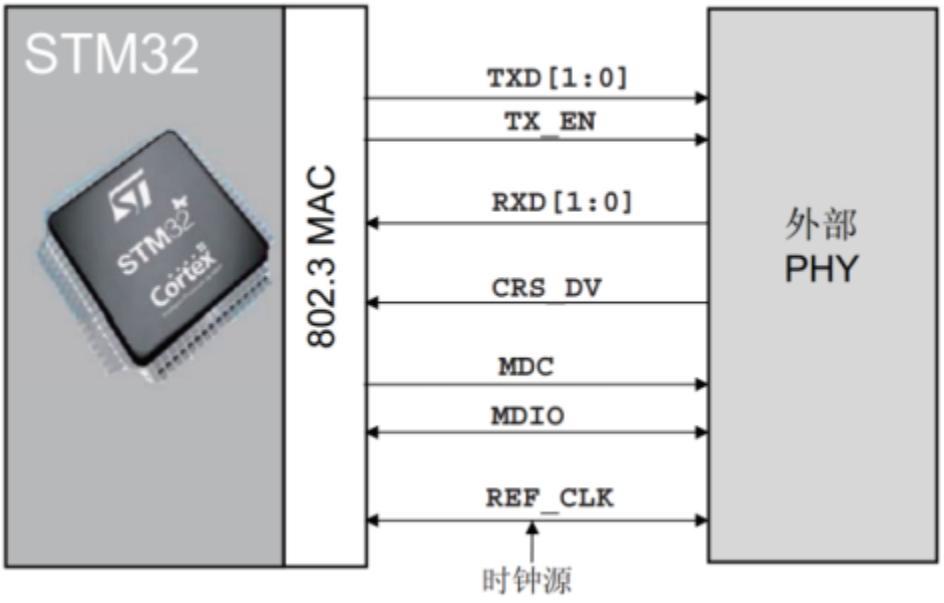


图 3-5 RMII 接口连接示意图

- TX_CLK：数据发送时钟线。标称速率为 10Mbit/s 时为 2.5MHz；速率为 100Mbit/s 时为

25MHz。RMII 接口没有该线。

- RX_CLK：数据接收时钟线。标称速率为 10Mbit/s 时为 2.5MHz；速率为 100Mbit/s 时为 25MHz。RMII 接口没有该线。
- TX_EN：数据发送使能。在整个数据发送过程保存有效电平。
- TXD[3:0] 或 TXD[1:0]：数据发送数据线。对于 MII 有 4 位，RMII 只有 2 位。只有在 TX_EN 处于有效电平数据线才有效。
- CRS：载波侦听信号，由 PHY 芯片负责驱动，当发送或接收介质处于非空闲状态时使能该信号。在全双工模式该信号线无效。
- COL：冲突检测信号，由 PHY 芯片负责驱动，检测到介质上存在冲突后该线被使能，并且保持至冲突解除。在全双工模式该信号线无效。
- RXD[3:0] 或 RXD[1:0]：数据接收数据线，由 PHY 芯片负责驱动。对于 MII 有 4 位，RMII 只有 2 位。在 MII 模式，当 RX_DV 禁止、RX_ER 使能时，特定的 RXD[3:0] 值用于传输来自 PHY 的特定信息。
- RX_DV：接收数据有效信号，功能类似 TX_EN，只不过用于数据接收，由 PHY 芯片负责驱动。对于 RMII 接口，是把 CRS 和 RX_DV 整合成 CRS_DV 信号线，当介质处于不同状态时会自切换该信号状态。
- RX_ER：接收错误信号线，由 PHY 驱动，向 MAC 控制器报告在帧某处检测到错误。
- REF_CLK：仅用于 RMII 接口，由外部时钟源提供 50MHz 参考时钟。因为要达到 100Mbit/s 传输速度，MII 和 RMII 数据线数量不同，使用 MII 和 RMII 在时钟线的设计是完全不同的。对于 MII 接口，一般是外部为 PHY 提供 25MHz 时钟源，再由 PHY 提供 TX_CLK 和 RX_CLK 时钟。对于 RMII 接口，一般需要外部直接提供 50MHz 时钟源，同时接入 MAC 和 PHY。

开发板板载的 PHY 芯片型号为 LAN8720A，该芯片只支持 RMII 接口，电路设计时参考图 *3_6*。

ETH(AF11)		GPIO
MII	MII_TX_CLK	PC3
	MII_TXD0	PB12/PG13
	MII_TXD1	PB13/PG14
	MII_TXD2	PC2
	MII_TXD3	PB8/PE2
	MII_TX_EN	PB11/PG11
	MII_RX_CLK	PA1
	MII_RXD0	PC4
	MII_RXD1	PC5
	MII_RXD2	PB0/PH6
	MII_RXD3	PB1/PH7
	MII_RX_ER	PB10/PI10
	MII_RX_DV	PA7
	MII_CRS	PA0/PH2
	MII_COL	PA3/PH3
RMII	RMII_TXD0	PB12/PG13
	RMII_TXD1	PB13/PG14
	RMII_TX_EN	PG11
	RMII_RXD0	PC4
	RMII_RXD1	PC5
	RMII_CRS_DV	PA7
	RMII_REF_CLK	PA1
SMI	MDIO	PA2
	MDC	PC1
其他	PPS_OUT	PB5/PG8

图 3-6 ETH 复用引脚

注：其中，PPS_OUT 是 IEEE 1588 定义的一个时钟同步机制。

3.4 PHY：LAN8720A

LAN8720A 是 SMSC 公司 (已被 Microchip 公司收购) 设计的一个体积小、功耗低、全能型 10/100Mbps 的以太网物理层收发器。它是针对消费类电子和企业应用而设计的。LAN8720A 总共只有 24Pin，仅支持 RMII 接口。由它组成的网络结构见图 *3_7*。

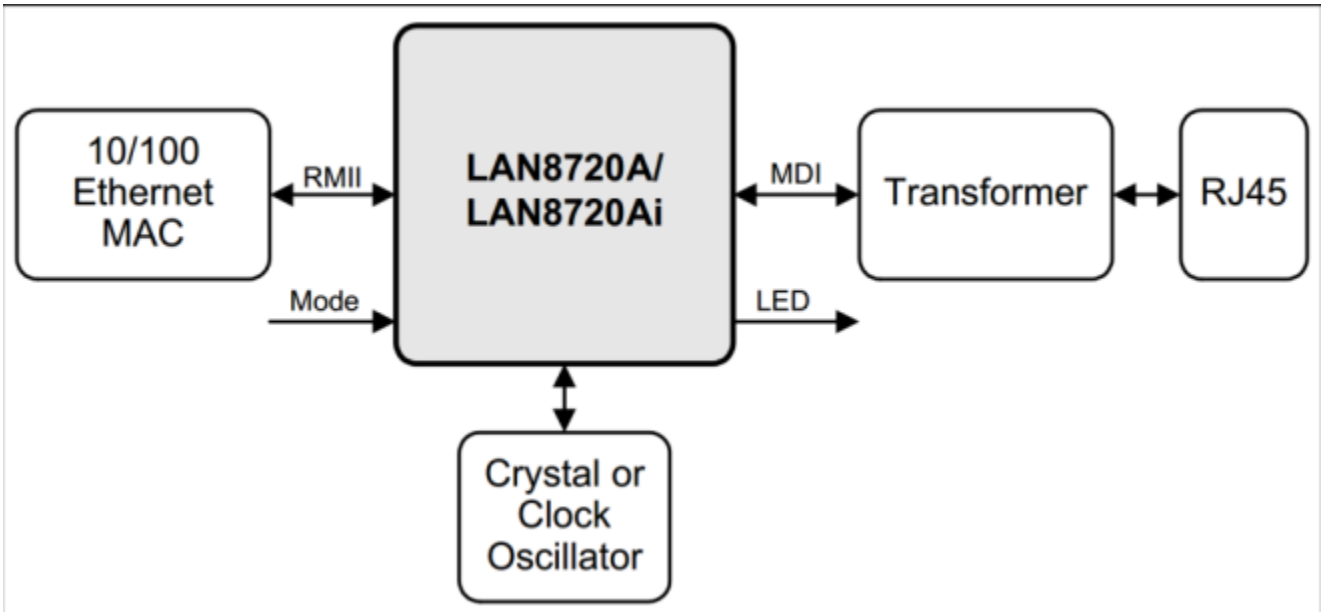


图 3-7 由 LAN8720A 组成的网络系统结构

LAN8720A 通过 RMII 与 MAC 连接。RJ45 是网络插座，在与 LAN8720A 连接之间还需要一个变压器，所以一般使用带电压转换和 LED 指示灯的 HY911105A 型号的插座。一般来说，必须为使用 RMII 接口的 PHY 提供 50MHz 的时钟源输入到 REF_CLK 引脚，不过 LAN8720A 内部集成 PLL，可以将 25MHz 的时钟源陪频到 50MHz 并在指定引脚输出该时钟，所以我们可以直接使其与 REF_CLK 连接达到提供 50MHz 时钟效果。

LAN8720A 内部系统结构见图 *3_8*。

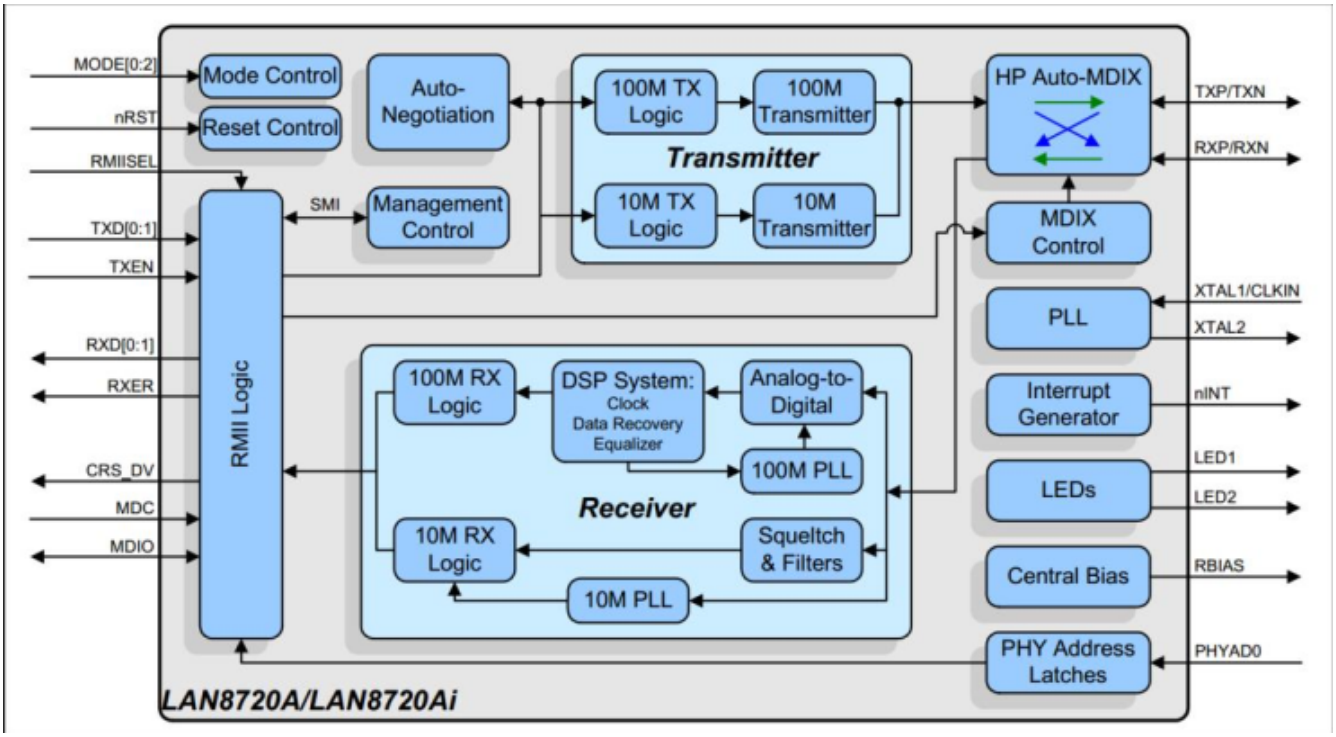


图 3-8LAN8720A 内部系统结构

LAN8720A 有各个不同功能模块组成，最重要的要数接收控制器和发送控制器，其它的基本上都是与外部引脚挂钩，实现信号传输。部分引脚是具有双重功能的，比如 PHYAD0 与 RXER 引脚是共用的，在系统上电后 LAN8720A 会马上读取这部分共用引脚的电平，以确定系统的状态并保存在相关寄存器内，之后则自动转入作为另一功能引脚。

PHYAD[0] 引脚用于配置 SMI 通信的 LAN8720A 地址，在芯片内部该引脚已经自带下拉电阻，默认认为 0(即使外部悬空不接)，在系统上电时会检测该引脚获取得到 LAN8720A 的地址为 0 或者 1，并保存在特殊模式寄存器 (R18) 的 PHYAD 位中，该寄存器的 PHYAD 有 5 个位，在需要超过 2 个 LAN8720A 时可以通过软件设置不同 SMI 通信地址。PHYAD[0] 是与 RXER 引脚共用。

MODE[2:0] 引脚用于选择 LAN8720A 网络通信速率和工作模式，可选 10Mbps 或 100Mbps 通信速度，半双工或全双工工作模式，另外 LAN8720A 支持 HP Auto-MDIX 自动翻转功能，即可自动识别直连或交叉网线并自适应。一般将 MODE 引脚都设置为 1，可以让 LAN8720A 启动自适应功能，它会自动寻找最优工作方式。MODE[0] 与 RXD0 引脚共用、MODE[1] 与 RXD1 引脚共用、MODE[2] 与 CRS_DV 引脚共用。

nINT/REFCLKO 引脚用于 RMII 接口中 REF_CLK 信号线，当 nINTSEL 引脚为低电平是，它也

可以被设置成 50MHz 时钟输出，这样可以直接与 STM32F42x 的 REF_CLK 引脚连接为其提供 50MHz 时钟源，这种模式要求为 XTAL1 与 XTAL2 之间或为 TAL1/CLKIN 提供 25MHz 时钟，由 LAN8720A 内部 PLL 电路陪频得到 50MHz 时钟，此时 nIN/REFCLKO 引脚的中断功能不可用，用于 50MHz 时钟输出。当 nINTSEL 引脚为高电平时，LAN8720A 被设置为时钟输入，即外部时钟源直接提供 50MHz 时钟接入 STM32F42x 的 REF_CLK 引脚和 LAN8720A 的 XTAL1/CLKIN 引脚，此时 INT/REFCLKO 可用于中断功能。nINTSEL 与 LED2 引脚共用，一般使用下拉。

REGOFF 引脚用于配置内部 +1.2V 电压源，LAN8720A 内部需要 +1.2V 电压，可以通过 VDDCR 引脚输入 +1.2V 电压提供，也可以直接利用 LAN8720A 内部 +1.2V 稳压器提供。当 REGOFF 引脚为低电平时选择内部 +1.2V 稳压器。REGOFF 与 LED1 引脚共用。

SMI 支持寻址 32 个寄存器，LAN8720A 只用到其中 14 个，参考图 *3_9*。

序号	寄存器名称	分组
0	Basic Control Register	Basic
1	Basic Status Register	Basic
2	PHY Identifier 1	Extended
3	PHY Identifier 2	Extended
4	Auto-Negotiation Advertisement Register	Extended
5	Auto-Negotiation Link Partner Ability Register	Extended
6	Auto-Negotiation Expansion Register	Extended
17	Mode Control/Status Register	Vendor-specific
18	Special Modes	Vendor-specific
26	Symbol Error Counter Register	Vendor-specific
27	Control / Status Indication Register	Vendor-specific
29	Interrupt Source Register	Vendor-specific
30	Interrupt Mask Register	Vendor-specific
31	PHY Special Control/Status Register	Vendor-specific

图 3-9LAN8720A 寄存器列表

序号与 SMI 数据帧中的 RADDR 是对应的，这在编写驱动时非常重要，本文将它们标记为

R0~R31。寄存器可规划为三个组：Basic、Extended 和 Vendor-specific。Basic 是 IEEE802.3 要求的，R0 是基本控制寄存器，其位 15 为 Soft Reset 位，向该位写 1 启动 LAN8720A 软件复位，还包括速度、自适应、低功耗等等功能设置。R1 是基本状态寄存器。Extended 是扩展寄存器，包括 LAN8720A 的 ID 号、制造商、版本号等等信息。Vendorspecific 是供应商自定义寄存器，R31 是特殊控制/状态寄存器，指示速度类型和自适应功能。

3.5 硬件设计

在讲解移植步骤之前，有必须先介绍我们的实验硬件设计，主要是 LAN8720A 通过 RMII 和 SMI 接口与 STM32F42x 控制器连接，见图 *3_10*。

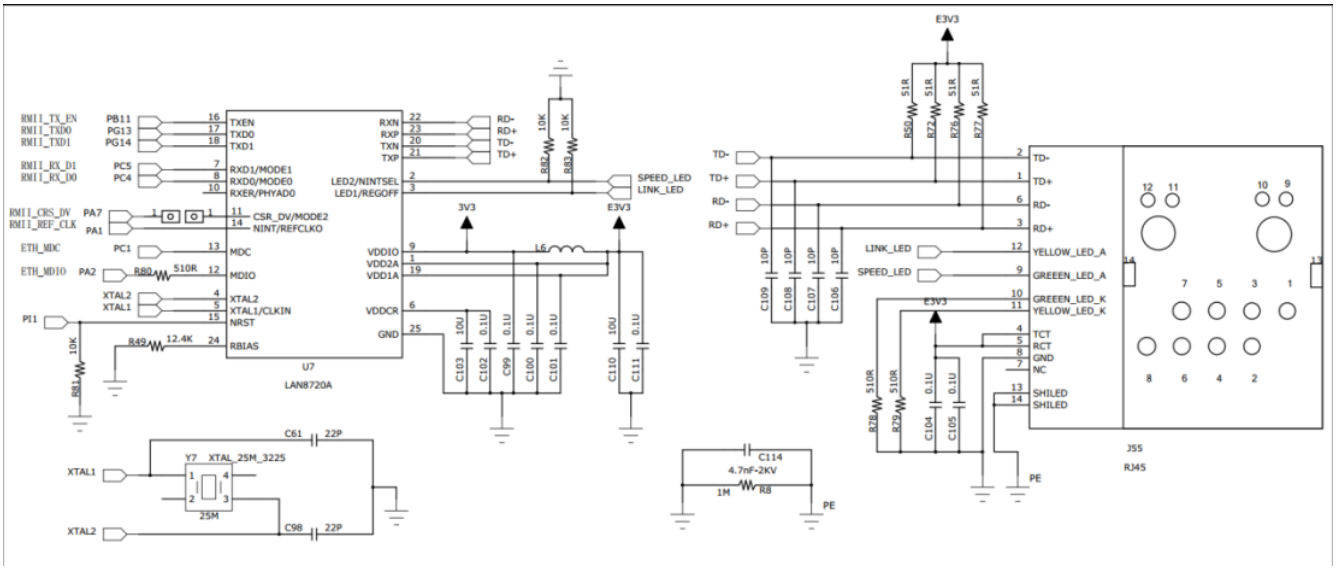


图 3-10PHY 硬件设计

电路设计时，将 NINTSEL 引脚通过下拉电阻拉低，设置 NINT/FEFCLKO 为输出 50MHz 时钟，当然前提是在 XTAL1 和 XTAL2 接入了 25MHz 的时钟源。另外也把 REGOFF 引脚通过下拉电阻拉低，使能使用内部 +1.2V 稳压器。

3.6 软件设计

3.6.1 获取 STM32 的裸机工程模板

STM32 的裸机工程模板我们直接使用野火 STM32 开发板配套的 HAL 库例程即可。这里我们选取比较简单的例程—“GPIO 输出—使用固件库点亮 LED”作为裸机工程模板。该裸机工程模板均可以在对应板子的 A 盘/程序源码目录下获取到，下面以野火 STM32F429-挑战者开发板的光盘目录为例，获取到一个简单的裸机例程，具体见图 *3_11*。



图 3-11 获取 STM32 的裸机工程模板

3.6.2 添加 bsp_eth.c 与 bsp_eth.h

我们打开裸机工程之后，就创建一个文件夹，命名为 eth，并且在该文件夹下创建两个文件，分别为 bsp_eth.c 与 bsp_eth.h 文件，具体见图 *3_12*。



图 3-12 创建 bsp_eth.c 与 bsp_eth.h 文件

然后再将 bsp_eth.c 文件添加到工程分组中，具体见图 *3_13*。

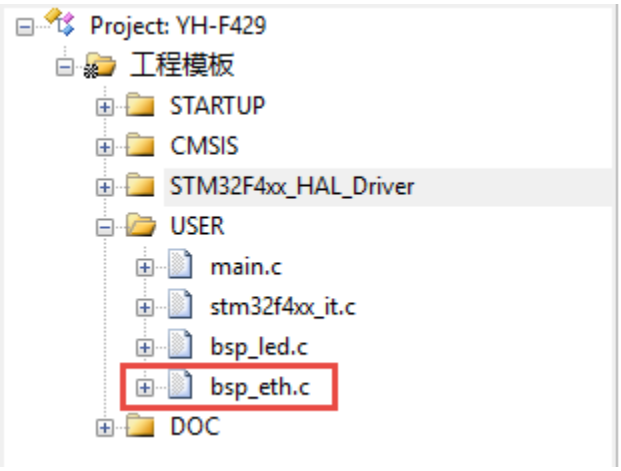


图 3-13 将 bsp_eth.c 添加到工程中

然后我们就可以在 bsp_eth.c 文件中进行初始化 eth 驱动了，暂时加入以下代码，具体见代码清单 *3_1*。

代码清单 3-1 bsp_eth.c 内容

```
/**
*****************************************************************************

* @file    main.c
* @author  fire
* @version V1.0
* @date    2019-xx-xx
* @brief   eth
*****************************************************************************
* @attention
*
* 实验平台: 野火 STM32 F429 开发板
* 论坛     :http://www.firebbs.cn
* 淘宝     :http://firestm32.taobao.com
```

(下页继续)

(续上页)

```
*
*******************************************************************************
*/
#include "./eth/bsp_eth.h"
#include "main.h"

#ifndef PRINT_DEBUG
#define PRINT_DEBUG
#endif

#ifndef PRINT_ERR
#define PRINT_ERR
#endif

/* Global Ethernet handle */
ETH_HandleTypeDef heth;

#if defined ( __ICCARM__ ) /*!< IAR Compiler */
#pragma data_alignment=4
#endif
__ALIGN_BEGIN ETH_DMADescTypeDef  DMARxDscrTab[ETH_RXBUFNB] __ALIGN_END;
/* Ethernet Rx MA Descriptor */

#if defined ( __ICCARM__ ) /*!< IAR Compiler */
#pragma data_alignment=4
#endif
__ALIGN_BEGIN ETH_DMADescTypeDef  DMATxDscrTab[ETH_TXBUFNB] __ALIGN_END;
/* Ethernet Tx DMA Descriptor */

#if defined ( __ICCARM__ ) /*!< IAR Compiler */
#pragma data_alignment=4
#endif
```

(下页继续)

(续上页)

```
__ALIGN_BEGIN uint8_t Rx_Buff[ETH_RXBUFNB][ETH_RX_BUF_SIZE] __ALIGN_END;
/* Ethernet Receive Buffer */

#if defined ( __ICCARM__ ) /*!< IAR Compiler */
#pragma data_alignment=4
#endif
__ALIGN_BEGIN uint8_t Tx_Buff[ETH_TXBUFNB][ETH_TX_BUF_SIZE] __ALIGN_END;
/* Ethernet Transmit Buffer */


void HAL_ETH_MspInit(ETH_HandleTypeDef* ethHandle)
{
    GPIO_InitTypeDef GPIO_InitStruct;
    if (ethHandle->Instance==ETH)
    {
        /* USER CODE BEGIN ETH_MspInit 0 */

        /* USER CODE END ETH_MspInit 0 */
//    /* Enable Peripheral clock */
//    __HAL_RCC_ETH_CLK_ENABLE();

        /**ETH GPIO Configuration
        PC1     ------> ETH_MDC
        PA1     ------> ETH_REF_CLK
        PA2     ------> ETH_MDIO
        PA7     ------> ETH_CRS_DV
        PC4     ------> ETH_RXD0
        PC5     ------> ETH_RXD1
        PB11     ------> ETH_TX_EN
        PG13     ------> ETH_TXD0
        PG14     ------> ETH_TXD1
        */
```

(下页继续)

(续上页)

```
    GPIO_InitStruct.Pin = ETH_MDC_Pin|ETH_RXD0_Pin|ETH_RXD1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF11_ETH;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = ETH_REF_CLK_Pin|ETH_MDIO_Pin|ETH_CRS_DV_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF11_ETH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = ETH_TX_EN_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF11_ETH;
    HAL_GPIO_Init(ETH_TX_EN_GPIO_Port, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = ETH_TXD0_Pin|ETH_TXD1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate = GPIO_AF11_ETH;
    HAL_GPIO_Init(GPIOG, &GPIO_InitStruct);

    /* USER CODE BEGIN ETH_MspInit 1 */
    /* Enable the Ethernet global Interrupt */
    HAL_NVIC_SetPriority(ETH_IRQn, 6, 0);
    HAL_NVIC_EnableIRQ(ETH_IRQn);
```

(下页继续)

(续上页)

```
        /* Enable ETHERNET clock  */
        __HAL_RCC_ETH_CLK_ENABLE();
        /* USER CODE END ETH_MspInit 1 */
    }
}

static void Eth_Reset(void)
{
    /* PHY RESET: PI1 */
    GPIO_InitTypeDef GPIO_InitStructure;
    __HAL_RCC_GPIOI_CLK_ENABLE();

    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull  = GPIO_PULLUP;
    GPIO_InitStructure.Speed = GPIO_SPEED_FAST;
    GPIO_InitStructure.Pin = GPIO_PIN_1;
    HAL_GPIO_Init(GPIOI, &GPIO_InitStructure);
    HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_RESET);
    HAL_Delay(5);
    HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_SET);
    HAL_Delay(5);
}

void HAL_ETH_MspDeInit(ETH_HandleTypeDef* ethHandle)
{
    if (ethHandle->Instance==ETH)
    {
        /* USER CODE BEGIN ETH_MspDeInit 0 */

        /* USER CODE END ETH_MspDeInit 0 */
        /* Peripheral clock disable */
```

(下页继续)

(续上页)

```
        __HAL_RCC_ETH_CLK_DISABLE();

        /**ETH GPIO Configuration
        PC1     ------> ETH_MDC
        PA1     ------> ETH_REF_CLK
        PA2     ------> ETH_MDIO
        PA7     ------> ETH_CRS_DV
        PC4     ------> ETH_RXD0
        PC5     ------> ETH_RXD1
        PB11     ------> ETH_TX_EN
        PG13     ------> ETH_TXD0
        PG14     ------> ETH_TXD1
        */
        HAL_GPIO_DeInit(GPIOC, ETH_MDC_Pin|ETH_RXD0_Pin|ETH_RXD1_Pin);

        HAL_GPIO_DeInit(GPIOA, ETH_REF_CLK_Pin|ETH_MDIO_Pin|ETH_CRS_DV_
↪Pin);

        HAL_GPIO_DeInit(ETH_TX_EN_GPIO_Port, ETH_TX_EN_Pin);

        HAL_GPIO_DeInit(GPIOG, ETH_TXD0_Pin|ETH_TXD1_Pin);

        /* USER CODE BEGIN ETH_MspDeInit 1 */

        /* USER CODE END ETH_MspDeInit 1 */
    }
}

HAL_StatusTypeDef Bsp_Eth_Init(void)
{
    HAL_StatusTypeDef ret;
```

(下页继续)

(续上页)

```
    uint8_t MACAddr[6] ;

    HAL_ETH_DeInit(&heth);

    Eth_Reset();

    ETH->DMABMR |= ETH_DMABMR_SR;

    /* Init ETH */
    MACAddr[0] = 0x02;
    MACAddr[1] = 0x00;
    MACAddr[2] = 0x00;
    MACAddr[3] = 0x00;
    MACAddr[4] = 0x00;
    MACAddr[5] = 0x00;
    heth.Instance = ETH;
    heth.Init.AutoNegotiation = ETH_AUTONEGOTIATION_ENABLE;
    heth.Init.PhyAddress = LAN8720_PHY_ADDRESS;
    heth.Init.MACAddr = &MACAddr[0];
    heth.Init.RxMode = ETH_RXPOLLING_MODE;  // rx mode
    heth.Init.ChecksumMode = ETH_CHECKSUM_BY_HARDWARE;
    heth.Init.MediaInterface = ETH_MEDIA_INTERFACE_RMII;
    heth.Init.Speed = ETH_SPEED_100M;       //speed
    heth.Init.DuplexMode = ETH_MODE_FULLDUPLEX;

    /* configure ethernet peripheral (GPIOs, clocks, MAC, DMA) */
    ret = HAL_ETH_Init(&heth);
    if (ret == HAL_OK)
        PRINT_DEBUG("eth hardware init sucess...\n");
    else
        PRINT_DEBUG("eth hardware init faild...\n");
```

(下页继续)

(续上页)

```
    /* Initialize Tx Descriptors list: Chain Mode */
    HAL_ETH_DMATxDescListInit(&heth, DMATxDscrTab, &Tx_Buff[0][0], ETH_
↪TXBUFNB);

    /* Initialize Rx Descriptors list: Chain Mode  */
    HAL_ETH_DMARxDescListInit(&heth, DMARxDscrTab, &Rx_Buff[0][0], ETH_
↪RXBUFNB);
    /* Enable MAC and DMA transmission and reception */
    return ret;
}

void ETH_IRQHandler(void)
{
    HAL_ETH_IRQHandler(&heth);

}

/**
* @brief  Ethernet Rx Transfer completed callback
* @param  heth: ETH handle
* @retval None
*/

void HAL_ETH_RxCpltCallback(ETH_HandleTypeDef *heth)
{

}

void HAL_ETH_TxCpltCallback(ETH_HandleTypeDef *heth)
{
    ;
}
```

(下页继续)

(续上页)

```
void HAL_ETH_ErrorCallback(ETH_HandleTypeDef *heth)
{
    PRINT_ERR("eth err\n");
}
```

STM32 的 HAL 库使用一个数据结构对以太网进行描述，我们可以认为那是一个以太网的句柄，记录着以太网的注册基地址、连接状态、发送描述、接收描述等等，该数据结构是 ETH_HandleTypeDef，具体见*代码清单 3_2*。在 bsp_eth.c 我们需要定义一个用于描述以太网的数据结构 heth，这样子我们就能通过 heth 对以太网进行初始化、收发数据等操作。

代码清单 3-2 ETH_HandleTypeDef 结构

```
typedef struct
{
    ETH_TypeDef           *Instance;  /*!< Register base address */

    ETH_InitTypeDef       Init;       /*!< Ethernet Init Configuration */

    uint32_t              LinkStatus; /*!< Ethernet link status */

    ETH_DMADescTypeDef    *RxDesc;    /*!< Rx descriptor to Get */

    ETH_DMADescTypeDef    *TxDesc;    /*!< Tx descriptor to Set */

    ETH_DMARxFrameInfos   RxFrameInfos; /*!< last Rx frame infos */

    __IO HAL_ETH_StateTypeDef  State; /*!< ETH communication state */

    HAL_LockTypeDef       Lock;    /*!< ETH Lock */

} ETH_HandleTypeDef;
```

我们先看一下我们的 Bsp_Eth_Init() 函数，调用 HAL 库的 HAL_ETH_DeInit(&heth) 进行复位 ETH 配置，该复位函数内部会调用我们 bsp_eth.c 文件中的 HAL_ETH_MspDeInit() 函数，然后我们再对 heth 的参数进行初始化，如开启网络自适应功能，速度和工作模式无需配置、设置 PHY 的地址、设置 MAC 地址、设置接收网络数据的方式为中断方式、设置检查校验为硬件校验、设置以太网速度为 100M 等等，然后调用 HAL 库的 HAL_ETH_Init() 函数将以太网进行初始化，在初始化的时候，会调用 HAL_ETH_MspInit() 对以太网的接口进行初始化，所以，我们的 bsp_eth.c 文件需要对 HAL_ETH_MspInit() 进行封装，根据我们的硬件接口（IO 接口）进行初始化操作，再对以太网的收发数据描述列表，无需我们理会，这些 HAL 库已经帮我们处理好了，这样子，一个以太网接口基本就初始化完成，但是，收发数据的操作还需要我们自己写驱动，所以我们暂时还不可以使用它进行网络数据的收发操作，因为数据的收发需要配合 LwIP，会在后面的章节中进行介绍。

3.6.3 修改 stm32f4xx_hal_conf.h 文件

有人可能会问了，PHY 的初始化在哪呢？其实，调用 HAL_ETH_Init() 函数的时候，HAL 库就已经对我们的 PHY 进行初始化了，当然，每个不一样的 PHY 肯定是不一样的配置，所以，这就需要我们自己对 PHY 参数进行配置，我们开发板使用的是 LAN8720A 芯片，LAN8720A 复位时需要一段延时时间，这里需要定义延时时间长度，大约 5~50ms 即可，驱动代码中需要获取 PHY 的速度和工作模式，LAN8720A 的 R31 是特殊控制/状态寄存器，包括指示以太网速度和工作模式的状态位，所以，我们需要在 stm32f4xx_hal_conf.h 中添加我们自己的 PHY 配置，具体见代码清单 3_3。

代码清单 3-3PHY 配置

```
/* ############ Ethernet peripheral configuration ################ */

/* Section 1 : Ethernet peripheral configuration */

/* MAC ADDRESS: MAC_ADDR0:MAC_ADDR1:MAC_ADDR2:MAC_ADDR3:MAC_ADDR4:MAC_
↪ADDR5 */
#define MAC_ADDR0   2U
#define MAC_ADDR1   0U
```

(下页继续)

(续上页)

```
#define MAC_ADDR2   0U
#define MAC_ADDR3   0U
#define MAC_ADDR4   0U
#define MAC_ADDR5   0U

/* Definition of the Ethernet driver buffers size and count */
#define ETH_RX_BUF_SIZE                ETH_MAX_PACKET_SIZE
/* buffer size for receive */
#define ETH_TX_BUF_SIZE                ETH_MAX_PACKET_SIZE
/* buffer size for transmit */
#define ETH_RXBUFNB                    ((uint32_t)8U)
/* 4 Rx buffers of size ETH_RX_BUF_SIZE  */
#define ETH_TXBUFNB                    ((uint32_t)8U)
/* 4 Tx buffers of size ETH_TX_BUF_SIZE  */

/* Section 2: PHY configuration section */

/* LAN8720_PHY_ADDRESS Address*/
#define LAN8720_PHY_ADDRESS           0U
/* PHY Reset delay these values are based on a 1 ms Systick interrupt*/
#define PHY_RESET_DELAY                ((uint32_t)0x00000005U)
/* PHY Configuration delay */
#define PHY_CONFIG_DELAY               ((uint32_t)0x00000005U)

#define PHY_READ_TO                    ((uint32_t)0x0000FFFFU)
#define PHY_WRITE_TO                   ((uint32_t)0x0000FFFFU)

/* Section 3: Common PHY Registers */

#define PHY_BCR                        ((uint16_t)0x00U)
/*!< Transceiver Basic Control Register   */
#define PHY_BSR                        ((uint16_t)0x01U)
```

(下页继续)

(续上页)

```
/*!< Transceiver Basic Status Register    */

#define PHY_RESET               ((uint16_t)0x8000U)  /*!< PHY Reset */
#define PHY_LOOPBACK                    ((uint16_t)0x4000U)
/*!< Select loop-back mode */
#define PHY_FULLDUPLEX_100M             ((uint16_t)0x2100U)
/*!< Set the full-duplex mode at 100 Mb/s */
#define PHY_HALFDUPLEX_100M             ((uint16_t)0x2000U)
/*!< Set the half-duplex mode at 100 Mb/s */
#define PHY_FULLDUPLEX_10M              ((uint16_t)0x0100U)
/*!< Set the full-duplex mode at 10 Mb/s  */
#define PHY_HALFDUPLEX_10M              ((uint16_t)0x0000U)
/*!< Set the half-duplex mode at 10 Mb/s  */
#define PHY_AUTONEGOTIATION             ((uint16_t)0x1000U)
/*!< Enable auto-negotiation function     */
#define PHY_RESTART_AUTONEGOTIATION     ((uint16_t)0x0200U)
/*!< Restart auto-negotiation function    */
#define PHY_POWERDOWN                   ((uint16_t)0x0800U)
/*!< Select the power down mode           */
#define PHY_ISOLATE                     ((uint16_t)0x0400U)
/*!< Isolate PHY from MII */

#define PHY_AUTONEGO_COMPLETE           ((uint16_t)0x0020U)
/*!< Auto-Negotiation process completed   */
#define PHY_LINKED_STATUS               ((uint16_t)0x0004U)
/*!< Valid link established */
#define PHY_JABBER_DETECTION            ((uint16_t)0x0002U)
/*!< Jabber condition detected*/

/* Section 4: Extended PHY Registers */
#define PHY_SR                          ((uint16_t)0x1FU)
/*!< PHY status register Offset*/
```

(下页继续)

(续上页)

```
#define PHY_SPEED_STATUS                ((uint16_t)0x0004U)
/*!< PHY Speed mask */
#define PHY_DUPLEX_STATUS               ((uint16_t)0x0010U)
/*!< PHY Duplex mask*/
```

第 4 章　LwIP 的网络接口管理

本章主要是讲解 netif 结构体，了解底层硬件网络接口如何与 LwIP 对接。

4.1 netif 结构体

从前面的章节我们知道，网络接口（如以太网接口）是硬件接口，（提示：网络接口又可以称之为网卡，为了统一，下文均采用网卡表示网络接口），LwIP 是软件，那么怎么让硬件与软件无缝连接起来呢？而且，网卡又有多种多样，怎么能让 LwIP 使用同样的软件能兼容不同的硬件呢？原来 LwIP 使用一个数据结构——netif 来描述一个网卡，但是由于网卡是直接与硬件打交道的，硬件不同则处理基本是不同的，所以必须由用户提供最底层接口函数，LwIP 提供统一的接口，但是底层的实现需要用户自己去完成，比如网卡的初始化，网卡的收发数据，当 LwIP 底层得到了网络的数据之后，才会传入内核中去处理；同理，LwIP 内核需要发送一个数据包的时候，也需要调用网卡的发送函数，这样子才能把数据从硬件接口到软件内核无缝连接起来。LwIP 中的 ethernetif.c 文件即为底层接口的驱动的模版，用户为自己的网络设备实现驱动时应参照此模块做修改。ethernetif.c 文件中的函数通常为与硬件打交道的底层函数，当有数据需要通过网卡接收或者发送数据的时候就会被调用，经过 LwIP 协议栈内部进行处理后，从应用层就能得到数据或者可以发送数据。

简单来说，netif 是 LwIP 抽象出来的网卡，LwIP 协议栈可以使用多个不同的接口，而 ethernetif.c 文件则提供了 netif 访问各种不同的网卡，每个网卡有不同的实现方式，用户只需要修改 ethernetif.c 文件即可。

在单网卡中，这个 netif 结构体只有一个，可能还有人会问，那么一个设备中有多个网卡怎么办，很简单，LwIP 会将每个用 netif 描述的网卡连接成一个链表（单向链表），该链表就记录每个网卡的 netif。屏蔽硬件接口的差异，完成了对不同网卡的抽象，因此了解 netif 结构体是移植 LwIP 的关键。

我们可以理解将整个网络的数据传输理解为物流，那么网卡就是不同的运输工具，我们可以选择汽车、飞机、轮船等运输工具，不同的运输工具速度是不一样的，但是对于一个物流公司而言，

可能同时存在很多种运输的工具，这就需要物流公司去记录这些运输工具，当有一个包裹需要通过飞机运输出去，那么物流公司就会将这个包裹通过飞机发送出去，这就好比我们的网卡，需要哪个网卡发送或者接收网络数据的时候，就会让对应的网卡去工作。

下面一起来看看 netif 数据结构是怎么样的，具体见代码清单 4_1。

代码清单 4-1 netif 数据结构（已删减 IPv6 相关字段）

```
struct netif
{
#if !LWIP_SINGLE_NETIF
    /* 指向 netif 链表中的下一个 */
    struct netif *next;                                    (1)
#endif

#if LWIP_IPV4
    /* 网络字节中的 IP 地址、子网掩码、默认网关配置 */
    ip_addr_t ip_addr;
    ip_addr_t netmask;
    ip_addr_t gw;                                          (2)
#endif /* LWIP_IPV4 */

    /* 此函数由网络设备驱动程序调用，将数据包传递到 TCP/IP 协议栈。
    * 对于以太网物理层，这通常是 ethernet_input()*/
    netif_input_fn input;                                  (3)

#if LWIP_IPV4

    /* 此函数由 IP 层调用，在接口上发送数据包。通常这个功能，
        * 首先解析硬件地址，然后发送数据包。
        * 对于以太网物理层，这通常是 etharp_output() */
    netif_output_fn output;                                (4)

#endif /* LWIP_IPV4 */
```

(下页继续)

(续上页)

```
    /* 此函数由 ethernet_output() 调用，当需要在网卡上发送一个数据包时。
        * 底层硬件输出数据函数，一般是调用自定义函数 low_level_output*/
    netif_linkoutput_fn linkoutput;                    (5)

#if LWIP_NETIF_STATUS_CALLBACK
    /* 当 netif 状态设置为 up 或 down 时调用此函数 */
    netif_status_callback_fn status_callback;                (6)
#endif /* LWIP_NETIF_STATUS_CALLBACK */

#if LWIP_NETIF_LINK_CALLBACK
    /* 当 netif 链接设置为 up 或 down 时，将调用此函数 */
    netif_status_callback_fn link_callback;          (7)
#endif /* LWIP_NETIF_LINK_CALLBACK */

#if LWIP_NETIF_REMOVE_CALLBACK
    /* 当 netif 被删除时调用此函数 */
    netif_status_callback_fn remove_callback;                (8)
#endif /* LWIP_NETIF_REMOVE_CALLBACK */

    /* 此字段可由设备驱动程序设置并指向设备的状态信息。
    * 主要是将网卡的某些私有数据传递给上层，用户可以自由发挥，也可以不用。*/
    void *state;                                       (9)

#ifdef netif_get_client_data
    void* client_data[LWIP_NETIF_CLIENT_DATA_INDEX_MAX + LWIP_NUM_NETIF_
↪CLIENT_DATA];
#endif
#if LWIP_NETIF_HOSTNAME
    /* 这个 netif 的主机名，NULL 也是一个有效值 */
    const char*  hostname;
#endif /* LWIP_NETIF_HOSTNAME */
```

(下页继续)

(续上页)

```
#if LWIP_CHECKSUM_CTRL_PER_NETIF
    u16_t chksum_flags;
#endif /* LWIP_CHECKSUM_CTRL_PER_NETIF*/

    /** 最大传输单位 (以字节为单位), 对于以太网一般设为 1500 */
    u16_t mtu;                                              (10)

    /** 此网卡的链路层硬件地址 */
    u8_t hwaddr[NETIF_MAX_HWADDR_LEN];                      (11)

    /** 硬件地址长度, 对于以太网就是 MAC 地址长度, 为 6 字节 */
    u8_t hwaddr_len;                                        (12)

    /* 网卡状态信息标志位, 是很重要的控制字段,
    * 它包括网卡功能使能、广播使能、 ARP 使能等等重要控制位。 */
    u8_t flags;                                             (13)

    /* 字段用于保存每一个网卡的名字。用两个字符的名字来标识网络接
    * 口使用的设备驱动的种类, 名字由设备驱动来设置并且应该反映通过网卡
    * 表示的硬件的种类。比如蓝牙设备 ( bluetooth) 的网卡名字可以是 bt,
    * 而 IEEE 802.11b WLAN 设备的名字就可以是 wl, 当然设置什么名字用户是可
    * 以自由发挥的, 这并不影响用户对网卡的使用。当然, 如果两个网卡
    * 具有相同的网络名字, 我们就用 num 字段来区分相同类别的不同网卡 */
    char name[2];                                           (14)

    /* 用来标示使用同种驱动类型的不同网卡 */
    u8_t num;                                               (15)

#if MIB2_STATS
    /* 连接类型 */
    u8_t link_type;
    /* 连接速度 */
```

(下页继续)

(续上页)

```
    u32_t link_speed;
    /* 最后一次更改的时间戳 */
    u32_t ts;
    /** counters */
    struct stats_mib2_netif_ctrs mib2_counters;
#endif /* MIB2_STATS */

#if LWIP_IPV4 && LWIP_IGMP
    /** 可以调用此函数来添加或删除多播中的条目
        以太网 MAC 的过滤表。*/
    netif_igmp_mac_filter_fn igmp_mac_filter;
#endif /* LWIP_IPV4 && LWIP_IGMP */

#if LWIP_NETIF_USE_HINTS
    struct netif_hint *hints;
#endif /* LWIP_NETIF_USE_HINTS */

#if ENABLE_LOOPBACK
    /* List of packets to be queued for ourselves. */
    struct pbuf *loop_first;
    struct pbuf *loop_last;

#if LWIP_LOOPBACK_MAX_PBUFS
    u16_t loop_cnt_current;
#endif /* LWIP_LOOPBACK_MAX_PBUFS */

#endif /* ENABLE_LOOPBACK */
};
```

我们挑一些比较重要的 netif 字段进行讲解：

代码清单 4-1(**1**)：LwIP 使用链表来管理同一设备的多个网卡。在 netif.c 文件中定义两个全局指针：struct netif *netif_list 和 struct netif *netif_default，其中 netif_list 就是网卡链表指针，指向网卡

链表的首节点（第一个网卡），后者表示默认情况下（有多网口时）使用哪个网卡。next 字段指向下一个 netif 结构体指针，在一个设备中有多个网卡时，才使用该字段。

代码清单 4-1**(2)**：ip_addr 字段记录的是网络中的 IP 地址，netmask 字段记录的是子网掩码，gw 记录的是网关地址，这些字段是用于描述网卡的网络地址属性。

IP 地址必须与网卡对应，即设备拥有多少个网卡那就必须有多少个 IP 地址；子网掩码可以用来判断某个 IP 地址与当前网卡是否处于同一个子网中，IP 在发送数据包的时候会选择与目标 IP 地址处于同一子网的网卡来发送；网关地址在数据包的发送、转发过程非常重要，如果要向不属于同一子网的主机（主机目标 IP 地址与网卡不属于同一子网）发送一个数据包，那么 LwIP 就会将数据包发送到网关中，网关设备会对该数据包进行正确的转发，除此之外，网关还提供很多高级功能，如 DNS，DHCP 等。

代码清单 4-1**(3)**：input 是一个函数指针，指向一个函数，该函数由网络设备驱动程序调用，将数据包传递到 TCP/IP 协议栈（IP 层）。对于以太网物理层，这通常是 ethernet_input()，参数为 pbuf 和 netif 类型，其中 pbuf 为接收到的数据包。

代码清单 4-1**(4)**：output 也是一个函数指针，指向一个函数，此函数由 IP 层调用，在接口上发送数据包。用户需要编写该函数并使 output 指向它，通这个函数的处理步骤是首先解析硬件地址，然后发送数据包。对于以太网物理层，该函数通常是 etharp_output()，参数为 pbuf、netif 和 ip_addr 类型，其中，ipaddr 代表要将该数据包发送到的地址，但不一定是数据包最终到到达的 IP 地址，比如，要发送 IP 数据报到一个并不在本网络的主机上，该数据包要被发送到一个路由器上，这里的 ipaddr 就是路由器 IP 地址。

代码清单 4-1**(5)**：linkoutput 字段和 output 类似，也需要用户自己实现一个函数，但只有两个参数，它是由 ARP 模块调用的，一般是自定义函数 low_level_output()。当需要在网卡上发送一个数据包时，该函数会被 ethernet_output() 函数调用。

代码清单 4-1**(6)**：当 netif 状态设置为 up 或 down 时，将调用此函数。

代码清单 4-1**(7)**：当 netif 连接设置为 up 或 down 时，将调用此函数。

代码清单 4-1**(8)**：当 netif 被删除时调用此函数。

代码清单 4-1**(9)**：此字段可由设备驱动程序设置并指向设备的状态信息。主要是将网卡的某些私有数据传递给上层，用户可以自由发挥，也可以不用。

代码清单 4-1**(10)**：最大传输单位（以字节为单位），对于以太网一般为 1500，在 IP 层发送数据

的时候，LwIP 会使用该字段决定是否需要对数据包进行分片处理，为什么是在 IP 层进行分片处理？因为链路层不提供任何的差错处理机制，如果在网卡中接收的数据包不满足网卡自身的属性，那么网卡可能就会直接丢弃该数据包，也可能在底层进行分包发送，但是这种分包在 IP 层看来是不可接受的，因为它打乱了数据的结构，所以只能由 IP 层进行分片处理。

代码清单 4-1**(11)**：此网卡的链路层硬件地址。

代码清单 4-1**(12)**：硬件地址长度，对于以太网就是 MAC 地址长度，为 6 字节

代码清单 4-1**(13)**：网卡状态信息标志位，是很重要的控制字段，它包括网卡功能使能、广播使能、ARP 使能等等重要控制位。

代码清单 4-1**(14)**：name 字段用于保存每一个网卡的名字。用两个字符的名字来标识网卡使用的设备驱动的种类，名字由设备驱动来设置并且应该反映通过网卡表示的硬件的种类。比如蓝牙设备（bluetooth）的网卡名字可以是 bt，而 IEEE 802.11b WLAN 设备的名字就可以是 wl，当然设置什么名字用户是可以自由发挥的，这并不影响用户对网卡的使用。当然，如果两个网卡具有相同的网络名字，我们就用 num 字段来区分相同类别的不同网卡。

代码清单 4-1**(15)**：用来标识使用同种驱动类型的不同网卡。

4.2 netif 使用

那么 netif 具体该如何使用呢？其实使用还是非常简单的。首先我们需要根据我们的网卡定义一个 netif 结构体变量 struct netif gnetif，我们首先要把网卡挂载到 netif_list 链表上才能使用，因为 LwIP 是通过链表来管理所有的网卡，所有第一步是通过 netif_add() 函数将我们的网卡挂载到 netif_list 链表上，netif_add() 函数具体见*代码清单 4_2*。

代码清单 4-2netif_add() 函数（已删减）

```
struct netif *
netif_add(struct netif *netif,const ip4_addr_t *ipaddr,
        const ip4_addr_t *netmask, const ip4_addr_t *gw,
        void *state, netif_init_fn init, netif_input_fn input)
{
```

(下页继续)

(续上页)

```
    LWIP_ASSERT_CORE_LOCKED();

    if (ipaddr == NULL)
    {
        ipaddr = ip_2_ip4(IP4_ADDR_ANY);
    }
    if (netmask == NULL)
    {
        netmask = ip_2_ip4(IP4_ADDR_ANY);
    }
    if (gw == NULL)
    {
        gw = ip_2_ip4(IP4_ADDR_ANY);
    }

    /* reset new interface configuration state */
    ip_addr_set_zero_ip4(&netif->ip_addr);                    (1)
    ip_addr_set_zero_ip4(&netif->netmask);
    ip_addr_set_zero_ip4(&netif->gw);
    netif->output = netif_null_output_ip4;

    NETIF_SET_CHECKSUM_CTRL(netif, NETIF_CHECKSUM_ENABLE_ALL);
    netif->mtu = 0;
    netif->flags = 0;

    memset(netif->client_data, 0, sizeof(netif->client_data));

    /* remember netif specific state information data */
    netif->state = state;
    netif->num = netif_num;
    netif->input = input;                                     (2)
```

(下页继续)

(续上页)

```
    NETIF_RESET_HINTS(netif);


    netif_set_addr(netif, ipaddr, netmask, gw);                          (3)


    /* call user specified initialization function for netif */
    if (init(netif) != ERR_OK)                                           (4)
    {
        return NULL;
    }

    {
        struct netif *netif2;
        int num_netifs;
        do
        {
            if (netif->num == 255)
            {
                netif->num = 0;
            }
            num_netifs = 0;
        for(netif2 = netif_list; netif2 != NULL; netif2 = netif2->next)
            {
                num_netifs++;
                if (netif2->num == netif->num)
                {
                    netif->num++;
                    break;
                }
            }
        }
```

(下页继续)

(续上页)

```
        while (netif2 != NULL);
    }
    if (netif->num == 254)
    {
        netif_num = 0;
    }
    else
    {
        netif_num = (u8_t)(netif->num + 1);                    (5)
    }

    /* add this netif to the list */
    netif->next = netif_list;
    netif_list = netif;                                        (6)

    mib2_netif_added(netif);


    ip4_addr_debug_print(NETIF_DEBUG, ipaddr);

    ip4_addr_debug_print(NETIF_DEBUG, netmask);

    ip4_addr_debug_print(NETIF_DEBUG, gw);


    netif_invoke_ext_callback(netif, LWIP_NSC_NETIF_ADDED, NULL);

    return netif;
}
```

代码清单 4-2**(1)**：清空主机 IP 地址、子网掩码、网关等字段信息。

代码清单 4-2**(2)**：根据传递进来的参数填写网卡 state、input 等字段的相关信息。

代码清单 4-2**(3)**：调用网卡设置函数 netif_set_addr() 设置网卡 IP 地址、子网掩码、网关等信息。

代码清单 4-2**(4)**：通过传递进来的回调函数 init() 进行网卡真正的初始化操作，所以该函数是由用户实现的，对于不同网卡就使用不一样的初始化，而此处是以太网，则该回调函数一般为 ethernetif_init()，该函数在后文详细讲解。

代码清单 4-2**(5)**：初始化网卡成功，则遍历当前设备拥有多少个网卡，并为当前网卡分配唯一标识 num。

代码清单 4-2**(6)**：将当前网卡插入 netif_list 链表中。

在使用之前需要进行初始化主机 IP 地址、子网掩码、网关等，并且在调用 netif_add() 函数之后会触发 netif_init_fn 的回调函数。

总之一句话，在开始使用 LwIP 协议栈的时候，我们就需要将网卡底层移植完成，才能开始使用，而移植的第一步，就是将网络进行初始化，并且设置该网卡为默认网卡，让 LwIP 能通过网卡进行收发数据，具体见*代码清单 4_3* 加粗部分。

代码清单 4-3 挂载网卡

```
/*Static IP ADDRESS: IP_ADDR0.IP_ADDR1.IP_ADDR2.IP_ADDR3 */
#define IP_ADDR0                    192
#define IP_ADDR1                    168
#define IP_ADDR2                      1
#define IP_ADDR3                    122

/*NETMASK*/
#define NETMASK_ADDR0               255
#define NETMASK_ADDR1               255
#define NETMASK_ADDR2               255
#define NETMASK_ADDR3                 0

/*Gateway Address*/
#define GW_ADDR0                    192
#define GW_ADDR1                    168
#define GW_ADDR2                      1
```

(下页继续)

(续上页)

```
#define GW_ADDR3                        1

struct netif gnetif;
ip4_addr_t ipaddr;
ip4_addr_t netmask;
ip4_addr_t gw;
uint8_t IP_ADDRESS[4];
uint8_t NETMASK_ADDRESS[4];
uint8_t GATEWAY_ADDRESS[4];

void TCPIP_Init(void)
{

    tcpip_init(NULL, NULL);

    /* IP addresses initialization */
    /* USER CODE BEGIN 0 */
#ifdef USE_DHCP
    ip_addr_set_zero_ip4(&ipaddr);
    ip_addr_set_zero_ip4(&netmask);
    ip_addr_set_zero_ip4(&gw);
#else
    IP4_ADDR(&ipaddr,IP_ADDR0,IP_ADDR1,IP_ADDR2,IP_ADDR3);
    IP4_ADDR(&netmask,NETMASK_ADDR0,NETMASK_ADDR1,NETMASK_ADDR2,NETMASK_
↪ADDR3);
    IP4_ADDR(&gw,GW_ADDR0,GW_ADDR1,GW_ADDR2,GW_ADDR3);
#endif /* USE_DHCP */
    /* USER CODE END 0 */
    /* Initilialize the LwIP stack without RTOS */
    /* add the network interface (IPv4/IPv6) without RTOS */
    netif_add(&gnetif, &ipaddr, &netmask, &gw, NULL, &ethernetif_init,
                            &tcpip_input);
```

(下页继续)

(续上页)

```
    /* Registers the default network interface */
    netif_set_default(&gnetif);

    if (netif_is_link_up(&gnetif))
    {
/* When the netif is fully configured this function must be called */
        netif_set_up(&gnetif);
    }
    else
    {
        /* When the netif link is down this function must be called */
        netif_set_down(&gnetif);
    }
}
```

挂载网卡的过程是非常简单的，如果一个设备当前是还没有网卡的，当调用 netif_add() 函数挂载网卡后，其过程如图 *4_1* 所示，当设备需要挂载多个网卡的时候，就多次调用 netif_add() 函数即可，新挂载的网卡会在链表的最前面，具体见图 *4_2*。

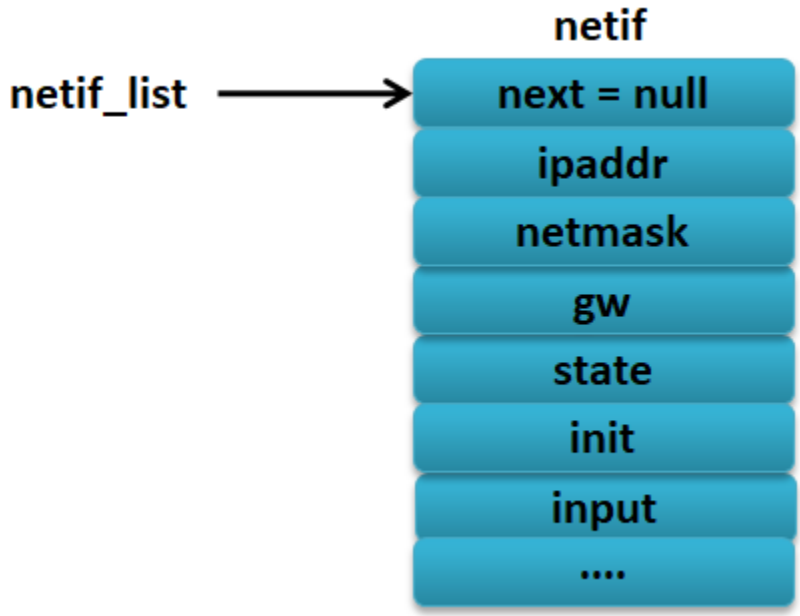


图 4-1 挂载第一个网卡

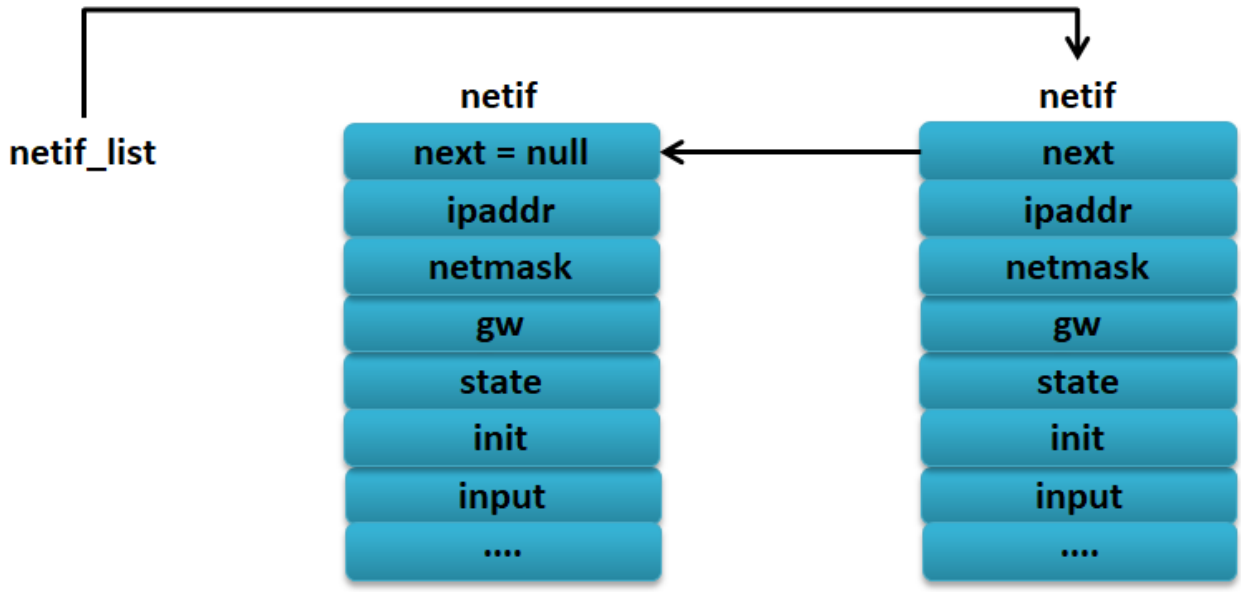


图 4-2 挂载第二个网卡（右边是新挂载的网卡）

4.3 与 netif 相关的底层函数

通过前面的学习，我们知道每个 netif 接口都需要一个底层接口文件提供访问硬件的支持，而 LwIP 作者将这种支持做成一个框架供我们参考，如 ethernetif.c 文件就是实现为一个框架的形式，我们在移植的时候只需要根据实际的网卡特性完善这里面的函数即可，该文件在后文会讲解。框架中的函数名、参数等都已经实现，我们只需往里面填充完善即可，当然，网卡的驱动与这些函数名字我们也可以进行修改，只要 LwIP 内核能正确识别网卡中的功能即可，为了方便，我们还是使用 LwIP 作者提供的框架进行移植操作，当一个设备使用了多个网卡的时候，那就需要编写多个不同的网卡驱动。与网卡驱动密切相关的函数有三个，分别是：

代码清单 4-4

```
static void low_level_init(struct netif *netif);
static err_t low_level_output(struct netif *netif, struct pbuf *p);
```

(下页继续)

(续上页)

```
static struct pbuf * low_level_input(struct netif *netif);
```

low_level_init() 为网卡初始化函数，它主要完成网卡的复位及参数初始化，根据实际的网卡属性进行配置 netif 中与网卡相关的字段，例如网卡的 MAC 地址、长度，最大发送单元等。

low_level_output() 函数为网卡的发送函数，它主要将内核的数据包发送出去，数据包采用 pbuf 数据结构进行描述，该数据结构是一个比较复杂的数据结构，后续我们会详细讲解。

low_level_input() 函数为网卡的数据接收函数，该函数会接收一个数据包，为了内核易于对数据包的管理，该函数必须将接收的数据封装成 pbuf 的形式。

在这一章节中我们仅讲解 low_level_init() 函数，后面的两个函数涉及到 pbuf 数据结构，在后面会详细讲解。

除此之外，还有两个函数也与网卡与关系，分别是：

```
err_t ethernetif_init(struct netif *netif);
void ethernetif_input(void *pParams);
```

ethernetif_init() 函数是在上层管理网卡 netif 的到时候会被调用的函数，如使用 netif_add() 添加网卡的时候，就会调用 ethernetif_init() 函数对网卡进行初始化，其实该函数的最终调用的初始化函数就是 low_level_init() 函数，我们目前只有一个网卡，就暂时不用对该函数进行改写，直接使用即可，它内部会将网卡的 name、output、linkoutput 等字段进行初始化，这样子就能将内核与网卡无缝连接起来。

ethernetif_input() 函数的主要作用就是调用 low_level_input() 函数从网卡中读取一个数据包，然后解析该数据包的类型是属于 ARP 数据包还是 IP 数据包，再将包递交给上层，在无操作系统的时候 ethernetif_input() 就是一个可以直接使用的函数，已经无需我们自己去修改，内核会周期性处理该接收函数。而在多线程操作系统的时候，我们一般会将其改写成一个线程的形式，可以周期性去调用 low_level_input() 网卡接收函数；也可以使用中断的形式去处理，当这个线程将在尚未接收到数据包的时候，处于阻塞状态，当收到数据包的时候，中断利用操作系统的 IPC 通信机制来唤醒线程去处理接收到的数据包，并将数据包递交上层，这样子的效率会更加高效，事实上我们也是这样子处理的。

4.4 ethernetif.c 文件内容

4.4.1 ethernetif 数据结构

既然已经了解了与网卡有关的底层驱动函数，那么我们先看看 ethernetif.c 文件到底是怎么样子的，又是怎么将网卡与 LwIP 内核连接在一起。

首先在 ethernetif.c 文件的开始时，就定义了一个 ethernetif 数据结构，具体见*代码清单 4_6*。

代码清单 4-6 ethernetif 数据结构

```
struct ethernetif
{
    struct eth_addr *ethaddr;
    /* Add whatever per-interface state that is needed here. */
};
```

ethernetif 数据结构用来描述底层硬件设备的一些私有信息，如 MAC 地址等，该结构体唯一不可或缺的是 MAC 地址，它是 LwIP 用于相应 ARP 查询的核心数据。用户可以对该结构进行添加其他的网卡描述信息，如果没有特殊需要，就不用添加其他成员数据，该数据结构在初始化的时候，会通过 netif 的 state 成员变量将这些硬件的私有信息传递给上层。

4.4.2 ethernetif_init()

该函数是直接拿来用即可，如果没有特别的需求，基本不需要怎么修改它，它是 LwIP 中默认的网卡初始化函数，内部封装了 low_level_init() 函数，具体见*代码清单 4_7*。

代码清单 4-7 ethernetif_init() 源码

```
err_t ethernetif_init(struct netif *netif)
{
    struct ethernetif *ethernetif;
```

(下页继续)

(续上页)

```
//  LWIP_ASSERT("netif != NULL", (netif != NULL));

    ethernetif = mem_malloc(sizeof(struct ethernetif));

    if (ethernetif == NULL)
    {
        PRINT_ERR("ethernetif_init: out of memory\n");
        return ERR_MEM;
    }

    LWIP_ASSERT("netif != NULL", (netif != NULL));
//
#if LWIP_NETIF_HOSTNAME
    /* Initialize interface hostname */
    netif->hostname = "lwip";
#endif /* LWIP_NETIF_HOSTNAME */
    netif->state = ethernetif;                                    (1)
    netif->name[0] = IFNAME0;
    netif->name[1] = IFNAME1;

#if LWIP_IPV4
#if LWIP_ARP || LWIP_ETHERNET
#if LWIP_ARP
    netif->output = etharp_output;
#else

    netif->output = low_level_output_arp_off;
#endif /* LWIP_ARP */
#endif /* LWIP_ARP || LWIP_ETHERNET */
#endif /* LWIP_IPV4 */

#if LWIP_IPV6
```

(下页继续)

(续上页)

```
    netif->output_ip6 = ethip6_output;
#endif /* LWIP_IPV6 */

    netif->linkoutput = low_level_output;

    /* initialize the hardware */
    low_level_init(netif);                                  (2)
    ethernetif->ethaddr = (struct eth_addr *) &(netif->hwaddr[0]);

    return ERR_OK;
}
```

代码清单 4-7**(1)**：通过 netif 的 state 成员变量将 ethernetif 结构传递给上层。

代码清单 4-7**(2)**：调用 low_level_init() 函数对网卡进行初始化，而该函数需要我们根据网卡的实际情况进行编写。

4.4.3 low_level_init()

该函数主要是根据实际情况对网卡进行一系列的初始化工作，例如：初始化 MAC 地址、长度，设置最大传输包的大小，设置网卡的属性字段，支持广播、多播、ARP 等功能，如果是使用操作系统的话，还需要建立接收数据、发送数据的任务以及一些需要的消息队列、信号量等，此处讲解的是裸机底层驱动的编写，low_level_init() 源码具体见*代码清单 4_8*。

代码清单 4-8low_level_init() 源码

```
static void low_level_init(struct netif *netif)
{
    HAL_StatusTypeDef hal_eth_init_status;

    //初始化 bsp—eth
    hal_eth_init_status = Bsp_Eth_Init();                   (1)
```

(下页继续)

(续上页)

```
    if (hal_eth_init_status == HAL_OK)
    {
        /* Set netif link flag */
        netif->flags |= NETIF_FLAG_LINK_UP;
    }
#if LWIP_ARP || LWIP_ETHERNET

    /* set MAC hardware address length */
    netif->hwaddr_len = ETH_HWADDR_LEN;

    /* set MAC hardware address */
    netif->hwaddr[0] =  heth.Init.MACAddr[0];
    netif->hwaddr[1] =  heth.Init.MACAddr[1];
    netif->hwaddr[2] =  heth.Init.MACAddr[2];
    netif->hwaddr[3] =  heth.Init.MACAddr[3];
    netif->hwaddr[4] =  heth.Init.MACAddr[4];
    netif->hwaddr[5] =  heth.Init.MACAddr[5];

    /* maximum transfer unit */
    netif->mtu = NETIF_MTU;

#if LWIP_ARP
    netif->flags |= NETIF_FLAG_BROADCAST | NETIF_FLAG_ETHARP;
#else
    netif->flags |= NETIF_FLAG_BROADCAST;
#endif /* LWIP_ARP */

#endif /* LWIP_ARP || LWIP_ETHERNET */

    HAL_ETH_Start(&heth);                                   (2)
}
```

代码清单 4-8**(1)**：调用我们自己实现的以太网驱动初始化函数 Bsp_Eth_Init()，这是根据网卡的驱动所编写的函数，不同的网卡是不一样的，由用户实现，该函数在代码清单 3-1 中实现，此处就不再赘述。

代码清单 4-8**(1)**：初始化完成就需要启动网卡，才能进行数据的收发操作。

本章就暂时讲解网卡初始相关的函数，而对于网卡的接收数据和发送数据都涉及到 pbuf，我们会在后续进行详细讲解。

第 5 章　LwIP 的内存管理

从本章开始，我们就开始正式进入 LwIP 内核的学习，LwIP 本质就是对数据的处理，网络中的数据是非常多的，那么 LwIP 对这些数据的处理必然是需要消耗系统资源的，而有好的内存管理策略就显得非常必要了，内存分配策略、内存分配效率等都是衡量系统性能的重要因素。

5.1 几种内存分配策略

常见的内存分配策略有两种，一种是分配固定大小的内存块；另一种是利用内存堆进行动态分配，属于可变长度的内存块。这两种内存分配策略都会在 LwIP 中被使用到，他们各有所长，LwIP 的作者根据不同的应用场景选择不同的内存分配策略，这样子使得系统的内存开销、分配效率等都得到很大的提高。此外 LwIP 还支持使用 C 标准库中的 malloc 和 free 进行内存分配，但是这种内存分配我们不建议使用，因为 C 标准库在嵌入式设备中使用会有很多问题，系统每次调用这些函数执行的时间可能都不一样，这是致命的，因为内存分配中最重要的就是分配时间效率的问题。内存分配的本质就是事先准备一大块内存堆（可以理解为一个巨大的数组），然后将该空间起始地址返回给申请者，这就需要内核必须采用自己独有的一套数据结构来描述、记录哪些内存空间已经分配，哪些内存空间是未使用的，根据使用的机制不同，延伸出多种类型的内存分配策略。

5.1.1 固定大小的内存块

使用固定大小的内存块分配策略，用户只能申请大小固定的内存块，在内存初始化的时候，系统会将所有可用的内存区域划分为 N 块固定大小的内存，然后将这些内存块通过单链表的方式连接起来，用户在申请内存块的时候就直接从链表的头部取出一个内存块进行分配，同理释放内存块的时候也是很简单，直接将内存块释放到链表的头部即可，这样子分配内存的时间就是固定的，非常高效。但是缺点也是很明显的，用户只能申请固定大小的内存块，如果内存块无法满足用户的需求，那么则无法申请成功，而如果将内存块大小变大，那么在用户需要极小的内存的时候就会造成内存的浪费，这也是不适合的。

可能会有人问了，那这种内存分配策略不好用，为什么 LwIP 作者会使用呢？其实不然，LwIP 中有很多固定的数据结构空间，如 TCP 首部、UDP 首部，IP 首部，以太网首部等都是固定的数据结构，其大小就是一个固定的值，那么我们就能采用这种方式分配这些固定大小的内存空间，这样子的效率就会大大提高，并且无论怎么申请与释放，都不会产生内存碎片，这就让系统能很稳定地运行。这种分配策略在 LwIP 中被称之为动态内存池分配策略，内存池示意图具体见图 *5_1*。

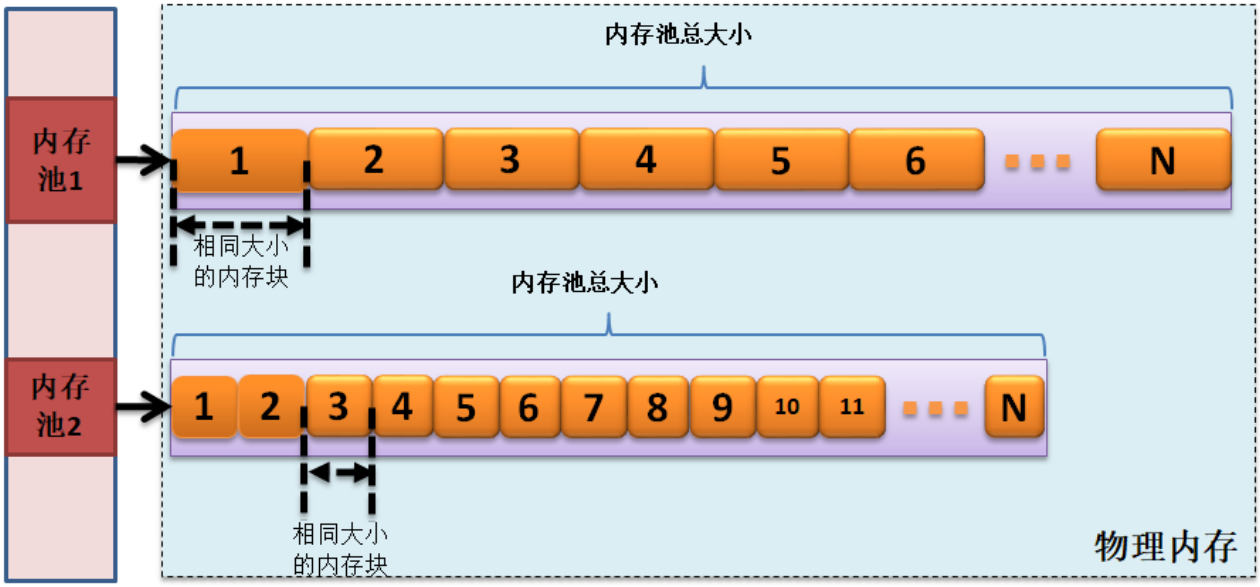


图 5-1 动态内存池

5.1.2 可变长度分配

这种内存分配策略在很多系统中都会被使用到，系统运行的时候，各个空闲内存块的大小是不固定的，它会随着用户的申请而改变，刚开始的时候，系统就是一块大的内存堆，随着系统的运行，用户会申请与释放内存块，所以系统的内存块的大小。数量都会随之改变，并且对于这种内存分配策略是有多种不同的算法的，具体的我们就不再细说，可以参考操作系统相关的书籍。

LwIP 中也会使用这种内存分配策略，它采用 First Fit（首次拟合）内存管理算法，申请内存时只要找到一个比所请求的内存大的空闲块，就从中切割出合适的块，并把剩余的部分返回到动态内存堆中，这种分配策略分配的内存块大小有限制，要求请求的分配大小不能小于 MIN_SIZE，否则请求会被分配到 MIN_SIZE 大小的内存空间，一般 MIN_SIZE 大小为 12 字节，在这 12 个字节

中前几个字节会存放内存分配器管理用的私有数据，该数据区域不能被用户程序修改，否则导致致命问题。内存释放的过程是相反的过程，但分配器会查看该节点前后相邻的内存块是否空闲，如果空闲则合并成一个大的内存空闲块。当然，采用这种内存堆的分配方式，在申请和释放的时候肯定需要消耗时间，可以类似地看做是以时间换空间的策略。采用这种分配策略，其优点就是内存浪费小，比较简单，适合用于小内存的管理，其缺点就是如果频繁的动态分配和释放，可能会造成严重的内存碎片，如果在碎片情况严重的话，可能会导致内存分配不成功从而导致系统崩溃。

补充：存碎片导致系统崩溃的原因并不是因为系统没有可用内存了，而是内存块被分割成很多不连续的小内存块，当用户需要申请一个更大的内存块的时候，系统没办法提供这样子的内存块，就会导致申请失败。

当然 LwIP 也支持 C 标准库的 malloc() 和 free()，因为不建议使用这种情况，所以此处我们就不做过多的讲解。

5.2 动态内存池（POOL）

申请大小必须是指定固定大小字节的值（如 4、8、16 等等），系统将所有可用区域以固定大小的字节单位进行划分，然后用单链表将所有空闲内存块连接起来。链表中所有节点大小相同，分配，释放都非常简单。。

LwIP 源文件中 memp.c 和 memp.h 就是动态内存池分配策略，为什么 LWIP 需要有 POOL？因为协议栈里面有大量的协议首部，这些协议首部长度都是固定不变的，所以我们可以首先分配固定内存，给这些固定长度的协议首部，以后每次需要处理协议首部的时候，都直接使用这些已经分配的内存，不需要重新分配内存区域，这样子就达到一个地方分配，多个地方使用的方便与效率。

5.2.1 内存池的预处理

在内核初始化时，会事先在内存中初始化相应的内存池，内核会将所有可用的区域根据宏定义的配置以固定的大小为单位进行划分，然后用一个简单的链表将所有空闲块连接起来，这样子就组成一个个的内存池。由于链表中所有节点的大小相同，所以分配时不需要查找，直接取出第一个节点中的空间分配给用户即可。

注意了，内核在初始化内存池的时候，是根据用户配置的宏定义进行初始化的，比如，用户定义了 LWIP_UDP 这个宏定义，在编译的时候，编译器就会将与 UDP 协议控制块相关的数据构编译编译进去，这样子就将 LWIP_MEMPOOL(UDP_PCB, MEMP_NUM_UDP_PCB, sizeof(struct udp_pcb),” UDP_PCB”) 包含进去，在初始化的时候，UDP 协议控制块需要的 POOL 资源就会被初始化，其数量由 MEMP_NUM_UDP_PCB 宏定义决定，注意了，不同协议的 POOL 内存块的大小是不一样的，这由协议的性质决定，如 UDP 协议控制块的内存块大小是 sizeof(struct udp_pcb)，而 TCP 协议控制块的 POOL 大小则为 sizeof(struct tcp_pcb)。通过这种方式，就可以将一个个用户配置的宏定义功能需要的 POOL 包含进去，就使得编程变得更加简便。

在这里有一个很有意思的文件，那就是 memp_std.h 文件，该文件位于 include/lwip/priv 目录下，它里面全是宏定义，LwIP 为什么要这样子写呢，其实很简单，当然是为了方便，在不同的地方调用 #include “lwip/priv/memp_std.h” 就能产生不同的效果。

该文件中的宏值定义全部依赖于宏 LWIP_MEMPOOL(name,num,size,desc)，这样，只要外部提供的该宏值不同，则包含该文件的源文件在编译器的预处理后，就会产生不一样的结果。这样，就可以通过在不同的地方多次包含该文件，前面必定提供宏值 MEMPOOL 以产生不同结果。可能有些人看得一脸懵逼，其实我一开始也是这样子，不得不说 LwIP 源码的作者还是很厉害的。

简单来说，就是在外边提供 LWIP_MEMPOOL 宏定义，然后在包含 memp_std.h 文件，编译器就会帮我们处理，我们先来看看 memp_std.h 文件到底有什么内容，具体见*代码清单 5_1*，再举个简单的例子说明一下，具体见*代码清单 5_2*。

代码清单 5-1memp_std.h 文件（已删减）

```
#if LWIP_RAW
LWIP_MEMPOOL(RAW_PCB,         MEMP_NUM_RAW_PCB,
             sizeof(struct raw_pcb),         "RAW_PCB")
#endif /* LWIP_RAW */

#if LWIP_UDP
LWIP_MEMPOOL(UDP_PCB,         MEMP_NUM_UDP_PCB,
             sizeof(struct udp_pcb),         "UDP_PCB")
#endif /* LWIP_UDP */
```

(下页继续)

(续上页)

```
#if LWIP_TCP
LWIP_MEMPOOL(TCP_PCB,          MEMP_NUM_TCP_PCB,
             sizeof(struct tcp_pcb),         "TCP_PCB")

LWIP_MEMPOOL(TCP_PCB_LISTEN, MEMP_NUM_TCP_PCB_LISTEN,
             sizeof(struct tcp_pcb_listen), "TCP_PCB_LISTEN")

LWIP_MEMPOOL(TCP_SEG,          MEMP_NUM_TCP_SEG,
             sizeof(struct tcp_seg),         "TCP_SEG")
#endif /* LWIP_TCP */

#if LWIP_ALTCP && LWIP_TCP
LWIP_MEMPOOL(ALTCP_PCB,        MEMP_NUM_ALTCP_PCB,
             sizeof(struct altcp_pcb),       "ALTCP_PCB")
#endif /* LWIP_ALTCP && LWIP_TCP */

#if LWIP_IPV4 && IP_REASSEMBLY
LWIP_MEMPOOL(REASSDATA,        MEMP_NUM_REASSDATA,
             sizeof(struct ip_reassdata),    "REASSDATA")
#endif /* LWIP_IPV4 && IP_REASSEMBLY */

#if LWIP_NETCONN || LWIP_SOCKET
LWIP_MEMPOOL(NETBUF,           MEMP_NUM_NETBUF,
             sizeof(struct netbuf),          "NETBUF")

LWIP_MEMPOOL(NETCONN,          MEMP_NUM_NETCONN,
             sizeof(struct netconn),         "NETCONN")
#endif /* LWIP_NETCONN || LWIP_SOCKET */
#undef LWIP_MEMPOOL
```

代码清单 5-2 memp_std.h 使用方式的例子

```
typedef enum
{
#define LWIP_MEMPOOL(name,num,size,desc)  MEMP_##name,
#include "lwip/priv/memp_std.h"
    MEMP_MAX
} memp_t;
```

可能很多人一看到代码清单 5_2 的例子，就懵逼了，这写的是什么鬼东西，完全不知道 LwIP 作者想要干什么，但是当你读懂这段代码的时候，你就不得不佩服 LwIP 作者的水平了，那是真的厉害。

先说说“#define LWIP_MEMPOOL(name,num,size,desc) MEMP_##name,”这个宏定义，此处先补充一下 C 语言的连接符“##”相关的知识，## 被称为连接符（concatenator），用来将两个 Token 连接为一个 Token。注意这里连接的对象是 Token 就行，而不一定是宏的变量。在编译器编译的时候，它会扫描源码，将代码分解为一个个的 Token，Token 可以是 C 语言的关键字，如 int、for、while 等，也可以是用户自定义的变量，如，a、num、name 等，当我们经过“#define LWIP_MEMPOOL(name,num,size,desc) MEMP_##name,”这个宏定义后，在编译过程中遇到了“LWIP_MEMPOOL(EmbedFire,num,size,desc)”这句代码，编译器就会将它替换为“MEMP_EmbedFire,”注意，这里有一个英文的逗号“,”，因为现在是定义枚举类型的数据，那么经过编译器处理的代码清单 5_2 代码后，这个枚举变量就会变成以下的内容（假设所有的宏定义都是使能状态），具体见代码清单 5_3。

代码清单 5-3 经过编译器处理的代码

```
typedef enum
{
    MEMP_RAW_PCB,
    MEMP_UDP_PCB,
    MEMP_TCP_PCB,
    MEMP_TCP_PCB_LISTEN,
    MEMP_TCP_SEG,
    MEMP_ALTCP_PCB,
    MEMP_REASSDATA,
```

(下页继续)

(续上页)

```
    MEMP_NETBUF,
    MEMP_NETCONN,
    MEMP_MAX
} memp_t;
```

memp_t 类型在整个内存池的管理中是最重要的存在，通过内存池申请函数申请内存的时候，唯一的参数就是 memp_t 类型的，它将告诉分配的函数在哪种类型的 POOL 中去分配对应的内存块，这样子就直接管理了系统中所有类型的 POOL。

这个枚举变量的 MEMP_MAX 不代表任何类型的 POOL，它只是记录这系统中所有的 POOL 的数量，比如例子中的 MEMP_RAW_PCB 的值为 0，而 MEMP_MAX 的值为 9，就表示当前系统中有 9 种 POOL。

这样子的编程方式，是不是大开眼界了？不过还有一点需要注意的是，在 memp_std.h 文件的最后需要对 LWIP_MEMPOOL 宏定义进行撤销，因为该文件很会被多个地方调用，在每个调用的地方会重新定义这个宏定义的功能，所以在文件的末尾添加这句 #undef LWIP_MEMPOOL 代码是非常有必要的。

在这里还要给大家提个醒，如果以后看 LwIP 源码的时候，发现某个找不到定义，但是编译是没有问题的，那么很可能就是通过“##”连接符产生的宏定义了，例如图 *5_2* 出现的情况，MEMP_RAW_PCB 定义就在 memp_t 类型中，是通过“##”连接符产生的。

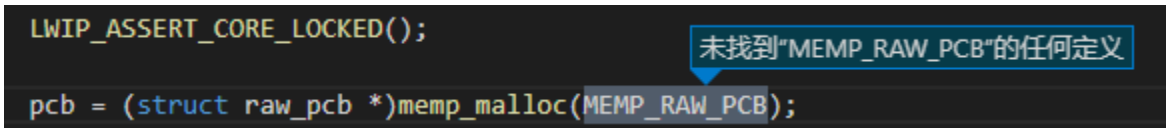


图 5-2 未找到 MEMP_RAW_PCB 的定义

按照这种包含头文件的原理，只需要定义 LWIP_MEMPOOL 宏的作用，就能产生很大与内存池相关的操作，如在 memp.c 文件的开头就定义了如下代码：

代码清单 5-4

```
#define LWIP_MEMPOOL(name,num,size,desc) LWIP_MEMPOOL_DECLARE(name,num,
↪size,desc)
```

(下页继续)

(续上页)

```
#include "lwip/priv/memp_std.h"
```

经过包含 memp_std.h 文件后，再经过编译器的处理，就能得到下面的结果，具体见代码清单 5_5 加粗部分。其实这些编译器处理的代码我们并不需要怎么理会，简单了解一下即可

代码清单 5-5

```
#define LWIP_MEMPOOL(name,num,size,desc) \
        LWIP_MEMPOOL_DECLARE(name,num,size,desc)

#define LWIP_DECLARE_MEMORY_ALIGNED(variable_name, size) \
        u8_t variable_name[LWIP_MEM_ALIGN_BUFFER(size)]

LWIP_MEM_ALIGN_BUFFER(size) (((size) + MEM_ALIGNMENT - 1U))

#define LWIP_MEMPOOL_DECLARE(name,num,size,desc) \
LWIP_DECLARE_MEMORY_ALIGNED(memp_memory_ ## name ## _base, \
((num) * (MEMP_SIZE + MEMP_ALIGN_SIZE(size)))); \
\
LWIP_MEMPOOL_DECLARE_STATS_INSTANCE(memp_stats_ ## name) \
\
static struct memp *memp_tab_ ## name; \
\
const struct memp_desc memp_ ## name = { \
DECLARE_LWIP_MEMPOOL_DESC(desc) \
LWIP_MEMPOOL_DECLARE_STATS_REFERENCE(memp_stats_ ## name) \
LWIP_MEM_ALIGN_SIZE(size), \
(num), \
memp_memory_ ## name ## _base, \
&memp_tab_ ## name \
};
```

(下页继续)

(续上页)

```
/* 编译时候的宏定义 */
LWIP_MEMPOOL(RAW_PCB,MEMP_NUM_RAW_PCB,sizeof(struct raw_pcb),"RAW_PCB")

/* 通过转换后得到的结果，例子是 RAW_PCB */
LWIP_DECLARE_MEMORY_ALIGNED(memp_memory_RAW_PCB_base,
((MEMP_NUM_RAW_PCB) * (MEMP_SIZE + MEMP_ALIGN_SIZE(sizeof(struct raw_
↪pcb)))));

LWIP_MEMPOOL_DECLARE_STATS_INSTANCE(memp_stats_RAW_PCB)

static struct memp *memp_tab_RAW_PCB;

const struct memp_desc memp_RAW_PCB =
{
    DECLARE_LWIP_MEMPOOL_DESC("RAW_PCB")
    LWIP_MEMPOOL_DECLARE_STATS_REFERENCE(memp_stats_RAW_PCB)
    LWIP_MEM_ALIGN_SIZE(sizeof(struct raw_pcb)),
    (MEMP_NUM_RAW_PCB),
    memp_memory_RAW_PCB_base,
    &memp_tab_RAW_PCB
};

/* 再次转换 */
u8_t memp_memory_RAW_PCB_base[(((((MEMP_NUM_RAW_PCB) *
                (MEMP_SIZE + MEMP_ALIGN_SIZE(sizeof(struct raw_pcb)))))
                + MEM_ALIGNMENT - 1U))];

static struct memp *memp_tab_RAW_PCB;

const struct memp_desc memp_RAW_PCB ={
(((sizeof(struct raw_pcb)) + MEM_ALIGNMENT - 1U) & ~(MEM_ALIGNMENT-1U)),
    LWIP_MEM_ALIGN_SIZE(sizeof(struct raw_pcb)),
```

(下页继续)

(续上页)

```
    (MEMP_NUM_RAW_PCB),
    memp_memory_RAW_PCB_base,
    &memp_tab_RAW_PCB
};

/* 代入数据得到，注意，数据是根据自己配置的宏定义得到的 */
u8_t memp_memory_RAW_PCB_base[((4 * 24) + 4 - 1U)];

static struct memp *memp_tab_RAW_PCB;

const struct memp_desc memp_RAW_PCB ={
    ((24) + 4 - 1U) & ~(4-1U)),
    (24),
    4
    memp_memory_RAW_PCB_base,
    &memp_tab_RAW_PCB
};
```

关于包含 memp_std.h 文件的处理就不再过多说明了，用户也不需要了解太多，我们就来看看关于内存池的主要参数，就使用上面的 RAW_PCB 的例子，每种 POOL 在经过编译器都会得到一个结构体，memp_desc memp_XXXX，XXXX 表示对应的 POOL 类型，如 RAW_PCB 的结构体就是 memp_desc memp_RAW_PCB，这里面就记录了该内存块对其后的大小 LWIP_MEM_ALIGN_SIZE(sizeof(struct raw_pcb))。也就是说，在经过编译器的处理，该结构体就保存了每种 POOL 的内存对齐后的大小。

同理该结构体也记录了每种 POOL 的其他参数，如内存块的个数 num，比如 MEMP_NUM_RAW_PCB，这些就是用户配置的宏定义，都会被记录在里面，还有每种 POOL 的描述“DECLARE_LWIP_MEMPOOL_DESC(“RAW_PCB”)”，当然这个参数可用可不用，这只是一个字符串，在输出信息的时候用到。

除了这些信息，还有一个最重要的信息，那就是真正的内存池区域，使用 u8_t memp_memory_XXXX_base 进行定义，XXXX 表示对应的 POOL 类型，每个类型都有自己的内存池区域，是编译器开辟出来的内存空间，简单来说就是一个数组，我们知道这个区域的

的起始地址，就能对它进行操作。

5.2.2 内存池的初始化

在 LwIP 协议栈初始化的时候，memp_init() 会对内存池进行初始化，真正的内存初始化函数是 memp_init_pool() 函数，该函数源码具体见代码清单 5_6。

代码清单 5-6 内存池的初始化（已删减）

```
void
memp_init(void)
{
    u16_t i;

    /* for every pool: */
    for (i = 0; i < LWIP_ARRAYSIZE(memp_pools); i++)
    {
        memp_init_pool(memp_pools[i]);
    }
}

void
memp_init_pool(const struct memp_desc *desc)
{
    int i;
    struct memp *memp;

    *desc->tab = NULL;
    memp = (struct memp *)LWIP_MEM_ALIGN(desc->base);

    memset(memp, 0, (size_t)desc->num * (MEMP_SIZE + desc->size));

    /* 将内存块链接成链表形式 */
```

(下页继续)

(续上页)

```
    for (i = 0; i < desc->num; ++i)
    {
        memp->next = *desc->tab;
        *desc->tab = memp;

    /* 地址偏移 */
    memp = (struct memp *)(void *)((u8_t *)memp + MEMP_SIZE + desc->size);
    }
}
```

该函数是比较简单的函数，就是根据每种 POOL 的 memp_desc 描述进行初始化，在每种类型的 POOL 中将空闲内存块连接成单链表，并且使用 memset() 函数将其内容清零，这样子就初始化完成了。

初始化完成的示意图（只有部分 POOL）具体见图 *5_3*。

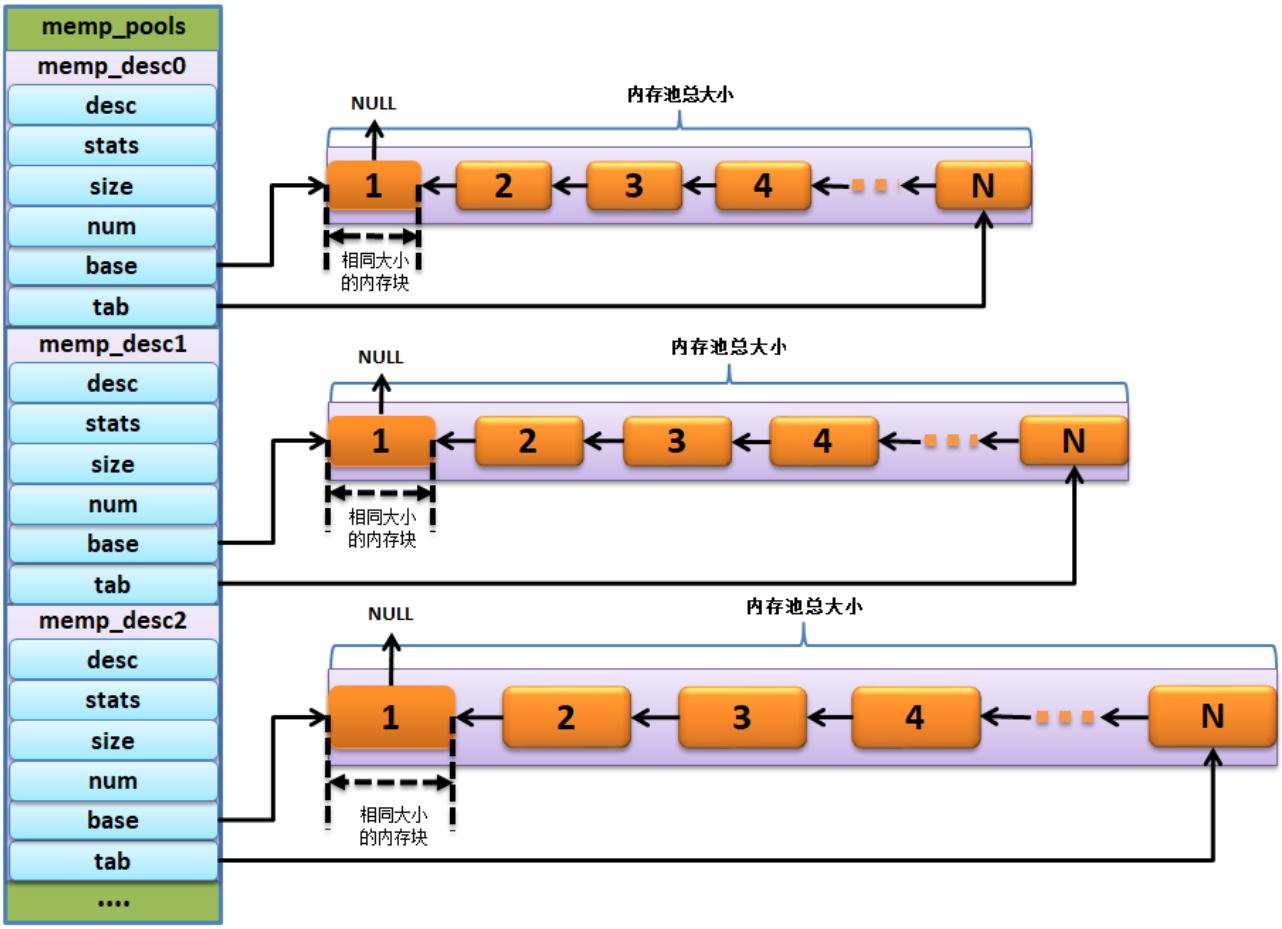


图 5-3 内存池初始化完成示意图

5.2.3 内存分配

内存池的初始化之后，这些内存池中的内存块就可以使用了，这就需要用户通过 memp_malloc 函数进行申请内存块，而内存块的大小就是指定的大小，其过程很简单，就是根据内存池的类型去选择从哪个内存池进行分配，因为不同类型的内存池中内存块大小是不一样的，比如 TCP_PCB 与 UDP_PCB 的大小就不一样，所以申请内存的时候传入的参数是内存池的类型而并非要申请的内存大小，系统中所有的内存池类型都会被记录在 memp_pools 数组中，我们可以将该数组称之为内存池描述表，它负责将系统所有的内存池信息描述并且记录下来，这样子在申请内存的时候就能很迅速得到一个对应的内存块，内存分配函数的源码具体见代码清单 5_7。

代码清单 5-7 内存分配函数（已删减）

```
void *
memp_malloc(memp_t type)
{
    void *memp;
LWIP_ERROR("memp_malloc: type < MEMP_MAX", (type < MEMP_MAX), return NULL;
↪);

    memp = do_memp_malloc_pool(memp_pools[type]);

    return memp;
}

static void *
do_memp_malloc_pool(const struct memp_desc *desc)
{
    struct memp *memp;
    SYS_ARCH_DECL_PROTECT(old_level);
    memp = *desc->tab;
    if (memp != NULL)
    {

        *desc->tab = memp->next;

        LWIP_ASSERT("memp_malloc: memp properly aligned",
                    ((mem_ptr_t)memp % MEM_ALIGNMENT) == 0);

        SYS_ARCH_UNPROTECT(old_level);
        /* cast through u8_t* to get rid of alignment warnings */
        return ((u8_t *)memp + MEMP_SIZE);
    }
    else
```

(下页继续)

(续上页)

```
    {
        SYS_ARCH_UNPROTECT(old_level);
        LWIP_DEBUGF(MEMP_DEBUG | LWIP_DBG_LEVEL_SERIOUS,
                ("memp_malloc: out of memory in pool %s\n", desc->desc));
    }
    return NULL;
}
```

内存池申请函数的核心代码就一句，那就是 memp = *desc->tab;，通过这句代码，能直接得到对应内存块中的第一个空闲内存块，并将其取出，并且移动 *desc->tab 指针，指向下一个空闲内存块，然后将 ((u8_t *)memp + MEMP_SIZE) 返回，MEMP_SIZE 偏移的空间大小，因为内存块需要一些空间存储内存块相关的信息，该宏定义的值是 (LWIP_MEM_ALIGN_SIZE(sizeof(struct memp)) + MEM_SANITY_REGION_BEFORE_ALIGNED)，我们暂时无需理会它，只要知道申请内存块后返回的地址是直接可用的地址即可，而偏移的 MEMP_SIZE 这部分内容是内存分配器管理的空间，用户是不允许触碰的地方，否则就很可能发生错误。

5.2.4 内存释放

同样的，内存释放函数也非常简单的，只需要把使用完毕的内存添加到对应内存池中的空闲内存块链表即可，只不过释放内存有两个参数，一个是 POOL 的类型，还有就是内存块的起始地址，源码具体见*代码清单 5_8*。

代码清单 5-8 内存释放函数（已删减）

```
void
memp_free(memp_t type, void *mem)
{
    LWIP_ERROR("memp_free: type < MEMP_MAX",
            (type < MEMP_MAX), return;);

    if (mem == NULL)
```

(下页继续)

(续上页)

```
    {
        return;
    }
    do_memp_free_pool(memp_pools[type], mem);
}

static void
do_memp_free_pool(const struct memp_desc *desc, void *mem)
{
    struct memp *memp;
    SYS_ARCH_DECL_PROTECT(old_level);

    LWIP_ASSERT("memp_free: mem properly aligned",
                ((mem_ptr_t)mem % MEM_ALIGNMENT) == 0);

    /* cast through void* to get rid of alignment warnings */
    memp = (struct memp *)(void *)((u8_t *)mem - MEMP_SIZE);          (1)

    SYS_ARCH_PROTECT(old_level);

    memp->next = *desc->tab;                                          (2)
    *desc->tab = memp;                                                (3)

    SYS_ARCH_UNPROTECT(old_level);
}
```

代码清单 5-8**(1)**：根据内存块的地址偏移得到内存块的起始地址，因为前面也说了，内存块中有一部分内容是内存分配器操作的，所以需要进行偏移。

代码清单 5-8**(2)**：内存块的下一个就是链表中的第一个空闲内存块。

代码清单 5-8**(3)**：将内存块插入到对应内存池的 *desc->tab 中。

LwIP 对内存池的设计，采用了很多巧妙的地方，特别是对编译全局变量的过程，根据用户配置

的宏定义决定是否编译进去。而且，内存池还采用内存池描述表进行管理系统中所有的内存池，在用户需要某种类型的内存块时候，就直接将其需要的类型传递进去就能得到对应大小的内存块，分配的方式是非常方便并且高效的。

5.3 动态内存堆

在嵌入式开发中，内存管理以及使用是至关重要的，内存使用的多少、内存泄漏等时刻需要注意。合理的内存管理策略将从根本上决定内存分配和回收效率，最终决定系统的整体性能。LwIP 为了能够灵活的使用内存，为使用者提供两种简单却又高效的动态内存管理策略：动态内存堆管理（heap）、动态内存池管理（pool），而内存池管理策略在前面的章节已经讲解，那么现在就来看看内存堆的管理。

其中，动态内存堆管理（heap）又可以分为两种：一种是 C 标准库自带的内存管理策略，另一种是 LwIP 自身实现的内存堆管理策略。这两者的选择需要通过宏值 MEM_LIBC_MALLOC 来选择，且二者只能选择其一。

其次，LwIP 在自身内存堆和内存池的实现上设计得非常灵活。内存池可由内存堆实现，反之，内存堆也可以由内存池实现。通过 MEM_USE_POOLS 和 MEMP_MEM_MALLOC 这两个宏定义来选择，且二者只能选择其一。

5.3.1 内存堆的组织结构

在学习内存堆之前，我们先看看内存堆的的组织结构，它包括了内存数据结构与某些重要的全局变量，具体见*代码清单 5_9*。

代码清单 5-9 内存堆的组织结构（部分）

```
struct mem
{
    /** index (-> ram[next]) of the next struct */
    mem_size_t next;                                                    (1)
    /** index (-> ram[prev]) of the previous struct */
```

（下页继续）

(续上页)

```
    mem_size_t prev;                                                    (2)
    /** 1: this area is used; 0: this area is unused */
    u8_t used;                                                          (3)
#if MEM_OVERFLOW_CHECK
    /** this keeps track of the user allocation size for guard checks */
    mem_size_t user_size;
#endif
};

#define MIN_SIZE             12                                         (4)

LWIP_DECLARE_MEMORY_ALIGNED(ram_heap, MEM_SIZE_ALIGNED+(2U*SIZEOF_STRUCT_
↪MEM)); (5)

#define LWIP_RAM_HEAP_POINTER       ram_heap                                (6)

/** pointer to the heap (ram_heap):
for alignment, ram is now a pointer instead of an array */
static u8_t *ram;                                                       (7)

/** the last entry, always unused! */
static struct mem *ram_end;                                             (8)

#if !NO_SYS
static sys_mutex_t mem_mutex;                                               (9)
#endif

static struct mem * LWIP_MEM_LFREE_VOLATILE lfree;                      (10)
```

代码清单 5-9**(1)(2)**：可能很多人都会认为 next 与 prev 是一个指针，分别指向下一个内存块与上一个内存块，但是其实这两个字段表示的是目的地址的偏移量，基地址是整个内存堆的起始地址。

代码清单 5-9**(3)**：used 字段用于标记该内存是否已经被使用。

代码清单 5-9(**4**)：申请的内存最小为 12 字节，因为一个内存块最起码需要保持 mem 结构体的信息，以便于对内存块进行操作，而该结构体在对齐后的内存大小就是 12 字节。

代码清单 5-9(**5**)：内存堆的大小是由这个宏定义的，该语句在编译器处理之后就是 u8_t ram_heap[(((MEM_SIZE_ALIGNED + (2U * SIZEOF_STRUCT_MEM)) + MEM_ALIGNMENT - 1U))];，其中 MEM_SIZE_ALIGNED 宏是内存堆大小 MEM_SIZE 经过内存对齐后的大小；而 SIZEOF_STRUCT_MEM 则是结构体 mem 经过内存对其后的大小，MEM_ALIGNMENT 则是 CPU 按多少字节对其的宏定义，一般为 4。

代码清单 5-9(**6**)：ram_heap[] 就是内核的内存堆空间，LWIP_RAM_HEAP_POINTER 这个宏定义相对于重新命名 ram_heap。

代码清单 5-9(**7**)：ram 是一个全局指针变量，指向内存堆对齐后的起始地址，因为真正的内存堆起始地址不一定是按照 CPU 的对齐方式对齐的，而此处就要确保内存堆的起始地址是对齐的。

代码清单 5-9(**8**)：mem 类型指针，指向内存堆中最后一个内存块。

代码清单 5-9(**9**)：互斥量，用户保护内存堆的互斥量，暂时未用。

代码清单 5-9(**10**)：mem 类型指针，指向内存堆中低地址的空闲内存块，简单来说就是空闲内存块链表指针。

5.3.2 内存堆初始化

在内核初始化的时候，会调用 mem_init() 函数进行内存堆的初始化，内存堆初始化主要的过程就是对上述所属的内存堆组织结构进行初始化，主要设置内存堆的起始地址，以及初始化空闲列表。根据用户配置的宏定义进行相关初始化，配置不同其实现也不同（可能为空），该函数源码具体见*代码清单 5_10*。

代码清单 5-10 mem_init() 源码

```
void
mem_init(void)
{
    struct mem *mem;
```

(下页继续)

(续上页)

```
    LWIP_ASSERT("Sanity check alignment",
                (SIZEOF_STRUCT_MEM & (MEM_ALIGNMENT - 1)) == 0);

    /* align the heap */
    ram = (u8_t *)LWIP_MEM_ALIGN(LWIP_RAM_HEAP_POINTER);                (1)
    /* initialize the start of the heap */
    mem = (struct mem *)(void *)ram;                                    (2)
    mem->next = MEM_SIZE_ALIGNED;                                       (3)
    mem->prev = 0;                                                      (4)
    mem->used = 0;                                                      (5)
    /* initialize the end of the heap */
    ram_end = ptr_to_mem(MEM_SIZE_ALIGNED);                             (6)
    ram_end->used = 1;                                                  (7)
    ram_end->next = MEM_SIZE_ALIGNED;
    ram_end->prev = MEM_SIZE_ALIGNED;
    MEM_SANITY();

    /* initialize the lowest-free pointer to the start of the heap */
    lfree = (struct mem *)(void *)ram;                                  (8)

    MEM_STATS_AVAIL(avail, MEM_SIZE_ALIGNED);

    if (sys_mutex_new(&mem_mutex) != ERR_OK)                            (9)
    {
        LWIP_ASSERT("failed to create mem_mutex", 0);
    }
}
```

代码清单 5-10(**1**)：内存堆空间对齐，LWIP_RAM_HEAP_POINTER 宏定义就是 ram_mem，内存堆对齐后的起始地址被记录在 ram 中。

代码清单 5-10(**2**)：在内存堆起始位置放置一个 mem 类型的结构体，因为初始化后的内存堆就是

一个大的空闲内存块，每个空闲内存块的前面都需要放置一个 mem 结构体。

代码清单 5-10(**3**)：下一个内存块的偏移量为 MEM_SIZE_ALIGNED，这相对于直接到内存堆的结束地址了。

代码清单 5-10(**4**)：上一个内存块为空。

代码清单 5-10(**5**)：标记未被使用。

代码清单 5-10(**6**)：指针移动到内存堆末尾的位置，并且在那里放置一个 mem 类型的结构体，并初始化表示内存堆结束的内存块。

代码清单 5-10(**7**)：标记已经使用了该内存块，因为结束的地方是没有内存块的，不能被分配出去，只能表示已经使用。同时 mem 结构体的 next 与 prev 字段都指向自身，此处仅表示已经到了内存堆的结束的地方，并无内存可以分配。

代码清单 5-10(**8**)：空闲内存块链表指针指向内存堆的起始地址，因为当前只有一个内存块。

代码清单 5-10(**9**)：创建一个内存堆分配时候使用的互斥量，如果是无操作系统的情况，该语句等效于空。

经过 mem_init() 函数后，内存堆会被初始化为两个内存块，第一个内存块的大小就是整个内存堆的大小，而第二个内存块就是介绍内存块，其大小为 0，并且被标记为已使用状态，无法进行分配。值得注意的是，系统在运行的时候，随着内存的分配与释放，lfree 指针的指向地址不断改变，都指向内存堆中低地址空闲内存块，而 ram_end 则不会改变，它指向系统中最后一个内存块，也就是内存堆的结束地址。初始化完成的示意图具体见图 *5_4*。

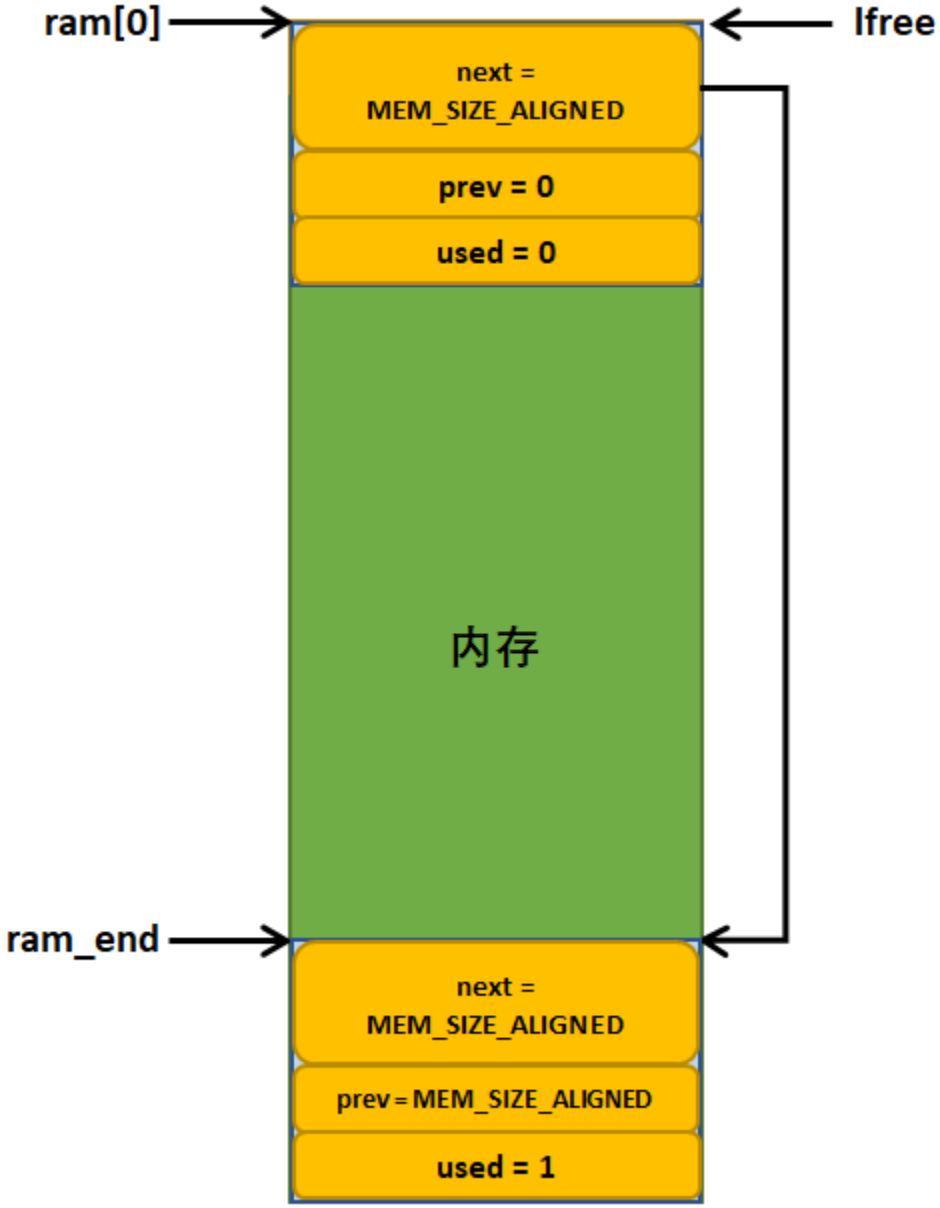


图 5-4 内存堆初始化完成示意图

5.3.3 内存分配

内存分配函数根据用户指定申请大小的内存空间进行分配内存，其大小要大于 MIN_SIZE，LwIP 中使用内存分配算法是首次拟合方法，其分配原理就是在空闲内存块链表中遍历寻找，直到找到第一个合适用户需求大小的内存块进行分配，如果该内存块能进行分割，则将用户需要大小的内存块分割出来，剩下的空闲内存块则重新插入空闲内存块链表中。经过多次的分配与释放，很可能会出现内存碎片，当然，LwIP 也有解决的方法，在内存释放中会进行讲解。

mem_malloc() 函数是 LwIP 中内存分配函数，其参数是用户指定大小的内存字节数，如果申请成功则返回内存块的地址，如果内存没有分配成功，则返回 NULL，分配的内存空间会受到内存对其的影响，可能会比申请的内存略大，比如用户需要申请 22 个字节的内存，而 CPU 是按照 4 字节内存对齐的，那么分配的时候就会申请 24 个字节的内存块。

内存块在申请成功后返回的是内存块的起始地址，但是该内存并未进行初始化，可能包含任意的随机数据，用户可以立即对其进行初始化或者写入有效数据以防止数据错误。此外内存堆是一个全局变量，在操作系统的环境中进行申请内存块是不安全的，所以 LwIP 使用互斥量实现了对临界资源的保护，在多个线程同时申请或者释放的时候，会因为互斥量的保护而产生延迟。内存分配函数具体见*代码清单 5_11*。

代码清单 5-11 mem_malloc() 源码

```
void *
mem_malloc(mem_size_t size_in)
{
    mem_size_t ptr, ptr2, size;
    struct mem *mem, *mem2;
    LWIP_MEM_ALLOC_DECL_PROTECT();

    if (size_in == 0)
    {
        return NULL;
    }

    size = (mem_size_t)LWIP_MEM_ALIGN_SIZE(size_in);          (1)
```

(下页继续)

(续上页)

```
if (size < MIN_SIZE_ALIGNED)
{

    size = MIN_SIZE_ALIGNED;                    (2)
}

if ((size > MEM_SIZE_ALIGNED) || (size < size_in))
{
    return NULL;                                        (3)
}

sys_mutex_lock(&mem_mutex);                     (4)

LWIP_MEM_ALLOC_PROTECT();

/* 遍历空闲内存块链表 */
for (ptr = mem_to_ptr(lfree); ptr < MEM_SIZE_ALIGNED - size;
        ptr = ptr_to_mem(ptr)->next)            (5)
{
    mem = ptr_to_mem(ptr);                              (6)

    if((!mem->used)&&(mem->next-(ptr + SIZEOF_STRUCT_MEM))>= size) (7)
    {
        if (mem->next - (ptr + SIZEOF_STRUCT_MEM) >=
                (size + SIZEOF_STRUCT_MEM + MIN_SIZE_ALIGNED))
        {
            ptr2 = (mem_size_t)(ptr + SIZEOF_STRUCT_MEM + size);    (8)
            LWIP_ASSERT("invalid next ptr",ptr2 != MEM_SIZE_ALIGNED);
            /* create mem2 struct */
            mem2 = ptr_to_mem(ptr2);                    (9)
            mem2->used = 0;                             (10)
```

(下页继续)

(续上页)

```
            mem2->next = mem->next;
            mem2->prev = ptr;
            /* and insert it between mem and mem->next */
            mem->next = ptr2;
            mem->used = 1;                                          (11)

            if (mem2->next != MEM_SIZE_ALIGNED)
            {
                ptr_to_mem(mem2->next)->prev = ptr2;                (12)
            }
            MEM_STATS_INC_USED(used, (size + SIZEOF_STRUCT_MEM));
        }
        else
        {
            mem->used = 1;                                          (13)
            MEM_STATS_INC_USED(used, mem->next - mem_to_ptr(mem));
        }

        if (mem == lfree)                                           (14)
        {
            struct mem *cur = lfree;
        /*Find next free block after mem and update lowest free␣
↪pointer */
            while (cur->used && cur != ram_end)
            {
                cur = ptr_to_mem(cur->next);                        (15)
            }
            lfree = cur;                                            (16)
            LWIP_ASSERT("mem_malloc: !lfree->used",
                        ((lfree == ram_end) || (!lfree->used)));
        }
        LWIP_MEM_ALLOC_UNPROTECT();
```

(下页继续)

(续上页)

```
            sys_mutex_unlock(&mem_mutex);                          (17)
            LWIP_ASSERT("mem_malloc: allocated memory not above ram_end.",
            (mem_ptr_t)mem +SIZEOF_STRUCT_MEM+size <=(mem_ptr_t)ram_end);
            LWIP_ASSERT("mem_malloc: allocated memory properly aligned.",
            ((mem_ptr_t)mem + SIZEOF_STRUCT_MEM) % MEM_ALIGNMENT == 0);
            LWIP_ASSERT("mem_malloc: sanity check alignment",
                        (((mem_ptr_t)mem) & (MEM_ALIGNMENT - 1)) == 0);


        MEM_SANITY();
        return (u8_t *)mem + SIZEOF_STRUCT_MEM + MEM_SANITY_OFFSET; (18)
        }
    }
    MEM_STATS_INC(err);
    LWIP_MEM_ALLOC_UNPROTECT();
    sys_mutex_unlock(&mem_mutex);                                  (19)
    LWIP_DEBUGF(MEM_DEBUG | LWIP_DBG_LEVEL_SERIOUS,
        ("mem_malloc: could not allocate %"S16_F" bytes\n", (s16_t)size));
    return NULL;
}
```

代码清单 5-11**(1)**：将用户申请的内存大小进行对齐操作。

代码清单 5-11**(2)**：如果用户申请的内存大小小于最小的内存对齐大小 MIN_SIZE_ALIGNED，则设为最小的默认值。

代码清单 5-11**(3)**：如果申请的内存大小大于整个内存堆对齐后的大小，则返回 NULL，申请内存失败。

代码清单 5-11**(4)**：获得互斥量，这一句代码在操作系统环境才起作用。

代码清单 5-11**(5)**：遍历空闲内存块链表，直到找到第一个适合用户需求的内存块大小。

代码清单 5-11**(6)**：得到这个内存块起始地址。

代码清单 5-11(**7**)：如果该内存块是未使用的，并且它的大小不小于用户需要的大小加上 mem 结构体的大小，那么就满足用户的需求。

代码清单 5-11(**8**)：既然满足用户需求，那么这个内存块可能很大，不能直接分配给用户，否则就是太浪费了，那就看看这个内存块能不能切开，如果能就将一部分分配给用户即可，程序能执行到这里，说明内存块能进行分割，那就通过内存块的起始地址与用户需求大小进行偏移，得到剩下的的内存起始块地址 ptr2。

代码清单 5-11(**9**)：将该地址后的内存空间作为分割之后新内存块 mem2，将起始地址转换为 mem 结构体用于记录内存块的信息。

代码清单 5-11(**10**)：标记为未使用的内存块，并且将其插入空闲内存块链表中。

代码清单 5-11(**11**)：被分配出去的内存块 mem 标记为已使用状态。

代码清单 5-11(**12**)：如果 mem2 内存块的下一个内存块不是链表中最后一个内存块（结束地址），那就将它下一个的内存块的 prve 指向 mem2。

代码清单 5-11(**13**)：如果不能分割，直接将分配的内存块标记为已使用即可。

代码清单 5-11(**14**)：如果被分配出去的内存块是 lfree 指向的内存块，那么就需要重新给 lfree 赋值。

代码清单 5-11(**15**)：找到第一个低地址的空闲内存块。

代码清单 5-11(**16**)：将 lfree 指向该内存块。

代码清单 5-11(**17**)：释放互斥量。

代码清单 5-11(**18**)：返回内存块可用的起始地址，因为内存块的块头需要使用 mem 结构体保存内存块的基本信息。

代码清单 5-11(**19**)：如果没法分配成功，就释放互斥量并且退出。

如果在初始化后的内存堆中分配了一个大小为 24 字节的内存块出去，则分配完成的示意图具体见图 *5_5*。

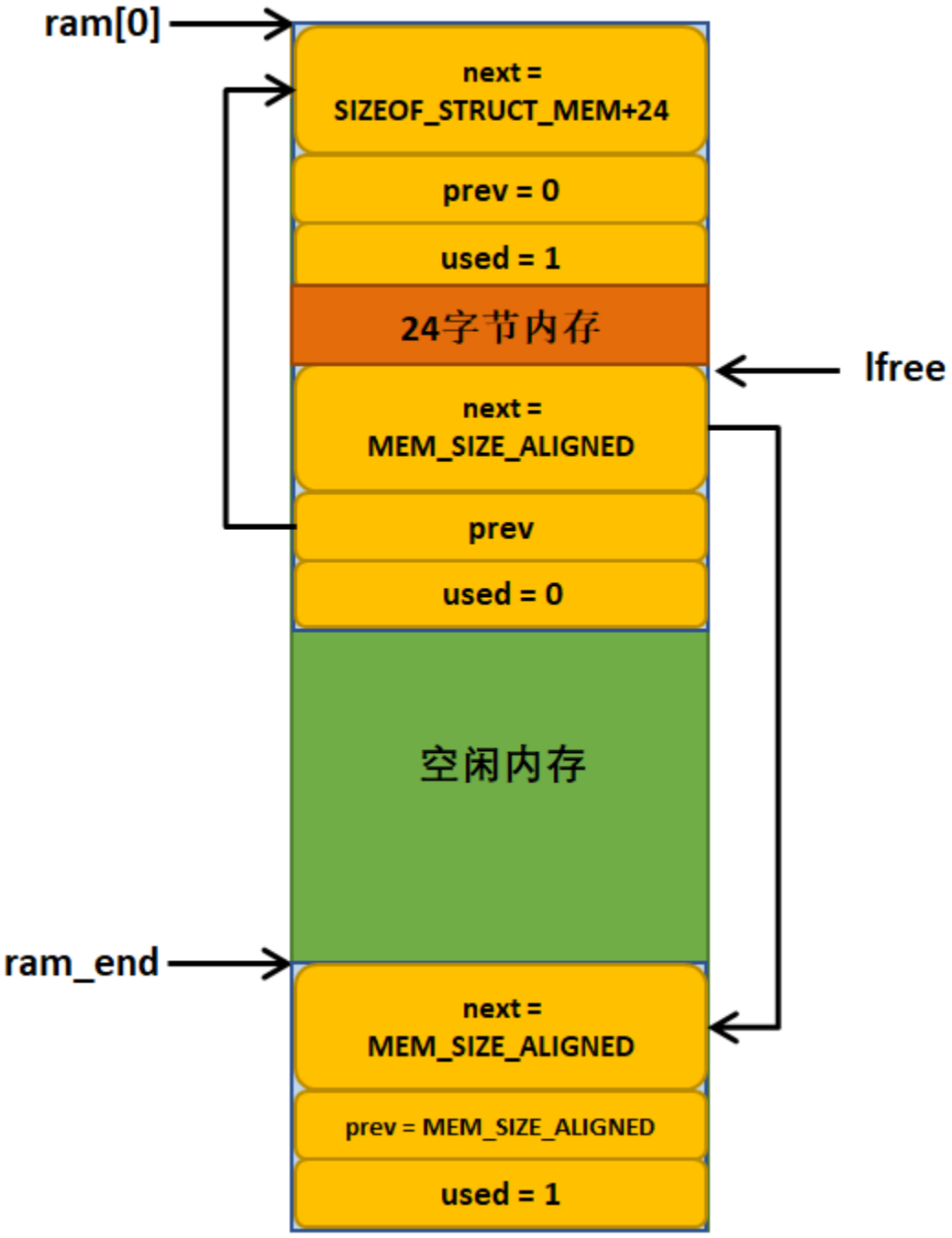


图 5-5 申请 24 字节内存块完成示意图

5.3.4 内存释放

内存释放的操作也是比较简单的，LwIP 是这样子做的：它根据用户释放的内存块地址，通过偏移 mem 结构体大小得到正确的内存块起始地址，并且根据 mem 中保存的内存块信息进行释放、合并等操作，并将 used 字段清零，表示该内存块未被使用。

LwIP 为了防止内存碎片的出现，通过算法将内存相邻的两个空闲内存块进行合并，在释放内存块的时候，如果内存块与上一个或者下一个空闲内存块在地址上是连续的，那么就将这两个内存块进行合并，其源码具体见*代码清单 5_12*。

代码清单 5-12 源码

```
void
mem_free(void *rmem)
{
    struct mem *mem;
    LWIP_MEM_FREE_DECL_PROTECT();

    if (rmem == NULL)                                          (1)
    {
        LWIP_DEBUGF(MEM_DEBUG | LWIP_DBG_TRACE |
                    LWIP_DBG_LEVEL_SERIOUS,
                    ("mem_free(p == NULL) was called.\n"));
        return;
    }
    if ((((mem_ptr_t)rmem) & (MEM_ALIGNMENT - 1)) != 0)
    {
        LWIP_MEM_ILLEGAL_FREE("mem_free: sanity check alignment");
        LWIP_DEBUGF(MEM_DEBUG | LWIP_DBG_LEVEL_SEVERE,
                    ("mem_free: sanity check alignment\n"));
        /* protect mem stats from concurrent access */
        MEM_STATS_INC_LOCKED(illegal);
        return;
    }
```

(下页继续)

(续上页)

```
mem = (struct mem *)(void *)((u8_t *)rmem -
    (SIZEOF_STRUCT_MEM + MEM_SANITY_OFFSET));    (2)

if ((u8_t *)mem < ram ||
        (u8_t *)rmem + MIN_SIZE_ALIGNED > (u8_t *)ram_end) (3)
{
    LWIP_MEM_ILLEGAL_FREE("mem_free: illegal memory");
    LWIP_DEBUGF(MEM_DEBUG | LWIP_DBG_LEVEL_SEVERE,
                ("mem_free: illegal memory\n"));
    /* protect mem stats from concurrent access */
    MEM_STATS_INC_LOCKED(illegal);
    return;
}

/* protect the heap from concurrent access */
LWIP_MEM_FREE_PROTECT();

/* mem has to be in a used state */
if (!mem->used)                                    (4)
{
    LWIP_MEM_ILLEGAL_FREE("mem_free: illegal \
memory: double free");
    LWIP_MEM_FREE_UNPROTECT();
    LWIP_DEBUGF(MEM_DEBUG | LWIP_DBG_LEVEL_SEVERE,
                ("mem_free: illegal memory: double free?\n"));
    /* protect mem stats from concurrent access */
    MEM_STATS_INC_LOCKED(illegal);
    return;
}

if (!mem_link_valid(mem))                          (5)
```

(下页继续)

(续上页)

```
    {
        LWIP_MEM_ILLEGAL_FREE("mem_free: illegal memory:\
    non-linked: double free");
        LWIP_MEM_FREE_UNPROTECT();
        LWIP_DEBUGF(MEM_DEBUG | LWIP_DBG_LEVEL_SEVERE,
                ("mem_free: illegal memory: non-linked: double free?\n"));
        /* protect mem stats from concurrent access */
        MEM_STATS_INC_LOCKED(illegal);
        return;
    }

    /* mem is now unused. */
    mem->used = 0;                                          (6)

    if (mem < lfree)
    {
        /* the newly freed struct is now the lowest */
        lfree = mem;                                                (7)
    }

    MEM_STATS_DEC_USED(used, mem->next -
                    (mem_size_t)(((u8_t *)mem - ram)));

    /* finally, see if prev or next are free also */
    plug_holes(mem);                                                (8)

    MEM_SANITY();

    LWIP_MEM_FREE_UNPROTECT();
}
```

代码清单 5-12**(1)**：如果释放的地址为空，则直接返回。

代码清单 5-12**(2)**：对释放的地址进行偏移，得到真正内存块的起始地址。

代码清单 5-12**(3)**：判断一下内存块的起始地址是否合法，如果不合法则直接返回。

代码清单 5-12**(4)**：判断一下内存块是否被使用，如果是未使用的也直接返回。

代码清单 5-12**(5)**：判断一下内存块在链表中的连接是否正常，如果不正常也直接返回。

代码清单 5-12**(6)**：程序执行到这里，表示内存块能正常释放，就将 used 置 0 表示已经释放了内存块。

代码清单 5-12**(7)**：如果刚刚释放的内存块地址比 lfree 指向的内存块地址低，则更新 lfree 指针。

代码清单 5-12**(8)**：调用 plug_holes() 函数尝试进行内存块合并，如果能合并则合并，该函数就是我们说的内存块合并算法，只要新释放的内存块与上一个或者下一个空闲内存块在地址上是连续的，则进行合并，该函数的代码具体见*代码清单 5_13*，该函数比较容易理解，就不做过多赘述。

代码清单 5-13 plug_holes() 源码

```
static void
plug_holes(struct mem *mem)
{
    struct mem *nmem;
    struct mem *pmem;

    LWIP_ASSERT("plug_holes: mem >= ram", (u8_t *)mem >= ram);
    LWIP_ASSERT("plug_holes: mem < ram_end",
                (u8_t *)mem < (u8_t *)ram_end);
    LWIP_ASSERT("plug_holes: mem->used == 0", mem->used == 0);

    /* plug hole forward */
    LWIP_ASSERT("plug_holes: mem->next <= MEM_SIZE_ALIGNED",
                mem->next <= MEM_SIZE_ALIGNED);

    nmem = ptr_to_mem(mem->next);
```

(下页继续)

(续上页)

```
    if (mem != nmem && nmem->used == 0 &&
            (u8_t *)nmem != (u8_t *)ram_end)
    {
    /* if mem->next is unused and not end of ram, combine mem and mem->
↪next */
        if (lfree == nmem)
        {
            lfree = mem;
        }
        mem->next = nmem->next;
        if (nmem->next != MEM_SIZE_ALIGNED)
        {
            ptr_to_mem(nmem->next)->prev = mem_to_ptr(mem);
        }
    }

    /* plug hole backward */
    pmem = ptr_to_mem(mem->prev);
    if (pmem != mem && pmem->used == 0)
    {
        /* if mem->prev is unused, combine mem and mem->prev */
        if (lfree == mem)
        {
            lfree = pmem;
        }
        pmem->next = mem->next;
        if (mem->next != MEM_SIZE_ALIGNED)
        {
            ptr_to_mem(mem->next)->prev = mem_to_ptr(pmem);
        }
    }
}
```

对内存释放函数的操作要非常小心，尤其是传递给函数的参数，该参数必须是内存申请返回的地址，这样子才能保证系统根据该地址去寻找内存块中的 mem 结构体，最终通过操作 mem 结构体才能实现内存块的释放操作，并且这样子才有可能进行内存块的合并，否则就没法正常合并内存块，还会把整个内存堆打乱，这样子就会很容易产生内存碎片。

此外，用户在申请内存的时候要注意及时释放内存块，否则就会造成内存泄漏，什么是内存泄漏呢？就是用户在调用内存分配函数后，没有及时或者进行错误的内存释放操作，一次两次这样子的操作并没有什么影响，如果用户周期性调用 mem_malloc() 函数进行内存申请，并且在内存使用完的时候么有释放，这样子程序就会用完内存堆中的所有内存，最终导致内存耗尽无法申请内存，出现死机等现象。

5.4 使用 C 库的 malloc 和 free 来管理内存

在前面的章节也说了，LwIP 支持使用 C 标准库的 malloc 与 free 进行内存的管理，当宏定义 MEM_LIBC_MALLOC 被定义的时候，编译器就会把以下代码编译进去，就会采用 C 标准库的 malloc 与 free 函数，具体见代码清单 5_14。

代码清单 5-14 使用 C 标准库进行内存管理相关的宏定义

```
#if MEM_LIBC_MALLOC
void
mem_init(void)
{
}
void *
mem_trim(void *mem, mem_size_t size)
{
    LWIP_UNUSED_ARG(size);
    return mem;
}

#ifndef mem_clib_free
```

(下页继续)

(续上页)

```
#define mem_clib_free free
#endif
#ifndef mem_clib_malloc
#define mem_clib_malloc malloc
#endif
#ifndef mem_clib_calloc
#define mem_clib_calloc calloc
#endif

#define MEM_LIBC_STATSHELPER_SIZE 0

#endif
```

如果用户选择了使用 C 标准库这种方式管理内存，那么内存处理函数 void mem_init(void) 和 void* mem_trim(void *mem, mem_size_t size) 将没有实际的实现内容，因为既然选择了 C 库策略，那肯定必然是没法实现的，当然 ram_heap 也不会被编译，用户申请的内存块是在 C 标准库管理的系统堆中进行的，这就意味着我们必须将整个 C 标准库初始化好，并为其建立内存堆空间。所以我们不建议使用这种方式进行内存管理。

5.5 LwIP 中的配置

LwIP 中，内存的选择是通过以下这几个宏值来决定的，根据用户对宏值的定义值来判断使用那种内存管理策略，具体如下：

- MEM_LIBC_MALLOC：该宏定义是否使用 C 标准库自带的内存分配策略。该值默认情况下为 0，表示不使用 C 标准库自带的内存分配策略。即默认使用 LwIP 提供的内存堆分配策略。如果要使用 C 标准库自带的分配策略，则需要把该值定义为 1。

当该宏定义为 0 表示使用 LwIP 自己实现的动态内存管理策略。LwIP 的动态内存管理策略又分为两种实现形式：一种通过内存堆 (HEAP) 管理策略来实现内存管理 (大数组)，另一种是通过内存池 (POOL) 管理策略来实现内存管理 (事先开辟好的内存池)。

- MEMP_MEM_MALLOC：该宏定义表示是否使用 LwIP 内存堆分配策略实现内存池分配（即：要从内存池中获取内存时，实际是从内存堆中分配）。默认情况下为 0，表示不从内存堆中分配，内存池为独立一块内存实现。与 MEM_USE_POOLS 只能选择其一。
- MEM_USE_POOLS：该宏定义表示是否使用 LwIP 内存池分配策略实现内存堆的分配（即：要从内存堆中获取内存时，实际是从内存池中分配）。默认情况下为 0，表示不使用从内存池中分配，内存堆为独立一块内存实现。与 MEMP_MEM_MALLOC 只能选择其一。

要使用内存池的方式实现内存堆分配，则需要将 MEM_USE_POOLS 与 MEMP_USE_CUSTOM_POOLS 定义为 1，并且宏定义 MEMP_MEM_MALLOC 必须为 0，除此之外还需要做一下处理：

创建一个 lwippools.h 文件，在该文件中添加类似*代码清单 5_15* 初始化内存池相关的代码，内存池的大小及数量是由用户自己决定的。

代码清单 5-15lwippools.h 文件

```
LWIP_MALLOC_MEMPOOL_START

LWIP_MALLOC_MEMPOOL(20, 256)

LWIP_MALLOC_MEMPOOL(10, 512)

LWIP_MALLOC_MEMPOOL(5, 1512)

LWIP_MALLOC_MEMPOOL_END
```

此处需要注意一点的是，内存池的大小要依次增大，在编译阶段，编译器就会将这些内存个数及大小添加到系统的内存池之中，用户在申请内存的时候，根据其需要的大小在这些内存池中选择最合适的大小的内存块进行分配，如果具有最匹配的内存池中的内存块已经用完，则选择更大的内存池进行分配，只不过这样子会浪费更多的内存，当然，内存池的分配效率也是最高的，也相对于是我们常说的以空间换时间。

关于如何选择这些宏定义及其分配策略，具体见表格 5-1。

表格 5-1 宏定义及其分配策略

MEMP_MEM_MALLOC	MEM_USE_POOLS	内存分配策略
0	**0**	LwIP 中默认的宏定义，内存池与内存堆独立实现，互不相干。
0	**1**	内存堆的实现由内存池实现。
1	**0**	内存池的实现由内存堆实现。
1	**1**	不允许的方式。

总结来说，无论宏值怎么配置，LWIP 都有两种内存管理策略：内存堆和内存池。

第 6 章　网络数据包

TCP/IP 是一种数据通信机制，因此，协议栈的实现本质上就是对数据包进行处理，为了实现高效的效率，LwIP 数据包管理要提供一种高效处理的机制。协议栈各层能对数据包进行灵活的处理，同时减少数据在各层间传递时的时间与空间开销，这是提高协议栈工作效率的关键点。在 BSD 的实现中，一个描述数据包的结构体叫做 mbuf，同样的 在 LwIP 中，也有个类似的结构，称之为 pbuf，本章所有知识点将围绕 pbuf 而展开。

什么是数据包？数据包的种类可谓是五花八门，无奇不有，就比如网卡上的接收到的数据，它可以是一个一千多字节的数据包，也可以是几十个字节的 ARP 数据包，还有用户的数据，这些数据可能存在 RAM、ROM 上，重点是这些数据大小不一，可以是几个字节，也看是上千个字节，并且 LwIP 各层在数据处理的时候极力避免进行数据的拷贝，所以就需要一个数据包这些数据进行统一的管理，使得 LwIP 处理效率更加高效。

6.1 TCP/IP 协议的分层思想

在标准的 TCP/IP 协议栈中，各层之间都是一个独立的模块，它有着很清晰的层次结构，每一层只负责完成该层的处理，不会越界到其他层次去读写数据。而 LwIP 只是一个轻量级 TCP/IP 协议栈，它只是一个较完整的 TCP/IP 协议，多应用在嵌入式领域中，由于处理器的性能有限，LwIP 并没有采用很明确的分层结构，它假设各层之间的部分数据和结构体和实现原理在其他层是可见的，简单来说就传输层知道 IP 层是如何封装数据、传递数据的，IP 层知道链路层是怎么封装数据的等等。

为什么要模糊分层的处理？简单来说就是为了提高效率，例如链路层完成数据包在物理线路上传输的封装；IP 层完成数据包的选择和路由，负责将数据包发送到目标主机；传输层负责根据 IP 地址将数据包传输到指定主机，端口号识别一台主机的线程，向不同的应用层递交数据；但是，如果按照标准的 TCP/IP 协议栈这种严格的分层思想，在数据传输的时候就需要层层拷贝，因为各层之间的内存都不是共用的，在链路层递交到 IP 层需要拷贝，在 IP 层递交到传输层需要拷贝，反之亦然，这样子每当收到或者发送一个数据的时候都要 CPU 去拷贝数据，这个效率就太慢了，

所以 LwIP 假设各层之间的资源都是共用的，各层之间的实现方式也是已知的，那么在 IP 层往传输层递交数据的时候，链路层往 IP 层递交数据的时候就无需再次拷贝，直接操作协议栈中属于其他层次的字段，得到相应的信息，然后直接读取传递的数据即可，这样子处理的方式就无需拷贝，各个层次之间存在交叉存取数据的现象，既节省系统的空间也节省处理的时间，而且更加灵活。例如在传输层，在计算 TCP 报文段的校验以及 TCP 在重装无需报文时，TCP 层必须知道该报文的源 IP 地址和目的 IP 地址，为了得到这些信息，传输层并不是调用 IP 层的接口，而是直接访问数据包中的 IP 数据报首部，因为传输层已经知道 IP 层的数据报的格式及作用，直接访问读取这些信息即可。

在小型嵌入式设备中，LwIP 与用户程序之间通常没有太严格的分层结构，这种方式允许用户处理数据与内核之间变得更加宽松。LwIP 假设用户完全了解协议栈内部的数据处理机制，用户程序可以直接访问协议栈内部各层的数据包，可以让协议栈与用户使用同样的内存区域，允许用户直接对这片区域进行读写操作，这样子就很好地避免了拷贝的现象，当然这样子的做法也有缺陷，取决于用户对协议栈处理过程的了解程度，因为数据是公共的，如果处理不正确那就让协议栈也没法正常工作。

当然，除了标准的 TCP/IP 协议，还存在很多其他的 TCP/IP 协议，即使这些协议栈内部存在着模糊分层、交叉存取现象，但是对协议栈外部的应用层则保持着明显的分层结构，在操作系统中，TCP/IP 协议栈往往被设计为内核代码的一部分，用户可以的函数仅仅是协议栈为用户提供的那些，或者直接完全封装起来，用户的操作类似于读写文件的方式进行（如 BSD Socket），这样子用户就无法避免数据的拷贝，在数据发送的时候，用户数据必须从用户区域拷贝到协议栈内部，在数据接收的时候，协议栈内部数据也将被拷贝到用户区域。

严格按照 TCP 协议的分层思想会导致数据包在各层递交产生内存拷贝问题，影响性能。为了节省时间和空间的开销，LwIP 没有遵循严格的分层机制，各层次之间存在交叉存取的现象，提高效率。

由于 LwIP 的内存共享机制，使得应用程序能直接对协议栈内核的内存区域直接操作，减少时间和空间的损耗。

6.2 LwIP 的线程模型

线程模型可以理解为协议栈的实现被划分在多个线程之中，如让协议栈的各个层次都独立成为一个线程，在这种模式下，各个层次都有严格分层结构，各个层次的提供的 API 接口也是分层清晰的，这样子能使得编程的时候更加简便，代码的组织也更加灵活，当然，按照前面所说的，在嵌入式设备中，这种严格的分层结构并不是最好的，在数据包向其他层递交的时候，都需要进行拷贝以及切换线程，这是一个很大的开销，当数据量太大的时候，这种开销就足以导致系统没法处理大量的数据，如一个数据包在各个层次间的递交至少需要进行 3 次切换线程：底层网卡接收到一个数据包，此时是链路层的线程在工作，当它往上层递交该数据包的时候，就需要切换线程，该数据包被拷贝到 IP 层，此时是 IP 层的线程在工作，当 IP 层处理完毕之后，要向传输层递交数据包，那么此时又需要切换线程，到传输层的线程中处理该数据包，这样子使得协议栈的效率非常低下。

还有一种方式就是协议栈与操作系统融合，成为操作系统的一部分，这样子用户线程与协议栈内核之间都是通过操作系统提供的函数来实现的，这种情况让协议栈各层之间与用户线程就没有很严格的分层结构，各层之间能交叉存取，从而提高效率。

但是 LwIP 采用了另一种方式，让协议栈内核与操作系统相互隔离，协议栈仅仅作为操作系统的一个独立线程存在，用户程序能驻留在协议栈内部，协议栈通过回调函数实现用户与协议栈之间的数据交互；也可以让用户程序单独实现一个线程，与协议栈使用系统的信号量和邮箱等 IPC 通信机制联系起来，进行数据的交互。当使用第一种通过回调函数进行交互情况的时候，也就是我们所说的 RAW API 编程，而使用第二种通过操作系统 IPC 通信机制的时候，就是另外两种 API 编程，即 NETCONN API 和 Socket API。当然这样子既有优点也有缺点，优点就是能在任何的操作系统中移植，缺点就是受到操作系统的影响，因为即使 LwIP 作为一个独立的线程，也是需要借助操作系统进行调度的，因此，协议栈的响应的实时性会有一定影响，并且建议设置 LwIP 线程的优先级为最高优先级。

6.3 pbuf 结构体说明

pbuf 就是一个描述协议栈中数据包的数据结构，LwIP 中在 pbuf.c 和 pubf.h 实现了协议栈数据包管理的所有函数与数据结构，pbuf 数据结构的定义具体见*代码清单 6_1*。

代码清单 6-1pbuf 结构体

```
/** Main packet buffer struct */
struct pbuf
{
    /** next pbuf in singly linked pbuf chain */
    struct pbuf *next;                                          (1)

    /** pointer to the actual data in the buffer */
    void *payload;                                              (2)

    u16_t tot_len;                                              (3)

    /** length of this buffer */
    u16_t len;                                                  (4)

    u8_t type_internal;                                         (5)

    /** misc flags */
    u8_t flags;                                                 (6)

    LWIP_PBUF_REF_T ref;                                        (7)

    /** For incoming packets, this contains the input netif's index */
    u8_t if_idx;                                                (8)
};
```

代码清单 6-1**(1)**：next 是一个 pbuf 类型的指针，指向下一个 pbuf，因为网络中的数据包可能很大，而 pbuf 能管理的数据包大小有限，就会采用链表的形式将所有的 pbuf 包连接起来，这样子才能

完整描述一个数据包，这些连接起来的 pbuf 包会组成一个链表，我称之为 pbuf 链表。

代码清单 6-1(**2**)：payload 是一个指向数据区域的指针，指向该 pbuf 管理的数据区域起始地址，这里的数据区域可以是紧跟在 pbuf 结构体地址后面的 RAM 空间，也可以是 ROM 中的某个地址上，取决于 pbuf 的类型。

代码清单 6-1(**3**)：tot_len 中记录的是当前 pbuf 及其后续 pbuf 所有数据的长度，例如如果当前 pbuf 是 pbuf 链表上第一个数据结构，那么 tot_len 就记录着整个 pbuf 链表中所有 pbuf 中数据的长度；如果当前 pbuf 是链表上最后一个数据结构，那就记录着当前 pbuf 的长度。

代码清单 6-1(**4**)：len 表示当前 pbuf 中有效的数据长度。

代码清单 6-1(**5**)：type_internal 表示 pbuf 的类型，LwIP 中有 4 种 pbuf 的类型，并且使用了一个枚举类型的数据结构定义他们，具体见代码清单 6-2。

代码清单 6-1(**6**)：flags 字段在初始化的时候一般被初始化为 0，此处就不对 flags 字段进行过多讲解。

代码清单 6-1(**7**)：ref 表示该 pbuf 被引用的次数，引用表示有其他指针指向当前 pbuf，这里的指针可以是 pbuf 的 next 指针，也可以是其他任意形式的指针，初始化一个 pbuf 的时候，ref 会被设置为 1，因为该 pbuf 的地址一点会被返回一个指针变量，当有其他指针指向 pbuf 的时候，就必须调用相关函数将 ref 字段加 1。

代码清单 6-1(**8**)：if_idx 用于记录传入的数据包中输入 netif 的索引，也就是 netif 中 num 字段。

6.4 pbuf 的类型

代码清单 6-2pbuf 的类型

```
typedef enum
{
    PBUF_RAM = (PBUF_ALLOC_FLAG_DATA_CONTIGUOUS |
                PBUF_TYPE_FLAG_STRUCT_DATA_CONTIGUOUS |
                PBUF_TYPE_ALLOC_SRC_MASK_STD_HEAP),
```

(下页继续)

(续上页)

```
    PBUF_ROM = PBUF_TYPE_ALLOC_SRC_MASK_STD_MEMP_PBUF,

    PBUF_REF = (PBUF_TYPE_FLAG_DATA_VOLATILE |
                PBUF_TYPE_ALLOC_SRC_MASK_STD_MEMP_PBUF),

    PBUF_POOL = (PBUF_ALLOC_FLAG_RX |
                 PBUF_TYPE_FLAG_STRUCT_DATA_CONTIGUOUS |
                 PBUF_TYPE_ALLOC_SRC_MASK_STD_MEMP_PBUF_POOL)

} pbuf_type;
```

pbuf 的类型有 4 种，分别为 PBUF_RAM、PBUF_POOL 、PBUF_ROM、PBUF_REF。

6.4.1 PBUF_RAM 类型的 pbuf

PBUF_RAM 类型的 pbuf 空间是通过内存堆分配而来的，这种类型的 pbuf 在协议栈中使用得最多，一般协议栈中要发送的数据都是采用这种形式，在申请这种 pbuf 内存块的时候，协议栈会在管理的内存堆中根据需要的大小进行分配对应的内存空间，这种 pbuf 内存块包含数据空间以及 pbuf 数据结构区域，在连续的 RAM 内存空间中。很多人又会有疑问了，不是说各个协议层都有首部吗，这些内存空间在哪呢？能想到这一层的读者是非常聪明的，我很欣慰，你们有认真看前面的章节，内核申请这类型的 pbuf 时，也算上了协议首部的空间，当然是根据协议栈不同层次需要的首部进行申请，LwIP 也使用一个枚举类型对不同的协议栈分层需要的首部大小进行定义，关于各层间的首部区域我们在后续讲解，此处只需知道即可。那么申请这种 pbuf 是怎么样申请的呢？具体见*代码清单 6_3*。

代码清单 6-3 申请 PBUF_RAM 类型的 pbuf（伪代码）

```
/* 函数原型 */
struct pbuf *
pbuf_alloc(pbuf_layer layer, u16_t length, pbuf_type type);
```

(下页继续)

(续上页)

```
//例子
struct pbuf *p;

p = pbuf_alloc(PBUF_RAW, (u16_t)(req_len + 1), PBUF_RAM);

p = pbuf_alloc(PBUF_TRANSPORT, 1472, PBUF_RAM);
```

PBUF_RAM 类型的 pbuf 示意图具体见图 *6_1*，图中可以看出整个 pbuf 就是一个连续的内存区域，layer（offset）就是各层协议的首部，如 TCP 报文首部、IP 首部、以太网帧首部等，预留出来的这些空间是为了在各个协议层中灵活地处理这些数据，当然 layer 的大小也可以是 0，具体是多少就与数据包的申请方式有关，具体在后面的章节中讲解。

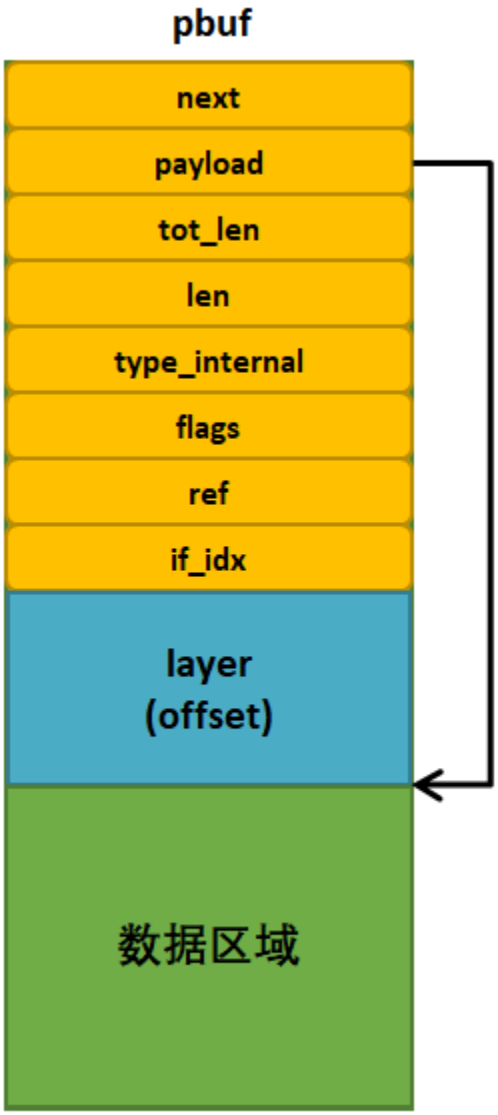


图 6-1PBUF_RAM 类型的 pbuf

6.4.2 PBUF_POOL 类型的 pbuf

PBUF_POOL 类型的 pbuf 与 PBUF_RAM 类型的 pbuf 都是差不多的，其 pbuf 结构体与数据缓冲区也是存在于连续的内存块中，但它的空间是通过内存池分配的，这种类型的 pbuf 可以在极短的时间内分配得到，因为这是内存池分配策略的优势，在网卡接收数据的时候，LwIP 一般就使用这种类型的 pbuf 来存储接收到的数据，申请 PBUF_POOL 类型时，协议栈会在内存池中分配适当的内存池个数以满足需要的数据区域大小。

除此之外，在系统进行内存池初始化的时候，还好初始化两个与 pbuf 相关的内存池，分别为 MEMP_PBUF、MEMP_ PBUF_POOL，具体见*代码清单 6_4*。

代码清单 6-4 与 pbuf 相关的内存池

```
LWIP_MEMPOOL(PBUF, MEMP_NUM_PBUF, sizeof(struct pbuf),"PBUF_REF/ROM")
LWIP_PBUF_MEMPOOL(PBUF_POOL,PBUF_POOL_SIZE,PBUF_POOL_BUFSIZE,"PBUF_POOL")
```

MEMP_PBUF 内存池是专门用于存放 pbuf 数据结构的内存池，主要用于 PBUF_ROM、PBUF_REF 类型的 pbuf，其大小为 sizeof(struct pbuf)，内存块的数量为 MEMP_NUM_PBUF；而 MEMP_PBUF_POOL 则包含 pbuf 结构与数据区域，也就是 PBUF_POOL 类型的 pbuf，内存块的大小为 PBUF_POOL_BUFSIZE，其值由用户自己定义，默认为 590（536+40+0+14）字节，当然也可以由我们定义 TCP_MSS 的大小改变该宏定义，我们将宏定义 TCP_MSS 的值定义为 1460，这样子我们 PBUF_POOL 类型的 pbuf 的内存池大小为 1514（1460+40+0+14），内存块的个数为 PBUF_POOL_SIZE。

如果按照默认的内存大小，对于有些很大的以太网数据包，可能就需要多个 pbuf 才能将这些数据存放下来，这就需要申请多个 pbuf，因为是 PBUF_POOL 类型的 pbuf，所以申请内存空间只需要调用 memp_malloc() 函数进行申请即可。然后再将这些 pbuf 通过链表的形式连接起组成 pbuf 链表上，以保证用户的空间需求，分配与连接成功的 pbuf 示意图具体见图 *6_2*。

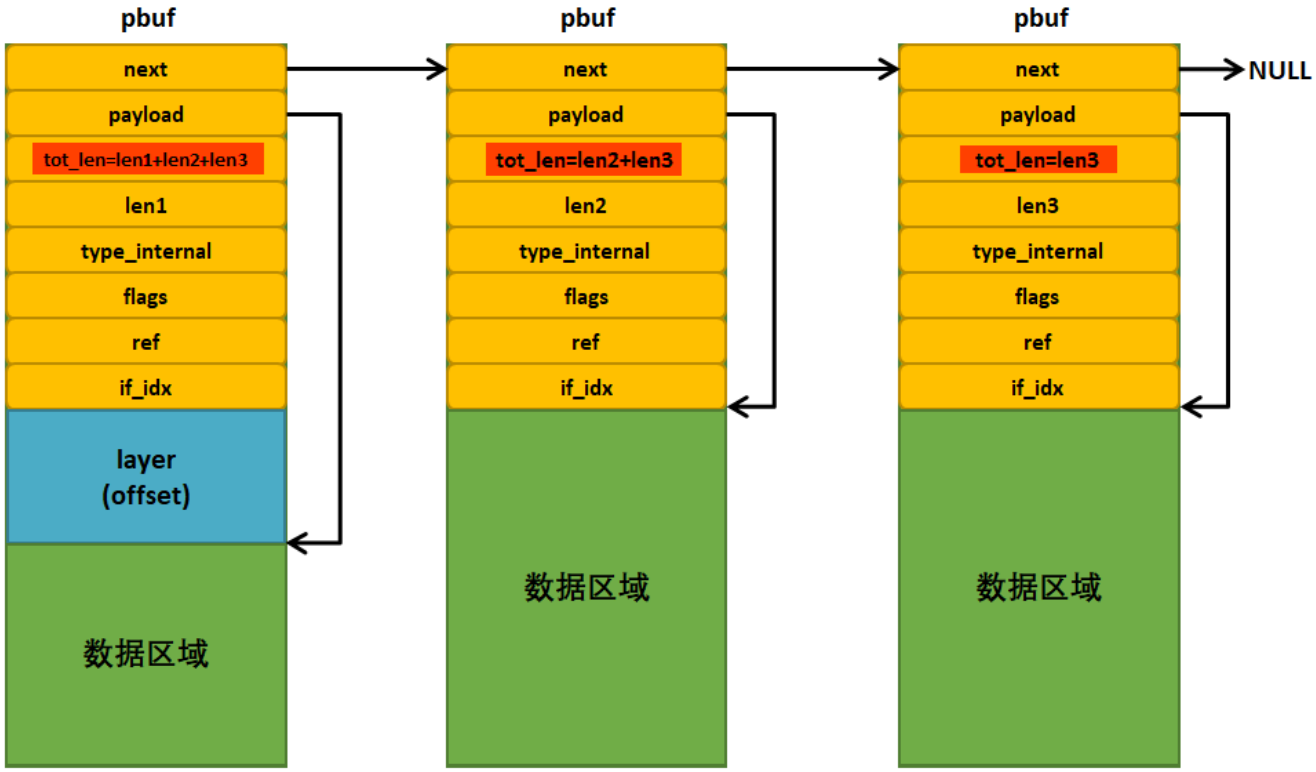


图 6-2 PBUF_POOL 类型 pbuf（组成 pbuf 链表）

注意了，pbuf 链表中第一个 pbuf 是有 layer 字段的，用于存放协议头部，而在它后面的 pbuf 则是没有该字段，由于 PBUF_POOL 类型 pbuf 都是以固定长度分配的，在最后一个 pbuf 中，可能会被浪费大量的空间，并且，每个 pbuf 的 tot_len 字段记录的就是自身及其后面的 pbuf 总大小。

6.4.3 PBUF_ROM 和 PBUF_REF 类型 pbuf

PBUF_ROM 和 PBUF_REF 类型的 pbuf 基本是一样的，它们在内存池申请的 pbuf 不包含数据区域，只包含 pbuf 结构体，即 MEMP_PBUF 类型的 POOL，这也是 PBUF_ROM 和 PBUF_REF 与前面两种类型的 pbuf 最大的差别。

PBUF_ROM 类型的 pbuf 的数据区域存储在 ROM 中，是一段静态数据，而 PBUF_REF 类型的 pbuf 的数据区域存储在 RAM 空间中。申请这两种类型的 pbuf 时候也是只需要调用 memp_malloc() 函数从内存池中申请即可，申请内存的大小就是 MEMP_PBUF，它只是一个 pbuf 结构体大小，正确分配到的 pbuf 内存块示意图具体见图 *6_3*。

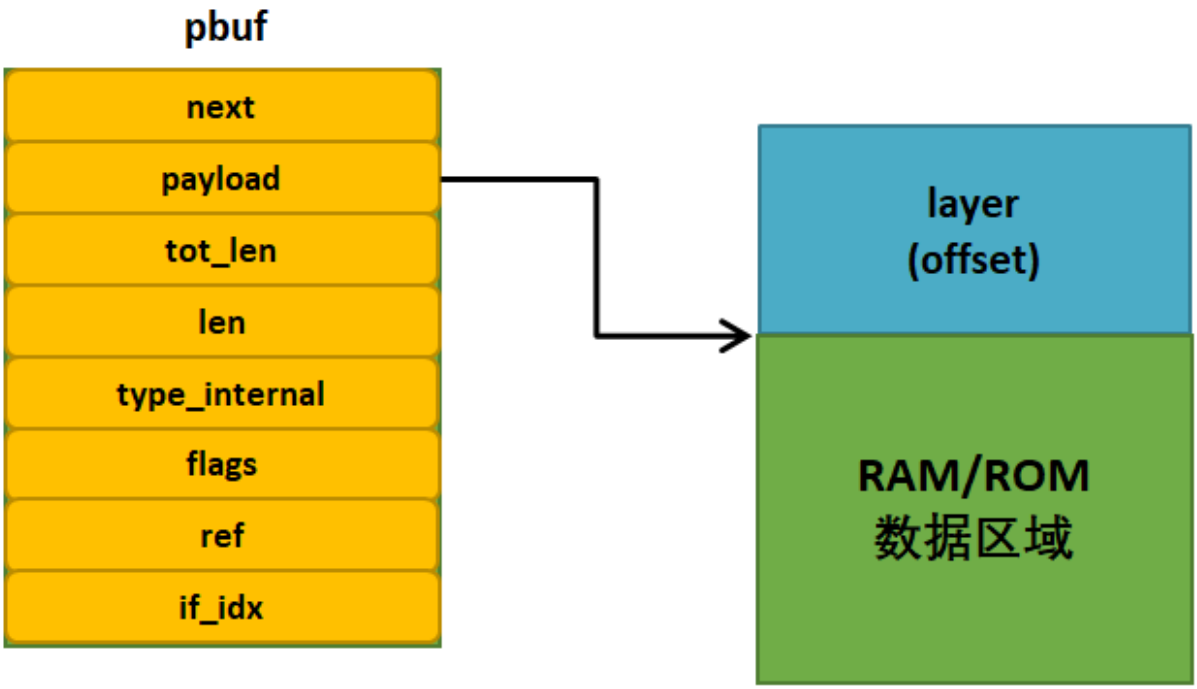


图 6-3 PBUF_ROM/PBUF_REF 类型 pbuf

最后，作者想要提醒一下大家，对于一个数据包，它可能会使用任意类型的 pbuf 进行描述，也可能使用多种不同的 pbuf 一起描述，如图 *6_4* 所示，就是采用多种 pbuf 描述一个数据包，但是无论怎么样描述，数据包的处理都是不变的，payload 指向的始终是数据区域，采用链表的形式连接起来的数据包，其 tot_len 字段永远是记录当前及其后续 pbuf 的总大小。

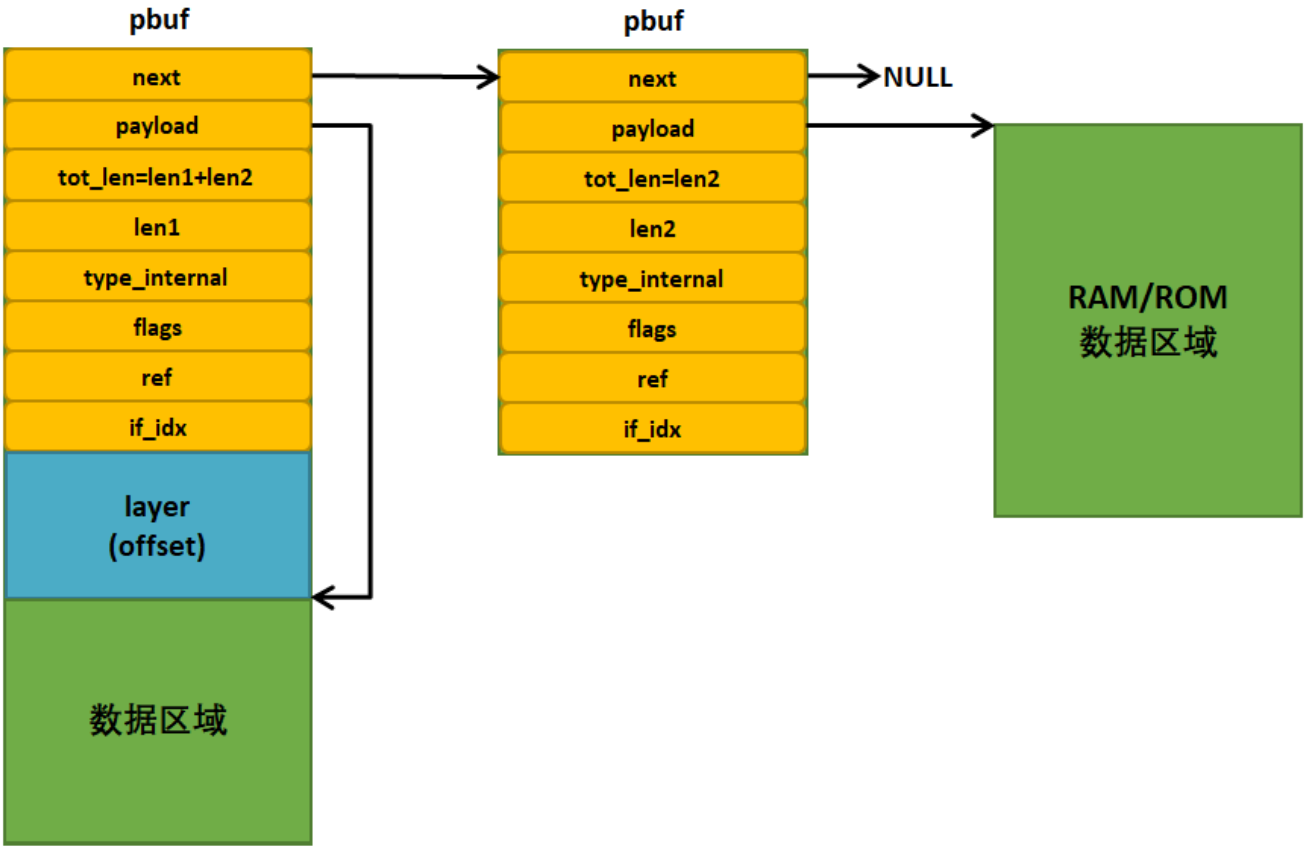


图 6-4 不同类型的 pbuf 组成 pbuf 链表

6.5 pbuf_alloc()

数据包申请函数 pbuf_alloc() 在系统中的许多地方都会用到，例如在网卡接收数据时，需要申请一个数据包，然后将网卡中的数据填入数据包中；在发送数据的时候，协议栈会申请一个 pbuf 数据包，并将即将发送的数据装入到 pbuf 中的数据区域，同时相关的协议首部信息也会被填入到 pbuf 中的 layer 区域内，所以 pbuf 数据包的申请函数几乎无处不在，存在协议栈于各层之中，当然，在不同层的协议中，layer 字段的大小是不一样的，因为不一样的协议其首部大小是不同的，这个知识点会在后文讲解各协议的时候讲解，此处只需了解一下即可。协议栈中各层首部的大小都会被预留出来，LwIP 采用枚举类型的变量将各个层的首部大小记录下来，在申请的时候就把 layer 需要空间的大小根据协议进行分配，具体见*代码清单 6_5*。

代码清单 6-5 各层协议首部大小

```
#define PBUF_TRANSPORT_HLEN 20
#define PBUF_IP_HLEN        20

typedef enum
{
    PBUF_TRANSPORT = PBUF_LINK_ENCAPSULATION_HLEN +
            PBUF_LINK_HLEN + PBUF_IP_HLEN + PBUF_TRANSPORT_HLEN,(1)

    PBUF_IP = PBUF_LINK_ENCAPSULATION_HLEN +
            PBUF_LINK_HLEN + PBUF_IP_HLEN,                    (2)

    PBUF_LINK = PBUF_LINK_ENCAPSULATION_HLEN + PBUF_LINK_HLEN,       (3)

    PBUF_RAW_TX = PBUF_LINK_ENCAPSULATION_HLEN,                      (4)

    PBUF_RAW = 0                                                     (5)
} pbuf_layer;
```

代码清单 6-5**(1)**：传输层协议首部内存空间，如 UDP、TCP 报文协议首部。

代码清单 6-5**(2)**：网络层协议首部内存空间，如 IP 协议。

代码清单 6-5**(3)**：链路层协议首部内存空间，如以太网。

代码清单 6-5**(4)(5)**：原始层，不预留空间，PBUF_LINK_ENCAPSULATION_HLEN 宏定义默认为 0。

数据包申请函数有两个重要的参数：数据包 pbuf 的类型和数据包在哪一层被申请。数据包类型就是我们之前讲的那四种，数据包在哪一层申请这个参数主要是为了预留各层协议的内存大小，也就是前面所说的 layer 值，当数据包申请时，所处的层次不同，就会导致预留空间的的 layer 值不同。

pbuf 分配函数 pbuf_alloc() 的实现具体见*代码清单 6_6*。

代码清单 6-6 pbuf_alloc() 源码

```
struct pbuf *
pbuf_alloc(pbuf_layer layer, u16_t length, pbuf_type type)
{
    struct pbuf *p;
    u16_t offset = (u16_t)layer;

    switch (type)
    {
    case PBUF_REF: /* fall through */
    case PBUF_ROM:
        p = pbuf_alloc_reference(NULL, length, type);                (1)
        break;
    case PBUF_POOL:                                                  (2)
    {
        struct pbuf *q, *last;
        u16_t rem_len; /* remaining length */
        p = NULL;
        last = NULL;
        rem_len = length;
        do
        {
            u16_t qlen;
            q = (struct pbuf *)memp_malloc(MEMP_PBUF_POOL);          (3)
            if (q == NULL)                                           (4)
            {
                PBUF_POOL_IS_EMPTY();
                /* free chain so far allocated */
                if (p)
                {
                    pbuf_free(p);                                    (5)
                }
                /* bail out unsuccessfully */
```

(下页继续)

(续上页)

```
            return NULL;
        }
        qlen = LWIP_MIN(rem_len,(u16_t)(PBUF_POOL_BUFSIZE_ALIGNED -
                        LWIP_MEM_ALIGN_SIZE(offset)));   (6)
        pbuf_init_alloced_pbuf(q, LWIP_MEM_ALIGN((void *)
                    ((u8_t *)q + SIZEOF_STRUCT_PBUF + offset)),
                        rem_len, qlen, type, 0);                 (7)

        if (p == NULL)
        {
            /* allocated head of pbuf chain (into p) */
            p = q;
        }
        else
        {
            /* make previous pbuf point to this pbuf */
            last->next = q;                                      (8)
        }
        last = q;
        rem_len = (u16_t)(rem_len - qlen);                       (9)
        offset = 0;
    }
    while (rem_len > 0);                                         (10)
    break;
}
case PBUF_RAM:                                                   (11)
{
    u16_t payload_len = (u16_t)(LWIP_MEM_ALIGN_SIZE(offset) +
                        LWIP_MEM_ALIGN_SIZE(length));
    mem_size_t alloc_len = (mem_size_t)
        (LWIP_MEM_ALIGN_SIZE(SIZEOF_STRUCT_PBUF) + payload_len); (12)
```

(下页继续)

(续上页)

```
        if ((payload_len < LWIP_MEM_ALIGN_SIZE(length)) ||
                (alloc_len < LWIP_MEM_ALIGN_SIZE(length)))
        {
            return NULL;
        }

        /* If pbuf is to be allocated in RAM, allocate memory for it. */
        p = (struct pbuf *)mem_malloc(alloc_len);                    (13)
        if (p == NULL)
        {
            return NULL;
        }
        pbuf_init_alloced_pbuf(p, LWIP_MEM_ALIGN((void *)
                               ((u8_t *)p + SIZEOF_STRUCT_PBUF + offset)),
                               length, length, type, 0);             (14)

        break;
    }
    default:
        return NULL;                                                 (15)
    }
    return p;
}
```

代码清单 6-6(**1**)：根据具体的 pbuf 类型进行分配，对于 PBUF_ROM 与 PBUF_REF 类型的 pbuf，只分配 pbuf 结构体空间大小。

代码清单 6-6(**2**)：对于 PBUF_POOL 这种类型的 pbuf，可能需要进行分配几个内存块才能描述一个数据包。

代码清单 6-6(**3**)：调用 memp_malloc(MEMP_PBUF_POOL) 分配内存块吗，内存块类型为 MEMP_PBUF_POOL。

代码清单 6-6(**4**)：分配失败，可能内存块已经用完。

代码清单 6-6**(5)**：如果前面分配内存块成功，但是这次分配失败，无法描述一个完整的数据包，则将之前分配的内存块都释放掉。

代码清单 6-6**(6)**：分配成功，得到实际数据区域长度。

代码清单 6-6**(7)**：初始化 pbuf 结构体的成员变量。

代码清单 6-6**(8)**：将这些 pbuf 连接成 pbuf 链表。

代码清单 6-6**(9)**：计算存下所有数据需要的长度。

代码清单 6-6**(10)**：继续分配内存块，直到将所有的数据装下为止

代码清单 6-6**(11)**：对于 PBUF_RAM 这种类型的 pbuf，内核将从内存堆中申请 pbuf。

代码清单 6-6**(12)**：计算要申请的内存大小。

代码清单 6-6**(13)**：调用 mem_malloc() 函数申请内存。

代码清单 6-6**(14)**：初始化 pbuf 结构体的成员变量。

代码清单 6-6**(15)**：类型超出预期，直接返回。

pbuf_alloc() 函数的思路很清晰，根据传入的 pbuf 类型及协议层次 layer，去申请对应的 pbuf，就能预留出对应的协议首部空间，对于 PBUF_ROM 与 PBUF_REF 类型的 pbuf，内核不会申请数据区域，因此，pbuf 结构体中 payload 指针就需要用户自己去设置，我们通常在申请 PBUF_ROM 与 PBUF_REF 类型的 pbuf 成功后，紧接着就将 payload 指针指向某个数据区域。

举个例子，假设 TCP 协议需要申请一个 pbuf 数据包，那么就会调用下面代码进行申请：

代码清单 6-7 TCP 协议申请数据包（伪代码）

```
p = pbuf_alloc(PBUF_TRANSPORT, 1472, PBUF_RAM);
```

内核就会根据这句代码进行分配一个 PBUF_RAM 类型的 pbuf，其数据区域大小是 1472 字节，并且会根据协议层次进行预留协议首部空间，由于是传输层，所以内核需要预留 54 个字节空间，即以太网帧首部长度 PBUF_LINK_HLEN（14 字节）、IP 数据报首部长度 PBUF_IP_HLEN（20 字节）、TCP 首部长度 PBUF_TRANSPORT_HLEN（20 字节）。当数据报往下层递交的时候，其他层直接填充对应的协议首部即可，无需对数据进行拷贝等操作，这也是 LwIP 能快速处理的优势。

6.6 pbuf_free()

数据包 pbuf 的释放是必须的，因为当内核处理完数据就要将这些资源进行回收，否则就会造成内存泄漏，在后续的数据处理中无法再次申请内存。当底层将数据发送出去后或者当应用层将数据处理完毕的时候，数据包就要被释放掉。

当然，既然要释放数据包，那么肯定有条件，pbuf 中 ref 字段就是记录 pbuf 数据包被引用的次数，在申请 pbuf 的时候，ref 字段就被初始化为 1，当释放 pbuf 的时候，先将 ref 减 1，如果 ref 减 1 后为 0，则表示能释放 pbuf 数据包，此外，能被内核释放的 pbuf 数据包只能是首节点或者其他地方未被引用过的节点，如果用户错误地调用 pbuf 释放函数，将 pbuf 链表中的某个中间节点删除了，那么必然会导致错误。

前面我们也说了，一个数据包可能会使用链表的形式将多个 pbuf 连接起来，那么假如删除一个首节点，怎么保证删除完属于一个数据包的数据呢？很简单，LwIP 的数据包释放函数会自动删除属于一个数据包中连同首节点在内所有 pbuf，举个例子，假设一个数据包需要 3 个 pbuf 连接起来，那么在删除第一个 pbuf 的时候，内核会检测一下它下一个 pbuf 释放与首节点是否存储同一个数据包的数据，如果是那就将第二个节点也删除掉，同理第三个也会被删除。但如果删除某个 pbuf 链表的首节点时，链表中第二个节点的 pbuf 中 ref 字段不为 0，则表示该节点还在其他地方被引用，那么第二个节点不与第一个节点存储同一个数据包，那么就不会删除第二个节点。

下面用示意图来解释一下删除的过程，假设有 4 个 pbuf 链表，链表中每个 pbuf 的 ref 都有一个值，具体见图 *6_5*，当调用 pbuf_free() 删除第一个节点的时候，剩下的 pbuf 变化情况，具体见。

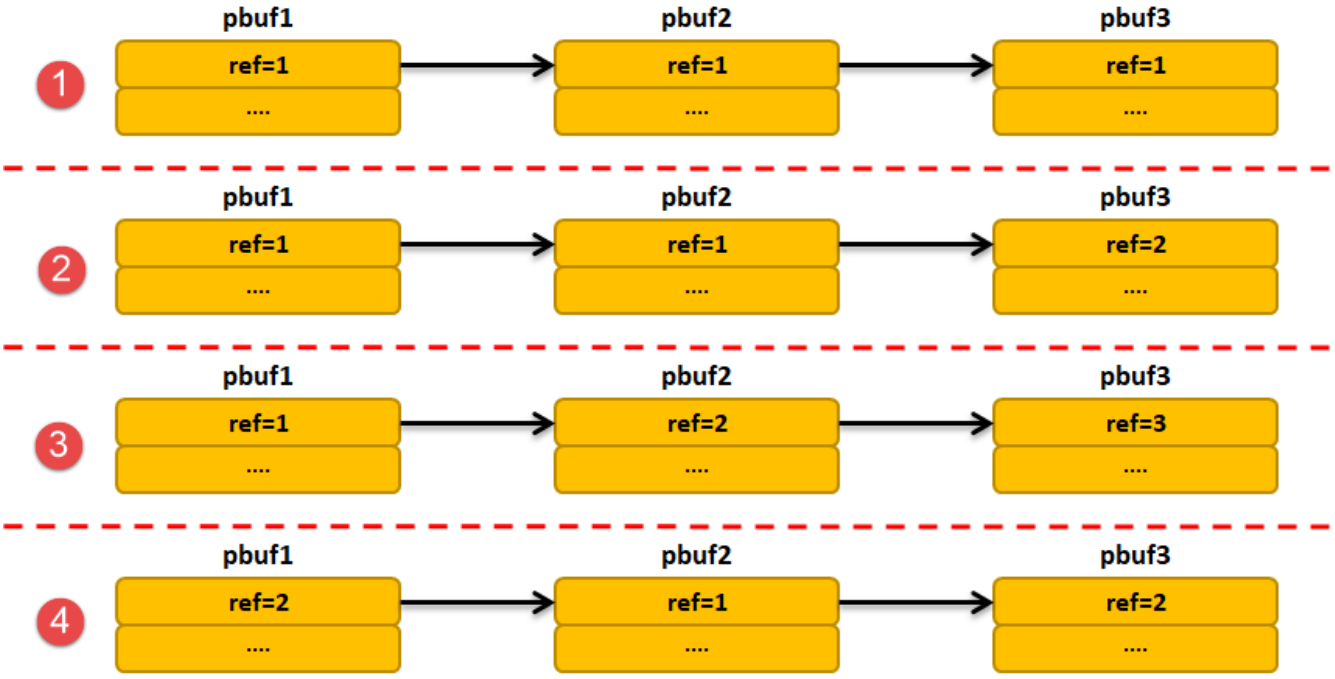


图 6-5 开始时 4 个 pbuf 链表

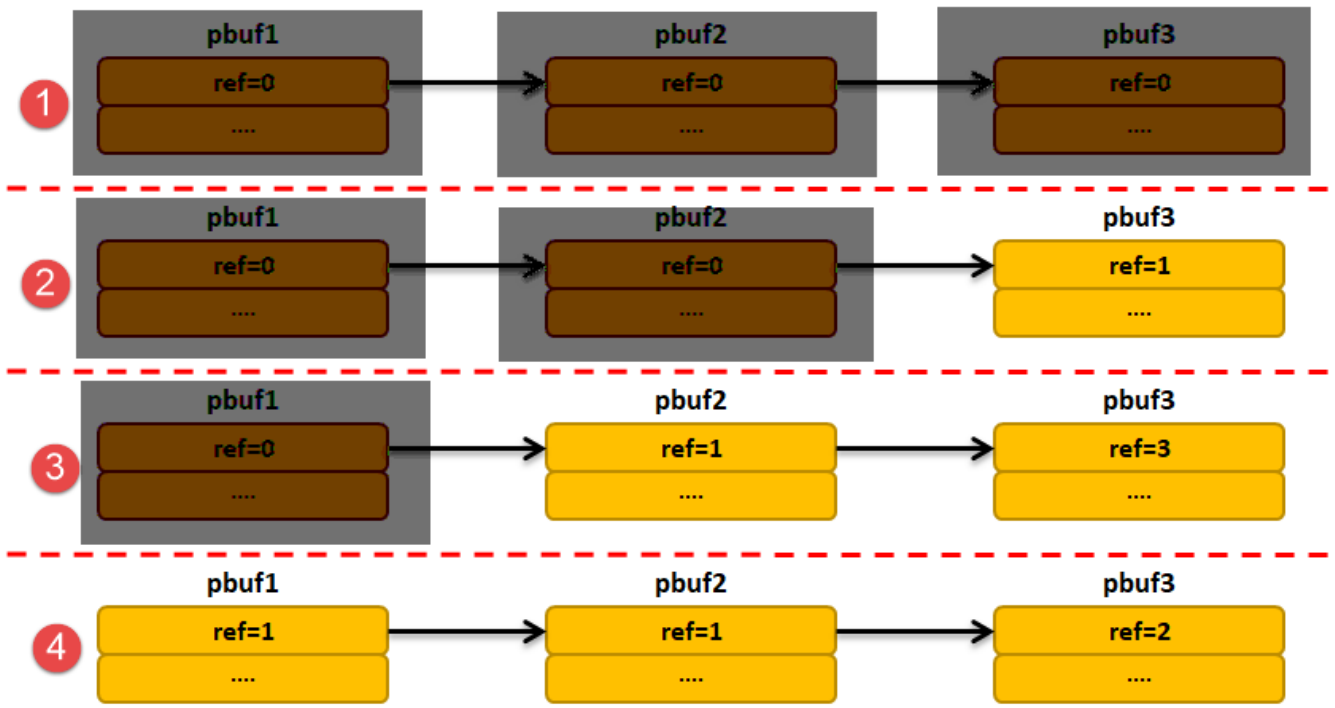


图 6-6 删除第一个节点后的 pbuf’ 链表

从这两张图中我们也看到了，当删除第一个节点后，如果后续的 pbuf 的 ref 为 1（即与第一个节点存储同一个数据包），那么该节点也会被删除。第一个 pbuf 链表在删除首节点之后就不存在节点；第二个 pbuf 链表在删除首节点后只存在 pbuf3；第三个 pbuf 链表在删除首节点后还存在 pbuf2 与 pbuf3；第四个链表还不能删除首节点，因为该数据包还在其他地方被引用了。

pbuf_free() 函数源码具体见*代码清单 6_8*。

代码清单 6-8pbuf_free() 函数

```
u8_t
pbuf_free(struct pbuf *p)
{
    u8_t alloc_src;
    struct pbuf *q;
    u8_t count;

    if (p == NULL)
    {
        return 0;                                          (1)
    }

    PERF_START;

    count = 0;

    while (p != NULL)
    {
        LWIP_PBUF_REF_T ref;

        SYS_ARCH_DECL_PROTECT(old_level);

        SYS_ARCH_PROTECT(old_level);
```

(下页继续)

(续上页)

```
    ref = --(p->ref);                                   (2)
    SYS_ARCH_UNPROTECT(old_level);
    /* this pbuf is no longer referenced to? */
    if (ref == 0)
    {
        /* remember next pbuf in chain for next iteration */
        q = p->next;                                    (3)

        alloc_src = pbuf_get_allocsrc(p);                       (4)
        /* is this a pbuf from the pool? */
        if (alloc_src == PBUF_TYPE_ALLOC_SRC_MASK_STD_MEMP_PBUF_POOL)
        {
            memp_free(MEMP_PBUF_POOL, p);
            /* is this a ROM or RAM referencing pbuf? */
        }
        else if (alloc_src == PBUF_TYPE_ALLOC_SRC_MASK_STD_MEMP_PBUF)
        {
            memp_free(MEMP_PBUF, p);
            /* type == PBUF_RAM */
        }
        else if (alloc_src == PBUF_TYPE_ALLOC_SRC_MASK_STD_HEAP)
        {
            mem_free(p);
        }
        else
        {
            /* @todo: support freeing other types */
            LWIP_ASSERT("invalid pbuf type", 0);
        }
        count++;                                                (5)
        /* proceed to next pbuf */
```

(下页继续)

(续上页)

```
            p = q;                                          (6)
            /* p->ref > 0, this pbuf is still referenced to */
            /* (and so the remaining pbufs in chain as well) */
        }
        else
        {
            /* stop walking through the chain */
            p = NULL;
        }
    }
    PERF_STOP("pbuf_free");
    /* return number of de-allocated pbufs */
    return count;
}
```

代码清单 6-8(**1**)：如果释放的 pbuf 地址为空，则直接返回。

代码清单 6-8(**2**)：将 pbuf 中 ref 字段减一。

代码清单 6-8(**3**)：若 ref 为 0，表示该 pbuf 被引用次数为 0，则可以删除该 pbuf，用 q 记录下当前 pbuf 的下一个 pbuf。

代码清单 6-8(**4**)：获取当前 pbuf 的类型，根据不一样的类型进行不一样的释放操作，如果是从内存池中申请的 pbuf，则调用 memp_free() 函数进行释放，如 PBUF_POOL、PBUF_ROM 和 PBUF_REF 类型的 pbuf，如果是从内存堆中申请的，就调用 mem_free() 函数进行释放内存，如 PBUF_RAM 类型的 pbuf。

代码清单 6-8(**5**)：记录删除的 pbuf 个数。

代码清单 6-8(**6**)：处理链表中的下一个 pbuf，直到 pbuf 中引用次数不为 0 才退出。

pbuf 的释放要小心，如果 pbuf 是串成链表的话，pbuf 在释放的时候，就会把 pbuf 的 ref 值减 1，然后函数会判断 ref 减完之后是不是变成 0，如果是 0 就会根据 pbuf 的类型调用内存池或者内存堆回收函数进行回收。然后这里就有个很危险的事，对于这个 pbuf_free() 函数，用户传递的参数必须是链表头指针，假如不是链表头而是指向链表中间的某个 pbuf 的指针，那就很容易出现问

题，因为这个 pbuf_free() 函数可不会帮我们检查是不是链表头，这样子势必会导致一部分 pbuf 没被回收，意味着一部分内存池就这样被泄漏了，以后没办法用了。同时，还可能将一些尚未处理的数据回收了，这样子整个系统就乱套了。

6.7 其它 pbuf 操作函数

6.7.1 pbuf_realloc()

pbuf_realloc() 函数在相应 pbuf(链表）尾部释放一定的空间，将数据包 pbuf 中的数据长度减少为某个长度值。对于 PBUF_RAM 类型的 pbuf，函数将调用内存堆管理中介绍到的 mem_realloc() 函数，释放这些多余的空间。对于其他三种类型的 pbuf，该函数只是修改 pbuf 中的长度字段值，并不释放对应的内存池空间。

6.7.2 pbuf_header()

pbuf_header() 函数用于调整 pbuf 的 payload 指针（向前或向后移动一定字节数），在前面也说到过，在 pbuf 的数据区前可能会预留一些协议首部空间，而 pbuf 被创建时，payload 指针是指向数据区的，为了实现对这些预留空间的操作，可以调用 pbuf_header() 函数使 payload 指针指向数据区前的首部字段，这就为各层对数据包首部的操作提供了方便。当然，进行这个操作的时候，len 和 tot_len 字段值也会随之更新。

6.7.3 pbuf_take()

pbuf_take() 函数用于向 pbuf 的数据区域拷贝数据。pbuf_copy() 函数用于将一个任何类型的 pbuf 中的数据拷贝到一个 PBUF_RAM 类型的 pbuf 中。pbuf_chain() 函数用于连接两个 pbuf（链表）为一个 pbuf 链表。pbuf_ref 函数用于将 pbuf 中的值加 1。

6.8 网卡中使用的 pbuf

在第 4 章讲解网卡的时候，我们仅仅讲解了网卡初始化相关的内容，但是对于网卡的接收数据与发送数据并未做过多的讲解，现在我们学习了 pbuf 数据包，那么就能编写网卡底层的接收与发送数据相关的代码了。

6.8.1 low_level_output()

网卡发送数据是通过 low_level_output() 函数实现的，该函数是一个底层驱动函数，这要求用户熟悉网卡底层特性，还要熟悉 pbuf 数据包。首先说说发送数据的过程，用户在应用层想要通过一个网卡发送数据，那么就要将数据传入 LwIP 内核中，经过内核的传输层封装、IP 层封装等等，简单来说就是上层将要发送的数据层层封装，存储在 pbuf 数据包中，可能数据很大，想要多个 pbuf 才能存放得下，这时候 pbuf 就以链表的形式存在，当数据发送的时候，就要将属于一个数据包的数据全部发送出去，此处需要注意的是，属于同一个数据包中的所有数据都必须放在同一个以太网帧中发送。low_level_output() 函数的实现具体见代码清单 6_9。

代码清单 6-9 low_level_output() 实现

```
static err_t low_level_output(struct netif *netif, struct pbuf *p)
{
    err_t errval;
    struct pbuf *q;
    uint8_t *buffer = (uint8_t *)(heth.TxDesc->Buffer1Addr);
    __IO ETH_DMADescTypeDef *DmaTxDesc;
    uint32_t framelength = 0;
    uint32_t bufferoffset = 0;
    uint32_t byteslefttocopy = 0;
    uint32_t payloadoffset = 0;
    DmaTxDesc = heth.TxDesc;
    bufferoffset = 0;

    /* copy frame from pbufs to driver buffers */
```

(下页继续)

(续上页)

```
    for (q = p; q != NULL; q = q->next)                                    (1)
    {
        /* Is this buffer available? If not, goto error */
        if ((DmaTxDesc->Status & ETH_DMATXDESC_OWN) != (uint32_t)RESET)
        {
            errval = ERR_USE;
            goto error;                                                    (2)
        }

        /* Get bytes in current lwIP buffer */
        byteslefttocopy = q->len;                                          (3)
        payloadoffset = 0;

        /* Check if the length of data to copy is bigger than Tx buffer␣
↪size*/
        while ( (byteslefttocopy + bufferoffset) > ETH_TX_BUF_SIZE ) (4)
        {
            /* Copy data to Tx buffer*/
            memcpy( (uint8_t*)((uint8_t*)buffer + bufferoffset),
                    (uint8_t*)((uint8_t*)q->payload + payloadoffset),
                    (ETH_TX_BUF_SIZE - bufferoffset) );                    (5)

            /* Point to next descriptor */
            DmaTxDesc = (ETH_DMADescTypeDef *)(DmaTxDesc->
↪Buffer2NextDescAddr); (6)

            /* Check if the buffer is available */
            if((DmaTxDesc->Status & ETH_DMATXDESC_OWN) != (uint32_t)RESET)
            {
                errval = ERR_USE;
                goto error;                                                (7)
            }
```

(下页继续)

(续上页)

```
            buffer = (uint8_t *)(DmaTxDesc->Buffer1Addr);    (8)

            byteslefttocopy = byteslefttocopy - (ETH_TX_BUF_SIZE -␣
↪bufferoffset);
            payloadoffset = payloadoffset + (ETH_TX_BUF_SIZE -␣
↪bufferoffset);
            framelength = framelength + (ETH_TX_BUF_SIZE - bufferoffset);
            bufferoffset = 0;                                          (9)
        }

        /* Copy the remaining bytes */
        memcpy( (uint8_t*)((uint8_t*)buffer + bufferoffset),
                (uint8_t*)((uint8_t*)q->payload + payloadoffset),␣
↪byteslefttocopy );
        bufferoffset = bufferoffset + byteslefttocopy;
        framelength = framelength + byteslefttocopy;                   (10)
    }

    /* Prepare transmit descriptors to give to DMA */
    HAL_ETH_TransmitFrame(&heth, framelength);                         (11)

    errval = ERR_OK;

error:

    /* When Transmit Underflow flag is set, clear it and issue a
    Transmit Poll Demand to resume transmission */
    if ((heth.Instance->DMASR & ETH_DMASR_TUS) != (uint32_t)RESET)
    {
        /* Clear TUS ETHERNET DMA flag */
        heth.Instance->DMASR = ETH_DMASR_TUS;                          (12)
```

(下页继续)

(续上页)

```
        /* Resume DMA transmission*/
        heth.Instance->DMATPDR = 0;                                    (13)
    }
    return errval;
}
```

代码清单 6-9(**1**)：前面也说了，可能一个 pbuf 没法存储所有的数据，LwIP 就会使用链表形式的 pbuf 将所有属于一个数据包的数据存储起来，那么在发送的时候就要变量这个 pbuf 链表，将所有数据都取出来。

代码清单 6-9(**2**)：判断一下要发送数据的缓冲区可用吗？如果不可用，就跳转错误。

代码清单 6-9(**3**)：获取 pbuf 中的数据长度。

代码清单 6-9(**4**)：检查要拷贝的数据长度是否大于 ETH_TX_BUF_SIZE 的大小，当数据长度大于以太网发送缓冲区的时候，就需要分批次拷贝了。

代码清单 6-9(**5**)：将 pbuf 中 payload 指向数据区域的数据拷贝到缓冲区，拷贝的大小就是缓冲区的大小。

代码清单 6-9(**6**)：指向下一个发送的描述符（描述符是 STM32 以太网中的一种数据结构，此处了解一下即可）。

代码清单 6-9(**7**)：检查一下要发送数据的缓冲区可用吗？如果不可用，就跳转错误。

代码清单 6-9(**8**)：得到新的 buff 地址。

代码清单 6-9(**9**)：计算还需要拷贝的长度、得到要偏移的数据地址、记录已经拷贝的长度，重新进行拷贝，直到要拷贝的数据长度小于 ETH_TX_BUF_SIZE 的大小。

代码清单 6-9(**10**)：退出了 while 循环，拷贝剩余的数据，并记录拷贝了多大的数据。

代码清单 6-9(**11**)：提供传输描述符及长度给 DMA 启动发送。

代码清单 6-9(**12**)：当发送错误的时候，清除 TUS ETHERNET DMA 标志位。

代码清单 6-9(**13**)：重新恢复 DMA 传输。

6.8.2 low_level_input()

与 low_level_output() 函数相反的是 low_level_input() 函数，该函数用于从网卡中接收一个数据包，并将数据包封装在 pbuf 中递交给上层。low_level_input() 函数的编写也需要用户熟悉 pbuf 与网卡底层驱动，该函数的实现具体见*代码清单 6_10*。

代码清单 6-10 low_level_input() 实现

```
static struct pbuf * low_level_input(struct netif *netif)
{
    struct pbuf *p = NULL;
    struct pbuf *q = NULL;
    uint16_t len = 0;
    uint8_t *buffer;
    __IO ETH_DMADescTypeDef *dmarxdesc;
    uint32_t bufferoffset = 0;
    uint32_t payloadoffset = 0;
    uint32_t byteslefttocopy = 0;
    uint32_t i=0;

    /* get received frame */
    if (HAL_ETH_GetReceivedFrame(&heth) != HAL_OK)                (1)
    {
        PRINT_ERR("receive frame faild\n");
        return NULL;
    }
/*Obtain the size of the packet and put it into the "len" variable. */
    len = heth.RxFrameInfos.length;
    buffer = (uint8_t *)heth.RxFrameInfos.buffer;                 (2)

    PRINT_INFO("receive frame len : %d\n", len);

    if (len > 0)
```

(下页继续)

(续上页)

```
    {
        /* We allocate a pbuf chain of pbufs from the Lwip buffer pool */
        p = pbuf_alloc(PBUF_RAW, len, PBUF_POOL);                    (3)
    }

    if (p != NULL)
    {
        dmarxdesc = heth.RxFrameInfos.FSRxDesc;
        bufferoffset = 0;
        for (q = p; q != NULL; q = q->next)                          (4)
        {
            byteslefttocopy = q->len;
            payloadoffset = 0;

            /* Check if the length of bytes to copy in
            current pbuf is bigger than Rx buffer size*/
            while ((byteslefttocopy + bufferoffset) > ETH_RX_BUF_SIZE) (5)
            {
                /* Copy data to pbuf */
                memcpy( (uint8_t*)((uint8_t*)q->payload + payloadoffset),
                        (uint8_t*)((uint8_t*)buffer + bufferoffset),
                        (ETH_RX_BUF_SIZE - bufferoffset));    (6)

                /* Point to next descriptor */
                dmarxdesc = (ETH_DMADescTypeDef *)(dmarxdesc->
↪Buffer2NextDescAddr);
            buffer = (uint8_t *)(dmarxdesc->Buffer1Addr);     (7)

                byteslefttocopy = byteslefttocopy -(ETH_RX_BUF_SIZE -␣
↪bufferoffset);
                payloadoffset = payloadoffset + (ETH_RX_BUF_SIZE -␣
↪bufferoffset);
```

(下页继续)

(续上页)

```
                bufferoffset = 0;                                    (8)
            }
            /* Copy remaining data in pbuf */
            memcpy( (uint8_t*)((uint8_t*)q->payload + payloadoffset),
                    (uint8_t*)((uint8_t*)buffer + bufferoffset), ␣
↪byteslefttocopy);
            bufferoffset = bufferoffset + byteslefttocopy;   (9)
        }
    }
    /* Release descriptors to DMA */
    /* Point to first descriptor */
    dmarxdesc = heth.RxFrameInfos.FSRxDesc;                          (10)
    /* Set Own bit in Rx descriptors: gives the buffers back to DMA */
    for (i=0; i< heth.RxFrameInfos.SegCount; i++)
    {
        dmarxdesc->Status |= ETH_DMARXDESC_OWN;                      (11)
        dmarxdesc = (ETH_DMADescTypeDef *)(dmarxdesc->Buffer2NextDescAddr);
    }

    /* Clear Segment_Count */
    heth.RxFrameInfos.SegCount =0;                                   (12)

    /* When Rx Buffer unavailable flag is set: clear it and resume␣
↪reception */
    if ((heth.Instance->DMASR & ETH_DMASR_RBUS) != (uint32_t)RESET)
    {
        /* Clear RBUS ETHERNET DMA flag */
        heth.Instance->DMASR = ETH_DMASR_RBUS;                                (13)
        /* Resume DMA reception */
        heth.Instance->DMARPDR = 0;
    }
    return p;                                                                 (14)
```

(下页继续)

(续上页)

```
}
```

代码清单 6-10**(1)**：看看是否接收到数据，如果没有直接返回 NULL，如果有则将数据获取到 heth 数据结构中。

代码清单 6-10**(2)**：获取接收到数据包大小并将其放入 len 变量中，以及获取数据区域存放在 buffer 指针中。

代码清单 6-10**(3)**：调用 pbuf_alloc() 函数从 LwIP 的内存池中分配 pbuf。

代码清单 6-10**(4)**：如果 pbuf 分配成功，则将内存拷贝到 pbuf 中，因为 pbuf 可能不止一个，就需要遍历 pbuf 链表将所有数据都存储进来。

代码清单 6-10**(5)**：检查一下要拷贝的数据大小于接收缓冲区 ETH_RX_BUF_SIZE 的大小，如果要拷贝的数据大于 ETH_RX_BUF_SIZE 的大小，那么就要分几次拷贝。

代码清单 6-10**(6)**：将接收到的数据拷贝到 pbuf 中 payload 指向的数据区域。

代码清单 6-10**(7)**：指向下一个描述符，得到未拷贝的数据区域地址 buffer。

代码清单 6-10**(8)**：重新计算还需要拷贝的数据大小 byteslefttocopy，pbuf 中数据的偏移量 payloadoffset，重置接收缓冲区偏移量 bufferoffset，然后重复进行拷贝，直到要拷贝的数据小于 ETH_RX_BUF_SIZE 的值，才退出 while 循环。

代码清单 6-10**(9)**：将剩余的数据拷贝到 pbuf 中。

代码清单 6-10**(10)**：dmarxdesc 变量指向第一个接收描述符。

代码清单 6-10**(11)**：进行遍历描述符列表，将描述符中 Status 状态设置为 ETH_DMARXDESC_OWN。

代码清单 6-10**(12)**：清除描述符中 SegCount 变量的值。

代码清单 6-10**(13)**：清除 DMA 标志。

代码清单 6-10**(14)**：重新恢复 DMA 接收。

6.8.3 ethernetif_input()

low_level_output() 函数只是完成了网卡驱动接收，但是还没将 pbuf 数据包递交给上层，那么又是谁将 pbuf 数据包递交给上层的呢？我们在前面讲解 4.3 小节的时候，就知道 ethernetif_input() 函数会被周期性调用（如果是采用操作系统方式，往往会将该函数变成线程形式运行），这样子就能接收网卡的数据，在接收完毕，就能将数据通过网卡 netif 的 input 接口将 pbuf 递交给上层，该函数的实现具体见*代码清单 6_11*。

代码清单 6-11 ethernetif_input() 实现

```
void ethernetif_input(struct netif *netif)
{
    err_t err;
    struct pbuf *p;

    /* move received packet into a new pbuf */
    p = low_level_input(netif);

    /* no packet could be read, silently ignore this */
    if (p == NULL) return;

    /* entry point to the LwIP stack */
    err = netif->input(p, netif);

    if (err != ERR_OK)
    {
        LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input error\n"));
        pbuf_free(p);
        p = NULL;
    }
}
```

第 7 章　无操作系统移植 LwIP

本章开始正式进入 LwIP 移植的学习，在前面的章节中，都是打基础的部分，俗话说“基础不牢，地动山摇”，我们只有在叫了解 LwIP 的时候才开始移植，这样子会更加加深我们的印象。

LwIP 的官方源代码中并没有提供对任何芯片的底层驱动移植程序，因此这部分程序需要由开发者自己完成，在前面的章节，我们也对底层驱动进行编写，网卡的初始化、收发数据等都已经实现了，那么直接拿过来用即可。

7.1 将 LwIP 添加到裸机工程

首先我们将 3.6 小节中得到的裸机（已经写好以太网驱动）工程拿过来，将工程名字改为“LwIP 移植”；然后再将我们已经下载的 LwIP 源码拿过来，准备将 LwIP 源码添加到工程中，但是 LwIP 源码太大了，我们不需要那么多东西，所以我们只需要将 LwIP 源码中的 src 文件文件夹添加进去即可，具体见图 *7_1*，但是我发现 src 表示源码，但是不够明显，无法一眼就知道它是 LwIP 中的源码，那我们直接把文件夹重命名为“LwIP”，这样子就能让别人一目了然知道文件夹是什么内容了，具体见图 *7_2*。

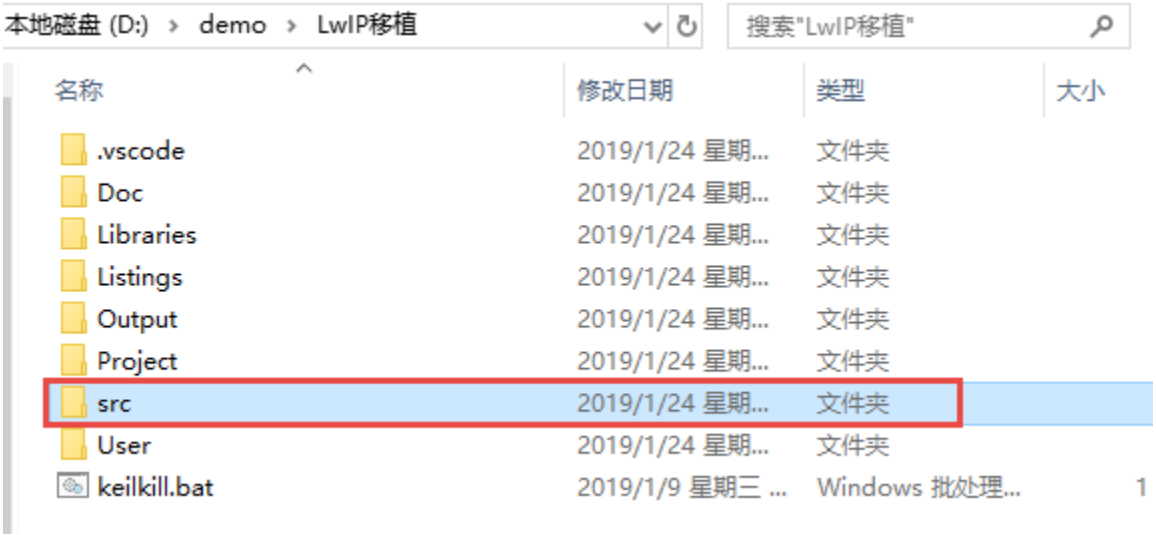


图 7-1 将 LwIP 源码添加到工程中

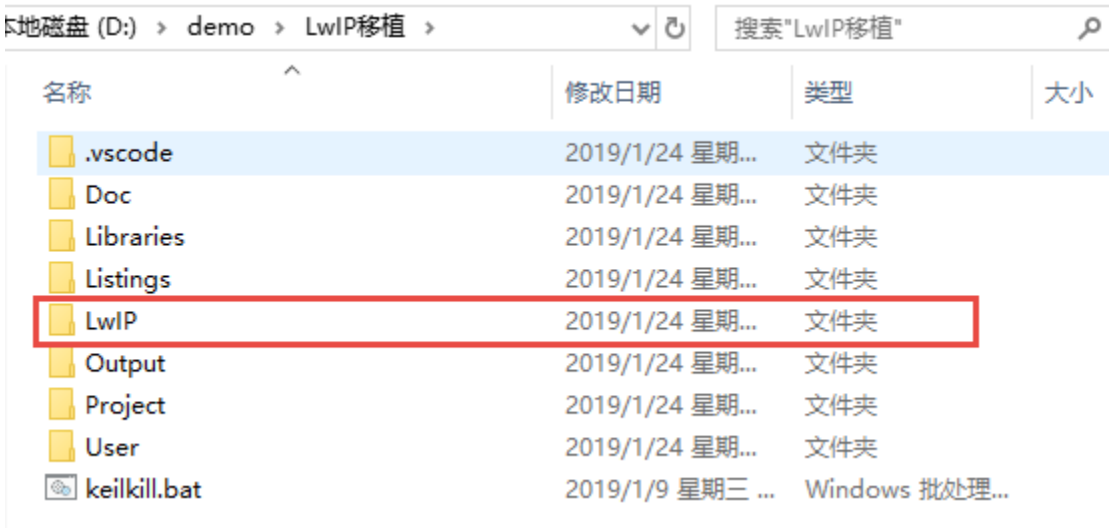


图 7-2 将 src 文件夹重命名为 LwIP

在前面的章节中也讲解了 LwIP 源码文件夹的功能及作用，那么我们现在直接添加到工程分组中即可，首先我们在工程中创建 4 个与 LwIP 相关的工程分组，其中三个分别用于存放 LwIP 源码中的三个文件，其对应关系具体见图 *7_3*。

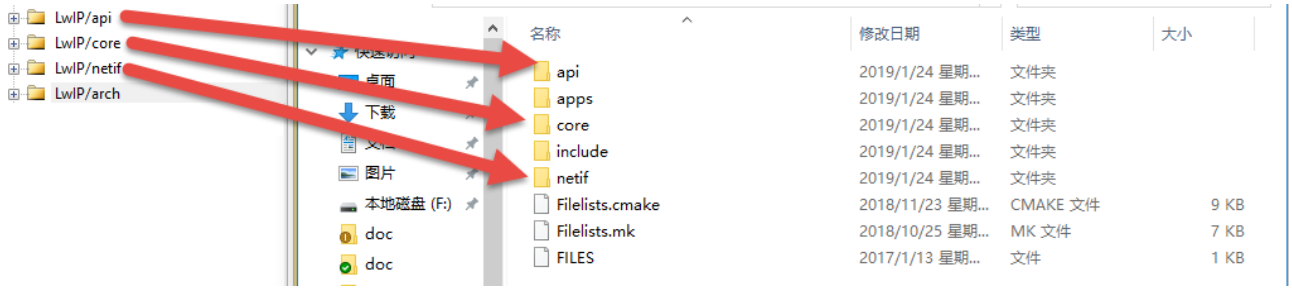


图 7-3 工程中创建分组存放 LwIP 源码

每个分组中都添加对应文件夹下的源码，添加完成后的源码具体见图 7-4、图 7-5、图 7-6 与图 7-7。

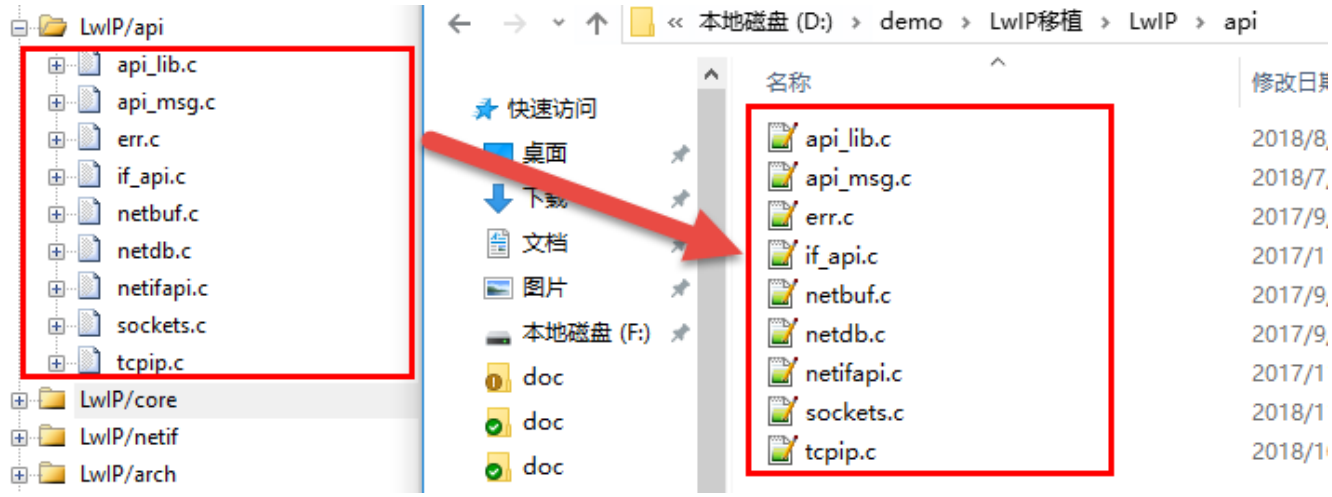

LwIP/api
api_lib.c
api_msg.c
err.c
if_api.c
netbuf.c
netdb.c
netifapi.c
sockets.c
tcpip.c
LwIP/core
LwIP/netif
LwIP/arch
本地磁盘 (D:) › demo › LwIP移植 › LwIP › api
快速访问
桌面
下载
文档
图片
本地磁盘 (F:)
doc
doc
doc
名称
修改日期
api_lib.c 2018/8
api_msg.c 2018/7
err.c 2017/9
if_api.c 2017/1
netbuf.c 2017/9
netdb.c 2017/9
netifapi.c 2017/1
sockets.c 2018/1
tcpip.c 2018/1

图 7-4

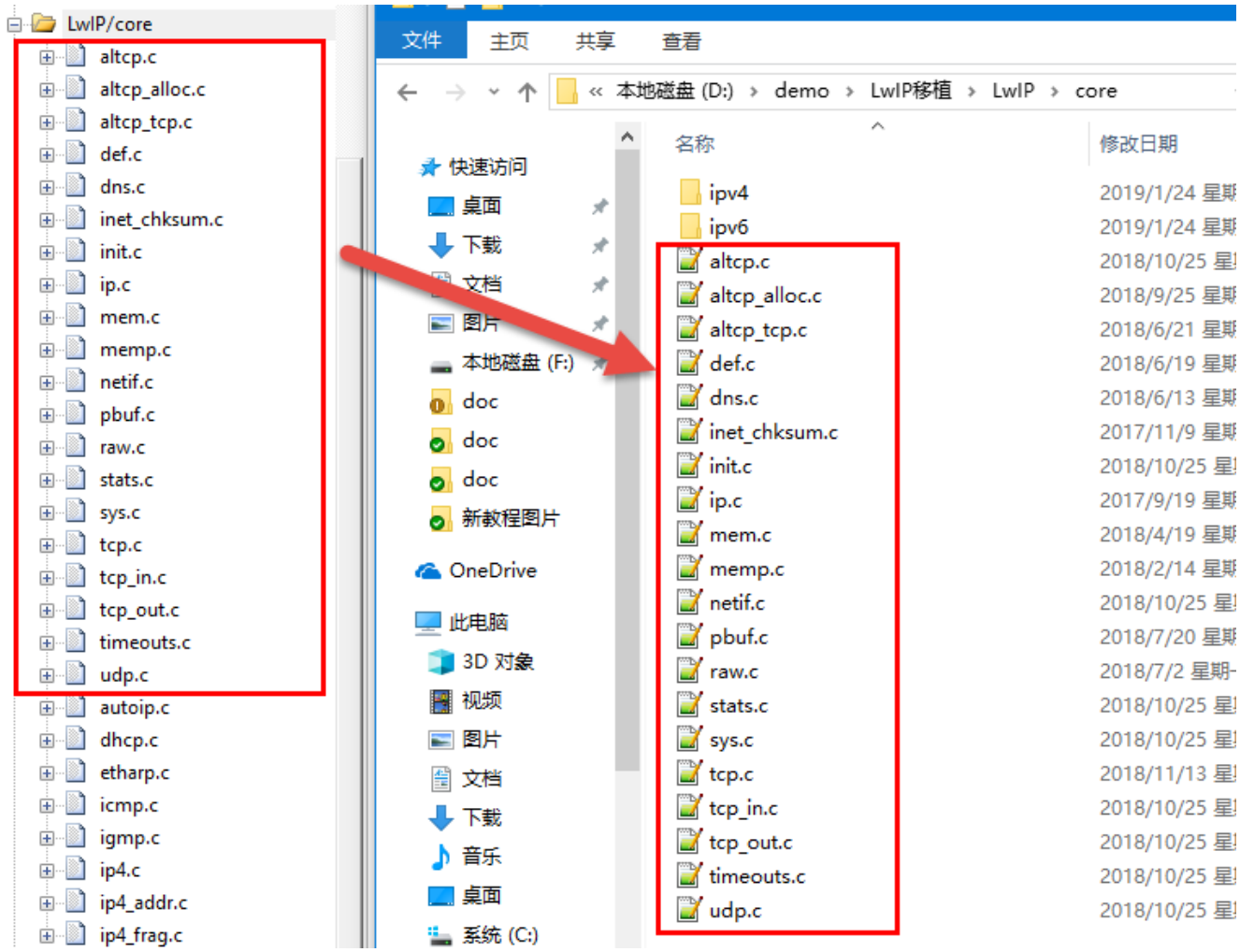

LwIP/core
altcp.c
altcp_alloc.c
altcp_tcp.c
def.c
dns.c
inet_chksum.c
init.c
ip.c
mem.c
memp.c
netif.c
pbuf.c
raw.c
stats.c
sys.c
tcp.c
tcp_in.c
tcp_out.c
timeouts.c
udp.c
autoip.c
dhcp.c
etharp.c
icmp.c
igmp.c
ip4.c
ip4_addr.c
ip4_frag.c
文件
主页
共享
查看
本地磁盘 (D:) › demo › LwIP移植 › LwIP › core
快速访问
桌面
下载
文档
图片
本地磁盘 (F:)
doc
doc
doc
新教程图片
OneDrive
此电脑
3D 对象
视频
图片
文档
下载
音乐
桌面
系统 (C:)
名称
修改日期
ipv4
ipv6
altcp.c
altcp_alloc.c
altcp_tcp.c
def.c
dns.c
inet_chksum.c
init.c
ip.c
mem.c
memp.c
netif.c
pbuf.c
raw.c
stats.c
sys.c
tcp.c
tcp_in.c
tcp_out.c
timeouts.c
udp.c

图 7-5

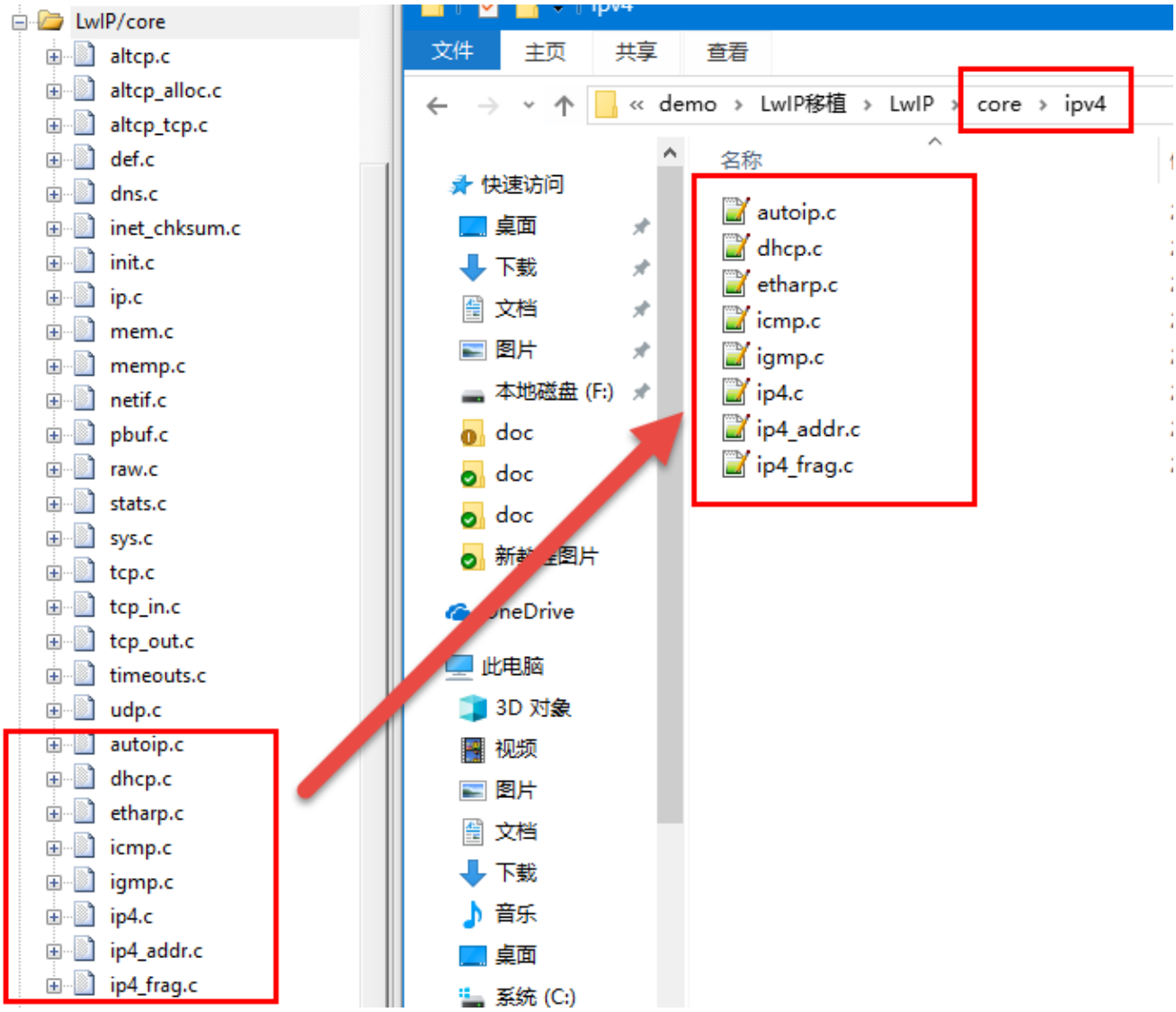

LwIP/core
altcp.c
altcp_alloc.c
altcp_tcp.c
def.c
dns.c
inet_chksum.c
init.c
ip.c
mem.c
memp.c
netif.c
pbuf.c
raw.c
stats.c
sys.c
tcp.c
tcp_in.c
tcp_out.c
timeouts.c
udp.c
autoip.c
dhcp.c
etharp.c
icmp.c
igmp.c
ip4.c
ip4_addr.c
ip4_frag.c
文件
主页
共享
查看
« demo › LwIP移植 › LwIP › core › ipv4
名称
快速访问
桌面
下载
文档
图片
本地磁盘 (F:)
doc
doc
doc
此电脑
3D 对象
视频
图片
文档
下载
音乐
桌面
系统 (C:)
autoip.c
dhcp.c
etharp.c
icmp.c
igmp.c
ip4.c
ip4_addr.c
ip4_frag.c

图 7-6

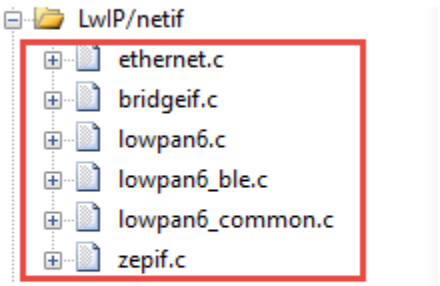


图 7-7

然后在工程中添加 LwIP 头文件路径，并且把 C99 模式也勾选上，添加完成的示意图具体见图 *7_8*。

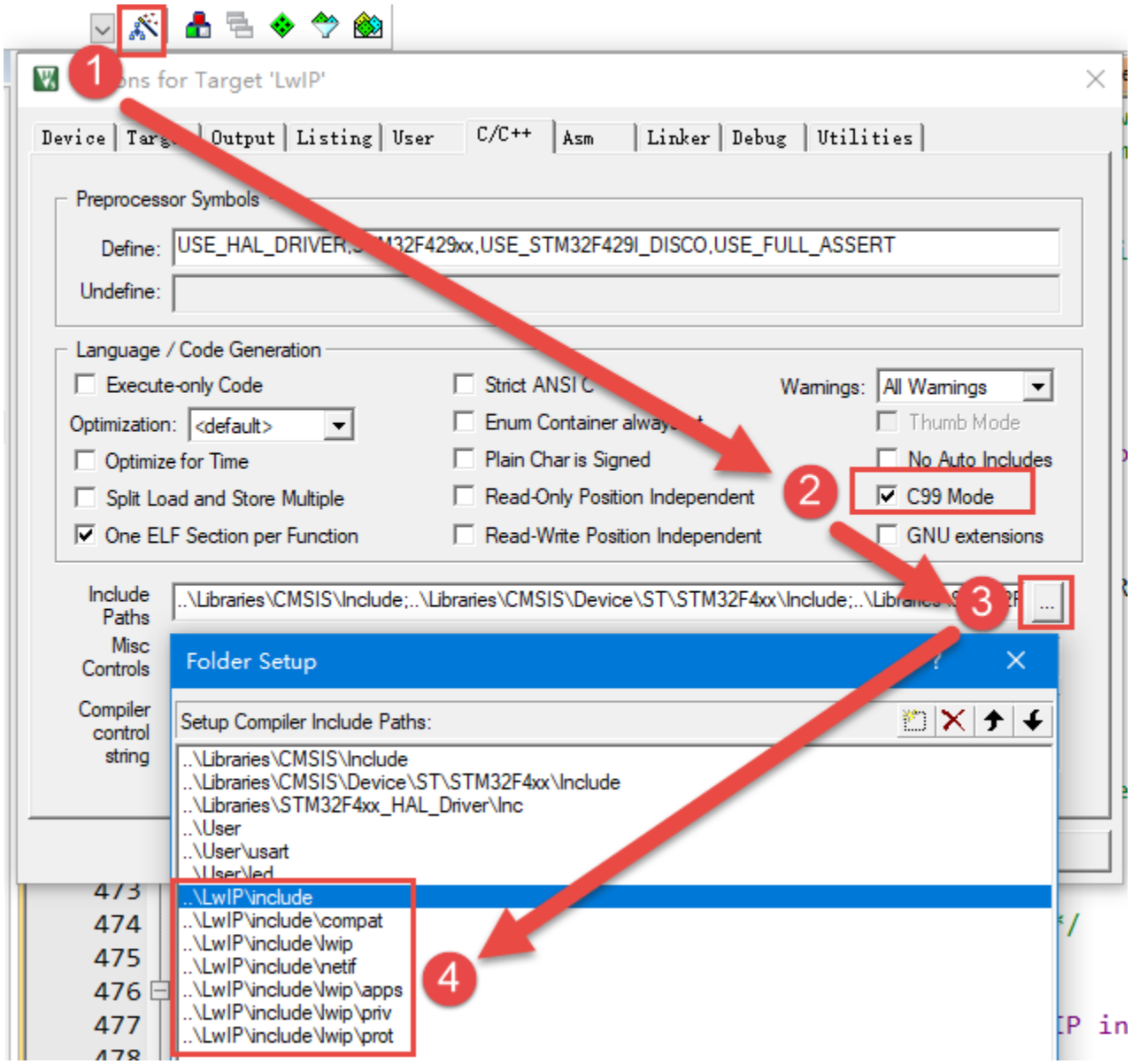


图 7-8 在工程中添加 LwIP 头文件路径

7.2 移植头文件

经过上面的步骤，我们的工程基本就添加完成，但是要想 LwIP 跑起来，还需一些头文件的支持，分别是 lwipopts.h、cc.h、pref.h 等。lwipopts.h 就是用于配置 LwIP 的相关参数的，一般来说 LwIP 默认会有参数的配置，存放在 opt.h 文件中，如果用户没有在 lwipopts.h 文件进行配置，那么 LwIP 就会使用 opt.h 默认的参数，注意，在移植的时候出现定义某些参数是非常重要的，这对我们 LwIP 的性能至关重要，甚至在配置错误的时候能直接导致 LwIP 的运行崩溃，下面一起来看看 lwipopts.h 文件的内容，这个文件的内容是由我们自己添加进来的，这里仅仅是介绍一部分配置。

我们先在工程的 User 文件夹下面新建一个 arch 文件夹，用于存放与底层接口相关的文件，然后打开 LwIP 的 contrib 包，把路径“contrib-2.1.0examplesexample_app”下的 lwipopts.h 文件拷贝到 arch 文件夹中；为了方便，我们把 cc.h 于 pref.h 也添加进来，这两个文件也是存放在 LwIP 的 contrib 包中，路径为“contrib-2.1.0portsunixportincludearch”，我们把这些拷贝到 arch 中，并且将工程的头文件路径添加进去，这样子就能对 LwIP 进行配置了，具体见图 7_9。

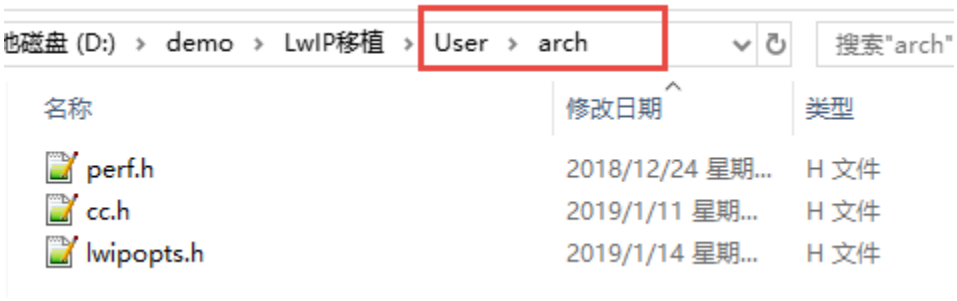


图 7-9 arch 的文件

然后我们打开 lwipopts.h，并在文件内部写入以下配置，具体见代码清单 7_1。

代码清单 7-1lwipopts.h 文件内容

```
#ifndef __LWIPOPTS_H__
#define __LWIPOPTS_H__

/**
* SYS_LIGHTWEIGHT_PROT==1: if you want inter-task protection for certain
```

(下页继续)

(续上页)

```
* critical regions during buffer allocation, deallocation and memory
* allocation and deallocation.
*/
#define SYS_LIGHTWEIGHT_PROT    0

/**
* NO_SYS==1: Provides VERY minimal functionality. Otherwise,
* use lwIP facilities.
*/
#define NO_SYS                  1                   (1)

/**
* NO_SYS_NO_TIMERS==1: Drop support for sys_timeout when NO_SYS==1
* Mainly for compatibility to old versions.
*/
#define NO_SYS_NO_TIMERS        0

/* ---------- Memory options ---------- */
/* MEM_ALIGNMENT: should be set to the alignment of the CPU for which
    lwIP is compiled. 4 byte alignment -> define MEM_ALIGNMENT to 4, 2
    byte alignment -> define MEM_ALIGNMENT to 2. */
#define MEM_ALIGNMENT           4                   (2)

/* MEM_SIZE: the size of the heap memory. If the application will send
a lot of data that needs to be copied, this should be set high. */
#define MEM_SIZE                (30*1024)           (3)

/* MEMP_NUM_PBUF: the number of memp struct pbufs. If the application
    sends a lot of data out of ROM (or other static memory), this
    should be set high. */
#define MEMP_NUM_PBUF           50
/* MEMP_NUM_UDP_PCB: the number of UDP protocol control blocks. One
```

(下页继续)

(续上页)

```
    per active UDP "connection". */
#define MEMP_NUM_UDP_PCB        6
/* MEMP_NUM_TCP_PCB: the number of simulatenously active TCP
    connections. */
#define MEMP_NUM_TCP_PCB        10
/* MEMP_NUM_TCP_PCB_LISTEN: the number of listening TCP
    connections. */
#define MEMP_NUM_TCP_PCB_LISTEN 6
/* MEMP_NUM_TCP_SEG: the number of simultaneously queued TCP
    segments. */
#define MEMP_NUM_TCP_SEG        12
/* MEMP_NUM_SYS_TIMEOUT: the number of simulateously active
    timeouts. */
#define MEMP_NUM_SYS_TIMEOUT    10

/* ---------- Pbuf options ---------- */
/* PBUF_POOL_SIZE: the number of buffers in the pbuf pool. */
#define PBUF_POOL_SIZE          10                  (4)

/* PBUF_POOL_BUFSIZE: the size of each pbuf in the pbuf pool. */
#define PBUF_POOL_BUFSIZE       500                 (5)

/* ---------- TCP options ---------- */
#define LWIP_TCP                1
#define TCP_TTL                 255

/* Controls if TCP should queue segments that arrive out of
    order. Define to 0 if your device is low on memory. */
#define TCP_QUEUE_OOSEQ         0

/* TCP Maximum segment size. */
#define TCP_MSS                 (1500 - 40)                 (6)
```

(下页继续)

(续上页)

```
/* TCP sender buffer space (bytes). */
#define TCP_SND_BUF              (4*TCP_MSS)          (7)

/*  TCP_SND_QUEUELEN: TCP sender buffer space (pbufs). This must be at␣
↪least
as much as (2 * TCP_SND_BUF/TCP_MSS) for things to work. */

#define TCP_SND_QUEUELEN         (2* TCP_SND_BUF/TCP_MSS)

/* TCP receive window. */
#define TCP_WND                  (2*TCP_MSS)          (8)

/* ---------- ICMP options ---------- */
#define LWIP_ICMP                           1

/* ---------- DHCP options ---------- */
/* Define LWIP_DHCP to 1 if you want DHCP configuration of
interfaces. DHCP is not implemented in lwIP 0.5.1, however, so
turning this on does currently not work. */
#define LWIP_DHCP                1

/* ---------- UDP options ---------- */
#define LWIP_UDP                 1
#define UDP_TTL                  255

/* ---------- Statistics options ---------- */
#define LWIP_STATS 0
#define LWIP_PROVIDE_ERRNO 1

/* ---------- link callback options ---------- */
/* LWIP_NETIF_LINK_CALLBACK==1: Support a callback function from an␣
↪interface
```

(下页继续)

(续上页)

```
* whenever the link changes (i.e., link down)
*/
#define LWIP_NETIF_LINK_CALLBACK        0
/*
    ----------------------------------------
    ---------- Checksum options ----------
    ----------------------------------------
*/

/*The STM32F4x7 allows comput
ing and verifying the IP, UDP, TCP and ICMP checksums by hardware:
- To use this feature let the following define uncommented.
- To disable it and process by CPU comment the  the checksum.
*/
#define CHECKSUM_BY_HARDWARE

#ifdef CHECKSUM_BY_HARDWARE
/* CHECKSUM_GEN_IP==0: Generate checksums by hardware for outgoing IP␣
↪packets.*/
#define CHECKSUM_GEN_IP                 0
/* CHECKSUM_GEN_UDP==0: Generate checksums by hardware for outgoing UDP␣
↪packets.*/
#define CHECKSUM_GEN_UDP                0
/* CHECKSUM_GEN_TCP==0: Generate checksums by hardware for outgoing TCP␣
↪packets.*/
#define CHECKSUM_GEN_TCP                0
/* CHECKSUM_CHECK_IP==0: Check checksums by hardware for incoming IP␣
↪packets.*/
#define CHECKSUM_CHECK_IP               0
/* CHECKSUM_CHECK_UDP==0: Check checksums by hardware for incoming UDP␣
↪packets.*/
#define CHECKSUM_CHECK_UDP              0
```

(下页继续)

(续上页)

```
/* CHECKSUM_CHECK_TCP==0: Check checksums by hardware for incoming TCP␣
↪packets.*/
#define CHECKSUM_CHECK_TCP              0
/*CHECKSUM_CHECK_ICMP==0: Check checksums by hardware for incoming ICMP␣
↪packets.*/
#define CHECKSUM_GEN_ICMP               0
#else
/* CHECKSUM_GEN_IP==1: Generate checksums in software for outgoing IP␣
↪packets.*/
#define CHECKSUM_GEN_IP                 1
/* CHECKSUM_GEN_UDP==1: Generate checksums in software for outgoing UDP␣
↪packets.*/
#define CHECKSUM_GEN_UDP                1
/* CHECKSUM_GEN_TCP==1: Generate checksums in software for outgoing TCP␣
↪packets.*/
#define CHECKSUM_GEN_TCP                1
/* CHECKSUM_CHECK_IP==1: Check checksums in software for incoming IP␣
↪packets.*/
#define CHECKSUM_CHECK_IP               1
/* CHECKSUM_CHECK_UDP==1: Check checksums in software for incoming UDP␣
↪packets.*/
#define CHECKSUM_CHECK_UDP              1
/* CHECKSUM_CHECK_TCP==1: Check checksums in software for incoming TCP␣
↪packets.*/
#define CHECKSUM_CHECK_TCP              1
/*CHECKSUM_CHECK_ICMP==1: Check checksums by hardware for incoming ICMP␣
↪packets.*/
#define CHECKSUM_GEN_ICMP               1
#endif

/*
    ----------------------------------------------
```

(下页继续)

(续上页)

```
    ---------- Sequential layer options ----------
    ----------------------------------------------
*/
/**
* LWIP_NETCONN==1: Enable Netconn API (require to use api_lib.c)
*/
#define LWIP_NETCONN                    0             (9)

/*
    ------------------------------------
    ---------- Socket options ----------
    ------------------------------------
*/
/**
* LWIP_SOCKET==1: Enable Socket API (require to use sockets.c)
*/
#define LWIP_SOCKET                     0             (10)

/*
    ---------------------------------------
    ---------- Lwip Debug options ----------
    ---------------------------------------
*/
//#define LWIP_DEBUG                      1

#endif /* __LWIPOPTS_H__ */

/************************ (C) COPYRIGHT STMicroelectronics *****END OF␣
↪FILE****/
```

代码清单 7-1**(1)**：NO_SYS 表示无操作系统模拟层，这个宏非常重要，因为无操作系统与有操作系统的移植和编写是完全不一样的，我们现在是无操作系统移植，所以将这个宏定义为 1。

代码清单 7-1**(2)**：内存对齐，按照 4 字节对齐。

代码清单 7-1**(3)**：堆内存的大小。如果应用程序将发送很多需要复制的数据应该设置得大一点。

代码清单 7-1**(4)**：PBUF_POOL 内存池中内存块数量。

代码清单 7-1**(5)**：PBUF_POOL 内存池中每个内存块大小。

代码清单 7-1**(6)**：TCP 协议报文最大长度。

代码清单 7-1**(7)**：允许 TCP 协议使用的最大发送缓冲区空间（字节）。

代码清单 7-1**(8)**：TCP 接收窗口大小。

代码清单 7-1**(9)(10)**：因为现在是无操作系统，就不使能 NETCONN API 和 Socket API 编程。

cc.h 文件中包含处理器相关的变量类型、数据结构及字节对齐的相关宏。

LwIP 中使用的基本变量类型均以位数进行命名，为抽象的变量类型定义，开发者需要根据所用处理器及编译器特性进行定义，一般我们直接将变量直接定义为 C 语言的基本类型，如 unsigned char、int 等，这样子可以保证 LwIP 协议栈就与平台无关了。除此之外我们还可以定义大小端模式，输出调试的宏等，cc.h 文件内容具体见代码清单 7_2。

代码清单 7-2 cc.h 文件内容

```
#ifndef __CC_H__
#define __CC_H__

#include "stdio.h"

#include "main.h"

#define LWIP_NO_STDINT_H  1

typedef unsigned   char    u8_t;
typedef signed     char    s8_t;
typedef unsigned   short   u16_t;
typedef signed     short   s16_t;
```

(下页继续)

(续上页)

```
typedef unsigned   long    u32_t;
typedef signed     long    s32_t;
typedef u32_t mem_ptr_t;
typedef int sys_prot_t;


#define U16_F "hu"
#define S16_F "d"
#define X16_F "hx"
#define U32_F "u"
#define S32_F "d"
#define X32_F "x"
#define SZT_F "uz"



/* 选择小端模式 */
#define BYTE_ORDER LITTLE_ENDIAN

/* define compiler specific symbols */
#if defined (__ICCARM__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x
#define PACK_STRUCT_USE_INCLUDES

#elif defined (__CC_ARM)

#define PACK_STRUCT_BEGIN __packed
#define PACK_STRUCT_STRUCT
```

(下页继续)

(续上页)

```
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#elif defined (__GNUC__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT __attribute__ ((__packed__))
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#elif defined (__TASKING__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#endif


#define LWIP_PLATFORM_ASSERT(x) do {printf(x);}while(0)


extern u32_t sys_now(void);

#endif /* __CC_H__ */
```

perf.h 文件是与系统统计与测量相关的头文件，我们暂时无需使用任何统计与测量功能，因此该头文件的量宏定义直接为空即可，具体见代码清单 7_3。

代码清单 7-3 perf.h 文件内容

```
#ifndef __PERF_H__
#define __PERF_H__

#define PERF_START    /* null definition */
#define PERF_STOP(x)  /* null definition */

#endif /* __PERF_H__ */
```

7.3 移植网卡驱动

在前面的 4.4 章节与 6.8 章节中已经讲解了关于底层驱动的函数，这些函数在网卡中至关重要，而 ethernetif.c 文件就是存放这些函数的，LwIP 的 contrib 包中就包含这个文件的模板，我们直接拿过来修改即可，该文件的路径为 “contrib-2.1.0examplesethernetif”，然后我们拷贝到 arch 文件夹下，并且创建一个 ethernetif.h 文件，一同添加到我们的工程中即可。我们直接使用 4.4 章节与 6.8 章节编写的网卡驱动代码，在编写完成的 ethernetif.c 文件内容具体见代码清单 7_4。

代码清单 7-4 ethernetif.c 文件全貌

```
#include "main.h"
#include "lwip/opt.h"
#include "lwip/mem.h"
#include "lwip/memp.h"
#include "lwip/timeouts.h"
#include "netif/ethernet.h"
#include "netif/etharp.h"
#include "lwip/ethip6.h"
#include "ethernetif.h"
#include <string.h>

/* Network interface name */
#define IFNAME0 's'
```

(下页继续)

(续上页)

```
#define IFNAME1 't'

struct ethernetif
{
    struct eth_addr *ethaddr;
    /* Add whatever per-interface state that is needed here. */
};

extern ETH_HandleTypeDef heth;

static void arp_timer(void *arg);

static void low_level_init(struct netif *netif)
{
    HAL_StatusTypeDef hal_eth_init_status;

    //初始化 bsp—eth
    hal_eth_init_status = Bsp_Eth_Init();

    if (hal_eth_init_status == HAL_OK)
    {
        /* Set netif link flag */
        netif->flags |= NETIF_FLAG_LINK_UP;
    }

#if LWIP_ARP || LWIP_ETHERNET

    /* set MAC hardware address length */
    netif->hwaddr_len = ETH_HWADDR_LEN;

    /* set MAC hardware address */
    netif->hwaddr[0] =  heth.Init.MACAddr[0];
```

(下页继续)

(续上页)

```
    netif->hwaddr[1] =  heth.Init.MACAddr[1];
    netif->hwaddr[2] =  heth.Init.MACAddr[2];
    netif->hwaddr[3] =  heth.Init.MACAddr[3];
    netif->hwaddr[4] =  heth.Init.MACAddr[4];
    netif->hwaddr[5] =  heth.Init.MACAddr[5];

    /* maximum transfer unit */
    netif->mtu = NETIF_MTU;

#if LWIP_ARP
    netif->flags |= NETIF_FLAG_BROADCAST | NETIF_FLAG_ETHARP;
#else
    netif->flags |= NETIF_FLAG_BROADCAST;
#endif /* LWIP_ARP */

#endif /* LWIP_ARP || LWIP_ETHERNET */

    HAL_ETH_Start(&heth);
}


static err_t low_level_output(struct netif *netif, struct pbuf *p)
{

    err_t errval;
    struct pbuf *q;

    uint8_t *buffer = (uint8_t *)(heth.TxDesc->Buffer1Addr);

    __IO ETH_DMADescTypeDef *DmaTxDesc;

    uint32_t bufferoffset = 0;
```

(下页继续)

(续上页)

```
    uint32_t framelength = 0;

    uint32_t byteslefttocopy = 0;

    uint32_t payloadoffset = 0;

    DmaTxDesc = heth.TxDesc;
    bufferoffset = 0;

    if ((DmaTxDesc->Status & ETH_DMATXDESC_OWN) != (uint32_t)RESET)
    {
        errval = ERR_USE;
        goto error;
    }

    /* copy frame from pbufs to driver buffers */
    for (q = p; q != NULL; q = q->next)
    {
        /* Get bytes in current lwIP buffer */
        byteslefttocopy = q->len;
        payloadoffset = 0;

    /*Check if the length of data to copy is bigger than Tx buffer size*/
        while ( (byteslefttocopy + bufferoffset) > ETH_TX_BUF_SIZE )
        {
            /* Copy data to Tx buffer*/
            memcpy( (uint8_t*)((uint8_t*)buffer + bufferoffset),
                    (uint8_t*)((uint8_t*)q->payload + payloadoffset),
                    (ETH_TX_BUF_SIZE - bufferoffset) );

            /* Point to next descriptor */
            DmaTxDesc = (ETH_DMADescTypeDef *)(DmaTxDesc->
↪Buffer2NextDescAddr);
```

(下页继续)

(续上页)

```
            /* Check if the buffer is available */
            if ((DmaTxDesc->Status & ETH_DMATXDESC_OWN) != (uint32_t)RESET)
            {
                errval = ERR_USE;
                goto error;
            }

            buffer = (uint8_t *)(DmaTxDesc->Buffer1Addr);

            byteslefttocopy = byteslefttocopy - (ETH_TX_BUF_SIZE -␣
↪bufferoffset);
            payloadoffset = payloadoffset + (ETH_TX_BUF_SIZE -␣
↪bufferoffset);
            framelength = framelength + (ETH_TX_BUF_SIZE - bufferoffset);
            bufferoffset = 0;
        }

        /* Copy the remaining bytes */
        memcpy( (uint8_t*)((uint8_t*)buffer + bufferoffset),
            (uint8_t*)((uint8_t*)q->payload + payloadoffset),␣
↪byteslefttocopy );
        bufferoffset = bufferoffset + byteslefttocopy;
        framelength = framelength + byteslefttocopy;
    }

    /* Prepare transmit descriptors to give to DMA */
    HAL_ETH_TransmitFrame(&heth, framelength);

    errval = ERR_OK;

error:
```

(下页继续)

(续上页)

```
    if ((heth.Instance->DMASR & ETH_DMASR_TUS) != (uint32_t)RESET)
    {
        /* Clear TUS ETHERNET DMA flag */
        heth.Instance->DMASR = ETH_DMASR_TUS;

        /* Resume DMA transmission*/
        heth.Instance->DMATPDR = 0;
    }

    return errval;
}

static struct pbuf * low_level_input(struct netif *netif)
{
    struct pbuf *p = NULL;
    struct pbuf *q = NULL;
    uint16_t len = 0;
    uint8_t *buffer;
    __IO ETH_DMADescTypeDef *dmarxdesc;
    uint32_t bufferoffset = 0;
    uint32_t payloadoffset = 0;
    uint32_t byteslefttocopy = 0;
    uint32_t i=0;

    /* get received frame */
    if (HAL_ETH_GetReceivedFrame(&heth) != HAL_OK)
    {
        PRINT_ERR("receive frame faild\n");
        return NULL;
    }
/*Obtain the size of the packet and put it into the "len" variable.*/
```

(下页继续)

(续上页)

```
    len = heth.RxFrameInfos.length;
    buffer = (uint8_t *)heth.RxFrameInfos.buffer;

    PRINT_INFO("receive frame len : %d\n", len);

    if (len > 0)
    {
    /* We allocate a pbuf chain of pbufs from the Lwip buffer pool */
        p = pbuf_alloc(PBUF_RAW, len, PBUF_POOL);
    }

    if (p != NULL)
    {
        dmarxdesc = heth.RxFrameInfos.FSRxDesc;
        bufferoffset = 0;
        for (q = p; q != NULL; q = q->next)
        {
            byteslefttocopy = q->len;
            payloadoffset = 0;


            while ( (byteslefttocopy + bufferoffset) > ETH_RX_BUF_SIZE )
            {
                /* Copy data to pbuf */
                memcpy( (uint8_t*)((uint8_t*)q->payload + payloadoffset),
                        (uint8_t*)((uint8_t*)buffer + bufferoffset),
                        (ETH_RX_BUF_SIZE - bufferoffset));

                /* Point to next descriptor */
                dmarxdesc = (ETH_DMADescTypeDef *)(dmarxdesc->
↪Buffer2NextDescAddr);
                buffer = (uint8_t *)(dmarxdesc->Buffer1Addr);
```

(下页继续)

(续上页)

```
            byteslefttocopy = byteslefttocopy - (ETH_RX_BUF_SIZE -␣
↪bufferoffset);
                payloadoffset = payloadoffset + (ETH_RX_BUF_SIZE -␣
↪bufferoffset);
                bufferoffset = 0;
            }
            /* Copy remaining data in pbuf */
            memcpy( (uint8_t*)((uint8_t*)q->payload + payloadoffset),
                    (uint8_t*)((uint8_t*)buffer + bufferoffset),␣
↪byteslefttocopy);
            bufferoffset = bufferoffset + byteslefttocopy;
        }
    }

    /* Release descriptors to DMA */
    /* Point to first descriptor */
    dmarxdesc = heth.RxFrameInfos.FSRxDesc;
    /* Set Own bit in Rx descriptors: gives the buffers back to DMA */
    for (i=0; i< heth.RxFrameInfos.SegCount; i++)
    {
        dmarxdesc->Status |= ETH_DMARXDESC_OWN;
        dmarxdesc = (ETH_DMADescTypeDef *)(dmarxdesc->Buffer2NextDescAddr);
    }

    /* Clear Segment_Count */
    heth.RxFrameInfos.SegCount =0;

    /* When Rx Buffer unavailable flag is set: clear it and resume␣
↪reception */
    if ((heth.Instance->DMASR & ETH_DMASR_RBUS) != (uint32_t)RESET)
    {
```

(下页继续)

(续上页)

```
        /* Clear RBUS ETHERNET DMA flag */
        heth.Instance->DMASR = ETH_DMASR_RBUS;
        /* Resume DMA reception */
        heth.Instance->DMARPDR = 0;
    }
    return p;
}


void ethernetif_input(struct netif *netif)
{
    err_t err;
    struct pbuf *p;

    /* move received packet into a new pbuf */
    p = low_level_input(netif);

    /* no packet could be read, silently ignore this */
    if (p == NULL) return;

    /* entry point to the LwIP stack */
    err = netif->input(p, netif);

    if (err != ERR_OK)
    {
        LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input error\n"));
        pbuf_free(p);
        p = NULL;
    }
}

#if !LWIP_ARP
```

(下页继续)

(续上页)

```
static err_t low_level_output_arp_off(struct netif *netif, struct pbuf *q,␣
↪con
ip4_addr_t *ipaddr)
{
    err_t errval;
    errval = ERR_OK;

    return errval;

}
#endif /* LWIP_ARP */

err_t ethernetif_init(struct netif *netif)
{
    struct ethernetif *ethernetif;

//  LWIP_ASSERT("netif != NULL", (netif != NULL));

    ethernetif = mem_malloc(sizeof(struct ethernetif));

    if (ethernetif == NULL)
    {
        PRINT_ERR("ethernetif_init: out of memory\n");
        return ERR_MEM;
    }

    LWIP_ASSERT("netif != NULL", (netif != NULL));
//
#if LWIP_NETIF_HOSTNAME
    /* Initialize interface hostname */
    netif->hostname = "lwip";
```

(下页继续)

(续上页)

```
#endif /* LWIP_NETIF_HOSTNAME */
    netif->state = ethernetif;
    netif->name[0] = IFNAME0;
    netif->name[1] = IFNAME1;

#if LWIP_IPV4
#if LWIP_ARP || LWIP_ETHERNET
#if LWIP_ARP
    netif->output = etharp_output;
#else
    netif->output = low_level_output_arp_off;
#endif /* LWIP_ARP */
#endif /* LWIP_ARP || LWIP_ETHERNET */
#endif /* LWIP_IPV4 */

#if LWIP_IPV6
    netif->output_ip6 = ethip6_output;
#endif /* LWIP_IPV6 */

    netif->linkoutput = low_level_output;

    /* initialize the hardware */
    low_level_init(netif);
    ethernetif->ethaddr = (struct eth_addr *) &(netif->hwaddr[0]);

    return ERR_OK;
}
static void arp_timer(void *arg)
{
    etharp_tmr();
    sys_timeout(ARP_TMR_INTERVAL, arp_timer, NULL);
}
```

(下页继续)

(续上页)

```
/* USER CODE BEGIN 6 */


void ethernetif_update_config(struct netif *netif)
{
    __IO uint32_t tickstart = 0;
    uint32_t regvalue = 0;

    if (netif_is_link_up(netif))
    {
        /* Restart the auto-negotiation */
        if (heth.Init.AutoNegotiation != ETH_AUTONEGOTIATION_DISABLE)
        {
            /* Enable Auto-Negotiation */
            HAL_ETH_WritePHYRegister(&heth, PHY_BCR, PHY_AUTONEGOTIATION);

            /* Get tick */
            tickstart = HAL_GetTick();

            /* Wait until the auto-negotiation will be completed */
            do
            {
                HAL_ETH_ReadPHYRegister(&heth, PHY_BSR, &regvalue);

                /* Check for the Timeout ( 1s ) */
                if ((HAL_GetTick() - tickstart ) > 1000)
                {
                    /* In case of timeout */
                    goto error;
                }
            }
```

(下页继续)

(续上页)

```
            while (((regvalue & PHY_AUTONEGO_COMPLETE) != PHY_AUTONEGO_
↪COMPLETE));

            /* Read the result of the auto-negotiation */
            HAL_ETH_ReadPHYRegister(&heth, PHY_SR, &regvalue);


            if ((regvalue & PHY_DUPLEX_STATUS) != (uint32_t)RESET)
            {

                heth.Init.DuplexMode = ETH_MODE_FULLDUPLEX;
            }
            else
            {

                heth.Init.DuplexMode = ETH_MODE_HALFDUPLEX;
            }

            if (regvalue & PHY_SPEED_STATUS)
            {
        /* Set Ethernet speed to 10M following the auto-negotiation */
                heth.Init.Speed = ETH_SPEED_10M;
            }
            else
            {
        /* Set Ethernet speed to 100M following the auto-negotiation */
                heth.Init.Speed = ETH_SPEED_100M;
            }
        }
        else /* AutoNegotiation Disable */
        {
error :
```

(下页继续)

(续上页)

```
            /* Check parameters */
            assert_param(IS_ETH_SPEED(heth.Init.Speed));
            assert_param(IS_ETH_DUPLEX_MODE(heth.Init.DuplexMode));

            /* Set MAC Speed and Duplex Mode to PHY */
            HAL_ETH_WritePHYRegister(&heth, PHY_BCR,
                                ((uint16_t)(heth.Init.DuplexMode >> 3) |
                                (uint16_t)(heth.Init.Speed >> 1)));
        }

        /* ETHERNET MAC Re-Configuration */
        HAL_ETH_ConfigMAC(&heth, (ETH_MACInitTypeDef *) NULL);

        /* Restart MAC interface */
        HAL_ETH_Start(&heth);
    }
    else
    {
        /* Stop MAC interface */
        HAL_ETH_Stop(&heth);
    }

    ethernetif_notify_conn_changed(netif);
}
```

当然，我们还需要一个 ethernetif.h 文件，主要是对函数的一些声明，其内容具体见代码清单 7_5。

代码清单 7-5 ethernetif.h 文件内容

```
#ifndef __ETHERNETIF_H__
#define __ETHERNETIF_H__
```

(下页继续)

(续上页)

```
#include "lwip/err.h"
#include "lwip/netif.h"

err_t ethernetif_init(struct netif *netif);

void ethernetif_input(struct netif *netif);
void ethernetif_update_config(struct netif *netif);
void ethernetif_notify_conn_changed(struct netif *netif);

#endif
```

7.4 LwIP 时基

LwIP 也是一个内核，与操作系统一样，也是由时基驱动的，LwIP 作者为了能让内核正常运行，也引入了一个时钟来驱动，这样子可以处理内核中各种定时事件，如 ARP 定时、TCP 定时等，LwIP 已经实现处理超时（定时）事件的函数 sys_check_timeouts()，具体怎么处理的就无需用户关心。由于时钟的来源是由用户提供的，这就需要用户实现一个 sys_now() 函数来获取系统的时钟，以毫秒为单位，LwIP 通过两次获取的时间就能判断是否有超时，从而让内核去处理对应的事件。

我们在 STM32 中，一般采用 SysTick 作为 LwIP 的时基定时器，将 SysTick 产生中断的频率设置为 1000HZ，也就是 1ms 触发一次中断，每次产生中断的时候，系统变量就会加 1，当然，在 HAL 库中已经实现了获取系统时间的函数 HAL_GetTick()，那么很简单，我们在 sys_now() 函数中直接返回 HAL_GetTick() 函数得到的值即可，具体见。但是有个问题，如果 SysTick 的频率不是 1000HZ，那就需要将 HAL_GetTick() 函数得到的值转换为时间（ms），这就由用户自己实现即可，这也是很简单的，当我们使用操作系统的时候，就直接可以转换了使用操作系统的宏进行 tick 与 ms 的转换了，在后续讲解。

代码清单 7-6 sys_now() 实现

```
u32_t sys_now(void)
{
```

(下页继续)

(续上页)

```
    return HAL_GetTick();
}


void SysTick_Handler(void)
{
    HAL_IncTick();
}
```

7.5 协议栈初始化

想要使用 LwIP，那就必须先将协议栈初始化，我们就创建一个函数，在函数中初始化协议栈，注册网卡，设置主机的 IP 地址、子网掩码、网关地址等，比如作者个人电脑的 IP 地址是 192.163.1.181，那么我们在开发板上设置的 IP 地址必须是与路由器处于同一子网的，我就设置为 192.168.1.122，因为这个地址必须是路由器承认的合法地址，否则路由器不会对这个 IP 地址的数据包进行转发，网关就写对应的网关（路由器 IP 地址）192.168.1.1 即可，255.255.255.0 为整个局域网的子网掩码。

然后挂载我们的网卡，挂载网卡的函数我们也讲解过了，就是 netif_add() 函数，如果我们了解了前面章节的内容，移植起来是一点都不费劲的。这里主要讲解一下 ethernet_input() 函数，这个函数在 ethernet.c 文件中（在以前的版本如 1.4.1，这个函数在 etharp.c 文件），主要是用于无操作系统时候 LwIP 去处理接收到的数据，接收网卡的数据然后往上层递交，对于不同的数据包进行不同的处理，如果是 ARP 包，则调用 etharp_input() 函数交给 ARP 去处理，更新 ARP 缓存表；如果是 IP 包，则调用 ip4_input() 函数递交给 IP 层去处理，在后续会详细讲解，此处了解一下即可，协议栈初始化的源码具体见代码清单 7_7。

提示：该函数由用户编写，可以随意放在任何地方，作者个人就将它放在 main.c 文件中。

代码清单 7-7 协议栈初始化

```
/*Static IP ADDRESS: IP_ADDR0.IP_ADDR1.IP_ADDR2.IP_ADDR3 */
#define IP_ADDR0                        192
#define IP_ADDR1                        168
#define IP_ADDR2                          1
#define IP_ADDR3                        122

/*NETMASK*/
#define NETMASK_ADDR0                   255
#define NETMASK_ADDR1                   255
#define NETMASK_ADDR2                   255
#define NETMASK_ADDR3                     0

/*Gateway Address*/
#define GW_ADDR0                        192
#define GW_ADDR1                        168
#define GW_ADDR2                          1
#define GW_ADDR3                          1
/* USER CODE END 0 */

void LwIP_Init(void)
{

    IP4_ADDR(&ipaddr,IP_ADDR0,IP_ADDR1,IP_ADDR2,IP_ADDR3);
    IP4_ADDR(&netmask,NETMASK_ADDR0,NETMASK_ADDR1,
            NETMASK_ADDR2,NETMASK_ADDR3);
    IP4_ADDR(&gw,GW_ADDR0,GW_ADDR1,GW_ADDR2,GW_ADDR3);

    /* Initilialize the LwIP stack without RTOS */
    lwip_init();

    /* add the network interface (IPv4/IPv6) without RTOS */
    netif_add(&gnetif, &ipaddr, &netmask, &gw, NULL,
```

(下页继续)

(续上页)

```
            &ethernetif_init, &ethernet_input);

    /* Registers the default network interface */
    netif_set_default(&gnetif);

    if (netif_is_link_up(&gnetif))
    {
    /*When the netif is fully configured this function must be called */
        netif_set_up(&gnetif);
    }
    else
    {
        /* When the netif link is down this function must be called */
        netif_set_down(&gnetif);
    }

}
```

7.6 获取数据包

通过上面的步骤，我们能使用开发板获取网络的数据包了，但是获取数据包的方式有两种，一种是查询方式，另一种是中断方式。查询方式通过主函数的 while 循环进行周期性处理，去获取网卡中是否接收到数据包，然后递交给上层协议去处理，而中单方式则不一样，在网卡接收到一个数据包的时候，就触发中断，通知 CPU 去处理，这样子效率就会高很多，特别是在操作系统环境下，我们都采用中断方式去获取数据包。当然，查询方式与中断方式的网卡底层初始化是不一样的，主要是通过网卡接收方式的配置进行初始化，在初始化的时候，如果网卡接收模式被配置为 ETH_RXINTERRUPT_MODE，则表示使用中断方式获取数据包，而如果网卡接收模式被配置为 ETH_RXPOLLING_MODE 则表明用查询方式获取数据包。

提示：网卡底层初始化函数是 Bsp_Eth_Init()，在 bsp_eth.c 文件中，用户可以自行修改。

7.6.1 查询方式

使用查询方式获取数据包的时候，我们只需要在程序中周期性调用网卡接收函数即可，具体见代码清单 7_8.

代码清单 7-8 查询方式获取数据包

```
int main(void)
{
    //板级外设初始化
    BSP_Init();

    //LwIP 协议栈初始化
    LwIP_Init();

    while (1)
    {
        //调用网卡接收函数
        ethernetif_input(&gnetif);

        //处理 LwIP 中定时事件
        sys_check_timeouts();
    }
}
```

然后我们编译一下程序，并且将程序下载到开发板，然后我们打开 Windows 的控制台，可以直接通过 win+r 快捷键输入“cmd”快速打开，具体见图 7_10。

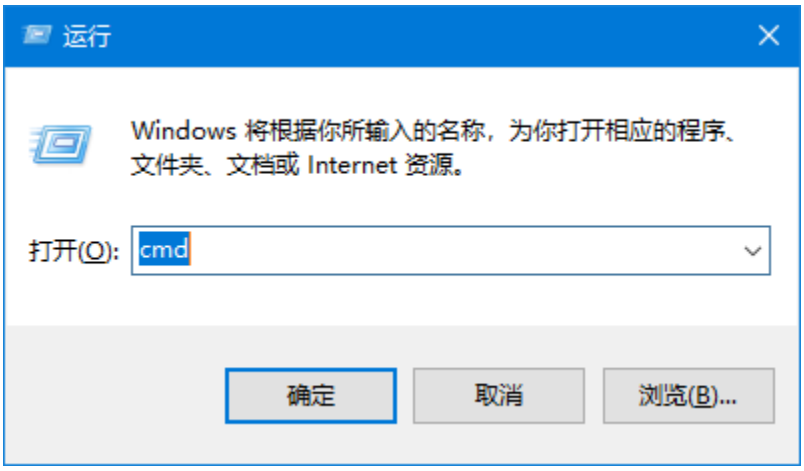


图 7-10 打开 Windows 的控制台

如果不知道自己所处的局域网网关与电脑 IP 地址的，我们可以通过在 Windows 的控制台中执行“ipconfig”命令，就能得到电脑的 IP 地址，相应的就能得到网关地址，我们只需要将电脑 IP 地址中最后的 3 位数改成我们开发板上的地址即可，具体见图 *7_11*。

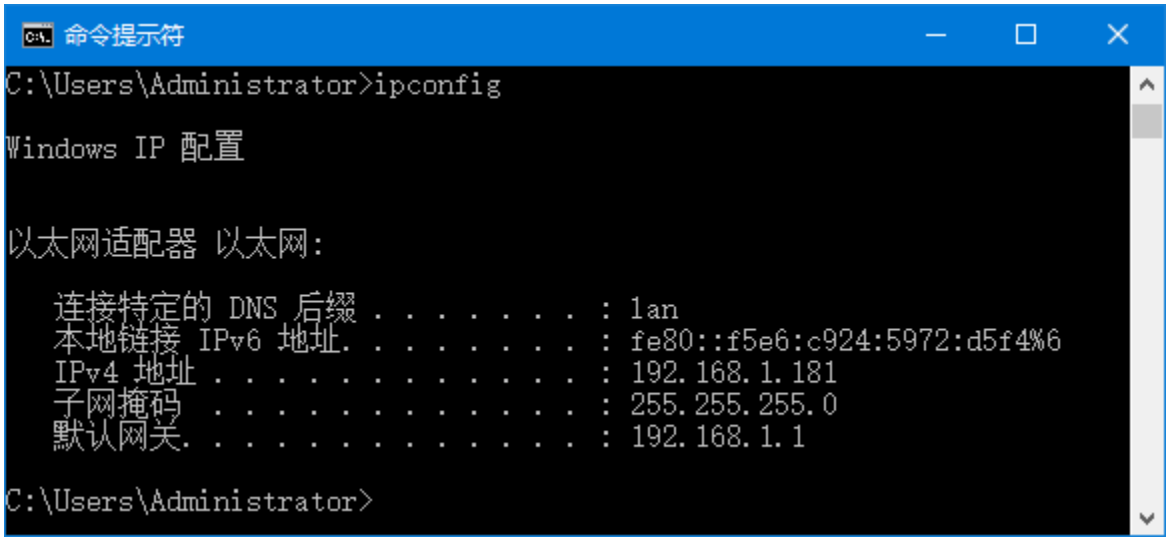


图 7-11 执行“ipconfig”命令查看 IP 地址

7.6.2 ping 命令详解

然后我们在命令行中执行 ping 命令，看看网卡驱动与协议栈是否能正常工作，关于 ping 命令的基本介绍如下（“[ ]”中的命令是非必须填充字段）：

ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS] [-r count] [-s count] [[-j host-list] | [-k host-list]][-w timeout] [-R] [-S srcaddr] [-c compartment] [-p][-4] [-6] target_name

选项：

- -t Ping 指定的主机，直到停止。若要查看统计信息并继续操作，请键入 Ctrl+Break；若要停止，请键入 Ctrl+C。
- -a 将地址解析为主机名。
- -n count 要发送的回显请求数，默认值为 4。
- -l size 发送缓冲区大小。
- -f 在数据包中设置“不分段”标记 (仅适用于 IPv4)，数据包就不会被路由上的网关分段。
- -i TTL 指定生存时间。
- -v TOS 服务类型 (仅适用于 IPv4。该设置已被弃用，对 IP 标头中的服务类型字段没有任何影响)
- -r count 记录计数跃点的路由 (仅适用于 IPv4)。
- -s count 计数跃点的时间戳 (仅适用于 IPv4)。
- -j host-list 与主机列表一起使用的松散源路由 (仅适用于 IPv4)。
- -k host-list 与主机列表一起使用的严格源路由 (仅适用于 IPv4)。
- -w timeout 等待每次回复的超时时间 (毫秒)。
- -R 同样使用路由标头测试反向路由 (仅适用于 IPv6)。根据 RFC 5095，已弃如果使用此标头，某些系统可能丢弃用此路由标头，如果使用此标头，某些系统可能丢弃回显请求
- -S srcaddr 要使用的源地址。
- -c compartment 路由隔离舱标识符。

- -p Ping Hyper-V 网络虚拟化提供程序地址。
- -4 强制使用 IPv4。
- -6 强制使用 IPv6。
- target_name 指定要 ping 的远程计算机。

当然，那么多命令肯定也是记不住的，那我们可以通过在控制台中执行“ping /?”命令，就能得到 ping 命令说明了，具体见图 *7_12*。

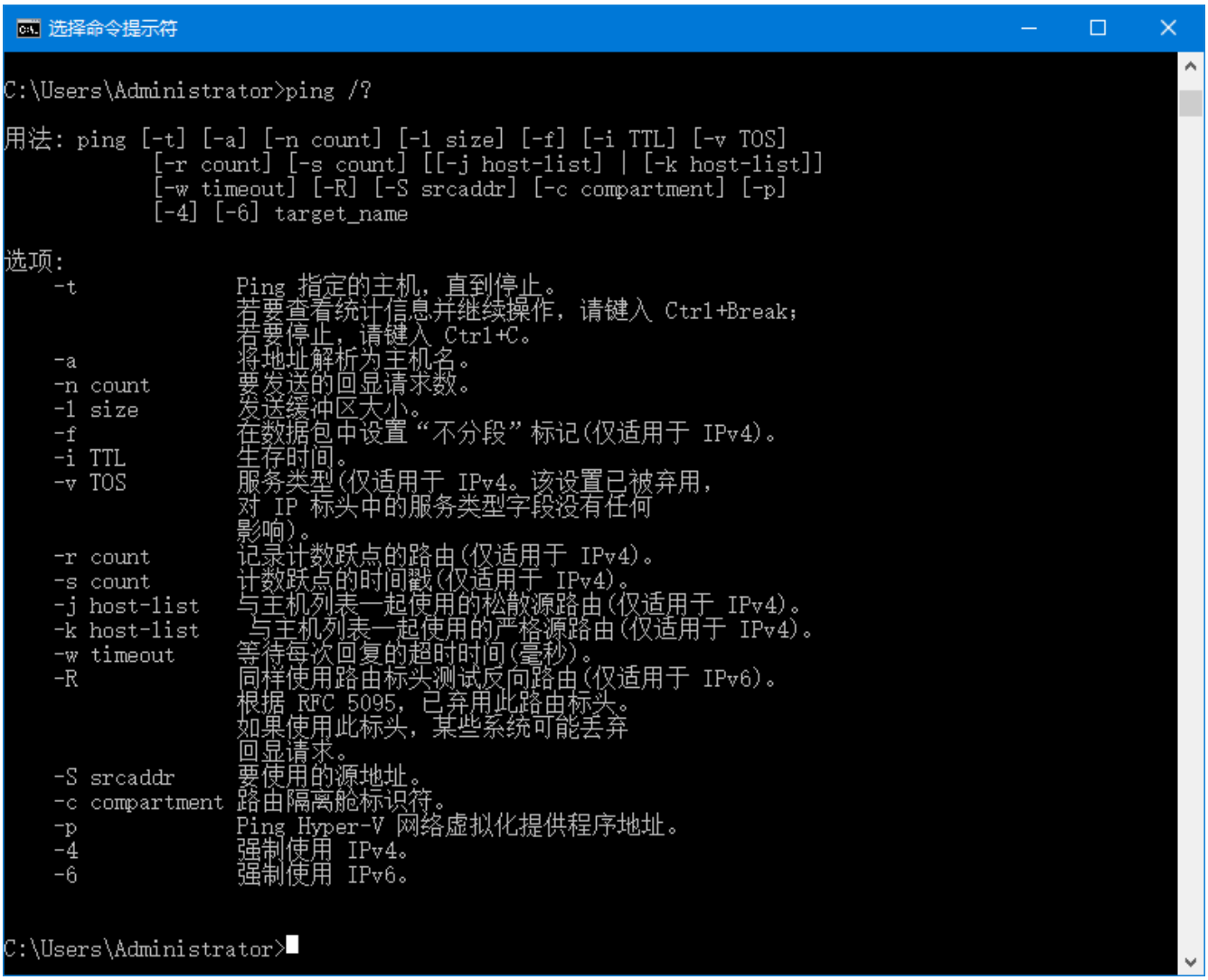


图 7-12 “ping /?”命令

学习了 ping 命令，我们直接执行“ping xxx.xxx.xxx.xxx”命令，看一下我们的开发板看看能否 ping 得通，如果电脑发出的 ping 包能被开发板接收并且正确返回，这代表这电脑与开发板在网络上已经是连通的，xxx 对应自己开发板的 IP 地址，执行命令的结果具体见图 *7_13*，然后我们再执行“arp -a”命令，可以看到电脑主机的 ARP 表，我们开发板的 IP 地址与 MAC 地址都正确出现在 ARP 表中，具体见图 *7_14*。

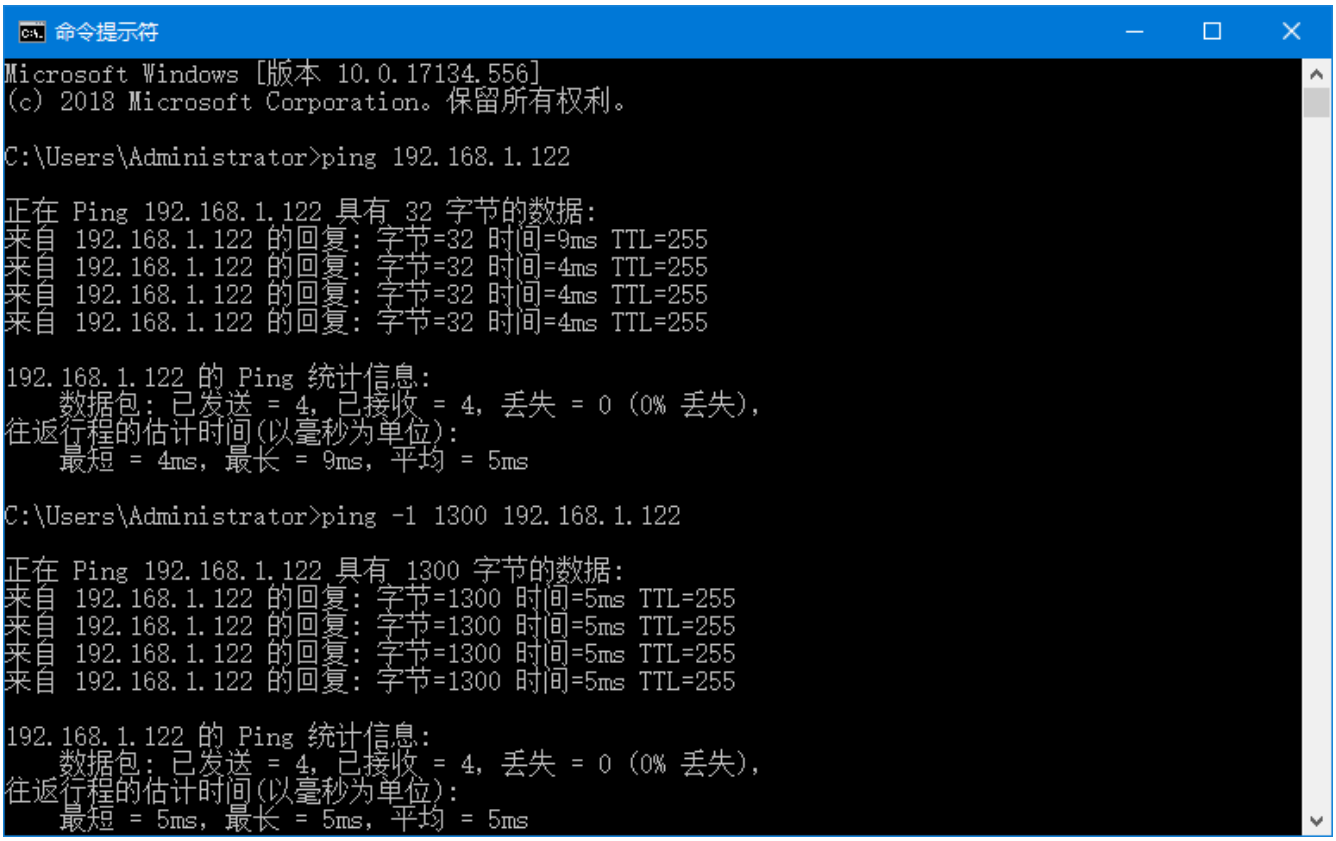


图 7-13 ping 结果

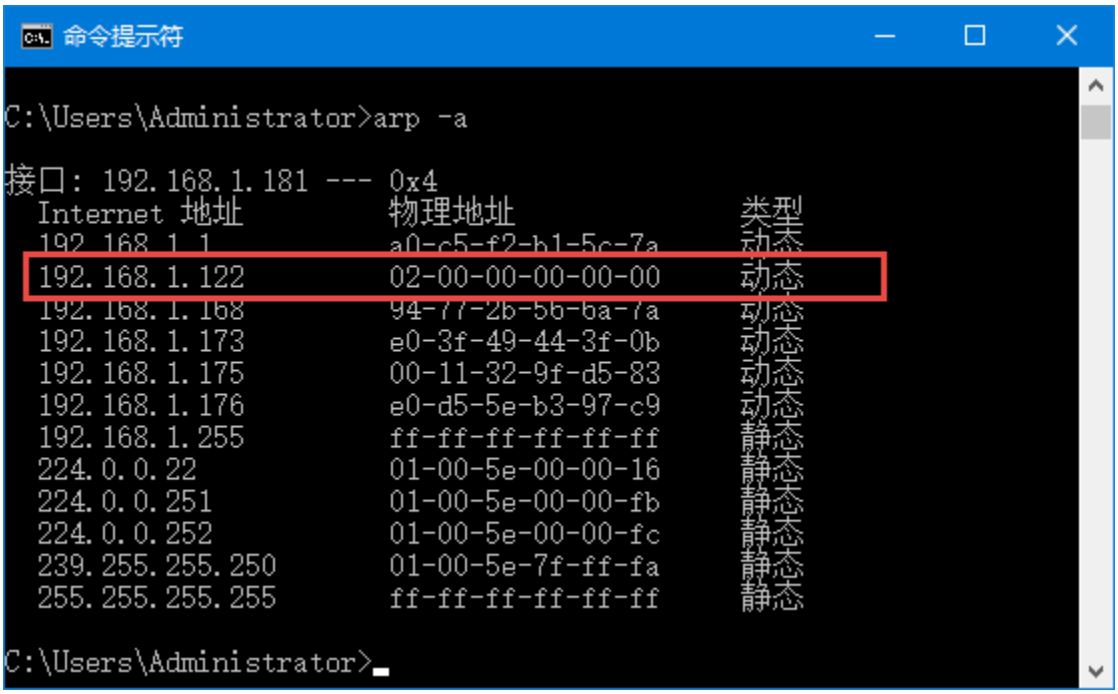


图 7-14 电脑主机 ARP 表

7.6.3 中断方式

采用查询的方式虽然可行，但是这种方式的效率不高，因为查询就要 CPU 去看看有没有数据，就像一个人在房子中等朋友过来，但是不知道朋友什么时候来，那主人就要每隔一段时间去看看朋友有没有过来，这样子就占用了大量的资源，主人也没能做其他事情，而如果在门口装一个门铃，朋友来的时候就按下门铃，主人就知道朋友来了，就出去迎接，这样子就不会占用主人的时间，主人可以做其他事情。同样的，我们可以使用中断方式来接收数据，当接收完成的时候，就通知 CPU 来处理即可，当然，还需要我们编写对应的中断服务函数 ETH_IRQHandler()，具体见代码清单 7_9。

代码清单 7-9 中断方式接收数据包

```
int flag = 0;
int main(void)
{
```

(下页继续)

(续上页)

```
    //板级外设初始化
    BSP_Init();

    //LwIP 协议栈初始化
    LwIP_Init();

    while (1)
    {
        if (flag)
        {
            flag = 0;
            //调用网卡接收函数
            ethernetif_input(&gnetif);
        }
        //处理 LwIP 中定时事件
        sys_check_timeouts();
    }
}

void ETH_IRQHandler(void)
{

    HAL_ETH_IRQHandler(&heth);

}

void HAL_ETH_RxCpltCallback(ETH_HandleTypeDef *heth)
{
    flag = 1;
//  LWIP_Process();
}
```

当我们编译并下载好程序，同样用 ping 命令去 ping 一下开发板，其结果具体见图 *7_15*，说明说明我们的中断方式的接收也是能正常工作的。

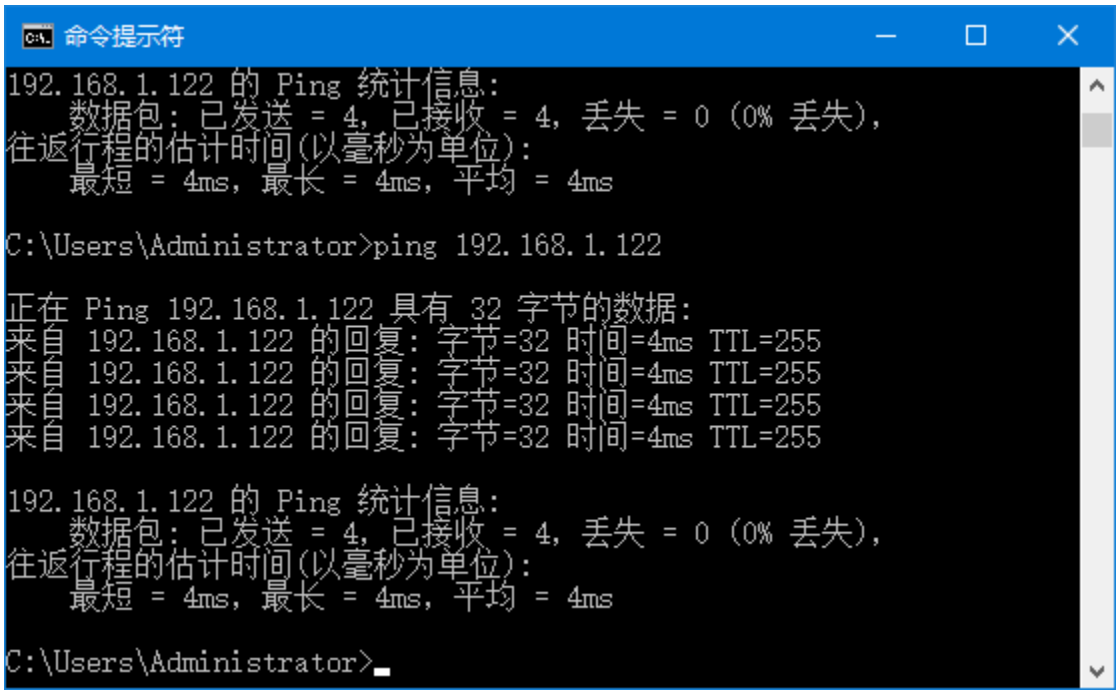


图 7-15 中断方式 ping 结果

第 8 章　有操作系统移植 LwIP

LwIP 不仅能在裸机上运行，也能在操作系统环境下运行，而且在操作系统环境下，用户能使用 NETCONN API 与 Socket API 编程，相比 RAW API 编程会更加简便。操作系统环境下，这意味着多线程环境，一般来说 LwIP 作为一个独立的处理线程运行，用户程序也独立为一个/多个线程，这样子在操作系统中就相互独立开，并且借助操作系统的 IPC 通信机制，更好地实现功能的需求。

LwIP 在设计之初，设计者无法预测 LwIP 运行的环境是怎么样的，而且世界上操作系统那么多，根本没法统一，而如果 LwIP 要运行在操作系统环境中，那么就必须产生依赖，即 LwIP 需要依赖操作系统自身的通信机制，如信号量、互斥量、消息队列（邮箱）等，所以 LwIP 设计者在设计的时候就提供一套与操作系统相关的接口，由用户根据操作系统的不同进行移植，这样子就能降低耦合度，让 LwIP 内核不受其运行的环境影响，因为往往用户并不能完全了解内核的运作，所以只需要用户在移植的时候对 LwIP 提供的接口根据不同操作系统进行完善即可。

8.1 LwIP 中添加操作系统

8.1.1 拷贝 FreeRTOS 源码到工程文件夹

我们先把无操作性移植的代码拷贝过来，再往工程中添加操作系统的源码，操作系统的源码可以从我们对外发布的例程中获取，也可以从官网获取，此处以 FreeRTOS 为例进行移植操作。

首先将 FreeRTOS 源码拷贝到工程文件夹下，具体见

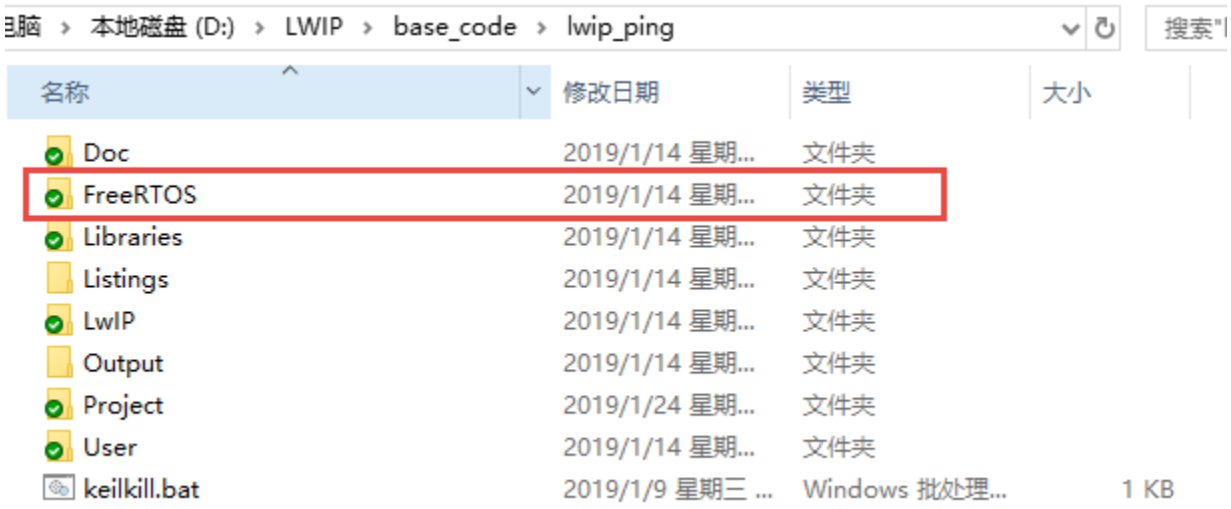


图 8-1 拷贝 FreeRTOS 源码到工程文件夹下

8.1.2 添加 FreeRTOS 源码到工程组文件夹

在上一步我们只是将 FreeRTOS 的源码放到了本地工程目录下，还没有添加到开发环境里面的组文件夹里面，FreeRTOS 也就没有移植到我们的工程中去。

接下来我们在开发环境里面新建 FreeRTOS/src 和 FreeRTOS/port 两个组文件夹，其中 FreeRTOS/src 用于存放 src 文件夹的所有内容，FreeRTOS/port 用于存放 portMemMang 文件夹与 portRVDSARM_CM？文件夹的内容，“？”表示 3、4 或者 7，具体选择哪个得看你使用的是野火哪个型号的 STM32 开发板，具体见表 8-1。

表 8-1 野火 STM32 开发板型号对应 FreeRTOS 的接口文件

野火 STM32 开发板型号	具体芯片型号	FreeRTOS 不同内核的接口文件
MINI	STM32F103RCT6	portRVDSARM_CM3
指南者	STM32F103VET6	
霸道	STM32F103ZET6	
霸天虎	STM32F407ZGT6	portRVDSARM_CM4
F429-挑战者	STM32F429IGT6	
F767-挑战者	STM32F767IGT6	portRVDSARM_CM7
H743-挑战者	STM32H743IIT6	

然后我们将工程文件中 FreeRTOS 的内容添加到工程中去，按照已经新建的分组添加我们的 FreeRTOS 工程源码。

在 FreeRTOS/port 分组中添加 MemMang 文件夹中的文件只需选择其中一个即可，我们选择“heap_4.c”，这是 FreeRTOS 的一个内存管理源码文件。同时，需要根据自己的开发板型号在 FreeRTOS\port\RVDS\ARM_CM? 中选择，“?”表示 3、4 或者 7，具体选择哪个得看你使用的是野火哪个型号的 STM32 开发板，具体见上表 8-1。

至此我们的 FreeRTOS 添加到工程中就已经完成，完成的效果具体见图 *8_2*。

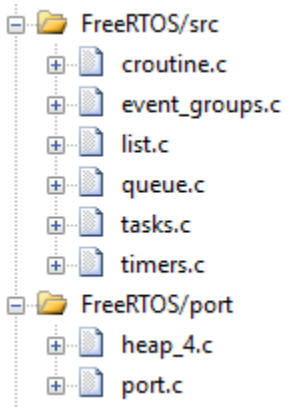


图 8-2 添加 FreeRTOS 源码到工程分组中

8.1.3 指定 FreeRTOS 头文件的路径

FreeRTOS 的源码已经添加到开发环境的组文件夹下面，编译的时候需要为这些源文件指定头文件的路径，不然编译会报错。FreeRTOS 的源码里面只有 FreeRTOS\include 和 FreeRTOS\port\RVDS\ARM_CM 这两个文件夹下面有头文件，只需要将这两个头文件的路径在开发环境里面指定即可。同时我们还将 FreeRTOSConfig.h 这个头文件拷贝到了工程根目录下的 user 文件夹下，所以 user 的路径也要加到开发环境里面。FreeRTOS 头文件的路径添加完成后的效果具体见图 *8_3*。

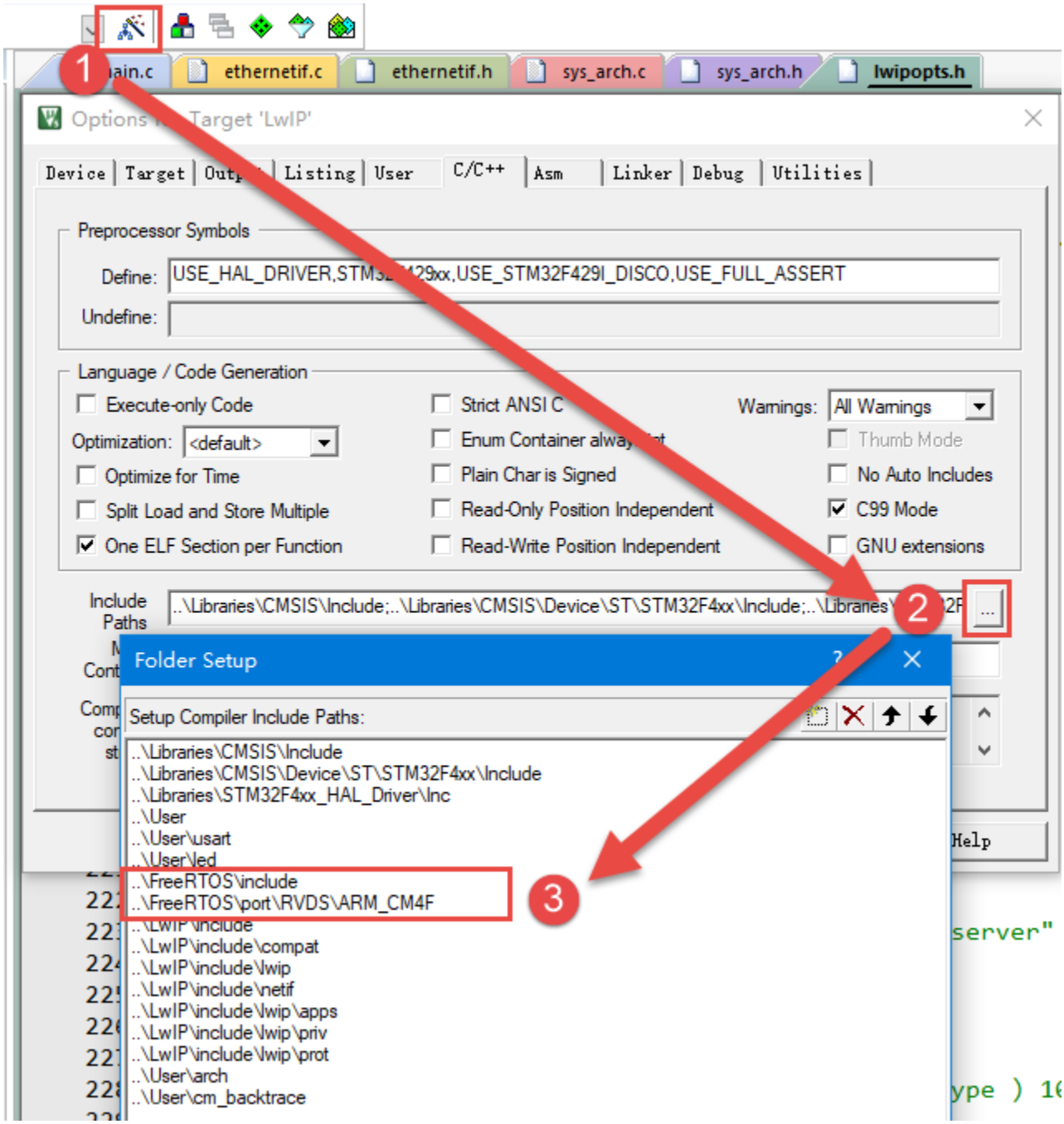


图 8-3 在开发环境中指定 FreeRTOS 的头文件的路径

至此，FreeRTOS 的整体工程基本移植完毕，我们也不需要修改 FreeRTOS 配置文件，直接使用我们 FreeRTOS 例程中的头文件即可。

8.1.4 修改 stm32f4xx_it.c

SysTick 中断服务函数是一个非常重要的函数，FreeRTOS 所有跟时间相关的事情都在里面处理，SysTick 就是 FreeRTOS 的一个心跳时钟，驱动着 FreeRTOS 的运行，就像人的心跳一样，假如没有心跳，我们就相当于“死了”，同样的，FreeRTOS 没有了心跳，那么它就会卡死在某个地方，不能进行任务调度，不能运行任何的东西，因此我们需要实现一个 FreeRTOS 的心跳时钟，FreeRTOS 帮我们实现了 SysTick 的启动的配置：在 port.c 文件中已经实现 vPortSetupTimerInterrupt() 函数，并且 FreeRTOS 通用的 SysTick 中断服务函数也实现了：在 port.c 文件中已经实现 xPortSysTickHandler() 函数，所以移植的时候只需要我们在 stm32f4xx_it.c 文件中实现我们对应（STM32）平台上的 SysTick_Handler() 函数即可。FreeRTOS 为开发者考虑得特别多，PendSV_Handler() 与 SVC_Handler() 这两个很重要的函数都帮我们实现了，在在 port.c 文件中已经实现 xPortPendSVHandler() 与 vPortSVCHandler() 函数，防止我们自己实现不了，那么在 stm32f4xx_it.c 中就需要我们注释掉或者删除掉 PendSV_Handler() 与 SVC_Handler() 这两个函数了，具体实现见*代码清单 8_1* 加粗部分。

代码清单 8-1stm32f4xx_it.c 文件内容

```
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "stm32f4xx_it.h"
#include "./usart/bsp_debug_usart.h"

#include "FreeRTOS.h"          //FreeRTOS 使用
#include "task.h"

/** @addtogroup STM32F4xx_HAL_Examples
* @{
*/

/** @addtogroup GPIO_EXTI
* @{
*/
```

(下页继续)

(续上页)

```
/* Private typedef -----------------------------------------------*/
/* Private define ------------------------------------------------*/
/* Private macro -------------------------------------------------*/
/* Private variables ---------------------------------------------*/
/* Private function prototypes -----------------------------------*/
/* Private functions ---------------------------------------------*/

/******************************************************************/
/*          Cortex-M4 Processor Exceptions Handlers               */
/******************************************************************/

/**
* @brief  This function handles NMI exception.
* @param  None
* @retval None
*/
void NMI_Handler(void)
{
}

/**
* @brief  This function handles Hard Fault exception.
* @param  None
* @retval None
*/
//void HardFault_Handler(void)
//{
//  /* Go to infinite loop when Hard Fault exception occurs */
//  while (1)
//  {
//  }
//}
```

(下页继续)

(续上页)

```
/**
* @brief  This function handles Memory Manage exception.
* @param  None
* @retval None
*/
void MemManage_Handler(void)
{
    /* Go to infinite loop when Memory Manage exception occurs */
    while (1)
    {
    }
}

/**
* @brief  This function handles Bus Fault exception.
* @param  None
* @retval None
*/
void BusFault_Handler(void)
{
    /* Go to infinite loop when Bus Fault exception occurs */
    while (1)
    {
    }
}

/**
* @brief  This function handles Usage Fault exception.
* @param  None
* @retval None
*/
```

(下页继续)

(续上页)

```
void UsageFault_Handler(void)
{
    /* Go to infinite loop when Usage Fault exception occurs */
    while (1)
    {
    }
}

/**
* @brief  This function handles Debug Monitor exception.
* @param  None
* @retval None
*/
void DebugMon_Handler(void)
{
}

/**
* @brief  This function handles SysTick Handler.
* @param  None
* @retval None
*/
extern void xPortSysTickHandler(void);

void SysTick_Handler(void)
{
    uint32_t ulReturn;
    /* 进入临界段，临界段可以嵌套 */
    ulReturn = taskENTER_CRITICAL_FROM_ISR();

    HAL_IncTick();
#if (INCLUDE_xTaskGetSchedulerState  == 1 )
```

(下页继续)

(续上页)

```
    if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
    {
#endif  /* INCLUDE_xTaskGetSchedulerState */
        xPortSysTickHandler();
#if (INCLUDE_xTaskGetSchedulerState  == 1 )
    }
#endif  /* INCLUDE_xTaskGetSchedulerState */

    /* 退出临界段 */
    taskEXIT_CRITICAL_FROM_ISR( ulReturn );
}
```

至此，将 FreeRTOS 添加到 LwIP 裸机工程中的步骤就基本完成了，编译的时候就基本也不会有错，下面就正式使用这个工程进行 LwIP 与操作系统的移植，因为还需要根据操作系统的特性修改很多接口文件。

8.2 lwipopts.h 文件需要加入的配置

在前面的章节也说了 lwipopts.h 文件的作用，而此刻在操作系统中移植，我们首先要将添加了操作系统的工程拿过来，把 lwipopts.h 文件修改一下，该文件最重要的宏定义就是 NO_SYS，我们把它定义为 0 就表示使用操作系统，当然，在使用操作系统的时候我们一般都会使用 NETCONN API 与 Socket API 编程，那么就需要将宏 LWIP_NETCONN 与 LWIP_SOCKET 定义为 1，表示使能这两种 API 编程，lwipopts.h 简单修改一下即可，然后再添加一下线程运行的一些宏定义，必须修改的部分具体见*代码清单 8_2* 加粗部分，而其他宏定义是根据实际情况进行修改即可。

代码清单 8-2 lwipopts.h 文件

```
#ifndef __LWIPOPTS_H__
#define __LWIPOPTS_H__

/**
```

(下页继续)

(续上页)

```
* SYS_LIGHTWEIGHT_PROT==1: if you want inter-task protection for certain
* critical regions during buffer allocation, deallocation and memory
* allocation and deallocation.
*/
#define SYS_LIGHTWEIGHT_PROT    1

/**
* NO_SYS==1: Provides VERY minimal functionality. Otherwise,
* use lwIP facilities.
*/
#define NO_SYS                  0

/**
* NO_SYS_NO_TIMERS==1: Drop support for sys_timeout when NO_SYS==1
* Mainly for compatibility to old versions.
*/
#define NO_SYS_NO_TIMERS        0

/* ---------- Memory options ---------- */
/* MEM_ALIGNMENT: should be set to the alignment of the CPU for which
lwIP is compiled. 4 byte alignment -> define MEM_ALIGNMENT to 4, 2
byte alignment -> define MEM_ALIGNMENT to 2. */
#define MEM_ALIGNMENT           4

/* MEM_SIZE: the size of the heap memory. If the application will send
a lot of data that needs to be copied, this should be set high. */
#define MEM_SIZE                (15*1024)

/* MEMP_NUM_PBUF: the number of memp struct pbufs. If the application
sends a lot of data out of ROM (or other static memory), this
should be set high. */
#define MEMP_NUM_PBUF           25
```

(下页继续)

(续上页)

```
/* MEMP_NUM_UDP_PCB: the number of UDP protocol control blocks. One
per active UDP "connection". */
#define MEMP_NUM_UDP_PCB        4
/* MEMP_NUM_TCP_PCB: the number of simulatenously active TCP
connections. */
#define MEMP_NUM_TCP_PCB        6
/* MEMP_NUM_TCP_PCB_LISTEN: the number of listening TCP
connections. */
#define MEMP_NUM_TCP_PCB_LISTEN 6
/* MEMP_NUM_TCP_SEG: the number of simultaneously queued TCP
segments. */
#define MEMP_NUM_TCP_SEG        150
/* MEMP_NUM_SYS_TIMEOUT: the number of simulateously active
timeouts. */
#define MEMP_NUM_SYS_TIMEOUT    6


/* ---------- Pbuf options ---------- */
/* PBUF_POOL_SIZE: the number of buffers in the pbuf pool. */
#define PBUF_POOL_SIZE          45
/* PBUF_POOL_BUFSIZE: the size of each pbuf in the pbuf pool. */
#define PBUF_POOL_BUFSIZE    \
LWIP_MEM_ALIGN_SIZE(TCP_MSS+40+PBUF_LINK_ENCAPSULATION_HLEN+PBUF_LINK_HLEN)

/* ---------- TCP options ---------- */
#define LWIP_TCP                1
#define TCP_TTL                 255

/* Controls if TCP should queue segments that arrive out of
order. Define to 0 if your device is low on memory. */
#define TCP_QUEUE_OOSEQ         0
```

(下页继续)

(续上页)

```
/* TCP Maximum segment size. */
#define TCP_MSS                   (1500 - 40)

/* TCP sender buffer space (bytes). */
#define TCP_SND_BUF               (10*TCP_MSS)

/*  TCP_SND_QUEUELEN: TCP sender buffer space (pbufs). This must be at␣
↪least
as much as (2 * TCP_SND_BUF/TCP_MSS) for things to work. */

#define TCP_SND_QUEUELEN          (8* TCP_SND_BUF/TCP_MSS)

/* TCP receive window. */
#define TCP_WND                   (11*TCP_MSS)


/* ---------- ICMP options ---------- */
#define LWIP_ICMP                           1


/* ---------- DHCP options ---------- */
/* Define LWIP_DHCP to 1 if you want DHCP configuration of
interfaces. DHCP is not implemented in lwIP 0.5.1, however, so
turning this on does currently not work. */
#define LWIP_DHCP                 1


/* ---------- UDP options ---------- */
#define LWIP_UDP                  1
#define UDP_TTL                   255
```

(下页继续)

(续上页)

```
/* ---------- Statistics options ---------- */
#define LWIP_STATS 0
#define LWIP_PROVIDE_ERRNO 1

/* ---------- link callback options ---------- */
/* LWIP_NETIF_LINK_CALLBACK==1: Support a callback function from an␣
↪interface
* whenever the link changes (i.e., link down)
*/
#define LWIP_NETIF_LINK_CALLBACK        0
/*
    ----------------------------------------
    ---------- Checksum options ----------
    ----------------------------------------
*/

/* The STM32F4x7 allows computing and verifying the IP,
    UDP, TCP and ICMP checksums by hardware:
- To use this feature let the following define uncommented.
- To disable it and process by CPU comment the  the checksum.
*/
#define CHECKSUM_BY_HARDWARE

#ifdef CHECKSUM_BY_HARDWARE
/* CHECKSUM_GEN_IP==0: Generate checksums by hardware for outgoing IP␣
↪packets.*/
#define CHECKSUM_GEN_IP                 0
/* CHECKSUM_GEN_UDP==0: Generate checksums by hardware for outgoing UDP␣
↪packets.*/
#define CHECKSUM_GEN_UDP                0
/* CHECKSUM_GEN_TCP==0: Generate checksums by hardware for outgoing TCP␣
↪packets.*/
```

(下页继续)

(续上页)

```
#define CHECKSUM_GEN_TCP                0
/* CHECKSUM_CHECK_IP==0: Check checksums by hardware for incoming IP␣
↪packets.*/
#define CHECKSUM_CHECK_IP               0
/* CHECKSUM_CHECK_UDP==0: Check checksums by hardware for incoming UDP␣
↪packets.*/
#define CHECKSUM_CHECK_UDP              0
/* CHECKSUM_CHECK_TCP==0: Check checksums by hardware for incoming TCP␣
↪packets.*/
#define CHECKSUM_CHECK_TCP              0
/*CHECKSUM_CHECK_ICMP==0: Check checksums by hardware for incoming ICMP␣
↪packets.*/
#define CHECKSUM_GEN_ICMP               0
#else
/* CHECKSUM_GEN_IP==1: Generate checksums in software for outgoing IP␣
↪packets.*/
#define CHECKSUM_GEN_IP                 1
/* CHECKSUM_GEN_UDP==1: Generate checksums in software for outgoing UDP␣
↪packets.*/
#define CHECKSUM_GEN_UDP                1
/* CHECKSUM_GEN_TCP==1: Generate checksums in software for outgoing TCP␣
↪packets.*/
#define CHECKSUM_GEN_TCP                1
/* CHECKSUM_CHECK_IP==1: Check checksums in software for incoming IP␣
↪packets.*/
#define CHECKSUM_CHECK_IP               1
/* CHECKSUM_CHECK_UDP==1: Check checksums in software for incoming UDP␣
↪packets.*/
#define CHECKSUM_CHECK_UDP              1
/* CHECKSUM_CHECK_TCP==1: Check checksums in software for incoming TCP␣
↪packets.*/
#define CHECKSUM_CHECK_TCP              1
```

(下页继续)

(续上页)

```
/*CHECKSUM_CHECK_ICMP==1: Check checksums by hardware for incoming ICMP␣
↪packets.*/
#define CHECKSUM_GEN_ICMP               1
#endif

/*
    ----------------------------------------------
    ---------- Sequential layer options ----------
    ----------------------------------------------
*/
/**
* LWIP_NETCONN==1: Enable Netconn API (require to use api_lib.c)
*/
#define LWIP_NETCONN                    1

/*
    ----------------------------------
    ---------- Socket options ----------
    ----------------------------------
*/
/**
* LWIP_SOCKET==1: Enable Socket API (require to use sockets.c)
*/
#define LWIP_SOCKET                     1

/*
    ---------------------------------
    ---------- OS options ----------
    ---------------------------------
*/

#define DEFAULT_UDP_RECVMBOX_SIZE       10
```

(下页继续)

(续上页)

```
#define DEFAULT_TCP_RECVMBOX_SIZE       10
#define DEFAULT_ACCEPTMBOX_SIZE         10
#define DEFAULT_THREAD_STACKSIZE        1024

#define TCPIP_THREAD_NAME              "lwip"
#define TCPIP_THREAD_STACKSIZE          512
#define TCPIP_MBOX_SIZE                 8
#define TCPIP_THREAD_PRIO               3
```

8.3 sys_arch.c/h 文件的编写

操作系统环境下，LwIP 移植的核心就是编写与操作系统相关的接口文件 sys_arch.c 和 sys_arch.h，这两个文件可以自己创建也可以从 contrib 包中获取，路径分别为“contrib-2.1.0\ports\freertos”与“contrib-2.1.0\ports\freertos\includearch”，用户在移植的时候必须根据操作系统的功能为协议栈提供相应的接口，如邮箱（因为本次移植以 FreeRTOS 为例子，FreeRTOS 中没有邮箱这种概念，但是可以使用消息队列替代，为了迎合 LwIP 中的命名，下文统一采用邮箱表示）、信号量、互斥量等，这些 IPC 通信机制是保证内核与上层 API 接口通信的基本保障，也是内核实现管理的继承，同时在 sys.h 文件中声明了用户需要实现的所有函数框架，这些函数具体见表格 8-2。

表格 8-2 需要用户实现的函数

名称	属性	功能	所在文件
sys_sem_t	数据类型	指针类型，指向系统信号量	sys_arch.h
sys_mutex_t	数据类型	指针类型，指向系统互斥量	sys_arch.h
sys_mbox_t	数据类型	指针类型，指向系统邮箱	sys_arch.h
sys_thread_t	数据类型	指针类型，指向系统任务	sys_arch.h
SYS_MBOX_NULL	宏定义	系统邮箱的空值	sys_arch.h
SYS_SEM_NULL	宏定义	系统信号量的空值	sys_arch.h
SYS_MRTEX_NULL	宏定义	系统互斥量的空值	sys_arch.h

下页继续

表 1 – 续上页

名称	属性	功能	所在文件
sys_now	函数	内核时钟	sys_arch.c
sys_init	函数	初始化系统	sys_arch.c
sys_arch_protect	函数	进入临界段	sys_arch.c
sys_arch_unprotect	函数	退出临界段	sys_arch.c
sys_sem_new	函数	创建一个信号量	sys_arch.c
sys_sem_free	函数	删除一个信号量	sys_arch.c
sys_sem_valid	函数	判断信号量是否有效	sys_arch.c
sys_sem_set_invalid	函数	将信号量设置无效状态	sys_arch.c
sys_arch_sem_wait	函数	等待一个信号量	sys_arch.c
sys_sem_signal	函数	释放一个信号量	sys_arch.c
sys_mutex_new	函数	创建一个互斥量	sys_arch.c
sys_mutex_free	函数	删除一个互斥量	sys_arch.c
sys_mutex_set_invalid	函数	设置互斥量为无效状态	sys_arch.c
sys_mutex_lock	函数	获取一个互斥量	sys_arch.c
sys_mutex_unlock	函数	释放一个互斥量	sys_arch.c
sys_mbox_new	函数	创建一个邮箱	sys_arch.c
sys_mbox_free	函数	删除一个邮箱	sys_arch.c
sys_mbox_valid	函数	判断邮箱是否有效	sys_arch.c
sys_mbox_set_invalid	函数	设置邮箱为无效状态	sys_arch.c
sys_mbox_post	函数	向邮箱发送消息，一直阻塞	sys_arch.c
sys_mbox_trypost	函数	向邮箱发送消息，非阻塞	sys_arch.c
sys_mbox_trypost_fromisr	函数	在中断中向邮箱发送消息	sys_arch.c
sys_arch_mbox_fetch	函数	从邮箱中获取消息，阻塞	sys_arch.c
sys_arch_mbox_tryfetch	函数	从邮箱中获取消息，非阻塞	sys_arch.c
sys_thread_new	函数	创建一个线程	sys_arch.c

看到那么多函数，是不是头都大了，其实这些函数的实现都是很简单的，首先讲解一下邮箱函数的实现。在 LwIP 中，用户代码与协议栈内部之间是通过邮箱进行数据的交互的，邮箱本质上就是一个指向数据的指针，API 将指针传递给内核，内核通过这个指针访问数据，然后去处理，反

之内核将数据传递给用户代码也是通过邮箱将一个指针进行传递。

在操作系统环境下，LwIP 会作为一个线程运行，线程的名字叫 tcpip_thread，在初始化 LwIP 的时候，内核就会自动创建这个线程，并且在线程运行的时候阻塞在邮箱上，等待数据进行处理，这个邮箱数据的来源可能在底层网卡接收到的数据或者上层应用程序的数据，总之，tcpip_thread 线程在获取到邮箱中的数据时候，就会退出阻塞态，去处理数据，在处理完毕数据后又进入阻塞态中等待数据的到来，如此反复。

信号量与互斥量的实现为内核提供同步与互斥的机制，比如当用户想要发送一个数据的时候，就会调用上层 API 接口，API 接口就会去先发送一个数据给内核去处理，然后尝试获取一个信号量，因为此时是没有信号量的，所以就会阻塞用户线程；内核在知道用户想要发送数据后，就会调用对应的网卡去发送数据，当数据发送完成后就释放一个信号量告知用户线程发送完成，这样子用户线程就得以继续执行。

所以这些函数的接口都必须由用户实现，下面具体看看这些函数的实现，具体见*代码清单 8_3*。

代码清单 8-3 sys_arch.c 文件内容

```
#include "debug.h"

#include <lwip/opt.h>
#include <lwip/arch.h>

#include "tcpip.h"
#include "lwip/init.h"
#include "lwip/netif.h"
#include "lwip/sio.h"
#include "ethernetif.h"

#if !NO_SYS
#include "sys_arch.h"
#endif
#include <lwip/stats.h>
#include <lwip/debug.h>
#include <lwip/sys.h>
```

(下页继续)

(续上页)

```
#include <string.h>

int errno;


u32_t lwip_sys_now;

struct sys_timeouts
{
    struct sys_timeo *next;
};

struct timeoutlist
{
    struct sys_timeouts timeouts;
    xTaskHandle pid;
};

#define SYS_THREAD_MAX 4

static struct timeoutlist s_timeoutlist[SYS_THREAD_MAX];

static u16_t s_nextthread = 0;

u32_t
sys_jiffies(void)
{
    lwip_sys_now = xTaskGetTickCount();
    return lwip_sys_now;
}
```

(下页继续)

(续上页)

```
u32_t
sys_now(void)
{
    lwip_sys_now = xTaskGetTickCount();
    return lwip_sys_now;
}

void
sys_init(void)
{
    int i;
    // Initialize the the per-thread sys_timeouts structures
    // make sure there are no valid pids in the list
    for (i = 0; i < SYS_THREAD_MAX; i++)
    {
        s_timeoutlist[i].pid = 0;
        s_timeoutlist[i].timeouts.next = NULL;
    }
    // keep track of how many threads have been created
    s_nextthread = 0;
}

struct sys_timeouts *sys_arch_timeouts(void)
{
    int i;
    xTaskHandle pid;
    struct timeoutlist *tl;
    pid = xTaskGetCurrentTaskHandle( );
    for (i = 0; i < s_nextthread; i++)
    {
        tl = &(s_timeoutlist[i]);
        if (tl->pid == pid)
```

(下页继续)

(续上页)

```
        {
            return &(tl->timeouts);
        }
    }
    return NULL;
}

sys_prot_t sys_arch_protect(void)
{
    vPortEnterCritical();           //进入临界段
    return 1;
}

void sys_arch_unprotect(sys_prot_t pval)
{
    ( void ) pval;
    vPortExitCritical();        //退出临界段
}

#if !NO_SYS

err_t
sys_sem_new(sys_sem_t *sem, u8_t count)
{
    /* 创建 sem */
    if (count <= 1)
    {
        *sem = xSemaphoreCreateBinary();        //创建二值信号量
        if (count == 1)
        {
            sys_sem_signal(*sem);   //新创建的信号量是无效的，需要释放一个信号量
        }
```

(下页继续)

(续上页)

```
    }
    else
        *sem = xSemaphoreCreateCounting(count,count); //创建计数信号量

#if SYS_STATS
    ++lwip_stats.sys.sem.used;
    if (lwip_stats.sys.sem.max < lwip_stats.sys.sem.used)
    {
        lwip_stats.sys.sem.max = lwip_stats.sys.sem.used;
    }
#endif /* SYS_STATS */

    if (*sem != SYS_SEM_NULL)
        return ERR_OK;          //创建成功返回 ERR_OK
    else
    {
#if SYS_STATS
        ++lwip_stats.sys.sem.err;
#endif /* SYS_STATS */
        printf("[sys_arch]:new sem fail!\n");
        return ERR_MEM;
    }
}

void
sys_sem_free(sys_sem_t *sem)
{
#if SYS_STATS
    --lwip_stats.sys.sem.used;
#endif /* SYS_STATS */
    /* 删除 sem */
    vSemaphoreDelete(*sem);    //删除一个信号量
```

(下页继续)

(续上页)

```
    *sem = SYS_SEM_NULL;  //删除之后置空
}

int sys_sem_valid(sys_sem_t *sem)
{
    return (*sem != SYS_SEM_NULL);    //返回信号量是否有效
}

void
sys_sem_set_invalid(sys_sem_t *sem)
{
    *sem = SYS_SEM_NULL;     //信号量设置为无效
}

/*
如果 timeout 参数不为零，则返回值为
等待信号量所花费的毫秒数。如果
信号量未在指定时间内发出信号，返回值为
SYS_ARCH_TIMEOUT。如果线程不必等待信号量
该函数返回零。 */
u32_t
sys_arch_sem_wait(sys_sem_t *sem, u32_t timeout)
{
    u32_t wait_tick = 0;
    u32_t start_tick = 0 ;

    //看看信号量是否有效
    if (*sem == SYS_SEM_NULL)
        return SYS_ARCH_TIMEOUT;

    //首先获取开始等待信号量的时钟节拍
    start_tick = xTaskGetTickCount();
```

(下页继续)

(续上页)

```
    //timeout != 0, 需要将 ms 换成系统的时钟节拍
    if (timeout != 0)
    {
        //将 ms 转换成时钟节拍
        wait_tick = timeout / portTICK_PERIOD_MS;
        if (wait_tick == 0)
            wait_tick = 1;
    }
    else
        wait_tick = portMAX_DELAY;  //一直阻塞

    //等待成功, 计算等待的时间, 否则就表示等待超时
    if (xSemaphoreTake(*sem, wait_tick) == pdTRUE)
        return ((xTaskGetTickCount()-start_tick)*portTICK_RATE_MS);
    else
        return SYS_ARCH_TIMEOUT;
}

void
sys_sem_signal(sys_sem_t *sem)
{
    if (xSemaphoreGive( *sem ) != pdTRUE)   //释放信号量
        printf("[sys_arch]:sem signal fail!\n");
}

err_t
sys_mutex_new(sys_mutex_t *mutex)
{
    /* 创建 sem */
    *mutex = xSemaphoreCreateMutex(); //创建互斥量
    if (*mutex != SYS_MRTEX_NULL)
```

(下页继续)

(续上页)

```
        return ERR_OK;       //创建成功返回 ERR_OK
    else
    {
        printf("[sys_arch]:new mutex fail!\n");
        return ERR_MEM;
    }
}

void
sys_mutex_free(sys_mutex_t *mutex)
{
    vSemaphoreDelete(*mutex);    //删除互斥量
}

void
sys_mutex_set_invalid(sys_mutex_t *mutex)
{
    *mutex = SYS_MRTEX_NULL;     //设置互斥量为无效
}

void
sys_mutex_lock(sys_mutex_t *mutex)
{
    xSemaphoreTake(*mutex,/* 互斥量句柄 */
                   portMAX_DELAY); /* 等待时间 */
}

void
sys_mutex_unlock(sys_mutex_t *mutex)
{
    xSemaphoreGive( *mutex );//给出互斥量
}
```

(下页继续)

(续上页)

```
sys_thread_t
sys_thread_new(const char *name, lwip_thread_fn function,
        void *arg, int stacksize, int prio)
{
    sys_thread_t handle = NULL;
    BaseType_t xReturn = pdPASS;
    /* 创建一个线程 */
    xReturn = xTaskCreate((TaskFunction_t )function,  /* 线程入口函数 */
                          (const char*    )name,/* 线程名字 */
                          (uint16_t       )stacksize,  /* 线程栈大小 */
                          (void*          )arg,/* 线程入口函数参数 */
                          (UBaseType_t    )prio, /* 线程的优先级 */
                          (TaskHandle_t*  )&handle);/* 线程控制块指针 */
    if (xReturn != pdPASS)
    {
        printf("[sys_arch]:create task fail!err:%#lx\n",xReturn);
        return NULL;
    }
    return handle;
}

err_t
sys_mbox_new(sys_mbox_t *mbox, int size)
{
    /* 创建一个邮箱 */
    *mbox = xQueueCreate((UBaseType_t ) size,/* 邮箱的长度 */
                         (UBaseType_t ) sizeof(void *));/* 消息的大小 */
#if SYS_STATS
    ++lwip_stats.sys.mbox.used;
    if (lwip_stats.sys.mbox.max < lwip_stats.sys.mbox.used)
    {
```

(下页继续)

(续上页)

```
        lwip_stats.sys.mbox.max = lwip_stats.sys.mbox.used;
    }
#endif /* SYS_STATS */
    if (NULL == *mbox)
        return ERR_MEM;    // 创建成功返回 ERR_OK

    return ERR_OK;
}

void
sys_mbox_free(sys_mbox_t *mbox)
{
    if ( uxQueueMessagesWaiting( *mbox ) )
    {
        /* Line for breakpoint.  Should never break here! */
        portNOP();
#if SYS_STATS
        lwip_stats.sys.mbox.err++;
#endif /* SYS_STATS */
    }

    vQueueDelete(*mbox);     //删除一个邮箱

#if SYS_STATS
    --lwip_stats.sys.mbox.used;
#endif /* SYS_STATS */
}

int sys_mbox_valid(sys_mbox_t *mbox)
{
    if (*mbox == SYS_MBOX_NULL)      //判断邮箱是否有效
        return 0;
```

(下页继续)

(续上页)

```
    else
        return 1;
}

void
sys_mbox_set_invalid(sys_mbox_t *mbox)
{
    *mbox = SYS_MBOX_NULL;  //设置有效为无效状态
}

void
sys_mbox_post(sys_mbox_t *q, void *msg)
{
    while (xQueueSend( *q, /* 邮箱的句柄 */
                       &msg,/* 发送的消息内容 */
                       portMAX_DELAY) != pdTRUE); /* 等待时间 */
}

err_t
sys_mbox_trypost(sys_mbox_t *q, void *msg)
{
    if (xQueueSend(*q,&msg,0) == pdPASS) //尝试发送一个消息，非阻塞发送
        return ERR_OK;
    else
        return ERR_MEM;
}

err_t
sys_mbox_trypost_fromisr(sys_mbox_t *q, void *msg)
{
    uint32_t ulReturn;
    err_t err = ERR_MEM;
```

(下页继续)

(续上页)

```
    BaseType_t pxHigherPriorityTaskWoken;

    /* 进入临界段，临界段可以嵌套 */
    ulReturn = taskENTER_CRITICAL_FROM_ISR();

    if (xQueueSendFromISR(*q,&msg,&pxHigherPriorityTaskWoken)==pdPASS)
    {
        err = ERR_OK;
    }
    //如果需要的话进行一次线程切换
    portYIELD_FROM_ISR(pxHigherPriorityTaskWoken);

    /* 退出临界段 */
    taskEXIT_CRITICAL_FROM_ISR( ulReturn );

    return err;
}

u32_t
sys_arch_mbox_fetch(sys_mbox_t *q, void **msg, u32_t timeout)
{
    void *dummyptr;
    u32_t wait_tick = 0;
    u32_t start_tick = 0 ;

    if ( msg == NULL )  //看看存储消息的地方是否有效
        msg = &dummyptr;

    //首先获取开始等待信号量的时钟节拍
    start_tick = sys_now();

    //timeout != 0, 需要将 ms 换成系统的时钟节拍
```

(下页继续)

(续上页)

```
    if (timeout != 0)
    {
        //将 ms 转换成时钟节拍
        wait_tick = timeout / portTICK_PERIOD_MS;
        if (wait_tick == 0)
            wait_tick = 1;
    }
    //一直阻塞
    else
        wait_tick = portMAX_DELAY;

    //等待成功，计算等待的时间，否则就表示等待超时
    if (xQueueReceive(*q,&(*msg), wait_tick) == pdTRUE)
        return ((sys_now() - start_tick)*portTICK_PERIOD_MS);
    else
    {
        *msg = NULL;
        return SYS_ARCH_TIMEOUT;
    }
}

u32_t
sys_arch_mbox_tryfetch(sys_mbox_t *q, void **msg)
{
    void *dummyptr;
    if ( msg == NULL )
        msg = &dummyptr;

    //等待成功，计算等待的时间
    if (xQueueReceive(*q,&(*msg), 0) == pdTRUE)
        return ERR_OK;
    else
```

(下页继续)

(续上页)

```
        return SYS_MBOX_EMPTY;
}

#if LWIP_NETCONN_SEM_PER_THREAD
#error LWIP_NETCONN_SEM_PER_THREAD==1 not supported
#endif /* LWIP_NETCONN_SEM_PER_THREAD */

#endif /* !NO_SYS */
```

这些函数都是对操作系统的 IPC 通信机制进行简单的封装，在这里用户只需要稍微注意一下 sys_arch_sem_wait() 函数与 sys_arch_mbox_fetch() 函数，因为 LwIP 中使用的时间是以毫秒（ms）为单位的，而操作系统中则以时钟节拍（tick）为单位，那么在返回等待信号量或者邮箱所使用的时间就是要转换成 ms，而操作系统并未提供等待这些信息的时间，那么我们可以使用一个折中的方法，在获取的时候开始记录时间戳，在获取结束后再次记录一次时间戳，两次时间戳相减就得到等待的时间，但是需要将这些时间（tick）转换为毫秒，这种做法当然是不精确的，但是对 LwIP 来说影响不大。

8.4 网卡底层的编写

在无操作性移植的时候，我们的网卡收发数据就是单纯的收发数据，ethernetif_input() 函数就是处理接收网卡数据的，但是使用了操作系统的话，我们一般将接收数据函数独立成为一个网卡接收线程，这样子在收到数据的时候才去处理数据，然后递交给内核线程，所以我们只需要稍作修改即可，将函数转换成线程就行了，并且在初始化网卡的时候创建网卡接收线程。当然，我们也能将发送函数独立成一个线程，我们暂时没有必要去处理它，此处只创建一个网卡接收线程，具体见代码清单 8_4。

代码清单 8-4 网卡接收线程 ethernetif_input()

```
void ethernetif_input(void *pParams)
{
```

(下页继续)

(续上页)

```
    struct netif *netif;
    struct pbuf *p = NULL;
    netif = (struct netif*) pParams;
    LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input error\n"));

    while (1)
    {
        if (xSemaphoreTake( s_xSemaphore, portMAX_DELAY ) == pdTRUE)
        {
            /* move received packet into a new pbuf */
            taskENTER_CRITICAL();
            p = low_level_input(netif);
            taskEXIT_CRITICAL();
            /* points to packet payload, which starts with an Ethernet␣
↪header */
            if (p != NULL)
            {
                taskENTER_CRITICAL();
                /* full packet send to tcpip_thread to process */
                if (netif->input(p, netif) != ERR_OK)
                {
                LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input␣
↪error\n"));
                    pbuf_free(p);
                    p = NULL;
                }
                taskEXIT_CRITICAL();
            }
        }
    }
}
```

在网卡接收线程中需要留意一下以下内容：网卡接收线程是需要通过信号量机制去接收数据的，一般来说我们都是使用中断的方式去获取网络数据包，当产生中断的时候，我们一般不会在中断中处理数据，而是告诉对应的线程去处理，也就是我们的网卡接收线程去处理数据，那么就会通过信号量进行同步，当网卡接收到了数据就会产生中断释放一个信号量，然后线程从阻塞中恢复，去获取网卡的数据并且向上层递交。

当然我们还需要在中断中对网卡底层进行编写，具体见*代码清单 8_5*

代码清单 8-5 网卡中断处理（bsp_eth.c）

```
void ETH_IRQHandler(void)
{
    uint32_t ulReturn;
    /* 进入临界段，临界段可以嵌套 */
    ulReturn = taskENTER_CRITICAL_FROM_ISR();

    HAL_ETH_IRQHandler(&heth);

    /* 退出临界段 */
    taskEXIT_CRITICAL_FROM_ISR( ulReturn );
}


extern xSemaphoreHandle s_xSemaphore;
void HAL_ETH_RxCpltCallback(ETH_HandleTypeDef *heth)
{
//  LED2_TOGGLE;
    portBASE_TYPE xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR( s_xSemaphore, &xHigherPriorityTaskWoken );
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

此外我们还需要在网卡初始化的时候创建网卡接收线程与对应的信号量，网卡初始化函数还是 low_level_init() 函数，并且初始化的其他信息都未修改，只是添加了线程与信号量的创建，具体

见代码清单 8_6 加粗部分。

代码清单 8-6 网卡初始化函数 low_level_init()

```
static void low_level_init(struct netif *netif)
{
    HAL_StatusTypeDef hal_eth_init_status;

    //初始化 bsp—eth
    hal_eth_init_status = Bsp_Eth_Init();
    if (hal_eth_init_status == HAL_OK)
    {
        /* Set netif link flag */
        netif->flags |= NETIF_FLAG_LINK_UP;
    }

#if LWIP_ARP || LWIP_ETHERNET

    /* set MAC hardware address length */
    netif->hwaddr_len = ETH_HWADDR_LEN;

    /* set MAC hardware address */
    netif->hwaddr[0] =  heth.Init.MACAddr[0];
    netif->hwaddr[1] =  heth.Init.MACAddr[1];
    netif->hwaddr[2] =  heth.Init.MACAddr[2];
    netif->hwaddr[3] =  heth.Init.MACAddr[3];
    netif->hwaddr[4] =  heth.Init.MACAddr[4];
    netif->hwaddr[5] =  heth.Init.MACAddr[5];

    /* maximum transfer unit */
    netif->mtu = NETIF_MTU;

    /* Accept broadcast address and ARP traffic */
    /* don't set NETIF_FLAG_ETHARP if this device is not an ethernet one */
```

(下页继续)

(续上页)

```
#if LWIP_ARP
    netif->flags |= NETIF_FLAG_BROADCAST | NETIF_FLAG_ETHARP;
#else
    netif->flags |= NETIF_FLAG_BROADCAST;
#endif /* LWIP_ARP */

    /* USER CODE BEGIN PHY_PRE_CONFIG */

    s_xSemaphore = xSemaphoreCreateCounting(40,0);

    /* create the task that handles the ETH_MAC */
    sys_thread_new("ETHIN",
                   ethernetif_input,  /* 任务入口函数 */
                   netif,             /* 任务入口函数参数 */
                   NETIF_IN_TASK_STACK_SIZE,/* 任务栈大小 */
                   NETIF_IN_TASK_PRIORITY); /* 任务的优先级 */

#endif /* LWIP_ARP || LWIP_ETHERNET */

    /* Enable MAC and DMA transmission and reception */
    HAL_ETH_Start(&heth);
}
```

8.5 协议栈初始化

经过上面的移植，我们的底层与操作系统接口都基本移植完毕，想要让 LwIP 在操作系统中能跑起来，还需要最后一步，将协议栈进行初始化，前面我们也说了，内核在操作系统中是作为一个线程独立存在的，在初始化的时候，我们不仅要挂载网卡，也要创建 tcpip_thread() 线程，当然，这个线程 LwIP 会在初始的时候自动创建，而挂载网卡的内容与无操作系统是一样的，就无需过多修改，协议栈初始化的源码具体见*代码清单 8_7*。

代码清单 8-7 协议栈初始化

```
/*Static IP ADDRESS: IP_ADDR0.IP_ADDR1.IP_ADDR2.IP_ADDR3 */
#define IP_ADDR0                        192
#define IP_ADDR1                        168
#define IP_ADDR2                          1
#define IP_ADDR3                        122

/*NETMASK*/
#define NETMASK_ADDR0                   255
#define NETMASK_ADDR1                   255
#define NETMASK_ADDR2                   255
#define NETMASK_ADDR3                     0

/*Gateway Address*/
#define GW_ADDR0                        192
#define GW_ADDR1                        168
#define GW_ADDR2                          1
#define GW_ADDR3                          1
/* USER CODE END 0 */

struct netif gnetif;
ip4_addr_t ipaddr;
ip4_addr_t netmask;
ip4_addr_t gw;
uint8_t IP_ADDRESS[4];
uint8_t NETMASK_ADDRESS[4];
uint8_t GATEWAY_ADDRESS[4];

void LwIP_Init(void)
{

    tcpip_init(NULL, NULL);
```

(下页继续)

(续上页)

```
    /* IP addresses initialization */
    /* USER CODE BEGIN 0 */
#ifdef USE_DHCP
    ip_addr_set_zero_ip4(&ipaddr);
    ip_addr_set_zero_ip4(&netmask);
    ip_addr_set_zero_ip4(&gw);
#else
    IP4_ADDR(&ipaddr,IP_ADDR0,IP_ADDR1,IP_ADDR2,IP_ADDR3);
    IP4_ADDR(&netmask,NETMASK_ADDR0,NETMASK_ADDR1,NETMASK_ADDR2,NETMASK_
↪ADDR3);
    IP4_ADDR(&gw,GW_ADDR0,GW_ADDR1,GW_ADDR2,GW_ADDR3);
#endif /* USE_DHCP */
    /* USER CODE END 0 */
    /* Initilialize the LwIP stack without RTOS */
    /* add the network interface (IPv4/IPv6) without RTOS */
    netif_add(&gnetif, &ipaddr, &netmask, &gw, NULL,
            &ethernetif_init, &tcpip_input);

    /* Registers the default network interface */
    netif_set_default(&gnetif);
    if (netif_is_link_up(&gnetif))
    {
    /* When the netif is fully configured this function must be called */
        netif_set_up(&gnetif);
    }
    else
    {
        /* When the netif link is down this function must be called */
        netif_set_down(&gnetif);
    }
```

(下页继续)

(续上页)

```
}
```

在 tcpip_init() 函数中，LwIP 会调用 lwip_init() 将内核进行初始化，并且创建一个 tcpip_mbox 邮箱，邮箱的大小是 TCPIP_MBOX_SIZE，用于接收从底层或者上层传递过来的消息，并且最重要的是创建一个 tcpip_thread 线程，这就是 LwIP 在操作系统中作为一个独立的线程运行，所有处理的数据都要这个线程去处理，这个线程我们会在后续讲解。lwip_init() 函数的源码具体见代码清单 8_8。

代码清单 8-8 lwip_init() 源码

```
void
tcpip_init(tcpip_init_done_fn initfunc, void *arg)
{
    lwip_init();

    tcpip_init_done = initfunc;
    tcpip_init_done_arg = arg;
    if (sys_mbox_new(&tcpip_mbox, TCPIP_MBOX_SIZE) != ERR_OK)
    {
        LWIP_ASSERT("failed to create tcpip_thread mbox", 0);
    }
#if LWIP_TCPIP_CORE_LOCKING
    if (sys_mutex_new(&lock_tcpip_core) != ERR_OK)
    {
        LWIP_ASSERT("failed to create lock_tcpip_core", 0);
    }
#endif /* LWIP_TCPIP_CORE_LOCKING */

    sys_thread_new(TCPIP_THREAD_NAME, tcpip_thread, NULL,
                TCPIP_THREAD_STACKSIZE, TCPIP_THREAD_PRIO);
}
```

对于代码清单 8-7 中 netif_add() 函数，与裸机移植的时候有一点细微的差别，它最后一个参数就

是 tcpip_input，是一个函数，而不是原来的 ethernet_input()，因为 tcpip_input() 函数会将网卡收到的数据包打包成为一个消息，发送到 tcpip_mbox 邮箱中，传递给 tcpip_thread 线程去处理，不过本质上也是调用 ethernet_input() 函数去递交这个数据包，只不过是绕了一大圈，因此，想要 LwIP 能正常运行，消息机制是不可或缺的，关于这个运作机制我们在后文详细讲解。

8.6 移植后使用 ping 测试基本响应

在移植完操作系统与 LwIP，我们也应该用电脑去 ping 一下开发版，看看是否能 ping 通，如果通了就表明我们的移植是正常的，那么我们才能继续去用 LwIP 完成更高级的应用，ping 结果具体见图 *8_4*。

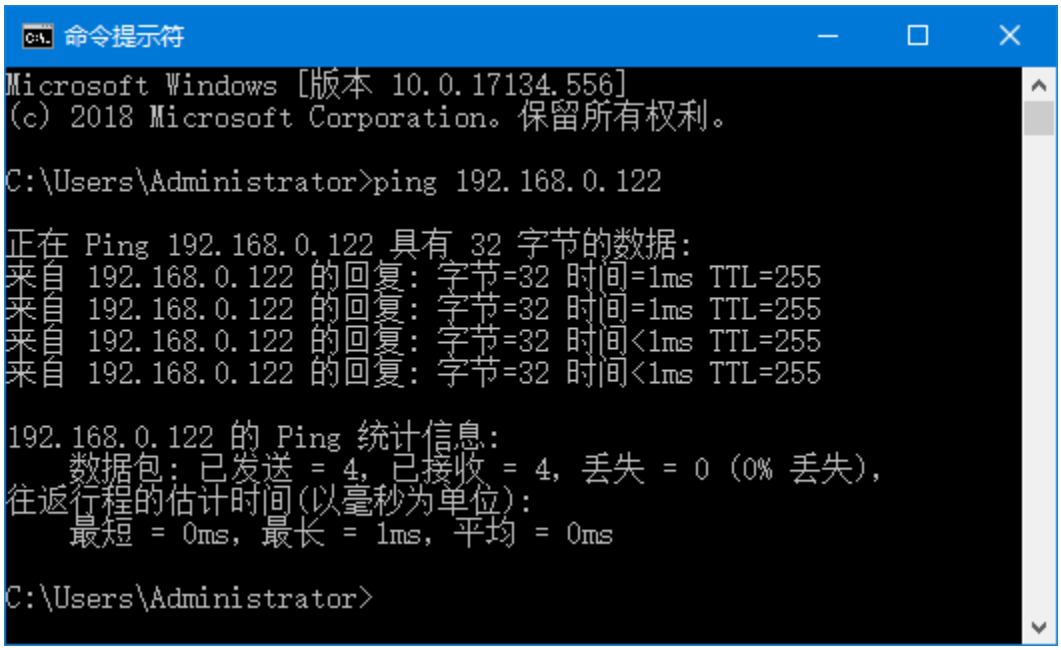


图 8-4ping 响应结果

第 9 章　LwIP 一探究竟

可能很多同学看完了前面的章节，是不是感觉像是在云里雾里一样，完全没有头绪，不得不说中文是很棒的语言，所有的文字都认识，但是连在一起就不知道说什么了。其实我也是过来人，刚开始研究 LwIP 的时候，也是不知所云，但是，当你对 LwIP 有一个框架的时候，这些都不是事。

而本章，就是让你们对 LwIP 有一个框架，让你们知道网卡中的数据是怎么传递到 LwIP 内核中的，然后 LwIP 内核又是怎么处理这些数据的，本章会简化这些流程，看完本章，你们就会惊叹，原来 LwIP 的处理是那么的优秀！

注意：由于不推荐使用 RAW API 编程，所以我们只会讲解 LwIP 在操作系统环境下网卡数据传递到内核的运作流程。

9.1 网卡接收数据的流程

网卡是怎么接收数据的，我想也无需多说了，基本就是开发板上 eth 接收完数据后产生一个中断，然后释放一个信号量通知网卡接收线程去处理这些接收的数据，然后将数据这些数据封装成消息，投递到 tcpip_mbox 邮箱中，LwIP 内核线程得到这个消息，就对消息进行解析，根据消息中数据包类型进行处理，实际上是调用 ethernet_input() 函数决定是否递交到 IP 层，如果是 ARP 包，内核就不会递交给 IP 层，而是更新 ARP 缓存表，对于 IP 数据包则递交给 IP 层去处理，这就是一个数据从网卡到内核的过程，具体见图 *9_1*。

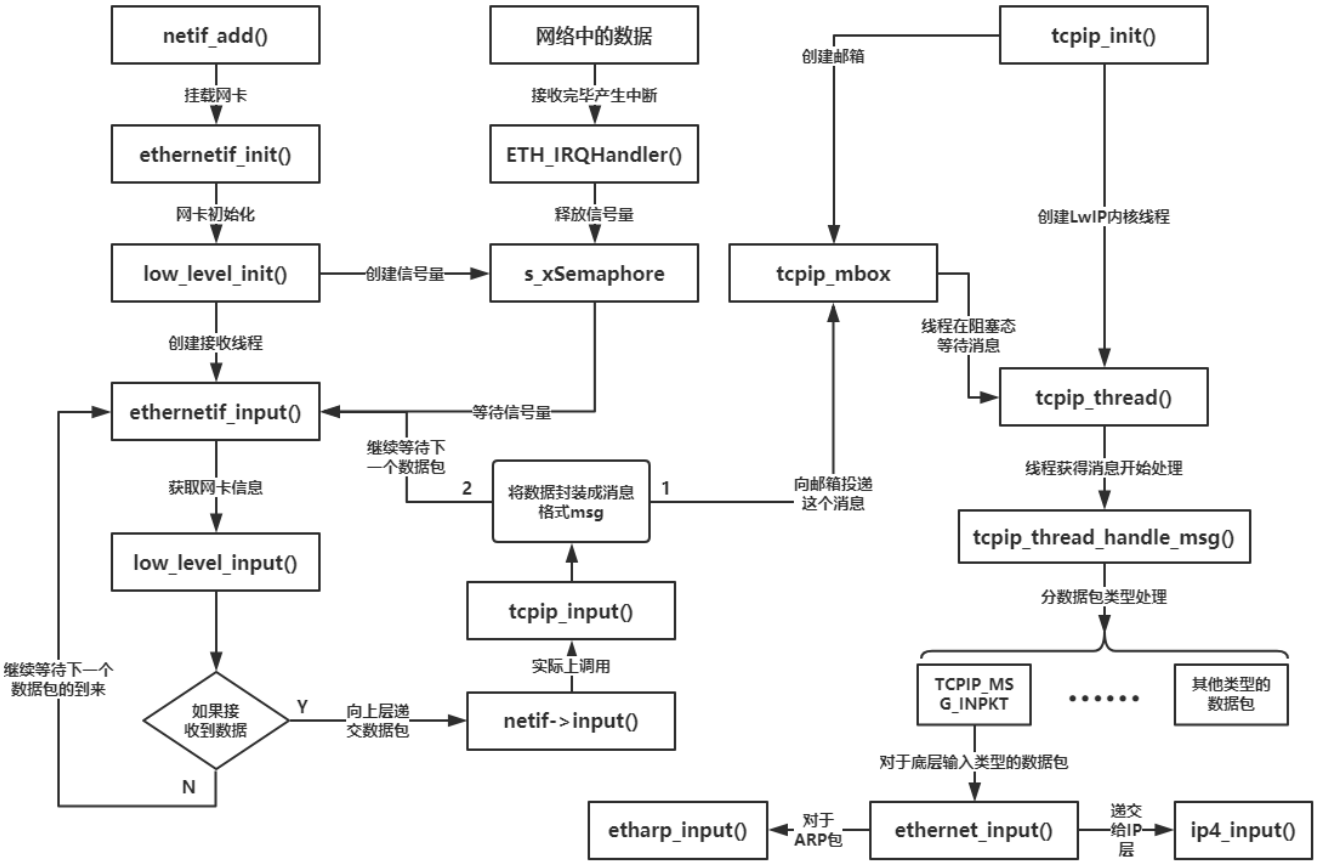


图 9-1 网卡数据传入 LwIP 内核

从这个图中，我们也能很明显看到，用户程序与内核是完全独立的，只是通过操作系统的 IPC 通信机制进行数据交互。

9.2 内核超时处理

在 LwIP 中很多时候都要用到超时处理，例如 ARP 缓存表项的时间管理、IP 分片数据报的重装等待超时、TCP 中的建立连接超时、重传超时机制等，因此超时处理的实现是 TCP/IP 协议栈中一个重要部分，LwIP 为每个与外界网络连接的任务都有设定了 timeout 属性，即等待超时时间，超时处理的相关代码实现在 timeouts.c 与 timeouts.h 中。

在旧版本的 LwIP 中（如 LwIP 1.4.1 版本），超时处理被称之为定时器（其实现在我们也能这样子

称之为定时器，但是为了下文统一，我们一律使用超时处理），但是，在最新版的 LwIP 中，原来的 timer.c 已经被删除，转而使用了 timeouts.c 来代替，并且该源码在实现上也有一定的区别。

由于 LwIP 是软件，它自身并没有硬件定时器，更不会对硬件定时器进行管理，所以 LwIP 作者就采用软件定时器对这些超时进行处理，因为软件定时器很容易维护，并且与平台无关，只需要用户提供一个较为准确的时基即可。

9.2.1 sys_timeo 结构体与超时链表

LwIP 通过一个 sys_timeo 类型的数据结构管理与超时链表相关的所有超时事件。LwIP 使用这个结构体记录下内核中所有被注册的超时事件，这些结构体会以链表的形式一个个连接在超时链表中，而内核中只有一条超时链表，那么怎么对超时链表进行管理呢？LwIP 定义了一个 sys_timeo 类型的指针 next_timeout，并且将 next_timeout 指向当前内核中链表头部，所有被注册的超时事件都会按照被处理的先后顺序排列在超时链表上。sys_timeo 结构体与超时链表源码具体见*代码清单 9_1*。

代码清单 9-1 sys_timeo 结构体与超时链表

```
typedef void (* sys_timeout_handler)(void *arg);

struct sys_timeo
{
    struct sys_timeo *next;                    (1)
    u32_t time;                                (2)
    sys_timeout_handler h;                     (3)
    void *arg;                                 (4)
};

/** The one and only timeout list */
static struct sys_timeo *next_timeout;         (5)
```

代码清单 9-1**(1)**：指向下一个超时事件的指针，用于超时链表的连接。

代码清单 9-1**(2)**：当前超时事件的等待时间。

代码清单 9-1(**3**)：指向超时的回调函数，该事件超时后就执行对应的回调函数。

代码清单 9-1(**4**)：向回调函数传入参数。

代码清单 9-1(**5**)：指向超时链表第一个超时事件。

9.2.2 注册超时事件

LwIP 虽然使用超时链表进行管理所有的超时事件，那么它首先需要知道有哪些超时事件才能去管理，而这些超时事件就是通过注册的方式被挂载在链表上，简单来说就是这些超时事件要在内核中登记一下，内核才会去处理，LwIP 中注册超时事件的函数是 sys_timeout()，但是实际上是调用 sys_timeout_abs() 函数，具体见*代码清单 9_2*。

代码清单 9-2 sys_timeout() 源码

```
void
sys_timeout(u32_t msecs, sys_timeout_handler handler, void *arg)
{
u32_t next_timeout_time;
LWIP_ASSERT_CORE_LOCKED();

/* overflow handled by TIME_LESS_THAN macro */
next_timeout_time = (u32_t)(sys_now() + msecs);              (1)

    sys_timeout_abs(next_timeout_time, handler, arg);
}

static void
sys_timeout_abs(u32_t abs_time, sys_timeout_handler handler, void *arg)
{
    struct sys_timeo *timeout, *t;

    timeout = (struct sys_timeo *)memp_malloc(MEMP_SYS_TIMEOUT); (2)
    if (timeout == NULL)
```

(下页继续)

(续上页)

```
    {
        return;
    }

    timeout->next = NULL;
    timeout->h = handler;
    timeout->arg = arg;
    timeout->time = abs_time;                                          (3)

    if (next_timeout == NULL)
    {
        next_timeout = timeout;                                        (4)
        return;
    }
    if (TIME_LESS_THAN(timeout->time, next_timeout->time))
    {
        timeout->next = next_timeout;
        next_timeout = timeout;                                        (5)
    }
    else
    {
        for (t = next_timeout; t != NULL; t = t->next)
        {
            if ((t->next == NULL) ||
                    TIME_LESS_THAN(timeout->time, t->next->time))
            {
                timeout->next = t->next;
                t->next = timeout;                                     (6)
                break;
            }
        }
    }
```

(下页继续)

(续上页)

```
}
```

代码清单 9-2**(1)**：根据当前时间计算出超时的时间，然后调用 sys_timeout_abs() 函数将当前事件插入超时链表。

代码清单 9-2**(2)**：从内存池中申请一个 MEMP_SYS_TIMEOUT 类型内存，保存对应超时事件的相关信息。

代码清单 9-2**(3)**：填写对应的超时事件信息，超时回调函数、函数参数、超时的时间。

代码清单 9-2**(4)**：如果超时链表中没有超时事件，那么新添加的事件就是链表的第一个。

代码清单 9-2**(5)**：如果新插入的超时事件比链表上第一个事件的时间短，则将新插入的超时事件设置成链表的第一个。

代码清单 9-2**(6)**：遍历链表，寻找合适的插入节点，超时链表根据超时事件的时间升序排列。

在 timeouts.c 中，有一个名字为 lwip_cyclic_timer 的结构，LwIP 使用该结构存放了其内部使用的循环超时事件。这些超时事件在 LwIP 初始化时通过函数 sys_timeouts_init() 调用定时器注册函数 sys_timeout() 注册进入超时链表中，lwip_cyclic_timer 的结构具体见代码清单 9_3。

代码清单 9-3 lwip_cyclic_timer 结构（已删减）

```
#define   TCP_TMR_INTERVAL       250
#define   IP_TMR_INTERVAL        1000
#define   ARP_TMR_INTERVAL       1000

struct lwip_cyclic_timer
{
    u32_t interval_ms;
    lwip_cyclic_timer_handler handler;
};

const struct lwip_cyclic_timer lwip_cyclic_timers[] =
{
```

(下页继续)

(续上页)

```
    {TCP_TMR_INTERVAL, HANDLER(tcp_tmr)},

    {IP_TMR_INTERVAL, HANDLER(ip_reass_tmr)},

    {ARP_TMR_INTERVAL, HANDLER(etharp_tmr)},
};
```

lwip_cyclic_timers 数组中存放了每个周期性的超时事件回调函数及超时时间，在 LwIP 初始化的时候就将这些事件一个个插入超时链表中，具体见*代码清单 9_4*。

代码清单 9-4 sys_timeouts_init() 源码

```
void sys_timeouts_init(void)
{
    size_t i;

    for(i =(LWIP_TCP ? 1 : 0);i < LWIP_ARRAYSIZE(lwip_cyclic_timers); i++)
    {
        sys_timeout(lwip_cyclic_timers[i].interval_ms,lwip_cyclic_timer,
                    LWIP_CONST_CAST(void *, &lwip_cyclic_timers[i]));
    }
}
```

插入超时链表后的示意图具体见图 *9_2*。

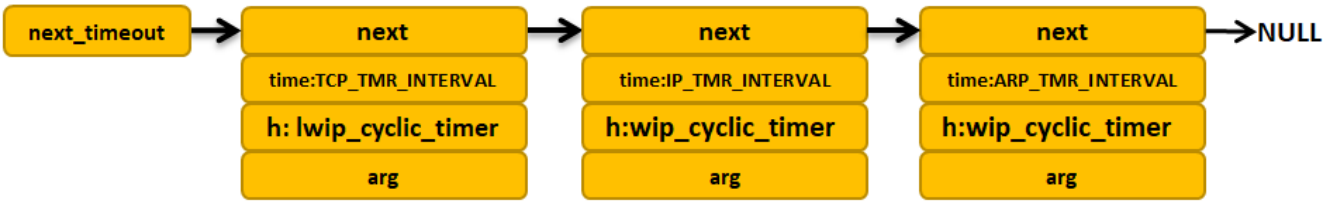


图 9-2 初始化插入的超时事件示意图

每个 sys_timeo 结构体中的 h 成员变量记录着对应的超时回调函数，对于周期性的回调函数，LwIP

是这样子处理的：在初始化的时候将他们注册到 lwip_cyclic_timer() 函数中，每次在处理回调函数之后，就调用 sys_timeout_abs() 函数将其重新注册到超时链表中，具体见代码清单 9_5。

代码清单 9-5 lwip_cyclic_timer() 源码（已删减）

```
lwip_cyclic_timer(void *arg)
{
    u32_t now;
    u32_t next_timeout_time;
    const struct lwip_cyclic_timer *cyclic = (const struct lwip_cyclic_
↪timer *)arg;

    cyclic->handler();

    now = sys_now();
    next_timeout_time = (u32_t)(current_timeout_due_time + cyclic->
↪interval_ms);

    if (TIME_LESS_THAN(next_timeout_time, now))
    {
        sys_timeout_abs((u32_t)(now + cyclic->interval_ms), lwip_cyclic_
↪timer, arg);
    }
    else
    {
        sys_timeout_abs(next_timeout_time, lwip_cyclic_timer, arg);
    }
}
```

9.2.3 超时检查

有的同学可能会发现，前面讲解的超时处理根本是不需要我们用户去考虑的，为什么还有讲解呢，其实不是这样子的，学习讲究的是一个循序渐进的过程，本书讲解的东西自然有其要讲解的道理，LwIP 实现了超时处理，那么无论我们的开发平台是否使用操作系统，都可以对其进行超时检查并且去处理，lwip 中以下两个函数可以实现对超时的处理：

void sys_check_timeouts(void)：这是用于裸机的函数，用户需要在裸机应用程序中周期性调用该函数，每次调用的时候 LwIP 都会检查超时链表上第一个 sys_timeo 结构体是否到期，如果没有到期，直接退出该函数，否则，执行 sys_timeo 结构体中对应的超时回调函数，并从链表上删除它，然后继续检查下一个 sys_timeo 结构体，直到 sys_timeo 结构体没有超时才退出。

tcpip_timeouts_mbox_fetch(sys_mbox_t *mbox, void **msg)：这个函数在操作系统的线程中循环调用，主要是等待 tcpip_mbox 消息，是可阻塞的，如果在等待 tcpip_mbox 的过程中发生超时事件，则会同时执行超时事件处理，即调用超时回调函数。LwIP 是这样子处理的，如果已经发生超时，LwIP 就会内部调用 sys_check_timeouts() 函数去检查超时的 sys_timeo 结构体并调用其对应的回调函数，如果没有发生超时，那就一直等待消息，其等待的时间为下一个超时时间的时间，一举两得。LwIP 中 tcpip 线程就是靠这种方法，即处理了上层及底层的 tcpip_mbox 消息，同时处理了所有需要超时处理的事件。具体见代码清单 9_6。

代码清单 9-6 tcpip_timeouts_mbox_fetch()

```
#define TCPIP_MBOX_FETCH(mbox, msg) tcpip_timeouts_mbox_fetch(mbox, msg)

static void
tcpip_timeouts_mbox_fetch(sys_mbox_t *mbox, void **msg)
{
    u32_t sleeptime, res;

again:
    LWIP_ASSERT_CORE_LOCKED();

    sleeptime = sys_timeouts_sleeptime();            (1)
    if (sleeptime == SYS_TIMEOUTS_SLEEPTIME_INFINITE)
```

(下页继续)

(续上页)

```
    {
        UNLOCK_TCPIP_CORE();
        sys_arch_mbox_fetch(mbox, msg, 0);          (2)
        LOCK_TCPIP_CORE();
        return;
    }
    else if (sleeptime == 0)
    {
        sys_check_timeouts();                       (3)
        goto again;
    }

    UNLOCK_TCPIP_CORE();
    res = sys_arch_mbox_fetch(mbox, msg, sleeptime);    (4)
    LOCK_TCPIP_CORE();
    if (res == SYS_ARCH_TIMEOUT)
    {
        sys_check_timeouts();
        goto again;
    }
}
```

代码清单 9-6**(1)**：调用 sys_timeouts_sleeptime() 函数得到距离事件超时的时间并保存在 sleeptime 变量中。

代码清单 9-6**(2)**：如果 sleeptime 为 SYS_TIMEOUTS_SLEEPTIME_INFINITE，表示当前系统无超时事件，那只需一直等待 mbox 消息即可，所以调用 sys_arch_mbox_fetch() 函数进行等待消息，等待时间是一直等待。

代码清单 9-6**(3)**：如果 sleeptime 为 0 表示已经发生超时了，那就调用 sys_check_timeouts() 去检查一下到底是哪个事件发生超时并且去处理其超时回调函数。

代码清单 9-6**(4)**：对于其他时间，LwIP 就在等待 tcpip_mbox 的消息的同时就去处理超时事件，等待 tcpip_mbox 的消息的时间为 sleeptime，然后在时间到达的时候就处理超时事件。如果接收

到消息，并且超时时间还没到，那就去处理 tcpip_mbox 的消息，然后再回来重新计算等待时间 sleeptime，如此反复，这样子既不会错过 tcpip_mbox 的消息，也不会错过超时的事件。

9.3 tcpip_thread 线程

从前面的章节我们也知道，LwIP 在操作系统的环境下，LwIP 内核是作为操作系统的一个线程运行的，在协议栈初始化的时候就会创建 tcpip_thread 线程，那么我们现在来看看 tcpip_thread 线程到底是怎么样的东西，具体见*代码清单 9_7*。

代码清单 9-7 tcpip_thread 线程

```
static void
tcpip_thread(void *arg)
{
    struct tcpip_msg *msg;
    LWIP_UNUSED_ARG(arg);

    LWIP_MARK_TCPIP_THREAD();

    LOCK_TCPIP_CORE();
    if (tcpip_init_done != NULL)
    {
        tcpip_init_done(tcpip_init_done_arg);
    }

    while (1)
    {
        LWIP_TCPIP_THREAD_ALIVE();
        /* 等待消息，等待时处理超时 */
        TCPIP_MBOX_FETCH(&tcpip_mbox, (void **)&msg);          (1)
        if (msg == NULL)
        {
```

(下页继续)

(续上页)

```
            continue;                                                    (2)
        }
        tcpip_thread_handle_msg(msg);                                    (3)
    }
}
```

代码清单 9-7**(1)**：LwIP 将函数 tcpip_timeouts_mbox_fetch() 定义为带参宏 TCPIP_MBOX_FETCH，所以在这里就是等待消息并且处理超时事件。

代码清单 9-7**(2)**：如果没有等到消息就继续等待。

代码清单 9-7**(3)**：等待到消息就对消息进行处理，这个函数具体见代码清单 9-8。

代码清单 9-8 tcpip_thread_handle_msg() 源码

```
static void
tcpip_thread_handle_msg(struct tcpip_msg *msg)
{
    switch (msg->type)
    {
#if !LWIP_TCPIP_CORE_LOCKING
    case TCPIP_MSG_API:
        msg->msg.api_msg.function(msg->msg.api_msg.msg);                 (1)
        break;
    case TCPIP_MSG_API_CALL:
        msg->msg.api_call.arg->err =
            msg->msg.api_call.function(msg->msg.api_call.arg);           (2)
        sys_sem_signal(msg->msg.api_call.sem);
        break;
#endif /* !LWIP_TCPIP_CORE_LOCKING */

#if !LWIP_TCPIP_CORE_LOCKING_INPUT
    case TCPIP_MSG_INPKT:
```

(下页继续)

(续上页)

```
    if (msg->msg.inp.input_fn(msg->msg.inp.p, msg->msg.inp.netif) != ERR_
↪OK) (3)
        {
            pbuf_free(msg->msg.inp.p);
        }
        memp_free(MEMP_TCPIP_MSG_INPKT, msg);
        break;
#endif /* !LWIP_TCPIP_CORE_LOCKING_INPUT */

#if LWIP_TCPIP_TIMEOUT && LWIP_TIMERS
    case TCPIP_MSG_TIMEOUT:
    sys_timeout(msg->msg.tmo.msecs, msg->msg.tmo.h, msg->msg.tmo.arg);(4)
        memp_free(MEMP_TCPIP_MSG_API, msg);
        break;
    case TCPIP_MSG_UNTIMEOUT:
        sys_untimeout(msg->msg.tmo.h, msg->msg.tmo.arg);              (5)
        memp_free(MEMP_TCPIP_MSG_API, msg);
        break;
#endif /* LWIP_TCPIP_TIMEOUT && LWIP_TIMERS */

    case TCPIP_MSG_CALLBACK:
        msg->msg.cb.function(msg->msg.cb.ctx);                        (6)
        memp_free(MEMP_TCPIP_MSG_API, msg);
        break;

    case TCPIP_MSG_CALLBACK_STATIC:
        msg->msg.cb.function(msg->msg.cb.ctx);                        (7)
        break;

    default:
        break;
    }
```

(下页继续)

(续上页)

```
}
```

代码清单 9-8**(1)(2)**：根据消息中的不同类型进行不同的处理，对于 TCPIP_MSG_API 类型，就执行对应的 API 函数。

代码清单 9-8**(3)**：对于 TCPIP_MSG_INPKT 类型，直接交给 ARP 层处理。

代码清单 9-8**(4)**：对于 TCPIP_MSG_TIMEOUT 类型，表示上层注册一个超时事件，直接执行注册超时事件即可。

代码清单 9-8**(5)**：相反的，对于 TCPIP_MSG_ UNTIMEOUT 类型，表示上层删除一个超时事件，直接执行删除超时事件即可。

代码清单 9-8**(6)(7)**：对于 TCPIP_MSG_CALLBACK 或者是 TCPIP_MSG_CALLBACK_STATIC 类型，表示上层通过回调方式执行一个回调函数，那么就执行对应的回调函数即可。

9.4 LwIP 中的消息

本小节主要讲解数据包消息与 API 消息，这最常用的两种消息类型，而且是 LwIP 中必须存在的消息，整个内核的运作都要依赖他们。

9.4.1 消息结构

从前面的章节，我们知道消息有多种类型，LwIP 中消息是有多种结构的的，对于不同的消息类型其封装是不一样的，tcpip_thread 线程是通过 tcpip_msg 描述消息的，tcpip_thread 线程接收到消息后，根据消息的类型进行不同的处理。LwIP 中使用 tcpip_msg_type 枚举类型定义了系统中可能出现的消息的类型，消息结构 msg 字段是一个共用体，其中定义了各种消息类型的具体内容，每种类型的消息对应了共用体中的一个字段，其中注册与删除事件的消息使用了同一个 tmo 字段。LwIP 中的 API 相关的消息内容很多，不适合直接放在 tcpip_msg 中，所以 LwIP 用一个 api_msg 结构体来描述 API 消息，在 tcpip_msg 中只存放指向 api_msg 结构体的指针，具体见代码清单 9_9。

代码清单 9-9 消息结构

```
enum tcpip_msg_type
{
    TCPIP_MSG_API,
    TCPIP_MSG_API_CALL,      //API 函数调用
    TCPIP_MSG_INPKT,         //底层数据包输入
    TCPIP_MSG_TIMEOUT,       //注册超时事件
    TCPIP_MSG_UNTIMEOUT,     //删除超时事件
    TCPIP_MSG_CALLBACK,
    TCPIP_MSG_CALLBACK_STATIC        //执行回调函数
};

struct tcpip_msg
{
    enum tcpip_msg_type type;                                    (1)
    union
    {
        struct
        {
            tcpip_callback_fn function;
            void* msg;
        } api_msg;                                               (2)

        struct
        {
            tcpip_api_call_fn function;
            struct tcpip_api_call_data *arg;
            sys_sem_t *sem;
        } api_call;                                              (3)

        struct
        {
            struct pbuf *p;
```

(下页继续)

(续上页)

```
            struct netif *netif;
            netif_input_fn input_fn;
        } inp;                                          (4)

        struct
        {
            tcpip_callback_fn function;
            void *ctx;
        } cb;                                                (5)

        struct
        {
            u32_t msecs;
            sys_timeout_handler h;
            void *arg;
        } tmo;                                          (6)

    } msg;
};
```

代码清单 9-9(**1**)：消息的类型，目前有 7 种。

代码清单 9-9(**2**)：API 消息主要由两部分组成，一部分是用于表示内核执行的 API 函数，另一部分是执行函数时候的参数，都会被记录在 api_msg 中。

代码清单 9-9(**3**)：与 API 消息差不多，也是由两部分组成，一部分是 tcpip_api_call_fn 类型的函数，另一部分是其对应的形参，此外还有用于同步的信号量。

代码清单 9-9(**4**)：inp 用于记录数据包消息的内容，p 指向接收到的数据包；netif 表示接收到数据包的网卡；input_fn 表示输入的函数接口，在 tcpip_inpkt 进行配置。

代码清单 9-9(**5**)：cb 用于记录回调函数与其对应的形参。

代码清单 9-9(**6**)：tmo 用于记录超时相关信息，如超时的时间，超时回调函数，参数等。

9.4.2 数据包消息

对于每种类型的消息，LwIP 内核都必须有一个产生与之对应的消息函数，在产生该类型的消息后就将其投递到系统邮箱 tcpip_mbox 中，这样子 tcpip_thread 线程就会从邮箱中得到消息并且处理，从而能使内核完美运作，从图 *9_1* 中我们可以很直观看到对应数据包的消息，是通过 tcpip_input() 函数对消息进行构造并且投递的，但是真正执行这些操作的函数是 tcpip_inpkt()，其源码具体见*代码清单 9_10*。

代码清单 9-10 tcpip_input() 源码

```
err_t
tcpip_input(struct pbuf *p, struct netif *inp)
{
    if (inp->flags & (NETIF_FLAG_ETHARP | NETIF_FLAG_ETHERNET))
    {
        return tcpip_inpkt(p, inp, ethernet_input);          (1)
    }
}

err_t
tcpip_inpkt(struct pbuf *p, struct netif *inp, netif_input_fn input_fn)
{
    struct tcpip_msg *msg;

    LWIP_ASSERT("Invalid mbox", sys_mbox_valid_val(tcpip_mbox));

    msg = (struct tcpip_msg *)memp_malloc(MEMP_TCPIP_MSG_INPKT); (2)
    if (msg == NULL)
    {
        return ERR_MEM;
    }

    msg->type = TCPIP_MSG_INPKT;
```

(下页继续)

(续上页)

```
    msg->msg.inp.p = p;
    msg->msg.inp.netif = inp;
    msg->msg.inp.input_fn = input_fn;                          (3)
    if (sys_mbox_trypost(&tcpip_mbox, msg) != ERR_OK)          (4)
    {
        memp_free(MEMP_TCPIP_MSG_INPKT, msg);                  (5)
        return ERR_MEM;
    }
    return ERR_OK;
}
```

代码清单 9-10**(1)**：调用 tcpip_inpkt() 函数将 ethernet_input() 函数作为结构体的一部分传递给内核，然后内核接收到这个数据包就调用该函数。

代码清单 9-10**(2)**：申请存放消息的内存空间。

代码清单 9-10**(3)**：构造消息，消息的类型是数据包消息，初始化消息结构中 msg 共用体的 inp 字段，p 指向数据包，网卡就是对应的网卡，处理的函数就是我们熟悉的 ethernet_input() 函数。

代码清单 9-10**(4)**：构造消息完成，就调用 sys_mbox_trypost 进行投递消息，这其实就是对操作系统的 API 简单封装，如果投递成功则返回 ERR_OK。

代码清单 9-10**(5)**：如果投递失败，就释放对应的消息结构空间。

总的来说，万变不离其宗，无论是裸机编程还是操作系统，都是通过 ethernet_input() 函数去处理接收到的数据包，只不过操作系统通过线程与线程间数据通信，使用了消息进行传递，这样子能使接收线程与内核线程互不干扰，相互独立开，在操作系统环境下，接收线程只负责接收数据包、构造消息并且完成投递消息即可，这样子处理完又能接收下一个数据包，这样子的效率更加高效，而内核根据这些消息做对应处理即可。

其运作示意图具体见图 *9_3*。

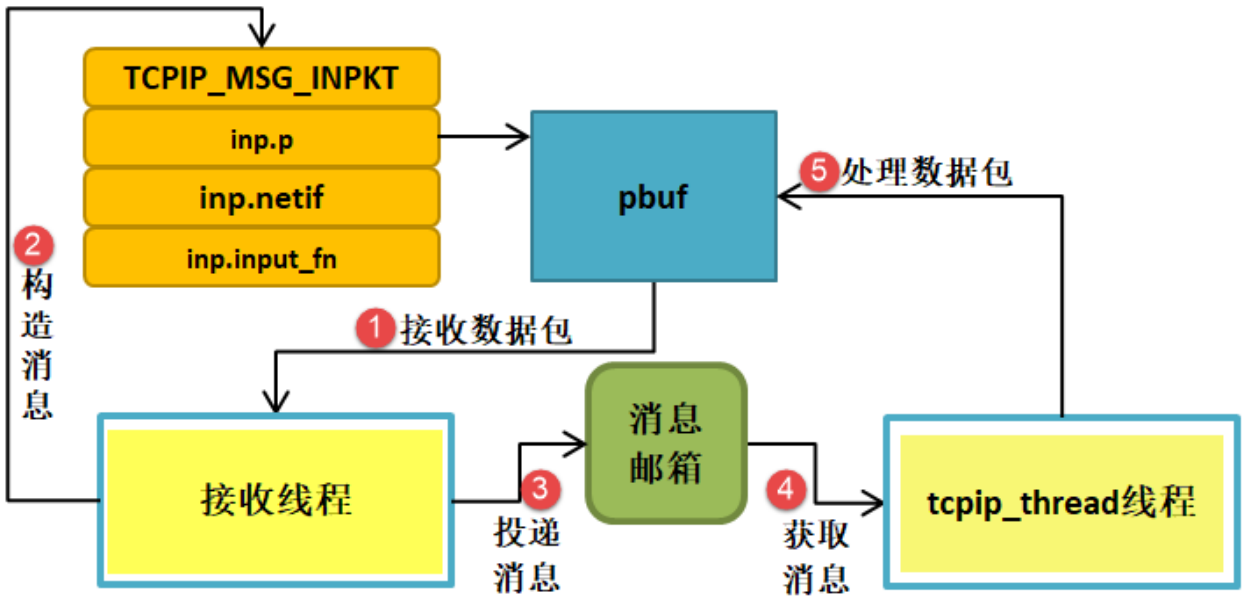


图 9-3 数据包消息运作

9.4.3 API 消息

LwIP 使用 api_msg 结构体描述一个 API 消息的内容，具体见*代码清单 9_11*。

代码清单 9-11api_msg 结构体（已删减）

```
struct api_msg
{
    struct netconn *conn; //当前连接
    err_t err;        //执行结果

    union
    {
        struct netbuf *b; //执行 lwip_netconn_do_send 需要的参数，待发送数据

        struct
```

（下页继续）

(续上页)

```
{
    u8_t proto; //执行 lwip_netconn_do_newconn 需要的参数，连接类型
} n;

//执行 lwip_netconn_do_bind 和 lwip_netconn_do_connect 需要的参数
struct
{
    API_MSG_M_DEF_C(ip_addr_t, ipaddr);    //ip 地址
    u16_t port;                            //端口号
    u8_t if_idx;
} bc;

//执行 lwip_netconn_do_getaddr 需要的参数
struct
{
    ip_addr_t API_MSG_M_DEF(ipaddr);//ip 地址
    u16_t API_MSG_M_DEF(port);      //端口号
    u8_t local;
} ad;

//执行 lwip_netconn_do_write 需要的参数
struct
{
    const struct netvector *vector; //要写入的当前向量
    u16_t vector_cnt;               //未写入的向量的数量
    size_t vector_off;              //偏移到当前向量
    size_t len;                     //总长度
    size_t offset;                  //偏移量
    u8_t apiflags;
} w;

//执行 lwip_netconn_do_write 需要的参数
```

(下页继续)

(续上页)

```
        struct
        {
            size_t len;    //长度
        } r;
    } msg;
};
```

api_msg 只包含 3 个字段，描述连接信息的 conn、内核返回的执行结果 err、还有 msg，msg 是一个共用体，根据不一样的 API 接口使用不一样的数据结构。在 conn 中，它保存了当前连接的重要信息，如信号量、邮箱等，lwip_netconn_do_xxx（xxx 表示不一样的 NETCONN API 接口）类型的函数执行需要用这些信息来完成与应用线程的通信与同步；内核执行 lwip_netconn_do_xxx 类型的函数返回结果会被记录在 err 中；msg 的各个产业记录各个函数执行时需要的详细参数。

我们了解底层的数据包消息，那么同理对于上层的 API 函数，想要与内核进行数据交互，也是通过 LwIP 的消息机制，API 消息由用户线程发出，与内核进行交互，因为用户的应用程序并不是与内核处于同一线程中，简单来说就是用户使用 NETCONN API 接口的时候，LwIP 会将对应 API 函数与参数构造成消息传递到 tcpip_thread 线程中，然后根据对应的 API 函数执行对应的操作，LwIP 这样子处理是为了简单用户的编程，这样子就不要求用户对内核很熟悉，与数据包消息类似，也是有独立的 API 消息投递函数去处理，那就是 netconn_apimsg() 函数，在 NETCONN API 中构造完成数据包，就会调用 netconn_apimsg() 函数进行投递消息，具体见*代码清单 9_12*。

代码清单 9-12NETCONN API 构造消息（以 netconn_bind 为例，已删减）

```
err_t
netconn_bind(struct netconn *conn, const ip_addr_t *addr, u16_t port)
{
    API_MSG_VAR_DECLARE(msg);
    err_t err;

    if (addr == NULL)
    {
        addr = IP4_ADDR_ANY;
```

(下页继续)

(续上页)

```
    }

    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;
    API_MSG_VAR_REF(msg).msg.bc.ipaddr = API_MSG_VAR_REF(addr);
    API_MSG_VAR_REF(msg).msg.bc.port = port;                              (1)
    err = netconn_apimsg(lwip_netconn_do_bind, &API_MSG_VAR_REF(msg)); (2)
    API_MSG_VAR_FREE(msg);

    return err;
}

static err_t
netconn_apimsg(tcpip_callback_fn fn, struct api_msg *apimsg)
{
    err_t err;

    err = tcpip_send_msg_wait_sem(fn, apimsg, LWIP_API_MSG_SEM(apimsg));
    if (err == ERR_OK)
    {
        return apimsg->err;
    }
    return err;
}

err_t
tcpip_send_msg_wait_sem(tcpip_callback_fn fn, void *apimsg, sys_sem_t *sem)
{
    TCPIP_MSG_VAR_DECLARE(msg);

    TCPIP_MSG_VAR_ALLOC(msg);
    TCPIP_MSG_VAR_REF(msg).type = TCPIP_MSG_API;
```

(下页继续)

(续上页)

```
    TCPIP_MSG_VAR_REF(msg).msg.api_msg.function = fn;
    TCPIP_MSG_VAR_REF(msg).msg.api_msg.msg = apimsg;                    (3)
    sys_mbox_post(&tcpip_mbox, &TCPIP_MSG_VAR_REF(msg));                (4)
    sys_arch_sem_wait(sem, 0);                                          (5)
    TCPIP_MSG_VAR_FREE(msg);
    return ERR_OK;
}
```

代码清单 9-12**(1)**：根据 netconn_bind() 传递的参数初始化 api_msg 结构体。

代码清单 9-12**(2)**：调用 netconn_apimsg() 函数投递这个 api_msg 结构体，这个函数实际上是调用 tcpip_send_msg_wait_sem() 函数投递 API 消息的，并且需要等待 tcpip_thread 线程的回应。

代码清单 9-12**(3)**：构造 API 消息，类型为 TCPIP_MSG_API，函数为 API 对应的函数 lwip_netconn_do_bind，将 msg 的指针指向 api_msg 结构体。

代码清单 9-12**(4)**：调用 sys_mbox_post() 函数向内核进行投递消息。

代码清单 9-12**(5)**：同时调用 sys_arch_sem_wait() 函数等待消息处理完毕

总的来说，用户的应用线程与内核也是相互独立的，依赖操作系统的 ICP 通信机制进行数据交互与同步（邮箱、信号量等），LwIP 提供上层 NETCONN API 接口，会自动帮我们处理这些事情，只需要我们根据 API 接口传递正确的参数接口，当然，NETCONN API 的使用我们会在后面的章节具体介绍，此处仅做了解一下即可，只是为了让大家对 LwIP 整个内核的运作有个详细的了解，其运作示意图具体见图 *9_4*。

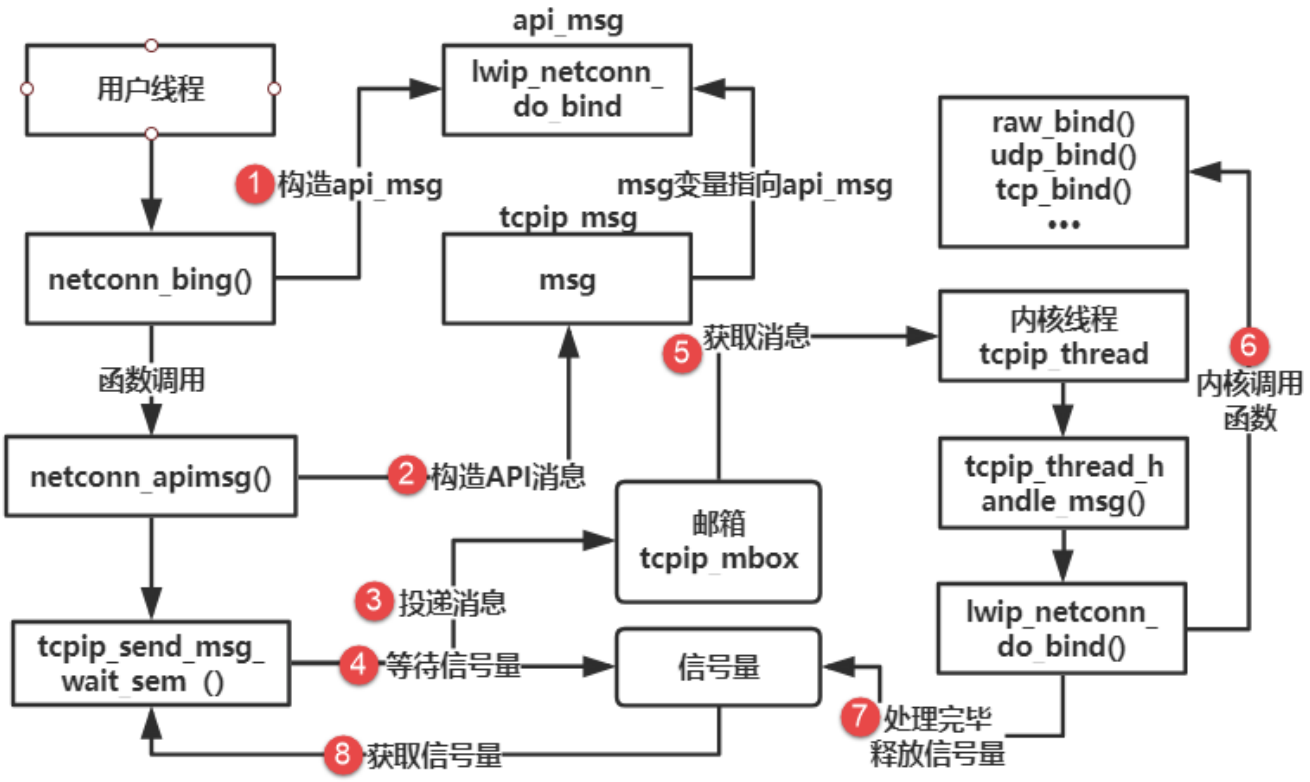


图 9-4API 消息运作

其实这个运作示意图并不是最优的，这种运作的方式在每次发送数据的时候，会进行一次线程的调度，这无疑是增大了系统的开销，而将 LWIP_TCPIP_CORE_LOCKING 宏定义设置为 1 则无需操作系统邮箱与信号量的参与，直接在用户线程中通过回调函数调用对应的处理，当然在这个过程中，内核线程是无法获得互斥量而运行的，因为是通过互斥量进行保护用户线程的处理，当然，LwIP 的作者也是这样子建议的。

更多详细内容请看 tcpip_send_msg_wait_sem() 源码，具体见*代码清单 9_13*。注意：此处的源码是无删减的，它通过宏定义 LWIP_TCPIP_CORE_LOCKING 决定运行哪部分代码。

代码清单 9-13 tcpip_send_msg_wait_sem() 源码

```
err_t
tcpip_send_msg_wait_sem(tcpip_callback_fn fn, void *apimsg, sys_sem_t *sem)
{
#if LWIP_TCPIP_CORE_LOCKING
```

(下页继续)

(续上页)

```
    LWIP_UNUSED_ARG(sem);
    LOCK_TCPIP_CORE();
    fn(apimsg);                          //调用对应的回调函数去处理
    UNLOCK_TCPIP_CORE();
    return ERR_OK;
#else /* LWIP_TCPIP_CORE_LOCKING */
    TCPIP_MSG_VAR_DECLARE(msg);

    LWIP_ASSERT("semaphore not initialized", sys_sem_valid(sem));
    LWIP_ASSERT("Invalid mbox", sys_mbox_valid_val(tcpip_mbox));

    TCPIP_MSG_VAR_ALLOC(msg);
    TCPIP_MSG_VAR_REF(msg).type = TCPIP_MSG_API;
    TCPIP_MSG_VAR_REF(msg).msg.api_msg.function = fn;
    TCPIP_MSG_VAR_REF(msg).msg.api_msg.msg = apimsg;
    sys_mbox_post(&tcpip_mbox, &TCPIP_MSG_VAR_REF(msg));
    sys_arch_sem_wait(sem, 0);
    TCPIP_MSG_VAR_FREE(msg);
    return ERR_OK;
#endif /* LWIP_TCPIP_CORE_LOCKING */
}
```

9.5 揭开 LwIP 神秘的面纱

通过本章的学习，是不是对 LwIP 没那么陌生了，我敢保证，如果认真学习了本章，那么对 LwIP 的内核整体运作过程已经熟悉了，从底层数据包输入到内核，从应用程序到内核间的数据交互，都依赖操作系统的 IPC 通信机制，回过头来，看看本章的图解与源码，是不是觉得 LwIP 没有那么神秘了，这就是本章需要达到的效果。

第 10 章　ARP 协议

10.1 链路层概述

我们首先学习一些有用的术语。在本章中为方便讨论，将运行链路层协议（即第 2 层）协议的任何设备均称为结点（node）。结点包括主机、路由器、交换机和 WiFi 接入点。我们也把沿着通信路径连接相邻结点的通信信道称为链路（link）。为了将一个数据报从源主机传输到目标主机，数据报必须通过端到端路径上的各段链路才能到达目标主机。在通过特定的链路时，传输结点将数据报封装在链路层帧中，并将该帧传送到链路中。

链路层的主体部分是网卡中实现的，在发送的数据帧的时候，协议栈取得由高层传递过的数据报（注意，数据帧与数据报不是一个东西，数据帧一般形容链路层的数据，是一帧一帧的，也是链路层的服务——数据成帧），在链路层中封装该数据报，也就是填写数据帧的各个字段，然后遵循链路接入协议将该帧数据进行传输；在接收端，控制器接收了整个数据帧，抽取出网络层的数据报，往上层传递。

其实这些知识过于底层，我们暂时也无需了解太多，只需要知道所有不同主机的信息都是必须通过链路层才能传递数据，所有链路层的完整与稳定是传递信息的必要条件。

10.2 MAC 地址的基本概念

MAC Address(Media Access Control Address)，亦称为 EHA（Ethernet Hardware Address）、硬件地址、物理地址（Physical Address）。MAC 地址就是在媒体接入层上使用的地址，也叫物理地址、硬件地址或链路地址，其被固化在网卡的 ROM 中。可见 MAC 地址实际上就是网卡地址或网卡标识符。当某台计算机使用某块网卡后，网卡上的标识符就成为该计算机的 MAC 地址。MAC 地址长度为 6 字节（48 比特），其前 3 个字节表示组织唯一标志符（Organizationally Unique Identifier，即 OUI），由 IEEE 的注册管理机构给不同厂家分配的代码，以区分不同的厂家，后 3 个字节由厂家自行分配，称为扩展标识符。同一个厂家生产的网卡中 MAC 地址后 24 位是不同的。

10.3 初识 ARP

地址解析协议（Address Resolution Protocol，ARP）是通过解析 IP 地址得到数据链路层地址的，是一个在网络协议包中极其重要的网络传输协议，它与网卡有着极其密切的关系，在 TCP/IP 分层结构中，把 ARP 划分为网络层，为什么呢，因为在网络层看来，源主机与目标主机是通过 IP 地址进行识别的，而所有的数据传输又依赖网卡底层硬件，即链路层，那么就需要将这些 IP 地址转换为链路层可以识别的东西，在所有的链路中都有着自己的一套寻址机制，如在以太网中使用 MAC 地址进行寻址，标识不同的主机，那么就需要有一个协议将 IP 地址转换为 MAC 地址，由此就出现了 ARP 协议，ARP 协议在网络层被应用，它是网络层与链路层连接的重要枢纽。

在局域网中，网络中实际传输的是“帧”，帧里面是有目标主机的 MAC 地址的。在以太网中，一个主机要和另一个主机进行直接通信，必须要知道目标主机的 MAC 地址，那就需要 ARP 进行地址解析，所谓“地址解析”就是主机在发送帧前将目标 IP 地址转换成目标 MAC 地址的过程。ARP 协议的基本功能就是通过目标设备的 IP 地址，查询目标设备的 MAC 地址，以保证通信的顺利进行。

10.4 以太网帧结构

既然谈到 MAC 地址，那就不得不说一下以太网帧结构了，每个网卡都有唯一一个物理地址，在硬件中进行数据帧传输的时候就必须有正确的目的物理地址，例如以太网的 48 位 MAC 地址就是存储在网卡内部存储器中。

以太网帧结构如图 *10_1* 所示：

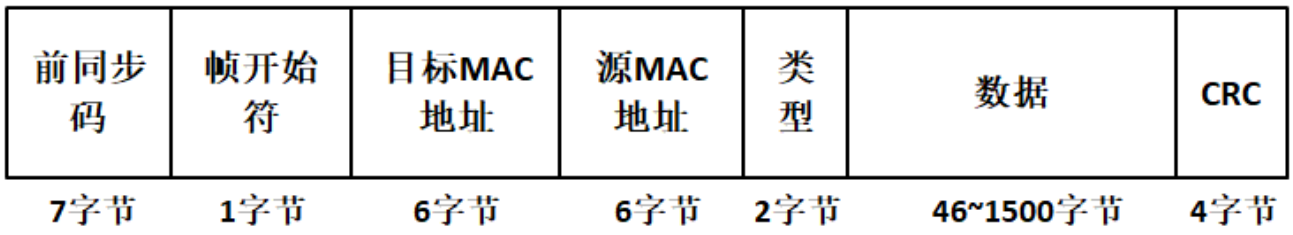


图 10-1 以太网帧结构

一看这个以太网帧结构，就觉得它有 7 个字段，但是事实上，前同步码与帧开始符字段不能算是真正的以太网帧数据结构，他们是在网卡发送的时候添加进去的，为了数据的准确传输。

以太网帧以一个 7 字节的前同步码（Preamble）字段开始。该前同步码的值都是 10101010（0x55，大端模式）；而后紧接着一个字节的帧开始符，其值是 10101011（0xD5，大端模式）。前同步码字段的作用是实现物理层帧输入输出的同步，而帧开始符表示着以太网帧的开始，剩下的 5 个字段才是真正的以太网数据帧结构。

目标 MAC 地址（6 字节）：这个字段包含目标网卡的 MAC 地址，当一个网卡收到一个以太网数据帧，如果该数据帧的目标地址是网卡自身的 MAC 地址或者是 MAC 广播地址，它都将该帧的数据字段的内容传递给网络层；如果它收到了具有任何其他 MAC 地址的帧，则将该数据帧丢弃。

源 MAC 地址（6 字节）：这个字段包含了传输该帧到局域网上的适配器的 MAC 地址。

类型字段（2 字节）：类型字段允许以太网复用多种网络层协议。为了理解这点，我们需要记住主机能够使用除了 IP 以外的其他网络层协议。事实上，一台给定的主机可以支持多种网络层协议，以对不同的应用采用不同的协议。因此，当以太网帧到达网卡中，网卡需要知道它应该将数据字段的内容传递给哪个网络层协议。如 IP 协议、ARP 协议等。

注意了：当这个字段的值小于 1518 时，它表示后面数据字段的数据长度，当大于 1518 的时候才表示递交给哪个协议。

数据字段（46~1500 字节）：这个字段承载了 IP 数据报。以太网的最大传输单元（MTU）是 1500 字节。这意味着如果 IP 数据报超过了 1500 字节，则主机必须将该数据报分片（关于分片会在后续讲解）。数据字段的最小长度是 46 字节，这意味着如果 IP 数据报小于 46 字节，数据报必须被填充到 46 字节。当采用填充时，传递到网络层的数据包括 IP 数据报和填充部分，网络层使用 IP 数据报首部中的长度字段来去除填充部分。

CRC（4 字节）：CRC 字段包含了以太网的差错校验信息。

在以太网帧中，目标 MAC 地址可以分成三类，单播地址、多播地址和广播地址。单播地址通常是与某个网卡的 MAC 地址对应，它要求以太网第一个字节的 bit0（最先发出去的位）必须是 0；而多播地址则要求第一个字节的 bit0 为 1，这样子多播地址就不会与任何网卡的 MAC 地址相同，可以被多个网卡同时接收；广播地址的 48 位 MAC 地址全为 1，也就是 FF-FF-FF-FF-FF-FF，同一局域网内的所有网卡都会收到广播的数据包。

所有的以太网计算都向网络层提供不可靠的无连接服务，也就是说在网卡发送数据的时候，不会

向目标网卡进行事先的通知（握手），网卡只在以太网帧中封装好来自上层的数据报，然后把数据报发送到局域网上。同样的当一个网卡接收一个以太网数据帧的时候，它也不会进行回复确认，如果当网卡执行 CRC 校验不通过的时候，它也不会发送否定确认，这样子当一些以太网帧不通过 CRC 校验时，网卡只是将其丢弃，而发送的一方就不会知道它传输的数据是否达到并且通过校验。

10.5 IP 地址映射为物理地址

TCP/IP 协议有自己的 IP 地址，IP 地址（IPv4）是一个 32 位的 IP 地址，网络层发送数据包只需要知道目标主机 IP 地址即可，而以太网发送数据则必须知道对方的硬件 MAC 地址，同时 IP 地址的分配与硬件 MAC 地址是没有关系的，为了让网络层只需要知道 IP 地址就可以完成通信工作，那就需要有一个协议将 IP 地址映射成为对应的 MAC 地址，此外还需要考虑 IP 地址可能是动态的，非常灵活，使用静态的映射方法是行不通的，所以 ARP 协议就提供优质的地址动态解析的机制，让 32 位的 IP 地址能映射成为 48 位的 MAC 地址，让上层应用于底层完全分离开，这样子在上层应用就能灵活使用 IP 地址作为标识，进行通信。

10.6 ARP 缓存表

既然已经解释了 ARP 的用途，我们再来看看它是如何工作的。为了实现 IP 地址与网卡 MAC 地址的查询与转换，ARP 协议引入了 ARP 缓存表的概念，每台主机或路由器在其内存中具有一个 ARP 缓存表（ARP table），这张表包含 IP 地址到 MAC 地址的映射关系，表中记录了 <IP 地址，MAC 地址 > 对，它们是主机最近运行时获得关于其他主机的 IP 地址到物理地址的映射，当需要发送 IP 数据的时候，主机就会根据目标 IP 地址到 ARP 缓存表中进行查找对应的 MAC 地址，然后通过网卡将数据发送出去。ARP 表也包含一个寿命（TTL）值，它指示了从表中删除每个映射的时间。从一个表项放置到某 ARP 表中开始，一个表项通常的过期时间是 10 分钟。

我们电脑也是有自己的 ARP 缓存表的，可以在控制台中通过“arp -a”命令进行查看，具体见图 *10_2*。

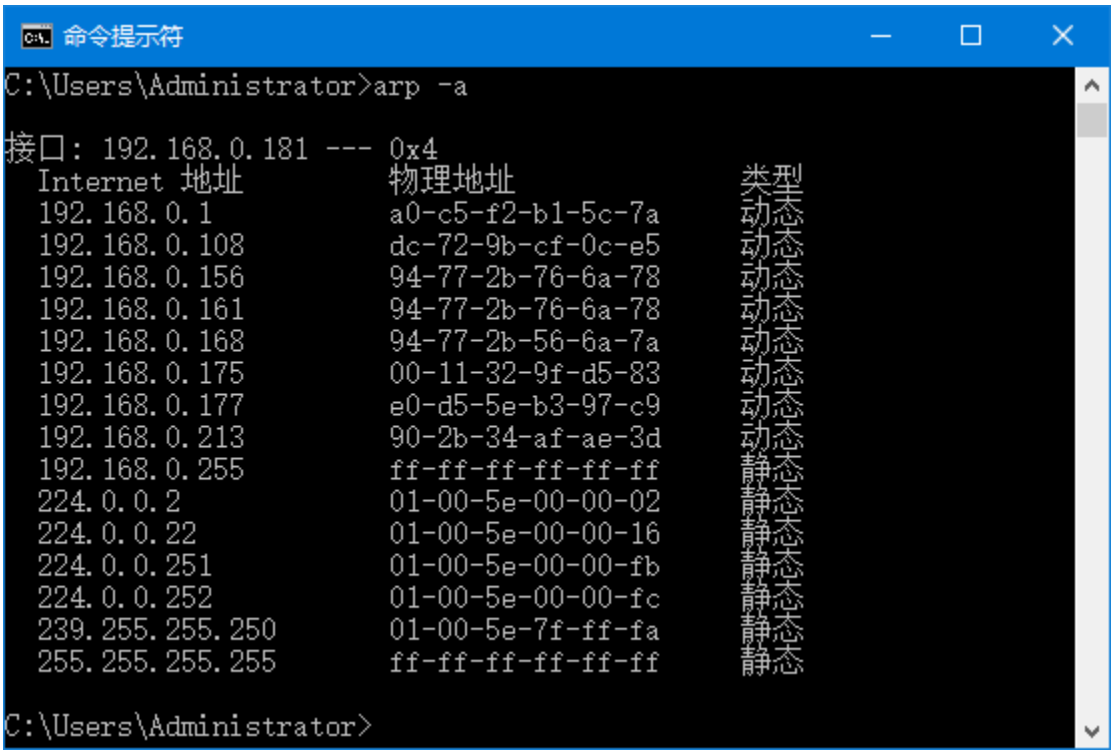


图 10-2ARP 缓存表

从图中可以看到作者的电脑有很多这样子的缓存表，其中 192.168.0.181 是我电脑的 IP 地址，而 192.168.0.xxx 这些 IP 是公司的其他电脑与其对应的物理地址（MAC 地址），比如 IP 地址为 192.168.0.108 的电脑 MAC 地址为 dc-72-9b-cf-0c-e5，如果我想向该电脑发送一个数据包，那么我的电脑就会从已有的 ARP 缓存表中寻找这个 IP 地址对应的物理地址，然后直接将数据写入以太网数据帧中让网卡进行发送即可，而如果没有找到这个 IP 地址，那么这个数据就没法立即发送，电脑会先在局域网上广播一个 ARP 请求（目标 MAC 地址为 FF-FF-FF-FF-FF-FF），广播的 ARP 请求发出后，处于同一局域网内的所有主机都会接收到这个请求，如果目标 IP 地址与主机自身 IP 地址吻合就会返回一个 ARP 应答，告诉请求者自身的 MAC 地址，当我的电脑收到这个 ARP 应答后，就去更新 ARP 缓存表，并且重新将数据发送出去。

ARP 协议的核心就是对缓存表的操作，发送数据包的时候，查找 ARP 缓存表以得到对应的 MAC 地址，必要时进行 ARP 缓存表的更新，此外 ARP 还需要不断处理其他主机的 ARP 请求，在 ARP 缓存表中的 TTL 即将过期的时候更新缓存表以保证缓存表中的表项有效。

其运作过程大致可以理解为：

1. 如果主机 A 想发送数据给主机 B，主机 A 首先会检查自己的 ARP 缓存表，查看是否有主机 B 的 IP 地址和 MAC 地址的对应关系，如果有，则会将主机 B 的 MAC 地址作为源 MAC 地址封装到数据帧中。如果本地 ARP 缓存中没有对应关系，主机 A 就会向局域网中广播 ARP 请求（包括发送方的 IP 地址、MAC 地址、接收方的 IP 地址），每台主机接收到 ARP 请求后都检查自己的 IP 地址是否与 ARP 请求中的接收方 IP 地址相同，若不相同则丢弃 ARP 请求包。

2. 当交换机接受到此数据帧之后，发现此数据帧是广播帧，因此，会将此数据帧从非接收的所有接口发送出去。

3. 当主机 B 接受到此数据帧后，会校对 IP 地址是否是自己的，并将主机 A 的 IP 地址和 MAC 地址的对应关系记录到自己的 ARP 缓存表中，同时会发送一个 ARP 响应，其中包括自己的 MAC 地址。

4. 主机 A 在收到这个回应的数据帧之后，在自己的 ARP 缓存表中记录主机 B 的 IP 地址和 MAC 地址的对应关系。而此时交换机已经学习到了主机 A 和主机 B 的 MAC 地址了。

那么在 LwIP 中缓存表是如何实现的呢？下面我们就结合源码进行讲解。

ARP 协议的核心是 ARP 缓存表，ARP 的实质就是对缓存表的建立、更新、查询等操作，ARP 缓存表的核心是表项（entry）。LwIP 使用一个 arp_table 数组描述 ARP 缓存表，数组的内容是表项的内容，具体见*代码清单 10_1*。每个表项都必须记录一对 IP 地址与 MAC 地址的映射关系，此外还有一些基本的信息，如表项的状态、生命周期（生存时间）以及对应网卡的基本信息，LwIP 使用一个 etharp_entry 结构体对表项进行描述，具体见*代码清单 10_2*。

代码清单 10-1ARP 缓存表

```
static struct etharp_entry arp_table[ARP_TABLE_SIZE];
```

编译器预先定义了缓存表的大小，ARP_TABLE_SIZE 默认为 10，也就是最大能存放 10 个表项，由于这个表很小，LwIP 对表的操作直接采用遍历方式，遍历每个表项并且更改其中的内容。

代码清单 10-2 etharp_entry 结构体

```
struct etharp_entry
{
#if ARP_QUEUEING
```

(下页继续)

(续上页)

```
    /** 指向此 ARP 表项上挂起的数据包队列的指针。 */
    struct etharp_q_entry *q;                                    //(1)-□
#else
    /** 指向此 ARP 表项上的单个挂起数据包的指针。 */
    struct pbuf *q;                                        //(1)-□
#endif
    ip4_addr_t ipaddr;                                           (2)
    struct netif *netif;                                   (3)
    struct eth_addr ethaddr;                                     (4)
    u16_t ctime;                                           (5)
    u8_t state;                                            (6)
};
```

代码清单 10-2**(1)**：这里使用了一个宏定义进行预编译，默认配置该宏定义是不打开的，其实都是一样，q 都是指向数据包，**(1)-□** 中的 etharp_q_entry 指向的是数据包缓存队列，etharp_q_entry 是一个结构体，具体见*代码清单 10_3*，如果将 ARP_QUEUEING 宏定义打开，系统会为 etharp_q_entry 结构体开辟一些 MEMP_ARP_QUEUEING 类型的内存池，以便快速申请内存。而 **(1)-□** 中的 q 直接指向单个数据包。具体见图 *10_3* 与图 *10_4*。

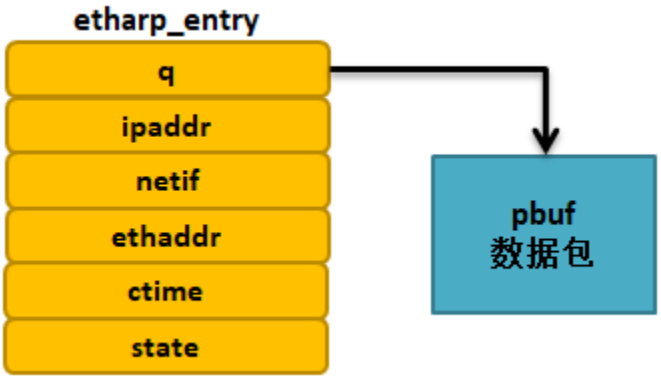


图 10-3 ARP 表项上的单个挂起数据包

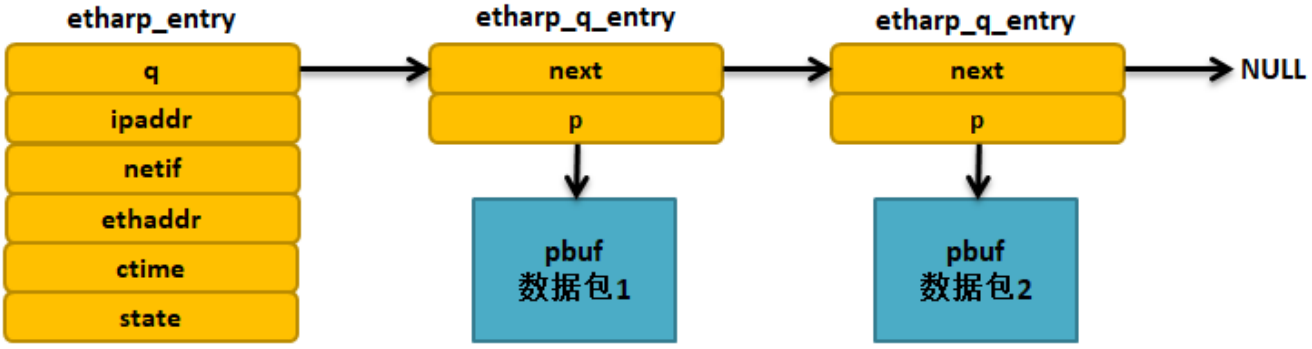


图 10-4 ARP 表项上挂起的数据包队列

代码清单 10-3etharp_q_entry 结构体

```
struct etharp_q_entry
{
    struct etharp_q_entry *next;
    struct pbuf *p;
};
```

代码清单 10-2(**2**)：记录目标 IP 地址。

代码清单 10-2(**3**)：对应网卡信息。

代码清单 10-2(**4**)：记录与目标 IP 地址对应的 MAC 地址。

代码清单 10-2(**5**)：生存时间。

代码清单 10-2(**6**)：表项的状态，LwIP 中用枚举类型定义了不同的状态，具体见*代码清单 10_4*。

代码清单 10-4 表项的状态

```
/** ARP states */
enum etharp_state
{
    ETHARP_STATE_EMPTY = 0,
    ETHARP_STATE_PENDING,
```

(下页继续)

(续上页)

```
    ETHARP_STATE_STABLE,
    ETHARP_STATE_STABLE_REREQUESTING_1,
    ETHARP_STATE_STABLE_REREQUESTING_2
#if ETHARP_SUPPORT_STATIC_ENTRIES
    , ETHARP_STATE_STATIC
#endif /* ETHARP_SUPPORT_STATIC_ENTRIES */
};
```

ARP 缓存表在初始化的时候，所有的表项都会被初始化为 ETHARP_STATE_EMPTY，也就是空状态，表示这些表项能被使用，在需要添加表项的时候，LwIP 内核就会遍历 ARP 缓存表，找到合适的表项，进行添加。如果 ARP 表项处于 ETHARP_STATE_PENDING 状态，表示 ARP 已经发出了一个 ARP 请求包，但是还未收到目标 IP 地址主机的应答，处于这个状态的缓存表项是有等待时间的，它通过宏定义 ARP_MAXPENDING 指定，默认为 5 秒钟，如果从发出 ARP 请求包后的 5 秒内还没收到应答，那么该表项又会被删除；而如果收到应答后，ARP 就会更新缓存表的信息，记录目标 IP 地址与目标 MAC 地址的映射关系并且开始记录表项的生存时间，同时该表项的状态会变成 ETHARP_STATE_STABLE 状态。当要发送数据包的时候，而此时表项为 ETHARP_STATE_PENDING 状态，那么这些数据包就会暂时被挂载到表项的数据包缓冲队列上，直到表项的状态为 ETHARP_STATE_STABLE，才进行发送数据包。对于状态为 ETHARP_STATE_STABLE 的表项，这些表项代表着 ARP 记录了 IP 地址与 MAC 地址的映射关系，能随意通过 IP 地址进行数据的发送，但是这些表项是具有生存时间的，通过宏定义 ARP_MAXAGE 指定，默认为 5 分钟，在这些时间，LwIP 会不断维护这些缓存表以保持缓存表的有效。当表项是 ETHARP_STATE_STABLE 的时候又发送一个 ARP 请求包，那么表项状态会暂时被设置为 ETHARP_STATE_STABLE_REREQUESTING_1，然后被设置为 ETHARP_STATE_STABLE_REREQUESTING_2 状态，这些是一个过渡状态，当收到 ARP 应答后，表项又会被设置为 ETHARP_STATE_STABLE，这样子能保持表项的有效。

所以 ARP 缓存表是一个动态更新的过程，为什么要动态更新呢？因为以太网的物理性质并不能保证数据传输的是可靠的。以太网发送数据并不会知道对方是否已经介绍成功，而两台主机的物理线路不可能一直保持有效畅通，那么如果不是动态更新的话，主机就不会知道另一台主机是否在工作中，这样子发出去的数据是没有意义的。比如两台主机 A 和 B，一开始两台主机都是处于连接状态，能正常进行通信，但是某个时刻主机 B 断开了，但是主机 A 不会知道主机 B 是否正常运行，因为以太网不会提示主机 B 已经断开，那么主机 A 会一直按照 MAC 地址发送数据，

而此时在物理链路层就已经是不通的，那么这些数据是没有意义的，而如果 ARP 动态更新的话，主机 A 就会发出 ARP 请求包，如果得不到主机 B 的回应，则说明无法与主机 B 进行通信，那么就会删除 ARP 表项，就无法进行通信。

10.7 ARP 缓存表的超时处理

从前面的章节也知道，ARP 是动态处理的，现在总结一下：ARP 表项的生存时间是 5 分钟，而 ARP 请求的等待时间是 5 秒钟，当这些时间到达后，就会更新 ARP 表项，如果在物理链路层无法连通则会删除表项。这就需要 ARP 层有一个超时处理函数对 ARP 进行管理，这些操作都是根据 ARP 表项的 ctime 字段进行的，它记录着对应表项的生存时间，而超时处理函数是 etharp_tmr()，它是一个周期性的超时处理函数，每隔 1 秒就调用一次，当 ctime 的值大于指定的时间，就会删除对应的表项，具体见*代码清单 10_5*。

代码清单 10-5 etharp_tmr() 源码

```
void
etharp_tmr(void)
{
    int i;

    LWIP_DEBUGF(ETHARP_DEBUG, ("etharp_timer\n"));
    /* 遍历 ARP 表，从 ARP 表中删除过期的表项 */
    for (i = 0; i < ARP_TABLE_SIZE; ++i)     (1)
    {
        u8_t state = arp_table[i].state;
        if (state != ETHARP_STATE_EMPTY
#if ETHARP_SUPPORT_STATIC_ENTRIES
                && (state != ETHARP_STATE_STATIC)
#endif /* ETHARP_SUPPORT_STATIC_ENTRIES */
        )
        {
            arp_table[i].ctime++;           (2)
```

(下页继续)

(续上页)

```
            if ((arp_table[i].ctime >= ARP_MAXAGE) ||
                    ((arp_table[i].state == ETHARP_STATE_PENDING)  &&
                    (arp_table[i].ctime >= ARP_MAXPENDING))) (3)
            {
                /* 等待表项稳定或者表项已经过期 */
        LWIP_DEBUGF(ETHARP_DEBUG,("etharp_timer: expired %s entry %d.\n",
            arp_table[i].state >= ETHARP_STATE_STABLE ? "stable" : "pending
↪", i));
                /* 从 ARP 表中删除过期的表项 */
                etharp_free_entry(i);                (4)
            }
            else if (arp_table[i].state == ETHARP_STATE_STABLE_
↪REREQUESTING_1)
            {
                /* 过渡 */
                arp_table[i].state = ETHARP_STATE_STABLE_REREQUESTING_2;
            }
        else if (arp_table[i].state == ETHARP_STATE_STABLE_REREQUESTING_2)
            {
        /* 进入 ETHARP_STATE_STABLE 状态 */

                arp_table[i].state = ETHARP_STATE_STABLE;
            }
            else if (arp_table[i].state == ETHARP_STATE_PENDING)
            {
                /* 仍然挂起，重新发送 ARP 请求 */
                etharp_request(arp_table[i].netif, &arp_table[i].ipaddr);
            }
        }
    }
}
```

代码清单 10-5(**1**)：由于 LwIP 的 ARP 表是比较小的，直接遍历表即可，更新 ARP 表的内容。

代码清单 10-5(**2**)：如果 ARP 表项不是空的，那么就记录表项的时间。

代码清单 10-5(**3**)(**4**)：当表项的时间大于表项的生存时间（5 分钟），或者表项状态是 ETHARP_STATE_PENDING 处于等待目标主机回应 ARP 请求包，并且等待的时间超过 ARP_MAXPENDING（5 秒），那么 LwIP 就认为这些表项是无效了，就调用 etharp_free_entry() 函数删除表项。

10.8 ARP 报文

ARP 的请求与应答都是依赖 ARP 报文结构进行的，ARP 报文是放在以太网数据帧中进行发送的，所以下图会将以太网首部一同画出来，具体见图 *10_5*。

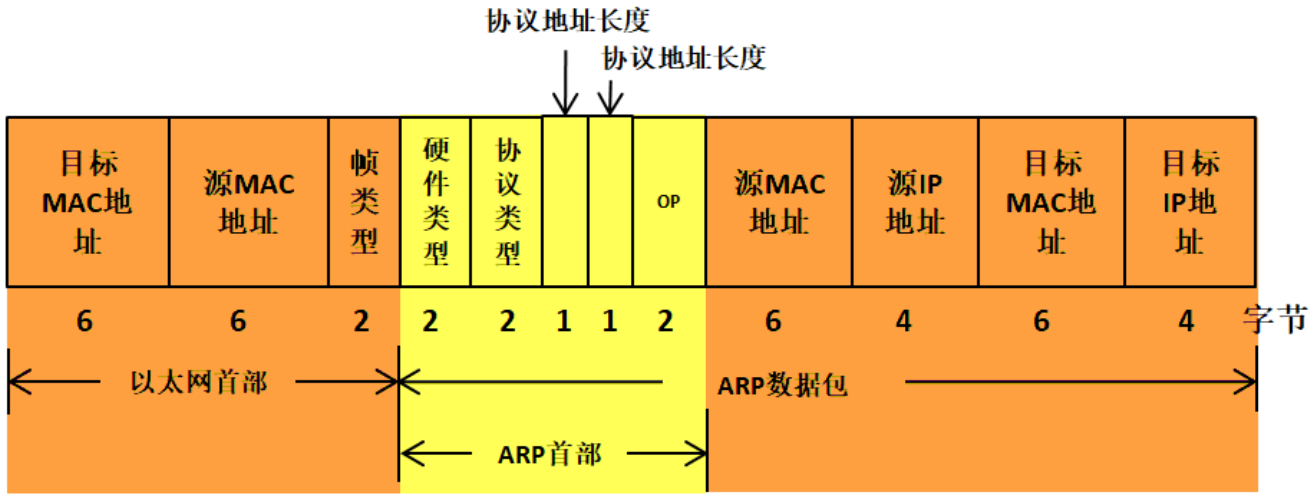


图 10-5ARP 报文

在 ARP 表建立前，主机并不知道目标 MAC 地址，所以在一开始的时候只能通过广播的方式将 ARP 请求包发送出去，处于同一局域网的主机都能接收到广播的数据包。所以一开始目标 MAC 地址是 FF-FF-FF-FF-FF-FF，而以太网首部的帧类型是有多种，对于 ARP 数据包来说，其值为 0x0806，对于 IP 数据报来说，其值为 0x0800，此处我们只需简单了解一下即可，无需记住。

接下来就是 ARP 报文部分，ARP 也是一种协议，也有 ARP 首部，在 ARP 首部一开始的 2 个字节

存储的是硬件类型，表示要知道目标网卡的硬件类型，其中，值为 1 表示以太网地址，其他还可能表示令牌环地址；接下来还有 2 字节的协议类型，表示硬件地址要映射的协议地址类型，其中，0x0800 表示 IP 地址，其他还可能是 ICMP/IGMP 等；接下来有 1 个字节表示硬件地址长度，指出该报文中硬件地址的长度，对于以太网，它的值为 6；还有 1 字节的协议地址长度，对于 ARP 请求或应答来说，该值为 4；ARP 首部最后的 op 字段用于记录 ARP 操作的类型，分别是：

- ARP 请求，其值为 1。
- ARP 应答，其值为 2。
- RARP 请求，其值为 3。
- RARP 应答，其值为 4。

我们只关心 ARP 的请求与应答即可，RARP 是逆地址解析协议，在这里我们就不用去了解，它在网络中基本已经被淘汰，用于主机在启动的时候获得自己的 IP 地址。

对于 ARP 首部后面的四个字段分别是源 MAC 地址、源 IP 地址、目标 MAC 地址、目标 IP 地址，这些就是比较简单的了。

在 ARP 请求包中，除了目标 MAC 地址是未知以外，其他地址 3 个字段都应该填写正确，然后通过广播的形式将该 ARP 请求包发送出去，目标主机接收到该请求包后判断目标 IP 地址与自身 IP 地址是否一致，如果一致则返回 ARP 应答；对应 ARP 应答包，只需要把自己的 MAC 地址填充进去，并且请求包的源主机信息与目标主机信息进行交换位置，然后把 op 字段设置为 2，就返回 ARP 应答包即可。

注意，在发送 ARP 请求包的时候，以太网首部的目标 MAC 地址是 FF-FF-FF-FF-FF-FF，而 ARP 首部目标 MAC 地址为 00-00-00-00-00-00，这里千万不要混淆。

在 LwIP 中，使用了大量的数据结构对 ARP 进行描述，比较麻烦，我们暂时不用去学它，只要知道原理是这样子的即可，关于这些数据结构的定义位于 etharp.h 、ethernet.h 等头文件中，具体见代码清单 10_6。

代码清单 10-6 与 ARP 报文相关的数据结构（已删减）

```
#define ETH_HWADDR_LEN     6      //以太网地址长度
```

(下页继续)

(续上页)

```
truct eth_addr    //以太网地址结构体
{
    PACK_STRUCT_FLD_8(u8_t addr[ETH_HWADDR_LEN]);
} PACK_STRUCT_STRUCT;

struct eth_hdr    //以太网首部
{
    PACK_STRUCT_FLD_S(struct eth_addr dest);  //以太网目标 MAC 地址
    PACK_STRUCT_FLD_S(struct eth_addr src);   //以太网源 MAC 地址
    PACK_STRUCT_FIELD(u16_t type);            //帧类型
} PACK_STRUCT_STRUCT;

struct etharp_hdr                //ARP 报文
{
    PACK_STRUCT_FIELD(u16_t hwtype);    //硬件类型
    PACK_STRUCT_FIELD(u16_t proto);     //协议类型
    PACK_STRUCT_FLD_8(u8_t  hwlen);     //硬件地址长度
    PACK_STRUCT_FLD_8(u8_t  protolen);  //协议地址长度
    PACK_STRUCT_FIELD(u16_t opcode);    //op 字段
    /* 以上是 ARP 报文首部 */

    PACK_STRUCT_FLD_S(struct eth_addr shwaddr);   //源 MAC 地址
    PACK_STRUCT_FLD_S(struct ip4_addr_wordaligned sipaddr);//源 ip 地址
    PACK_STRUCT_FLD_S(struct eth_addr dhwaddr);   //目标 MAC 地址
    PACK_STRUCT_FLD_S(struct ip4_addr_wordaligned dipaddr);//目标 ip 地址
} PACK_STRUCT_STRUCT;

enum etharp_opcode    //op 字段操作
{
    ARP_REQUEST = 1,      //请求包
    ARP_REPLY   = 2       //应答包
};
```

为了加深理解，我们使用 wireshark 网络抓包工具形象地讲解报文格式与内容，关于 wireshark 网络抓包工具的使用方式我们就不做过多讲解，打开工具，然后抓取电脑网络中的数据包，具体见图 *10_6*。

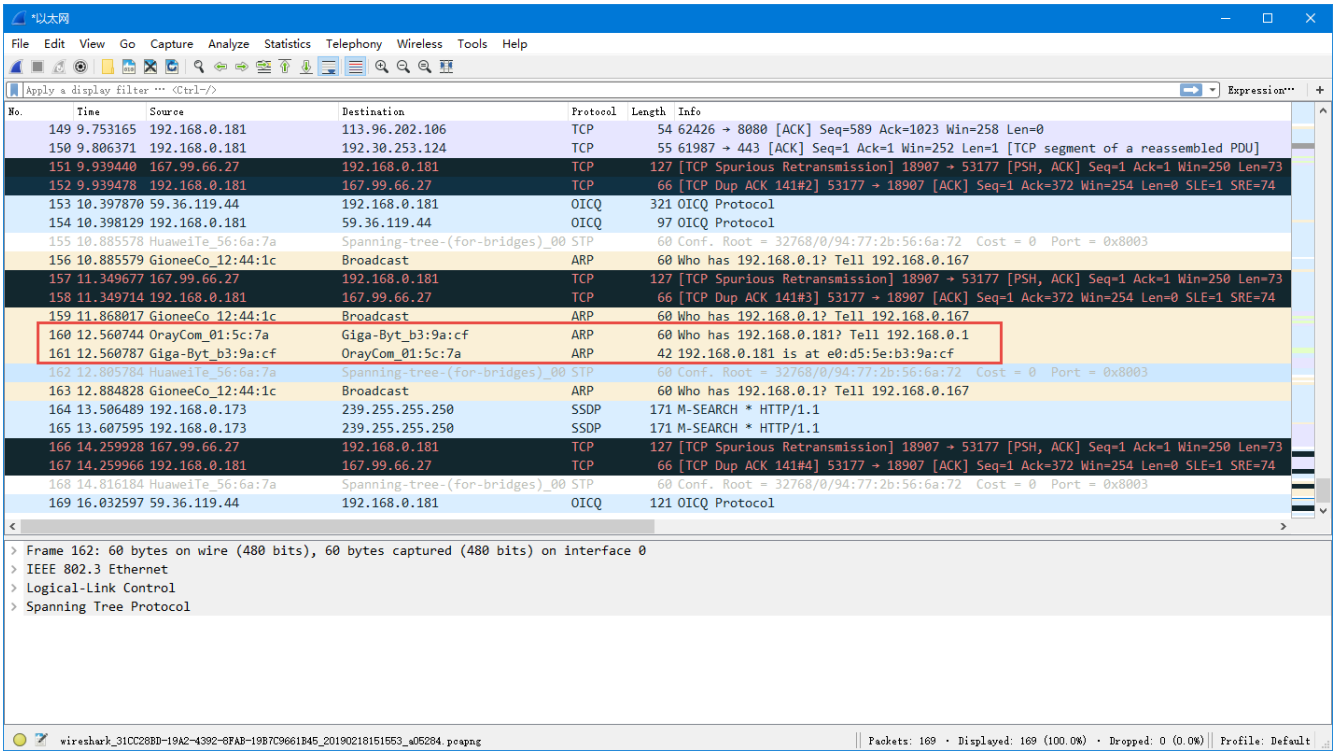


图 10-6 抓包界面

然后我们找到 ARP 协议，双击进行查看 ARP 协议中的数据包，然后我们可以看到第一个 ARP 协议是一个请求包，而第二个 ARP 协议是一个应答包，具体见图 *10_7* 与图 *10_8*。

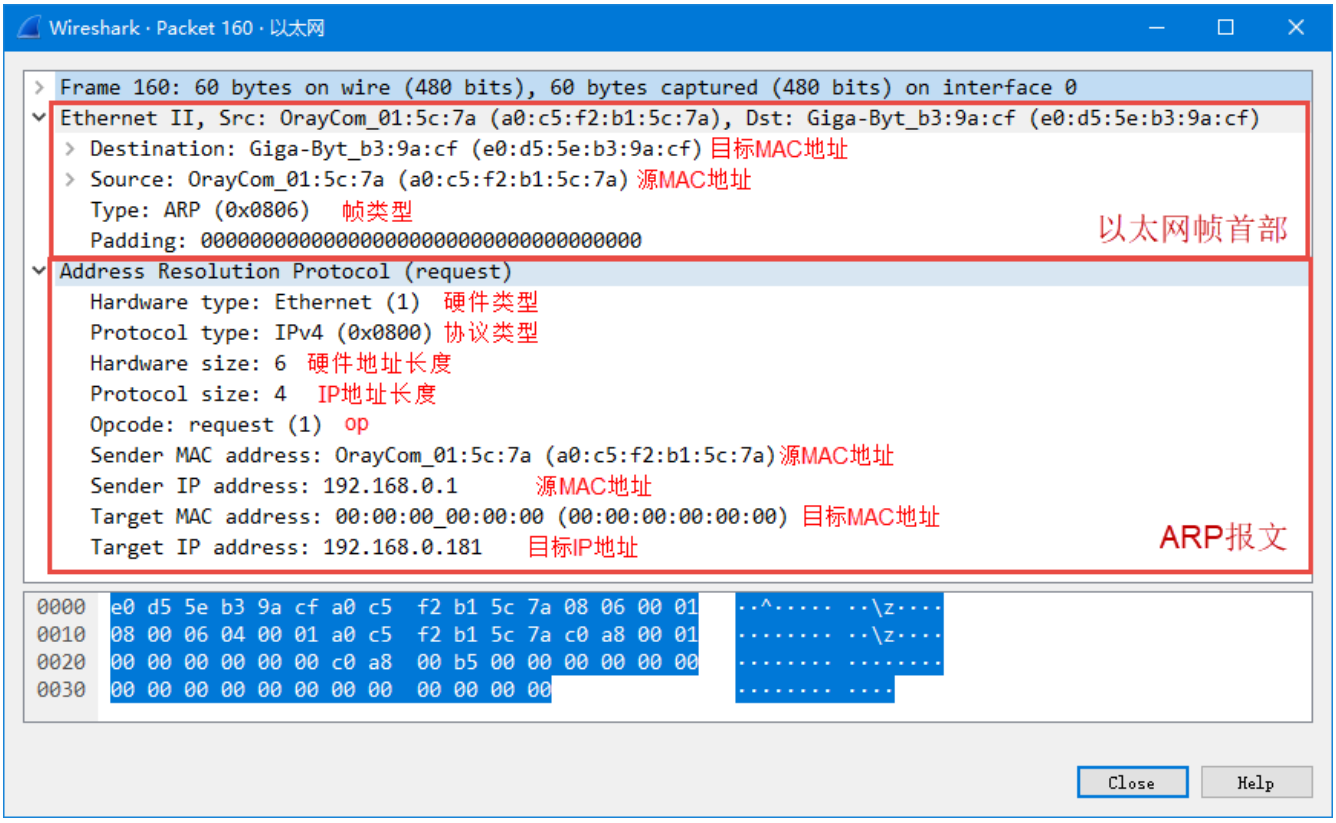


图 10-7ARP 请求包（op 字段为 1）

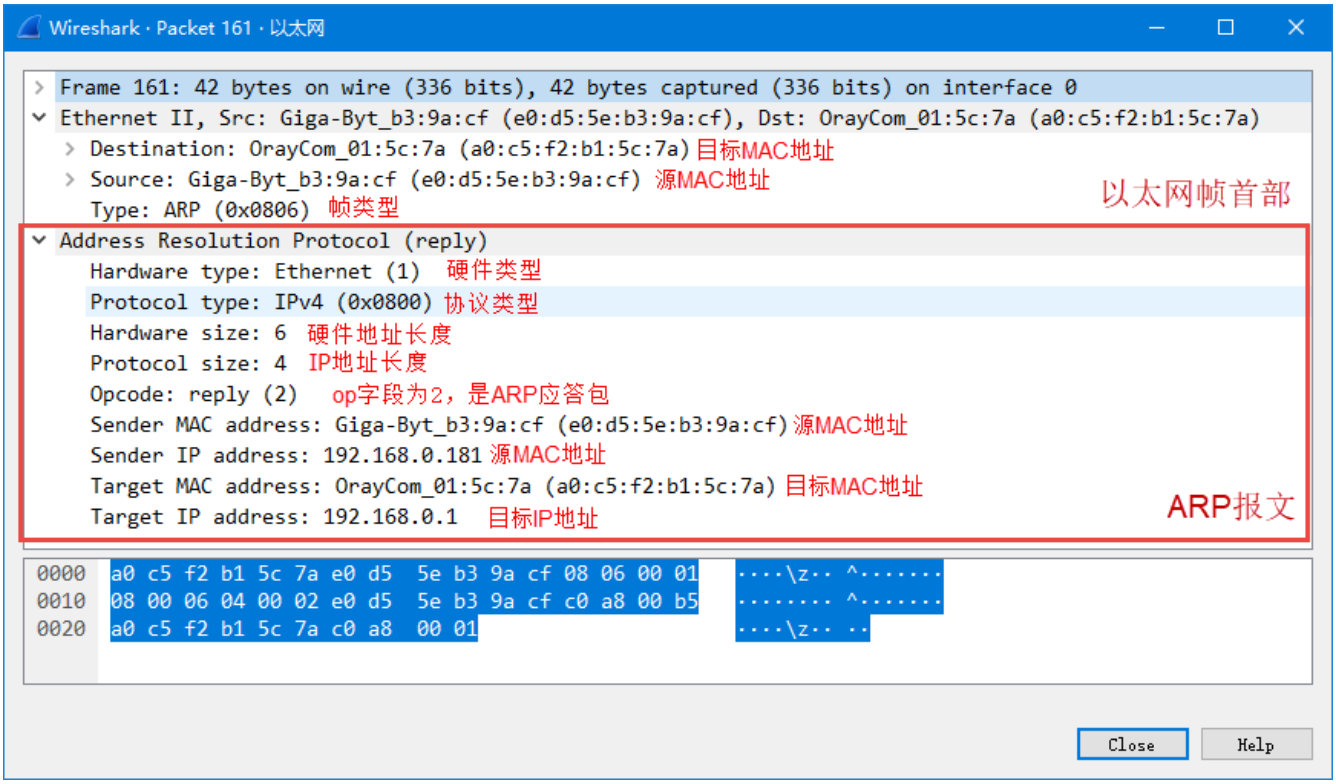


图 10-8 ARP 应答包（op 字段为 2）

10.9 发送 ARP 请求包

发送 ARP 请求包的时候，需要填充已知的目标 IP 地址、源 MAC 地址、源 IP 地址等，并且需要该 ARP 包进行广播出去，所以以太网首部的目标 MAC 地址为 FF-FF-FF-FF-FF-FF，源码具体见*代码清单 10_7*。

代码清单 10-7 发送 ARP 请求包相关源码

```
/* 发送原始 ARP 数据包（操作码和所有地址都可以修改）
* @param netif 用于发送 ARP 数据包的 lwip 网络接口
* @param ethsrc_addr 以太网头的源 MAC 地址
* @param ethdst_addr 以太网头的目标 MAC 地址
* @param hwsrc_addr ARP 协议头的源 MAC 地址
```

（下页继续）

(续上页)

```
* @param ipsrc_addr ARP 协议头的源 IP 地址
* @param hwdst_addr ARP 协议头的目标 MAC 地址
* @param ipdst_addr ARP 协议头的目标 IP 地址
* @param 操作编码 ARP 数据包的类型
* @return ERR_OK 如果已发送 ARP 数据包
* 如果无法分配 ARP 数据包，则为 ERR_MEM
*/
static err_t
etharp_raw(struct netif *netif, //用于发送 ARP 数据包的 lwip 网络接口
        const struct eth_addr *ethsrc_addr,//以太网头的源 MAC 地址
        const struct eth_addr *ethdst_addr,//以太网头的目标 MAC 地址
        const struct eth_addr *hwsrc_addr,//ARP 协议头的源 MAC 地址
        const ip4_addr_t *ipsrc_addr,//ARP 协议头的源 IP 地址
        const struct eth_addr *hwdst_addr, //ARP 协议头的目标 MAC 地址
        const ip4_addr_t *ipdst_addr,// ARP 协议头的目标 IP 地址
        const u16_t opcode)//操作编码 ARP 数据包的类型 (op 字段)
{
    struct pbuf *p;
    err_t result = ERR_OK;
    struct etharp_hdr *hdr;

    //申请 ARP 报文的内存空间
    p = pbuf_alloc(PBUF_LINK, SIZEOF_ETHARP_HDR, PBUF_RAM);

    if (p == NULL)
    {
        ETHARP_STATS_INC(etharp.memerr);//内存申请失败，返回错误代码
        return ERR_MEM;
    }

    //ARP 报文的数据区域，并且强制将起始地址转化成 ARP 报文首部
    hdr = (struct etharp_hdr *)p->payload;
```

(下页继续)

(续上页)

```
    hdr->opcode = lwip_htons(opcode); //填写 ARP 数据包的 op 字段

    //填写源 MAC 地址
    SMEMCPY(&hdr->shwaddr, hwsrc_addr, ETH_HWADDR_LEN);
    //填写目标 MAC 地址
    SMEMCPY(&hdr->dhwaddr, hwdst_addr, ETH_HWADDR_LEN);

    //以太网首部源 MAC 地址
    IPADDR_WORDALIGNED_COPY_FROM_IP4_ADDR_T(&hdr->sipaddr, ipsrc_addr);
    /
    //以太网首部目标 MAC 地址
    IPADDR_WORDALIGNED_COPY_FROM_IP4_ADDR_T(&hdr->dipaddr, ipdst_addr);

    //填写 ARP 首部硬件类型
    hdr->hwtype = PP_HTONS(LWIP_IANA_HWTYPE_ETHERNET);
    //填写 ARP 首部协议类型
    hdr->proto = PP_HTONS(ETHTYPE_IP);

    //填写 ARP 数据包硬件地址长度
    hdr->hwlen = ETH_HWADDR_LEN;
    //填写 ARP 数据包协议地址长度
    hdr->protolen = sizeof(ip4_addr_t);

    //调用底层发送函数将以太网数据帧发送出去
    ethernet_output(netif, p, ethsrc_addr, ethdst_addr, ETHTYPE_ARP);

    ETHARP_STATS_INC(etharp.xmit);

    pbuf_free(p);//发送完成释放内存
    p = NULL;
```

(下页继续)

(续上页)

```
    return result;  //返回结果
}

//FF-FF-FF-FF-FF-FF
const struct eth_addr ethbroadcast =
            {{0xff, 0xff, 0xff, 0xff, 0xff, 0xff}};
//00-00-00-00-00-00
const struct eth_addr ethzero = {{0, 0, 0, 0, 0, 0}};

//发送 ARP 请求包，指定目标 MAC 地址
static err_t
etharp_request_dst(struct netif *netif,
                const ip4_addr_t *ipaddr,
                const struct eth_addr *hw_dst_addr)
{
    return etharp_raw(netif,
                    (struct eth_addr *)netif->hwaddr,
                    hw_dst_addr,
                    (struct eth_addr *)netif->hwaddr,
                    netif_ip4_addr(netif), &ethzero,
                    ipaddr, ARP_REQUEST);
}

//发送 ARP 请求包，目标 MAC 地址为 ethbroadcast
err_t
etharp_request(struct netif *netif, const ip4_addr_t *ipaddr)
{
    return etharp_request_dst(netif, ipaddr, &ethbroadcast);
}
```

总的来说就是先调用 etharp_request() 函数进行发送 ARP 请求包，在 etharp_request() 函数中会调用 etharp_request_dst() 函数进行发送，此时指定的目标 MAC 地址是 ethbroadcast，而在

etharp_request_dst() 函数中会调用 etharp_raw() 进行发送 ARP 请求包，层层调用，并且每层的参数都是越来越多的，这样子封装对于上层程序来说更加好处理，在 etharp_raw() 函数中，会对 ARP 数据包进行封装，然后再封装到以太网数据帧中，最终调用以太网底层发送函数进行将以太网数据帧发送出去。

10.10 数据包接收流程

10.10.1 以太网之数据包接收

以太网是有自己独立的寻址方式（MAC 地址），而对于 TCP/IP 的上层协议（如 TCP 协议、IP 协议），它们是以 IP 地址作为网络的标识，如果没有 IP 地址则无法进行收发数据。当数据通过网卡中接收回来的时候，LwIP 内核就需要将数据进行分解，如果是 IP 数据报则递交给 IP 协议去处理，如果是 ARP 数据包则交由 ARP 协议去处理。LwIP 中数据包从网卡接收的函数是 ethernetif_input()，从第 9 章我们可以知道，真正让 LwIP 内核去处理接收到的数据包是 ethernet_input() 函数，这两个函数是不一样的，名字稍微有点区别，LwIP 就是在这个函数中处理不同的数据包类型，其源码具体见*代码清单 10_8*。

代码清单 10-8 ethernet_input() 函数

```
err_t
ethernet_input(struct pbuf *p, struct netif *netif)
{
    struct eth_hdr *ethhdr;
    u16_t type;

#if LWIP_ARP || ETHARP_SUPPORT_VLAN || LWIP_IPV6

    u16_t next_hdr_offset = SIZEOF_ETH_HDR;

#endif
```

（下页继续）

(续上页)

```
    LWIP_ASSERT_CORE_LOCKED();

    //校验数据长度
    if (p->len <= SIZEOF_ETH_HDR)
    {
        ETHARP_STATS_INC(etharp.proterr);
        ETHARP_STATS_INC(etharp.drop);
        MIB2_STATS_NETIF_INC(netif, ifinerrors);
        goto free_and_return;
    }

    if (p->if_idx == NETIF_NO_INDEX)
    {
        p->if_idx = netif_get_index(netif);
    }

    /* ethhdr 指针指向以太网帧头部，并且强制转换成 eth_hdr 结构 */
    ethhdr = (struct eth_hdr *)p->payload;                    (1)

    type = ethhdr->type;

    if (ethhdr->dest.addr[0] & 1)
    {
        /* 这可能是多播或广播数据包 */
        if (ethhdr->dest.addr[0] == LL_IP4_MULTICAST_ADDR_0)
        {
            if ((ethhdr->dest.addr[1] == LL_IP4_MULTICAST_ADDR_1) &&
                    (ethhdr->dest.addr[2] == LL_IP4_MULTICAST_ADDR_2))
            {
                /* 将 pbuf 标记为链路层多播 */
                p->flags |= PBUF_FLAG_LLMCAST;                (2)
            }
```

(下页继续)

(续上页)

```
        }

        else if (eth_addr_cmp(&ethhdr->dest, &ethbroadcast))
        {
            /* 将 pbuf 标记为链路层广播 */
            p->flags |= PBUF_FLAG_LLBCAST;                    (3)
        }
    }

    switch (type)
    {
    /* 如果是 IP 数据报 */
    case PP_HTONS(ETHTYPE_IP):
        if (!(netif->flags & NETIF_FLAG_ETHARP))
        {
            goto free_and_return;
        }
        /* 跳过以太网首部 */
        if (pbuf_remove_header(p, next_hdr_offset))           (4)
        {
            goto free_and_return;
        }
        else
        {
            /* 传递到 IP 协议去处理 */
            ip4_input(p, netif);                              (5)
        }
        break;

    //对于是 ARP 包
    case PP_HTONS(ETHTYPE_ARP):
        if (!(netif->flags & NETIF_FLAG_ETHARP))
```

(下页继续)

(续上页)

```
        {
            goto free_and_return;
        }
        /* 跳过以太网首部 */
        if (pbuf_remove_header(p, next_hdr_offset))          (6)
        {
            ETHARP_STATS_INC(etharp.lenerr);
            ETHARP_STATS_INC(etharp.drop);
            goto free_and_return;
        }
        else
        {
            /* 传递到 ARP 协议处理 */
            etharp_input(p, netif);                          (7)
        }
        break;

//如果支持 PPPOE
#if PPPOE_SUPPORT
    case PP_HTONS(ETHTYPE_PPPOEDISC):
        pppoe_disc_input(netif, p);
        break;

    case PP_HTONS(ETHTYPE_PPPOE):
        pppoe_data_input(netif, p);
        break;
#endif /* PPPOE_SUPPORT */

//如果支持 ipv6
#if LWIP_IPV6
    case PP_HTONS(ETHTYPE_IPV6): /* IPv6 */
        /* skip Ethernet header */
```

(下页继续)

(续上页)

```
        if ((p->len < next_hdr_offset) ||
                pbuf_remove_header(p, next_hdr_offset))
        {
            goto free_and_return;
        }
        else
        {
            /* pass to IPv6 layer */
            ip6_input(p, netif);
        }
        break;
#endif /* LWIP_IPV6 */

    default:
#ifdef LWIP_HOOK_UNKNOWN_ETH_PROTOCOL
        if (LWIP_HOOK_UNKNOWN_ETH_PROTOCOL(p, netif) == ERR_OK)
        {
            break;
        }
#endif
        ETHARP_STATS_INC(etharp.proterr);
        ETHARP_STATS_INC(etharp.drop);
        MIB2_STATS_NETIF_INC(netif, ifinunknownprotos);
        goto free_and_return;
    }

    return ERR_OK;

free_and_return:
    pbuf_free(p);
    return ERR_OK;
}
```

代码清单 10-8(**1**)：ethhdr 指针指向以太网帧首部，并且强制转换成 eth_hdr 结构，这是为了方便对以太网帧首部进行操作。

代码清单 10-8(**2**)：如果目标 IP 地址的第一个字节的 bit0 是 1，那么有可能是多播或者是广播数据包，所以，还需要进行判断，如果是多播的，就将 pbuf 标记为链路层多播。

代码清单 10-8(**3**)：如果是广播的，就将 pbuf 标记为链路层广播。

代码清单 10-8(**4**)：如果数据包是 ETHTYPE_IP 类型，则调用 pbuf_remove_header() 函数跳过以太网帧首部，方便对数据进行操作。

代码清单 10-8(**5**)：除去以太网帧首部成功，调用 ip4_input() 函数将数据包递交到 IP 协议去处理，对于 IP 层的处理，我们在后面的章节中讲解。

代码清单 10-8(**6**)：跳过以太网帧首部。

代码清单 10-8(**7**)：除去以太网帧首部成功，调用 etharp_input () 函数将数据包递交到 ARP 协议去处理。

10.10.2 ARP 数据包处理

ARP 数据包的处理函数为 etharp _input()，在这里它完成两个任务：

1. 如果收到的是 ARP 应答包，说明本机之前发出的 ARP 请求包有了回应，就根据应答包更新自身的 ARP 缓存表；

2. 如果收到的是 ARP 请求包，如果包中的目标 IP 地址与主机 IP 地址匹配，除了记录原主机的 IP 与 MAC 地址，更新自身的 ARP 表外，还要向源主机发送一个 ARP 应答包。但是如果包中目标 IP 地址与主机 IP 地址不匹配，则尽可能记录源主机的 IP 与 MAC 地址，更新自身的 ARP 表，并丢弃该请求包，为什么说是尽可能呢，因为主机的 ARP 缓存表是有限的，不可能记录太多的 ARP 表项，所以在有空闲的表项时才记录，如果没有空闲的表项，ARP 觉得它自己已经尽力了，也记不住那么多表项。

etharp_input() 函数的源码具体见代码清单 10-9。

代码清单 10-9 etharp_input() 源码

```
void
etharp_input(struct pbuf *p, struct netif *netif)
{
    struct etharp_hdr *hdr;

    ip4_addr_t sipaddr, dipaddr;
    u8_t for_us;

    LWIP_ASSERT_CORE_LOCKED();

    LWIP_ERROR("netif != NULL", (netif != NULL), return;);

    hdr = (struct etharp_hdr *)p->payload;

    /* 判断 ARP 包的合法性 */
    if ((hdr->hwtype != PP_HTONS(LWIP_IANA_HWTYPE_ETHERNET)) ||
            (hdr->hwlen != ETH_HWADDR_LEN) ||
            (hdr->protolen != sizeof(ip4_addr_t)) ||
            (hdr->proto != PP_HTONS(ETHTYPE_IP)))            (1)
    {
        LWIP_DEBUGF(ETHARP_DEBUG | LWIP_DBG_TRACE | LWIP_DBG_LEVEL_WARNING,
        ("etharp_input: packet dropped, wrong hw type, hwlen, proto,
↪protolen or ethernet type(%"U16_F"/%"U16_F"/%"U16_F"/%"U16_F")\n",hdr->
↪hwtype,(u16_t)hdr->hwlen, hdr->proto, (u16_t)hdr->protolen));

        ETHARP_STATS_INC(etharp.proterr);
        ETHARP_STATS_INC(etharp.drop);

        pbuf_free(p);
        return;
    }
    ETHARP_STATS_INC(etharp.recv);
```

(下页继续)

(续上页)

```
//拷贝源 IP 地址与目标 IP 地址
IPADDR_WORDALIGNED_COPY_TO_IP4_ADDR_T(&sipaddr, &hdr->sipaddr); (2)
IPADDR_WORDALIGNED_COPY_TO_IP4_ADDR_T(&dipaddr, &hdr->dipaddr); (3)

/* 看看主机网卡是否配置了 IP 地址 */
if (ip4_addr_isany_val(*netif_ip4_addr(netif)))          (4)
{
    for_us = 0;
}
else
{
    /* 判断目标 IP 地址与主机 IP 地址是否一样 */
    for_us = (u8_t)ip4_addr_cmp(&dipaddr, netif_ip4_addr(netif)); (5)
}

/* 更新 ARP 缓存表 */
etharp_update_arp_entry(netif, &sipaddr, &(hdr->shwaddr),        (6)
            for_us ? ETHARP_FLAG_TRY_HARD : ETHARP_FLAG_FIND_ONLY);

/* 更新完毕，根据包的类型处理 */
switch (hdr->opcode)                                          (7)
{
/* ARP request? */
case PP_HTONS(ARP_REQUEST):                                   (8)
    /* ARP 请求包 */
    LWIP_DEBUGF (ETHARP_DEBUG | LWIP_DBG_TRACE,
                 ("etharp_input: incoming ARP request\n"));
    /* 是请求自己的 */
    if (for_us)
    {
        /* 做出回应 */
```

(下页继续)

(续上页)

```
            etharp_raw(netif,
                    (struct eth_addr *)netif->hwaddr, &hdr->shwaddr,
                (struct eth_addr *)netif->hwaddr, netif_ip4_addr(netif),
                    &hdr->shwaddr, &sipaddr,
                    ARP_REPLY);                                          (9)
            /* 不是给自己的 */
        }
        else if (ip4_addr_isany_val(*netif_ip4_addr(netif)))             (10)
        {
            LWIP_DEBUGF(ETHARP_DEBUG | LWIP_DBG_TRACE,
                    ("etharp_input: we are unconfigured, ARP request␣
↪ignored.\n"));
        }
        else                                                             (11)
        {
            LWIP_DEBUGF(ETHARP_DEBUG | LWIP_DBG_TRACE,
                        ("etharp_input: ARP request was not for us.\n"));
        }
        break;
    case PP_HTONS(ARP_REPLY):                                            (12)
        /* 对于 ARP 应答包 */
        LWIP_DEBUGF(ETHARP_DEBUG | LWIP_DBG_TRACE,
                    ("etharp_input: incoming ARP reply\n"));
        break;
    default:
        LWIP_DEBUGF(ETHARP_DEBUG | LWIP_DBG_TRACE,
                    ("etharp_input: ARP unknown opcode type %"S16_F"\n",
                    lwip_htons(hdr->opcode)));
        ETHARP_STATS_INC(etharp.err);                                    (13)
        break;
    }
    /* 释放内存 */
```

(下页继续)

(续上页)

```
    pbuf_free(p);                                                    (14)
}
```

代码清单 10-9(**1**)：判断 ARP 包的合法性，已经类型是否为以太网、硬件地址长度是否为 ETH_HWADDR_LEN、协议地址长度是否为 sizeof(ip4_addr_t) 以及协议是否为 ARP 协议，如果都满足则表示 ARP 包合法。

代码清单 10-9(**2**)：拷贝源 IP 地址到 sipaddr 变量中，因为在 ARP 包中的 IP 地址字段并不是对齐的，不能直接使用，所以需要拷贝到临时变量，方便直接操作。

代码清单 10-9(**3**)：同理拷贝目标 IP 地址到 dipaddr 变量中。

代码清单 10-9(**4**)：看看主机网卡是否配置了 IP 地址，如果没有配置，将 for_us 变量设置为 0，表示不是给主机自己的 ARP 包。

代码清单 10-9(**5**)：调用 ip4_addr_cmp() 函数判断目标 IP 地址与主机 IP 地址是否一样，如果一样则返回 1，将 for_us 变量设置为 1，反之设置为 0。

代码清单 10-9(**6**)：调用 etharp_update_arp_entry() 函数更新 ARP 缓存表，这个操作有点特殊，我们稍后讲解。

代码清单 10-9(**7**)：更新完毕，根据包的类型处理，即根据 ARP 数据包的 op 字段进行处理。

代码清单 10-9(**8**)：对于 ARP 请求包，首先要判断一下是否是给主机自己的，如果是则要回应，否则就直接丢弃即可。

代码清单 10-9(**9**)：是请求自己的，调用 etharp_raw() 函数作出应答。

代码清单 10-9(**10**)：如果不是给自己的，原因有两种，一种是网卡自身尚未配置 IP 地址，这样子就只打印相关调试信息。

代码清单 10-9(**11**)：另一种是 ARP 包中的目标 IP 地址与主机 IP 地址不符合，也不用做出回应，直接丢弃即可，并输出相关调试信息。

代码清单 10-9(**12**)：对于 ARP 应答包，理论上应该更新 ARP 缓存表的，毕竟发出去的 ARP 请求包得到回应，但是在前面已经更新了缓存表了，此处就不用重复更新了。

代码清单 10-9(**13**)：对于其他情况，直接返回错误代码。

代码清单 10-9(**14**)：释放内存。

10.10.3 更新 ARP 缓存表

etharp_update_arp_entry() 函数是用于更新 ARP 缓存表的，它会在收到一个 ARP 数据包的时候被调用，它会先查找一个 ARP 表项，如果没有找到这个 ARP 表项的记录，就会去新建一个 ARP 表项，然后重置 ARP 表项的参数（状态、网卡。IP 地址与对应的 MAC 地址以及生存时间等），然后检测 ARP 表项中是否挂载数据包，如果有就将这些数据包发送出去，其源码具体见*代码清单 10_10*。

代码清单 10-10 etharp_update_arp_entry() 函数源码

```
static err_t
etharp_update_arp_entry(struct netif *netif,                          (1)
                        const ip4_addr_t *ipaddr,                     (2)
                        struct eth_addr *ethaddr,                     (3)
                        u8_t flags)                                   (4)
{
    s16_t i;

    /* non-unicast address? */
    if (ip4_addr_isany(ipaddr) ||
            ip4_addr_isbroadcast(ipaddr, netif) ||
            ip4_addr_ismulticast(ipaddr))                             (5)
    {
        return ERR_ARG;
    }
    /* 查找或者创建 ARP 表项，并且返回索引值 */
    i = etharp_find_entry(ipaddr, flags, netif);                      (6)

    /* 如果索引值不合法，更新 ARP 表项失败 */
    if (i < 0)
    {
```

(下页继续)

(续上页)

```
        return (err_t)i;
    }

    /* 设置表项状态为 ETHARP_STATE_STABLE */
    arp_table[i].state = ETHARP_STATE_STABLE;                          (7)

    /* 记录网卡 */
    arp_table[i].netif = netif;                                        (8)
    /* 插入 ARP 索引树 */
    mib2_add_arp_entry(netif, &arp_table[i].ipaddr);                   (9)

    /* 更新缓存表中的 MAC 地址 */
    SMEMCPY(&arp_table[i].ethaddr, ethaddr, ETH_HWADDR_LEN);           (10)
    /* 重置生存时间 */
    arp_table[i].ctime = 0;                                            (11)

    /* 如果表项上与未发送的数据包，那就将这些数据包发送出去 */
#if ARP_QUEUEING
    while (arp_table[i].q != NULL)
    {
        struct pbuf *p;
        /* 定义 q 指向 ARP 表项中的数据包缓存队列 */
        struct etharp_q_entry *q = arp_table[i].q;                     (12)
        /* 指向下一个数据包节点 */
        arp_table[i].q = q->next;                                      (13)
        /* 获取 pbuf 数据包 */
        p = q->p;                                                      (14)
        /* 释放 MEMP_ARP_QUEUE 类型的内存块 */
        memp_free(MEMP_ARP_QUEUE, q);                                  (15)
#else /* ARP_QUEUEING */
    if (arp_table[i].q != NULL)
    {
```

(下页继续)

(续上页)

```
        struct pbuf *p = arp_table[i].q;                    (16)
        arp_table[i].q = NULL;
#endif /* ARP_QUEUEING */
        /* 发送缓存队列的数据包 */
        ethernet_output(netif, p,
                        (struct eth_addr *)(netif->hwaddr),
                        ethaddr,
                        ETHTYPE_IP);                                (17)
        /* 释放 pbuf 数据包的内存空间 */
        pbuf_free(p);                                               (18)
    }
    return ERR_OK;
}
```

代码清单 10-10**(1)**：对应表项中的网卡。

代码清单 10-10**(2)**：对应表项中的 IP 地址。

代码清单 10-10**(3)**：对应表项中的 MAC 地址。

代码清单 10-10**(4)**：表项的更新方式，动态表项有两种方式，分别为 ETHARP_FLAG_TRY_HARD 和 ETHARP_FLAG_FIND_ONLY。前者表示无论如何都要创建一个表项，如果 ARP 缓存表中没有空间了，那就需要回收较老的表项，将他们删除，然后建立新的表项。而如果是后者，就让内核尽量更新表项，如果 ARP 缓存表中没有空间了，那么也无能为力，实在是添加不了新的表项。

代码清单 10-10**(5)**：IP 地址验证，ARP 数据包中的目标 IP 地址不能是广播、多播地址。

代码清单 10-10**(6)**：调用 etharp_find_entry() 函数查找或者创建 ARP 表项，并且返回索引值，如果索引值不合法，表示更新 ARP 表项失败，该函数比较复杂，此处就不做过多讲解，想要了解的可以在源码中查看该函数。

代码清单 10-10**(7)**：设置表项状态为 ETHARP_STATE_STABLE。

代码清单 10-10**(8)**：记录网卡信息。

代码清单 10-10**(9)**：插入 ARP 索引树。

代码清单 10-10(**10**)：更新表项中的 MAC 地址。

代码清单 10-10(**11**)：重置表项的生存时间。

代码清单 10-10(**12**)：如果表项上与未发送的数据包，那就将这些数据包发送出去。这里通过宏定义 ARP_QUEUEING 采用哪种方式发送数据包，如果定义了缓存数据包队列，那就需要将队列上的所有数据包发送出去，定义 q 指向 ARP 表项中的数据包缓存队列。

代码清单 10-10(**13**)：指向下一个数据包节点。

代码清单 10-10(**14**)：获取 pbuf 数据包。

代码清单 10-10(**15**)：释放 MEMP_ARP_QUEUE 类型的内存块。

代码清单 10-10(**16**)：此处是单个数据包挂载到表项上，无需太多操作，直接将数据包获取到，然后发送出去即可。

代码清单 10-10(**17**)：调用 ethernet_output() 函数发送挂载在表项上的数据包。

代码清单 10-10(**18**)：释放 pbuf 数据包的内存空间。

整个 ARP 处理的流程示意图具体见图 *10_9*。

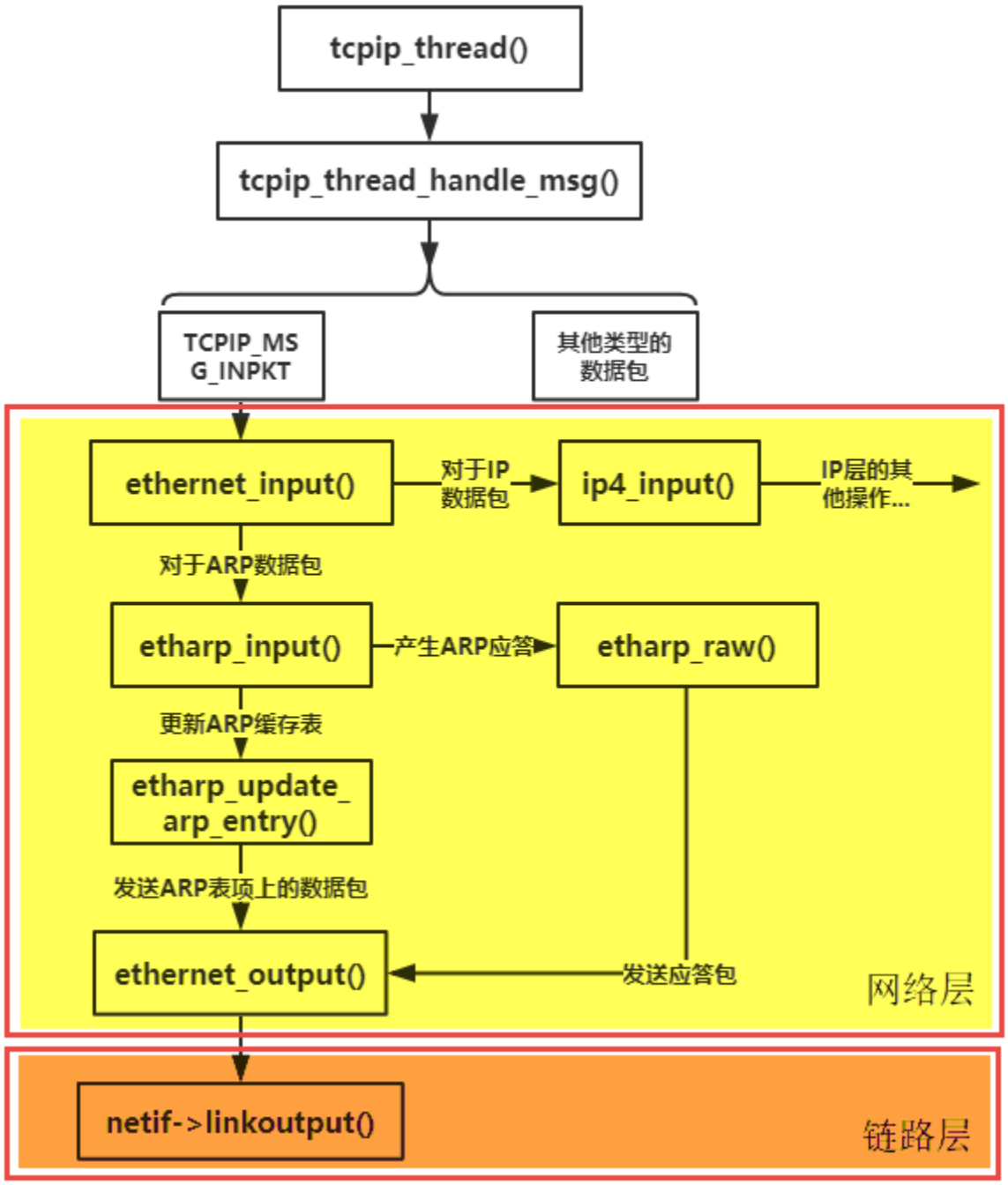


图 10-9ARP 协议处理的流程示意图

10.11 数据包发送流程

经过学习，我们知道一个数据包从底层传递进来的流程是怎么样的，如果是 ARP 数据包就会给 ARP 去处理，如果是 IP 数据报就使用 ip4_input() 函数传递到上层，这些处理在后面的章节讲解。那么如果上层协议想要发送数据，也肯定需要经过 ARP 协议将 IP 地址映射为 MAC 地址才能完成发送操作，IP 数据报通过 ip4_output() 函数将上层数据包传递到 ARP 协议处理，关于 IP 协议是怎么样传递的我们暂且不说，那么 ARP 通过 etharp_output() 函数接收到 IP 数据报后，就会进行发送，ARP 会先从数据包中进行分析，看看这个 IP 数据报是单播数据包还是多播或者是广播数据包，然后进行不同的处理：

- 对于多播或者是广播数据包，这种处理就很简单，直接将数据包丢给网卡就行了（调用 ethernet_output() 函数）。
- 对于单播包的处理稍微麻烦一点，ARP 协议需要根据 IP 地址找到对应的 MAC 地址，然后才能正确发送，如果找不到 MAC 地址的话，还要延迟发送数据包，ARP 协议首先会创建一个 ARP 表项，然后将数据包挂到 ARP 表项对应的缓存队列上，与此同时会发出一个 ARP 请求包，等待目标主机的回应后再发送 IP 数据报。

此处需要注意的是，对于 PBUFF_ERF、PBUF_POOL、PBUF_RAM 类型的数据包是不允许直接挂到 ARP 表项对应的缓存队列上的，因为此时内核需要等待目标主机的 ARP 应答，而这段时间里，这些数据有可能会被上层改动，这是不允许的，所以 LwIP 需要将这些 pbuf 数据包拷贝到新的空间，等待发送。

10.11.1 etharp_output() 函数

etharp_output() 函数被 IP 层的 ip4_output() 函数调用，IP 层传递一个数据包到 ARP 中，etharp_output() 会根据数据包的目标 IP 地址选择不同的处理，其源码具体见*代码清单 10_11*。

代码清单 10-11 etharp_output() 源码

```
const struct eth_addr ethbroadcast =
                {{0xff, 0xff, 0xff, 0xff, 0xff, 0xff}};
```

(下页继续)

(续上页)

```
const struct eth_addr ethzero = {{0, 0, 0, 0, 0, 0}};

/** 24 位 IANA IPv4 多播 OUI 为 01-00-5e:  */
#define LL_IP4_MULTICAST_ADDR_0 0x01
#define LL_IP4_MULTICAST_ADDR_1 0x00
#define LL_IP4_MULTICAST_ADDR_2 0x5e

err_t etharp_output
    (struct netif *netif, struct pbuf *q, const ip4_addr_t *ipaddr)
{
    const struct eth_addr *dest;
    struct eth_addr mcastaddr;
    const ip4_addr_t *dst_addr = ipaddr;

    LWIP_ASSERT_CORE_LOCKED();
    LWIP_ASSERT("netif != NULL", netif != NULL);
    LWIP_ASSERT("q != NULL", q != NULL);
    LWIP_ASSERT("ipaddr != NULL", ipaddr != NULL);

    if (ip4_addr_isbroadcast(ipaddr, netif))
    {
        /* 如果是广播数据包，目标 MAC 地址设置为 FF-FF-FF-FF-FF-FF-FF */
        dest = (const struct eth_addr *)&ethbroadcast;       (1)
        /* multicast destination IP address? */
    }
    else if (ip4_addr_ismulticast(ipaddr))
    {
        /* 如果是多播数据包，目标 MAC 地址设置为多播地址：01-00-5E-XX-XX-XX*/
        mcastaddr.addr[0] = LL_IP4_MULTICAST_ADDR_0;
        mcastaddr.addr[1] = LL_IP4_MULTICAST_ADDR_1;
        mcastaddr.addr[2] = LL_IP4_MULTICAST_ADDR_2;
        mcastaddr.addr[3] = ip4_addr2(ipaddr) & 0x7f;
```

(下页继续)

(续上页)

```
        mcastaddr.addr[4] = ip4_addr3(ipaddr);
        mcastaddr.addr[5] = ip4_addr4(ipaddr);

        dest = &mcastaddr;                                  (2)

    }
    else
    {
        /* 如果是单播目标地 IP 地址 */
        netif_addr_idx_t i;
        /* 判断目标 IP 地址是否与主机处于同一子网上,
        如果不是,则修改 IP 地址 */
        if (!ip4_addr_netcmp(ipaddr, netif_ip4_addr(netif),
                             netif_ip4_netmask(netif)) &&
                !ip4_addr_islinklocal(ipaddr))                (3)
        {
#if LWIP_AUTOIP
            struct ip_hdr *iphdr =
                LWIP_ALIGNMENT_CAST(struct ip_hdr *, q->payload);

            if (!ip4_addr_islinklocal(&iphdr->src))
#endif
            {
#ifdef LWIP_HOOK_ETHARP_GET_GW
                dst_addr = LWIP_HOOK_ETHARP_GET_GW(netif, ipaddr);
                if (dst_addr == NULL)
#endif
                {
                    /* 判断一下网关地址是否有效 */
                    if (!ip4_addr_isany_val(*netif_ip4_gw(netif))) (4)
                    {
                        /* 发送到默认网关,让网关进行转发 */
```

(下页继续)

(续上页)

```
                    dst_addr = netif_ip4_gw(netif);
                    /* 没有默认网关可用 */
                }
                else
                {
                    /* 返回错误 */
                    return ERR_RTE;
                }
            }
        }
        /* 遍历 ARP 缓存表 */
        for (i = 0; i < ARP_TABLE_SIZE; i++)
        {
            if ((arp_table[i].state >= ETHARP_STATE_STABLE) &&
                    (arp_table[i].netif == netif) &&
                    (ip4_addr_cmp(dst_addr, &arp_table[i].ipaddr)))
            {
                /* 如果找到目标 IP 地址对应的表项，直接发送 */
                ETHARP_SET_ADDRHINT(netif, i);
                return etharp_output_to_arp_index(netif, q, i); (5)
            }
        }
        /* 如果没有找到与目标 IP 地址对应的 ARP 表项 */
        return etharp_query(netif, dst_addr, q);              (6)
    }

    /* 对于多播、广播数据包，直接能得到对应的 MAC 地址，可以进行发送 */
    return ethernet_output(netif, q,
                (struct eth_addr *)(netif->hwaddr), dest, ETHTYPE_IP); (7)
}
```

代码清单 10-11(**1**)：如果是广播数据包，目标 MAC 地址设置为 FF-FF-FF-FF-FF-FF-FF。

代码清单 10-11(**2**)：如果是多播数据包，目标 MAC 地址设置为多播地址：01-00-5E-XX-XX-XX。

此处简单补充一下单播包、广播包与多播包的相关知识：

单播包：顾名思义，就是一对一通信，发送的目标主机 IP 地址是唯一的，就像是人们之间的对话一样，一个人对另外一个人说话。

多播包：“多播”可以理解为一个人向多个人（但不是在场的所有人）说话，比如在一个大餐厅中，一个人说话只能让一桌人知道，而其他桌上的人并不知道说了什么。同理的，主机发送的多播包只能让某些满足条件的目标主机接收到。

广播包：而广播就是类似于用大喇叭进行广播通知，在场的所有人都能知道。广播包是让所有处于同一子网的主机都能接收到数据包。

代码清单 10-11(**3**)：如果是单播目标地 IP 地址，首先判断目标 IP 地址是否与主机处于同一子网上，如果不是，则修改 IP 地址，IP 地址为网关的 IP 地址，目的是为了让网关进行转发。

代码清单 10-11(**4**)：判断一下网关地址是否有效，如果有效，则发送到默认网关，让网关进行转发，没有默认网关可用则返回错误代码。

代码清单 10-11(**5**)：遍历 ARP 缓存表，如果找到目标 IP 地址对应的表项，调用 etharp_output_to_arp_index() 函数直接发送，该函数源码具体见代码清单 10-12。

代码清单 10-11(**6**)：如果没有找到与目标 IP 地址对应的 ARP 表项，需要调用 etharp_query() 函数进行发送，这个函数在稍后讲解，具体见 10.11.3 小节。

代码清单 10-11(**7**)：对于多播、广播数据包，直接能得到对应的 MAC 地址，可以进行发送。

10.11.2 etharp_output_to_arp_index() 函数

这个函数是 ARP 找到了 IP 地址与 MAC 地址对应的表项，从而能直接进行发送，除此之外，ARP 还需要更新 ARP 表项，我们知道，LwIP 中的 ARP 表项生存时间是 5 分钟（300 秒），那么在 APP 表项的生存时间即将到来的时候，ARP 需要更新表项，为什么要在发送数据的时候更新呢？因为如果不发送数据，那就没必要更新 ARP 表项，这样子表项在生存时间到来的时候就会被系统删除，回收 ARP 表项空间，而一直使用的 ARP 表项需要是谁更新，更新的方式也有两种：

如果 ARP 表项还差 15 秒就过期了，LwIP 会通过广播的方式发送一个 ARP 请求包，试图得到主机的回应。

而如果 ARP 表项还差 30 秒就过期了，那么 LwIP 会通过单播的方式向目标主机发送一个请求包并试图得到回应。

在这种情况下发送 ARP 请求包的时候，表项的状态会由 ETHARP_STATE_STABLE 变成 ETHARP_STATE_STABLE_REREQUESTING_1，如果目标主机回应了，那就更新 ARP 缓存表中的表项。

当然，如果还没那么快到期的话，那就直接调用 ethernet_output() 函数将数据包传递给网卡进行发送。函数源码具体见*代码清单 10_12*

代码清单 10-12 etharp_output_to_arp_index() 函数源码

```
#define ARP_MAXAGE                          300

/* 即将到期的时间 */
#define ARP_AGE_REREQUEST_USED_UNICAST    (ARP_MAXAGE - 30)
#define ARP_AGE_REREQUEST_USED_BROADCAST (ARP_MAXAGE - 15)

static err_t
etharp_output_to_arp_index(struct netif *netif,
                           struct pbuf *q,
                           netif_addr_idx_t arp_idx)
{
    LWIP_ASSERT("arp_table[arp_idx].state >= ETHARP_STATE_STABLE",
                arp_table[arp_idx].state >= ETHARP_STATE_STABLE);
    /* 如果 arp 表项即将过期: LwIP 会重新请求它,
        但只有当它的状态是 ETHARP_STATE_STABLE 才能请求 */
    if (arp_table[arp_idx].state == ETHARP_STATE_STABLE)
    {
        /* 还差 15 秒到期 */
        if (arp_table[arp_idx].ctime >= ARP_AGE_REREQUEST_USED_BROADCAST)
        {
```

(下页继续)

(续上页)

```
            /* 使用广播方式发出请求包 */
        if (etharp_request(netif, &arp_table[arp_idx].ipaddr) == ERR_OK)
            {
            arp_table[arp_idx].state = ETHARP_STATE_STABLE_REREQUESTING_1;
            }
        }
        /* 还差 30 秒到期 */
    else if (arp_table[arp_idx].ctime >= ARP_AGE_REREQUEST_USED_UNICAST)
        {
            /* 发出单播请求（持续 15 秒），以防止不必要的广播 */
            if (etharp_request_dst(netif,
                                   &arp_table[arp_idx].ipaddr,
                                   &arp_table[arp_idx].ethaddr) == ERR_OK)
            {
            arp_table[arp_idx].state = ETHARP_STATE_STABLE_REREQUESTING_1;
            }
        }
    }

    return ethernet_output(netif, q,
                           (struct eth_addr *)(netif->hwaddr),
                           &arp_table[arp_idx].ethaddr, ETHTYPE_IP);
}
```

10.11.3 etharp_query() 函数

如果在 ARP 缓存表中没有找到目标 IP 地址对应的表项，那么 ARP 协议就会创建一个表项，这也是 ARP 协议的核心处理，对于刚创建的表项，它在初始化网卡信息后会被设置为 ETHARP_STATE_PENDING 状态，与此同时一个 ARP 请求包将被广播出去，这个时候的表项是无法发送数据的，只有等待到目标主机回应了一个 ARP 应答包才能发送数据，那么这些数据在这段时间中将被挂到表项的等待队列上，在 ARP 表项处于 ETHARP_STATE_STABLE 状态完

成数据的发送，函数源码具体见*代码清单 10_13*。

代码清单 10-13 etharp_query() 函数

```
err_t
etharp_query(struct netif *netif,
             const ip4_addr_t *ipaddr,
             struct pbuf *q)
{
    struct eth_addr *srcaddr = (struct eth_addr *)netif->hwaddr;
    err_t result = ERR_MEM;
    int is_new_entry = 0;
    s16_t i_err;
    netif_addr_idx_t i;

    /* 检是否为单播地址 */
    if (ip4_addr_isbroadcast(ipaddr, netif) ||
            ip4_addr_ismulticast(ipaddr) ||
            ip4_addr_isany(ipaddr))
    {
        return ERR_ARG;
    }

    /* 在 ARP 缓存中查找表项，如果没有则尝试创建表项 */
    i_err = etharp_find_entry(ipaddr, ETHARP_FLAG_TRY_HARD, netif);(1)

    /* 没有发现表项或者没有创建表项成功 */
    if (i_err < 0)
    {
        LWIP_DEBUGF(ETHARP_DEBUG | LWIP_DBG_TRACE,
                    ("etharp_query: could not create ARP entry\n"));
        if (q)
        {
            LWIP_DEBUGF(ETHARP_DEBUG | LWIP_DBG_TRACE,
```

(下页继续)

(续上页)

```
                    ("etharp_query: packet dropped\n"));
        ETHARP_STATS_INC(etharp.memerr);
    }
    return (err_t)i_err;  //返回错误代码
}
LWIP_ASSERT("type overflow", (size_t)i_err < NETIF_ADDR_IDX_MAX);

//找到对应的表项或者创建表项成功
i = (netif_addr_idx_t)i_err;

/* 将新表项标记为待处理 */
if (arp_table[i].state == ETHARP_STATE_EMPTY)
{
    is_new_entry = 1;
    arp_table[i].state = ETHARP_STATE_PENDING;
    /* 记录网络接口 */
    arp_table[i].netif = netif;                          (2)
}

/* 是否有新的表项 */
if (is_new_entry || (q == NULL))                                (3)
{
    /* 发送 ARP 请求包 */
    result = etharp_request(netif, ipaddr);
    if (result != ERR_OK)
    {
        /* 无法发送 ARP 请求 */
    }
    if (q == NULL)
    {
        return result;                                   (4)
    }
```

(下页继续)

(续上页)

```
    }

    LWIP_ASSERT("q != NULL", q != NULL);
    /* 表项状态是否稳定 */
    if (arp_table[i].state >= ETHARP_STATE_STABLE)
    {
        ETHARP_SET_ADDRHINT(netif, i);
        /* 发送数据包 */
        result = ethernet_output(netif, q,
                                 srcaddr,
                                 &(arp_table[i].ethaddr),
                                 ETHTYPE_IP);                          (5)
    }
    /* 如果表项是 ETHARP_STATE_PENDING 状态 */
    else if (arp_table[i].state == ETHARP_STATE_PENDING)
    {
        /* 将给数据包'q'排队 */
        struct pbuf *p;
        int copy_needed = 0;
        /* 如果 q 包含必须拷贝的 pbuf，请将整个链复制到一个新的 PBUF_RAM */
        p = q;
        while (p)
        {
            LWIP_ASSERT("no packet queues allowed!",
                        (p->len != p->tot_len) || (p->next == 0));
            if (PBUF_NEEDS_COPY(p))                                    (6)
            {
                //需要拷贝
                copy_needed = 1;
                break;
            }
        p = p->next;
```

(下页继续)

(续上页)

```
    }
    if (copy_needed)
    {
        /* 将整个数据包复制到新的 pbuf 中 */
        p = pbuf_clone(PBUF_LINK, PBUF_RAM, q);                    (7)
        }
        else
        {
            /* 引用旧的 pbuf 就足够了 */
            p = q;
            pbuf_ref(p);
        }

        if (p != NULL)
        {
            /* 如果使用队列 */
#if ARP_QUEUEING
            struct etharp_q_entry *new_entry;
            /* 分配一个新的 arp 队列表项 */                          (8)
        new_entry = (struct etharp_q_entry *)memp_malloc(MEMP_ARP_QUEUE);
            if (new_entry != NULL)
            {
                unsigned int qlen = 0;
                new_entry->next = 0;
                new_entry->p = p;
                if (arp_table[i].q != NULL)
                {
                    /* 队列已经存在，将新数据包插入队列后面 */
                    struct etharp_q_entry *r;
                    r = arp_table[i].q;
                    qlen++;
                    while (r->next != NULL)
```

(下页继续)

(续上页)

```
                    {
                        r = r->next;
                        qlen++;
                    }
                    r->next = new_entry;                    (9)
                }
                else
                {
                    /* 队列不存在，数据包就是队列的第一个节点 */
                    arp_table[i].q = new_entry;             (10)
                }
#if ARP_QUEUE_LEN
                if (qlen >= ARP_QUEUE_LEN)
                {
                    struct etharp_q_entry *old;
                    old = arp_table[i].q;
                    arp_table[i].q = arp_table[i].q->next;
                    pbuf_free(old->p);
                    memp_free(MEMP_ARP_QUEUE, old);
                }
#endif
                result = ERR_OK;
            }
            else
            {
                /* 申请内存失败 */
                pbuf_free(p);
                result = ERR_MEM;
            }
#else
            /* 如果只是挂载单个数据包，那么始终只为每个 ARP 请求排队一个数据包，
            就需要释放先前排队的数据包 */
```

(下页继续)

(续上页)

```
            if (arp_table[i].q != NULL)
            {
                pbuf_free(arp_table[i].q);                    (11)
            }
            arp_table[i].q = p;
            result = ERR_OK;

#endif
        }
        else
        {
            ETHARP_STATS_INC(etharp.memerr);
            result = ERR_MEM;
        }
    }
    return result;
}
```

代码清单 10-13**(1)(2)**：函数的处理逻辑是很清晰的，首先调用 etharp_find_entry() 函数在 ARP 缓存表中查找表项，如果没有找到就尝试创建表项并且返回表项的索引，当然 ARP 缓存表中可能存在表项，可能为新创建的表项（ETHARP_STATE_EMPTY），也可能为 ETHARP_STATE_PENDING 或者 ETHARP_STATE_STABLE 状态。如果是新创建的表项，那么表项肯定没有其他信息，LwIP 就会初始化一些信息，如网卡，然后就将表项设置为 ETHARP_STATE_PENDING 状态。

代码清单 10-13**(3)**：如果表项是刚创建的或者数据包是空的，那么就会调用 etharp_request() 函数发送一个 ARP 请求包。

代码清单 10-13**(4)**：如果数据包是空的，直接返回结果

代码清单 10-13**(5)**：如果表项的状态大于等于 ETHARP_STATE_STABLE，表示表项已经是稳定状态了，就调用 ethernet_output() 函数发送数据包。

代码清单 10-13**(6)**：通过宏定义 PBUF_NEEDS_COPY(p) 对数据包的类型进行判断，如果需要拷贝则将变量 copy_needed 设置为 1，表示需要拷贝。

代码清单 10-13(**7**)：将整个数据包复制到新的 pbuf 中。

代码清单 10-13(**8**)：如果 ARP_QUEUEING 宏定义为 1，则表示使用队列，那么 LwIP 会分配一个新的 ARP 数据包队列节点，然后插入队列中。

代码清单 10-13(**9**)：如果队列已经存在，将新数据包插入队列后面。

代码清单 10-13(**10**)：如果队列不存在，数据包就是队列的第一个节点。

代码清单 10-13(**11**)：如果只是挂载单个数据包，就需要释放先前排队的数据包，然后再挂载新的数据包。

挂载的这些数据在等待到目标主机产生 ARP 应答的时候会发送出去，此时的发送就是延时了，所以在没有 ARP 表项的时候，发送数据会产生延时，在指定等待 ARP 应答时间内如果等不到目标主机的应答，那么这个表项将被系统回收，同时数据也无法发送出去。

上层数据包通过 ARP 协议进行发送数据的流程示意图具体见图 *10_10*。

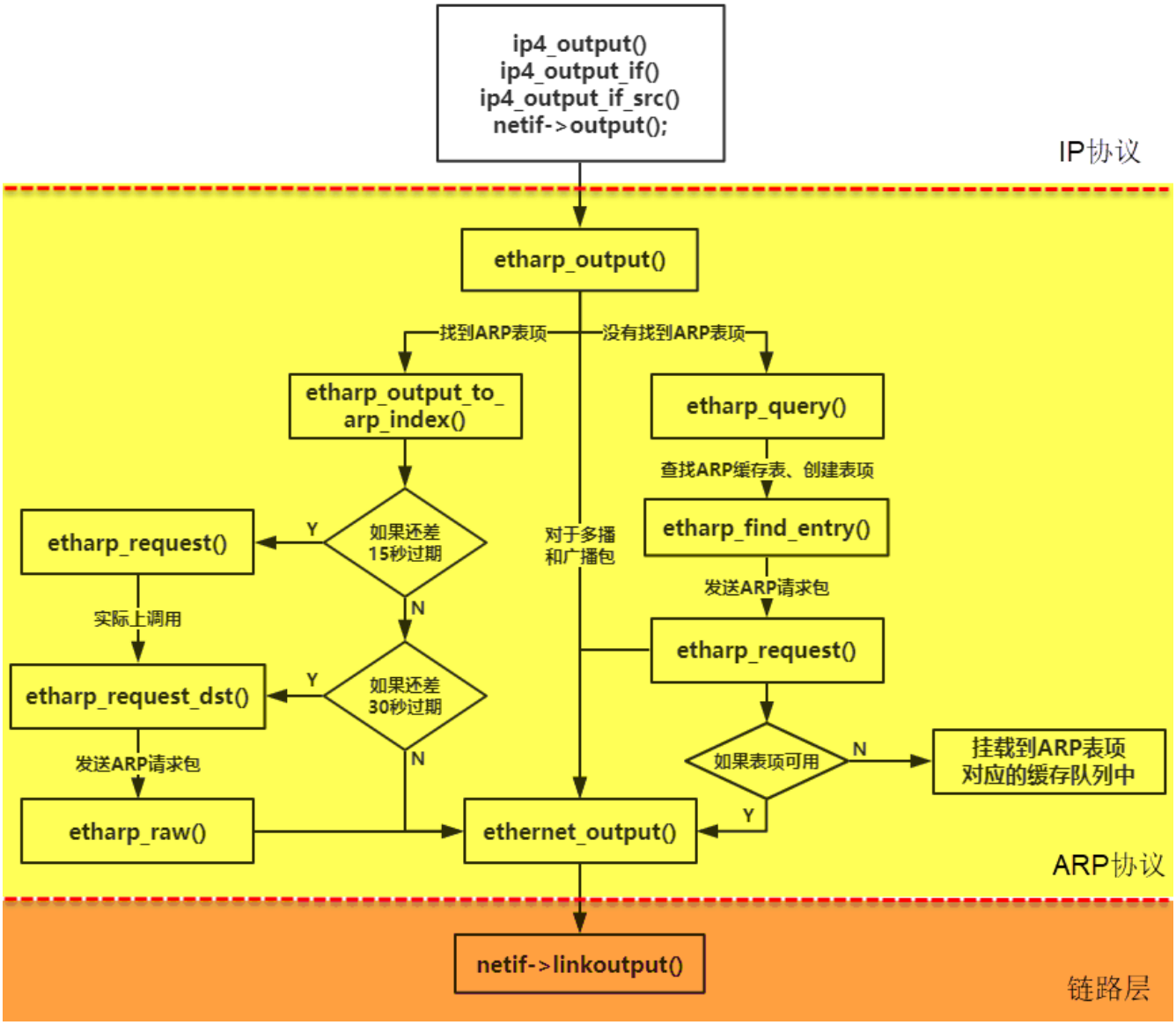


图 10-10 数据包通过 ARP 协议发送流程图

整个 ARP 协议运作示意图具体见图 *10_11*。

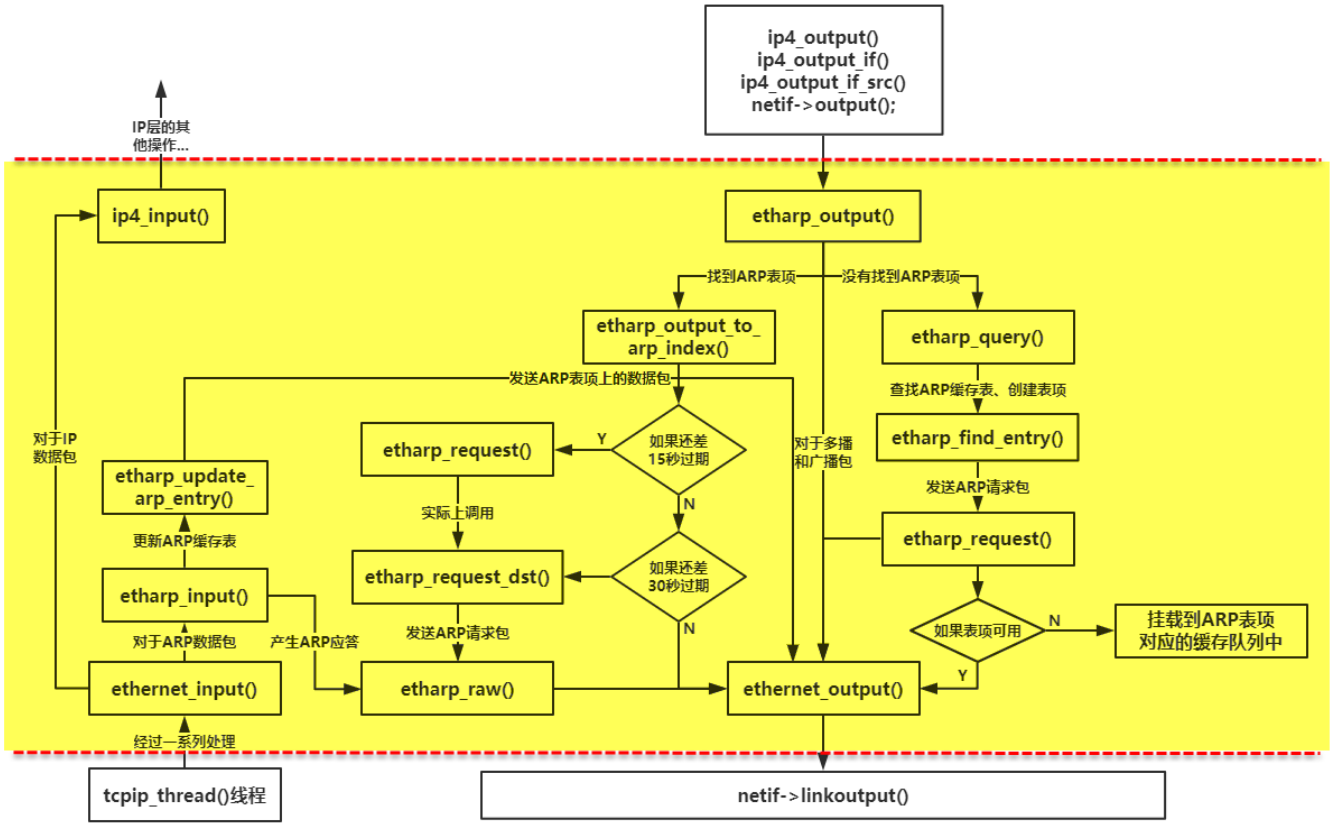


图 10-11 整个 ARP 协议运作示意图

第 11 章　IP 协议

IP 协议（Internet Protocol），又称之为网际协议，IP 协议处于 IP 层工作，它是整个 TCP/IP 协议栈的核心协议，上层协议都要依赖 IP 协议提供的服务，IP 协议负责将数据报从源主机发送到目标主机，通过 IP 地址作为唯一识别码，简单来说，不同主机之间的 IP 地址是不一样的，在发送数据报的过程中，IP 协议还可能对数据报进行分片处理，同时在接收数据报的时候还可能需要对分片的数据报进行重装等等。

IP 协议是一种无连接的不可靠数据报交付协议，协议本身不提供任何的错误检查与恢复机制。

11.1 IP 地址

11.1.1 概述

在 TCP/IP 设计之初，为了标识互联网中的每台主机的身份，设计人员为每个接入网络中的主机都分配一个 IP 地址（Internet Protocol Address），是一个 32 位的整数地址，只有合法的 IP 地址才能接入互联网中并且与其他主机进行通信，IP 地址是软件地址，不是硬件地址，硬件 MAC 地址是存储在网卡中的，应用于本地网络中寻找目标主机。而 IP 地址能让一个网络中的主机能够与另一个网络中的主机进行通信，无需理会这些主机之间的 MAC 地址。

在学习 IP 地址之前，我们需要简述一下主机与路由器连入网络的通信的方法：一台主机通常只有一条链路连接到网络，一般只有一个网卡；当主机中想发送一个数据报时，它就在该链路上发送，一个网卡对应一个 MAC 地址与 IP 地址，当然，主机也可能有多个网卡，这样子主机就有多个 MAC 地址与 IP 地址。

每个 IP 地址长度为 32 比特（4 字节），因此总共有 2^{32} 个可能的 IP 地址，大约有 40 亿个 IP 地址能被使用的。这些地址一般按所谓点分十进制记法（dotted-decimal notation）书写，即地址中的每个字节用它的十进制形式书写，各字节间以点分隔开。例如，IP 地址为 192.168.0.122，192 是该地址第一个 8 比特的十进制等价数，168 是该地址第二个 8 比特的十进制等价数，依次类推。

11.1.2 IP 地址编址

在全球的互联网中，每个主机都要唯一的一个 IP 地址作为身份识别，那么这么多 IP 地址是怎么样分配的呢？这不可能是随意自由选择 IP 地址的，实际上每个主机的 IP 地址的一部分都由其所在的子网决定的，所以又出现了 IP 地址分类编址的概念，网络的类型决定了 IP 地址将如何划分成网络部分和节点部分，在分类编址中，设计者把所有的 IP 地址划分为 5 大类，分别为 A、B、C、D、E 五类，每一类地址都觉定了其中 IP 地址的一部分组成，具体见图 *11_1*。

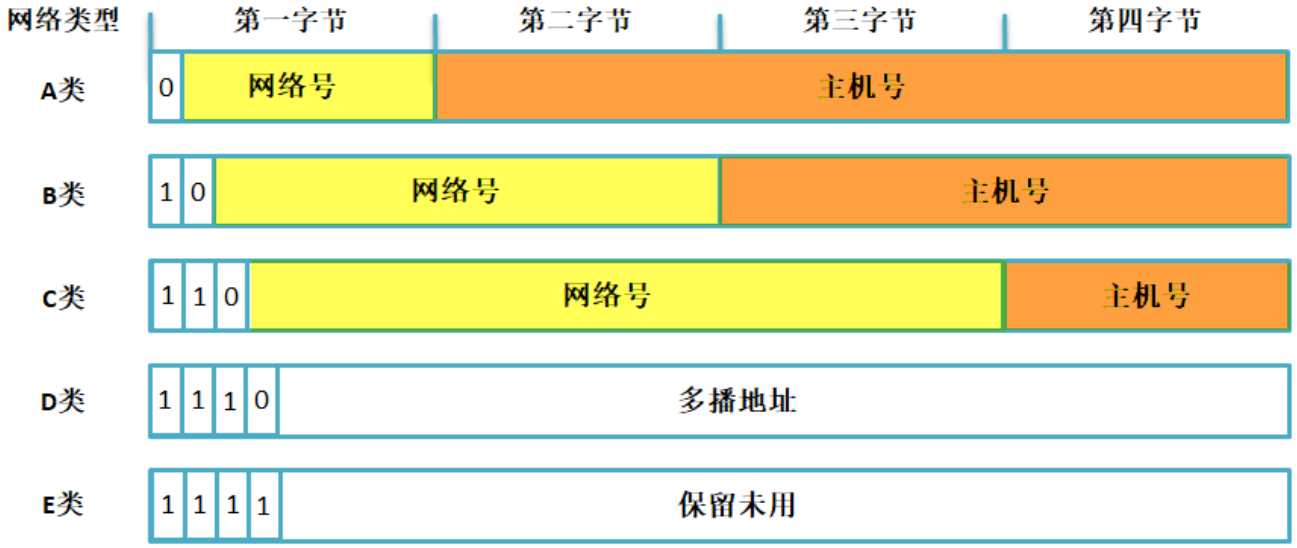


图 11-1 分类编址

A 类网络地址的第一个字节的第一位必须为 0 ，因此，A 类网络地址第一个字节的取值范围为 0~127（注意：但 0 和 127 不是有效的 A 类网络地址号），A 类地址总共有 128 个网络号，其中 3 个网络号用作特殊用途，因此可以在互联网上使用的还有 125 个，而每一个网络号后面的 3 字节主机号表示能用于 A 类网络地址不同网络号的的主机数量（多达 2^{24}-2 = 16777214 个），所以总的来说 A 类网络支持的主机数量为 125*16777214=2097151750 个，大约占据了 40 亿 IP 地址的半壁江山，这些 IP 地址通常被分配给世界上超大型的机构使用，但是通常没有任何一个机构能使用那么大量的 IP 地址，所以 A 类地址的很多 IP 地址都会被白白浪费掉。

B 类网络地址的第一个字节的第一位必须为 1 ，且第二位必须为 0，因此，B 类网络地址第一个字节的取值范围为 128~191，前 2 字节剩余的 14bit 表示网络号，多达 16384 个网络号，其中 16 个网络号被保留使用，因此可以分配给企业使用的网络号有 16368 个，每一个网络号中可以拥有

2^{16}-2 =65534 个主机，总的来说 B 类网络支持的主机个数为 16368*65534=1072660512，大约占据了所有 IP 地址的四分之一。

C 类网络地址的第一个字节的第一位必须为 1，第二位必须为 1，第三位必须为 0，因此 C 类网络第一个字节的取值范围为 192~223，C 类地址的前三个字节用于表示网络号（剩下的 21bit），所以拥有 2097152 个网络号，但是其中的 256 个网络号被保留出来，因此 C 类地址的可用网络号为 2096896 个，每个网络号下的主机个数为 254，因此 C 类网络支持的主机数量为 532611584，大约占据了所有 IP 地址的八分之一。

D 类 IP 地址的第一个字节前四位为 1110，那么它第一个字节的取值范围为 224~239，约占据了所有 IP 地址的十六分之一，R 它是一个专门保留的地址，它并不指向特定的网络，目前这一类地址被用于多播。

E 类 IP 地址的第一个字节前四位必须为 1111，那么它第一个字节的取值范围为 240~255，约占据了所有 IP 地址的十六分之一，是为将来使用而保留的，其中 32bit 全为 1 的 IP 地址（255.255.255.255）用作广播地址。

各类 IP 地址的特点具体见

表格 11-1 各类 IP 地址的特点

类别	第一字节 (二进制)	第一字节取值范围	网络号个数	主机号 个数	适用范围
A 类	0XXX XXXX	0~127	125	16777214	大型网络
B 类	10XX XXXX	128~191	16368	65534	中型网络
C 类	110X XXXX	192~223	2097152	254	小型网络
D 类	1110 XXXX	224~239	—	—	多播
E 类	1111 XXXX	240~255	—	—	保留

11.1.3 特殊 IP 地址

除了上述的网络地址外，还有一些特殊用途的地址，这些地址是不允许分配给任何一个网络的主机使用的，下面简单介绍几个常见的特殊地址。

11.1.3.1 受限广播地址

广播通信是一对所有的通信方式，受限广播地址用于定义整个互联网，如果设备想使 IP 数据报被整个网络所接收，就发送这个目的地址全为 1 的广播包，但这样会给整个互联网带来灾难性的负担，所以在任何情况下，路由器都会禁止转发目的地址为 255.255.255.255 的广播数据包，因此这样的数据包仅会出现在本地网络中（局域网），255.255.255.255 这个地址指本网段内的所有主机，相当于“房子里面的人都听着”通知所有主机。

注意：此处不要与以太网的广播地址（255-255-255-255-255-255）混淆了。

11.1.3.2 直接广播地址

受限广播地址是网络号与主机号都为 1 的地址，但是直接广播地址是主机号全为 1 而得到的地址，广播地址代表本网络内的所有主机，使用该地址可以向网络内的所有主机发送数据，比如一个 IP 地址是 192.168.0.181，这是 C 类地址，所以它的主机号只有一个字节，那么对主机号全取 1 得到一个广播地址 192.168.0.255，向这个地址发送数据就能让同一网络下的所有主机接收到。

A、B、C 三类地址的广播地址结构如下：

- A 类地址的广播地址为：XXX.255.255.255（XXX 为 A 类地址的第一个字节取值范围）。
- A 类地址的广播地址为：XXX. XXX.255.255（XXX 为 B 类地址的前两个字节取值范围）。
- A 类地址的广播地址为：XXX. XXX. XXX.255（XXX 为 C 类地址的前三个字节取值范围）。

注意：这个地址在 IP 数据报中只能作为目的地址。另外，直接广播地址使一个网段中可分配给设备的地址数减少了 1 个。

11.1.3.3 多播地址

多播地址用在一对多的通信中，即一个发送者，多个接收者，不论接受者员数量的多少，发送者只发送一次数据包。多播地址属于分类编址中的 D 类地址，D 类地址只能用作目的地址，而不能作为主机中的源地址。

11.1.3.4 环回地址

127 网段的所有地址都称为环回地址，主要用来测试网络协议是否工作正常的作用。比如在电脑中使用 ping 命令去 ping 127.1.1.1 就可以测试本地 TCP/IP 协议是否正常。用通俗的话表示，就是“我自己”，不能以 127 网段中的 IP 地址作为主机地址，因此 A 类地址又少了一个可用网络号。

11.1.3.5 本网络本主机

IP 地址 32bit 全为 0 的地址（0.0.0.0）表示的是本网络本主机，这个 IP 地址在 IP 数据报中只能用作源 IP 地址，这发生在当设备启动时但又不知道自己的 IP 地址情况下。在使用 DHCP 分配 IP 地址的网络环境中，这样的地址是很常见的，主机为了获得一个可用的 IP 地址，就给 DHCP 服务器发送 IP 数据报，并用这样的地址（0.0.0.0）作为源地址，目的地址为 255.255.255.255（因为主机这时还不知道 DHCP 服务器的 IP 地址），然后 DHCP 服务器就会知道这个主机暂时没有 IP 地址，那么就会分配一个 IP 给这个主机。

11.2 局域网和广域网的概念

11.2.1 什么是局域网

局域网（Local Area Network，缩写为 LAN），又称内网，指覆盖局部区域（如办公室或楼层）的计算机网络，局域网可以实现文件管理、应用软件共享、打印机共享、工作组内的日程安排、电子邮件和传真通信服务等功能，是在一定区域内由多个计算机连接组成的网络，比如腾讯、阿里的内网，在内部的计算机上的数据可以互联互通、数据共享等。简单来说，当我们使用的开发板接入路由器的时候，电脑的网络也接入路由器，那么开发板与电脑就组成局域网，数据在链路层

上是互联互通的。当然，板子也能通过网线直连电脑，这样子也是组成一个局域网络，数据可以在两个主机之间进行通信。

11.2.2 什么是广域网

广域网（Wide Area Network，缩写为 WAN），又称广域网、外网、公网。是连接不同地区计算机以进行通信的网络，这种网络通常会跨越很大的范围，覆盖的范围从几十公里到几千公里，它能连接多个地区、城市并能提供远距离通信服务，但是注意的是我们说的广域网并不等同于互联网。

简单来说（我使用比喻的方式表达），如果局域网是小溪，不同区域有很多的小溪，这些小溪汇聚成江河，那么这些江河就是广域网，然后这些江河再流入大海，这个大海就是互联网。互联网就是由无数个局域网，通过广域网线路汇聚互联起来，就形成了互联网。互联网的特点是开放、互联，如果一个公司的局域网没有连接到互联网，那这个局域网就不属于互联网，仅仅属于内部通信的网络。

举个例子，如果你家在某网络运营商办理了网络服务，可能他们将送你一个光猫，然后你拿着这个光猫就能上网了，你可能觉得需要多人使用网络，所以你又去买了一个无线路由器，让家人也能连接到网络上，那么这个家就是一个局域网，然后局域网的通信是通过运营商连接到广域网上的，示意图具体见图 *11_2*。

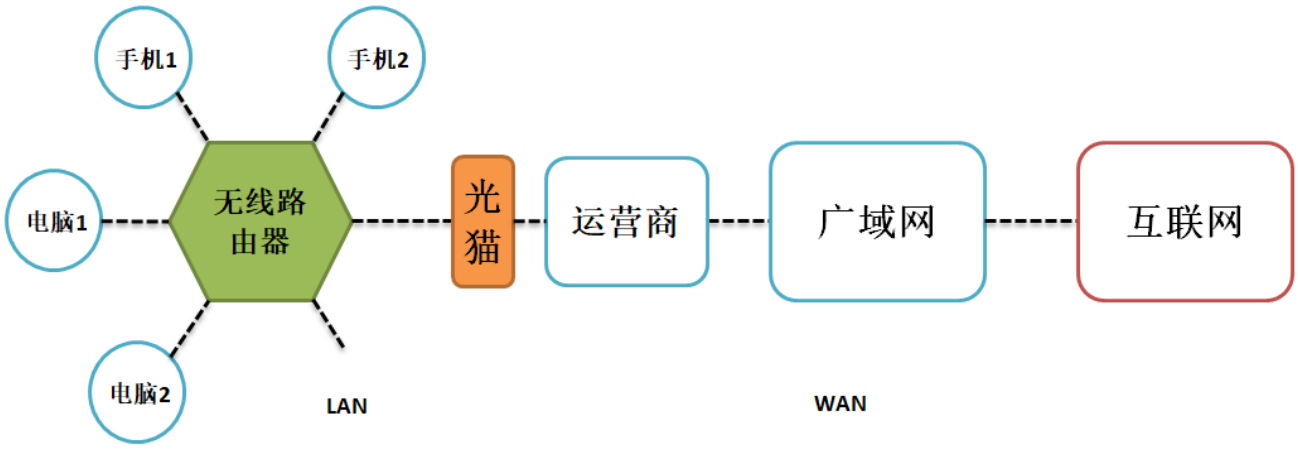


图 11-2 局域网与广域网示意图

这个示意图虽然简单，却把 LAN、WAN、Internet 三者全包含了。无线路由器把电脑、手机等设

备连接到局域网 LAN 上，并分配 IP 地址，即局域网 IP，我们可以称之为 LAN-IP，LAN-IP 所到之处，就是局域网的范围，像我们电脑的 IP 地址（192.168.0.xxx）就是一个局域网 IP，而路由器的地址就是运营商给我们的一个 IP 地址，这个 IP 地址是有效的，可以看做是 WAN-IP（实际上这个 IP 地址也是运营商的局域网 IP 地址（相对运营商来说），这个地址在运营商中转换成一个真正的广域网 IP 地址，但是这些事情我们暂时无需理会，只要把路由器的 IP 地址看做是 WAN-IP 即可）。

而运营商是一个边界，国家与国家之间有边界，网络之间也有边界，运营商就是局域网 LAN 与广域网 WAN 的边界。局域网 LAN-IP 可以在局域网内部有效，但是无法跨越边界进入广域网中，LAN-IP 是路由器分配给我们的 IP，那么我们想要跨越边界进入广域网中，就需要将 LAN-IP 变成有效的的 IP 地址，也就是 WAN-IP，那么在路由器中就需要将 IP 地址进行转换，完成 LAN-IP<—>WAN-IP 地址转换（NAT），关于地址转换的内容我们稍后讲解统一换成特别通行证才可以在广域网里继续遨游。

当持有 WAN-IP 的 IP 包顺利到达下一个边界 Internet Gateway，这是通往互联网 Internet 的最后一道关卡，即边界。左边是广域网，右边是互联网，也需要做 WAN-IP 与 Global-IP（互联网公共 IP）的转换才能进入互联网中，我们知道这种概念即可，无需过多深入。

11.3 网络地址转换（NAT）

NAT 英文全称是“Network Address Translation”，中文意思是“网络地址转换”，它是一个 IETF(Internet Engineering Task Force，Internet 工程任务组) 标准，允许一个整体机构以一个公用 IP（Internet Protocol）地址出现在 Internet 上。顾名思义，它是一种把内部私有网络地址（IP 地址）翻译成合法网络 IP 地址的技术。因此 NAT 在一定程度上，能够有效的解决 IP 地址不足的问题，它是一个方便且得到了广泛应用的技术，当然，NAT 也让主机之间的通信变得复杂，导致了通信效率的降低。

具有 NAT 功能的路由器必须拥有一个内部地址与一个外部地址，内部地址是为了与局域网的用户通信而使用的，它使用一个特定的内部 IP 地址，如 192.168.0.1（也是局域网的网关），外部地址是与广域网进行通信而使用的，这是一个有效的 IP 地址，通常为运营商分配给我们，假设运营商分配给我们的 IP 地址是一个 C 类网络地址 223.166.166.66，假设我们电脑上的 IP 地址是 192.168.0.181，端口号是 5555。

那么在局域网的电脑想要与外网进行通信的时候，比如我们想访问百度（假设百度的 IP 地址是 123.125.115.110，端口号是 80），那么电脑就会把这些数据报发送到路由器中，请求路由器帮忙转发，假设这些数据报格式简化为：

（源 IP 地址，端口号，目标 IP 地址，端口号）

（192.168.0.181，5555，123.125.115.110，80）

具有 NAT 功能的路由器会在内部维护一个 NAT 转换表，当路由器收到局域网的 IP 数据报时，就会为这个数据报分配一个路由器内部的 NAT 端口，假设为 6666，并且路由器会将原始 IP 数据报中源 IP 地址与端口号（192.168.0.181，5555）转换成一个有效 IP 地址与端口号（223.166.166.66，6666），然后转换后的路由器发出的数据就是：

（223.166.166.66，6666，123.125.115.110，80）

当百度响应这个数据报的时候，就会返回一个响应信息，它返回的目标 IP 地址就是我们路由器的有效 IP 地址和端口号（223.166.166.66，6666），那么路由器收到的数据报就是：

（123.125.115.110，80，223.166.166.66，6666）

当路由器收到这个数据报之后，会在 NAT 转换表中查找端口号为 6666 的连接，并且把数据报中的目标 IP 地址与端口号（223.166.166.66，6666）转换成局域网内我们电脑的 IP 地址与端口号（192.168.0.181，5555），并且将数据报转发到我们的主机上，那么我们电脑上收到的数据报就是：

（123.125.115.110，80，192.168.0.181，5555）

这样子的数据报对于电脑来说，是很正常的通信方式，但是它并不知道路由器已经转换过这些数据报的 IP 地址与端口内容，所以 NAT 对于所有用户来说是透明的，通过这样子的两次转换，局域网就实现了与广域网的通信，在 NAT 转换中，NAT 端口号是一个关键的因素，路由器应及时为每个连接分配唯一的端口号，并且要及时回收那些不使用的端口号。

11.4 IP 数据报

虽然前面我们也接触过 IP 数据报（也可以称之为 IP 数据报或者 IP 分组，为了统一，下文均采用 IP 数据报），IP 数据报与 ARP 报文都是一种报文格式，都有自己的组织形式，与 ARP 报文一样，由两部分组成，一部分是 IP 首部，另一部分是数据区域，一个 IP 数据报的首部长度是不定

的，通常为 20~60 字节，根据选项决定。而数据区域理论上可以多达 65535 个字节，但是很少有数据是那么大的，并且受限于网卡硬件。

IP 数据报的格式如所示图 *11_3*。

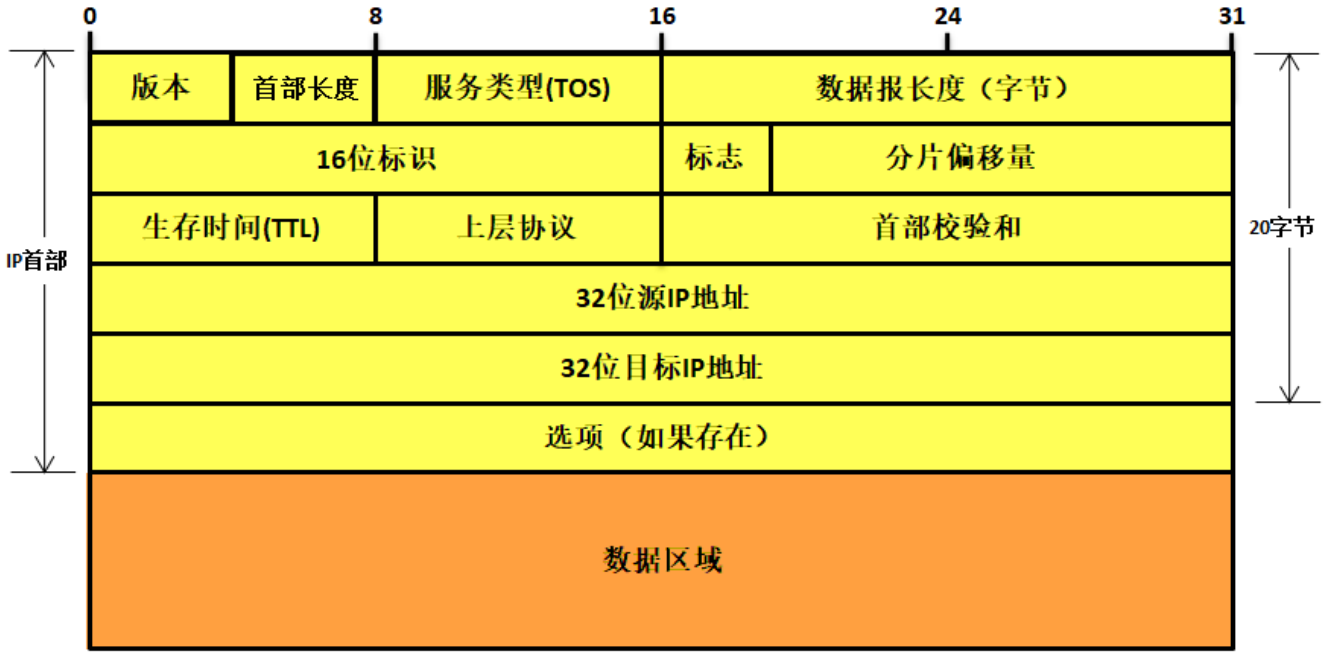


图 11-3IP 数据报格式（IPv4）

版本（号），占据 4bit 空间。这个字段规定了数据报的 IP 协议版本，对于 IPv4，该值为 4；对于 IPv6，该值为 6。通过查看版本号，路由器能够确定如何解释 IP 数据报的其他部分，不同的 IP 版本使用不同的数据报格式。目前版本的 IP（即 IPv4）的数据报格式如图 11-3 所示，而新版本的 IP（IPv6）的数据报格式又有所不同，此处我们就不做过多讨论 IPv6 版本的 IP 数据报格式，若无特殊说明，本书所写的 IP 数据报内容均为 IPv4。

部首长度字段占据 4bit 空间，用于记录 IP 首部的数据的长度，为什么需要记录首部长度呢？因为 IP 首部中包含了一些可变的数据选项，故需要这 4bit 记录首部的长度，以便区分数据部分的起始位置，当然啦，4bit 的部首长度单位是字，只有这样子才能最大记录 60 个字节的数据（15*4=60）。

服务类型 (TOS) 占据 8bit 空间，服务类型（TOS）包含在 IPv4 首部中，以便使不同类型的 IP 数据报（例如，一些特别要求低时延、高吞吐量或可靠性的数据报）能相互区别开来。提供特定等级的服务是一个由路由器管理员决定的策略问题，简单来说就路由器根据是这个字段的值来为

数据报提供（选择）最合理的路径。

数据报长度字段占据 16bit 空间。这是 IP 数据报的总长度（首部加上数据区域），以字节为单位。因为该字段长为 16bit，所以整个 IP 数据报的理论最大长度为 65535 字节，然而，数据报很少有超过 1500 字节的，这是因为底层链路硬件不允许那么大的数据报出现在链路上，以太网数据帧的最大长度为 1500 个字节，当有一个很大的 IP 数据报出现的时候，就需要进行分片处理；而如果 IP 数据报的数据很少的时候，比如少于 46 个字节，那么在以太网进行发送数据的时候会填充一定的字节以满足以太网帧最小长度，那么在接收方就要根据这个字段的内容进行提取有效数据。

标识、标志、分片偏移量这三个字段与 IP 数据报分片有关，这是一个我们将很快要深入学习的一个问题，在后文详细讲解。但更有趣的是，新版本的 IP（即 IPv6）数据报则不允许对数据进行分片处理。

标识字段用于表示 IP 层发送出去的每一份 IP 数据报，在发送每一份报文，该值加 1，在分片的时候，该字段会被复制到每个分片数据报中，在目标接收主机中，使用该字段判断这些数据是否属于同一个 IP 数据报。

标志位（3bit）的定义如下：第一位保留未用；第二位是不分片标志位，如果该位为 1，则表示 IP 数据报在发送的过程中不允许进行分片，如果这个 IP 数据报的大小超过链路层能承载的大小，这个 IP 数据报将被丢弃，如果该位为 0 则表示 IP 层在必要的时候可以对其进行分片处理；第三位为更多分片位，如果为 1 则表示该分片数据报不是整个 IP 数据报的最后一个分片，如果为 0 则表示是整个 IP 数据报的最后一个分片。

分片偏移量占据 13bit 空间，表示当前分片所携带的数据在整个 IP 数据报中的相对偏移位置（以 8 字节为单位），目标主机必须受到以 0 偏移量开始到最高偏移量的所有分片，才能将分片进行重装为一个完整的 IP 数据报，并且重装 IP 数据报的依据就是分片的偏移量。

生存时间（Time-To-Live，TTL），该字段用来确保数据报不会永远在网络中循环（例如由于长时间的路由选择环路）。每当 IP 数据报由一台路由器处理时，该字段的值减 1，若 TTL 字段减为 0，则该数据报必须丢弃，同时会返回一个 ICMP 差错报文给源主机，这样子数据就不会永远在网络中漂流而占据资源。

上层协议字段占据 8bit 空间。该字段仅在一个 IP 数据报到达其最终目的地才会有用。该字段的值指示了 IP 数据报的数据部分应交给哪个特定的传输层协议。例如，值为 6 表明数据部分要交给 TCP，而值为 17 表明数据要交给 UDP。在 IP 数据报中的协议号所起的作用，类似于运输层报

文段中端口号字段所起的作用。协议字段是将网络层与运输层绑定到一起的粘合剂，而端口号是将运输层和应用层绑定到一起的粘合剂，此处了解这个概念即可。

首部检验和字段占据 16bit 空间。首部检验和用于帮助路由器检测收到的 IP 数据报首部是否发生错误，而对应 IP 数据报中的数据区域校验那是上层协议处理的事情。首部检验和是这样计算的：将首部中的每 2 个字节当作一个数，用反码运算对这些数求和，该和的反码（被称为因特网检验和）存放在检验和字段中。路由器要对每个收到的 IP 数据报计算其首部检验和，如果数据报首部中携带的检验和与计算得到的检验和不一致，则表示出现错误，路由器一般会丢弃检测出错误的 IP 数据报。注意了：IP 数据报在到达每个路由器上都必须重新计算检验和并再次存放到原处，因为 TTL 字段以及可能的选项字段会改变。

可能很多人都会有疑问：为什么 TCP/IP 协议在传输层与网络层都执行差错检测？首先，在 IP 层只对 IP 首部计算了检验和，而传输层检验和是对整个 TCP/UDP 报文段进行的。

这样子在每个路由器对 IP 数据报首部检查校验能提高效率，直到 IP 数据报到达目标 IP 地址。

源 IP 地址与目标 IP 地址就不用过多解释了，源主机在生存 ip 数据包的时候会在源 IP 地址字段中插入它的 IP 地址，在目标 IP 地址字段中插入其想要发送的最终目标 IP 地址。

选项字段占据 0~40 个字节。它允许 IP 首部被扩展，首部选项在日常生活中还是比较少使用的，因此在每个 IP 数据报首部中必须存在的字段是不包括选项字段的，这样能够节约开销，如果与选项字段就添加，而如果没有就无需理会。在 LwIP 中只识别选项字段，但是不会处理选项字段的内容。因为选项的存在的确是件复杂的事，IP 数据报首部长度可变，故不能预先确定数据字段从何处开始，而且还因为有些数据报要求处理选项，而有些数据报则不要求，故导致一台路由器处理一个 IP 数据报所需的时间变化很大，这些考虑对于高性能路由器和主机上的 IP 协议处理来说特别重要，由于这样或那样的原因，在 IPv6 数据报首部中已去掉了 IP 数据报中的选项字段（此处我们不讲解选项字段的知识）。

数据区域（也可以称之为有效载荷），这是 IP 数据报的最后的一个字段，也是最重要的内容，因为有数据区域才会有数据报首部的存在，在大多数情况下，IP 数据报中的数据字段包含要交付给目标 IP 地址的运输层（TCP 协议或 UDP 协议），当然，数据区域也可承载其他类型的报文，如 ICMP 报文。

至此，整个 IP 数据报的字段内容已经全部讲解完毕，我们可以验证一下我们的数据报格式是否正确，打开 wireshark 网络抓包工具，然后随便抓数据，在这里我们抓取了一个 TCP 协议，然后查看它的 IP 层数据结构，具体见图 *11_4*。

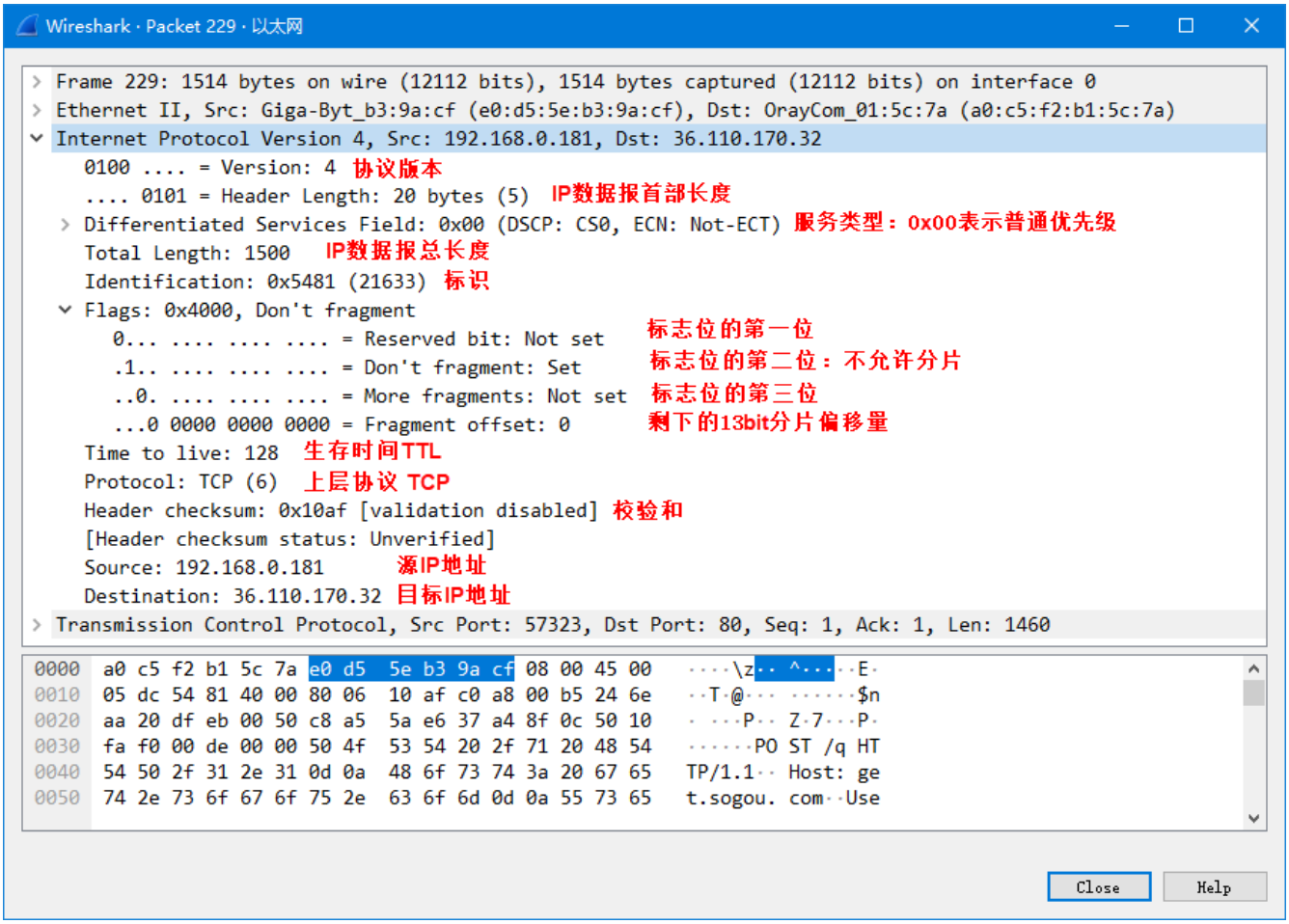


图 11-4IP 数据报格式

IP 数据报封装在以太网帧的格式具体见

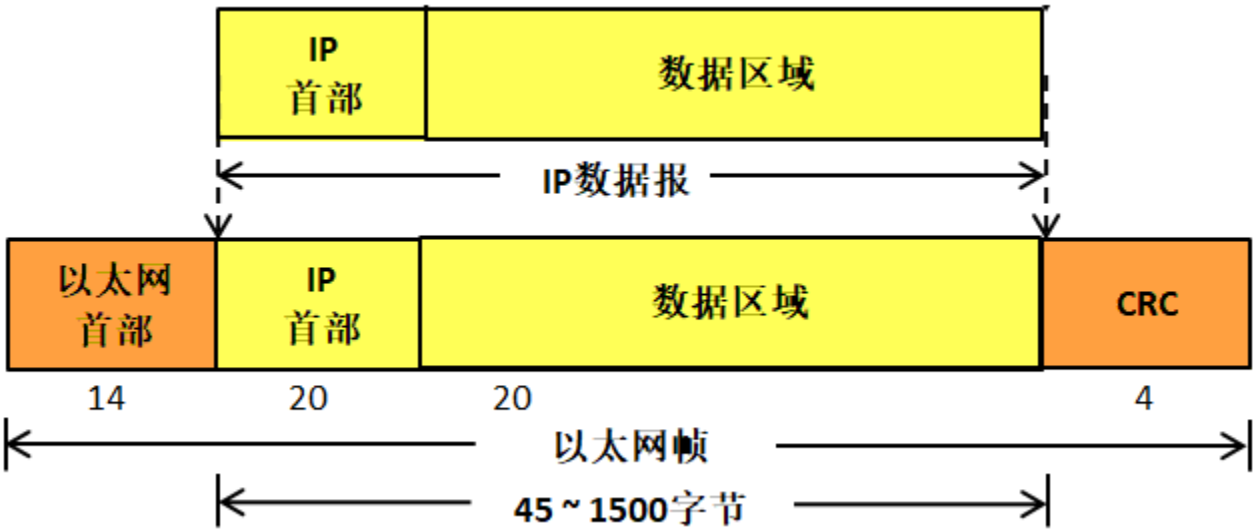


图 11-5IP 数据报在以太网帧上的封装

11.5 IP 数据报的数据结构

为了描述 IP 数据报首部的信息，LwIP 定义了一个 ip_hdr 的结构体作为描述 IP 数据报首部，同时还定义了很多获取 IP 数据报首部的宏定义与设置 IP 数据报首部的宏定义，具体见*代码清单 11_1*。

代码清单 11-1 LwIP 中对 IP 数据报首部描述的数据结构与宏定义

```
PACK_STRUCT_BEGIN
struct ip_hdr
{
    /* 版本 / 首部长度 */
    PACK_STRUCT_FLD_8(u8_t _v_hl);
    /* 服务类型 */
    PACK_STRUCT_FLD_8(u8_t _tos);
    /* 数据报总长度 */
```

(下页继续)

(续上页)

```
    PACK_STRUCT_FIELD(u16_t _len);
    /* 标识字段 */
    PACK_STRUCT_FIELD(u16_t _id);
    /* 标志与偏移 */
    PACK_STRUCT_FIELD(u16_t _offset);
#define IP_RF 0x8000U        /* 保留的标志位 */
#define IP_DF 0x4000U        /* 不分片标志位 */
#define IP_MF 0x2000U        /* 更多分片标志 */
#define IP_OFFMASK 0x1fffU   /* 用于分段的掩码 */
    /* 生存时间 */
    PACK_STRUCT_FLD_8(u8_t _ttl);
    /* 上层协议 */
    PACK_STRUCT_FLD_8(u8_t _proto);
    /* 校验和 */
    PACK_STRUCT_FIELD(u16_t _chksum);
    /* 源 IP 地址与目标 IP 地址 */
    PACK_STRUCT_FLD_S(ip4_addr_p_t src);
    PACK_STRUCT_FLD_S(ip4_addr_p_t dest);
} PACK_STRUCT_STRUCT;
PACK_STRUCT_END

/* 获取 IP 数据报首部各个字段信息的宏 */

//获取协议版本
#define IPH_V(hdr)  ((hdr)->_v_hl >> 4)
//获取首部长度（字）
#define IPH_HL(hdr) ((hdr)->_v_hl & 0x0f)
//获取获取首部长度字节
#define IPH_HL_BYTES(hdr) ((u8_t)(IPH_HL(hdr) * 4))
//获取服务类型
#define IPH_TOS(hdr) ((hdr)->_tos)
//获取数据报长度
```

(下页继续)

(续上页)

```
#define IPH_LEN(hdr) ((hdr)->_len)
//获取数据报标识
#define IPH_ID(hdr) ((hdr)->_id)
//获取分片标志位 + 偏移量
#define IPH_OFFSET(hdr) ((hdr)->_offset)
//获取偏移量大小 (字节)
#define IPH_OFFSET_BYTES(hdr) \
((u16_t)((lwip_ntohs(IPH_OFFSET(hdr)) & IP_OFFMASK) * 8U))
//获取生存时间
#define IPH_TTL(hdr) ((hdr)->_ttl)
//获取上层协议
#define IPH_PROTO(hdr) ((hdr)->_proto)
//获取校验和
#define IPH_CHKSUM(hdr) ((hdr)->_chksum)

/* 用于填写 IP 数据报首部的宏 */

//设置版本号跟首部长度
#define IPH_VHL_SET(hdr, v, hl) \
(hdr)->_v_hl = (u8_t)((((v) << 4) | (hl)))
//设置服务类型
#define IPH_TOS_SET(hdr, tos) (hdr)->_tos = (tos)
//设置数据报总长度
#define IPH_LEN_SET(hdr, len) (hdr)->_len = (len)
//设置标识
#define IPH_ID_SET(hdr, id) (hdr)->_id = (id)
//设置分片标志与偏移量
#define IPH_OFFSET_SET(hdr, off) (hdr)->_offset = (off)
//设置生存时间
#define IPH_TTL_SET(hdr, ttl) (hdr)->_ttl = (u8_t)(ttl)
//设置上层协议
#define IPH_PROTO_SET(hdr, proto) (hdr)->_proto = (u8_t)(proto)
```

(下页继续)

(续上页)

```
//设置校验和
#define IPH_CHKSUM_SET(hdr, chksum) (hdr)->_chksum = (chksum)
```

当然啦，在定义结构体的时候要禁止编译器进行对齐操作，因为结构体中的很多字段都是按位进行操作的。

11.6 IP 数据报分片

从 IP 首部我们就知道 IP 数据报分片这个概念，也知道不是每个底层网卡都能承载每个 IP 数据报长度的报文，例如以太网帧最大能承载 1500 个字节的数据，而某些广域网链路的帧可承载不超过 576 字节的数据。一个链路层帧能承载的最大数据量叫做最大传送单元（Maximum Transmission Unit，MTU）。因为每个 IP 数据报都必须封装在链路层帧中从一台路由器传输到下一台路由器，故链路层协议的 MTU 严格地限制着 IP 数据报的长度。对 IP 数据报长度具有严格限制并不是主要问题，问题在于在发送方与目的地路径上的每段链路可能使用不同的链路层协议，且不同的硬件可能具有不同的 MTU，这就需要有一个很好的处理方式，随之而来的就是 IP 数据报分片处理。

分片处理是将 IP 数据报中的数据分片成两个或更多个较小的 IP 数据报，用单独的链路层帧封装这些较小的 IP 数据报；然后向输出链路上发送这些帧，每个这些较小的数据报都称为分片，由于 IP 数据报的分片偏移量是用 8 的整数倍记录的，所以每个数据报中的分片数据大小也必须是 8 的整数倍。

所有分片数据报在其到达目标主机的传输层之前需要在 IP 层完成重新组装（也称之为重装）。IPv4 协议的设计者觉得如果在每个 IP 层中组装分片数据包，那么将严重影响路由器的性能，例如一台路由器，在收到数据分片后又进行重装，然后再转发，这样子的处理是万万不可的，所以 IPv4 的设计者决定将数据报的重新组装工作放到端系统中，而不是放到网络路由器中，什么是端系统呢？简单来说就是数据包中的目标 IP 地址的主机，在这台机器上的 IP 层进行数据分片的重装，这样子数据分片可以任意在各个路由之间进行转发，而路由器就无需理会数据分片是在哪里重装，只要数据分片不是给路由器的，那么就将其转发出去即可，当然，这样子的处理就会是的每个数据分片到达目标 IP 地址的主机时间是不一样的。

那么怎么样处理每个分片的数据呢？其实在发送主机中，它会把需要分片的数据进行切割（分片），按照数据的偏移量进行切割，切割后形成的每个 IP 数据报（即分片）具有与初始 IP 数据

报几乎一样的 IP 数据报首部，为什么说是几乎一样而不是全部一样呢，因为 IP 数据报首部的标志、分片偏移量这两个字段与分片有关，不同的分片，这些信息可能不一样，不同的分片数据报长度也是不一样的，校验和字段也是不一样的。但是源 IP 地址、目标 IP 地址与标识号肯定是一样的，每个分片上的分片偏移量字段是不一样的。

IP 是一种不可靠的服务，一个或多个分片可能永远到达不了目的地。因为这种原因，为了让目标主机相信它已经收到了初始 IP 数据报的最后一个分片，其最后一个分片上的标志字段（最后一位）被设置为 0。而所有其他分片的标志被设为 1。另外，为了让目的主机确定是否丢失了一个分片（且能按正确的顺序重新组装分片），使用偏移字段指定该分片应放在初始 IP 数据报的哪个位置。

可能使用文字描述的不够清晰，那么我们结合图片进行描述，如图 11-6 所示：

一个主机打算发送 4000 字节的 IP 数据报（20 字节 IP 首部加上 3980 字节 IP 数据区域，假设没有 IP 数据报首部选项字段），且该数据报必须通过一条 MTU 为 1500 字节的以太网链路。这就意味着源始 IP 数据报中 3980 字节数据必须被分配为 3 个独立的数据报分片（其中的每个分片也是一个 IP 数据报）。假定初始 IP 数据报贴上的标识号为 666，那么第一个分片的数据报总大小为 1500 字节（1480 字节数据大小 +20 字节 IP 数据报首部），分片偏移量为 0，第二个分片的数据报大小也为 1500 字节，分片偏移量为 185（185*8=1480），第三个分片的数据报大小为 1040（3980-1480-1480+20），分片偏移量为 370（185+185）。

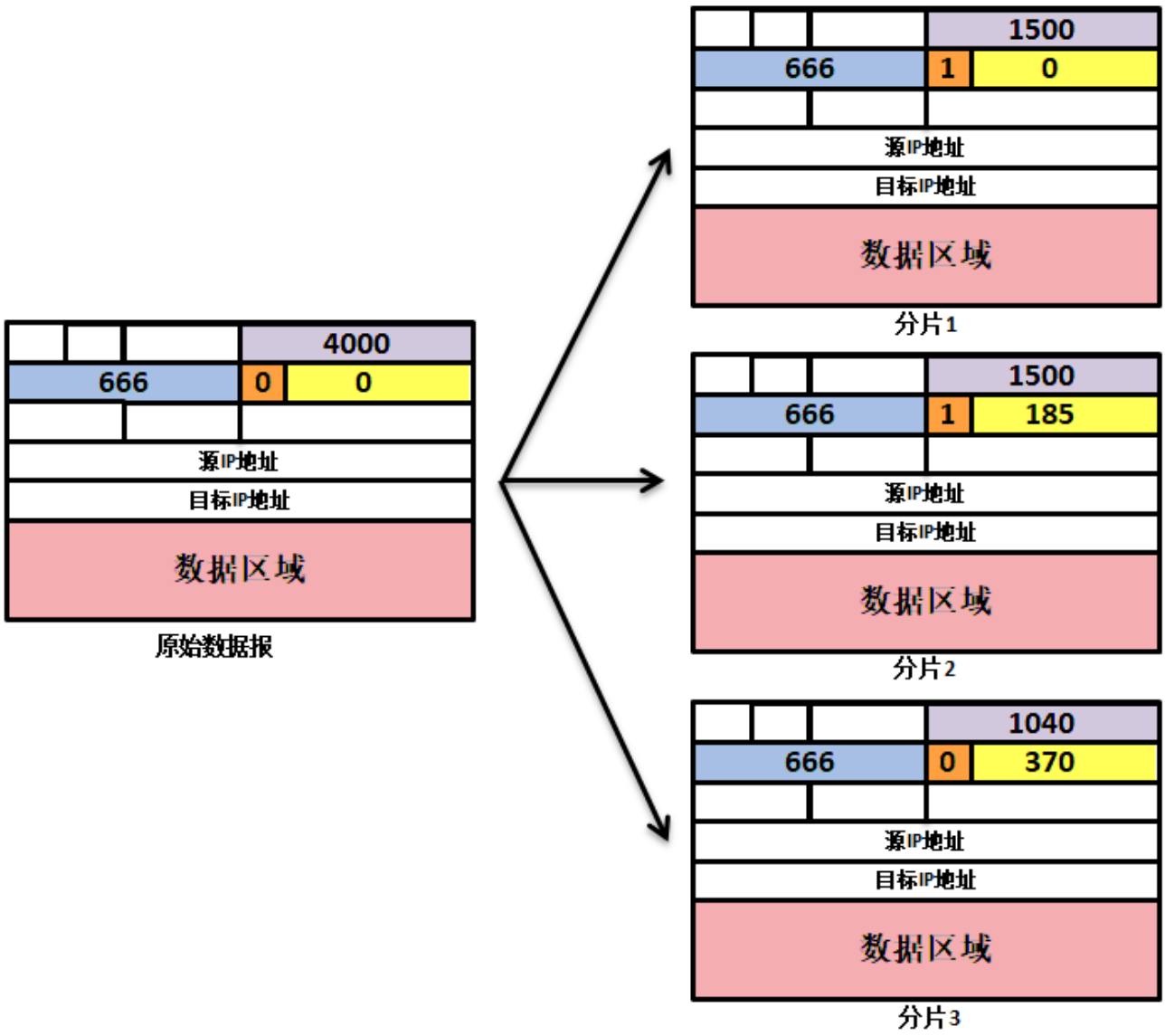


图 11-6IP 数据报分片示意图

下面看看源码是怎么样实现的，具体见*代码清单 11_2*。

代码清单 11-2IP 数据报分片源码

```
err_t
ip4_frag(struct pbuf *p,
         struct netif *netif,
```

(下页继续)

(续上页)

```
        const ip4_addr_t *dest)
{
    struct pbuf *rambuf;
    struct pbuf *newpbuf;
    u16_t newpbuflen = 0;
    u16_t left_to_copy;
    struct ip_hdr *original_iphdr;
    struct ip_hdr *iphdr;
    const u16_t nfb = (u16_t)((netif->mtu - IP_HLEN) / 8);
    u16_t left, fragsize;
    u16_t ofo;
    int last;
    u16_t poff = IP_HLEN;
    u16_t tmp;
    int mf_set;

    original_iphdr = (struct ip_hdr *)p->payload;
    iphdr = original_iphdr;
    if (IPH_HL_BYTES(iphdr) != IP_HLEN)
    {
        /* 如果 ip4_frag 不支持 IP 选项 */
        return ERR_VAL;
    }

    /* 保存原始偏移量 */
    tmp = lwip_ntohs(IPH_OFFSET(iphdr));
    ofo = tmp & IP_OFFMASK;
    /* 得到更多的分配标志位 */
    mf_set = tmp & IP_MF;

    /* 得到要发送数据的长度 */
    left = (u16_t)(p->tot_len - IP_HLEN);
```

(下页继续)

(续上页)

```
//要发送的数据长度大于 0
while (left)
{
    fragsize = LWIP_MIN(left, (u16_t)(nfb * 8));

    //申请分片 pbuf 结构
    rambuf = pbuf_alloc(PBUF_LINK, IP_HLEN, PBUF_RAM);
    if (rambuf == NULL)
    {
        goto memerr;
    }
    LWIP_ASSERT("this needs a pbuf in one piece!",
                (rambuf->len >= (IP_HLEN)));
    //拷贝原始数据的部分到分片中
    SMEMCPY(rambuf->payload, original_iphdr, IP_HLEN);

    //得到分片包存储区域
    iphdr = (struct ip_hdr *)rambuf->payload;

    //更新还需要拷贝的数据
    left_to_copy = fragsize;

    while (left_to_copy)
    {
        struct pbuf_custom_ref *pcr;

        //定义记录已经拷贝的数据大小变量 plen
        u16_t plen = (u16_t)(p->len - poff);

        //需要创建一个新 pbuf 拷贝剩下的
        newpbuflen = LWIP_MIN(left_to_copy, plen);
```

(下页继续)

(续上页)

```
        if (!newpbuflen)
        {
            poff = 0;
            p = p->next;
            continue;
        }
        //申请分片新的 pbuf
        pcr = ip_frag_alloc_pbuf_custom_ref();
        if (pcr == NULL)
        {
            pbuf_free(rambuf);
            goto memerr;
        }
        /* 初始化这个 pbuf */
        newpbuf = pbuf_alloced_custom(PBUF_RAW,
                                      newpbuflen,
                                      PBUF_REF,
                                      &pcr->pc,
                                      (u8_t *)p->payload + poff,
                                      newpbuflen);

        if (newpbuf == NULL)
        {
            ip_frag_free_pbuf_custom_ref(pcr);
            pbuf_free(rambuf);
            goto memerr;
        }

        pbuf_ref(p);
        pcr->original = p;
        pcr->pc.custom_free_function = ipfrag_free_pbuf_custom;
```

(下页继续)

(续上页)

```
            //将它添加到 rambuf 链的末尾
            pbuf_cat(rambuf, newpbuf);
            left_to_copy = (u16_t)(left_to_copy - newpbuflen);
            if (left_to_copy)
            {
                poff = 0;
                p = p->next;
            }
        }

        //更新数据报的偏移量
        poff = (u16_t)(poff + newpbuflen);

        last = (left <= netif->mtu - IP_HLEN);

        /* 设置新的偏移和 MF 标志 */
        tmp = (IP_OFFMASK & (ofo));
        if (!last || mf_set)
        {
            tmp = tmp | IP_MF;
        }
        //填写分片相关字段
        IPH_OFFSET_SET(iphdr, lwip_htons(tmp));
        IPH_LEN_SET(iphdr, lwip_htons((u16_t)(fragsize + IP_HLEN)));
        IPH_CHKSUM_SET(iphdr, 0);
#if CHECKSUM_GEN_IP
        //校验和
        IF__NETIF_CHECKSUM_ENABLED(netif, NETIF_CHECKSUM_GEN_IP)
        {
            IPH_CHKSUM_SET(iphdr, inet_chksum(iphdr, IP_HLEN));
        }
#endif
```

(下页继续)

(续上页)

```
            /* 发送数据报 */
            netif->output(netif, rambuf, dest);
            IPFRAG_STATS_INC(ip_frag.xmit);

            //释放分片空间
            pbuf_free(rambuf);
            //待发送数据减少
            left = (u16_t)(left - fragsize);
            //分片偏移增加
            ofo = (u16_t)(ofo + nfb);
        }
        MIB2_STATS_INC(mib2.ipfragoks);
        return ERR_OK;
memerr:
        MIB2_STATS_INC(mib2.ipfragfails);
        return ERR_MEM;
}
```

整个函数是比较复杂的，主要是循环处理数据报的分片，主要是处理偏移量与分片标志，拷贝原始数据的部分到分片空间中并发送出去，然后填写 IP 数据报首部的其他字段，如果是分片的最后一个数据报，则修改标志位并且发送出去，发送完成则释放分片空间。

11.7 IP 数据报发送

IP 协议是网络层的主要协议，在上层传输协议（如 TCP/UDP）需要发送数据时，就会将数据封装起来，然后传递到 IP 层，IP 层首先会根据上层协议的目标 IP 地址选择一个合适的网卡进行发送数据，当 IP 协议获得数据后将其封装成 IP 数据报的格式，填写 IP 数据报首部对应的各个字段，如目标 IP 地址、源 IP 地址、协议类型、生存时间等重要信息。最后在 IP 层通过回调函数 netif->output（即 etharp_output() 函数）将 IP 数据报投递给 ARP 协议，再调用网卡底层发送函数进行发送，这样子自上而下的数据就发送出去，IP 协议以目标 IP 地址作为目标主机的身份地址。

不过 IP 数据报发送流程是比较麻烦的，因为它嵌套了很多子函数，但是最终调用 ip4_output_if_src() 函数进行发送数据（此处不讲解带有选项字段的操作），IP 数据报发送相关的源码具体*代码清单 11_3*。

代码清单 11-3IP 数据报发送

```
err_t
ip4_output(struct pbuf *p,                                          (1)
        const ip4_addr_t *src,                                      (2)
        const ip4_addr_t *dest,                                     (3)
        u8_t ttl,                                                   (4)
        u8_t tos,                                                   (5)
        u8_t proto)                                                 (6)
{
    struct netif *netif;

    LWIP_IP_CHECK_PBUF_REF_COUNT_FOR_TX(p);

    //根据目标 IP 地址找到对应的网卡发送数据
    if ((netif = ip4_route_src(src, dest)) == NULL)                 (7)
    {
        return ERR_RTE;
    }

    return ip4_output_if(p,
                         src,
                         dest,
                         ttl,
                         tos,
                         proto,
                         netif); //新增了一个 netif 参数                (8)
}
```

(下页继续)

(续上页)

```
err_t
ip4_output_if(struct pbuf *p,
              const ip4_addr_t *src,
              const ip4_addr_t *dest,
              u8_t ttl,
              u8_t tos,
              u8_t proto,
              struct netif *netif)
{

    const ip4_addr_t *src_used = src;
    if (dest != LWIP_IP_HDRINCL)
    {
        //如果源 IP 地址是 0
        if (ip4_addr_isany(src))                                    (9)
        {
            //填写网卡的 IP 地址
            src_used = netif_ip4_addr(netif);
        }
    }
    return ip4_output_if_src(p, src_used, dest,
                             ttl, tos, proto, netif);
}

err_t
ip4_output_if_src(struct pbuf *p,
                  const ip4_addr_t *src,
                  const ip4_addr_t *dest,
                  u8_t ttl,
                  u8_t tos,
                  u8_t proto,
```

(下页继续)

(续上页)

```
                struct netif *netif)
{
    struct ip_hdr *iphdr;
    ip4_addr_t dest_addr;

    LWIP_ASSERT_CORE_LOCKED();
    LWIP_IP_CHECK_PBUF_REF_COUNT_FOR_TX(p);

    MIB2_STATS_INC(mib2.ipoutrequests);

    /* 如果 dest 不为 LWIP_IP_HDRINCL,
    表示 pbuf 中未填写 IP 数据报首部 */
    if (dest != LWIP_IP_HDRINCL)                                    (10)
    {
        u16_t ip_hlen = IP_HLEN;

        /* 调整数据区域指针以指向 IP 数据报首部 */
        if (pbuf_add_header(p, IP_HLEN))                            (11)
        {
            IP_STATS_INC(ip.err);
            MIB2_STATS_INC(mib2.ipoutdiscards);
            return ERR_BUF;    //操作失败返回 ERR_BUF 错误
        }

        //将数据首部区域强制转换成 IP 数据报首部的 ip_hdr 数据结构
        iphdr = (struct ip_hdr *)p->payload;                                (12)

        //填写生存时间
        IPH_TTL_SET(iphdr, ttl);                                    (13)
        //填写上层协议
        IPH_PROTO_SET(iphdr, proto);                                        (14)
```

(下页继续)

(续上页)

```
    /* 填写目标 IP 地址 */
    ip4_addr_copy(iphdr->dest, *dest);                    (15)

    //填写版本号 + 首部长度
    IPH_VHL_SET(iphdr, 4, ip_hlen / 4);                   (16)
    //填写服务类型
    IPH_TOS_SET(iphdr, tos);                              (17)
    //填写数据报总长度
    IPH_LEN_SET(iphdr, lwip_htons(p->tot_len));           (18)
    //填写标志位和分片偏移量
    IPH_OFFSET_SET(iphdr, 0);                             (19)
    //填写标识
    IPH_ID_SET(iphdr, lwip_htons(ip_id));                          (20)
    //标识加一
    ++ip_id;

    //填写源 IP 地址
    if (src == NULL)
    {
        ip4_addr_copy(iphdr->src, *IP4_ADDR_ANY4);
    }
    else
    {
        ip4_addr_copy(iphdr->src, *src);                          (21)
    }
    //不进行校验和检查
    IPH_CHKSUM_SET(iphdr, 0);                             (22)

}
//如果已经填写了 IP 首部
else
```

(下页继续)

(续上页)

```
    {
        //如果数据报总长度小于 IP 数据报首部长度
        if (p->len < IP_HLEN)                                          (23)
        {
            IP_STATS_INC(ip.err);
            MIB2_STATS_INC(mib2.ipoutdiscards);
            return ERR_BUF;    //返回错误
        }

        //将数据首部区域强制转换成 IP 数据报首部的 ip_hdr 数据结构
        iphdr = (struct ip_hdr *)p->payload;
        //利用 des 变量记录 IP 数据报中的目标 IP 地址
        ip4_addr_copy(dest_addr, iphdr->dest);
        dest = &dest_addr;                                             (24)
    }

    IP_STATS_INC(ip.xmit);
    ip4_debug_print(p);

    /* 如果目标 IP 地址是自己的网卡 IP 地址 */
    if (ip4_addr_cmp(dest, netif_ip4_addr(netif)))                     (25)
    {
        /* 调用环回输入 */
        LWIP_DEBUGF(IP_DEBUG, ("netif_loop_output()"));
        return netif_loop_output(netif, p);
    }
    /* 如果 IP 数据报太大，进行分片处理 */
    if (netif->mtu && (p->tot_len > netif->mtu))                       (26)
    {
        return ip4_frag(p, netif, dest);

        //直接调用注册的 netif->output 接口传递给 ARP 协议进行发送
```

(下页继续)

(续上页)

```
        return netif->output(netif, p, dest);                    (27)
    }
```

代码清单 11-3(**1**)：传输层协议需要发送的数据包 pbuf，payload 指针已经指向协议首部（注意：此处的协议首部并非 IP 数据报首部，是上层协议格式的首部）。

代码清单 11-3(**2**)：源 IP 地址。

代码清单 11-3(**3**)：目标 IP 地址，若为 LWIP_IP_HDRINCL 表示已经填写好 IP 数据报首部，且 payload 指针也指向了 IP 数据报首部。

代码清单 11-3(**4**)：设置生存时间。

代码清单 11-3(**5**)：设置服务类型。

代码清单 11-3(**6**)：设置上层协议。

代码清单 11-3(**7**)：根据目标 IP 地址找到合适的网卡发送数据，它会线性搜索网卡列表，如果找到匹配项就进行数据发送，netif 就指向找到的网卡，如果没有就返回错误。

代码清单 11-3(**8**)：调用 ip4_output_if() 函数进行发送，这里多增加了一个 netif 参数，是刚刚找到的网卡。

代码清单 11-3(**9**)：如果源 IP 地址是 0，就填写网卡中初始化的 IP 地址。

代码清单 11-3(**10**)：如果 dest 不为 LWIP_IP_HDRINCL，表示 pbuf 中未填写 IP 数据报首部，需要我们自己填写 IP 数据报首部。

代码清单 11-3(**11**)：调用 pbuf_add_header() 函数调整数据区域指针以指向 IP 数据报首部，若操作失败返回 ERR_BUF 错误。

代码清单 11-3(**12**)：将数据首部区域强制转换成 IP 数据报首部的 ip_hdr 数据结构，方便进行填写操作。

代码清单 11-3(**13**)：填写生存时间。

代码清单 11-3(**14**)：填写上层协议。

代码清单 11-3(**15**)：填写目标 IP 地址。

代码清单 11-3(**16**)：填写版本号与首部长度，这里的首部长度单位是字。

代码清单 11-3(**17**)：填写服务类型。

代码清单 11-3(**18**)：填写数据报总长度。

代码清单 11-3(**19**)：填写标志位和分片偏移量。

代码清单 11-3(**20**)：填写标识，并且将标识加一。

代码清单 11-3(**21**)：填写源 IP 地址

代码清单 11-3(**22**)：不进行校验和检查。

代码清单 11-3(**23**)：如果已经填写了 IP 首部，就无需填写 IP 数据报首部了，如果数据报总长度小于 IP 数据报首部长度，返回错误。

代码清单 11-3(**24**)：将数据首部区域强制转换成 IP 数据报首部的 ip_hdr 数据结构，并且利用 des 变量记录 IP 数据报中的目标 IP 地址，这样子能直接使用。

代码清单 11-3(**25**)：如果目标 IP 地址是自己的网卡 IP 地址，调用环回输入函数 netif_loop_output() 发送 IP 数据报给自己，这种处理一般是用于测试代码。

代码清单 11-3(**26**)：如果 IP 数据报太大，数据报总长度大于网卡的 MTU，则需要进行分片处理，调用 ip4_frag() 函数进行发送，这个函数在数据报分片处理的时候也讲解过。

代码清单 11-3(**27**)：直接调用注册的 netif->output 接口传递给 ARP 协议，实际上就是调用 etharp_output() 函数，在这里它会将 IP 地址解析成对应的 MAC 地址，并且调用网卡发送函数进行发送。

此处补充一下知识点：在上层应用一般不会调用 ip4_output() 这些函数，而是通过*代码清单 11_4* 所示的带参宏进行直接发送数据包，因为这些函数层层封装，直接调用底层的函数效率会更高，比如 UDP 协议就是通过调用 ip_output_if_src() 将数据包发送到 IP 层，因此 IP 数据报发送流程示意图具体见图 *11_7*。

代码清单 11-4

```
#define ip_output(p, src, dest, ttl, tos, proto) \
        ip4_output(p, src, dest, ttl, tos, proto)
```

(下页继续)

(续上页)

```
#define ip_output_if(p, src, dest, ttl, tos, proto, netif) \
        ip4_output_if(p, src, dest, ttl, tos, proto, netif)

#define ip_output_if_src(p, src, dest, ttl, tos, proto, netif) \
        ip4_output_if_src(p, src, dest, ttl, tos, proto, netif)

#define ip_output_hinted(p, src, dest, ttl, tos, proto, netif_hint) \
        ip4_output_hinted(p, src, dest, ttl, tos, proto, netif_hint)

#define ip_output_if_hdrincl(p, src, dest, netif) \
        ip4_output_if(p, src, LWIP_IP_HDRINCL, 0, 0, 0, netif)

#define ip_route(src, dest) \
        ip4_route_src(src, dest)

#define ip_netif_get_local_ip(netif, dest) \
        ip4_netif_get_local_ip(netif)
```

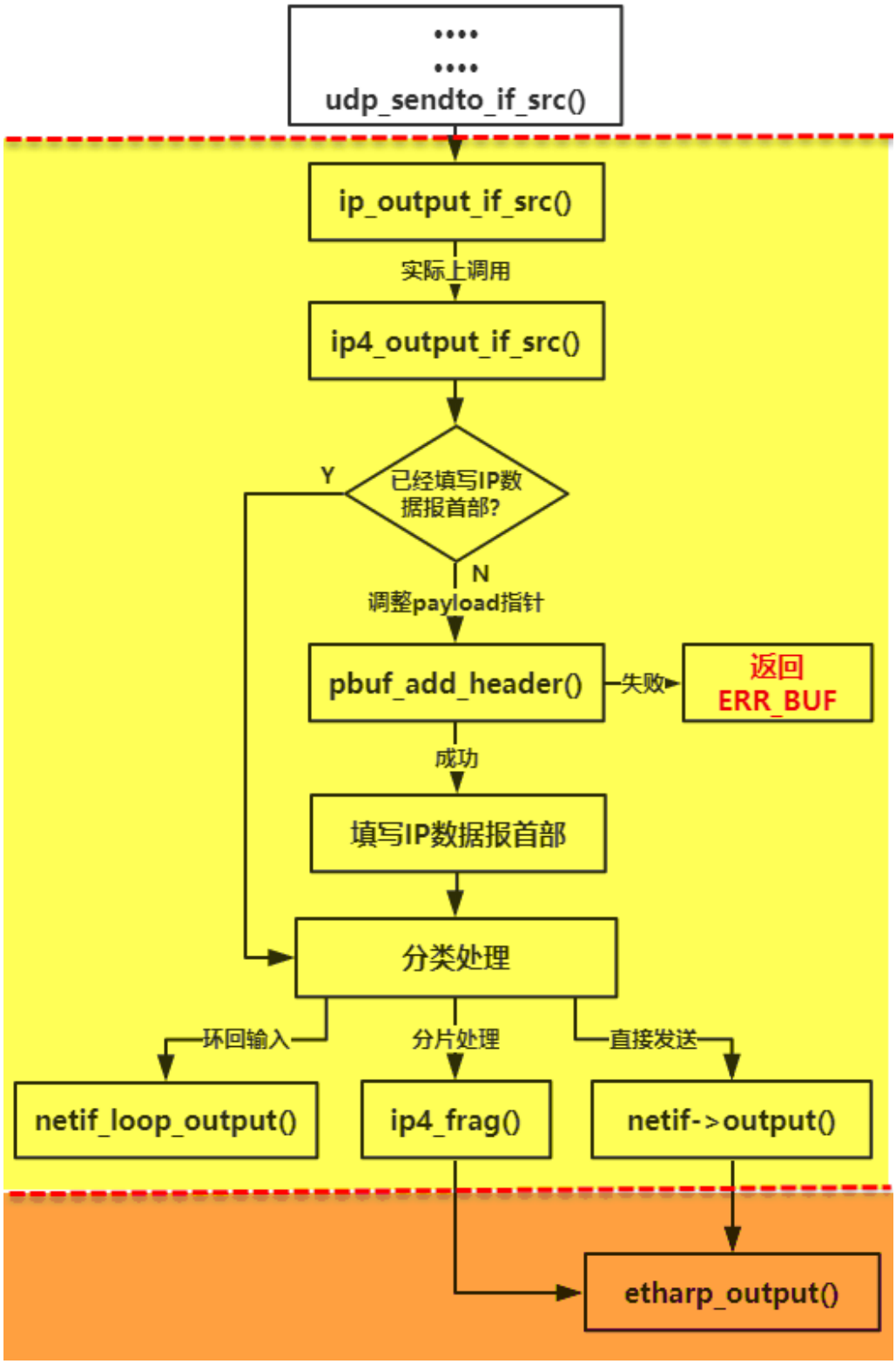

udp_sendto_if_src()
ip_output_if_src()
实际上调用
ip4_output_if_src()
已经填写IP数据报首部?
Y
N
调整payload指针
pbuf_add_header()
失败
返回ERR_BUF
成功
填写IP数据报首部
分类处理
环回输入
分片处理
直接发送
netif_loop_output()
ip4_frag()
netif->output()
etharp_output()

图 11-7IP 数据报发送流程

11.8 IP 数据报接收

从上一章的图 10_11 中我们就可以看出，一个 IP 数据报从网卡进入到 IP 协议，是通过 ethernet_input() 函数再到 ip4_input() 函数进入 IP 协议中被处理，对于 IPv4 版本的协议，所有的 IP 数据报都需要经过 ip4_input() 才能进入 IP 协议中，相对于 IP 数据报发送，接收的流程会更加复杂，因为对于所有输入的 IP 数据报，内核都要确认这些数据报是否是给自己的，并且还要保证这些数据报的格式必须是正确的；如不是给本地的数据报，IP 层还要将这些数据报进行转发或者丢弃，当然，如果对于分片的数据报，IP 层还需要负责将其组装起来，并且校验组装完成的数据报是否完整，如果不完整则丢弃它（关于组装数据报的部分比较麻烦，我们也无需了解太多，只要知道即可），当数据报是正确的，IP 层就会递交给上层协议（如 UDP 协议、TCP 协议），ip4_input() 源码具体见*代码清单 11_5*。

代码清单 11-5 ip4_input() 源码（已删减）

```
err_t
ip4_input(struct pbuf *p, struct netif *inp)
{
    const struct ip_hdr *iphdr;
    struct netif *netif;
    u16_t iphdr_hlen;
    u16_t iphdr_len;

    LWIP_ASSERT_CORE_LOCKED();

    IP_STATS_INC(ip.recv);
    MIB2_STATS_INC(mib2.ipinreceives);

    /* 识别 IP 数据报首部  */
    iphdr = (struct ip_hdr *)p->payload;                    (1)
    //如果不是 IPv4，删除 pbuf，返回 ERR_OK。
```

(下页继续)

(续上页)

```
    if (IPH_V(iphdr) != 4)                                          (2)
    {
        ip4_debug_print(p);
        pbuf_free(p);
        IP_STATS_INC(ip.err);
        IP_STATS_INC(ip.drop);
        MIB2_STATS_INC(mib2.ipinhdrerrors);
        return ERR_OK;
    }

    /* 以字节为单位获取 IP 头长度 */
    iphdr_hlen = IPH_HL_BYTES(iphdr);
    /* 以字节为单位获取数据报总长度 */
    iphdr_len = lwip_ntohs(IPH_LEN(iphdr));                         (3)

    if (iphdr_len < p->tot_len)
    {
        pbuf_realloc(p, iphdr_len);
    }

    /* 标头长度超过第一个 pbuf 长度，或者 ip 长度超过总 pbuf 长度 */
    if ((iphdr_hlen > p->len) ||
            (iphdr_len > p->tot_len) ||
            (iphdr_hlen < IP_HLEN))                                 (4)
    {
        /* 删除 pbuf，返回 ERR_OK */
        pbuf_free(p);
        IP_STATS_INC(ip.lenerr);
        IP_STATS_INC(ip.drop);
        MIB2_STATS_INC(mib2.ipindiscards);
        return ERR_OK;
    }
```

(下页继续)

(续上页)

```
/* 将源 IP 地址与目标 IP 地址复制到对齐的 ip_addr_t */
ip_addr_copy_from_ip4(ip_data.current_iphdr_dest, iphdr->dest);
ip_addr_copy_from_ip4(ip_data.current_iphdr_src, iphdr->src);

/* 匹配网卡的数据包，即这个包是发给我们的 */
if (ip4_addr_ismulticast(ip4_current_dest_addr()))                (5)
{
    //如果网卡已经挂载了并且 IP 地址有效
    if ((netif_is_up(inp)) &&
            (!ip4_addr_isany_val(*netif_ip4_addr(inp))))
    {
        netif = inp;
    }
    else
    {
        netif = NULL;
    }
}
//不是给我们的
else
{
    if (ip4_input_accept(inp))
    {
        netif = inp;
    }
    else
    {
        netif = NULL;
        /* 检查一下目标 IP 地址是否是环回地址 */
        if (!ip4_addr_isloopback(ip4_current_dest_addr())) (6)
        {
```

(下页继续)

(续上页)

```
                NETIF_FOREACH(netif)
                {
                    if (netif == inp)
                    {

                        continue;
                    }
                    if (ip4_input_accept(netif))
                    {
                        break;
                    }
                }
            }
        }
    }

    //校验源 IP 地址，源 IP 地址不能是多播或者广播地址
    if ((ip4_addr_isbroadcast(ip4_current_src_addr(), inp)) ||
            (ip4_addr_ismulticast(ip4_current_src_addr()))) (7)
    {
        pbuf_free(p);
        IP_STATS_INC(ip.drop);
        MIB2_STATS_INC(mib2.ipinaddrerrors);
        MIB2_STATS_INC(mib2.ipindiscards);
        return ERR_OK;
    }

    /* 到了这里，如果还没找到对应的网卡，说明数据包不是给我们的 */
    if (netif == NULL)
    {
        /* 路由转发或者丢弃。如果 IP_FORWARD 宏定义被使能，则进行转发 */
#if IP_FORWARD
```

(下页继续)

(续上页)

```
        /* 非广播包 */
        if (!ip4_addr_isbroadcast(ip4_current_dest_addr(), inp)) (8)
        {
            /* 尝试在（其他）网卡上转发 IP 数据包 */
            ip4_forward(p, (struct ip_hdr *)p->payload, inp);
        }
        else
#endif
        {
            /* 删除 pbuf，返回 ERR_OK */
            IP_STATS_INC(ip.drop);
            MIB2_STATS_INC(mib2.ipinaddrerrors);
            MIB2_STATS_INC(mib2.ipindiscards);
        }
        pbuf_free(p);                                                (9)
        return ERR_OK;
    }
    /* 如果数据报由多个片段组成，也就是数据报被分片处理了 */
    if ((IPH_OFFSET(iphdr) & PP_HTONS(IP_OFFMASK | IP_MF)) != 0)
    {
#if IP_REASSEMBLY /*  */
        /* 重装数据报 */
        p = ip4_reass(p);                                            (10)
        /* 如果重装没有完成 */
        if (p == NULL)
        {
            return ERR_OK;
        }
        //分片重装完成，将数据报首部强制转换为 ip_hdr 类型
        iphdr = (const struct ip_hdr *)p->payload;                   (11)
#else
        //不支持重装的数据，就删除掉
```

(下页继续)

(续上页)

```
        pbuf_free(p);                                                    (12)
        IP_STATS_INC(ip.opterr);
        IP_STATS_INC(ip.drop);
        MIB2_STATS_INC(mib2.ipinunknownprotos);
        return ERR_OK;
#endif
    }

    /* IP 数据报首部中不支持 IP 选项？ */
#if IP_OPTIONS_ALLOWED == 0

    //如果 IP 数据报首部长度大于 20 字节，就表示错误
    if (iphdr_hlen > IP_HLEN)                                            (13)
    {
        pbuf_free(p);
        IP_STATS_INC(ip.opterr);
        IP_STATS_INC(ip.drop);
        /* 不支持的协议功能 */
        MIB2_STATS_INC(mib2.ipinunknownprotos);
        return ERR_OK;
    }
#endif

    /* 发送到上层协议 */
    LWIP_DEBUGF(IP_DEBUG, ("ip4_input: \n"));
    ip4_debug_print(p);

    ip_data.current_netif = netif;
    ip_data.current_input_netif = inp;
    ip_data.current_ip4_header = iphdr;
    ip_data.current_ip_header_tot_len = IPH_HL_BYTES(iphdr);
```

(下页继续)

(续上页)

```
#if LWIP_RAW
    /* RAW API 输入 */
    raw_status = raw_input(p, inp);
    if (raw_status != RAW_INPUT_EATEN)
#endif /* LWIP_RAW */
    {
        /* 移动 payload 指针，指向有效的数据区域 */
        pbuf_remove_header(p, iphdr_hlen);                    (14)

        //根据 IP 数据报首部的协议的类型处理
        switch (IPH_PROTO(iphdr))
        {
        //对于 UDP 协议
        case IP_PROTO_UDP:
        case IP_PROTO_UDPLITE:
            MIB2_STATS_INC(mib2.ipindelivers);
            udp_input(p, inp);                                (15)
            break;

        //对于 TCP 协议
        case IP_PROTO_TCP:
            MIB2_STATS_INC(mib2.ipindelivers);
            tcp_input(p, inp);                                (16)
            break;

        //对于 ICMP 协议
        case IP_PROTO_ICMP:
            MIB2_STATS_INC(mib2.ipindelivers);
            icmp_input(p, inp);                               (17)
            break;

        //对于 IGMP 协议
```

(下页继续)

(续上页)

```
        case IP_PROTO_IGMP:
            igmp_input(p, inp, ip4_current_dest_addr());     (18)
            break;

        default:
            //找不到上层协议
#if LWIP_RAW
            if (raw_status == RAW_INPUT_DELIVERED)
            {
                MIB2_STATS_INC(mib2.ipindelivers);
            }
            else
#endif /* LWIP_RAW */
            {
#if LWIP_ICMP
                /* 除非是广播，否则发送 ICMP 目标协议不可达差错报文 */
                if (!ip4_addr_isbroadcast(ip4_current_dest_addr(), netif)
                        &&!ip4_addr_ismulticast(ip4_current_dest_addr()))
                {
                    pbuf_header_force(p, (s16_t)iphdr_hlen);
                    icmp_dest_unreach(p, ICMP_DUR_PROTO);    (19)
                }
#endif /* LWIP_ICMP */

                IP_STATS_INC(ip.proterr);
                IP_STATS_INC(ip.drop);
                MIB2_STATS_INC(mib2.ipinunknownprotos);
            }
            pbuf_free(p);
            break;
        }
    }
```

(下页继续)

(续上页)

```
    /* 全局变量清零 */
    ip_data.current_netif = NULL;                                    (20)
    ip_data.current_input_netif = NULL;
    ip_data.current_ip4_header = NULL;
    ip_data.current_ip_header_tot_len = 0;
    ip4_addr_set_any(ip4_current_src_addr());
    ip4_addr_set_any(ip4_current_dest_addr());

    return ERR_OK;
}
```

代码清单 11-5(**1**)：根据数据报的个数识别数据报首部，并且将首部强制转换成 ip_hdr 结构体格式。

代码清单 11-5(**2**)：检测 IP 数据报版本，如果不是 IPv4，删除 pbuf，返回 ERR_OK。

代码清单 11-5(**3**)：以字节为单位获取 IP 头长度保存在 iphdr_hlen 变量中，以字节为单位获取数据报总长度保存在 iphdr_len 变量中。

代码清单 11-5(**4**)：标头长度超过第一个 pbuf 长度，或者 ip 长度超过总 pbuf 长度，表示这些数据报是错误的，删除 pbuf，返回 ERR_OK。

代码清单 11-5(**5**)：将源 IP 地址与目标 IP 地址复制到对齐的 ip_addr_t，然后匹配网卡的数据包，即看看这个数据报是不是发给我们的，如果网卡已经挂载了并且 IP 地址有效，则继续。

代码清单 11-5(**6**)：如果不是给我们的，那就再检查一下目标 IP 地址是否是环回地址。

代码清单 11-5(**7**)：校验源 IP 地址，源 IP 地址不能是多播或者广播地址。

代码清单 11-5(**8**)：程序执行到了这里，如果还没找到对应的网卡，说明数据包不是给我们的，内核可以选择路由转发或者丢弃，如果 IP_FORWARD 宏定义被使能，则进行转发，对于非广播包，就尝试在（其他）网卡上转发 IP 数据报。

代码清单 11-5(**9**)：否则就删除 pbuf，返回 ERR_OK。

代码清单 11-5(**10**)：如果数据报由多个片段组成，也就是数据报被分片处理了，如果定义了

IP_REASSEMBLY 这个宏，那就调用 ip4_reass() 函数重装数据报，关于数据报的重装处理是比较麻烦的，我们暂且不讲解它。

代码清单 11-5(**11**)：分片重装完成，将数据报首部强制转换为 ip_hdr 类型。

代码清单 11-5(**12**)：如果不支持重装的数据，就删除掉数据报。

代码清单 11-5(**13**)：如果宏定义 IP_OPTIONS_ALLOWED 被定义为 0，表示 IP 数据报首部中不支持 IP 选项，如果 IP 数据报首部长度大于 20 字节，就表示错误。

代码清单 11-5(**14**)：程序执行到这里，就表示数据报是正确的，可以发送到上层协议，移动 payload 指针，指向有效的数据区域。

代码清单 11-5(**15**)：根据 IP 数据报首部的协议的类型处理，对于 UDP 协议，调用 UDP 协议的输入函数 udp_input() 接收数据报。

代码清单 11-5(**16**)：对于 TCP 协议，调用 tcp_input() 函数接收数据报。

代码清单 11-5(**17**)：对于 ICMP 协议，调用 icmp_input() 函数接收数据报。

代码清单 11-5(**18**)：对于 IGMP 协议，调用 igmp_input() 函数接收数据报。

代码清单 11-5(**19**)：如果找不到上层协议，除非是广播包，否则内核将发送 ICMP 目标协议不可达差错报文返回源主机。

代码清单 11-5(**20**)：全局变量清零。

整个 IP 数据报输入过程示意图具体见图 *11_8*。

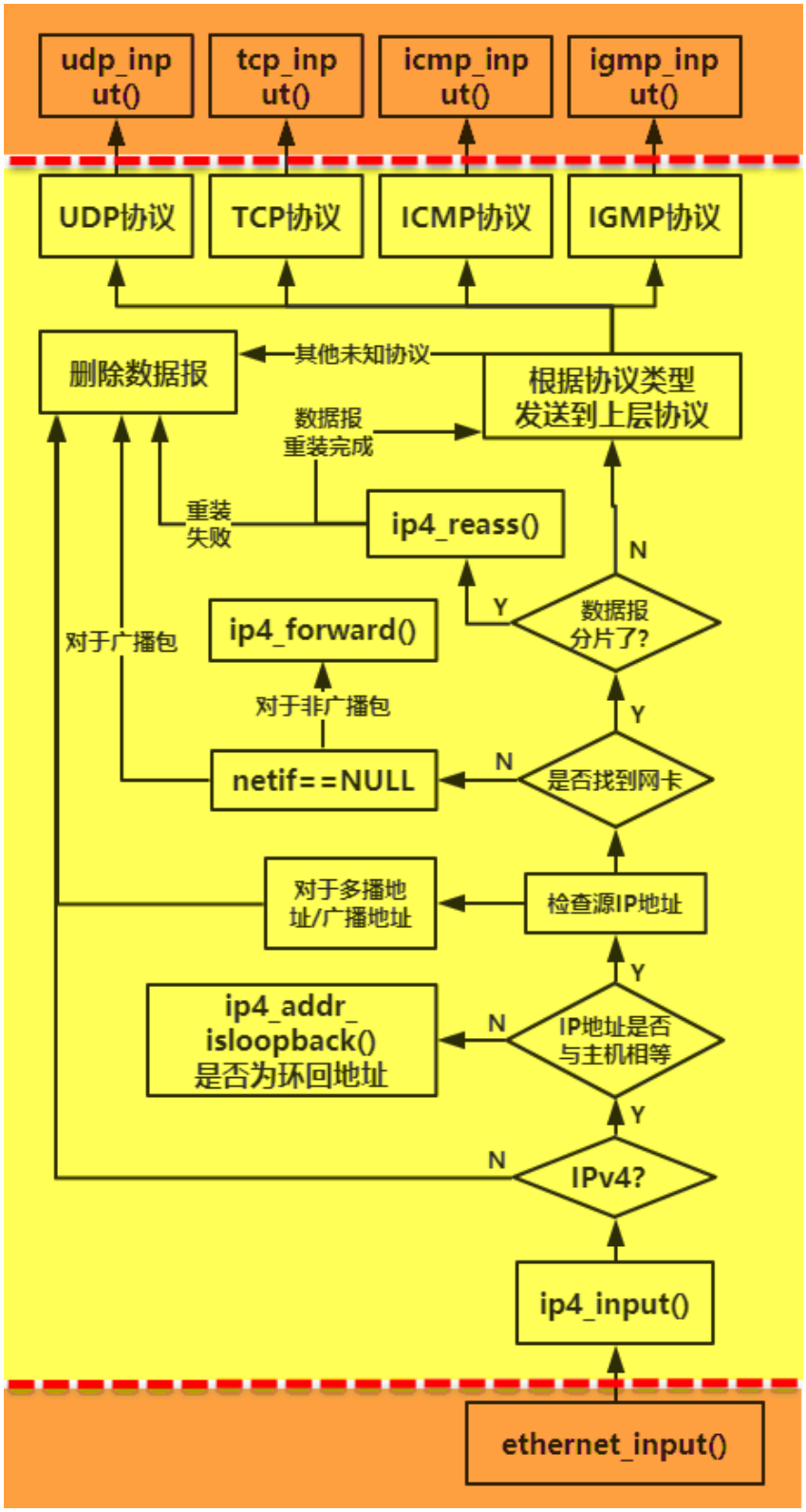

udp_input()
tcp_input()
icmp_input()
igmp_input()
UDP协议
TCP协议
ICMP协议
IGMP协议
删除数据报
其他未知协议
根据协议类型
发送到上层协议
数据报
重装完成
重装
失败
ip4_reass()
N
Y
数据报
分片了?
对于广播包
ip4_forward()
对于非广播包
Y
N
是否找到网卡
netif==NULL
对于多播地
址/广播地址
检查源IP地址
Y
ip4_addr_
isloopback()
是否为环回地址
N
IP地址是否
与主机相等
Y
N
IPv4?
ip4_input()
ethernet_input()

图 11-8IP 数据报接收处理流程

第 12 章　网际控制报文协议 ICMP

12.1 ICMP 功能简介

ICMP 是“Internet Control Message Protocol”（因特网控制报文协议）的缩写。在前面的学习中，我们也知道 IP 协议是一种不可靠、无连接的协议，只在各个主机间交付数据，但是对于数据的到达与否，IP 协议并不关心，为了提高数据交付的准确性，ICMP 就随之出现，在交付数据的时候，如果由于网络状况、链路不通等情况数据报无法到达目标主机，ICMP 就会返回一个差错报文，让源主机知道数据没能正常到达目标主机，接着进行重发或者放弃发送都可以。如在 IP 数据报生存时间 TTL 为 0 的时候，路由器不再对数据报进行转发操作，而是会直接丢弃这个数据报，并且返回一个 ICMP 报文，告诉源主机——你的数据报生存时间到了，还是没有送到目标主机那里，你可以选择重发或者放弃，你看着办就好。

为什么需要 ICMP，因为 IP 协议认为丢掉没用的数据是合理的，这样子能提高数据处理的效率，但是源主机更希望能得到当数据没能发送到目标的时候有个回应，不然目标主机都不知道发的数据到了哪里，到达接收没用，就像石沉大海一样，这是不可接受的。此外，ICMP 也被主机和路由器用来彼此沟通网络层的信息，如果链路都连不通，那发送数据就是没有意义的事情，ICMP 报文虽然不传输用户数据，但是对于用户数据的传递起着重要的作用。

ICMP 最典型的用途是差错报告。例如，当运行一个 FTP 或 HTTP 应用时，也许会遇到一些诸如“目的不可达”之类的错误报文，这种报文就是在 ICMP 中产生的，可能在转发的时候，在某个位置，路由器不能找到一条合适的路径转发该数据包以通往 FTP 或 HTTP 应用所指定的主机，该路由器就会向源主机创建和发出一个类型为 3 的 ICMP 报文以指示该错误（目的不可达）。

ICMP 通常被认为是 IP 层协议的一部分，但从体系结构上讲它是位于 IP 之上的，因为 ICMP 报文是承载在 IP 数据报中的。这就是说，ICMP 报文是作为 IP 数据报数据区域的（有一些书籍也称之为有效载荷），就像 TCP 与 UDP 报文段作为 IP 数据报数据区域那样。类似地，当一台主机收到一个指明上层协议为 ICMP 的 IP 数据报时，它将分解出该数据报的内容给 ICMP，就像分解出一个数据报的内容给 TCP 或 UDP 一样，但与 TCP 或 UDP 协议又有所不同，ICMP 出现的目的不是为上层应用程序提供服务，只是在 IP 层传递差错的报文，依赖于 IP 协议进行传输。

12.2 ICMP 报文结构

ICMP 报文是使用 IP 数据报来封装发送的，所以 ICMP 报文也是没有额外的可靠性与优先级，它一样会被别的路由器丢弃，与此同时，ICMP 报文封装在 IP 数据报中，IP 数据报封装在以太网帧中，因此 ICMP 报文是经过了两次的封装，具体见图 *12_1*。

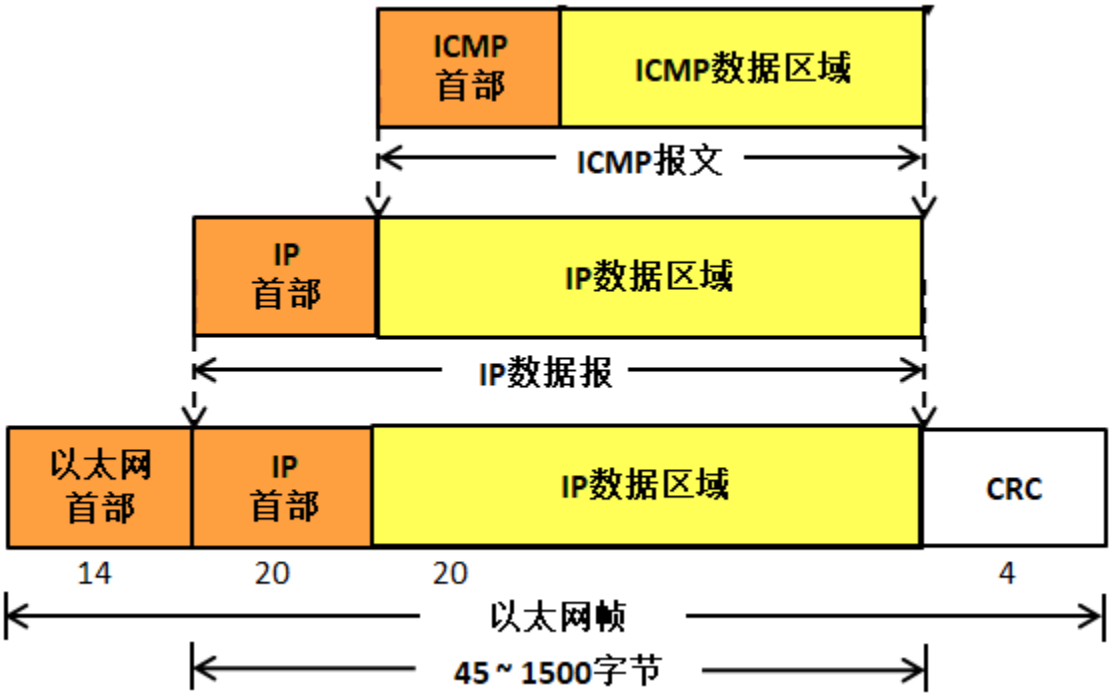


图 12-1ICMP 报文的封装

ICMP 报文与 IP 数据报一样，都是由首部与数据区域组成，ICMP 首部是 8 个字节，对于不同类型的 ICMP 报文，ICMP 报文首部的格式也会有点差异，但是首部的前 4 个字节都是通用的：

第一个字节（占据 8bit 空间）是类型（Type）字段，表示产生这种类型 ICMP 报文的原因。

第二个字节是代码（Code）字段，它进一步描述了产生这种类型 ICMP 报文的具体原因。因为每种类型的报文都可能有多个，比如目的不可达报文，产生这种原因可能有主机不可达、协议不可达、端口不可达等多个原因。

接下来的校验和字段（占据 16bit）用于记录包括 ICMP 报文数据部分在内的整个 ICMP 数据报的

校验和，以检验报文在传输过程中是否出现了差错，其计算方法与在我们介绍 IP 数据包首部中的校验和计算方法是一样的。

剩下的 4 个字节部分在讲解报文类型的时候详细讲解，因为不同类型的报文都有不一样的定义，并且数据部分的长度也存在差异，ICMP 报文格式具体见图 *12_2*。



图 12-2ICMP 报文格式

12.3 ICMP 报文类型

ICMP 报文有两大类型，可以划分为差错报告报文和查询报文，差错报告报文主要是用来向 IP 数据报源主机返回一个差错报告信息，而这个差错报告信息产生的原因是路由器或者主机不能对当前数据报进行正常的处理，简单来说就是源主机发送的数据报没法到目标主机中，或者到达了目标主机而无法递交给上层协议。

查询报文是用于一台主机向另一台主机发起一个请求，如果目标主机收到这个查询的请求后，就会按照查询报文的格式向源主机做出应答，比如我们使用的 ping 命令，它的本质就是一个 ICMP 查询报文。

差错报文与查询报文报文都有其具体的类型，具体见

ICMP 报文类型	具体类型	描述
差错报告报文	3	目的不可达
	4	源站抑制
	5	重定向
	11	超时
	12	参数错误报文
查询报文	0 或 8	回显请求或回显应答
	9 或 10	路由器询问或通告
	13 或 14	时间戳请求或应答
	15 或 16	信息请求或信息应答
	17 或 18	掩码请求或应答

虽然 ICMP 报文很多，但是它并不能纠正错误，它只是借助 IP 协议简单报告差错，差错报文是被返回源主机的，因为出现差错了，数据报中唯一可用的就是目标 IP 地址与源 IP 地址，源主机收到 ICMP 差错报告后，传递给上层协议，要怎么样处理就不是 ICMP 协议的事情了。

12.3.1 ICMP 差错报告报文

12.3.1.1 目的不可达

在日常生活中，邮寄包裹会经过多个传递环节，任意一环如果无法传下去，都会返回给寄件人，并附上无法邮寄的原因。同理，当路由器收到一个无法传递下去的 IP 数据报时，那么路由器就会丢弃这个数据报，然后返回一个 ICMP 目的不可达差错报告报文（类型为 3）给 IP 报文的源发送方，产生差错的原因有很多，如网络不可达、主机不可达、协议不可达、端口不可达等，在 ICMP 报文中的代码字段（Code）就用于记录引起差错的原因。

对于不同的差错代码字段的值是不一样的，但是 LwIP 实现的只有前 6 种，具体见表格 12-1。

表格 12-1 目的不可达的代码字段取值及说明

代码取值	说明	代码取值	说明
0	网络不可达	8	源主机被隔离（作废不用）
1	主机不可达	9	目标网络被禁止
2	协议不可达	10	目标主机被禁止
3	端口不可达	11	由于服务类型 TOS，网络不可达
4	需要分片但设置了不分片	12	由于服务类型 TOS，主机不可达
5	源站路由失败	13	由于过滤，通信被强行禁止
6	目标网络未知	14	主机越权
7	目标主机未知	15	优先级中止生效

同时 ICMP 目的不可达报文首部剩下的 4 字节全部未用，而 ICMP 报文数据区域则装载 IP 数据报首部及 IP 数据报的数据区域前 8 字节，为什么需要装载 IP 数据报的数据区域中前 8 个字节的数据呢？因为 IP 数据报的数据区域前 8 个字节刚好覆盖了传输层协议中的端口号字段，而 IP 数据报首部就拥有目标 IP 地址与源 IP 地址，当源主机收到这样子的 ICMP 报文后，它能根据 ICMP 报文的数据区域判断出是哪个数据包出现问题，并且 IP 层能够根据端口号将报文传递给对应的上层协议处理，ICMP 目的不可达报文的格式具体见图 *12_3*。



图 12-3ICMP 目的不可达报文

12.3.1.2 源站抑制

因为 IP 协议本身就是一个无反馈的协议，IP 层发送数据出去的时候就不会收到任何回复，当然也没不会知道目标主机那边的网络状况如何，可能源主机就一直发送数据到目标主机那边，但是由于处理性能、网络等其他因素，就会导致拥塞现象，就好比我们的公路一样，IP 数据报就是在公路上行驶的车辆，如果数据报太多了，路由/目标主机来不及处理这些数据报，那么就会堵在这里，就跟堵车是一个道理，而 ICMP 源站抑制报文的出现就是告诉源主机，你不要发送那么多数据报了。

这个对于我们来说也不太重要，就简单看看其格式，实际上 ICMP 源站抑制报文的格式与 ICMP 目的不可达报文的格式是一样的，但代码字段为 0。

12.3.1.3 重定向

一般来说，某个主机在启动的时候只有一个路由表（即默认路由），所以它发送的数据就是都发给了默认路由，让其帮忙转发，而路由器发现数据应该是发给另一个路由器的，那么它就会返回一个 ICMP 重定向报文给源主机，告诉它应该直接发给另一个路由器。举个例子，比如有人在你们班级找人，但是要找的人不在你们班，而在隔壁班，你们就会告诉他那个人在隔壁班，让他在隔壁班找就行了，路由器也是这样子。

重定向一般用来让刚启动的主机逐渐建立更完善的路由表，因为主机启动时路由表中可能只有一个默认路由。一旦默认路由器发现它可以转发给其他路由器的时候，默认路由器将通知它进行重定向，告诉主机对路由表作相应的改动，这样子就无需默认路由处理这些事情，而且数据报传输的效率更高了，当然啦，重定向报文是只能由路由器生成而不能由主机生成，但是使用重定向报文的只能是主机而非路由器，所以我们暂时不用理会这些报文的格式是怎么样的，而且 LwIP 对于这类数据报都不给予理会。

12.3.1.4 超时

IP 数据报首部有一个 TTL 字段，就是防止 IP 数据报永远在网络中漂流，当数据报每被转发一次，TTL 的值就减一，如果 TTL 为 0，那么路由或者主机就会丢弃该数据报，并且返回一个 ICMP 超时报文到源主机中；此外，在数据报分片重装的时候也使用了 ICMP 报文，当所有的 IP 分片数据报无法在规定的时间内完成重装，那么主机也会认为它超时了，那么这些数据报就会被删除，同时也返回一个 ICMP 超时报文到源主机中，ICMP 超时报文格式与 ICMP 目的站不可达报文格式一样，但代码字段中有两种取值，具体见表格 12-2。

表格 12-2 超时报文代码字段及说明

代码字段取值	说明
0	生存时间超时（TTL 为 0）
1	分片数据报重装超时

12.3.1.5 参数错误

IP 数据报在网络中传输的时候，都是根据其首部进行识别的，如果首部出现错误，那么就会产生严重的问题，因此如果 IP 数据报首部出现错误就会丢弃数据报，并且向源主机返回一个 ICMP 参数错误报文，当然啦，对于携带 ICMP 差错报文的数据报、非第一个分片的分片数据报、具有特殊目的地址的数据报（如环回、多播、广播）这些类型的数据报，即使是出现了差错也不会返回对应的差错报文。

12.3.2 ICMP 查询报文

我们知道，ping 命令使用的就是 ICMP 查询报文，若能 ping 成功，说明网卡、IP 层、ICMP 层都能通信正常，所以能证明 LwIP 已经移植成功了，我们一般在移植完成的时候都会测试一下 ping 命令，查看一下是否移植成功。

ping 这个名字源于声纳定位操作。ping 程序由 Mike Muuss 编写，目的是为了测试另一台主机是否可达。该程序发送一份 ICMP 回显请求报文给目标主机，并等待目标主机返回 ICMP 回显应答报文。

ICMP 回显请求报文和回显应答报文是 LwIP 中唯一实现的报文，而其他几种报文只是在以前的时候主机启动时确定自己的 IP 地址、掩码、路由状况等信息，现在基本用不到了，因为 DHCP 协议就已全部帮我们实现了，使用我们只讲解 ICMP 回显请求报文和回显应答报文。

我们称发送回显请求的 ping 程序为客户，而称被 ping 的主机为服务器。大多数的 TCP/IP 协议栈中都在内核中直接支持 ping 服务器，但是注意了，这种 ping 服务器不是一个用户线程，只是在内核线程中进行处理的。

ICMP 回显请求和回显应答报文格式具体见图 *12_4*。

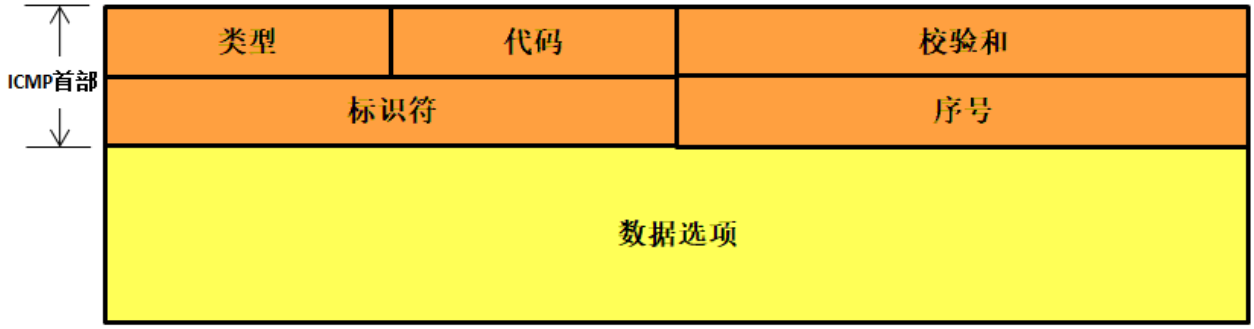


图 12-4ICMP 回显请求和回显应答报文格式

ICMP 报文首部剩下的 4 字节内容分为了两个字段，标识符用于标识在同一台主机上同时运行了多个 ping 程序，ping 程序也可以识别出返回的信息。序列号从 0 开始，每发送一次新的回显请求就进行加 1。ping 程序打印出返回的每个分组的序列号，允许我们查看是否有分组丢失、失序或重复。数据选项区域表示回显请求报文中可包含的数据，其长度是可选的，发送方选择合适的长度并且填充该段数据，在接收方，它会根据这个会回显请求返回一个回显应答报文，回显应答报文中的数据选项区域是与回显请求报文的数据选项区域完全一致。

我们可以通过 wireshark 抓包工具进行抓包查看 ICMP 回显请求报文与回显应答报文的格式，首先打开 wireshark 抓包工具开始抓包，然后打开 cmd 控制台，输入 “ping baidu.com”，接着我们在 wireshark 抓包工具就能看到 ICMP 的数据包了，点击对应的数据包查看 ICMP 回显请求报文与回显应答报文即可，具体见图 *12_5*

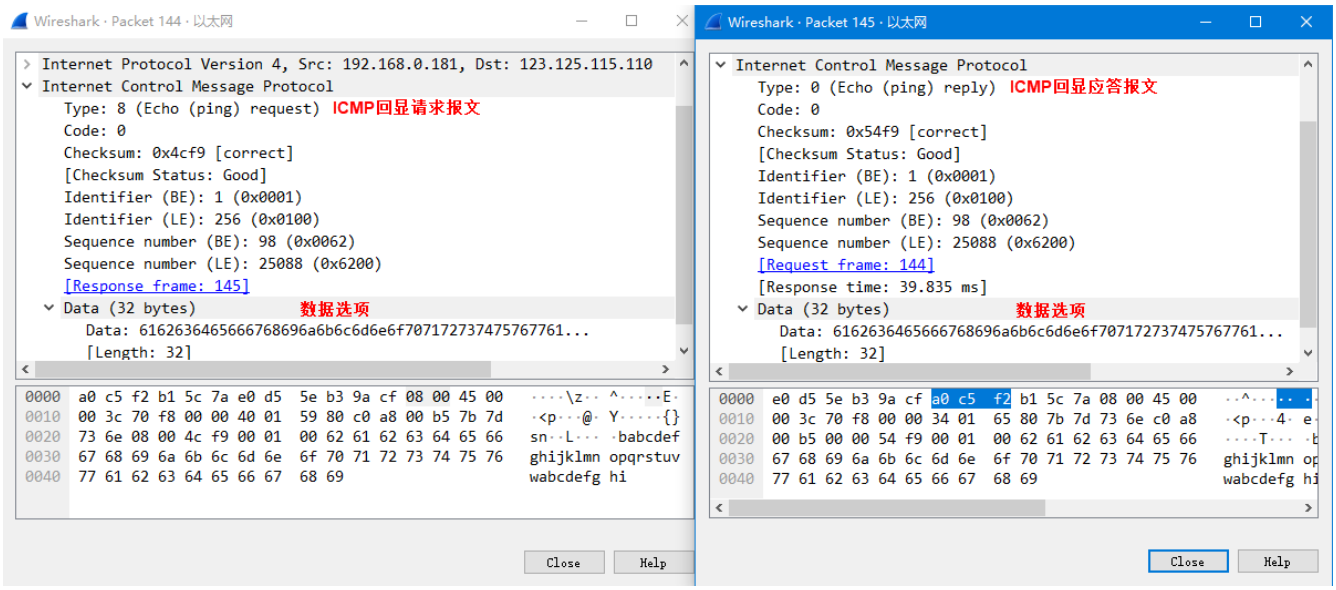


图 12-5ICMP 回显请求报文与回显应答报文

12.4 LwIP 中的 ICMP 实现

12.4.1 ICMP 报文数据结构

ICMP 报文与 IP 数据报的数据结构差不多，在 LwIP 中定义一个结构体对其进行描述，该结构体名字为 icmp_echo_hdr，看名字的话是不是很吃惊，居然是 ICMP 回显报文的首部，其实的确也是这样子的，因为各个类型的 ICMP 报文首部都是差不多的，所以能将 ICMP 回显报文首部用于其他 ICMP 报文首部，其内的成员变量分别对应 ICMP 报文首部的类型、代码、校验和、标识符、序号等字段，icmp_echo_hdr 结构体具体见*代码清单 12_1*。

代码清单 12-1 icmp_echo_hdr 结构体

```
PACK_STRUCT_BEGIN
struct icmp_echo_hdr
{
    PACK_STRUCT_FLD_8(u8_t type);
    PACK_STRUCT_FLD_8(u8_t code);
```

(下页继续)

(续上页)

```
    PACK_STRUCT_FIELD(u16_t chksum);
    PACK_STRUCT_FIELD(u16_t id);
    PACK_STRUCT_FIELD(u16_t seqno);
} PACK_STRUCT_STRUCT;
PACK_STRUCT_END
```

此外 LwIP 还定义了很多宏与枚举类型的变量对 ICMP 的类型及代码字段进行描述，具体见*代码清单 12_2*。

代码清单 12-2 LwIP 中实现的宏及枚举变量

```
#define ICMP_ER    0     /* 回显应答 */
#define ICMP_DUR   3     /* 目的不可达 */
#define ICMP_SQ    4     /* 源站抑制 */
#define ICMP_RD    5     /* 重定向 */
#define ICMP_ECHO  8     /* 回显请求 */
#define ICMP_TE   11     /* 超时 */
#define ICMP_PP   12     /* 参数错误 */
#define ICMP_TS   13     /* 时间戳请求 */
#define ICMP_TSR  14     /* 时间戳应答 */
#define ICMP_IRQ  15     /* 信息请求 */
#define ICMP_IR   16     /* 信息应答 */
#define ICMP_AM   17     /* 地址掩码请求 */
#define ICMP_AMR  18     /* 地址掩码应答 */

/** ICMP 目标不可达代码字段 */
enum icmp_dur_type
{
    /** 网络不可达 */
    ICMP_DUR_NET    = 0,
    /** 主机不可达 */
    ICMP_DUR_HOST   = 1,
```

(下页继续)

(续上页)

```
    /** 协议不可达 */
    ICMP_DUR_PROTO = 2,
    /** 端口不可达 */
    ICMP_DUR_PORT  = 3,
    /** 需要分片但设置了不分片 */
    ICMP_DUR_FRAG  = 4,
    /** 源站路由失败 */
    ICMP_DUR_SR    = 5
};

/** ICMP 超时代码字段 */
enum icmp_te_type
{
    /** 生存时间超时 */
    ICMP_TE_TTL  = 0,
    /** 分片重装超时 */
    ICMP_TE_FRAG = 1
};
```

看到这些宏定义是不是发现很多类型的 ICMP 报文都没被实现，在 LwIP 中确实也是这样子，我们也知道，LwIP 是一个较为完整的 TCP/IP 协议栈，只实现了对 ICMP 回显请求报文的处理，以及某些差错发送报告，如目的不可达报告报文、超时报文，而对于其他的 ICMP 报文，LwIP 均只做识别，而不给予理会。

此外为了快速操作 ICMP 报文的类型与代码字段，LwIP 也定义了两组宏定义对其进行读取和填写操作，具体见*代码清单 12_3*。

代码清单 12-3 操作 ICMP 报文首部的宏定义

```
#define ICMPH_TYPE(hdr) ((hdr)->type)                    /** 读取类型字段␣
↪*/
#define ICMPH_CODE(hdr) ((hdr)->code)                    /** 读取代码字段␣
↪*/
```

(下页继续)

(续上页)

```
#define ICMPH_TYPE_SET(hdr, t) ((hdr)->type = (t))  /** 填写类型字段 */
#define ICMPH_CODE_SET(hdr, c) ((hdr)->code = (c))        /** 填写代码字段 */
```

12.4.2 发送 ICMP 差错报文

无论是何种情况下，在数据报无法向上层递交的时候，协议栈就会向源主机返回一个类型为目的不可达的 ICMP 报文（具体的类型还要看代码字段），如 IP 数据报无法递交到传输层时，就会调用 icmp_dest_unreach() 函数返回一个 ICMP 协议不可达报文；而如果在传输层中，UDP 协议无法向应用层递交数据报，那么也将调用该函数返回一个 ICMP 端口不可达报文。另一种是超时差错报文，在转发数据报的时候如果数据报中 TTL 为 0，或者在分片数据报重装的时候超时，LwIP 将调用 icmp_time_exceeded() 函数会发送一个 ICMP 超时报文到源主机中，下面一起来看看 ICMP 差错报文的发送，具体见*代码清单 12_4*。

代码清单 12-4 发送 ICMP 差错报文

```
void
icmp_dest_unreach(struct pbuf *p, enum icmp_dur_type t) //发送目标不可达报文
{
    MIB2_STATS_INC(mib2.icmpoutdestunreachs);
    icmp_send_response(p, ICMP_DUR, t);
}

void
icmp_time_exceeded(struct pbuf *p, enum icmp_te_type t) //发送 ICMP 超时报文
{
    MIB2_STATS_INC(mib2.icmpouttimeexcds);
    icmp_send_response(p, ICMP_TE, t);
}

static void
icmp_send_response(struct pbuf *p, u8_t type, u8_t code)
```

(下页继续)

(续上页)

```
{
    struct pbuf *q;
    struct ip_hdr *iphdr;
    /* we can use the echo header here */
    struct icmp_echo_hdr *icmphdr;
    ip4_addr_t iphdr_src;
    struct netif *netif;

    /* 申请 pbuf 内存块，大小为 ICMP 首部 + IP 首部 + 8 字节数据 */
    q = pbuf_alloc(PBUF_IP, sizeof(struct icmp_echo_hdr)
                + IP_HLEN + ICMP_DEST_UNREACH_DATASIZE,
                PBUF_RAM);
    if (q == NULL)
    {
        MIB2_STATS_INC(mib2.icmpouterrors);
        return;
    }

    //指向 IP 数据报首部
    iphdr = (struct ip_hdr *)p->payload;

    LWIP_DEBUGF(ICMP_DEBUG, ("icmp_time_exceeded from "));
    ip4_addr_debug_print_val(ICMP_DEBUG, iphdr->src);
    LWIP_DEBUGF(ICMP_DEBUG, (" to "));
    ip4_addr_debug_print_val(ICMP_DEBUG, iphdr->dest);
    LWIP_DEBUGF(ICMP_DEBUG, ("\n"));

    //指针指向 pbuf 数据区域，并且强制将该地址转化为 ICMP 报文首部
    icmphdr = (struct icmp_echo_hdr *)q->payload;
    //填写类型字段
    icmphdr->type = type;
    //填写代码字段
```

(下页继续)

(续上页)

```
    icmphdr->code = code;
    icmphdr->id = 0;
    icmphdr->seqno = 0;

    /* 从原始数据包中复制字段，IP 数据报首部 +8 字节的数据区域 */
    SMEMCPY((u8_t *)q->payload + sizeof(struct icmp_echo_hdr),
            (u8_t *)p->payload,
            IP_HLEN + ICMP_DEST_UNREACH_DATASIZE);

    //得到源 IP 地址
    ip4_addr_copy(iphdr_src, iphdr->src);

    netif = ip4_route(&iphdr_src);

    if (netif != NULL)
    {
        icmphdr->chksum = 0;

        ICMP_STATS_INC(icmp.xmit);

        //发送出去
        ip4_output_if(q, NULL, &iphdr_src, ICMP_TTL, 0,
                      IP_PROTO_ICMP, netif);
    }
    pbuf_free(q);
}
```

发送的时候就单纯为 ICMP 申请 pbuf 空间，大小为 ICMP 报文首部 + IP 数据报首部 + 8 字节，然后直接填写 ICMP 报文首部区域，再拷贝 IP 数据报首部及 IP 数据报数据区域的前 8 字节内容，然后再直接调用发送函数将 ICMP 报文发送出去。

12.4.3 处理 ICMP 报文

LwIP 协议是轻量级 TCP/IP 协议栈，所以对 ICMP 报文中很多类型的报文都不做处理，LwIP 会将这些不处理的报文丢掉，但是对 ICMP 回显请求报文就做出处理，所以这也是为什么我们能 ping 通开发板的原因。从上一章的图 11_8 我们可以看到，当 IP 层收到一个 ICMP 报文的时候，会调用 icmp_input() 函数将报文传递到 ICMP 协议去处理，关于这个函数的处理也是比较简单明了的，只处理 ICMP 请求报文，然后返回一个 ICMP 应答报文，具体看注释即可，icmp_input() 函数具体见*代码清单 12_5*。

代码清单 12-5 icmp_input() 函数

```
void
icmp_input(struct pbuf *p, struct netif *inp)
{
    u8_t type;
    struct icmp_echo_hdr *iecho;
    const struct ip_hdr *iphdr_in;
    u16_t hlen;
    const ip4_addr_t *src;

    ICMP_STATS_INC(icmp.recv);
    MIB2_STATS_INC(mib2.icmpinmsgs);

    iphdr_in = ip4_current_header();
    hlen = IPH_HL_BYTES(iphdr_in);
    if (hlen < IP_HLEN)
    {
        goto lenerr;
    }
    if (p->len < sizeof(u16_t) * 2)
    {
        goto lenerr;
    }
```

(下页继续)

(续上页)

```
type = *((u8_t *)p->payload);

switch (type)
{
case ICMP_ER:
    MIB2_STATS_INC(mib2.icmpinechoreps);
    break;
case ICMP_ECHO:
    MIB2_STATS_INC(mib2.icmpinechos);
    src = ip4_current_dest_addr();

    if (ip4_addr_ismulticast(ip4_current_dest_addr()))
    {
        goto icmperr;
    }
if (ip4_addr_isbroadcast(ip4_current_dest_addr(),
                ip_current_netif()))
    {
        goto icmperr;
    }

    if (p->tot_len < sizeof(struct icmp_echo_hdr))
    {

        goto lenerr;
    }


    /* 调整回显报文请求中的相关字段以生成回显应答报文 */

    //强制将数据区域转换为 ICMP 报文首部
```

(下页继续)

(续上页)

```
    iecho = (struct icmp_echo_hdr *)p->payload;

    if (pbuf_add_header(p, hlen))
    {
        LWIP_DEBUGF(ICMP_DEBUG | LWIP_DBG_LEVEL_SERIOUS,
                    ("Can't move over header in packet"));
    }
    else
    {
        err_t ret;
        struct ip_hdr *iphdr = (struct ip_hdr *)p->payload;
        //拷贝源 IP 地址
        ip4_addr_copy(iphdr->src, *src);
        //拷贝目标 IP 地址
        ip4_addr_copy(iphdr->dest, *ip4_current_src_addr());
        //填写报文类型
        ICMPH_TYPE_SET(iecho, ICMP_ER);

        iecho->chksum = 0;

        //填写生存时间
        IPH_TTL_SET(iphdr, ICMP_TTL);
        IPH_CHKSUM_SET(iphdr, 0);

        ICMP_STATS_INC(icmp.xmit);
        MIB2_STATS_INC(mib2.icmpoutmsgs);
        MIB2_STATS_INC(mib2.icmpoutechoreps);

        /* 发送 ICMP 回显应答报文 */
        ret = ip4_output_if(p, src, LWIP_IP_HDRINCL,
                            ICMP_TTL, 0, IP_PROTO_ICMP, inp);
        if (ret != ERR_OK)
```

(下页继续)

(续上页)

```
            {
                LWIP_DEBUGF(ICMP_DEBUG, ("icmp_input: ip_output_if␣
↪returned an error: %s\n", lwip_strerr(ret)));
            }
        }
        break;
    default:
        //对于其他类型的报文，直接丢掉
        if (type == ICMP_DUR)
        {
            MIB2_STATS_INC(mib2.icmpindestunreachs);
        }
        else if (type == ICMP_TE)
        {
            MIB2_STATS_INC(mib2.icmpintimeexcds);
        }
        else if (type == ICMP_PP)
        {
            MIB2_STATS_INC(mib2.icmpinparmprobs);
        }
        else if (type == ICMP_SQ)
        {
            MIB2_STATS_INC(mib2.icmpinsrcquenchs);
        }
        else if (type == ICMP_RD)
        {
            MIB2_STATS_INC(mib2.icmpinredirects);
        }
        else if (type == ICMP_TS)
        {
            MIB2_STATS_INC(mib2.icmpintimestamps);
        }
```

(下页继续)

(续上页)

```
        else if (type == ICMP_TSR)
        {
            MIB2_STATS_INC(mib2.icmpintimestampreps);
        }
        else if (type == ICMP_AM)
        {
            MIB2_STATS_INC(mib2.icmpinaddrmasks);
        }
        else if (type == ICMP_AMR)
        {
            MIB2_STATS_INC(mib2.icmpinaddrmaskreps);
        }

        ICMP_STATS_INC(icmp.proterr);
        ICMP_STATS_INC(icmp.drop);
    }
    pbuf_free(p);
    return;
lenerr:
    pbuf_free(p);
    ICMP_STATS_INC(icmp.lenerr);
    MIB2_STATS_INC(mib2.icmpinerrors);
    return;
#if LWIP_ICMP_ECHO_CHECK_INPUT_PBUF_LEN || !LWIP_MULTICAST_PING ||
!LWIP_BROADCAST_PING
icmperr:
    pbuf_free(p);
    ICMP_STATS_INC(icmp.err);
    MIB2_STATS_INC(mib2.icmpinerrors);
    return;
#endif
}
```

第 13 章　TCP 协议

本章开始就进入了本书最难的部分，也是最重要的部分，TCP 协议（Transmission Control Protocol，传输控制协议）在 LwIP 协议栈中占据了大半的代码，它是最常用传输层协议，也是最稳定传输层协议，很多上层应用都是依赖于 TCP 协议进程传输数据，如 SMTP、FTP 等等，虽然说学习 TCP 协议是件很枯燥无味的事情，但是大家还是要坚持学下去，必有很大的收获。

13.1 TCP 服务简介

TCP 与 UDP 一样，都是传输层的协议，但是提供的服务却大不相同，UDP 为上层应用提供的是一种不可靠的，无连接的服务，而 TCP 则提供一种面向连接、可靠的字节流传输服务，TCP 让两个主机建立连接的关系，应用数据以数据流的形式进行传输，这与 UDP 协议是不一样：

UDP 运载的数据是以报文的形式，各个报文在网络中互不相干传输，UDP 每收到一个报文就递交给上层应用，因此如果对于大量数据来说，应用层的重装是非常麻烦的，因为 UDP 报文在网络中到达目标主机的顺序是不一样的；

而 TCP 采用数据流的形式传输，先后发出的数据在网络中虽然也是互不相干的传输，但是这些数据本身携带的信息却是紧密联系的，TCP 协议会给每个传输的字节进行编号，当然啦，两个主机方向上的数据编号是彼此独立的，在传输的过程中，发送方把数据的起始编号与长度放在 TCP 报文中，在接收方将所有数据按照编号组装起来，然后返回一个确认，当所有数据接收完成后才将数据递交到应用层中。

13.2 TCP 的特性

13.2.1 连接机制

TCP 是一个面向连接的协议，无论哪一方向另一方发送数据之前，都必须先在双方之间建立一个连接，否则将无法发送数据，一个 TCP 连接必须有双方 IP 地址与端口号，就像打电话一样，必须知道双方的电话号码才会打电话，关于具体的连接我们在后文讲解。

13.2.2 确认与重传

一个完整的 TCP 传输必须有数据的交互，接收方在接收到数据之后必须正面进行确认，向发送方报告接收的结果，而发送方在发送数据之后必须等待接收方的确认，同时发送的时候会启动一个定时器，在指定超时时间内没收到确认，发送方就会认为发送失败，然后进行重发操作，这就是重传报文。

TCP 提供可靠的运输层，但它依赖的是 IP 层的服务，IP 数据报的传输是无连接、不可靠的，因此它要通过确认来知道接收方确实已经收到数据了。但数据和确认都有可能会丢失，因此 TCP 通过在发送时设置一个超时机制（定时器）来解决这种问题，如果当超时时间到达的时候还没有收到对方的确认，它就重传该数据。

13.2.3 缓冲机制

在发送方想要发送数据的时候，由于应用程序的数据大小、类型都是不可预估的，而 TCP 协议提供了缓冲机制来处理这些数据，如在数据量很小的时候，TCP 会将数据存储在一个缓冲空间中，等到数据量足够大的时候在进行发送数据，这样子能提供传输的效率并且减少网络中的通信量，而且在数据发送出去的时候并不会立即删除数据，还是让数据保存在缓冲区中，因为发送出去的数据不一定能被接收方正确接收，它需要等待到接收方的确认再将数据删除。同样的，在接收方也需要有同样的缓冲机制，因为在网络中传输的数据报到达的时间是不一样的，而且 TCP 协议还需要把这些数据报组装成完整的数据，然后再递交到应用层中。

13.2.4 全双工通信

在 TCP 连接建立后，那么两个主机就是对等的，任何一个主机都可以向另一个主机发送数据，数据是双向流通的，所以 TCP 协议是一个全双工的协议，这种机制为 TCP 协议传输数据带来很大的方便，一般来说，TCP 协议的确认是通过捎带的方式来实现，即接收方把确认信息放到反向传来的是数据报文中，不必单独为确认信息申请一个报文，捎带机制减少了网络中的通信流量。由于双方主机是对等的存在，那么任意一方都可以断开连接，此时这个方向上的数据流就断开了，但是另一个方向上的数据仍是连通的状态，这种情况就称之为半双工。

13.2.5 流量控制

在前面讲过，一条 TCP 连接每一侧的主机都设置了缓冲区域。当该接收方收到数据后，它就将数据放入接收缓冲区，当确认这段数据是正常的时候，就会向发送方返回一个确认。并且向相关的应用层递交该数据，但不一定是数据刚一到达就立即递交。事实上，接收方应用也许正忙于其他任务，甚至要过很长时间后才会去处理这些数据。这样子如果接收方处理这些数据时相对缓慢，而发送方发送得太多、太快，就会很容易地使接收方的接收缓冲区发生溢出。

因此 TCP 提供了流量控制服务（flow-control service）以消除发送方使接收方缓冲区溢出的可能性。流量控制是一个速度匹配服务，即发送方的发送速率与接收方应用程序的读取速率相匹配，TCP 通过让发送方维护一个称为接收窗口（receive window）的变量来提供流量控制，是的，你没看错，是接收窗口（rwnd），它用于给发送方一个指示：接收方还能接收多少数据，接收方会将此窗口值放在 TCP 报文的首部中的窗口字段，然后传递给发送方，这个窗口的大小是在发送数据的时候动态调整的。

那可能有人问，这个窗口既然是动态调整的，那有没有可能是 0，这样子发送方不就是没法继续发送数据到接收方了？为了解决这个问题，TCP 协议的规范中有些要求，当接收方主机的接收窗口为 0 时，发送方继续发送只有一个字节的报文段，这些报文段将被接收方接收，直到缓存清空，并在确认报文中包含一个非 0 的接收窗口值。

流量控制是双方通信之间的控制信息，这是很有必要的，比如两个性能不对等的主机，建立了 TCP 协议连接，但是其中一个主机一直发送数据，但是接收的主机来不及处理，这样子的处理就不是最佳的，因此，TCP 协议中使用滑动窗口的流量控制方法，它允许接收方根据自身的处理能力来确定能接收数据的多少，因此会告诉发送方可以发送多少数据过来，即窗口的大小，而发送

方尽可能将数据都多发到对方那里，所以发送方会根据这个窗口的大小发送对应的数据，通俗地来说就是接收方告诉发送方“我还有能力处理那么多的数据，你就发那么多数据给我就行了，不要发多了，否则我处理不了”。

13.2.6 差错控制

除了确认与重传之外，TCP 协议也会采用校验和的方式来检验数据的有效性，主机在接收数据的时候，会将重复的报文丢弃，将乱序的报文重组，发现某段报文丢失了会请求发送方进行重发，因此在 TCP 往上层协议递交的数据是顺序的、无差错的完整数据。

13.2.7 拥塞控制

什么是拥塞？当数据从一个大的管道（如一个快速局域网）向一个较小的管道（如一个较慢的广域网）发送时便会发生拥塞。当多个输入流到达一个路由器，而路由器的输出流小于这些输入流的总和时也会发生拥塞，这种是网络状况的原因。

如果一个主机还是以很大的流量给另一个主机发送数据，但是其中间的路由器通道很小，无法承受这样大的数据流量的时候，就会导致拥塞的发生，这样子就导致了接收方无法在超时时间内完成接收（接收方此时完全有能力处理大量数据），而发送方又进行重传，这样子就导致了链路上的更加拥塞，延迟发送方必须实现一直自适应的机制，在网络中拥塞的情况下调整自身的发送速度，这种形式对发送方的控制被称为拥塞控制（congestion control），与前面我们说的流量控制是非常相似的，而且 TCP 协议采取的措施也非常相似，均是限制发送方的发送速度。

13.3 端口号的概念

TCP 协议的连接是包括上层应用间的连接，简单来说，TCP 连接是两个不同主机的应用连接，而传输层与上层协议是通过端口号进行识别的，如 IP 协议中以 IP 地址作为识别一样，端口号的取值范围是 0~65535，这些端口标识着上层应用的不同线程，一个主机内可能只有一个 IP 地址，但是可能有多个端口号，每个端口号表示不同的应用线程。一台拥有 IP 地址的主机可以提供许多服务，比如 Web 服务、FTP 服务、SMTP 服务等，这些服务完全可以通过 1 个 IP 地址来实现，主

机是怎样区分不同的网络服务呢？显然不能只靠 IP 地址，因为 IP 地址只能识别一台主机而非主机提供的服务，这些服务就是主机上的应用线程，因此是通过“IP 地址 + 端口号”来区分主机不同的线程。

常见的 TCP 协议端口号有 21、53、80 等等，更多端口描述具体见表格 13-1，其中 80 端口号是我们日常生活中最常见的一个端口号，它也是 HTTP 服务器默认开放的端口。

表格 13-1TCP 协议常用的端口号

端口号	协议	说明
20/21	FTP	文件传输协议，使得主机间可以共享文件。
23	Tel-net	终端远程登录，它为用户提供了在本地计算机上完成远程主机工作的能力。
25	SMTP	简单邮件传输协议，它帮助每台计算机在发送或中转信件时找到下一个目的地。
69	TFTP	普通文件传输协议。
80	HTTP	超文本传输协议，通过使用网页浏览器、网络爬虫或者其它的工具，客户端发起一个 HTT P 请求到服务器上指定端口（默认端口为 80 ），应答的服务器上存储着一些资源，比如 H TML 文件和图像，那么就会返回这些数据到客户端。
110	POP3	邮局协议版本 3，本协议主要用于支持使用客户端远程管理在服务器上的电子邮件。

举个例子，我们访问“https://www.baidu.com”这个网站，如果我们在浏览器中输入“https://www.baidu.com:80”，访问这个服务器的 80 端口，我们还是会进入一样的页面，但是如果我们输入“https://www.baidu.com:90”去访问这个服务器的 90 端口，这是不允许访问的，具体见图 13-1。

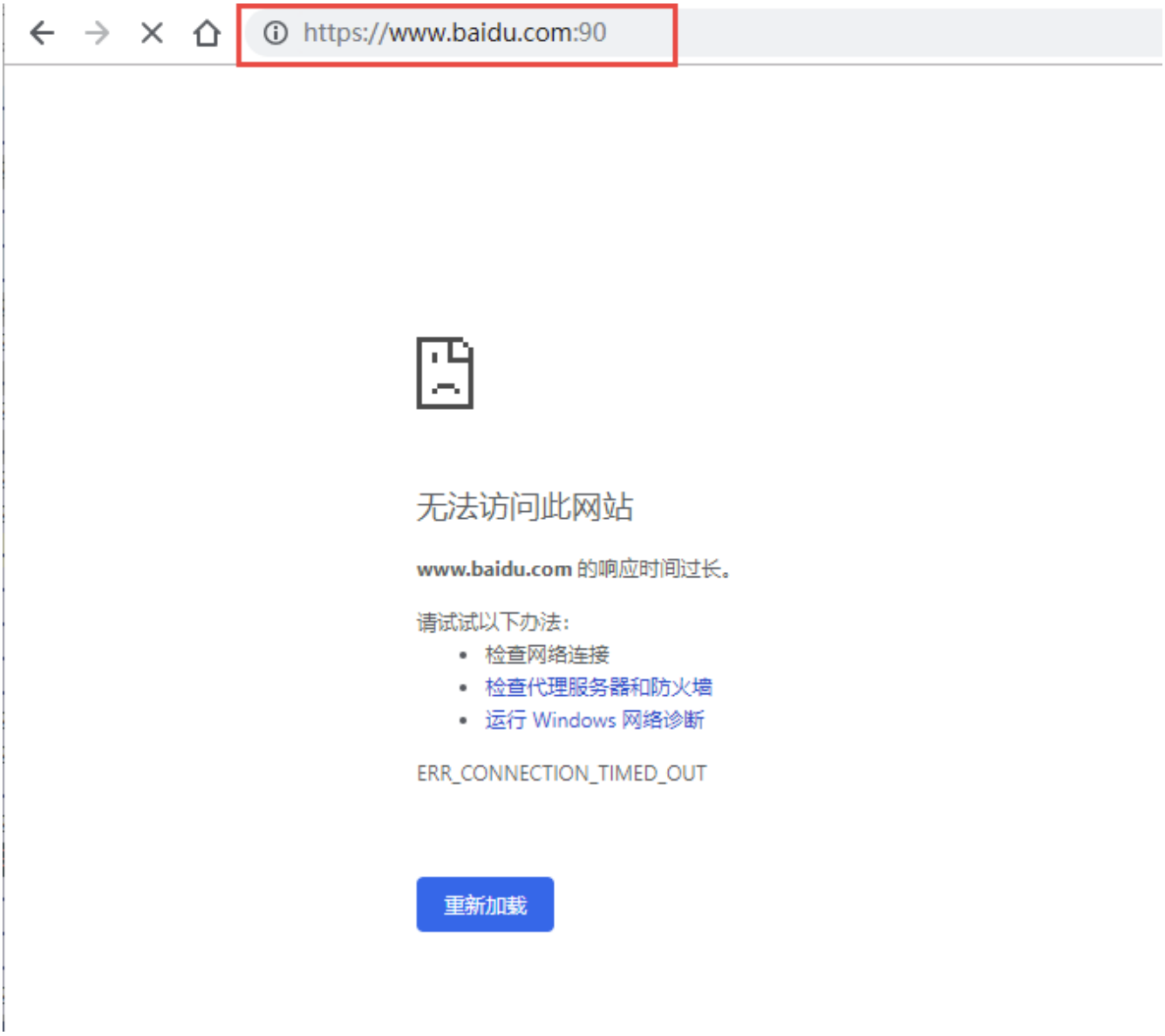


图 13-1 不允许访问 HTTP 服务器其他端口

13.4 TCP 报文段结构

在前面说了那么多，我们就直接来看 TCP 协议的报文是怎么样的，按照协议栈实现的方式，这 TCP 协议也肯定像 ARP 协议、IP 协议一样，都是使用报文进行描述，为了使用更加官方的描述，我们将 TCP 报文（数据包）称为报文段。

13.4.1 TCP 报文段的封装

如 ICMP 报文一样，TCP 报文段依赖 IP 协议进行发送，因此 TCP 报文段与 ICMP 报文一样，都是封装在 IP 数据报中，IP 数据报封装在以太网帧中，因此 TCP 报文段也是经过了两次的封装，然后发送出去，其封装具体见图 *13_2*。

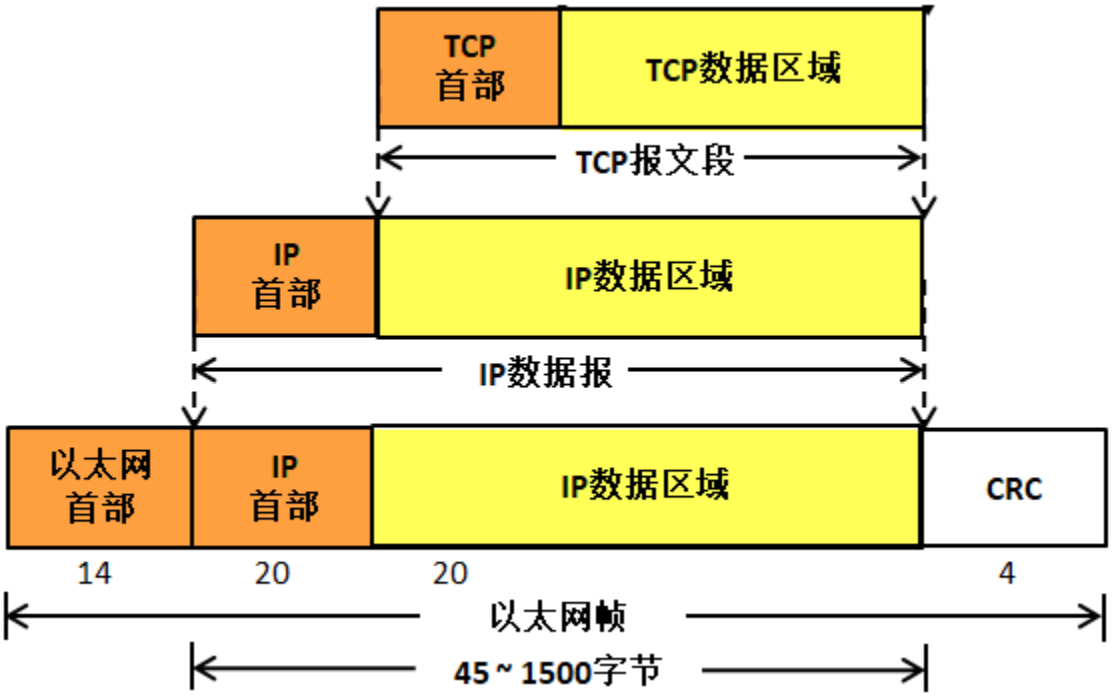


图 13-2TCP 报文段封装

13.4.2 TCP 报文段格式

TCP 报文段如 APR 报文、IP 数据报一样，也是由首部 + 数据区域组成，TCP 报文段的首部我们称之为 TCP 首部，其首部内推很丰富，各个字段都有不一样的含义，如果不计算选项字段，一般来说 TCP 首部只有 20 个字节，具体见图 *13_2*。在 LwIP 中，报文段首部采用一个名字叫 tcp_hdr 的结构体进行描述，具体见*代码清单 13_1*

代码清单 13-1 tcp_hdr 结构体

```
PACK_STRUCT_BEGIN
struct tcp_hdr
{
    PACK_STRUCT_FIELD(u16_t src);            /* 源端口 */
    PACK_STRUCT_FIELD(u16_t dest);           /* 目标端口 */
    PACK_STRUCT_FIELD(u32_t seqno);          /* 序号 */
    PACK_STRUCT_FIELD(u32_t ackno);          /* 确认序号 */
    PACK_STRUCT_FIELD(u16_t _hdrlen_rsvd_flags); /* 首部长度 + 保留位 + 标志
位 */
    PACK_STRUCT_FIELD(u16_t wnd);            /* 窗口大小 */
    PACK_STRUCT_FIELD(u16_t chksum);                 /* 校验和 */
    PACK_STRUCT_FIELD(u16_t urgp);           /* 紧急指针 */
} PACK_STRUCT_STRUCT;
PACK_STRUCT_END
```

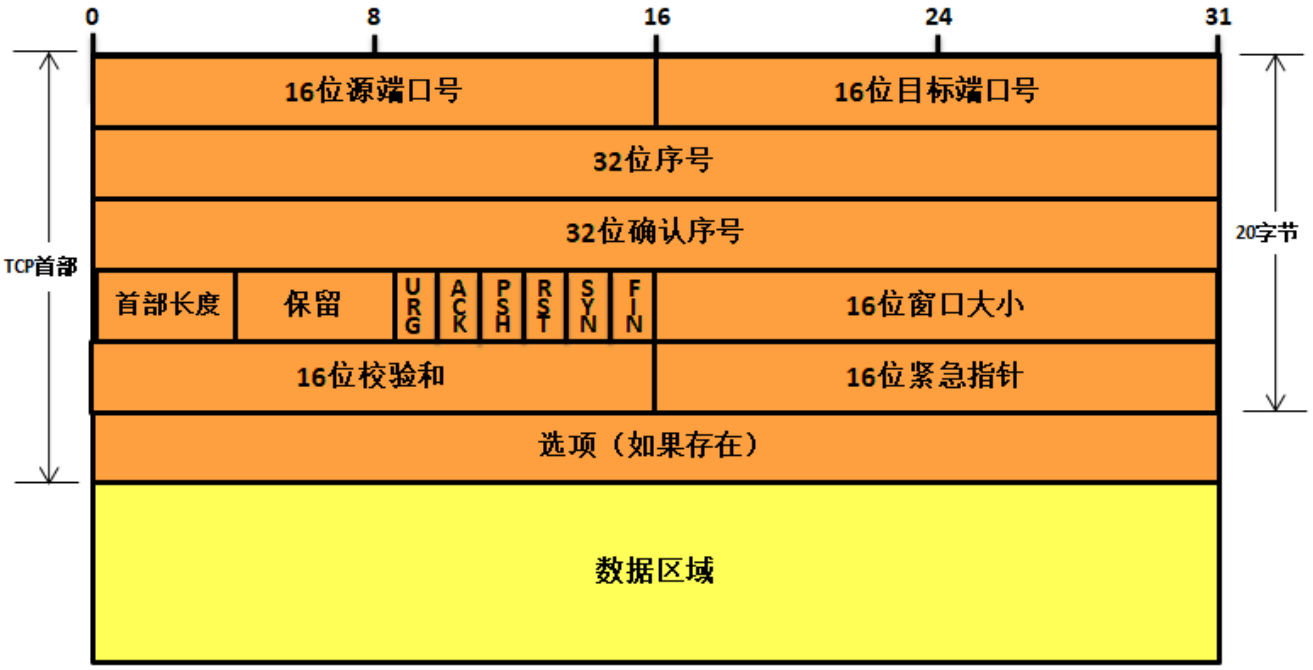


图 13-3TCP 报文段格式

每个 TCP 报文段都包含源主机和目标主机的端口号，用于寻找发送端和接收端应用线程，这两个值加上 I P 首部中的源 I P 地址和目标 I P 地址就能确定唯一一个 TCP 连接。

序号字段用来标识从 TCP 发送端向 TCP 接收端发送的数据字节流，它的值表示在这个报文段中的第一个数据字节所处位置吗，根据接收到的数据区域长度，就能计算出报文最后一个数据所处的序号，因为 TCP 协议会对发送或者接收的数据进行编号（按字节的形式），那么使用序号对每个字节进行计数，就能很轻易管理这些数据。序号是 32 bit 的无符号整数。

当建立一个新的连接时，TCP 报文段首部的 SYN 标志变 1，序号字段包含由这个主机随机选择的初始序号 ISN（Initial Sequence Number）。该主机要发送数据的第一个字节序号为 ISN+1，因为 SYN 标志会占用一个序号，在这里我们只需要了解一下即可，后面会讲解的。

既然 TCP 协议给每个传输的字节都了编号，那么确认序号就包含接收端所期望收到的下一个序号，因此，确认序号应当是上次已成功收到数据的最后一个字节序号加 1。当然，只有 ACK 标志为 1 时确认序号字段才有效，TCP 为应用层提供全双工服务，这意味数据能在两个方向上独立地进行传输，因此确认序号通常会与反向数据（即接收端传输给发送端的数据）封装在同一个报

文中（即捎带），所以连接的每一端都必须保持每个方向上的传输数据序号准确性。

首部长度字段占据 4bit 空间，它指出了 TCP 报文段首部长度，以字节为单位，最大能记录 15*4=60 字节的首部长度，因此，TCP 报文段首部最大长度为 60 字节。在字段后接下来有 6bit 空间是保留未用的。

此外还有 6bit 空间，是 TCP 报文段首部的标志字段，用于标志一些信息：

- URG：首部中的紧急指针字段标志，如果是 1 表示紧急指针字段有效。
- ACK：首部中的确认序号字段标志，如果是 1 表示确认序号字段有效。
- PSH：该字段置一表示接收方应该尽快将这个报文段交给应用层。
- RST：重新建立 TCP 连接。
- SYN：用同步序号发起连接。
- FIN：中止连接。

TCP 的流量控制由连接的每一端通过声明的窗口大小来提供，窗口大小为字节数，起始于确认序号字段指明的值，这个值是接收端正期望接收的数据序号，发送方根据窗口大小调整发送数据，以实现流量控制。窗口大小是一个占据 16 bit 空间的字段，因而窗口最大为 65535 字节，当接收方告诉发送方一个大小为 0 的窗口时，将完全阻止发送方的数据发送。

检验和覆盖了整个的 TCP 报文段：TCP 首部和 TCP 数据区域，由发送端计算和填写，并由接收端进行验证。

只有当 URG 标志置 1 时紧急指针才有效，紧急指针是一个正的偏移量，和序号字段中的值相加表示紧急数据最后一个字节的序号。简单来说，本 TCP 报文段的紧急数据在报文段数据区域中，从序号字段开始，偏移紧急指针的值结束。

选项字段我们暂时就无需理会了。

自此，我们已经把 TCP 报文段的内容讲解完毕，我们可以通过 wireshark 抓包工具进行抓包查看一下 TCP 报文段的数据，具体见图 *13_4*。

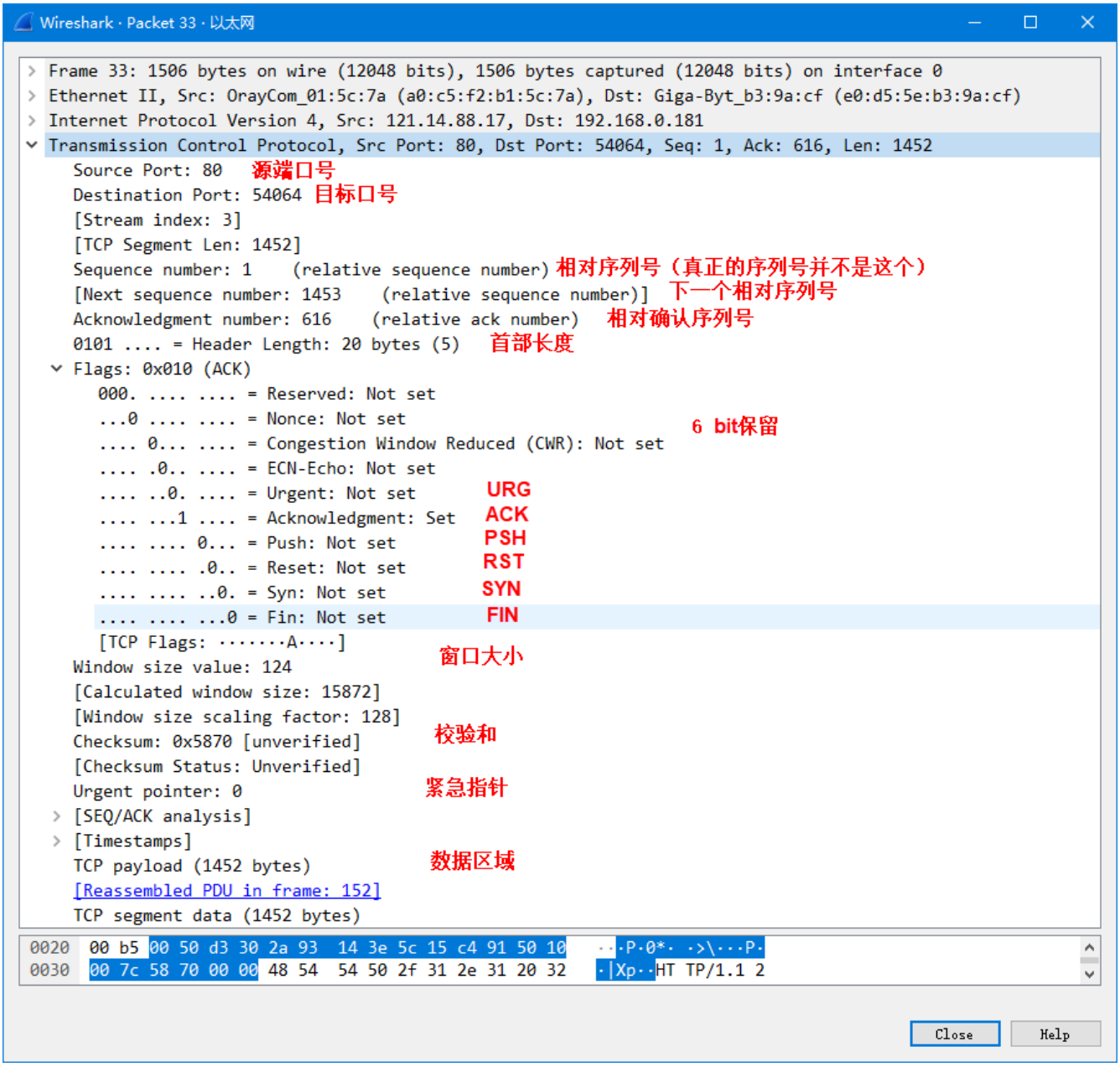


图 13-4wireshark 抓包工具的 TCP 报文段格式

此处有一个要注意的地方：当某个主机开启一个 TCP 会话时，他的初始序列号（ISN）是随机的，可能是 0~4294967295 之间的任意值，然而，像 wireshark 这种抓包工具，通常显示的都是相对序列号与确认号，而不是实际序列号与确认号，相对序列号与确认号是和 TCP 会话的初始序列号相关联的。因为比起真实序列号与确认号，wireshark 工具跟踪更小的相对序列号与确认号会相对

容易一些。

当然啦，我们也可以自己选择看真实的序列号，直接在 wireshark 抓包工具中取消相对序列号配置即可，在 wireshark 菜单栏中的 Edit -> Preferences ->protocols ->TCP，去掉 Relative sequence number 后面勾选框中的 √ 即可，具体见图 *13_5*，取消后的序列号具体见图 *13_6*。

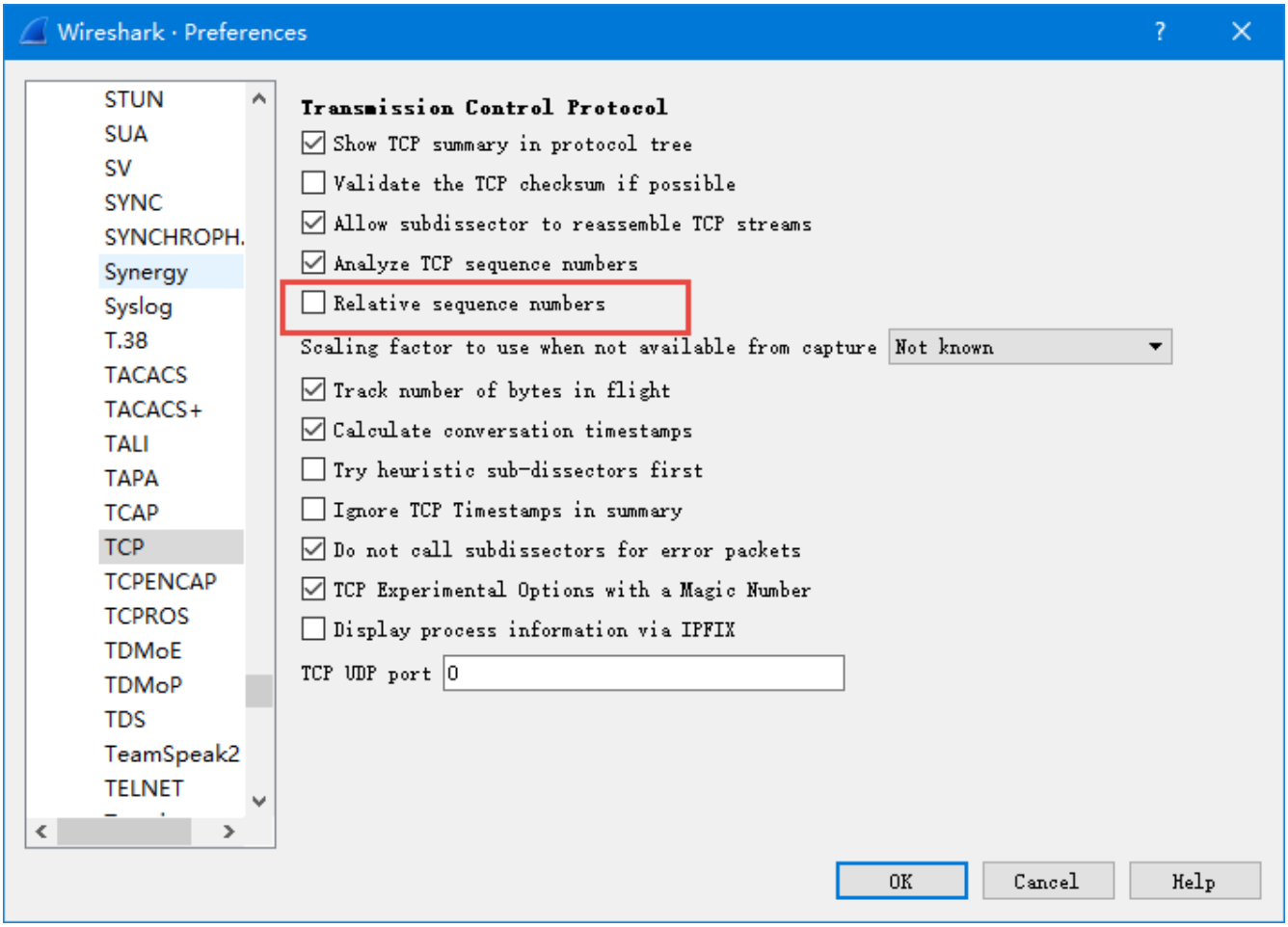


图 13-5 取消相对序列号配置

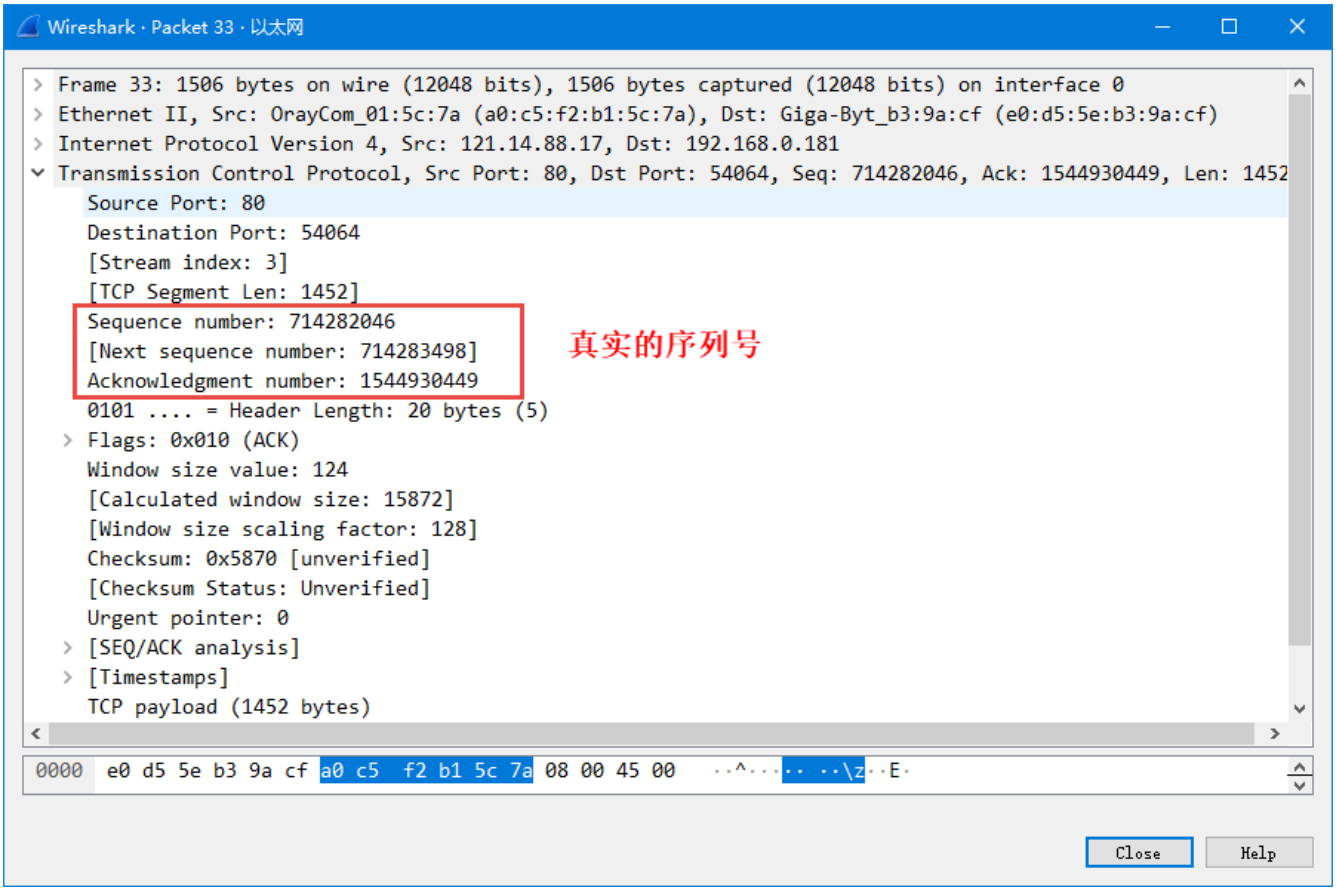


图 13-6 真实序列号

13.5 TCP 连接

TCP 是一个面向连接的协议，无论哪一方向另一方发送数据之前，都必须先在双方之间建立一条连接，俗称“握手”，可能在学习网络之前，大家或多或少都听过“三次握手”、“四次挥手”这两个词汇，那么“三次握手”、“四次挥手”是怎么样的呢？本小节将详细讨论一个 TCP 连接是如何建立的以及通信结束后是如何终止的。

13.5.1 “三次握手”建立连接

首先建立连接的过程是由客户端发起，而服务器无时无刻都在等待着客户端的连接，其示意图具体见图 *13_7*，TCP 连接一般来说会经历以下过程：

第一步：客户端的 TCP 首先向服务器端的 TCP 发送一个特殊的 TCP 报文段。该报文段中不包含应用层数据，但是在报文段的首部中的 SYN 标志位会被置为 1。因此，这个特殊报文段被称为 SYN 报文段（我们暂且称之为握手请求报文）。另外，客户会随机地选择一个初始序号（ISN，假设为 A），并将此序号放置于该 SYN 报文段的序号字段中；但 SYN 报文段中的 ACK 标志位 0，此时它的确认序号段是无效的。该报文段会被封装在一个 IP 数据报中，然后发送给目标服务器。

第二步：一旦服务器收到了客户端发出的 SYN 报文段，知道客户端要请求握手了，服务器便会从 SYN 报文段中提取对应的信息，为该 TCP 连接分配 TCP 缓存和变量，并向该客户 TCP 发送允许连接的报文段（握手应答报文）。这个报文段同样也不包含任何应用层数据，但是，在报文段的首部却包含 3 个重要的信息。

1. SYN 与 ACK 标志都被置为 1。
2. 将 TCP 报文段首部的确认序号字段设置为 A+1（这个 A（ISN）是从握手请求报文中得到）。
3. 服务器随机选择自己的初始序号（ISN，注意此 ISN 是服务器端的 ISN，假设为 B），并将其放置到 TCP 报文段首部的序号字段中。

这个允许连接的报文段实际上表明了：“我收到了你发起建立连接的请求，初始序号为 A，我同意建立该 TCP 连接，我自己的初始序号是 B。”该允许连接的报文段有时被称为 SYN ACK 报文段（SYN ACK segment），同时由于 ACK 标志位 1，所以 TCP 报文段首部的窗口大小字段是有效的。

第三步：当客户端收到服务器的握手应答报文后，会将 ACK 标志置位，此时客户端的 TCP 报文段的 ACK 标志被设置为 1，而对于 SYN 标志，因为连接已经建立了，所以该标志会被置为 0，同时客户端也要给该 TCP 连接分配缓存和变量，并且客户端还需要返回一个应答报文段，这个报文对服务器的应答报文段作出应答，将 TCP 报文段首部的确认序号字段设置为 B+1，同时也会告知服务器的窗口大小。

补充提示：在三次握手的第三个阶段可以在报文段数据区域中携带客户到服务器的数据。

在完成握手后，客户端与服务器就建立了连接，同时双方都得到了彼此的窗口大小，序列号等信

息，在传输 TCP 报文段的时候，每个 TCP 报文段首部的 SYN 标志都会被置 0，因为它只用于发起连接，同步序号。

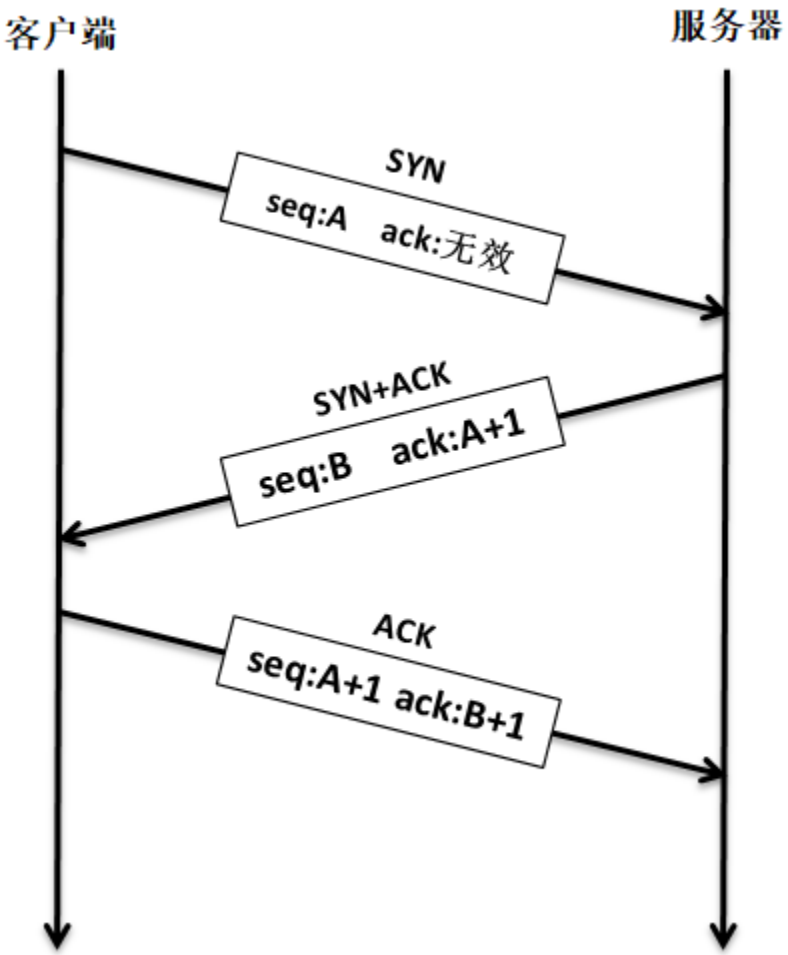


图 13-7 建立连接示意图

我们还可以通过 wireshark 抓包工具可以查看 TCP 协议三次握手的标志位，具体见图 *13_8*。

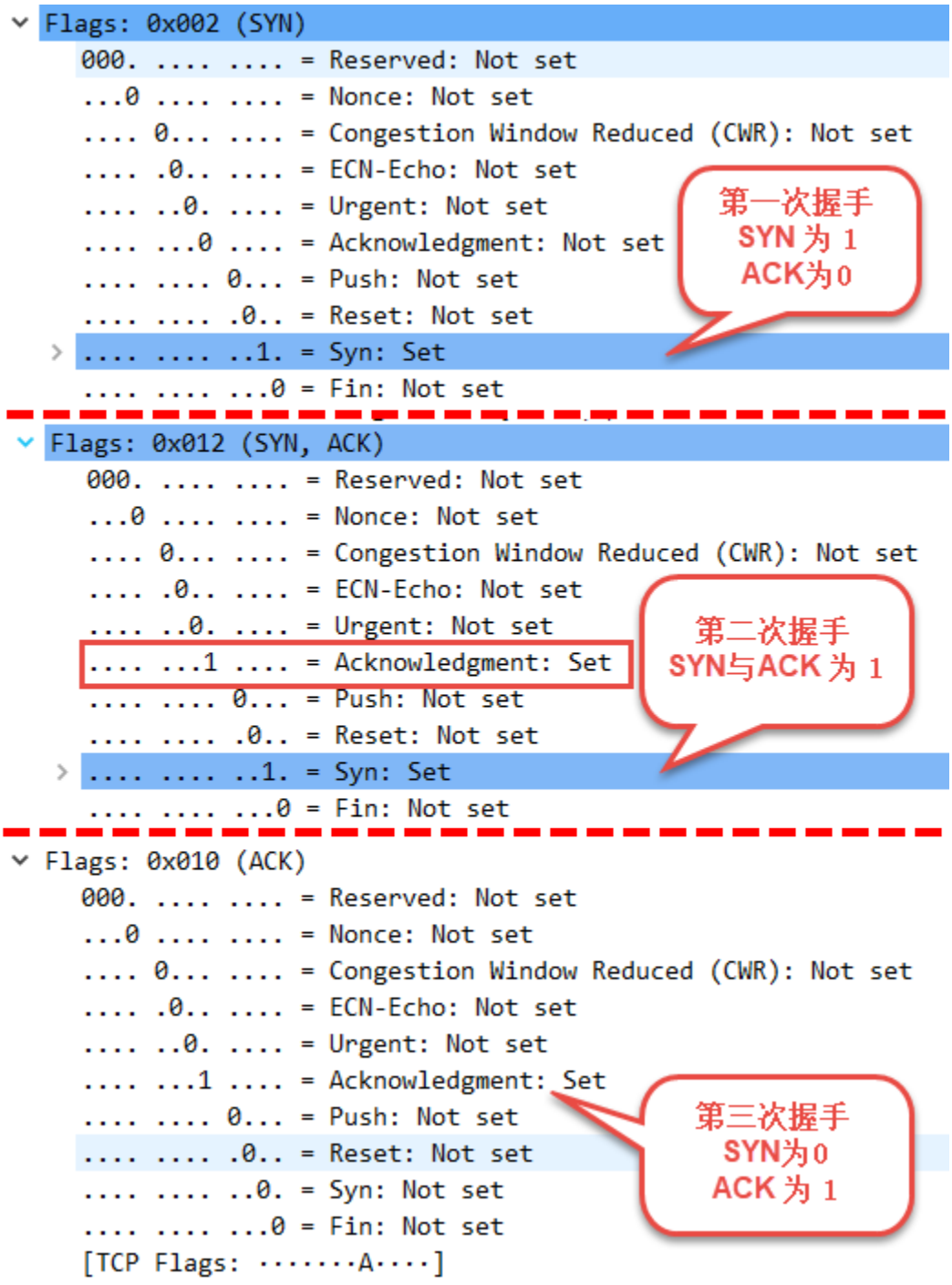


图 13-8 三次握手 TCP 报文段首部的标志

此外，为了更好的理解在整个 TCP 连接期间，TCP 序列号和确认号是如何工作的，我们可以使

用 wireshark 内置的绘制流功能，选择菜单栏中的 Statistics ->Flow Graph，具体见图 13-9。

当然啦，这样子肯定会得到一个比较乱的协议流程界面，因为有太多太多其他协议的内容了，而我们不需要分析这些协议，因此我们需要设置过滤，wireshark 有两种过滤器：

1. 捕捉过滤器（CaptureFilters）：用于决定将什么样的信息记录在捕捉结果中。
2. 显示过滤器（DisplayFilters）：用于在捕捉结果中进行详细查找。

捕捉过滤器在抓抱前进行设置，决定抓取怎样的数据；显示过滤器用于过滤抓包数据，便 stream 的追踪和排查。捕捉过滤器仅支持协议过滤，显示过滤器既支持协议过滤也支持内容过滤，两种过滤器它们支持的过滤语法并不一样。

对于我个人来说，我更喜欢第一种，我们就设置过滤规则，关于规则太多了，我们就不细分设置，我们只简单设置过滤的 IP 地址，比如说访问一个网页（HTTP 依赖 TCP 协议），我们就设置过滤规则为“ip.src ==47.95.47.253 or ip.dst ==47.95.47.253”，表示只抓取 IP 地址为 47.95.47.253 的数据，具体见图 *13_10*。

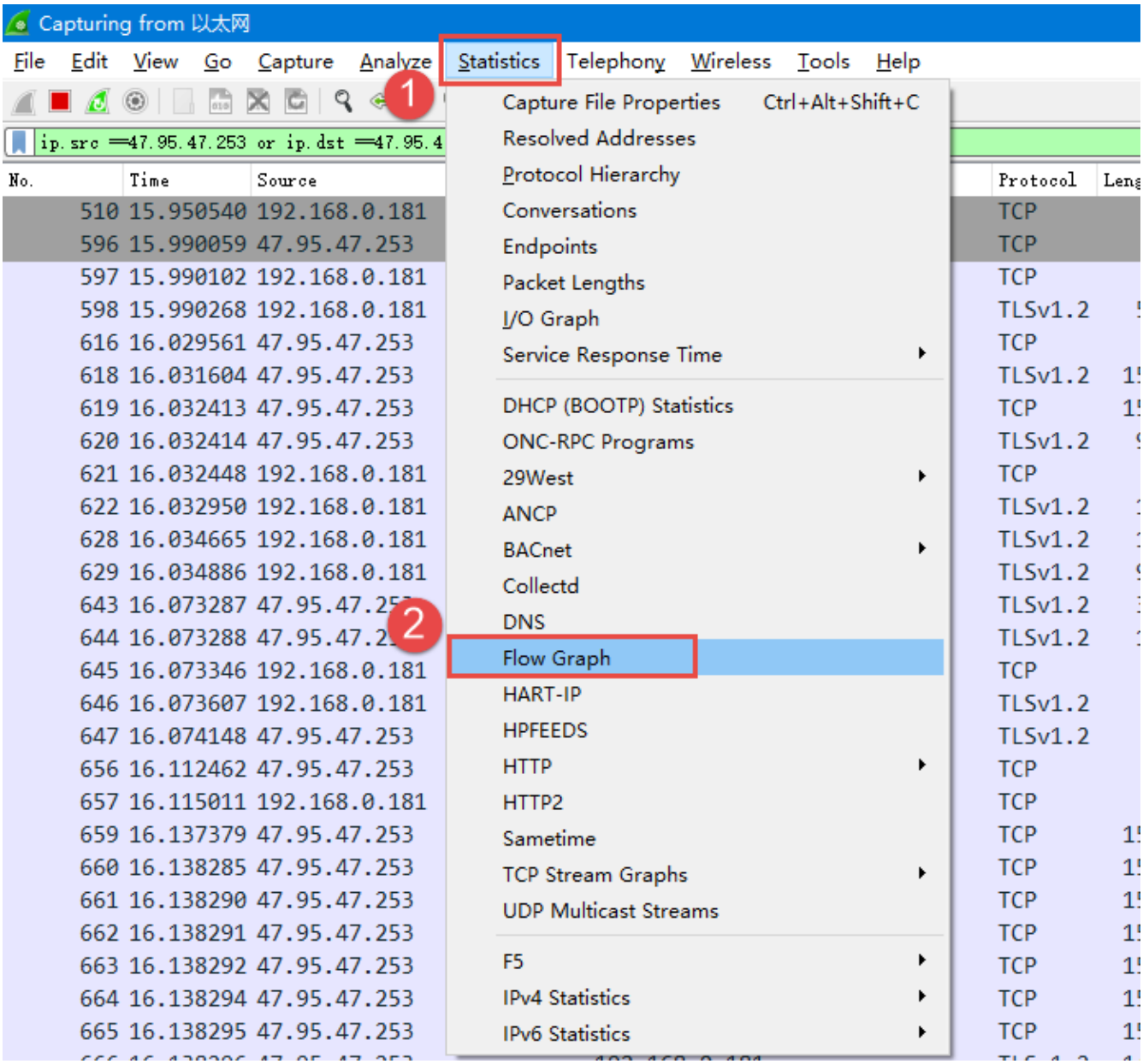


图 13-9wireshark 内置的绘制流功能

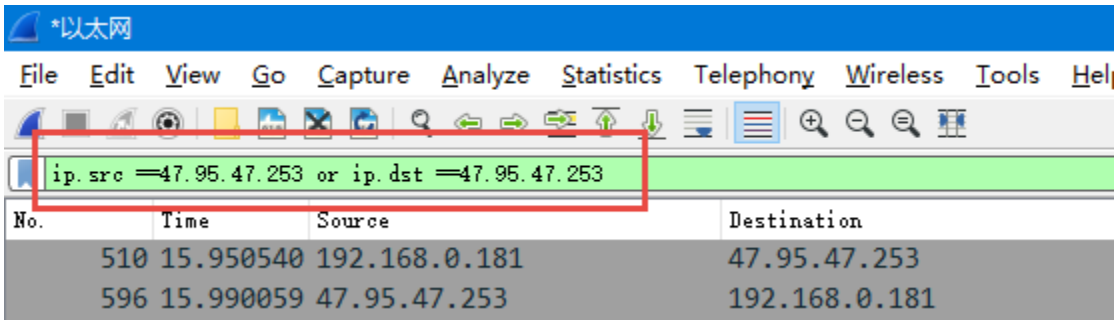


图 13-10 设置过滤规则

然后再次选择菜单栏中的 Statistics ->Flow Graph 绘制流功能，并且在 Flow 窗口吧 Limit to display filter 选项勾选，吧 Flow type 选择 TCP Flows，通过分析，与我们理论的结果是一致的，说明我们得到正确的抓取结果，具体见图 *13_11*。

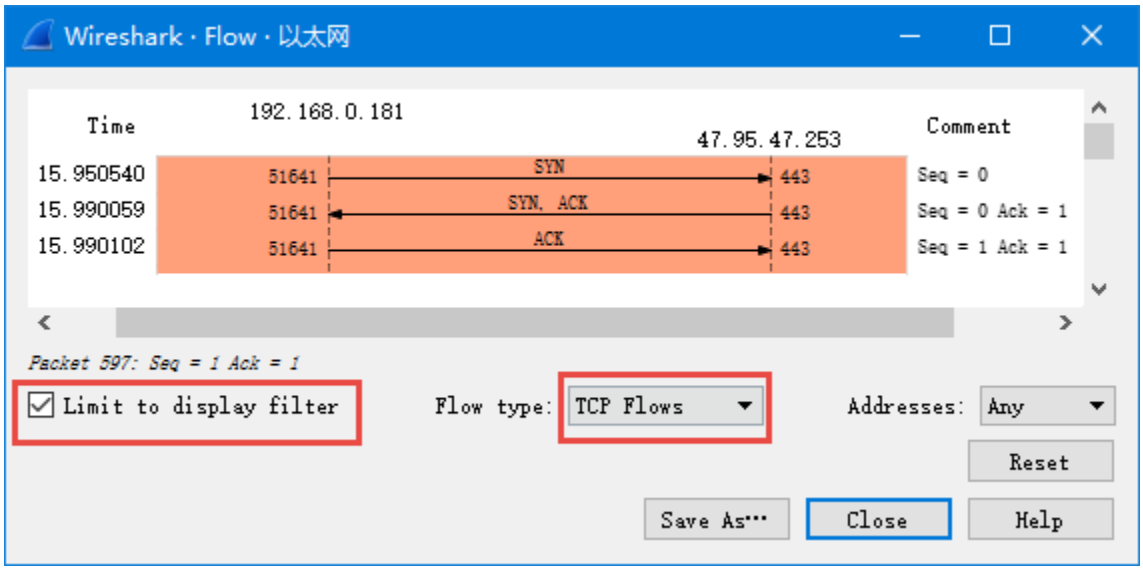


图 13-11 得到正确的 Flow 窗口（使用相对序号）

13.5.2 “四次挥手”终止连接

建立一个连接需要三次握手，而终止一个连接要经过四次挥手（有一些书上也会称为“四次握手”），这由 TCP 的特性造成的，因为 TCP 连接是全双工连接的服务，因此每个方向上的连接必须单独关闭。当一端完成它的数据发送任务后就能发送一个 FIN 报文段（可以称之为终止连接请求，其实就是 FIN 标志位被设置为 1）来终止这个方向上的连接。另一端收到一个 FIN 报文段，

它必须通知应用层对方几经终止了那个方向的连接，发送 FIN 报文段通常是应用层进行关闭的结果。

注意了：客户端发送一个 FIN 报文段只意味着在这一方向上没有数据流动，一个 TCP 连接在发送一个 FIN 后仍能接收数据，但是在实际应用中只有很少的 TCP 应用程序这样做。

“四次挥手”终止连接示意图具体见图 *13_12*，其具体过程如下：

第一步：客户端发出一个 FIN 报文段主动进行关闭连接，此时报文段的 FIN 标志位为 1，假设序号为 C，一般来说 ACK 标志也会被置一，但确认序号字段是无效的。

第二步：当服务器收到这个 FIN 报文段，它发回一个 ACK 报文段（此报文段是终止连接应答），确认序号为收到的序号加 1（C+1），和 SYN 一样，一个 FIN 将占用一个序号，此时断开客户端-> 服务器的方向连接。

第三步：服务器会向应用程序请求关闭与这个客户端的连接，接着服务器就会发送一个 FIN 报文段（这个报文段是服务器向客户端发出，请求终止连接），此时假设序号为 D，ACK 标志虽然也为 1，但是确认序号字段是无效的。

第四步：客户端返回一个 ACK 报文段来确认终止连接的请求，ACK 标志置一，并将确认序号设置为收到序号加 1（D+1），此时断开服务器-> 客户端的方向连接。

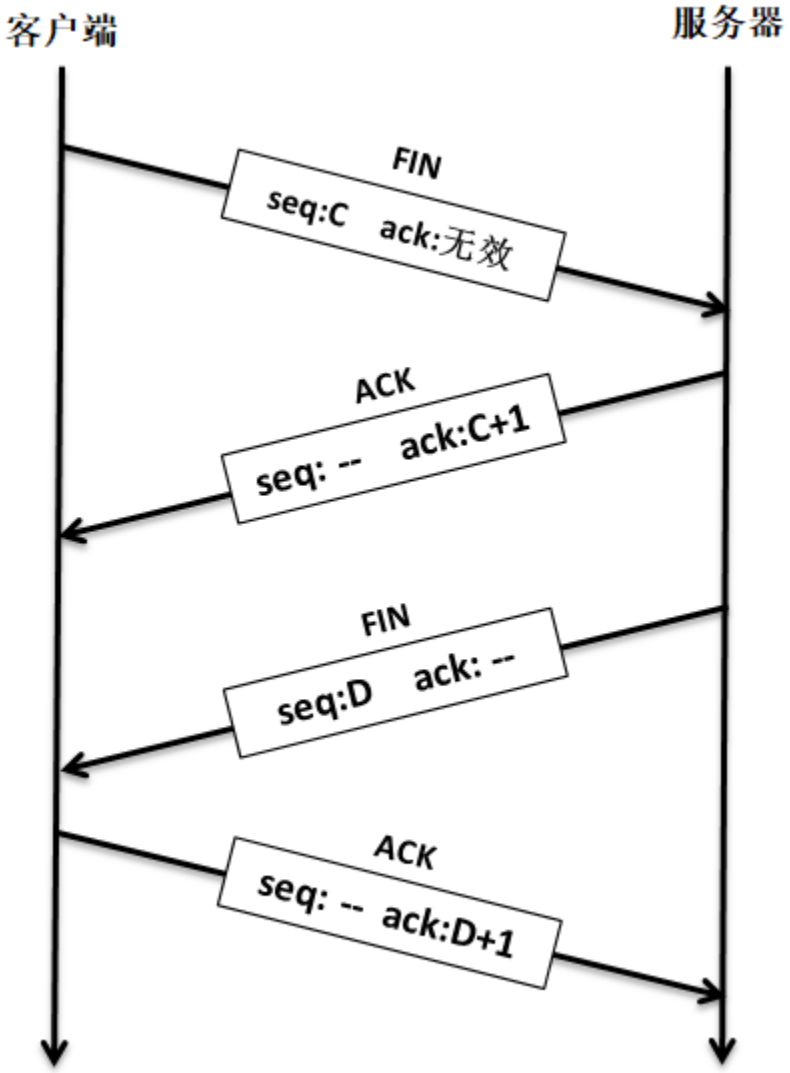


图 13-12 终止连接示意图

同样的，我们也能通过 wireshark 抓包工具抓取到终止连接的数据，具体见图 *13_13*。

296.615808	63996	FIN, ACK →	5001	Seq = 36 Ack = 36
296.616628	63996	← ACK	5001	Seq = 36 Ack = 37
296.616629	63996	← FIN, ACK	5001	Seq = 36 Ack = 37
296.616655	63996	ACK →	5001	Seq = 37 Ack = 37

图 13-13 终止连接 Flow 窗口（使用相对序号）

13.6 TCP 状态

13.6.1 LwIP 中定义的 TCP 状态

TCP 协议根据连接时接收到报文的不同类型，采取相应动作也不同，还要处理各个状态的关系，如当收到握手报文时候、超时的时候、用户主动关闭的时候等都需要不一样的状态去采取不一样的处理。在 LwIP 中，为了实现 TCP 协议的稳定连接，采用数组的形式定义了 11 种连接时候的状态，具体见*代码清单 13_2*。

代码清单 13-2LwIP 中的 TCP 连接状态

```
#if TCP_MSS > 536
#define INITIAL_MSS 536
#else
#define INITIAL_MSS TCP_MSS
#endif

static const char *const tcp_state_str[] =
{
    "CLOSED",      //关闭状态（无连接）
    "LISTEN",      //监听状态
    "SYN_SENT",    //已发起请求连接（等待确认）
    "SYN_RCVD",    //已收到请求连接
    "ESTABLISHED",//稳定连接状态
    "FIN_WAIT_1", //单向请求终止连接状态
    "FIN_WAIT_2", //对方已应答请求终止连接
    "CLOSE_WAIT", //等待终止连接
    "CLOSING",     //两端同时关闭
    "LAST_ACK",    //服务器等待对方接受关闭
    "TIME_WAIT"    //关闭成功（2MSL 等待状态）
};
```

在这里简单讲解几个状态：

1. ESTABLISHED 状态：这个状态是处于稳定连接状态，建立连接的 TCP 协议两端的主机都是处于这个状态，它们相互知道彼此的窗口大小、序列号、最大报文段等信息。

2. FIN_WAIT_1 与 FIN_WAIT_2 状态：处于这个状态一般都是单向请求终止连接，然后主机等待对方的回应，而如果对方产生应答，则主机状态转移为 FIN_WAIT_2，此时 {主机-> 对方} 方向上的 TCP 连接就断开，但是 {对方-> 主机} 方向上的连接还是存在的。此处有一个注意的地方：如果主机处于 FIN_WAIT_2 状态，说明主机已经发出了 FIN 报文段，并且对方也已对它进行确认，除非主机是在实行半关闭状态，否则将等待对方主机的应用层处理关闭连接，因为对方已经意识到它已收到 FIN 报文段，它需要主机发一个 FIN 来关闭 {对方-> 主机} 方向上的连接。只有当另一端的进程完成这个关闭，主机这端才会从 FIN_WAIT_2 状态进入 TIME_WAIT 状态。否则这意味着主机这端可能永远保持这个 FIN_WAIT_2 状态，另一端的主机也将处于 CLOSE_WAIT 状态，并一直保持这个状态直到应用层决定进行关闭。

3. TIME_WAIT 状态：TIME_WAIT 状态也称为 2MSL 等待状态。每个具体 TCP 连接的实现必须选择一个 TCP 报文段最大生存时间 MSL（Maximum Segment Lifetime），如 IP 数据报中的 TTL 字段，表示报文在网络中生存的时间，它是任何报文段被丢弃前在网络内的最长时间，这个时间是有限的，为什么需要等待呢？我们知道 IP 数据报是不可靠的，而 TCP 报文段是封装在 IP 数据报中，TCP 协议必须保证发出的 ACK 报文段是正确被对方接收，因此处于该状态的主机必须在这个状态停留最长时间为 2 倍的 MSL，以防最后这个 ACK 丢失，因为 TCP 协议必须保证数据能准确送达目的地。

13.6.2 TCP 状态转移

讲了那么多理论，下面就结合图文来详解一下 TCP 协议在建立连接与终止连接的时候所有的状态转移处理是怎么样的，具体见图 *13_14*

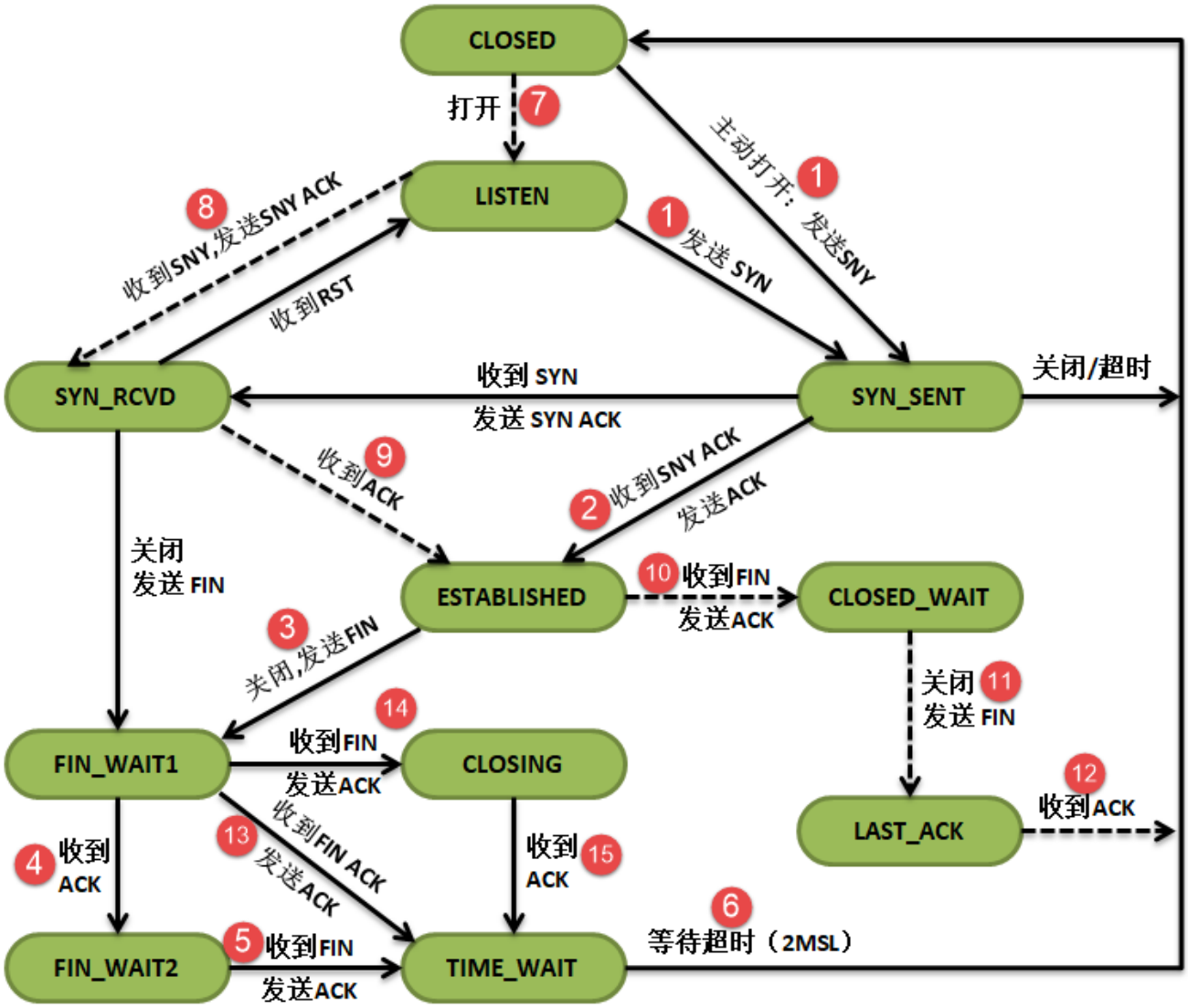


图 13-14TCP 协议状态转移图

对图 13-14 的补充说明（很重要，要牢牢记住）：

- 虚线：表示服务器的状态转移。
- 实线：表示客户端的状态转移。
- 图中所有“关闭”、“打开”都是应用程序主动处理。
- 图中所有的“超时”都是内核超时处理。

13.6.2.1 三次握手过程

图 13-14(**7**)：服务器的应用程序主动使服务器进入监听状态，等待客户端的连接请求。

图 13-14(**1**)：首先客户端的应用程序会主动发起连接，发送 SNY 报文段给服务器，在发送之后就进入 SYN_SENT 状态等待服务器的 SNY ACK 报文段进行确认，如果在指定超时时间内服务器不进行应答确认，那么客户端将关闭连接。

图 13-14(**8**)：处于监听状态的服务器收到客户端的连接请求（SNY 报文段），那么服务器就返回一个 SNY ACK 报文段应答客户端的响应，并且服务器进入 SYN_RCVD 状态。

图 13-14(**1**)：如果客户端收到了服务器的 SNY ACK 报文段，那么就进入 ESTABLISHED 稳定连接状态，并向服务器发送一个 ACK 报文段。

图 13-14(**9**)：同时，服务器收到来自客户端的 ACK 报文段，表示连接成功，进入 ESTABLISHED 稳定连接状态，这正是我们建立连接的三次握手过程。

13.6.2.2 四次挥手过程

图 13-14(**3**)：一般来说，都是客户端主动发送一个 FIN 报文段来终止连接，此时客户端从 ESTABLISHED 稳定连接状态转移为 FIN_WAIT_1 状态，并且等待来自服务器的应答确认。

图 13-14(**10**)：服务器收到 FIN 报文段，知道客户端请求终止连接，那么将返回一个 ACK 报文段到客户端确认终止连接，并且服务器状态由稳定状态转移为 CLOSE_WAIT 等待终止连接状态。

图 13-14(**4**)：客户端收到确认报文段后，进入 FIN_WAIT_2 状态，等待来自服务器的主动请求终止连接，此时 {客户端-> 服务器} 方向上的连接已经断开。

图 13-14(**11**)：一般来说，当客户端终止了连接之后，服务器也会终止 {服务器-> 客户端} 方向上的连接，因此服务器的原因程序会主动关闭该方向上的连接，发送一个 FIN 报文段给客户端。

图 13-14(**5**)：处于 FIN_WAIT_2 的客户端收到 FIN 报文段后，发送一个 ACK 报文段给服务器。

图 13-14(**12**)：服务器收到 ACK 报文段，就直接关闭，此时 {服务器-> 客户端} 方向上的连接已经终止，进入 CLOSED 状态。

图 13-14(**6**)：客户端还会等待 2MSL，以防 ACK 报文段没被服务器收到，这就是四次挥手的全部过程。

注意：对于图 13-14**(13)(14)(15)** 的这些状态都是一些比较特殊的状态，我们暂时就不讲解了，总的来说都是一样的。

13.7 TCP 中的数据结构

与其他协议一样，为了描述 TCP 协议，LwIP 定义了一个名字叫 tcp_pcb 的结构体，我们称之为 TCP 控制块，其内定义了大量的成员变量，基本定义了整个 TCP 协议运作过程的所有需要的东西，如发送窗口、接收窗口、数据缓冲区。超时处理、拥塞控制、滑动窗口等等，具体见代码清单 *13_3*。

代码清单 13-3 TCP 控制块

```
#define IP_PCB                                  \
/* 本地 ip 地址与远端 IP 地址 */                 \
ip_addr_t local_ip;                             \
ip_addr_t remote_ip;                            \
/* 绑定 netif 索引 */                           \
u8_t netif_idx;                                 \
/* 套接字选项 */                                \
u8_t so_options;                                \
/* 服务类型 */                                  \
u8_t tos;                                       \
/* 生存时间 */                                  \
u8_t ttl                                        \
/* 链路层地址解析提示 */                        \
IP_PCB_NETIFHINT

#define TCP_PCB_COMMON(type) \
type *next; /* 指向链表中的下一个控制块 */ \
void *callback_arg; \
TCP_PCB_EXTARGS \
enum tcp_state state; /* TCP 状态 */ \
```

(下页继续)

(续上页)

```
u8_t prio; \
/* 本地主机端口号 */ \
u16_t local_port

/** TCP 协议控制块 */
struct tcp_pcb
{
    IP_PCB;
    /** 协议特定的 PCB 成员 */
    TCP_PCB_COMMON(struct tcp_pcb);

    /* 远端端口号 */
    u16_t remote_port;

    tcpflags_t flags;
#define TF_ACK_DELAY   0x01U   /* 延迟发送 ACK */
#define TF_ACK_NOW     0x02U   /* 立即发送 ACK. */
#define TF_INFR        0x04U   /* 在快速恢复。 */
#define TF_CLOSEPEND   0x08U   /* 关闭挂起 */
#define TF_RXCLOSED    0x10U   /* rx 由 tcp_shutdown 关闭 */
#define TF_FIN         0x20U   /* 连接在本地关闭 */
#define TF_NODELAY     0x40U   /* 禁用 Nagle 算法 */
#define TF_NAGLEMEMERR 0x80U   /* 本地缓冲区溢出 */
#define TF_TIMESTAMP   0x0400U   /* Timestamp option enabled */
#endif
#define TF_RTO         0x0800U /* RTO 计时器 */

    u8_t polltmr, pollinterval;
    /* 控制块被最后一次处理的时间 */
    u8_t last_timer;
    u32_t tmr;
```

(下页继续)

(续上页)

```
    /* 接收窗口相关的字段 */
    u32_t rcv_nxt;   /* 下一个期望收到的序号 */
    tcpwnd_size_t rcv_wnd;   /* 接收窗口大小 */
    tcpwnd_size_t rcv_ann_wnd; /* 告诉对方窗口的大小 */
    u32_t rcv_ann_right_edge; /* 窗口的右边缘 */

    /* 重传计时器。*/
    s16_t rtime;

    u16_t mss;   /* 最大报文段大小 */

    /* RTT (往返时间) 估计变量 */
    u32_t rttest; /* RTT 估计，以为 500 毫秒递增 */
    u32_t rtseq;  /* 用于测试 RTT 的报文段序号 */
    s16_t sa, sv; /* RTT 估计得到的平均值与时间差 */

    s16_t rto;    /* 重传超时 */
    u8_t nrtx;    /* 重传次数 */

    /* 快速重传/恢复 */
    u8_t dupacks;
    u32_t lastack; /* 接收到的最大确认序号 */

    /* 拥塞避免/控制变量 */
    tcpwnd_size_t cwnd;      /* 连接当前的窗口大小 */
    tcpwnd_size_t ssthresh; /* 拥塞避免算法启动的阈值 */

    u32_t rto_end;

    u32_t snd_nxt;   /* 下一个要发送的序号 */
    u32_t snd_wl1, snd_wl2; /* 上一次收到的序号和确认号 */
    u32_t snd_lbb;       /* 要缓冲的下一个字节的序列号 */
```

(下页继续)

(续上页)

```
    tcpwnd_size_t snd_wnd;    /* 发送窗口大小 */
    tcpwnd_size_t snd_wnd_max; /* 对方的最大发送方窗口 */

    /* 可用的缓冲区空间（以字节为单位）。 */
    tcpwnd_size_t snd_buf;

    tcpwnd_size_t bytes_acked;

    struct tcp_seg *unsent;   /* 未发送的报文段 */
    struct tcp_seg *unacked;  /* 已发送但未收到确认的报文段 */
    struct tcp_seg *ooseq;  /* 已收到的无序报文 */
    /* 以前收到但未被上层处理的数据 */
    struct pbuf *refused_data;

#if LWIP_CALLBACK_API || TCP_LISTEN_BACKLOG
    struct tcp_pcb_listen* listener;
#endif

//TCP 协议相关的回调函数
#if LWIP_CALLBACK_API
    /* 当数据发送成功后被调用 */
    tcp_sent_fn sent;
    /* 接收数据完成后被调用 */
    tcp_recv_fn recv;
    /* 建立连接后被调用 */
    tcp_connected_fn connected;
    /* 该函数被内核周期调用 */
    tcp_poll_fn poll;
    /* 发送错误时候被调用 */
    tcp_err_fn errf;
#endif
```

(下页继续)

(续上页)

```
    /* 保持活性 */
    u32_t keep_idle;
    /* 坚持计时器计数器值 */
    u8_t persist_cnt;
    u8_t persist_backoff;
    u8_t persist_probe;

    /* 保持活性报文发送次数 */
    u8_t keep_cnt_sent;

};
```

其实 TCP 控制块我们暂时无需太深入理会，只要知道即可，在后续的使用中知道控制块的内容是干什么的即可。

LwIP 中除了定义了一个完整的 TCP 控制块之外，还定义了一个删减版的 TCP 控制块，叫 tcp_pcb_listen，用于描述处于监听状态的连接，因为分配完整的 TCP 控制块是比较消耗内存资源的，而 TCP 协议在连接之前，是无法进行数据传输的，那么在监听的时候只需要把对方主机的相关信息得到，然后无缝切换到完整的 TCP 控制块中，这样子就能节省不少资源，此外，LwIP 还定义了 4 个链表来维护 TCP 连接时的各种状态，具体见*代码清单 13_4*。

代码清单 13-4 监听 TCP 控制块与链表

```
/** 用于监听的 TCP 协议控制块 */
struct tcp_pcb_listen
{
    /** 所有 PCB 类型的通用成员 */
    IP_PCB;
    /** 协议特定的 PCB 成员 */
    TCP_PCB_COMMON(struct tcp_pcb_listen);
};

/* The TCP PCB lists. */
```

(下页继续)

(续上页)

```
/** 新绑定的端口 */
struct tcp_pcb *tcp_bound_pcbs;
/** 处于监听状态的 TCP 控制块 */
union tcp_listen_pcbs_t tcp_listen_pcbs;
/** 其他状态的 TCP 控制块 */
struct tcp_pcb *tcp_active_pcbs;
/** 处于 TIME_WAIT 状态的控制块 */
struct tcp_pcb *tcp_tw_pcbs;
```

tcp_bound_pcbs 链表上的 TCP 控制块可以看做是处于 CLOSED 状态，那些新绑定的端口初始的时候都是处于这个状态。tcp_listen_pcbs 链表用于记录处于监听状态的 TCP 控制块，一般就是记录的是 tcp_pcb_listen 控制块。tcp_tw_pcbs 链表用于记录连接中处于 TIME_WAIT 状态下的 TCP 控制块。而 tcp_active_pcbs 链表用于记录所有其他状态的 TCP 控制块，这些端口是活跃的，可以不断进行状态转移。

13.8 窗口的概念

相信大家在前面看到很多窗口的东西，但是不知道是怎么一回事，下面就来了解一下窗口是个什么东西，前面也说了，TCP 协议的发送和接收都会给每个字节的数据进行编号，这个编号可以理解为相对序号，如图 *13_15* 所示，就是每个字节的数据的编号。

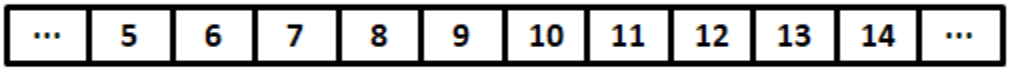


图 13-15 数据编号

13.8.1 接收窗口

TCP 控制块中关于接收窗口的成员变量有 rcv_nxt、rcv_wnd、rcv_ann_wnd、rcv_ann_right_edge，其中这些成员变量的定义在控制块中也讲解了，rcv_nxt 表示下次期望接收到的数据编号，rcv_wnd 表示接收窗口的大小，rcv_ann_wnd 用于告诉发送方窗口的大小，rcv_ann_right_edge 记录了窗口的右边界，这 4 个成员变量都会在数据传输的过程中动态改变的，其示意图具体见图 *13_16*。

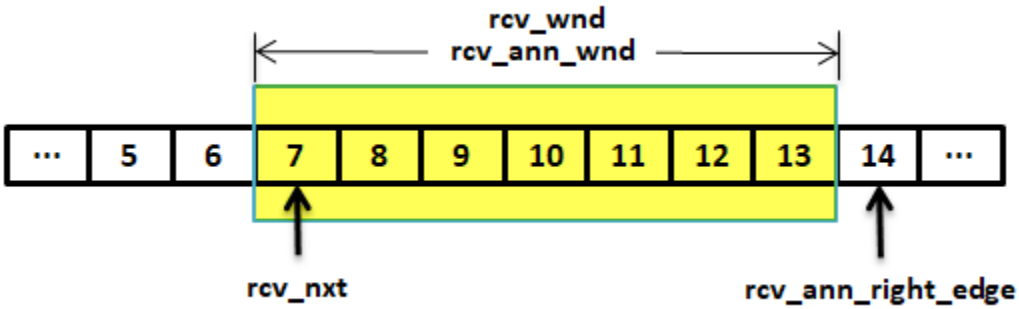


图 13-16 接收窗口示意图

如图 *13_16* 所示，比如在 7 字节之前的数据，都是已经接收确认的数据，而 7 字节正是主机想要接收到的下一个字节数据编号，而窗口的大小是 7，它会告诉发送方“你可以发送 7 个数据过来”，窗口的右边界为 14，当主机下一次接收到 N（不一定是 7）字节数据的时候，窗口就会向右移动 N 个字节，但是 rcv_wnd、rcv_ann_wnd、rcv_ann_right_edge 变量的值是不确定的，通过 LwIP 内部计算得出，而下一次想要接收的数据编号就为 7+N。

13.8.2 发送窗口

TCP 控制块中关于发送窗口的成员变量有 lastack、snd_nxt、snd_lbb、snd_wnd，lastack 记录了已经确认的最大序号，snd_nxt 表示下次要发送的序号，snd_lbb 是表示下一个将被应用线程缓冲的序号，而 snd_wnd 表示发送窗口的大小，是由接收已方提供的。这些值也是动态变化的，当发送的数据收到确认，就会更新 lastack，并且随着数据的发送出去，窗口会向右移动，即 snd_nxt 的值在增加，其示意图具体见图 *13_17*。

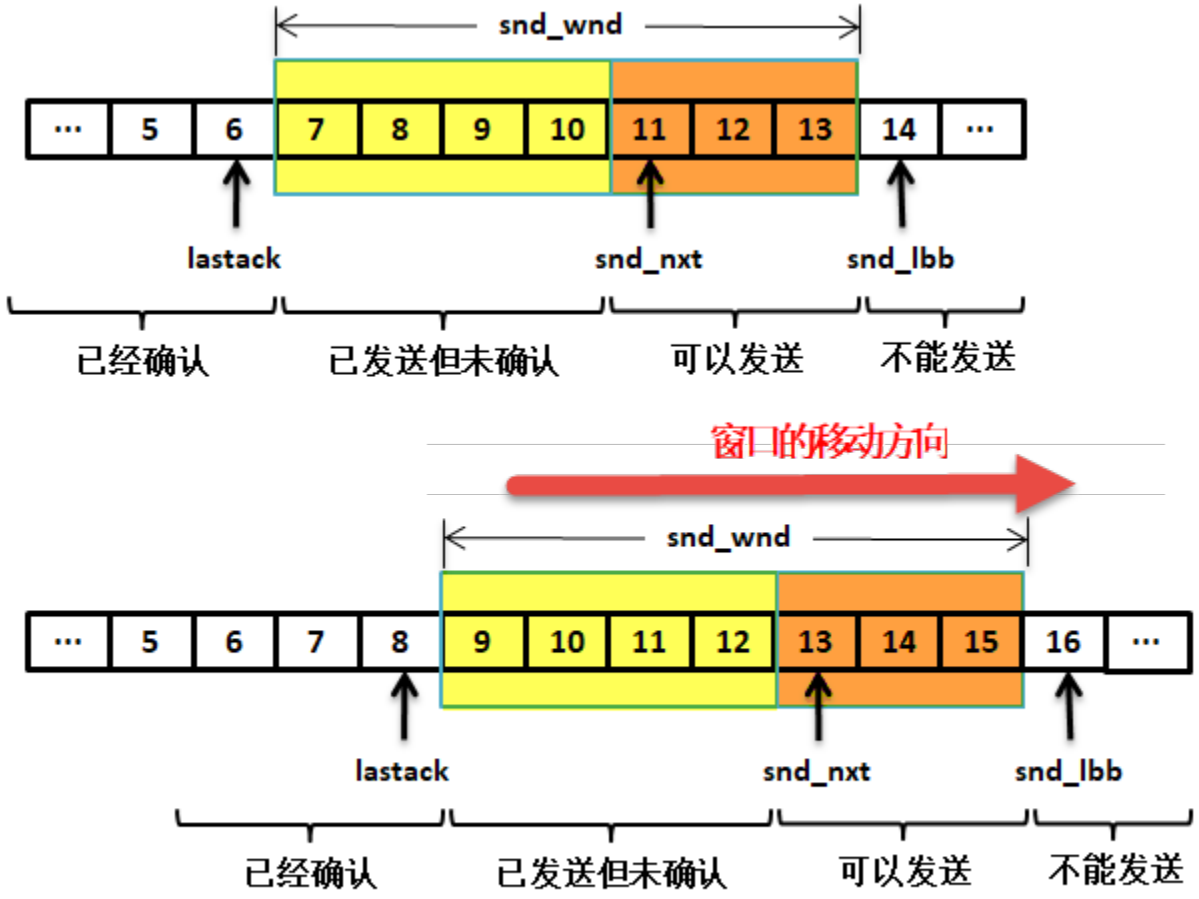


图 13-17 发送窗口

每条 TCP 连接的每一端都必须设有两个窗口——一个发送窗口和一个接收窗口，TCP 的可靠传输机制用字节的序号（编号）进行控制，TCP 所有的确认都是基于数据的序号而不是基于报文段，发送过的数据未收到确认之前必须保留，以便超时重传时使用，发送窗口在没收到确认序号之前是保持不动的，当收到确认序号就会向右移动，并且更新 lastack 的值。

发送缓冲区用来暂时存放应用程序发送给对方的数据，这是主机已发送出但未收到确认的数据。接收缓存用来暂时存放按序到达的、但尚未被接收应用程序读取的数据以及不按序到达的数据。

关于窗口的概念必须强调三点：

1. 发送方的发送窗口并不总是和接收方接收窗口一样大，因为有一定的时间滞后。
2. TCP 标准没有规定对不按序到达的数据应如何处理，通常是先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程。
3. TCP 要求接收方必须有确认的功能，这样可以减小传输开销。

13.9 TCP 报文段处理

13.9.1 报文段缓冲队列

我们知道，TCP 连接的每一端都有接收缓冲区与发送缓冲区（也可以称之为缓冲队列，下文均用缓冲队列），而 TCP 控制块只是维护缓冲区队列的指针，通过指针简单对这些缓冲区进行管理，LwIP 为了更好管理 TCP 报文段的缓冲队列数据，特地定义了一个数据结构，命名为 tcp_seg，使用它将所有的报文段连接起来，这些报文可能是无发送的、可能是已发送但未确认的或者是已经接收到的无序报文，都是需要缓冲在 TCP 控制块内部的，以便识别是哪个连接，而 TCP 控制块，又不可能单独为每个连接开辟那么大的空间，只能使用指针来管理，具体见代码清单 13_5。

代码清单 13-5tcp_seg 结构体

```
struct tcp_seg
{
    struct tcp_seg *next;     /* 指向下一个 tcp_seg */
    struct pbuf *p;           /* 指向报文段的 pbuf */
    u16_t len;                /* 报文段的数据长度 */

    u8_t  flags;              /* 报文段标志属性 */
#define TF_SEG_OPTS_MSS          (u8_t)0x01U
#define TF_SEG_OPTS_TS           (u8_t)0x02U
#define TF_SEG_DATA_CHECKSUMMED (u8_t)0x04U
#define TF_SEG_OPTS_WND_SCALE    (u8_t)0x08U
#define TF_SEG_OPTS_SACK_PERM    (u8_t)0x10U
```

(下页继续)

(续上页)

```
    struct tcp_hdr *tcphdr;   /* 指向报文段首部 */
};
```

每个已经连接的 TCP 控制块中维护了 3 个是指针，分别是 unsent、unacked、ooseq，unsent 指向未发送的报文段缓冲队列，unacked 指向已发送但未收到确认的报文段缓冲队列，ooseq 指向已经收到的无序报文段缓冲队列，当然啦，如果都没有这些报文段，那么这些指针都会指向 NULL，其指向示意图具体见图 *13_18*。

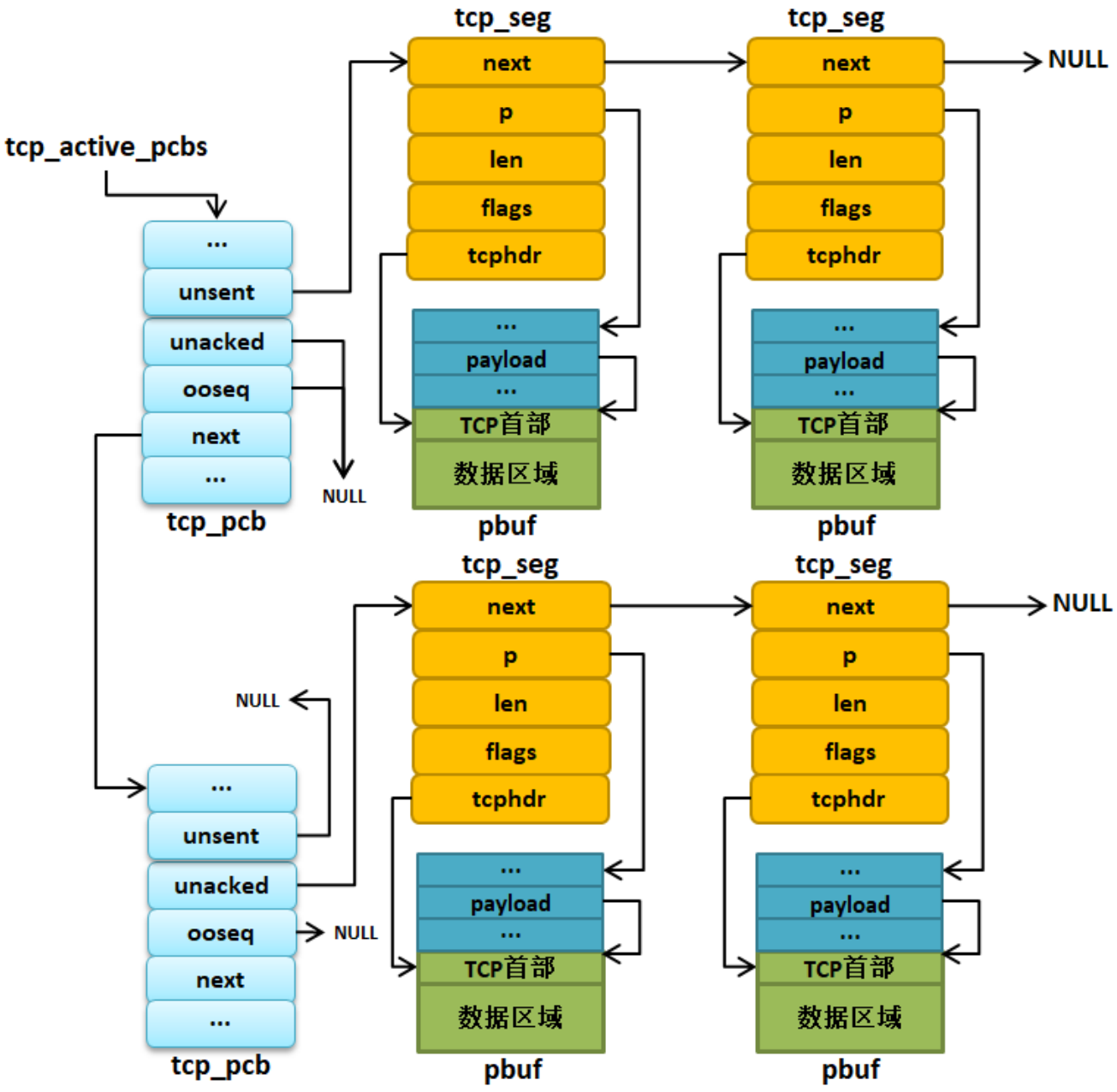


图 13-18TCP 报文段缓冲队列管理示意图

13.9.2 TCP 报文段发送

一般我们在应用层使用 NETCONN API 或者 Socket API 进行编程的时候，会将用户数据传递给传输层，那么本章关于应用层是如何传递数据到传输层的就暂时先不讲解，只需要知道数据到达传输层后是怎么输出的即可，如果我们使用的是 NETCONN API 对已经连接的 TCP 应用发送数据，那么经过内核的一系列处理，就会调用 lwip_netconn_do_writemore() 函数对发送数据，但是真正处理 TCP 报文段缓冲等操作是在 tcp_write() 函数中，在这个函数里，LwIP 会写入数据，但是不会立即发送，也就是存储在缓冲区里面，等待更多的数据进行高效的发送，这也是著名的 Nagle 算法，然后在调用 tcp_output() 函数进行发送出去，这样子一个应用层的数据就通过 TCP 协议传递给 IP 层了，tcp_output() 函数具体见*代码清单 13_6*。

代码清单 13-6 tcp_output() 源码（已删减）

```
err_t
tcp_output(struct tcp_pcb *pcb)
{
    struct tcp_seg *seg, *useg;
    u32_t wnd, snd_nxt;
    err_t err;
    struct netif *netif;

    //如果控制块有数据在处理，直接返回
    if (tcp_input_pcb == pcb)
    {
        return ERR_OK;
    }

    //得到合适的发送窗口
    wnd = LWIP_MIN(pcb->snd_wnd, pcb->cwnd);

    //找到控制块中的未发送数据缓冲区链表
    seg = pcb->unsent;
```

(下页继续)

(续上页)

```
    //根据控制块 IP 地址信息找到合适的网卡发送
    netif = tcp_route(pcb, &pcb->local_ip, &pcb->remote_ip);
    if (netif == NULL)
    {
        return ERR_RTE;
    }

    /* 如果没有本地 IP 地址，我们会从 netif 获得一个 */
    if (ip_addr_isany(&pcb->local_ip))
    {
        const ip_addr_t *local_ip =
            ip_netif_get_local_ip(netif, &pcb->remote_ip);
        if (local_ip == NULL)
        {
            return ERR_RTE;
        }
        ip_addr_copy(pcb->local_ip, *local_ip);
    }

    /* 处理当前不适合窗口的报文段 */
    if (lwip_ntohl(seg->tcphdr->seqno) - pcb->lastack + seg->len > wnd)
    {
        //开始持续定时器
        if (wnd == pcb->snd_wnd && pcb->unacked == NULL &&
                pcb->persist_backoff == 0)
        {
            pcb->persist_cnt = 0;
            pcb->persist_backoff = 1;
            pcb->persist_probe = 0;
        }
        /* 我们需要 ACK，但现在无法发送数据（无法捎带），所以发送一个 ACK 报文段 */
        if (pcb->flags & TF_ACK_NOW)
```

(下页继续)

(续上页)

```
        {
            return tcp_send_empty_ack(pcb);
        }
        goto output_done;
    }
    /* 停止持续计时器 */
    pcb->persist_backoff = 0;

    /* useg 指向未应答队列中的最后一个 tcp_seg 结构 */
    useg = pcb->unacked;
    if (useg != NULL)
    {
        for (; useg->next != NULL; useg = useg->next);
    }
    /* 可用数据和窗口允许它发送报文段，直到把数据全部发送出去或者填满发送窗口 */
    while (seg != NULL &&lwip_ntohl(seg->tcphdr->seqno)
            - pcb->lastack + seg->len <= wnd)
    {
        if ((tcp_do_output_nagle(pcb) == 0) &&
                ((pcb->flags & (TF_NAGLEMEMERR | TF_FIN)) == 0))
        {
            break;
        }

        if (pcb->state != SYN_SENT)
        {
            TCPH_SET_FLAG(seg->tcphdr, TCP_ACK);
        }

        //真正发送 TCP 报文的函数，此处发送 TCP 报文段
        err = tcp_output_segment(seg, pcb, netif);
```

(下页继续)

(续上页)

```
        if (err != ERR_OK)
        {
            tcp_set_flags(pcb, TF_NAGLEMEMERR);
            return err;
        }

        //得到下一个未发送的 tcp_seg
        pcb->unsent = seg->next;
        if (pcb->state != SYN_SENT)
        {
            tcp_clear_flags(pcb, TF_ACK_DELAY | TF_ACK_NOW);
        }
        //计算 snd_nxt 的值
        snd_nxt = lwip_ntohl(seg->tcphdr->seqno) + TCP_TCPLEN(seg);

        //更新下一个要发送的数据编号
        if (TCP_SEQ_LT(pcb->snd_nxt, snd_nxt))
        {
            pcb->snd_nxt = snd_nxt;
        }
        /* 如果发送出去的数据长度>0，则将这些报文段放在未确认链表中 */
        if (TCP_TCPLEN(seg) > 0)
        {
            seg->next = NULL;
            /* 未确认链表为空，插入即可 */
            if (pcb->unacked == NULL)
            {
                pcb->unacked = seg;
                useg = seg;

            }
            //如果不为空，按照顺序插入未确认链表中
```

(下页继续)

(续上页)

```
            else
            {
                if (TCP_SEQ_LT(lwip_ntohl(seg->tcphdr->seqno),
                               lwip_ntohl(useg->tcphdr->seqno)))
                {
                    struct tcp_seg **cur_seg = &(pcb->unacked);
                    while (*cur_seg &&
                            TCP_SEQ_LT(lwip_ntohl((*cur_seg)->tcphdr-
                            >seqno), lwip_ntohl(seg->tcphdr->seqno))){
                        cur_seg = &((*cur_seg)->next );
                    }
                    seg->next = (*cur_seg);
                    (*cur_seg) = seg;
                }
                else
                {
                    useg->next = seg;
                    useg = useg->next;
                }
            }
        }
        else
        {
            tcp_seg_free(seg);
        }
        seg = pcb->unsent;
    }

output_done:
    tcp_clear_flags(pcb, TF_NAGLEMEMERR);
    return ERR_OK;
}
```

总的来说，这个函数的流程还是很简单的，如果控制块的 flags 字段被设置为 TF_ACK_NOW，但是此时还没有数据发送，就只发送一个纯粹的 ACK 报文段，如果能发送数据，那就将 ACK 应答捎带过去，这样子就能减少网络中的流量，同时在发送的时候先找到未发送链表，然后调用 tcp_output_segment()->ip_output_if() 函数进行发送，直到把未发送链表的数据完全发送出去或者直到填满发送窗口，并且更新发送窗口相关字段，同时将这些已发送但是未确认的数据存储在未确认链表中，以防丢失数据进行重发操作，放入未确认链表的时候是按序号升序进行排序的。

13.9.3 TCP 报文段接收

我们在学习 IP 层的时候就知道，IP 数据报中如果是递交给 TCP 协议的数据，就会调用 tcp_input() 函数往上层传递，而 TCP 协议收到数据就会对这些数据进行一系列的处理与验证，因此这个函数是很麻烦的一个函数，源码足足有 476 行，由于太多我们就对其进行删减，删除绝大部分内容，简单看看这个函数的流程，具体见*代码清单 13_7*。

代码清单 13-7tcp_input() 函数（已删减）

```
void
tcp_input(struct pbuf *p, struct netif *inp)
{
    struct tcp_pcb *pcb, *prev;
    struct tcp_pcb_listen *lpcb;

    u8_t hdrlen_bytes;
    err_t err;

    LWIP_UNUSED_ARG(inp);

    PERF_START;

    TCP_STATS_INC(tcp.recv);
    MIB2_STATS_INC(mib2.tcpinsegs);
```

(下页继续)

(续上页)

```
    tcphdr = (struct tcp_hdr *)p->payload;

    /* 检查报文段是否有有效数据 */
    if (p->len < TCP_HLEN)
    {
        /* 如果没有就丢掉报文段 */
        TCP_STATS_INC(tcp.lenerr);
        goto dropped;
    }

    /* 不处理传入的广播/多播报文段。 */
    if (ip_addr_isbroadcast(ip_current_dest_addr(),
                            ip_current_netif()) ||
            ip_addr_ismulticast(ip_current_dest_addr()))
    {
        TCP_STATS_INC(tcp.proterr);
        goto dropped;
    }

    /* 检查 TCP 报文段首部长度 */
    hdrlen_bytes = TCPH_HDRLEN_BYTES(tcphdr);
    if ((hdrlen_bytes < TCP_HLEN) || (hdrlen_bytes > p->tot_len))
    {
        TCP_STATS_INC(tcp.lenerr);
        goto dropped;
    }

    /* 移动 pbuf 指针，指向 TCP 报文段数据区域 */
    tcphdr_optlen = (u16_t)(hdrlen_bytes - TCP_HLEN);
    tcphdr_opt2 = NULL;
    if (p->len >= hdrlen_bytes)
    {
```

(下页继续)

(续上页)

```
        tcphdr_opt1len = tcphdr_optlen;
        pbuf_remove_header(p, hdrlen_bytes);
    }

    /* 将 TCP 首部中的各字段内容提取出来。 */
    tcphdr->src = lwip_ntohs(tcphdr->src);
    tcphdr->dest = lwip_ntohs(tcphdr->dest);
    seqno = tcphdr->seqno = lwip_ntohl(tcphdr->seqno);
    ackno = tcphdr->ackno = lwip_ntohl(tcphdr->ackno);
    tcphdr->wnd = lwip_ntohs(tcphdr->wnd);

    flags = TCPH_FLAGS(tcphdr);
    tcplen = p->tot_len;

    if (flags & (TCP_FIN | TCP_SYN))
    {
        tcplen++;
        if (tcplen < p->tot_len)
        {
            /* u16_t 溢出，无法处理这个 */
            TCP_STATS_INC(tcp.lenerr);
            goto dropped;
        }
    }

    prev = NULL;

    //遍历 tcp_active_pcbs 链表寻找对应的 TCP 控制块
    for (pcb = tcp_active_pcbs; pcb != NULL; pcb = pcb->next)
    {
        /* 检查控制块是否与对应的网卡绑定 */
        if ((pcb->netif_idx != NETIF_NO_INDEX) &&
```

(下页继续)

(续上页)

```
            (pcb->netif_idx !=
            netif_get_index(ip_data.current_input_netif)))
        {
            prev = pcb;
            continue;
        }
        /* ... */
        /* 省略处理 */
        /* ... */

        if (pcb == NULL)
        {
            /* 如果 TCP 控制块没有处于连接状态，就去 tcp_tw_pcbs 链表中找 */
            for (pcb = tcp_tw_pcbs; pcb != NULL; pcb = pcb->next)
            {
                /* 检查控制块是否与对应的网卡绑定 */
                if ((pcb->netif_idx != NETIF_NO_INDEX) &&
                        (pcb->netif_idx != netif_get_index
                        (ip_data.current_input_netif)))
                {
                    continue;
                }

                if (pcb->remote_port == tcphdr->src &&
                        pcb->local_port == tcphdr->dest &&
                ip_addr_cmp(&pcb->remote_ip, ip_current_src_addr()) &&
                ip_addr_cmp(&pcb->local_ip, ip_current_dest_addr()))
                {
                    //找到了就处理它
                    tcp_timewait_input(pcb);

                    pbuf_free(p);
```

(下页继续)

(续上页)

```
                return;
            }
        }

        /* 还是找不到就去 tcp_listen_pcbs 链表中找 */
        prev = NULL;
        for (lpcb = tcp_listen_pcbs.listen_pcbs;
                lpcb != NULL; lpcb = lpcb->next)
        {
            /* 检查控制块是否与对应的网卡绑定 */
    if ((lpcb->netif_idx != NETIF_NO_INDEX) &&
(lpcb->netif_idx != netif_get_index(ip_data.current_input_netif)))
            {
                prev = (struct tcp_pcb *)lpcb;
                continue;
            }
            /* ... */
            /* 省略处理 */
            /* ... */

            //找到了处于监听状态的 TCP 控制块
            if (lpcb != NULL)
            {
                if (prev != NULL)
                {
                ((struct tcp_pcb_listen *)prev)->next = lpcb->next;
                    lpcb->next = tcp_listen_pcbs.listen_pcbs;
                    tcp_listen_pcbs.listen_pcbs = lpcb;
                }
                else
                {
                    TCP_STATS_INC(tcp.cachehit);
```

(下页继续)

(续上页)

```
                    }
                    //处理报文段
                    tcp_listen_input(lpcb);
                    pbuf_free(p);
                    return;
                }
            }

            /* ... */
            /* 省略处理 */
            /* ... */

            tcp_input_pcb = pcb;
            err = tcp_process(pcb);

            /* ... */
            /* 省略处理 */
            /* ... */

}
```

tcp_input() 函数会对传递进来的 IP 数据报进行处理，做一些校验数据报是否正确的操作，查看一下数据报中是否有数据，如果没有就丢掉，看一下是不是多播、广播报文，如果是就不做处理，释放 pbuf。将 TCP 首部中的各字段内容提取出来，首先在 tcp_active_pcbs 链表中寻找对应的 TCP 控制块，找到了就调用 tcp_process() 函数进行处理；如果找不到就去 tcp_tw_pcbs 链表中查找，找到了就调用 tcp_timewait_input() 函数处理它；如果还是找不到就去 tcp_listen_pcbs 链表中找，如果找到就调用 tcp_listen_input() 函数处理，如果找不到的话，就释放 pbu。

此外，还要补充，对于正常接收处理的数据，如果收到的报文段是复位报文或终止连接应答报文，那么就释放 pbuf，终止连接；如果 TCP 协议确认了报文段是新的数据，那么就调用带参宏 TCP_EVENT_SENT（其实是一个 sent 的回调函数）去处理，如果报文段中包含有效的数据，就调用 TCP_EVENT_RECV 去处理，如果是收到 FIN 报文，则调用 TCP_EVENT_CLOSED 去处理它。

因为 TCP 协议的代码量太多了，对于初学者就不用理会太过于深入，我们知道 TCP 层是怎么处理这些数据的即可，整个 TCP 层的工作流程示意图具体见。

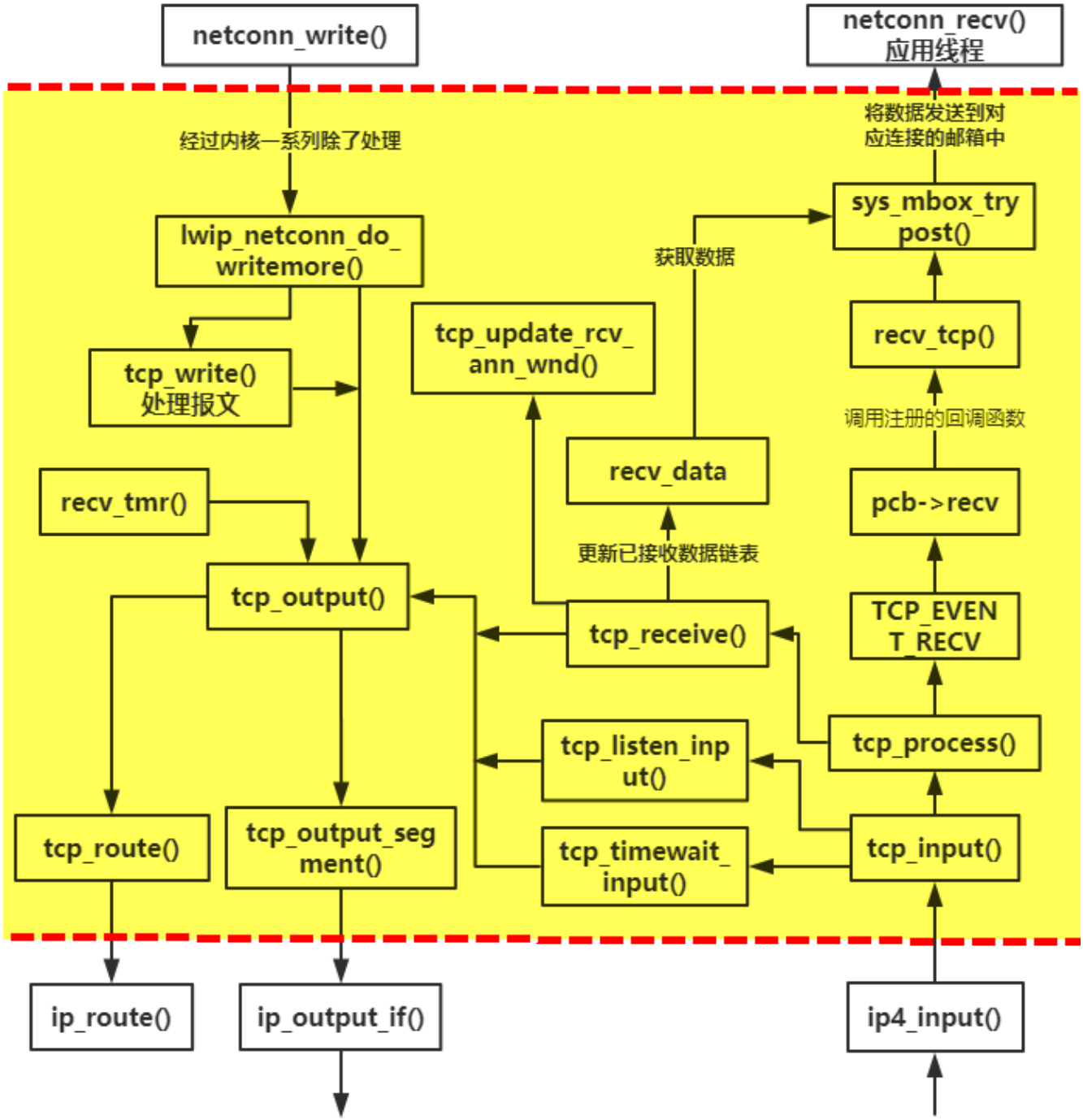


图 13-19 TCP 层的工作流程示意

第 14 章　UDP 协议

UDP 是一个简单的数据报的传输层协议：应用线程的每个输出数据都正好产生一个 UDP 数据报，并组装成一份待发送的 IP 数据报。这与面向数据流的 TCP 协议不同，TCP 协议的应用程序产生的全体数据与真正发送的单个 TCP 报文段可能没有什么联系。

14.1 UDP 协议简介

UDP 是 User Datagram Protocol 的简称，中文名是用户数据报协议，是一种无连接、不可靠的协议，它只是简单地实现从一端主机到另一端主机的数据传输功能，这些数据通过 IP 层发送，在网络中传输，到达目标主机的顺序是无法预知的，因此需要应用程序对这些数据进行排序处理，这就带来了很大的不方便，此外，UDP 协议更没有流量控制、拥塞控制等功能，在发送的一端，UDP 只是把上层应用的数据封装到 UDP 报文中，在差错检测方面，仅仅是对数据进行了简单的校验，然后将其封装到 IP 数据报中发送出去。而在接收端，无论是否收到数据，它都不会产生一个应答发送给源主机，并且如果接收到数据发送校验错误，那么接收端就会丢弃该 UDP 报文，也不会告诉源主机，这样子传输的数据是无法保障其准确性的，如果想要其准确性，那么就需要应用程序来保障了。

UDP 协议的特点：

1. 无连接、不可靠。
2. 尽可能提供交付数据服务，出现差错直接丢弃，无反馈。
3. 面向报文，发送方的 UDP 拿到上层数据直接添加个 UDP 首部，然后进行校验后就递交给 IP 层，而接收的一方在接收到 UDP 报文后简单进行校验，然后直接去除数据递交给上层应用。
4. 支持一对一，一对多，多对一，多对多的交互通信。
5. 速度快，UDP 没有 TCP 的握手、确认、窗口、重传、拥塞控制等机制，UDP 是一个无状态的传输协议，所以它在传递数据时非常快，即使在网络拥塞的时候 UDP 也不会降低发送的

数据。

UDP 虽然有很多缺点，但是也不排除其能用于很多场合，因为在如今的网络环境下，UDP 协议传输出现错误的概率是很小的，并且它的实时性是非常好，常用于实时视频的传输，比如直播、网络电话等，因为即使是出现了数据丢失的情况，导致视频卡帧，这也不是什么大不了的事情，所以，UDP 协议还是会被应用与对传输速度有要求，并且可以容忍出现差错的数据传输中。

14.2 UDP 常用端口号

与 TCP 协议一样，UDP 报文协议根据对应的端口号传递到目标主机的应用线程，同样的，传输层到应用层的唯一标识是通过端口号决定的，两个线程之间进行通信必须用端口号进行识别，同样的使用“IP 地址 + 端口号”来区分主机不同的线程。

常见的 UDP 协议端口号有

端口号	协议	说明
53	DNS	域名服务器，因特网上作为域名和 IP 地址相互映射的一个分布式数据库，能够使用户更方便的访问互联网，而不用去记住能够被机器直接读取的 IP 数串。
69	TFTP	小型文件传输协议
123	NTP	网络时间协议，它是用来同步网络中各个计算机时间的协议。
161	SNMP	简单网络管理协议

14.3 UDP 报文

UDP 报文也被称为用户数据报，与 TCP 协议一样，由报文首部与数据区域组成。在 UDP 协议中，它只是简单将应用层的数据进行封装（添加一个 UDP 报文首部），然后传递到 IP 层，再通过网卡发送出去，因此，UDP 数据也是经过两次封装，具体见图 *14_1*。

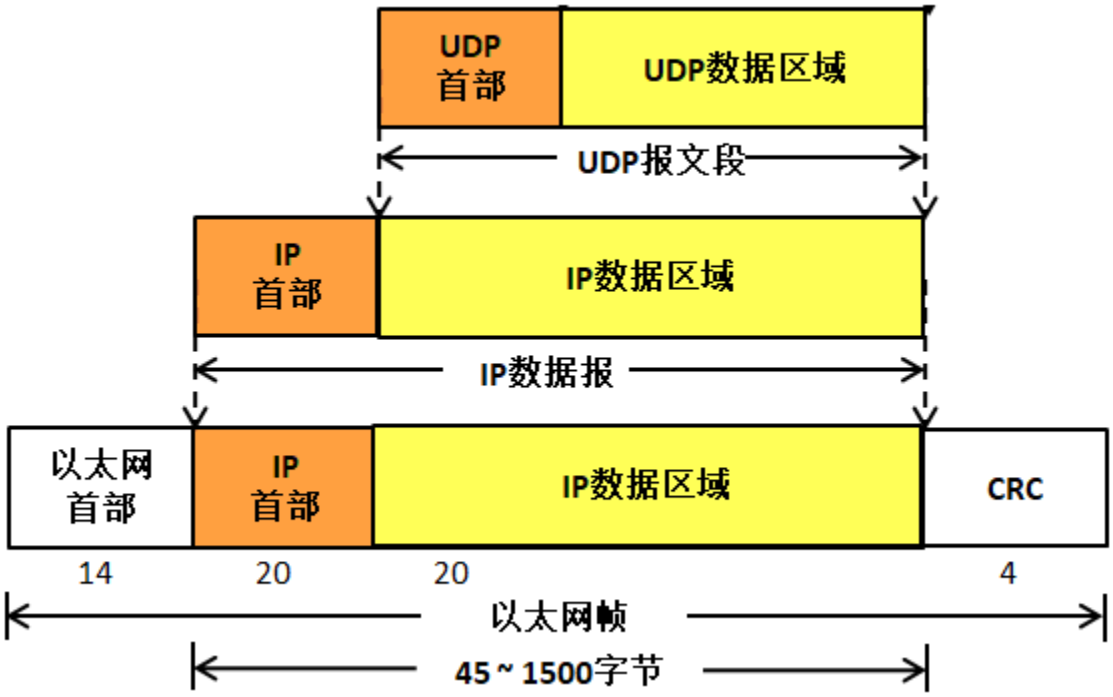


图 14-1UDP 报文封装

UDP 报文结构示意图具体见图 14-2。

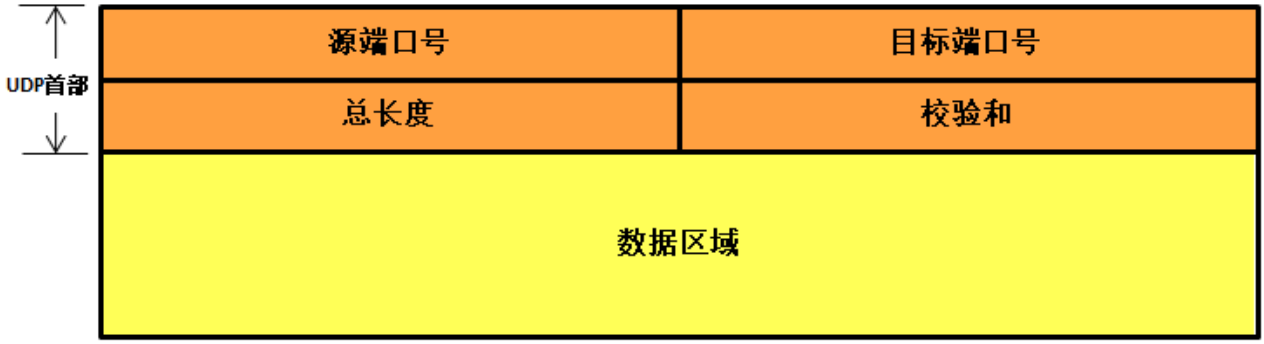


图 14-2UDP 报文结构

关于源端口号、目标端口号与校验和字段的作用与 TCP 报文段一样，端口号的取值在 0~65535 之间；16bit 的总长度用于记录 UDP 报文的总长度，包括 8 字节的首部长度与数据区域。

14.4 UDP 报文的数据结构

14.4.1 UDP 报文首部结构体

LwIP 定义了一个 UDP 报文首部数据结构，名字叫 udp_hdr，是一个结构体，它定义了 UDP 报文首部的各个字段，具体见*代码清单 14_1*。

代码清单 14-1 udp_hdr 结构体

```
PACK_STRUCT_BEGIN
struct udp_hdr
{
    PACK_STRUCT_FIELD(u16_t src);
    PACK_STRUCT_FIELD(u16_t dest);  /*src/dest UDP ports */
    PACK_STRUCT_FIELD(u16_t len);
    PACK_STRUCT_FIELD(u16_t chksum);
} PACK_STRUCT_STRUCT;
PACK_STRUCT_END
```

14.4.2 UDP 控制块

与 TCP 协议一样，为了更好管理 UDP 报文，LwIP 定义了一个 UDP 控制块，记录与 UDP 通信的所有信息，如源端口号、目标端口号、源 IP 地址、目标 IP 地址以及收到数据时候的回调函数等等，系统会为每一个基于 UDP 协议的应用线程创建一个 UDP 控制块，并且将其与对应的端口绑定，这样子就能进行 UDP 通信了。与 TCP 协议一样，LwIP 会把多个这样子的 UDP 控制块用一个链表连接起来，在处理的时候遍历列表，然后对控制块进行操作，具体见*代码清单 14_2*。

代码清单 14-2UDP 控制块

```
#define IP_PCB                                  \
/* 本地 ip 地址与远端 IP 地址 */                 \
ip_addr_t local_ip;                            \
```

(下页继续)

(续上页)

```
ip_addr_t remote_ip;                    \
/* 网卡 id */                            \
u8_t netif_idx;                         \
/* Socket 选项 */                       \
u8_t so_options;                        \
/* 服务类型    */                      \
u8_t tos;                               \
/* 生存时间 */                          \
u8_t ttl                                \
IP_PCB_NETIFHINT

/** UDP 控制块 */
struct udp_pcb
{

    IP_PCB;

    //指向下一个控制块
    struct udp_pcb *next;

    //控制块状态
    u8_t flags;

    /** 本地端口号与远端端口号 */
    u16_t local_port, remote_port;

    /** 接收回调函数 */
    udp_recv_fn recv;
    /** 回调函数参数 */
    void *recv_arg;
};
```

UDP 控制块会使用 IP 层的一个宏定义 IP_PCB，里面包括 IP 层需要使用的信息，如本地 IP 地址与目标 IP 地址（或者称为远端 IP 地址），服务类型、网卡、生存时间等，此外 UDP 控制块还要本地端口号与目标（远端）端口号，这两个字段很重要，UDP 协议就是根据这些端口号识别应用线程，当 UDP 收到一个报文的时候，会遍历链表上的所有控制块，根据报文的目标端口号找到与本地端口号相匹配的 UDP 控制块，然后递交数据到上层应用，而如果找不到对应的端口号，那么就会返回一个端口不可达 ICMP 差错控制报文。

除此之外 LwIP 会为我们注册一个接收数据回调函数，当然啦，如果我们使用 RAW API 编程，这个回调函数就需要我们自己实现，在 LwIP 接收到一个给本地的数据时候，就会调用这个回调函数，而 recv 字段就是指向这个回调函数的，其函数原型具体见*代码清单 14_3*。

代码清单 14-3 udp_recv_fn 函数原型

```
typedef void (*udp_recv_fn)(void *arg,
                            struct udp_pcb *pcb,
                            struct pbuf *p,
                            const ip_addr_t *addr,
                            u16_t port);
```

一般来说，我们使用 NETCONN API 或者是 Socket API 编程，是不需要我们自己去注册回调函数 recv_udp()，因为这个函数 LwIP 内核会自动给我们注册，具体见*代码清单 14_4*。

代码清单 14-4 注册接收回调函数

```
void
udp_recv(struct udp_pcb *pcb,
         udp_recv_fn recv,
         void *recv_arg)
{
    LWIP_ASSERT_CORE_LOCKED();


    /* 注册回调函数 */
    pcb->recv = recv;
```

(下页继续)

(续上页)

```
    pcb->recv_arg = recv_arg;
}

udp_recv(msg->conn->pcb.udp,
         recv_udp,
         msg->conn);
```

LwIP 中定义了一个名字为 udp_pcbs 的 UDP 控制块链表，记录主机中所有的 UDP 控制块，每个 UDP 协议的应用线程都能受到内核的处理，UDP 控制块链表将 UDP 控制块连接起来，在收到数据需要处理的时候，内核变量链表，查找 UDP 控制块的信息，从而调用对应的回调函数，当然，我们不使用 RAW API 编程的时候，回调函数只有一个，UDP 控制块链表示意图具体见图 *14_3*。

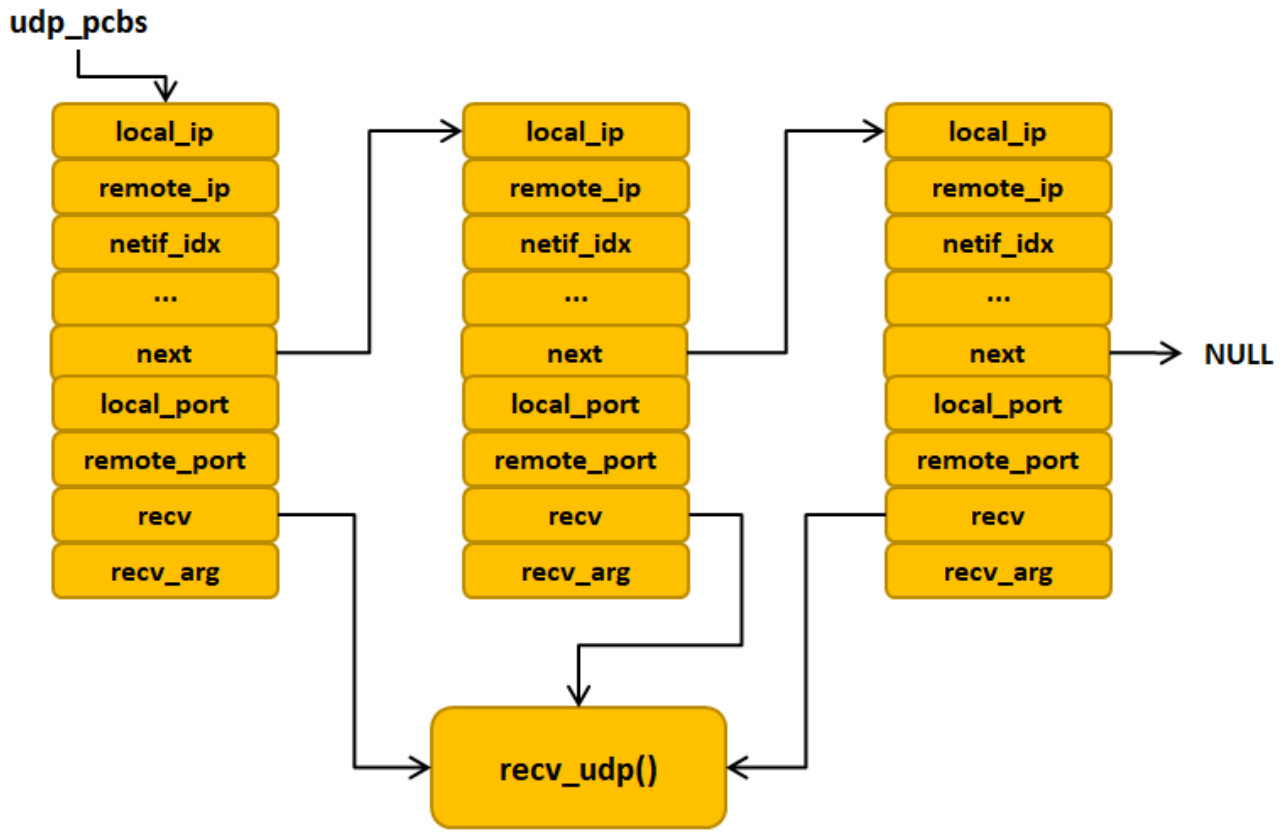


图 14-3UDP 控制块链表

14.5 UDP 报文发送

UDP 协议是传输层，所以需要从上层应用线程中得到数据，我们使用 NETCONN API 或者是 Socket API 编程，那么传输的数据经过内核的层层处理，最后调用 udp_sendto_if_src() 函数进行发送 UDP 报文，具体见*代码清单 14_5*。

代码清单 14-5 udp_sendto_if_src() 源码（已删减）

```
err_t
udp_sendto_if_src(struct udp_pcb *pcb, struct pbuf *p,
                  const ip_addr_t *dst_ip, u16_t dst_port,
                  struct netif *netif, const ip_addr_t *src_ip)
{
    struct udp_hdr *udphdr;
    err_t err;
    struct pbuf *q; /* q will be sent down the stack */
    u8_t ip_proto;
    u8_t ttl;

    /* 如果 UDP 控制块尚未绑定到端口，请将其绑定到这里 */
    if (pcb->local_port == 0)
    {
        err = udp_bind(pcb, &pcb->local_ip, pcb->local_port);
        if (err != ERR_OK)
        {
            return err;
        }
    }

    /* 数据包太大，无法添加 UDP 首部 */
    if ((u16_t)(p->tot_len + UDP_HLEN) < p->tot_len)
    {
        return ERR_MEM;
```

（下页继续）

(续上页)

```
    }
    /* 没有足够的空间将 UDP 首部添加到给定的 pbuf 中 */
    if (pbuf_add_header(p, UDP_HLEN))
    {
        /* 在一个单独的新 pbuf 中分配标头 */
        q = pbuf_alloc(PBUF_IP, UDP_HLEN, PBUF_RAM);
        /* 无法分配新的标头 pbuf */
        if (q == NULL)
        {
            return ERR_MEM;
        }
        if (p->tot_len != 0)
        {
            /* 把首部 pbuf 和数据 pbuf 连接到一个 pbuf 链表上 */
            pbuf_chain(q, p);
        }
    }
    else
    {
        /* 在数据 pbuf 中已经预留 UDP 首部空间 */
        /* q 指向 pbuf */
        q = p;

    }

    /* 填写 UDP 首部各个字段 */
    udphdr = (struct udp_hdr *)q->payload;
    udphdr->src = lwip_htons(pcb->local_port);
    udphdr->dest = lwip_htons(dst_port);
    /* in UDP, 0 checksum means 'no checksum' */
    udphdr->chksum = 0x0000;
```

(下页继续)

(续上页)

```
    udphdr->len = lwip_htons(q->tot_len);

    ip_proto = IP_PROTO_UDP;


    /* 发送到 IP 层 */
    NETIF_SET_HINTS(netif, &(pcb->netif_hints));

    err = ip_output_if_src(q, src_ip,
                           dst_ip, ttl, pcb->tos,
                           ip_proto, netif);

    NETIF_RESET_HINTS(netif);

    MIB2_STATS_INC(mib2.udpoutdatagrams);

    if (q != p)
    {
        /* 释放内存 */
        pbuf_free(q);
        q = NULL;

    }

    UDP_STATS_INC(udp.xmit);
```

相比于 TCP 协议的处理，UDP 发送的处理就简单太多了，即使我们加上校验那部分，也是非常简单的，就是直接将用户的数据添加 UDP 首部然后调用 ip_output_if_src() 函数发送到 IP 层，当然啦，在这个函数之前还是有很多操作的，比如找到本地合适的网卡发送出去，找到本地 IP 地址与本地端口、找到目标 IP 地址与目标端口等等

14.6 UDP 报文接收

根据前面第十一章图 11_8 我们知道：当有一个 UDP 报文被 IP 层接收的时候，IP 层会调用 udp_input() 函数将报文传递到传输层，LwIP 就会去处理这个 UDP 报文，UDP 协议会对报文进行一些合法性的检测，如果确认了这个报文是合法的，那么就遍历 UDP 控制块链表，在这些控制块中找到对应的端口，然后递交到应用层，首先要判断本地端口号、本地 IP 地址与报文中的目标端口号、目标 IP 地址是否匹配，如果匹配就说明这个报文是给我们的，然后调用用户的回调函数 recv_udp() 将受到的数据传递给上层应用。而如果找不到对应的端口，那么将返回一个端口不可达 ICMP 差错控制报文到源主机，当然，如果 LwIP 接收到这个端口不可达 ICMP 报文，也是不会去处理它的，udp_input() 函数源码具体见。

代码清单 14-6udp_input() 源码

```
void
udp_input(struct pbuf *p, struct netif *inp)
{
    struct udp_hdr *udphdr;
    struct udp_pcb *pcb, *prev;
    struct udp_pcb *uncon_pcb;
    u16_t src, dest;
    u8_t broadcast;
    u8_t for_us = 0;

    LWIP_UNUSED_ARG(inp);

    LWIP_ASSERT_CORE_LOCKED();

    PERF_START;

    UDP_STATS_INC(udp.recv);

    /* 检查最小长度，不能小于 UDP 首部 */
```

(下页继续)

(续上页)

```
    if (p->len < UDP_HLEN)
    {
        UDP_STATS_INC(udp.lenerr);
        UDP_STATS_INC(udp.drop);
        MIB2_STATS_INC(mib2.udpinerrors);
        pbuf_free(p);
        goto end;
    }
    //指向 UDP 报文首部，并且强制转换成 udp_hdr 类型，方便操作
    udphdr = (struct udp_hdr *)p->payload;

    /* 判断一下是不是广播包 */
    broadcast = ip_addr_isbroadcast(ip_current_dest_addr(),
                                    ip_current_netif());

    /* 得到 UDP 首部中的源主机和目标主机端口号 */
    src = lwip_ntohs(udphdr->src);
    dest = lwip_ntohs(udphdr->dest);

    udp_debug_print(udphdr);

    pcb = NULL;
    prev = NULL;
    uncon_pcb = NULL;

    //遍历 UDP 链表，找到对应的端口号，如果找不到，
    //那就用链表的第一个未使用的 UDP 控制块
    for (pcb = udp_pcbs; pcb != NULL; pcb = pcb->next)
    {
        /* 将 UDP 控制块本地地址 + 端口与 UDP 目标地址 + 端口进行比较 */
        if ((pcb->local_port == dest) &&
                (udp_input_local_match(pcb, inp, broadcast) != 0))
```

(下页继续)

(续上页)

```
        {
            if ((pcb->flags & UDP_FLAGS_CONNECTED) == 0)
            {
                if (uncon_pcb == NULL)
                {
                    /* 如果未找到使用第一个 UDP 控制块 */
                    uncon_pcb = pcb;
#if LWIP_IPV4
                }
                else if (broadcast && ip4_current_dest_addr()->addr
                        == IPADDR_BROADCAST)
                {
                    /* 对于全局广播地址 */
                    if (!IP_IS_V4_VAL(uncon_pcb->local_ip) ||
                            !ip4_addr_cmp(ip_2_ip4(&uncon_pcb->local_ip),
                                        netif_ip4_addr(inp)))
                    {
                        /* 当前 UDP 控制块与输入 netif 不匹配，检查此 UDP 控制块␣
↪*/
                        if (IP_IS_V4_VAL(pcb->local_ip) &&
                            ip4_addr_cmp(ip_2_ip4(&pcb->local_ip),netif_ip4_
↪addr(inp)))
                        {
                            /* 得到更好的匹配 */
                            uncon_pcb = pcb;
                        }
                    }
#endif /* LWIP_IPV4 */
                }
            }

            /* 将 UDP 控制块的目标地址 + 端口与 UDP 控制块源地址 + 端口进行比较 */
```

(下页继续)

(续上页)

```
            if ((pcb->remote_port == src) &&
                 (ip_addr_isany_val(pcb->remote_ip) ||
                 ip_addr_cmp(&pcb->remote_ip, ip_current_src_addr())))
            {
                /* 第一个完全匹配的 UDP 控制块 */
                if (prev != NULL)
                {
                    /* 将 UDP 控制块移动到 udp_pcbs 的前面,
                    这样就可以在下次查找的时候处理速度更快 */
                    prev->next = pcb->next;
                    pcb->next = udp_pcbs;
                    udp_pcbs = pcb;
                }
                else
                {
                    UDP_STATS_INC(udp.cachehit);
                }
                break;
            }
        }

        prev = pcb;
    }
    /*  找不到完全匹配的 UDP 控制块？
        将第一个未使用的 UDP 控制块作为匹配结果 */
    if (pcb == NULL)
    {
        pcb = uncon_pcb;
    }

    /* 检查校验和是否匹配或是否匹配。 */
    if (pcb != NULL)
```

(下页继续)

(续上页)

```
    {
        for_us = 1;
    }
    else
    {

        if (!ip_current_is_v6())
        {
            for_us = ip4_addr_cmp(netif_ip4_addr(inp),
                                  ip4_current_dest_addr());
        }
    }
    //匹配
    if (for_us)
    {
        //调整报文的数据区域指针
        if (pbuf_remove_header(p, UDP_HLEN))
        {
            UDP_STATS_INC(udp.drop);
            MIB2_STATS_INC(mib2.udpinerrors);
            pbuf_free(p);
            goto end;
        }
        //如果找到对应的控制块
        if (pcb != NULL)
        {
            MIB2_STATS_INC(mib2.udpindatagrams);
            /* 回调函数，将数据递交给上层应用 */
            if (pcb->recv != NULL)
            {
                /* recv 函数需要负责释放 p */
                pcb->recv(pcb->recv_arg, pcb, p,
```

(下页继续)

(续上页)

```
                        ip_current_src_addr(), src);
            }
            else
            {
                /* 如果 recv 函数没有注册，直接释放 p */
                pbuf_free(p);
                goto end;
            }
        }
        /* 没有找到匹配的控制块，返回端口不可达 ICMP 报文 */
        else
        {
            if (!broadcast &&
                    !ip_addr_ismulticast(ip_current_dest_addr()))
            {
                /* 将数据区域指针移回 IP 数据报首部 */
                pbuf_header_force(p,
                        (s16_t)(ip_current_header_tot_len() + UDP_HLEN));
                //返回一个端口不可达 ICMP 差错控制报文到源主机中
                icmp_port_unreach(ip_current_is_v6(), p);
            }
            UDP_STATS_INC(udp.proterr);
            UDP_STATS_INC(udp.drop);
            MIB2_STATS_INC(mib2.udpnoports);
            pbuf_free(p);
        }
    }
    else
    {
        pbuf_free(p);
    }
end:
```

(下页继续)

(续上页)

```
    PERF_STOP("udp_input");
    return;
}
```

虽然 udp_input() 函数看起来很长，但是其实是非常简单的处理，主要就是遍历 UDP 控制块链表 udp_pcbs 找到对应的 UDP 控制块，然后将去掉 UDP 控制块首部信息，提取 UDP 报文数据递交给应用程序，而递交的函数就是在 UDP 控制块初始化时注册的回调函数，即 recv_udp()，而这个函数会让应用能读取到数据，然后做对应的处理。

至此，UDP 系统的内容就讲解完毕，对比 TCP 协议是不是简单太多了，整个 UDP 协议的处理过程具体见图 *14_4*。

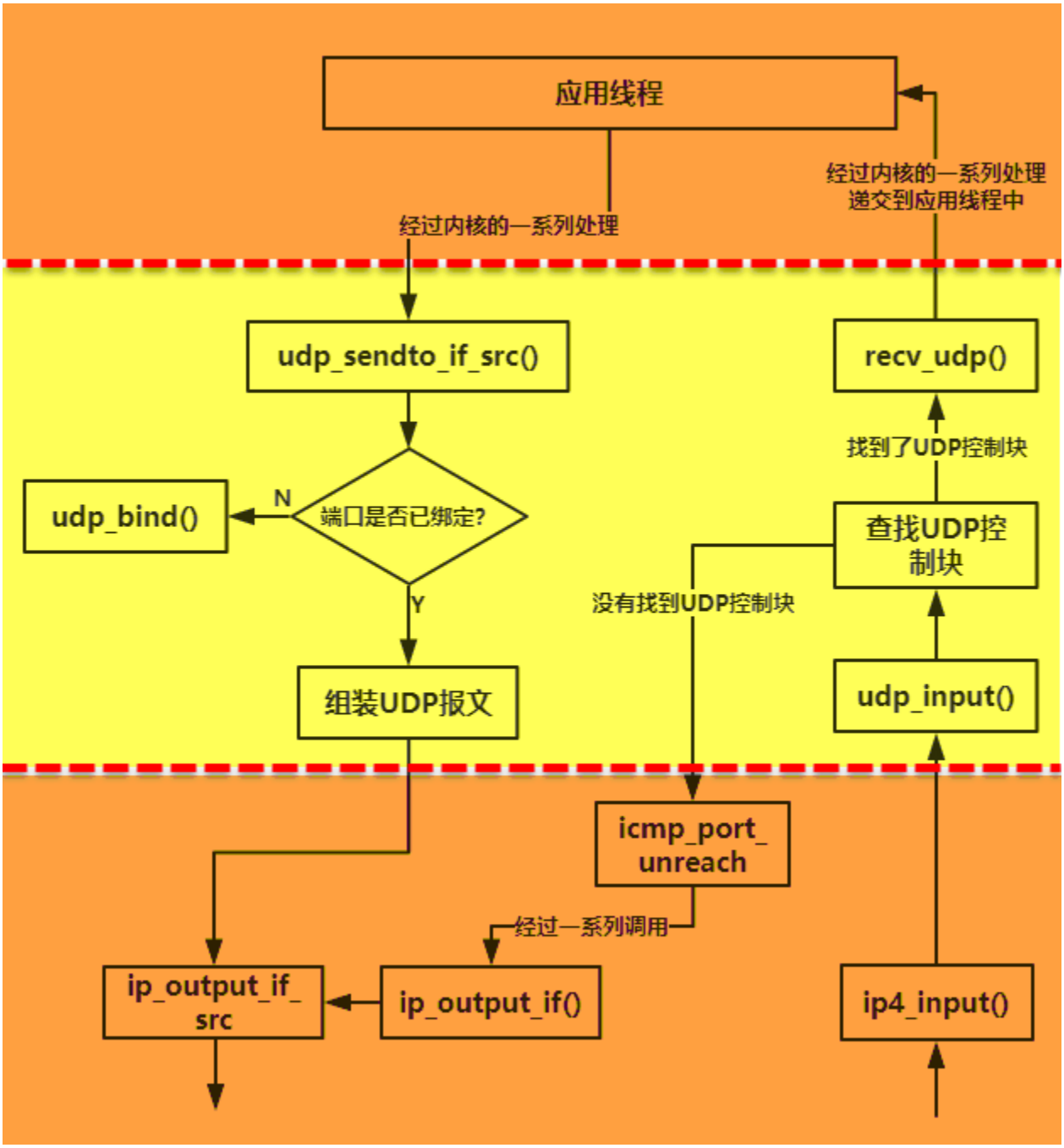


图 14-4UDP 协议的处理过程

第 15 章　使用 NETCONN 接口编程

当你学习到这章的时候，说明已经对 LwIP 中各个层的处理已经稔熟于心了，此时，再去回顾第 9 章的内容，相信，你会更加熟悉整个 LwIP 的运作过程，本书全是基于操作系统之上来讲解 LwIP，那么 NETCONN 接口编程的学习就是必须的，下面一起来学习一下 NETCONN API。

15.1 netbuf 结构体

LwIP 为了更好描述应用线程发送与接收的数据，并且为了更好管理这些数据的缓冲区，LwIP 定义了一个 netbuf 结构体，它是基于 pbuf 上更高一层的封装，记录了主机的 IP 地址与端口号，在这里再提醒一下大家，端口号对应的其实就是应用线程。在接收的时候，应用程序肯定需要知道到底是谁发数据给自己，而在发送的时候，应用程序需要将自己的端口号与 IP 地址填充到 netbuf 结构体对应字段中。netbuf 结构体具体见代码清单 15-1。

代码清单 15-1netbuf 结构体（已删减）

```
struct netbuf
{
    struct pbuf *p, *ptr;           (1)
    ip_addr_t addr;                 (2)
    u16_t port;                     (3)
};
```

代码清单 15-1**(1)**：netbuf 的 p 字段的指针指向 pbuf 链表，这是基于 pbuf 上封装的结构体，因此，ptr 字段的指针也是指向 pbuf，但是它与 p 字段的指针有一点不一样，因为它可以指向任意的 pbuf，由 netbuf_next() 与 netbuf_first() 函数来控制。

代码清单 15-1**(2)**：addr 字段记录了数据发送方的 IP 地址。

代码清单 15-1**(3)**：port 记录了数据发送方的端口号。

此外，LwIP 的一贯操作就是还会定义一些带参宏来快速操作这些结构体的字段，在这里操作 netbuf 也例外，如：

代码清单 15-2 处理 netbuf 字段的带参宏

```
#define netbuf_fromaddr(buf)          (&((buf)->addr))
#define netbuf_set_fromaddr(buf, fromaddr)  \
                ip_addr_set(&((buf)->addr), fromaddr)
#define netbuf_fromport(buf)          ((buf)->port)
```

netbuf 结构体指向示意图具体见图 *15_1* 与图 *15_2*，虚线表示 ptr 指针的指向位置是不固定的，它是由 netbuf_next() 函数与 netbuf_first() 函数来调整的。

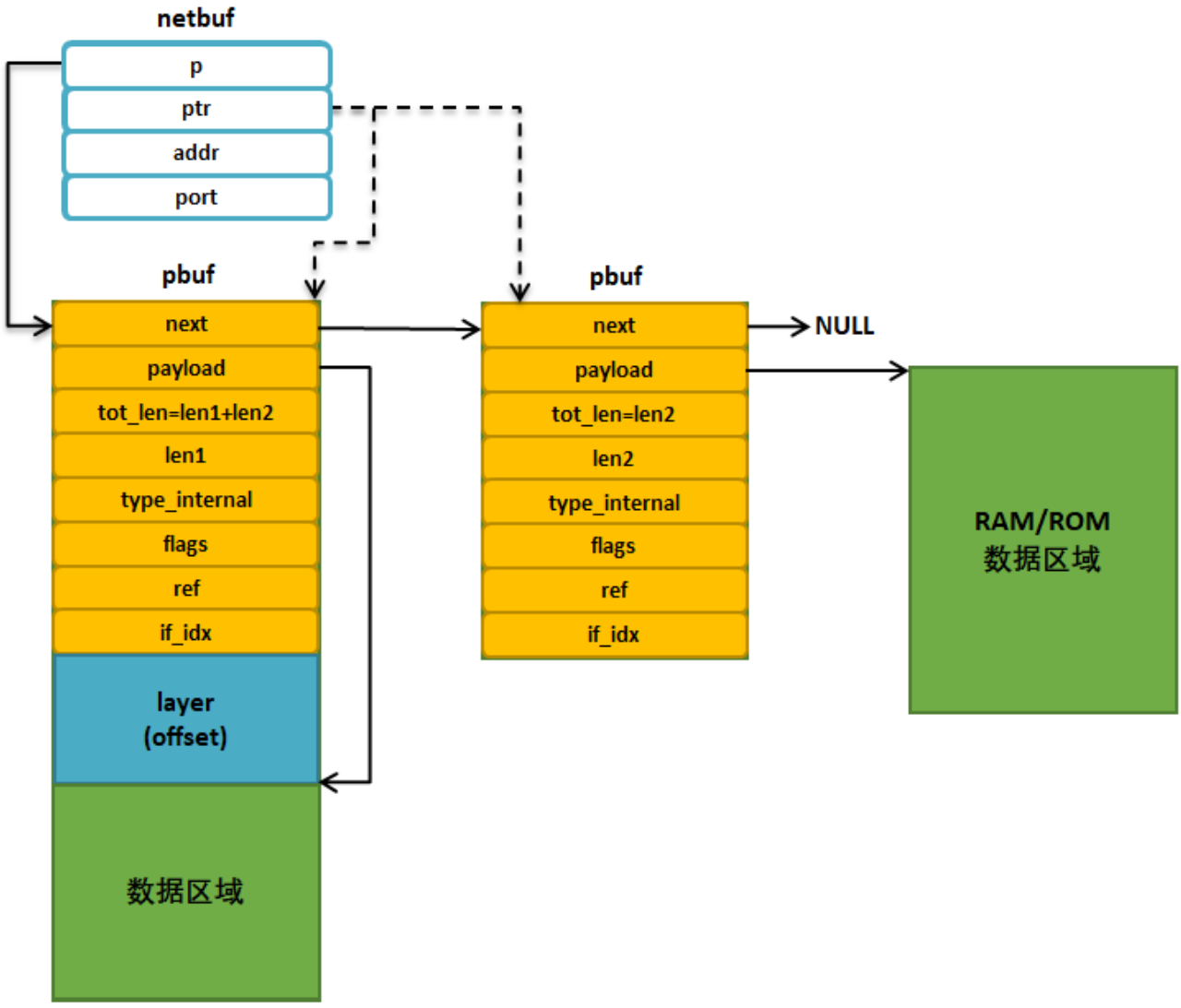


图 15-1 指向不同类型的 pbuf 链表

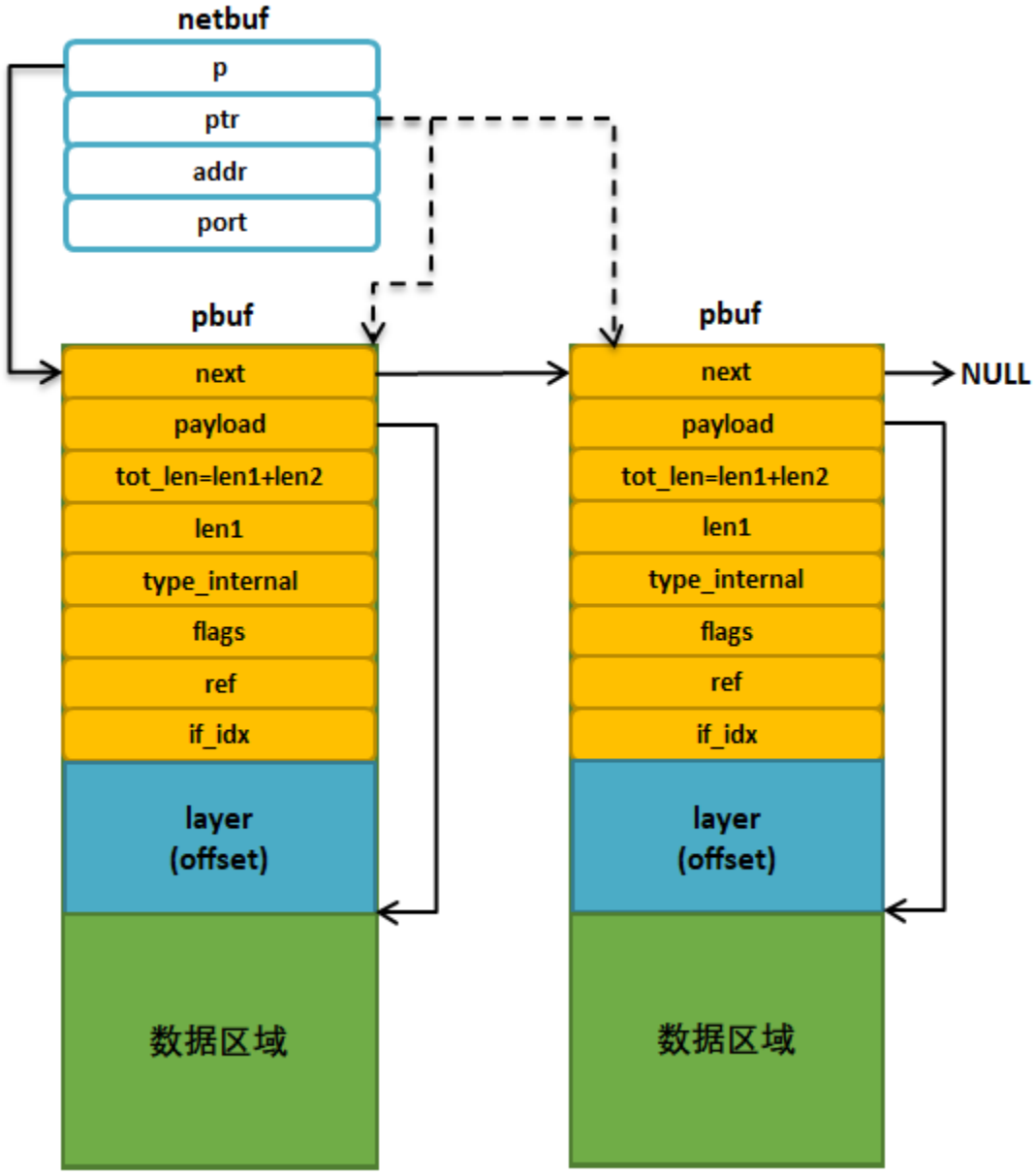


图 15-2 指向相同类型 pbuf 链表

15.2 netbuf 相关函数说明

netbuf 是 LwIP 描述用户数据很重要的一个结构体，因为 LwIP 是不可能让我们直接操作 pbuf 的，因为分层的思想，应用数据必然是由用户操作的，因此 LwIP 会提供很多函数接口让用户对 netbuf 进行操作，无论是 UDP 报文还是 TCP 报文段，其本质都是数据，要发送出去的数据都会封装在 netbuf 中，然后通过邮箱发送给内核线程（tcpip_thread 线程），然后经过内核的一系列处理，放入发送队列中，然后调用底层网卡发送函数进行发送，反之，应用线程接收到数据，也是通过 netbuf 进行管理，下面一起来看看 LwIP 提供给我们操作 netbuf 的相关函数。

15.2.1 netbuf_new()

这个函数的功能是申请一个新的 netbuf 结构体内存空间，通过 memp 内存池进行申请，大小为 MEMP_NETBUF，并且将 netbuf 结构体全部初始化为 0，并且返回一个指向 netbuf 结构体的指针，此时的 netbuf 结构体的 p 与 ptr 字段不指向任何的 pbuf，具体见*代码清单 15_3*。

代码清单 15-3 netbuf_new() 源码

```
struct
netbuf *netbuf_new(void)
{
    struct netbuf *buf;

    buf = (struct netbuf *)memp_malloc(MEMP_NETBUF);
    if (buf != NULL)
    {
        memset(buf, 0, sizeof(struct netbuf));
    }
    return buf;
}
```

15.2.2 netbuf_delete()

与 netbuf_new() 函数相反，释放一个 netbuf 结构体内存空间，如果 netbuf 结构体的 p 或者 ptr 字段指向的 pbuf 是拥有数据的，那么对应的 pbuf 也会被释放掉，具体见*代码清单 15_4*。

代码清单 15-4 netbuf_delete() 源码

```
void
netbuf_delete(struct netbuf *buf)
{
    if (buf != NULL)
    {
        if (buf->p != NULL)
        {
            pbuf_free(buf->p);
            buf->p = buf->ptr = NULL;
        }
        memp_free(MEMP_NETBUF, buf);
    }
}
```

15.2.3 netbuf_alloc()

这个函数比较有意思，为 netbuf 结构体中的 p 字段指向的数据区域分配指定大小的内存空间，简单来说就是申请 pbuf 内存空间，由于这个函数是在应用层调用的，因此这个内存会包含链路层首部、IP 层首部与传输层首部大小，当然，这些空间是附加上去的，用户指定的是数据区域大小，当然还有很重要的一点就是，如果当前 netbuf 中已经存在数据区域了，那么这个数据区域会被释放掉，然后重新申请用户指定大小的数据区域，而函数的返回是一个指向数据区域起始地址的指针（即 pbuf 的 payload 指针），具体见*代码清单 15_5*。

代码清单 15-5 netbuf_alloc() 源码

```
void *
netbuf_alloc(struct netbuf *buf, u16_t size)
{
    LWIP_ERROR("netbuf_alloc: invalid buf", (buf != NULL), return NULL;);

    /* Deallocate any previously allocated memory. */
    if (buf->p != NULL)
    {
        pbuf_free(buf->p);
    }
    buf->p = pbuf_alloc(PBUF_TRANSPORT, size, PBUF_RAM);
    if (buf->p == NULL)
    {
        return NULL;
    }
    LWIP_ASSERT("check that first pbuf can hold size",
                (buf->p->len >= size));
    buf->ptr = buf->p;
    return buf->p->payload;
}
```

15.2.4 netbuf_free()

这个函数的功能是比较简单的，直接释放 netbuf 结构体指向的 pbuf 内存空间，如果结构体中指向 pbuf 的内容为空，则不做任何释放操作，直接将 p 与 ptr 字段的指针设置为 NULL，具体见*代码清单 15_6*。

注意：这个函数释放的是 pbuf 内存空间，而不是 netbuf 结构体的内存空间。

代码清单 15-6 netbuf_free() 源码

```
void
```

(下页继续)

(续上页)

```
netbuf_free(struct netbuf *buf)
{
    LWIP_ERROR("netbuf_free: invalid buf", (buf != NULL), return;);
    if (buf->p != NULL)
    {
        pbuf_free(buf->p);
    }
    buf->p = buf->ptr = NULL;
}
```

15.2.5 netbuf_ref()

这个函数与 netbuf_alloc() 函数很像，都是申请内存空间，但是，有一个很大的不同，netbuf_ref() 函数只申请 pbuf 首部的内存空间，包含链路层首部、IP 层首部与传输层首部，而不会申请数据区域内存空间，然后把 pbuf 的 payload 指针指向用户指定的数据区域起始地址 dataptr，这种申请经常在发送静态数据的时候用到，因为数据保存的地址是固定的，而不用动态申请，如果 netbuf 的 p 或者 ptr 字段已经指向了 pbuf，那么这些 pbuf 将被释放掉，具体见*代码清单 15_7*。

注意：在使用该函数的时候用户需要传递有效的静态数据区域起始地址，比如某个静态字符串的起始地址。

代码清单 15-7netbuf_ref() 源码

```
err_t
netbuf_ref(struct netbuf *buf, const void *dataptr, u16_t size)
{
    LWIP_ERROR("netbuf_ref: invalid buf", (buf != NULL), return ERR_ARG;);
    if (buf->p != NULL)
    {
        pbuf_free(buf->p);
    }
    buf->p = pbuf_alloc(PBUF_TRANSPORT, 0, PBUF_REF);
```

(下页继续)

(续上页)

```
    if (buf->p == NULL)
    {
        buf->ptr = NULL;
        return ERR_MEM;
    }
    ((struct pbuf_rom *)buf->p)->payload = dataptr;
    buf->p->len = buf->p->tot_len = size;
    buf->ptr = buf->p;
    return ERR_OK;
}
```

15.2.6 netbuf_chain()

netbuf_chain() 函数是将 tail 中的 pbuf 数据连接到 head 中的 pbuf 后面，形成一个 pbuf 链表，在调用此函数之后，会将 tail 结构删除，具体见*代码清单 15_8*。

代码清单 15-8 netbuf_chain() 源码

```
void
netbuf_chain(struct netbuf *head, struct netbuf *tail)
{
    LWIP_ERROR("netbuf_chain: invalid head", (head != NULL), return;);
    LWIP_ERROR("netbuf_chain: invalid tail", (tail != NULL), return;);
    pbuf_cat(head->p, tail->p);
    head->ptr = head->p;
    memp_free(MEMP_NETBUF, tail);
}
```

15.2.7 netbuf_data()

该函数可以将 netbuf 结构体中的 ptr 指针指向的 pbuf 数据起始地址填写到 dataptr 中，同时将数据长度填入 len 中，netbuf 结构体中 p 字段的指针指向的数据可能是一个 pbuf 链表，但是这个函数的操作只能是将 ptr 指针指向的 pbuf 数据填写到 dataptr 中，如果想要操作 netbuf 中 p 指向的链表数据，想要使用 netbuf_next() 或者 netbuf_first() 函数来调整 ptr 指针指向的 pbuf，具体见*代码清单 15_9*。

代码清单 15-9 netbuf_data()

```
err_t
netbuf_data(struct netbuf *buf, void **dataptr, u16_t *len)
{
LWIP_ERROR("netbuf_data: invalid buf", (buf != NULL), return ERR_ARG;);
LWIP_ERROR("netbuf_data: invalid dataptr", (dataptr != NULL), return ERR_
↪ARG;);
LWIP_ERROR("netbuf_data: invalid len", (len != NULL), return ERR_ARG;);

    if (buf->ptr == NULL)
    {
        return ERR_BUF;
    }
    *dataptr = buf->ptr->payload;
    *len = buf->ptr->len;
    return ERR_OK;
}
```

15.2.8 netbuf_next() 与 netbuf_first()

netbuf_next() 用于移动 netbuf 的 ptr 数据指针，使 ptr 指针指向 pbuf 链表的下一个 pbuf。同样的 netbuf_first() 函数可以将 ptr 指针指向 pbuf 链表的第一个 pbuf。这两个函数是很有用的，比如 netbuf 中 p 字段的指针指向一个 pbuf 链表，并且 pbuf 链表中拥有多个 pbuf，那么需要配合 netbuf_data() 函数将链表中的所有的 pbuf 读取并且处理；如果 netbuf_next() 函数的返回值为 0，表

示调整成功，而如果返回值小于 0 时，则表示调整失败，当前 pbuf 已经是链表的最后一个 pbuf，具体见*代码清单 15_10*。

代码清单 15-10 netbuf_next() 与 netbuf_first() 源码

```
s8_t
netbuf_next(struct netbuf *buf)
{
    LWIP_ERROR("netbuf_next: invalid buf", (buf != NULL), return -1;);
    if (buf->ptr->next == NULL)
    {
        return -1;
    }
    buf->ptr = buf->ptr->next;
    if (buf->ptr->next == NULL)
    {
        return 1;
    }
    return 0;
}

void
netbuf_first(struct netbuf *buf)
{
    LWIP_ERROR("netbuf_first: invalid buf", (buf != NULL), return;);
    buf->ptr = buf->p;
}
```

15.2.9 netbuf_copy()

这个函数用于将 netbuf 结构体数据区域 pbuf 中的所有数据拷贝到 dataptr 指针指向的存储区，即使 pbuf（链表）中的数据被保存在多个 pbuf 中，它也会完全拷贝出来，len 参数指定要拷贝数据的最大长度，如果 netbuf 的数据区域空间小于 len 指定的大小，那么内核只会拷贝 netbuf 数据区域大小的数据，此外，该函数本质是一个宏定义，真正实现的函数在 pbuf.c 中，具体见*代码清单 15_11*。

代码清单 15-11 netbuf_copy() 源码

```
#define netbuf_copy_partial(buf, dataptr, len, offset) \
            pbuf_copy_partial((buf)->p, (dataptr), (len), (offset))
#define netbuf_copy(buf,dataptr,len)  \
            netbuf_copy_partial(buf, dataptr, len, 0)

u16_t pbuf_copy_partial(const struct pbuf *buf, void *dataptr,
            u16_t len, u16_t offset)
{
    const struct pbuf *p;
    u16_t left = 0;
    u16_t buf_copy_len;
    u16_t copied_total = 0;


    for (p = buf; len != 0 && p != NULL; p = p->next)
    {
        if ((offset != 0) && (offset >= p->len))
        {
            offset = (u16_t)(offset - p->len);
        }
        else
        {
        buf_copy_len = (u16_t)(p->len - offset);
```

(下页继续)

(续上页)

```
            if (buf_copy_len > len)
            {
                buf_copy_len = len;
            }
            MEMCPY(&((char *)dataptr)[left],
                   &((char *)p->payload)[offset], buf_copy_len);
            copied_total = (u16_t)(copied_total + buf_copy_len);
            left = (u16_t)(left + buf_copy_len);
            len = (u16_t)(len - buf_copy_len);
            offset = 0;
        }
    }
    return copied_total;
}
```

15.2.10 netbuf_take()

这个函数用于将用户指定区域的数据 dataptr 拷贝到 netbuf 结构体数据区域 pbuf 中，可能用户数据太多，一个 pbuf 存储不下用户的数据，那么内核将对数据进行切割处理，使用多个 pbuf 存储，len 参数指定要拷贝数据的长度，具体见*代码清单 15_12*。

代码清单 15-12 netbuf_take() 源码

```
#define netbuf_take(buf, dataptr, len) \
    pbuf_take((buf)->p, dataptr, len)

err_t
pbuf_take(struct pbuf *buf, const void *dataptr, u16_t len)
{
    struct pbuf *p;
    size_t buf_copy_len;
```

(下页继续)

(续上页)

```
    size_t total_copy_len = len;
    size_t copied_total = 0;

    if ((buf == NULL) || (dataptr == NULL) || (buf->tot_len < len))
    {
        return ERR_ARG;
    }

    /* 拷贝数据 */
    for (p = buf; total_copy_len != 0; p = p->next)
    {
        LWIP_ASSERT("pbuf_take: invalid pbuf", p != NULL);
        buf_copy_len = total_copy_len;
        if (buf_copy_len > p->len)
        {
            /* 此 pbuf 无法保存所有剩余数据 */
            buf_copy_len = p->len;
        }
        /* 从 dataptr 拷贝数据到 p->payload*/
        MEMCPY(p->payload, &((const char *)dataptr)[copied_total], buf_
↪copy_len);
        total_copy_len -= buf_copy_len;
        copied_total += buf_copy_len;
    }
    return ERR_OK;
}
```

15.2.11 其他操作 netbuf 的宏定义

除了上述的一些函数（宏定义）以外，LwIP 还定义了很多宏直接操作 netbuf，获得 netbuf 的信息，或者设置 netbuf 的字段信息，具体见

代码清单 15-13 其他操作 netbuf 的宏定义

```
//获取数据的总长度
#define netbuf_len(buf)                 ((buf)->p->tot_len)

//得到远端 IP 地址（目标 IP 地址）
#define netbuf_fromaddr(buf)            (&((buf)->addr))

//设置远端 IP 地址（目标 IP 地址）
#define netbuf_set_fromaddr(buf, fromaddr) \
ip_addr_set(&((buf)->addr), fromaddr)

//得到远端端口号
#define netbuf_fromport(buf)            ((buf)->port)
```

15.3 netconn 结构体

在 LwIP 中，如 TCP 连接，UDP 通信，都是需要提供一个编程接口给用户使用的，那么为了描述这样子的一个接口，LwIP 抽象出来一个 nettonn 结构体，它能描述一个连接，供应用程序使用，同时内核的 NETCONN API 接口也对各种连接操作函数进行了统一的封装，这样子，用户程序可以很方便使 netconn 和编程函数，我们暂且将 netconn 称之为连接结构体。

一个连接结构体中包含的成员变量很多，如描述连接的类型，连接的状态（主要是在 TCP 连接中使用），对应的控制块（如 UDP 控制块、TCP 控制块等等），还有对应线程的消息邮箱以及一些记录的信息，具体见*代码清单 15_14*。

代码清单 15-14 netconn 结构体

```
struct netconn
{
    /** netconn 类型 */
    enum netconn_type type;
    /** 当前 netconn 状态 */
    enum netconn_state state;
    /** LwIP 的控制块指针，如 TCP 控制块、UDP 控制块 */
    union
    {
        struct ip_pcb  *ip;
        struct tcp_pcb *tcp;
        struct udp_pcb *udp;
        struct raw_pcb *raw;
    } pcb;
    err_t pending_err;/** 这个 netconn 最后一个异步未报告的错误 */
    sys_sem_t op_completed; //信号量
    /** 消息邮箱，存储接收的数据，直到它们被提取 */
    sys_mbox_t recvmbox;
    /** 用于 TCP 服务器上的请求连接缓冲区 */
    sys_mbox_t acceptmbox;

    /** socket 描述符，用于 Socket API */
#if LWIP_SOCKET
    int socket;
#endif /* LWIP_SOCKET */

    /** 标志 */
    u8_t flags;
#if LWIP_TCP
    /** 当调用 netconn_write() 函数发送的数据不适合发送缓冲区时，
        数据会暂时存储在 current_msg 中，等待数据合适的时候进行发送 */
    struct api_msg *current_msg;
```

(下页继续)

(续上页)

```
#endif /* LWIP_TCP */
    /** 连接相关的回调函数 */
    netconn_callback callback;
};
```

在这里我们简单讲解一下 netconn 结构体中的部分字段，op_completed 信号量是非常重要的字段，它是等待对应的处理函数处理完成时两部分线程进行同步，具体可以参考图 9_4 的运作过程。

recvmbox 邮箱是接收数据的邮箱，内核线程会把接收到的数据投递到这个邮箱中，等待应用线程来读取这些数据。

同时 LwIP 为了描述连接的类型与状态，定义了一些枚举类型的变量，此外还定义了一个回调函数的指针类型，具体见代码清单 15_15。

代码清单 15-15 描述 netconn 的类型、状态及回调函数

```
enum netconn_type
{
    NETCONN_INVALID     = 0,
    /** TCP */
    NETCONN_TCP         = 0x10,
    /** UDP */
    NETCONN_UDP         = 0x20,
    /** UDP lite */
    NETCONN_UDPLITE     = 0x21,
    /** 无校验 UDP */
    NETCONN_UDPNOCHKSUM = 0x22,
    /** Raw */
    NETCONN_RAW         = 0x40
};

enum netconn_state
{
```

(下页继续)

(续上页)

```
    NETCONN_NONE,         //不处于任何状态
    NETCONN_WRITE,        //正在写（发送）数据
    NETCONN_LISTEN,       //处于监听状态
    NETCONN_CONNECT,      //处于连接状态
    NETCONN_CLOSE         //处于关闭状态
};

typedef void (* netconn_callback)(struct netconn *,
                                  enum netconn_evt,
                                  u16_t len);
```

15.4 netconn 函数接口说明

在前面一直都是 NETCONN API，但是都没真正使用到这些 API，下面我们就正式进入 NETCONN API 的学习阶段，下面这些函数都在 api_lib.c 文件中实现，在 api.h 头文件中声明。

提示：本章的内容建议结合第 9 章一起查看，更深入了解这些函数接口的运作。

15.4.1 netconn_new()

函数 netconn_new () 本质上是一个宏定义，它用来创建一个新的连接结构，连接结构的类型可以选择为 TCP 或 UDP 等，参数 type 描述了连接的类型，可以为 NETCONN_TCP 或 NETCONN_UDP 等，在这个函数被调用时，会初始化相关的字段，而并不会创建连接，具体见*代码清单 15_16*。

代码清单 15-16 netconn_new() 源码

```
//该函数本质是宏定义
#define netconn_new(t)    \
            netconn_new_with_proto_and_callback(t, 0, NULL)
```

(下页继续)

(续上页)

```
//真正实现的函数
struct netconn *
netconn_new_with_proto_and_callback(enum netconn_type t,
                                    u8_t proto,
                                    netconn_callback callback)
{
    struct netconn *conn;
    API_MSG_VAR_DECLARE(msg);
    API_MSG_VAR_ALLOC_RETURN_NULL(msg);

    conn = netconn_alloc(t, callback);              (1)
    if (conn != NULL)
    {
        err_t err;

        API_MSG_VAR_REF(msg).msg.n.proto = proto;
        API_MSG_VAR_REF(msg).conn = conn;
        err = netconn_apimsg(lwip_netconn_do_newconn,
                             &API_MSG_VAR_REF(msg));          (2)
        if (err != ERR_OK)
        {
            sys_sem_free(&conn->op_completed);
            sys_mbox_free(&conn->recvmbox);
            memp_free(MEMP_NETCONN, conn);
            API_MSG_VAR_FREE(msg);
            return NULL;
        }
    }
    API_MSG_VAR_FREE(msg);
    return conn;
}
```

代码清单 15-16**(1)**：我们先看看这个函数的执行过程，首先调用 netconn_alloc() 函数申请并且并且初始化一个 netconn 结构，初始化的过程包括设置 netconn 结构体中类型字段等，同时为该结构的 op_completed 创建一个信号量（用于同步）、在 netconn 结构体中的 recvmbox 字段创建一个接收邮箱。但是注意了，netconn_alloc () 函数并不是在 api_lib.c 文件中，而是在 api_msg.c 中，具体的实现过程大家可以自行去了解，在这里就不多讲解。

代码清单 15-16**(2)**：调用 netconn_apimsg() 函数构造一个消息，并且通过系统的消息邮箱发送给内核线程，请求 LwIP 内核去执行 lwip_netconn_do_newconn() 函数，并且在执行的时候，需要利用 op_completed 字段的信号量进行同步，直到内核处理完后，会释放一个信号量表示执行完成，这样子就形成两个线程间的同步，netconn_new() 函数才得以继续执行。

15.4.2 netconn_delete()

这个函数的功能与 netconn_new() 函数刚好是相反的，它用于删除一个 netconn 连接结构，对于 TCP 连接，如果此时是处于连接状态的，在调用该函数后，将请求内核执行终止连接操作，此时应用线程是无需理会到底是怎么运作的，因为 LwIP 内核将会完成所有的挥手过程，需要注意的是此时的 TCP 控制块还是不会立即被删除的，因为需要完成真正的断开挥手操作，这些状态可以参考 TCP 协议状态转移图。而对于 UDP 协议，UDP 控制块将被删除，终止通信，具体见*代码清单 15_17*。

代码清单 15-17 netconn_delete() 源码

```
err_t
netconn_delete(struct netconn *conn)
{
    err_t err;

    /* 判断一下 netconn 结构是否正确 */
    if (conn == NULL)
    {
        return ERR_OK;
    }
```

(下页继续)

(续上页)

```
    err = netconn_prepare_delete(conn);                    (1)

    if (err == ERR_OK)
    {
        netconn_free(conn);
    }
    return err;
}

err_t
netconn_prepare_delete(struct netconn *conn)
{
    err_t err;
    API_MSG_VAR_DECLARE(msg);

    if (conn == NULL)
    {
        return ERR_OK;
    }

    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;

    //记录时间
    API_MSG_VAR_REF(msg).msg.sd.polls_left =
        ((LWIP_TCP_CLOSE_TIMEOUT_MS_DEFAULT
        + TCP_SLOW_INTERVAL - 1) / TCP_SLOW_INTERVAL) + 1;

    err = netconn_apimsg(lwip_netconn_do_delconn,
                         &API_MSG_VAR_REF(msg));       (2)

    API_MSG_VAR_FREE(msg);
```

(下页继续)

(续上页)

```
    if (err != ERR_OK)
    {
        return err;
    }
    return ERR_OK;
}
```

代码清单 15-17**(1)(2)**：这个函数真正处理的函数是 netconn_prepare_delete()，它同样是调用 netconn_apimsg() 函数先构造一个 API 消息，然后投递到系统邮箱，请求 LwIP 内核线程去执行 lwip_netconn_do_delconn() 函数，这个函数会将对应的 netconn 连接结构删除，在执行完毕之后，通过信号量进行同步，应用线程得以继续执行。

15.4.3 netconn_getaddr()

netconn_getaddr() 函数的作用很简单，就是获取一个 netconn 连接结构的源 IP 地址、端口号与目标 IP 地址、端口号等信息，并且 IP 地址保存在 addr 中，端口号保存在 port 中，而 local 指定需要获取的信息是本地 IP 地址（源 IP 地址）还是远端 IP 地址（目标 IP 地址），如果是 1 则表示获取本地 IP 地址与端口号，如果为 0 表示远端 IP 地址与端口号。同样的，该函数会调用 netconn_apimsg() 函数构造一个 API 消息，并且请求内核执行 lwip_netconn_do_getaddr() 函数，然后通过 netconn 连接结构的信号量进行同步，具体见*代码清单 15_18*。

代码清单 15-18 netconn_getaddr() 源码

```
err_t
netconn_getaddr(struct netconn *conn,
                ip_addr_t *addr,
                u16_t *port,
                u8_t local)
{
    API_MSG_VAR_DECLARE(msg);
```

(下页继续)

(续上页)

```
    err_t err;

    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;
    API_MSG_VAR_REF(msg).msg.ad.local = local;
    msg.msg.ad.ipaddr = addr;
    msg.msg.ad.port = port;

    err = netconn_apimsg(lwip_netconn_do_getaddr, &msg);

    API_MSG_VAR_FREE(msg);

    return err;
}
```

15.4.4 netconn_bind()

netconn_bind() 函数用于将一个 IP 地址及端口号与 netconn 连接结构进行绑定，如果作为服务器端，这一步操作是必然需要的，同样的，该函数会调用 netconn_apimsg() 函数构造一个 API 消息，并且请求内核执行 lwip_netconn_do_bind() 函数，然后通过 netconn 连接结构的信号量进行同步，事实上内核线程的处理也是通过函数调用 xxx_bind（xxx_bind 可以是 udp_bind、tcp_bind、raw_bind，具体是哪个函数内核是根据 netconn 的类型决定的）完成相应控制块的绑定工作，具体见*代码清单 15_19*。

代码清单 15-19 netconn_bind() 源码

```
err_t
netconn_bind(struct netconn *conn,
             const ip_addr_t *addr,
             u16_t port)
{
```

(下页继续)

(续上页)

```
    API_MSG_VAR_DECLARE(msg);
    err_t err;
    /* 如果 IP 地址为空，将设置为 */
    if (addr == NULL)
    {
        addr = IP4_ADDR_ANY;
    }

    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;
    API_MSG_VAR_REF(msg).msg.bc.ipaddr = API_MSG_VAR_REF(addr);
    API_MSG_VAR_REF(msg).msg.bc.port = port;

    err = netconn_apimsg(lwip_netconn_do_bind,
                         &API_MSG_VAR_REF(msg));

    API_MSG_VAR_FREE(msg);

    return err;
}
```

15.4.5 netconn_connect()

netconn_connect() 函数是一个主动建立连接的函数，它一般在客户端中调用，将服务器端的 IP 地址和端口号与本地的 netconn 连接结构绑定，当 TCP 协议使用该函数的时候就是进行握手的过程，调用的应用线程将阻塞至握手完成；而对于 UDP 协议来说，调用该函数只是设置 UDP 控制块的目标 IP 地址与目标端口号，其实这个函数也是通过调用 netconn_apimsg() 函数构造一个 API 消息，并且请求内核执行 lwip_netconn_do_connect() 函数，然后通过 netconn 连接结构的信号量进行同步，在 lwip_netconn_do_connect() 函数中，根据 netconn 的类型不同，调用对应的 xxx_connect() 函数进行对应的处理，如果是 TCP 连接，将调用 tcp_connect()；如果是 UDP 协议，将调用 udp_connect()；如果是 RAW，将调用 raw_connect() 函数处理，具体见*代码清单 15_20*。

代码清单 15-20 netconn_connect() 源码

```
err_t
netconn_connect(struct netconn *conn,
                const ip_addr_t *addr,
                u16_t port)
{
    API_MSG_VAR_DECLARE(msg);
    err_t err;

    if (addr == NULL)
    {
        addr = IP4_ADDR_ANY;
    }

    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;
    API_MSG_VAR_REF(msg).msg.bc.ipaddr = API_MSG_VAR_REF(addr);
    API_MSG_VAR_REF(msg).msg.bc.port = port;

    err = netconn_apimsg(lwip_netconn_do_connect,
                         &API_MSG_VAR_REF(msg));

    API_MSG_VAR_FREE(msg);

    return err;
}
```

15.4.6 netconn_disconnect()

该函数是用于终止一个 UDP 协议的通信，注意，是 UDP 协议，而不是 TCP 协议，因为这个函数只能用于 UDP 协议，简单来说就是将 UDP 控制块的目标 IP 地址与目标端口号清除，不过麻雀虽小，但五脏俱全，同样的该函数也是构造 API 消息请求内核执行 lwip_netconn_do_disconnect() 函数，具体见*代码清单 15_21*。

代码清单 15-21 netconn_disconnect() 源码

```
err_t
netconn_disconnect(struct netconn *conn)
{
    API_MSG_VAR_DECLARE(msg);
    err_t err;

    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;

    err = netconn_apimsg(lwip_netconn_do_disconnect,
                         &API_MSG_VAR_REF(msg));

    API_MSG_VAR_FREE(msg);

    return err;
}
```

15.4.7 netconn_listen()

netconn_listen() 函数的本质是一个带参宏，其真正调用的函数是 netconn_listen_with_backlog()，只适用于 TCP 服务器中调用，它的作用是让 netconn 连接结构处于监听状态，同时让 TCP 控制块的状态处于 LISTEN 状态，以便客户端连接，同样的，它通过 netconn_apimsg() 函数请求内核执行 lwip_netconn_do_listen()，这个函数才是真正处理 TCP 连接的监听状态，并且在这个函数中会创建一个连接邮箱——acceptmbox 邮箱在 netconn 连接结构中，然后在 TCP 控制块中注册连接回

调函数——accept_function()，当有客户端连接的时候，这个回调函数被执行，并且向 acceptmbox 邮箱发送一个消息，通知应用程序有一个新的客户端连接，以便用户去处理这个连接。当然，在 wip_netconn_do_listen() 函数处理完成的时候会释放一个信号量，以进行线程间的同步，具体见*代码清单 15_22*。

代码清单 15-22 netconn_listen() 源码

```
#define netconn_listen(conn)         \
    netconn_listen_with_backlog(conn, TCP_DEFAULT_LISTEN_BACKLOG)

err_t
netconn_listen_with_backlog(struct netconn *conn, u8_t backlog)
{
    API_MSG_VAR_DECLARE(msg);
    err_t err;

    LWIP_UNUSED_ARG(backlog);

    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;

    err = netconn_apimsg(lwip_netconn_do_listen,
                         &API_MSG_VAR_REF(msg));

    API_MSG_VAR_FREE(msg);

    return err;
}
```

15.4.8 netconn_accept()

该函数用于 TCP 服务器中，接受远端主机的连接，内核会在 acceptmbox 邮箱中获取一个连接请求，如果邮箱中没有连接请求，将阻塞应用程序，直到接收到从远端主机发出的连接请求。调用这个函数的应用程序必须处于监听（LISTEN）状态，因此在调用 netconn_accept() 函数之前必须调用 netconn_listen() 函数进入监听状态，在与远程主机的连接建立后，函数返回一个连接结构 netconn；该函数在并不会构造一个 API 消息，而是直接获取 acceptmbox 邮箱中的连接请求，如果没有连接请求，将一直阻塞，当接收到远端主机的连接请求后，它会触发一个连接事件的回调函数（netconn 结构体中的回调函数字段），连接的信息由 accept_function() 函数完成。可能很多同学一看那源码，没发现这个回调函数啊，其实在 LwIP 在将 TCP 服务器进入监听状态的时候就已经注册了这个回调函数，在有连接的时候，就直接进行连接。在 lwip_netconn_do_listen() 函数中调用 tcp_accept() 函数进行注册连接时候的回调函数，netconn_accept() 源码具体见*代码清单 15_23*。

代码清单 15-23 netconn_accept() 源码

```
err_t
netconn_accept(struct netconn *conn, struct netconn **new_conn)
{
#if LWIP_TCP
    err_t err;
    void *accept_ptr;
    struct netconn *newconn;

    err = netconn_err(conn);
    if (err != ERR_OK)
    {
        /* return pending error */
        return err;
    }
    if (!NETCONN_ACCEPTMBOX_WAITABLE(conn))
    {
        /* 如果 acceptmbox 无效 */
        return ERR_CLSD;
```

(下页继续)

(续上页)

```
    }
    //如果 netconn 是不阻塞的
    if (netconn_is_nonblocking(conn))
    {
        //从 acceptmbox 邮箱获取远端主机的连接请求
        if (sys_arch_mbox_tryfetch(&conn->acceptmbox, &accept_ptr)
                == SYS_ARCH_TIMEOUT)
        {
            //如果超时
            return ERR_WOULDBLOCK;
        }
    }
    else
    {
        //一直等待着远端的连接请求
        sys_arch_mbox_fetch(&conn->acceptmbox, &accept_ptr, 0);
    }

    /* 触发连接事件的回调函数 */
    API_EVENT(conn, NETCONN_EVT_RCVMINUS, 0);

    if (lwip_netconn_is_err_msg(accept_ptr, &err))
    {
        /* 如果连接错误 */
        return err;
    }
    if (accept_ptr == NULL)
    {
        /* 连接已终止 */
        return ERR_CLSD;
    }
    newconn = (struct netconn *)accept_ptr;
```

(下页继续)

(续上页)

```
    *new_conn = newconn;

    return ERR_OK;
}
```

15.4.9 netconn_recv()

这个函数可能是我们在写代码中遇到最多的函数了，它可以接收一个 UDP 或者 TCP 的数据包，从 recvmbox 邮箱中获取数据包，如果该邮箱中没有数据包，那么线程调用这个函数将会进入阻塞状态以等待消息的到来，如果在等待 TCP 连接上的数据时，远端主机终止连接，将返回一个终止连接的错误代码（ERR_CLSD），应用程序可以根据错误的类型进行不一样的处理。

对应 TCP 连接，netconn_recv() 函数将调用 netconn_recv_data_tcp() 函数去获取 TCP 连接上的数据，在获取数据的过程中，调用 netconn_recv_data() 函数从 recvmbox 邮箱获取 pbuf，然后通过 netconn_tcp_recvd_msg()->netconn_apimsg() 函数构造一个 API 消息投递给系统邮箱，请求内核执行 lwip_netconn_do_recv() 函数，该函数将调用 tcp_recved() 函数去更新 TCP 接收窗口，同时 netconn_recv() 函数将完成 pbuf 数据包封装在 netbuf 中，返回个应用程序；而对于 UDP 协议、RAW 连接，将简单多了，将直接调用 netconn_recv_data() 函数获取数据，完成 pbuf 封装在 netbuf 中，返回给应用程序，具体见*代码清单 15_24*。

代码清单 15-24 netconn_recv() 源码

```
err_t
netconn_recv(struct netconn *conn, struct netbuf **new_buf)
{
#if LWIP_TCP
    struct netbuf *buf = NULL;
    err_t err;
#endif /* LWIP_TCP */

    *new_buf = NULL;
```

(下页继续)

(续上页)

```
#if LWIP_TCP
#if (LWIP_UDP || LWIP_RAW)

    //如果是 TCP 连接
    if (NETCONNTYPE_GROUP(conn->type) == NETCONN_TCP)
#endif /* (LWIP_UDP || LWIP_RAW) */
    {
        struct pbuf *p = NULL;
        /* 申请内存 */
        buf = (struct netbuf *)memp_malloc(MEMP_NETBUF);
        if (buf == NULL)
        {
            return ERR_MEM;
        }

        //等待数据 —— TCP 连接
        err = netconn_recv_data_tcp(conn, &p, 0);
        if (err != ERR_OK)
        {
            memp_free(MEMP_NETBUF, buf);
            return err;
        }
        LWIP_ASSERT("p != NULL", p != NULL);

        buf->p = p;
        buf->ptr = p;
        buf->port = 0;
        ip_addr_set_zero(&buf->addr);
        *new_buf = buf;

        return ERR_OK;
```

(下页继续)

(续上页)

```
    }
#endif /* LWIP_TCP */
#if LWIP_TCP && (LWIP_UDP || LWIP_RAW)
    //如果是 UDP 协议、RAW 连接
    else
#endif /* LWIP_TCP && (LWIP_UDP || LWIP_RAW) */
    {
#if (LWIP_UDP || LWIP_RAW)

        //等待数据
        return netconn_recv_data(conn, (void **)new_buf, 0);
#endif /* (LWIP_UDP || LWIP_RAW) */
    }
}
```

整个运作过程的示意图具体见图 15-3。

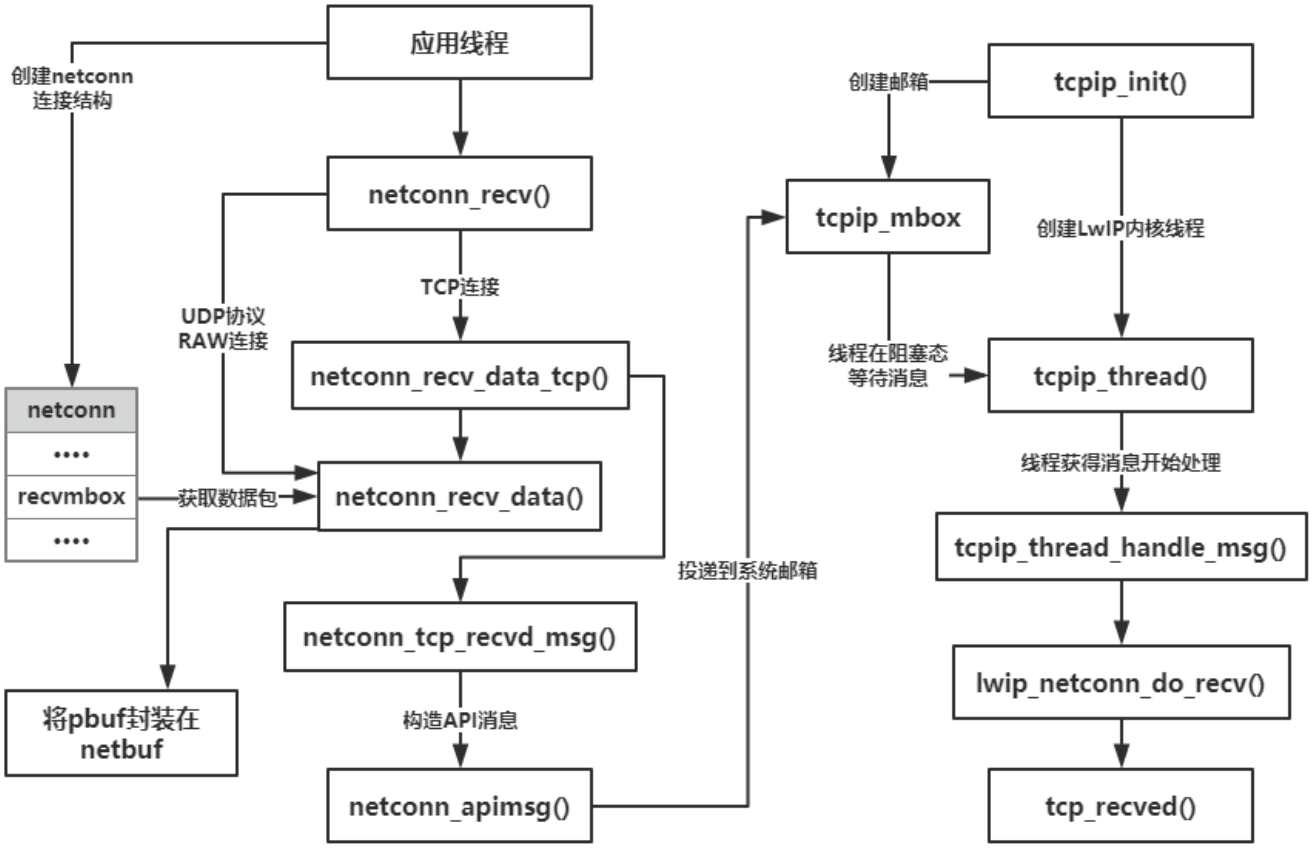


图 15-3 接收数据运作的示意图（已省略部分细节）

15.4.10 netconn_send()

整个数据发送函数我们在实际中使用的也是非常多的，它用于 UDP 协议、RAW 连接发送数据，通过参数 conn 选择指定的 UDP 或者 RAW 控制块发送参数 buf 中的数据，UDP/RAW 控制块中已经记录了目标 IP 地址与目标端口号了。这些数据被封装在 netbuf 中，如果没有使用 IP 数据报分片功能，那么这些数据不能太大，数据长度不能大于网卡最大传输单元 MTU，因为这个 API 目前还没有提供直接获取底层网卡最大传输单元 MTU 数值的函数，这就需要采用其它的途径来避免超过 MTU 值，所以规定了一个上限，即 netbuf 中包含的数据不能大于 1000 个字节，这就需要我们自己在发送数据的时候要注意，当然，使用了 IP 数据报分片功能的话，就不用管这些限制了。该函数会调用 netconn_apimsg() 函数构造一个 API 消息，并且请求内核执行 lwip_netconn_do_send() 函数，这个函数会通过消息得到目标 IP 地址与端口号以及 pbuf 数据报等

信息，然后调用 raw_send()/udp_send() 等函数发送数据，最后通过 netconn 连接结构的信号量进行同步，netconn_send() 函数源码具体见*代码清单 15_25*

代码清单 15-25netconn_send() 源码

```
err_t
netconn_send(struct netconn *conn, struct netbuf *buf)
{
    API_MSG_VAR_DECLARE(msg);
    err_t err;


    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;
    API_MSG_VAR_REF(msg).msg.b = buf;

    err = netconn_apimsg(lwip_netconn_do_send,
                         &API_MSG_VAR_REF(msg));

    API_MSG_VAR_FREE(msg);

    return err;
}
```

15.4.11 netconn_sendto()

这个函数与 netconn_send() 函数是一样的功能，只不过参数中直接指出目标 IP 地址与目标端口号，并且填写在 pbuf 中，具体见*代码清单 15_26*。

代码清单 15-26 netconn_sendto()

```
err_t
netconn_sendto(struct netconn *conn,
```

(下页继续)

(续上页)

```
                struct netbuf *buf,
                const ip_addr_t *addr,
                u16_t port)
{
    if (buf != NULL)
    {
        ip_addr_set(&buf->addr, addr);
        buf->port = port;
        return netconn_send(conn, buf);
    }
    return ERR_VAL;
}
```

15.4.12 netconn_write()

netconn_write() 函数的本质是一个宏，用于处于稳定连接状态的 TCP 协议发送数据，我们也知道，TCP 协议的数据是以流的方式传输的，只需要指出发送数据的其实地址与长度即可，LwIP 内核会帮我们直接处理这些数据，将这些数据按字节流进行编号，让它们按照 TCP 协议的方式进行传输，这样子就无需我们理会怎么传输了，对于数据的长度也没限制，内核会直接处理，使得它们变成最适的方式发送出去具体见

这个函数的功能是把 dataptr 指针指向的数据放在属于 conn 连接的 TCP 连接的发送队列中，size 参数指定了数据的长度，apiflags 参数有以下几种，具体见代码清单 *15_27*。

代码清单 15-27 apiflags 参数

```
/* 没有标志位（默认标志位） */
#define NETCONN_NOFLAG      0x00

/* 不拷贝数据到内核线程 */
#define NETCONN_NOCOPY      0x00
/* 拷贝数据到内核线程 */
```

(下页继续)

(续上页)

```
#define NETCONN_COPY        0x01

/* 尽快递交给上层应用 */
#define NETCONN_MORE        0x02

/* 当内核缓冲区满时，不会被阻塞，而是直接返回 */
#define NETCONN_DONTBLOCK   0x04

/* 不自动更新接收窗口，需要调用 netconn_tcp_recvd() 函数完成 */
#define NETCONN_NOAUTORCVD  0x08

/* 上层已经收到数据，将 FIN 保留在队列中直到再次调用 */
#define NETCONN_NOFIN       0x10
```

当 apiflags 的值为 NETCONN_COPY 时，dataptr 指针指向的数据将会被拷贝到为这些数据分配的内部缓冲区，这样的话，在调用本函数之后可以直接对这些数据进行修改而不会影响数据，但是拷贝的过程是需要消耗系统资源的，CPU 需要参与数据的拷贝，而且还会占用新的内存空间。

如果 apiflags 值为 NETCONN_NOCOPY，数据不会被拷贝而是直接使用 dataptr 指针来引用。但是这些数据在函数调用后不能立即被修改，因为这些数据可能会被放在当前 TCP 连接的重传队列中，以防对方未收到数据进行重传，而这段时间是不确定的。但是如果用户需要发送的数据在 ROM 中（静态数据），这样子就无需拷贝数据，直接引用数据即可。

如果 apiflags 值为 NETCONN_MORE，那么接收端在组装这些 TCP 报文段的时候，会将报文段首部的 PSH 标志置一，这样子，这些数据完成组装的时候，将会被立即递交给上层应用。

如果 apiflags 值为 NETCONN_DONTBLOCK，表示在内核发送缓冲区满的时候，再调用 netconn_write() 函数将不会被阻塞，而是会直接返回一个错误代码 ERR_VAL 告诉应用程序发送数据失败，应用程序可以自行处理这些数据，在适当的时候进行重传操作。

如果 apiflags 值为 NETCONN_NOAUTORCVD，表示在 TCP 协议接收到数据的时候，调用 netconn_recv_data_tcp() 函数的时候不会去更新接收窗口，只能由用户自己调用 netconn_tcp_recvd() 函数完成接收窗口的更新操作。

netconn_write() 函数的源码具体见*代码清单 15_28*，但是真正实现这些功能的函数是 netconn_write_vectors_partly()。

代码清单 15-28 netconn_write() 源码

```
err_t
netconn_write_partly(struct netconn *conn,
                     const void *dataptr,
                     size_t size,
                     u8_t apiflags,
                     size_t *bytes_written)
{
    struct netvector vector;
    vector.ptr = dataptr;
    vector.len = size;
    return netconn_write_vectors_partly(conn,
                                        &vector,
                                        1,
                                        apiflags,
                                        bytes_written);
}

err_t
netconn_write_vectors_partly(struct netconn *conn,
                             struct netvector *vectors,
                             u16_t vectorcnt,
                             u8_t apiflags,
                             size_t * bytes_written)
{
    API_MSG_VAR_DECLARE(msg);
    err_t err;
    u8_t dontblock;
    size_t size;
    int i;
```

(下页继续)

(续上页)

```
    dontblock = netconn_is_nonblocking(conn)
                || (apiflags & NETCONN_DONTBLOCK);

    if (dontblock && !bytes_written)
    {
        /* 不允许非阻塞写入 */
        return ERR_VAL;
    }

    /* 计算总大小 */
    size = 0;
    for (i = 0; i < vectorcnt; i++)
    {
        size += vectors[i].len;
        if (size < vectors[i].len)
        {
            /* 溢出 */
            return ERR_VAL;
        }
    }
    if (size == 0)
    {
        return ERR_OK;
    }
    else if (size > SSIZE_MAX)
    {
        ssize_t limited;
        /*  */
        if (!bytes_written)
        {
            return ERR_VAL;
```

(下页继续)

(续上页)

```
        }
        /* 限制最大长度 65535,TCP 数据流最大能记录这么大的数据 */
        limited = SSIZE_MAX;
        size = (size_t)limited;
    }

    API_MSG_VAR_ALLOC(msg);
    /* 尽可能多发送数据 */
    API_MSG_VAR_REF(msg).conn = conn;
    API_MSG_VAR_REF(msg).msg.w.vector = vectors;
    API_MSG_VAR_REF(msg).msg.w.vector_cnt = vectorcnt;
    API_MSG_VAR_REF(msg).msg.w.vector_off = 0;
    API_MSG_VAR_REF(msg).msg.w.apiflags = apiflags;
    API_MSG_VAR_REF(msg).msg.w.len = size;
    API_MSG_VAR_REF(msg).msg.w.offset = 0;

    /* 请求内核执行 lwip_netconn_do_write() 函数 */
    err = netconn_apimsg(lwip_netconn_do_write,
                         &API_MSG_VAR_REF(msg));
    if (err == ERR_OK)
    {
        if (bytes_written != NULL)
        {
            *bytes_written = API_MSG_VAR_REF(msg).msg.w.offset;
        }
    }
    API_MSG_VAR_FREE(msg);

    return err;
}
```

15.4.13 netconn_close()

netconn_close() 函数用于主动终止一个 TCP 连接，它通过调用 netconn_apimsg() 函数构造一个 API 消息，并且请求内核执行 lwip_netconn_do_close() 函数，然后通过 netconn 连接结构的信号量进行同步，内核会完成终止 TCP 连接的全过程，无需我们理会，具体见*代码清单 15_29*。

代码清单 15-29 netconn_close()

```
err_t
netconn_close(struct netconn *conn)
{
    return netconn_close_shutdown(conn, NETCONN_SHUT_RDWR);
}

static err_t
netconn_close_shutdown(struct netconn *conn, u8_t how)
{
    API_MSG_VAR_DECLARE(msg);
    err_t err;
    LWIP_UNUSED_ARG(how);


    API_MSG_VAR_ALLOC(msg);
    API_MSG_VAR_REF(msg).conn = conn;

    API_MSG_VAR_REF(msg).msg.sd.shut = how;

    API_MSG_VAR_REF(msg).msg.sd.polls_left =
        ((LWIP_TCP_CLOSE_TIMEOUT_MS_DEFAULT +
        TCP_SLOW_INTERVAL - 1) / TCP_SLOW_INTERVAL) + 1;

    err = netconn_apimsg(lwip_netconn_do_close, &API_MSG_VAR_REF(msg));
    API_MSG_VAR_FREE(msg);
```

(下页继续)

(续上页)

```
    return err;
}
```

15.5 实验

讲解了那么多 API，是时候来一次实战演练了，下面我们将使用 NETCONN API 搭建一个 TCP 客户端、TCP 服务器、以及一个 UDP 收发数据。

15.5.1 TCP Client

在这个实验中，我们将用野火开发板实现一个 TCP 客户端，去连接我们电脑的 TCP 上位机模拟的服务器，首先准备我们已经移植好的工程模板，在工程模板中添加两个文件，分别为 client.c 和 client.h，并且在 client.c 文件中添加*代码清单 15_30* 的内容。

代码清单 15-30client.c 文件内容

```
#include "client.h"

#include "lwip/opt.h"

#include "lwip/sys.h"
#include "lwip/api.h"

static void client(void *thread_param)
{
    struct netconn *conn;
    int ret;
    ip4_addr_t ipaddr;
```

(下页继续)

(续上页)

```
    uint8_t send_buf[]= "This is a TCP Client test...\n";

    while (1)
    {
        conn = netconn_new(NETCONN_TCP);                (1)
        if (conn == NULL)
        {
            printf("create conn failed!\n");
            vTaskDelay(10);
            continue;
        }

        IP4_ADDR(&ipaddr,192,168,0,181);                (2)

        ret = netconn_connect(conn,&ipaddr,5001);       (3)
        if (ret == -1)
        {
            printf("Connect failed!\n");
            netconn_close(conn);                        (4)
            vTaskDelay(10);
            continue;
        }

        printf("Connect to iperf server successful!\n");

        while (1)
        {
            ret = netconn_write(conn,send_buf,sizeof(send_buf),0); (5)

            vTaskDelay(1000);
        }
    }
```

(下页继续)

(续上页)

```
}

void
client_init(void)
{
    sys_thread_new("client", client, NULL, 512, 4);
}
```

代码清单 15-31client.h 文件内容

```
#ifndef _CLIENT_H
#define _CLIENT_H

void client_init(void);

#endif /* _CLIENT_H */
```

代码清单 15-30**(1)**：调用 netconn_new() 函数申请一个连接结构，指定参数是 NETCONN_TCP，即 TCP 连接，在这里将初始化 netconn 连接结构的相关参数，但并不会产生连接，返回指向连接结构的指针并且保存在 conn 中。

代码清单 15-30**(2)**：配置服务器的 IP 地址。

代码清单 15-30**(3)**：调用 netconn_connect() 函数连接服务器，指定端口为 5001。

代码清单 15-30**(4)**：如果连接失败就删除 conn 连接结构，然后尝试重连。

代码清单 15-30**(5)**：连接成功后，每隔 1 秒调用 netconn_write() 函数发送数据到服务器中。

除此之外，我们还需要配置一些宏定义，首先打开 lwipopts.h 配置头文件，在里面添加下面已经宏定义配置：

```
#define LWIP_NETCONN 1
```

然后我们打开工程的 sys_arch.h 头文件，根据自己的实际网络情况配置开发板的 IP 地址，网关等信息，具体见代码清单 *15_32* 加粗部分，如果不知道自己网络情况的话，可以通过 cmd 控制台，输入 ipconfig 就可以知道开发板所处的网络环境，这样子根据实际情况修改 IP 地址与网关即可，但是千万不要将 IP 地址改成与电脑 IP 地址是一致的，电脑的 IP 地址是我们要使用的连接服务器的 IP 地址，具体见图 *15_4*。

代码清单 15-32sys_arch.h 文件内容

```
#include <lwip/opt.h>
#include <lwip/arch.h>
#include "tcpip.h"

/* FreeRTOS 头文件 */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"

/* USER CODE BEGIN 0 */
#define DEST_IP_ADDR0               192
#define DEST_IP_ADDR1               168
#define DEST_IP_ADDR2                 0
#define DEST_IP_ADDR3               181

#define DEST_PORT                  6000

#define UDP_SERVER_PORT            5002
#define UDP_CLIENT_PORT            5002

#define LOCAL_PORT                 5001

/*Static IP ADDRESS: IP_ADDR0.IP_ADDR1.IP_ADDR2.IP_ADDR3 */
#define IP_ADDR0                    192
```

(下页继续)

(续上页)

```
#define IP_ADDR1                    168
#define IP_ADDR2                      0
#define IP_ADDR3                    122

/*NETMASK*/
#define NETMASK_ADDR0               255
#define NETMASK_ADDR1               255
#define NETMASK_ADDR2               255
#define NETMASK_ADDR3                 0

/*Gateway Address*/
#define GW_ADDR0                    192
#define GW_ADDR1                    168
#define GW_ADDR2                      0
#define GW_ADDR3                      1
/* USER CODE END 0 */

#define SYS_MBOX_NULL  (QueueHandle_t)0
#define SYS_SEM_NULL   (SemaphoreHandle_t)0
#define SYS_MRTEX_NULL SYS_SEM_NULL
#define SYS_DEFAULT_THREAD_STACK_DEPTH  configMINIMAL_STACK_SIZE

typedef SemaphoreHandle_t sys_sem_t;
typedef SemaphoreHandle_t sys_mutex_t;
typedef QueueHandle_t sys_mbox_t;
typedef TaskHandle_t sys_thread_t;

typedef int sys_prot_t;
void TCPIP_Init(void);
```

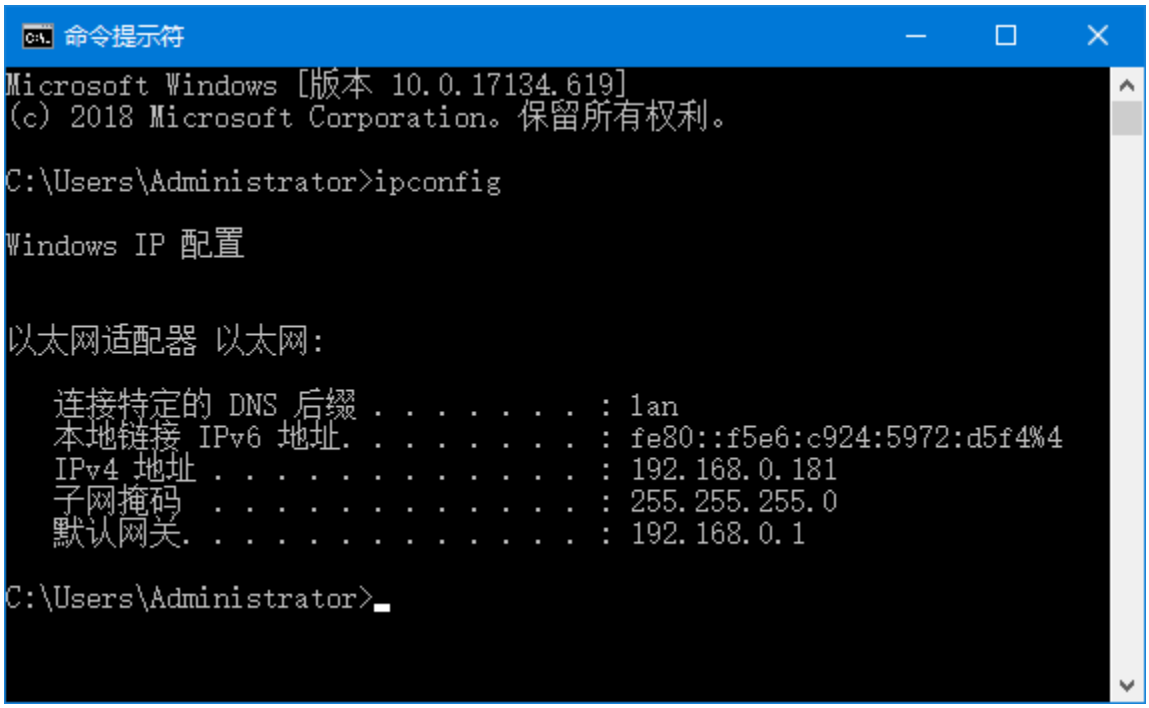


图 15-4 查看网络环境信息

同时在 main.c 文件中添加测试代码，关于 TCP 客户端的代码具体见*代码清单 15_33* 加粗部分。

代码清单 15-33main.c 文件内容

```
#include "main.h"
/* FreeRTOS 头文件 */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "client.h"
/**************************** 任务句柄 ********************************/
/*
* 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
* 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
* 这个句柄可以为 NULL。
*/
```

(下页继续)

(续上页)

```
static TaskHandle_t AppTaskCreate_Handle = NULL;/* 创建任务句柄 */
static TaskHandle_t Test1_Task_Handle = NULL;/* LED 任务句柄 */
static TaskHandle_t Test2_Task_Handle = NULL;/* KEY 任务句柄 */
/*
*************************************************************************
*                              函数声明
*************************************************************************
*/
static void AppTaskCreate(void);/* 用于创建任务 */

static void Test1_Task(void* pvParameters);/* Test1_Task 任务实现 */
static void Test2_Task(void* pvParameters);/* Test2_Task 任务实现 */

extern void TCPIP_Init(void);

/***********************************************************************
* @brief  主函数
* @param  无
* @retval 无
* @note   第一步：开发板硬件初始化
          第二步：创建 APP 应用任务
          第三步：启动 FreeRTOS，开始多任务调度
***********************************************************************/
int main(void)
{
    BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为 pdPASS */

    /* 开发板硬件初始化 */
    BSP_Init();

    /* 创建 AppTaskCreate 任务 */
    xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate,  /* 任务入口函数 */
```

(下页继续)

(续上页)

```
                        (const char*     )"AppTaskCreate",/* 任务名字 */
                        (uint16_t        )512,  /* 任务栈大小 */
                        (void*           )NULL,/* 任务入口函数参数 */
                        (UBaseType_t     )1, /* 任务的优先级 */
                        (TaskHandle_t*   )&AppTaskCreate_Handle);
    /* 启动任务调度 */
    if (pdPASS == xReturn)
        vTaskStartScheduler();   /* 启动任务，开启调度 */
    else
        return -1;

    while (1);  /* 正常不会执行到这里 */
}


/***********************************************************************
* @ 函数名  ： AppTaskCreate
* @ 功能说明： 为了方便管理，所有的任务创建函数都放在这个函数里面
* @ 参数    ： 无
* @ 返回值  ： 无
**********************************************************************/
static void AppTaskCreate(void)
{
    BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为 pdPASS */

    TCPIP_Init();

    client_init();

    taskENTER_CRITICAL();           //进入临界区

    /* 创建 Test1_Task 任务 */
```

(下页继续)

(续上页)

```
    xReturn = xTaskCreate((TaskFunction_t )Test1_Task, /* 任务入口函数 */
                          (const char*    )"Test1_Task",/* 任务名字 */
                          (uint16_t       )512,   /* 任务栈大小 */
                          (void*          )NULL,  /* 任务入口函数参数 */
                          (UBaseType_t    )1,     /* 任务的优先级 */
                          (TaskHandle_t*  )&Test1_Task_Handle);
    if (pdPASS == xReturn)
        printf("Create Test1_Task sucess...\r\n");

    /* 创建 Test2_Task 任务 */
    xReturn = xTaskCreate((TaskFunction_t )Test2_Task,  /* 任务入口函数 */
                          (const char*    )"Test2_Task",/* 任务名字 */
                          (uint16_t       )512,  /* 任务栈大小 */
                          (void*          )NULL,/* 任务入口函数参数 */
                          (UBaseType_t    )2, /* 任务的优先级 */
                          (TaskHandle_t*  )&Test2_Task_Handle);
    if (pdPASS == xReturn)
        printf("Create Test2_Task sucess...\n\n");

    vTaskDelete(AppTaskCreate_Handle); //删除 AppTaskCreate 任务

    taskEXIT_CRITICAL();            //退出临界区
}

/**********************************************************************
* @ 函数名  ： Test1_Task
* @ 功能说明： Test1_Task 任务主体
* @ 参数    ：
* @ 返回值  ： 无
**********************************************************************/
static void Test1_Task(void* parameter)
{
```

(下页继续)

(续上页)

```
    while (1)
    {
        LED1_TOGGLE;
//    PRINT_DEBUG("LED1_TOGGLE\n");
        vTaskDelay(1000);/* 延时 1000 个 tick */
    }
}

/**********************************************************************
* @ 函数名  ： Test2_Task
* @ 功能说明： Test2_Task 任务主体
* @ 参数    ：
* @ 返回值  ： 无
**********************************************************************/
static void Test2_Task(void* parameter)
{
    while (1)
    {
        LED2_TOGGLE;
//    PRINT_DEBUG("LED2_TOGGLE\n");
        vTaskDelay(2000);/* 延时 2000 个 tick */
    }
}
```

15.5.2 TCP Client 实验现象

将工程编译好，然后下载到开发板中，打开我们提供的网络调试工具——USR-TCP232-Test，然后在软件中按图 *15_5* 的步骤操作，按下开发板的复位按键，就得到实验现象，具体见图 *15_6*，刚好与我们写的程序是一致的。



图 15-5 配置 USR-TCP232-Test 调试工具

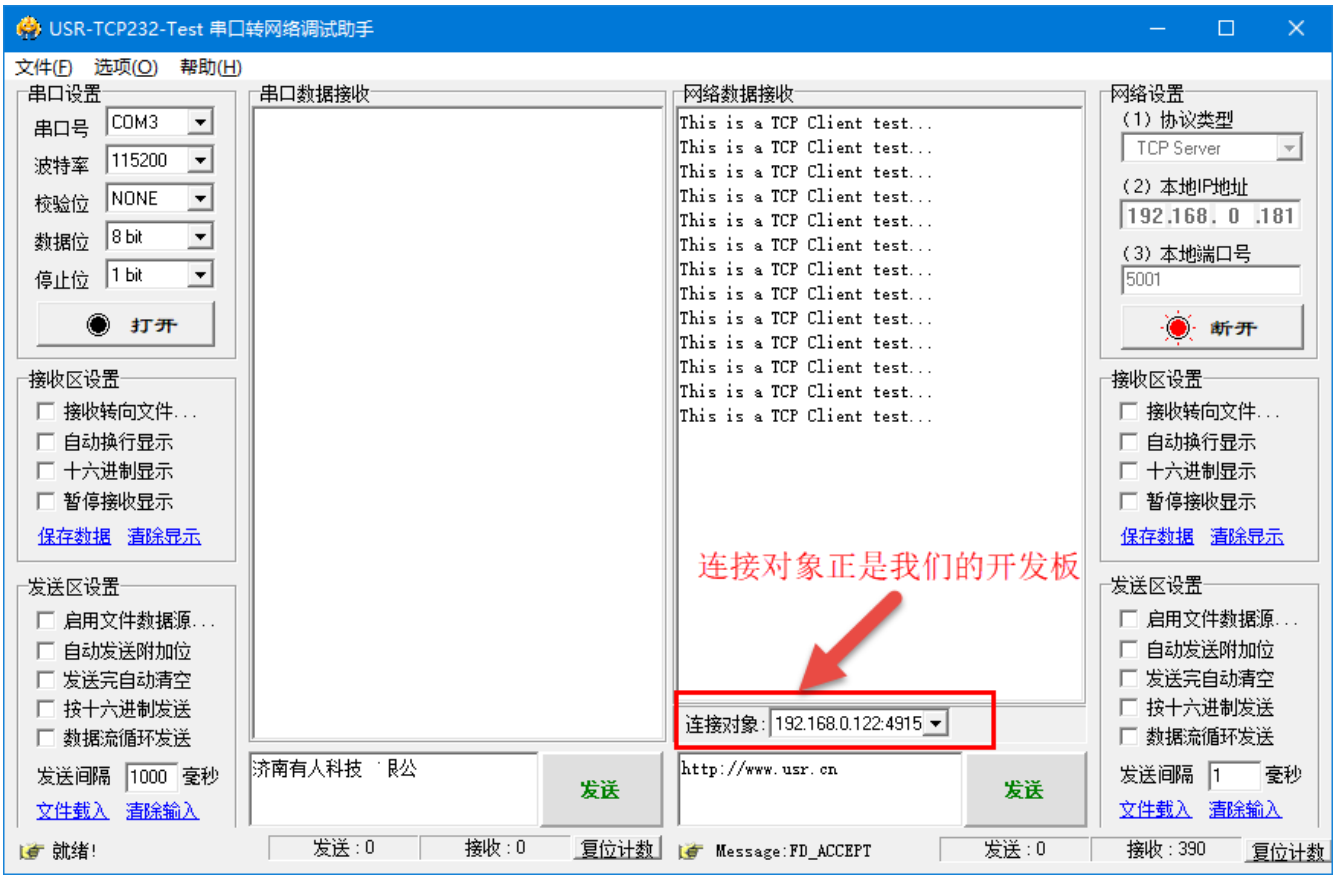


图 15-6 TCP Client 实验现象

15.5.3 TCP Server

在这个实验中，我们将用野火开发板实现一个 TCP 服务器，用电脑的 TCP 上位机模拟 TCP 客户端来连接我们的开发板，首先准备我们已经移植好的工程模板，在工程模板中添加两个文件，分别为 tcpecho.c 和 tcpecho.h，并且在 tcpecho.c 文件中添加代码清单 15-34 所示的代码，在 tcpecho.h 添加代码清单 15-35 所示的代码。

代码清单 15-34 tcpecho.c 文件内容

```
#include "tcpecho.h"
```

(下页继续)

(续上页)

```
#include "lwip/opt.h"

#if LWIP_NETCONN

#include "lwip/sys.h"
#include "lwip/api.h"
/*-----------------------------------------------------------------------*/
static void
tcpecho_thread(void *arg)
{
    struct netconn *conn, *newconn;
    err_t err;
    LWIP_UNUSED_ARG(arg);

    /* Create a new connection identifier. */
    /* Bind connection to well known port number 7. */
#if LWIP_IPV6
    conn = netconn_new(NETCONN_TCP_IPV6);
    netconn_bind(conn, IP6_ADDR_ANY, 5001);
#else /* LWIP_IPV6 */
    conn = netconn_new(NETCONN_TCP);                          (1)
    netconn_bind(conn, IP_ADDR_ANY, 5001);          (2)
#endif /* LWIP_IPV6 */
    LWIP_ERROR("tcpecho: invalid conn", (conn != NULL), return;);

    /* Tell connection to go into listening mode. */
    netconn_listen(conn);                                   (3)

    while (1)
    {

        /* Grab new connection. */
```

(下页继续)

(续上页)

```
        err = netconn_accept(conn, &newconn);                    (4)
        /*printf("accepted new connection %p\n", newconn);*/
        /* Process the new connection. */
        if (err == ERR_OK)
        {
            struct netbuf *buf;
            void *data;
            u16_t len;

            while ((err = netconn_recv(newconn, &buf)) == ERR_OK) (5)
            {
                /*printf("Recved\n");*/
                do
                {
                netbuf_data(buf, &data, &len);                   (6)
                err= netconn_write(newconn, data, len, NETCONN_COPY); (7)
#if 0
                    if (err != ERR_OK)
                    {
                        printf("tcpecho: netconn_write: error \"%s\"\n",
                            lwip_strerr(err));
                    }
#endif
                }
                while (netbuf_next(buf) >= 0);                          (8)
                netbuf_delete(buf);                              (9)
            }
            /*printf("Got EOF, looping\n");*/
            /* Close connection and discard connection identifier. */
            netconn_close(newconn);                              (10)
            netconn_delete(newconn);                                    (11)
        }
```

(下页继续)

(续上页)

```
    }
}
/*-----------------------------------------------------------------------*/
void
tcpecho_init(void)
{
    sys_thread_new("tcpecho_thread", tcpecho_thread, NULL, 512, 4);
}
/*-----------------------------------------------------------------------*/

#endif /* LWIP_NETCONN */
```

代码清单 15-35 tcpecho.h 文件内容

```
#ifndef LWIP_TCPECHO_H
#define LWIP_TCPECHO_H

void tcpecho_init(void);

#endif /* LWIP_TCPECHO_H */
```

我们来分析一下 tcpecho.c 的代码：

代码清单 15-34**(1)**：调用 netconn_new() 函数申请一个连接结构，指定参数是 NETCONN_TCP，即 TCP 连接，在这里将初始化 netconn 连接结构的相关参数，但并不会产生连接，返回指向连接结构的指针并且保存在 conn 中。

代码清单 15-34**(2)**：调用 netconn_bind() 函数绑定本地的 IP 地址与端口号，在这里我们指定任意一个网卡的 IP 地址 IP_ADDR_ANY，我们的网卡只有一个（以太网），那 IP 地址就是我们设定的网卡 IP 地址了，端口号指定为 5001。

代码清单 15-34**(3)**：调用 netconn_listen() 函数使得 TCP 服务器进入监听状态。

代码清单 15-34**(4)**：通过 netconn_accept() 函数处理客户端的连接请求，当只有当有客户端发送连

接请求的时候才会处理，否则将进入阻塞态，而客户端的信息保存在 newconn 连接结构中。

代码清单 15-34**(5)**：程序来到这里，说明已经有客户端连接了，那么就调用 netconn_recv() 函数来接收 newconn 客户端发来的数据。

代码清单 15-34**(6)**：如果接收到数据，我们用 netbuf_data() 函数将这些数据保存到 data 中，长度为 len。

代码清单 15-34**(7)**：再调用 netconn_write() 函数将数据返回给 newconn 客户端，信息为 data，长度为 len，并且指定为不拷贝方式 NETCONN_COPY。

代码清单 15-34**(8)**：可能客户端发送的数据很多，可能 netbuf 中还有数据，那就调用 netbuf_next() 函数移动 ptr 指针，指向下一个 pbuf。

代码清单 15-34**(9)**：循环处理完所有的消息，那就调用 netbuf_delete() 函数释放这些数据区域空间。

代码清单 15-34**(10)**：调用 netconn_close() 函数主动关闭与客户端的连接。

代码清单 15-34**(11)**：调用 netconn_delete() 函数释放 newconn 的空间。

提示：在代码中要注意的是，我们是单独使用一个线程来处理这些事情的，因此需要我们熟悉操作系统相关的知识，野火也有很多本 RTOS 系列的书籍，建议先学操作系统在学习 LwIP。

同样的，需要在 sys_arch.h 头文件中配置网络 IP 地址等相关参数，并且需要在 main.c 文件中添加测试代码，关于 TCP 服务器的代码具体见*代码清单 15_36* 加粗部分。

代码清单 15-36main.c 文件内容

```
#include "main.h"
/* FreeRTOS 头文件 */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

#include "tcpecho.h"
/**************************** 任务句柄 ********************************/
/*
```

(下页继续)

(续上页)

```
* 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
* 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
* 这个句柄可以为 NULL。
*/
static TaskHandle_t AppTaskCreate_Handle = NULL;/* 创建任务句柄 */
static TaskHandle_t Test1_Task_Handle = NULL;/* LED 任务句柄 */
static TaskHandle_t Test2_Task_Handle = NULL;/* KEY 任务句柄 */

/*
*************************************************************************
*                              函数声明
*************************************************************************
*/
static void AppTaskCreate(void);/* 用于创建任务 */

static void Test1_Task(void* pvParameters);/* Test1_Task 任务实现 */
static void Test2_Task(void* pvParameters);/* Test2_Task 任务实现 */

extern void TCPIP_Init(void);

/***********************************************************************
* @brief  主函数
* @param  无
* @retval 无
* @note   第一步：开发板硬件初始化
          第二步：创建 APP 应用任务
          第三步：启动 FreeRTOS，开始多任务调度
***********************************************************************/
int main(void)
{
    BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为 pdPASS */
```

(下页继续)

(续上页)

```
    /* 开发板硬件初始化 */
    BSP_Init();

    /* 创建 AppTaskCreate 任务 */
    xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
                          (const char*    )"AppTaskCreate",/* 任务名字 */
                          (uint16_t       )512,  /* 任务栈大小 */
                          (void*          )NULL,/* 任务入口函数参数 */
                          (UBaseType_t    )1, /* 任务的优先级 */
                          (TaskHandle_t*  )&AppTaskCreate_Handle);
    /* 启动任务调度 */
    if (pdPASS == xReturn)
        vTaskStartScheduler();   /* 启动任务，开启调度 */
    else
        return -1;

    while (1);  /* 正常不会执行到这里 */
}


/***********************************************************************
* @ 函数名  ： AppTaskCreate
* @ 功能说明： 为了方便管理，所有的任务创建函数都放在这个函数里面
* @ 参数    ： 无
* @ 返回值  ： 无
**********************************************************************/
static void AppTaskCreate(void)
{
    BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为 pdPASS */

    TCPIP_Init();
```

(下页继续)

(续上页)

```
    tcpecho_init();          //创建 tcp 服务器线程

    taskENTER_CRITICAL();           //进入临界区

    /* 创建 Test1_Task 任务 */
    xReturn = xTaskCreate((TaskFunction_t )Test1_Task, /* 任务入口函数 */
                          (const char*    )"Test1_Task",/* 任务名字 */
                          (uint16_t       )512,   /* 任务栈大小 */
                          (void*          )NULL,  /* 任务入口函数参数 */
                          (UBaseType_t    )1,     /* 任务的优先级 */
                          (TaskHandle_t*  )&Test1_Task_Handle);
    if (pdPASS == xReturn)
        printf("Create Test1_Task sucess...\r\n");

    /* 创建 Test2_Task 任务 */
    xReturn = xTaskCreate((TaskFunction_t )Test2_Task,  /* 任务入口函数 */
                          (const char*    )"Test2_Task",/* 任务名字 */
                          (uint16_t       )512,  /* 任务栈大小 */
                          (void*          )NULL,/* 任务入口函数参数 */
                          (UBaseType_t    )2, /* 任务的优先级 */
                          (TaskHandle_t*  )&Test2_Task_Handle);
    if (pdPASS == xReturn)
        printf("Create Test2_Task sucess...\n\n");

    vTaskDelete(AppTaskCreate_Handle); //删除 AppTaskCreate 任务

    taskEXIT_CRITICAL();            //退出临界区
}

static void Test1_Task(void* parameter)
{
    while (1)
```

(下页继续)

(续上页)

```
    {
        LED1_TOGGLE;
//    PRINT_DEBUG("LED1_TOGGLE\n");
        vTaskDelay(1000);/* 延时 1000 个 tick */
    }
}

static void Test2_Task(void* parameter)
{
    while (1)
    {
        LED2_TOGGLE;
//    PRINT_DEBUG("LED2_TOGGLE\n");
        vTaskDelay(2000);/* 延时 2000 个 tick */
    }
}
```

15.5.4 TCP Server 实验现象

将工程编译好，然后下载到开发板中，打开我们提供的网络调试工具——USR-TCP232-Test，然后在软件中按图 *15_7* 的步骤操作，按下开发板的复位按键，然后点击上位机的连接按钮，就得到实验现象，具体见图 *15_8* ，刚好与我们写的程序是一致的。

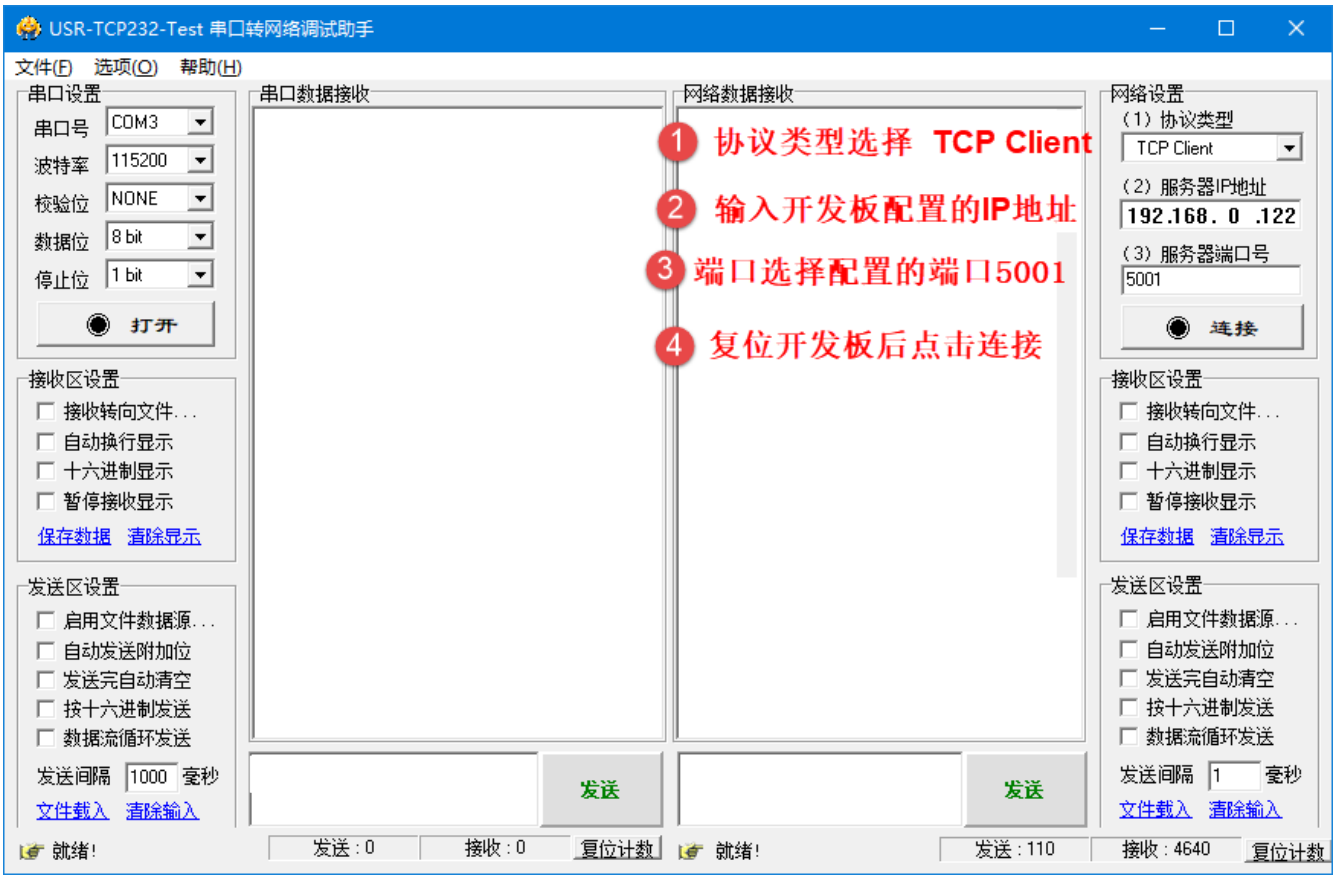


图 15-7 USR-TCP232-Test 工具操作步骤

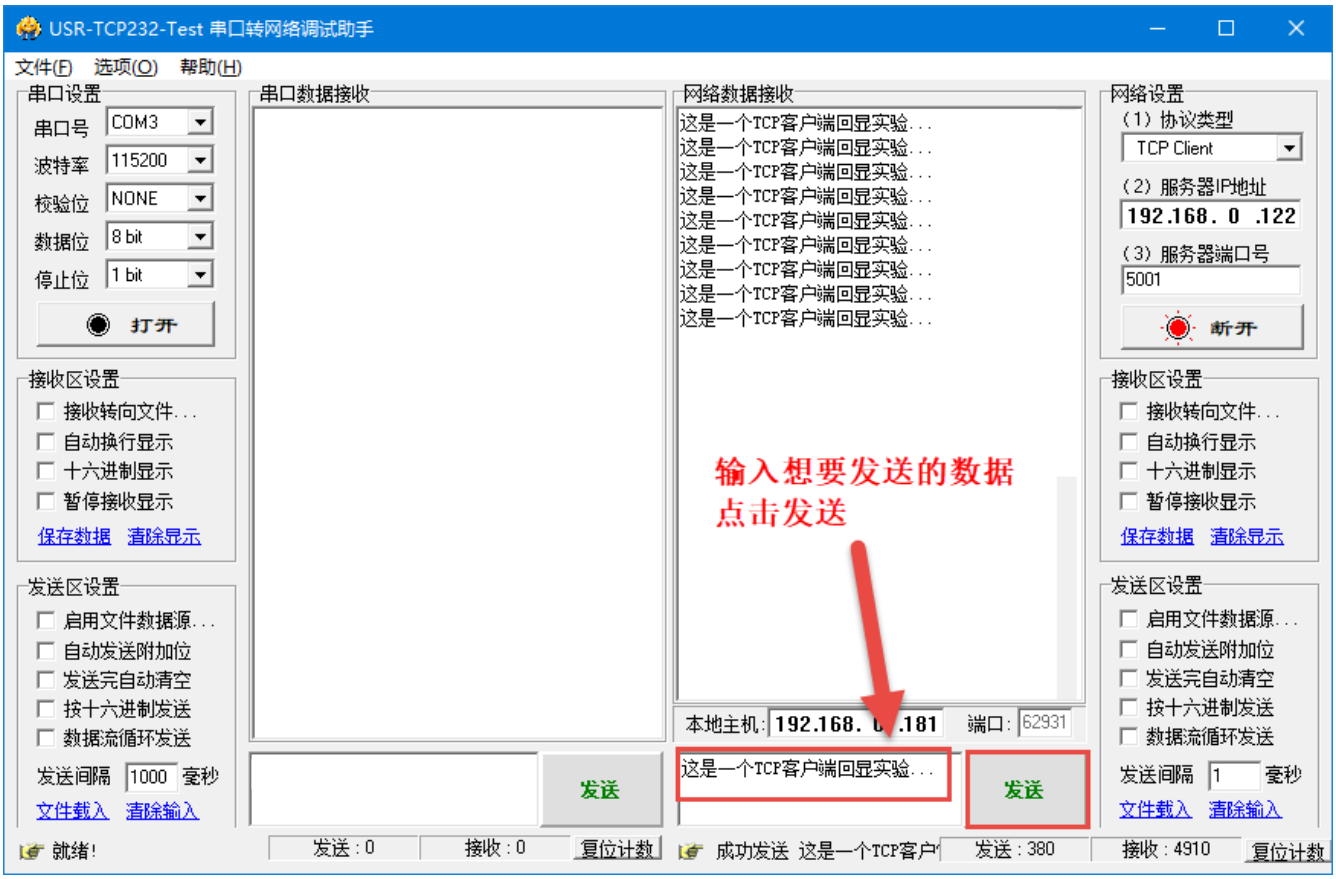


图 15-8 TCP Server 实验现象

15.5.5 UDP

在这个实验中，我们将用野火开发板实现一个 UDP 协议通信，用电脑的 UDP 协议上位机与我们的开发板进行通讯，因为 UDP 协议是没有任何连接的，因此我们在电脑上位机发送数据到开发板，然后开发板返回应用的数据，如果现实是一样的，那么表示通信正确，首先准备我们已经移植好的工程模板，在工程模板中添加两个文件，分别为 udpecho.c 和 udpecho.h，并且在 dupecho.c 文件中添加*代码清单 15_37* 所示的代码，在 udpecho.h 添加*代码清单 15_38* 所示的代码。

代码清单 15-37dupecho.c 文件内容

```
#include "udpecho.h"
```

(下页继续)

(续上页)

```
#include "lwip/opt.h"

#if LWIP_NETCONN

#include "lwip/api.h"
#include "lwip/sys.h"

/*-----------------------------------------------------------------------*/
static void
udpecho_thread(void *arg)
{
    struct netconn *conn;
    struct netbuf *buf;
    char buffer[4096];
    err_t err;
    LWIP_UNUSED_ARG(arg);

#if LWIP_IPV6
    conn = netconn_new(NETCONN_UDP_IPV6);
    netconn_bind(conn, IP6_ADDR_ANY, 5001);
#else /* LWIP_IPV6 */
    conn = netconn_new(NETCONN_UDP);
    netconn_bind(conn, IP_ADDR_ANY, 5001);
#endif /* LWIP_IPV6 */
    LWIP_ERROR("udpecho: invalid conn", (conn != NULL), return;);

    while (1)
    {
        err = netconn_recv(conn, &buf);
        if (err == ERR_OK)
        {
```

(下页继续)

(续上页)

```
            if (netbuf_copy(buf, buffer, sizeof(buffer))
                    != buf->p->tot_len)
            {
                LWIP_DEBUGF(LWIP_DBG_ON, ("netbuf_copy failed\n"));
            }
            else
            {
                buffer[buf->p->tot_len] = '\0';
                err = netconn_send(conn, buf);
                if (err != ERR_OK)
                {
                LWIP_DEBUGF(LWIP_DBG_ON, ("netconn_send failed: %d\n",␣
↪(int)err));
                }
                else
                {
                    LWIP_DEBUGF(LWIP_DBG_ON, ("got %s\n", buffer));
                }
            }
            netbuf_delete(buf);
        }
    }
}
/*-----------------------------------------------------------------------*/
void
udpecho_init(void)
{
    sys_thread_new("udpecho_thread", udpecho_thread, NULL, 2048, 4);
}

#endif /* LWIP_NETCONN */
```

代码清单 15-38udpecho.h 文件内容

```
#ifndef LWIP_UDPECHO_H
#define LWIP_UDPECHO_H

void udpecho_init(void);

#endif /* LWIP_UDPECHO_H */
```

这个 UDP 实验是非常简单的，都是首先通过 netconn_new() 函数创建一个 UDP 类型的连接结构（UDP 的目标 IP 地址与端口号是保存在 UDP 控制块中的），然后调用 netconn_bind() 函数进行绑定本地 IP 地址与端口，直接接受数据即可，无需连接，通过 netconn_recv() 函数接收数据。然后把接收到的数据拷贝出来，最后再调用 netconn_send() 函数返回给目标主机。

当然，我们也不能忘记修改 main.c 文件，具体见*代码清单 15_39* 加粗部分。

代码清单 15-39main.c 文件内容

```
#include "main.h"
/* FreeRTOS 头文件 */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "udpecho.h"

/**************************** 任务句柄 ********************************/
/*
* 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
* 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
* 这个句柄可以为 NULL。
*/
static TaskHandle_t AppTaskCreate_Handle = NULL;/* 创建任务句柄 */
static TaskHandle_t Test1_Task_Handle = NULL;/* LED 任务句柄 */
static TaskHandle_t Test2_Task_Handle = NULL;/* KEY 任务句柄 */
```

(下页继续)

(续上页)

```
/*
*************************************************************************
*                              函数声明
*************************************************************************
*/
static void AppTaskCreate(void);/* 用于创建任务 */

static void Test1_Task(void* pvParameters);/* Test1_Task 任务实现 */
static void Test2_Task(void* pvParameters);/* Test2_Task 任务实现 */

extern void TCPIP_Init(void);

/***********************************************************************
* @brief  主函数
* @param  无
* @retval 无
* @note   第一步：开发板硬件初始化
          第二步：创建 APP 应用任务
          第三步：启动 FreeRTOS，开始多任务调度
***********************************************************************/
int main(void)
{
    BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为 pdPASS */

    /* 开发板硬件初始化 */
    BSP_Init();

    /* 创建 AppTaskCreate 任务 */
    xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate,  /* 任务入口函数 */
                          (const char*    )"AppTaskCreate",/* 任务名字 */
                          (uint16_t       )512,  /* 任务栈大小 */
```

(下页继续)

(续上页)

```
                        (void*           )NULL,/* 任务入口函数参数 */
                        (UBaseType_t     )1, /* 任务的优先级 */
                        (TaskHandle_t*   )&AppTaskCreate_Handle); 53      /*␣
↪启
务调度 */
    if (pdPASS == xReturn)
        vTaskStartScheduler();   /* 启动任务，开启调度 */
    else
        return -1;

    while (1);  /* 正常不会执行到这里 */
}


/***********************************************************************
* @ 函数名  ： AppTaskCreate
* @ 功能说明： 为了方便管理，所有的任务创建函数都放在这个函数里面
* @ 参数    ： 无
* @ 返回值  ： 无
**********************************************************************/
static void AppTaskCreate(void)
{
    BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为 pdPASS */

    TCPIP_Init();

    udpecho_init();

    taskENTER_CRITICAL();           //进入临界区

    /* 创建 Test1_Task 任务 */
    xReturn = xTaskCreate((TaskFunction_t )Test1_Task, /* 任务入口函数 */
```

(下页继续)

(续上页)

```
                        (const char*    )"Test1_Task",/* 任务名字 */
                        (uint16_t       )512,   /* 任务栈大小 */
                        (void*          )NULL,  /* 任务入口函数参数 */
                        (UBaseType_t    )1,     /* 任务的优先级 */
                        (TaskHandle_t*  )&Test1_Task_Handle);
    if (pdPASS == xReturn)
        printf("Create Test1_Task sucess...\r\n");

    /* 创建 Test2_Task 任务 */
    xReturn = xTaskCreate((TaskFunction_t )Test2_Task,  /* 任务入口函数 */
                        (const char*    )"Test2_Task",/* 任务名字 */
                        (uint16_t       )512,  /* 任务栈大小 */
                        (void*          )NULL,/* 任务入口函数参数 */
                        (UBaseType_t    )2, /* 任务的优先级 */
                        (TaskHandle_t*  )&Test2_Task_Handle);
    if (pdPASS == xReturn)
        printf("Create Test2_Task sucess...\n\n");

    vTaskDelete(AppTaskCreate_Handle); //删除 AppTaskCreate 任务

    taskEXIT_CRITICAL();            //退出临界区
}

static void Test1_Task(void* parameter)
{
    while (1)
    {
        LED1_TOGGLE;
//    PRINT_DEBUG("LED1_TOGGLE\n");
        vTaskDelay(1000);/* 延时 1000 个 tick */
    }
}
```

(下页继续)

(续上页)

```
static void Test2_Task(void* parameter)
{
    while (1)
    {
        LED2_TOGGLE;
//    PRINT_DEBUG("LED2_TOGGLE\n");
        vTaskDelay(2000);/* 延时 2000 个 tick */
    }
}
```

15.5.6 UDP 实验现象

将工程编译好，然后下载到开发板中，打开我们提供的网络调试工具——USR-TCP232-Test，然后在软件中按图 *15_9* 的步骤操作，按下开发板的复位按键，然后点击上位机的连接按钮，就得到实验现象，具体见图 *15_10*，刚好与我们写的程序是一致的。

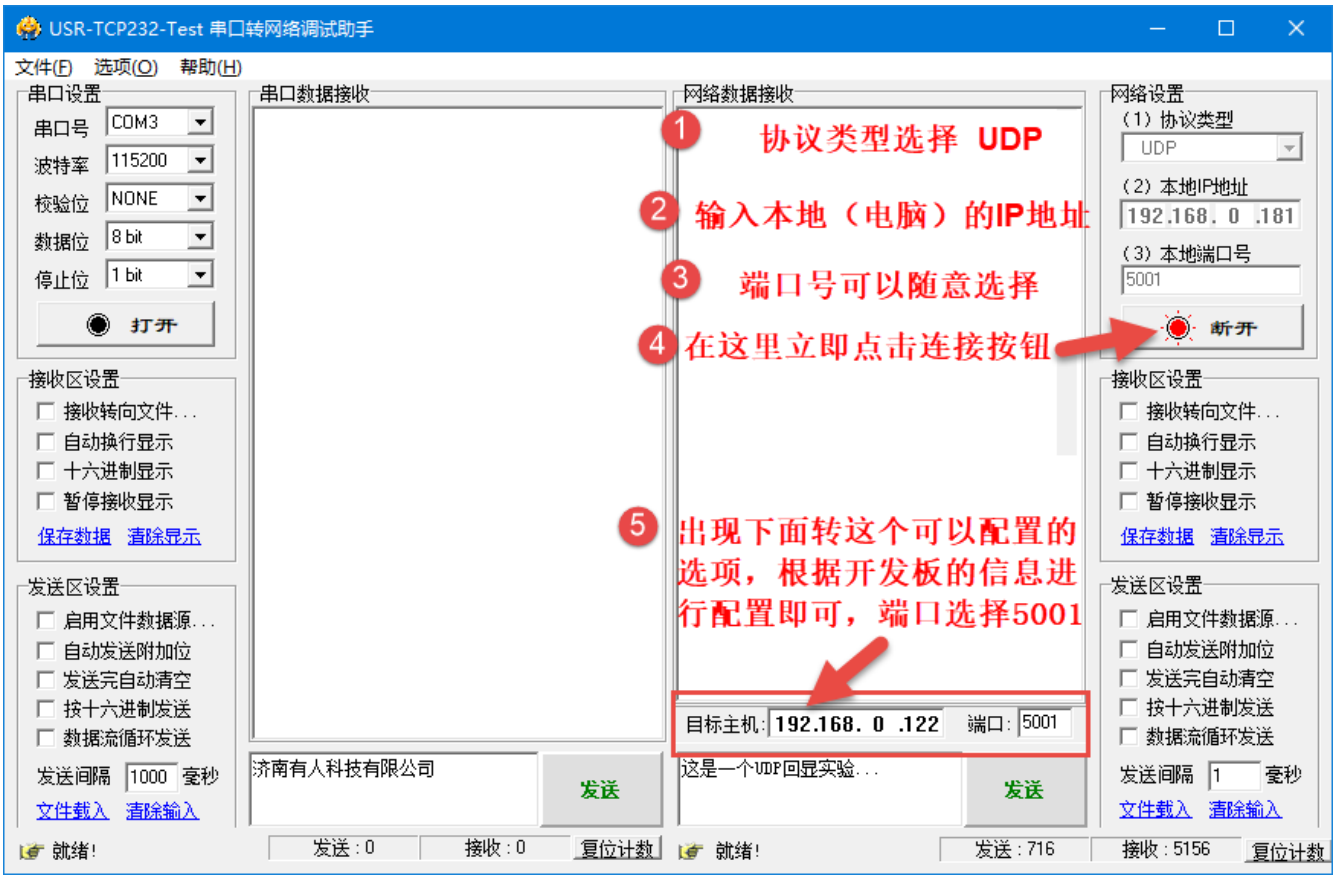


图 15-9 USR-TCP232-Test 工具操作步骤

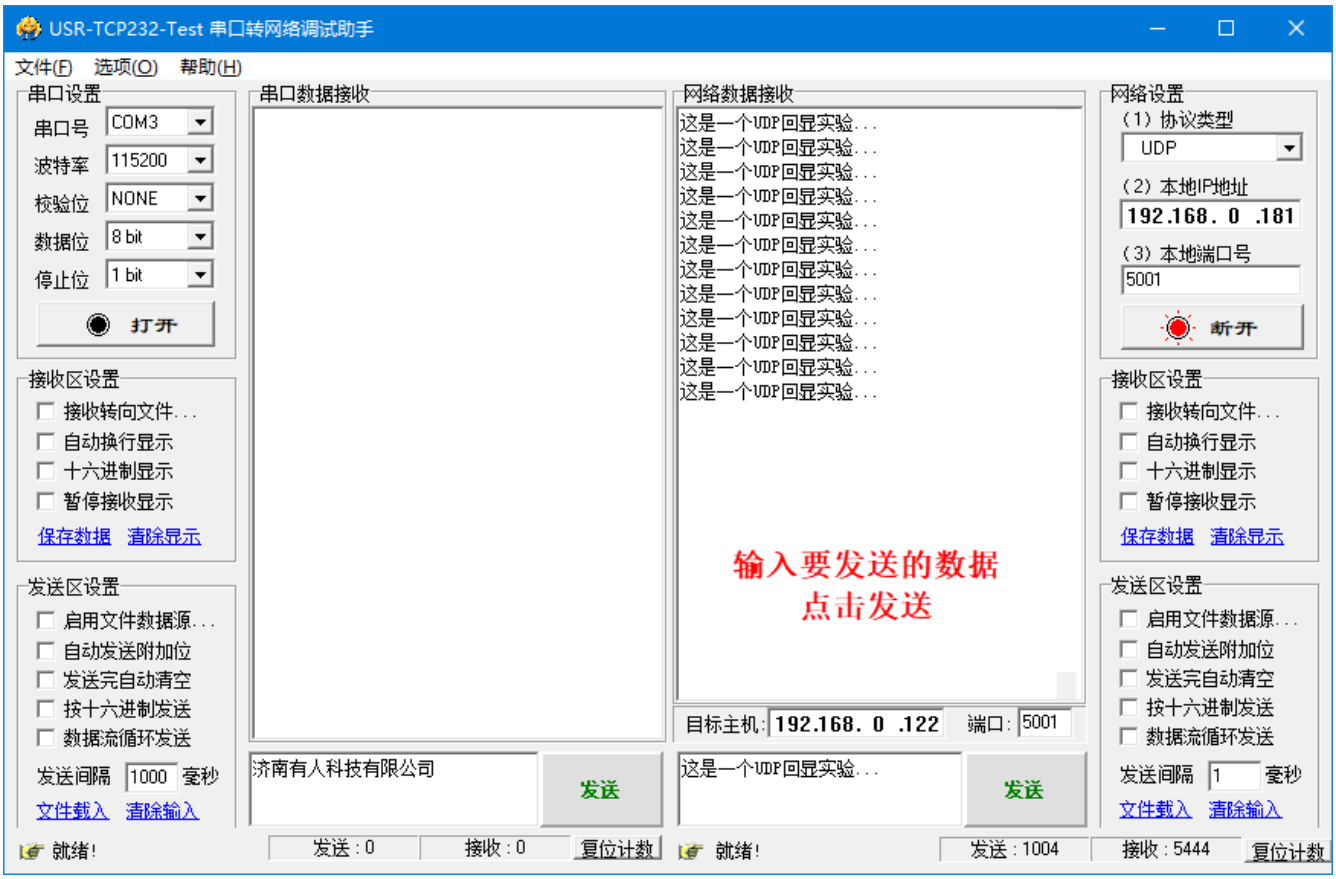


图 15-10 UDP 实验现象

第 16 章　使用 Socket 接口编程

16.1 什么是 Socket

Socket 英文原意是“孔”或者“插座”的意思，在网络编程中，通常将其称之为“套接字”，当前网络中的主流程序设计都是使用 Socket 进行编程的，因为它简单易用，更是一个标准，能在不同平台很方便移植。本章讲解的是 LwIP 中的 Socket 编程接口，因为 LwIP 作者为了能让更多开发者直接上手 LwIP 的编程，专门设计了 LwIP 的第三种编程接口——Socket API，它兼容 BSD Socket。

Socket 虽然是能在多平台移植，但是 LwIP 中的 Socket 并不完善，因为 LwIP 设计之初就是为了在嵌入式平台中使用，它只实现了完整 Socket 的部分功能，不过，在嵌入式平台中，这些功能早已足够。

在 Socket 中，它使用一个套接字来记录网络的一个连接，套接字是一个整数，就像我们操作文件一样，利用一个文件描述符，可以对它打开、读、写、关闭等操作，类似的，在网络中，我们也可以对 Socket 套接字进行这样子的操作，比如开启一个网络的连接、读取连接主机发送来的数据、向连接的主机发送数据、终止连接等操作。

16.2 LwIP 中的 Socket

在 LwIP 中，Socket API 是基于 NETCONN API 之上来实现的，系统最多提供 MEMP_NUM_NETCONN 个 netconn 连接结构，因此 Socket 套接字的个数也是那么多个，为了更好对 netconn 进行封装，LwIP 还定义了一个套接字结构体——lwip_sock（我称之为 Socket 连接结构），每个 lwip_sock 内部都有一个 netconn 的指针，实现了对 netconn 的再次封装，那怎么找到 lwip_sock 这个结构体呢？LwIP 定义了一个 lwip_sock 类型的 sockets 数组，通过套接字就可以直接索引并且访问这个结构体了，这也是为什么套接字是一个整数的原因，lwip_sock 结构体是比较简单的，因为基本上全是依赖 netconn 实现，具体见*代码清单 16_1*。

代码清单 16-1 LwIP 中的 Socket 相关数据结构

```
#define NUM_SOCKETS MEMP_NUM_NETCONN

/** 全局可用套接字数组（默认是 4） */
static struct lwip_sock sockets[NUM_SOCKETS];

union lwip_sock_lastdata
{
    struct netbuf *netbuf;
    struct pbuf *pbuf;
};

/** 包含用于套接字的所有内部指针和状态 */
struct lwip_sock
{
    /** 套接字当前是在 netconn 上构建的，每个套接字都有一个 netconn */
    struct netconn *conn;
    /** 从上一次读取中留下的数据 */
    union lwip_sock_lastdata lastdata;
    /** 收到数据的次数由 event_callback() 记录，下面的字段在 select 机制上使用 */
    s16_t rcvevent;
    /** 发送数据的次数，也是由回调函数记录的 */
    u16_t sendevent;
    /** Socket 上的发生的错误次数 */
    u16_t errevent;
    /** 使用 select 等待此套接字的线程数 */
    SELWAIT_T select_waiting;
};
```

16.3 Socket API

16.3.1 socket()

这个函数的功能是向内核申请一个套接字，在本质上该函数其实就是对 netconn_new() 函数进行了封装，虽然说不是直接调用它，但是主体完成的工作就做了 netconn_new() 函数的事情，而且该函数本质是一个宏定义，具体见代码清单 16_2。

代码清单 16-2 socket()

```
#define socket(domain,type,protocol)         \
    lwip_socket(domain,type,protocol)

int
lwip_socket(int domain, int type, int protocol);

#define AF_INET          2

/* Socket 服务类型 (TCP/UDP/RAW) */
#define SOCK_STREAM      1
#define SOCK_DGRAM       2
#define SOCK_RAW         3
```

参数 domain 表示该套接字使用的协议簇，对于 TCP/IP 协议来说，该值始终为 AF_INET。

参数 type 指定了套接字使用的服务类型，可能的类型有 3 种：

1. SOCK_STREAM：提供可靠的（即能保证数据正确传送到对方）面向连接的 Socket 服务，多用于资料（如文件）传输，如 TCP 协议。
2. SOCK_DGRAM：是提供无保障的面向消息的 Socket 服务，主要用于在网络上发广播信息，如 UDP 协议，提供无连接不可靠的数据报交付服务。
3. SOCK_RAW：表示原始套接字，它允许应用程序访问网络层的原始数据包，这个套接字用得比较少，暂时不用理会它。

参数 protocol 指定了套接字使用的协议，在 IPv4 中，只有 TCP 协议提供 SOCK_STREAM 这种可靠的服务，只有 UDP 协议提供 SOCK_DGRAM 服务，对于这两种协议，protocol 的值均为 0。

当申请套接字成功的时候，该函数返回一个 int 类型的值，也是 Socket 描述符，用户通过这个值可以索引到一个 Socket 连接结构——lwip_sock，当申请套接字失败时，该函数返回-1。

16.3.2 bind()

该函数的功能与 netconn_bind() 函数是一样的，用于服务器端绑定套接字与网卡信息，实际上就是对 netconn_bind() 函数进行了封装，可以将一个申请成功的套接字与网卡信息进行绑定，其函数原型具体见*代码清单 16_3*

代码清单 16-3 bind()

```
#define bind(s,name,namelen)   \
                    lwip_bind(s,name,namelen)

int lwip_bind(int s,
              const struct sockaddr *name,
              socklen_t namelen);
```

参数 s 是表示要绑定的 Socket 套接字，注意了，这个套机字必须是从 socket() 函数中返回的索引，否则将无法完成绑定操作。

参数 name 是一个指向 sockaddr 结构体的指针，其中包含了网卡的 IP 地址、端口号等重要的信息，LwIP 为了更好描述这些信息，使用了 sockaddr 结构体来定义了必要的信息的字段，它常被用于 Socket API 的很多函数中，我们在使用 bind() 的时候，只需要直接填写相关字段即可，sockaddr 结构体具体见*代码清单 16_4*。

参数 namelen 指定了 name 结构体的长度。

代码清单 16-4sockaddr 结构体

```
struct sockaddr
{
```

(下页继续)

(续上页)

```
    u8_t        sa_len;     /* 长度 */
    sa_family_t sa_family;  /* 协议簇 */
    char        sa_data[14];        /* 连续的 14 字节信息 */
};
```

咋一看这个结构体，好像没啥信息要我们填写的，确实也是这样子，我们需要填写的 IP 地址与端口号等信息，都在 sa_data 连续的 14 字节信息里面，但是这个数据对我们不友好，因此 LwIP 还定义了另一个对开发者更加友好的结构体——sockaddr_in，我们一般也是用这个结构体，具体见*代码清单 16_5*

代码清单 16-5sockaddr_in 结构体

```
struct sockaddr_in
{
    u8_t            sin_len;
    sa_family_t     sin_family;
    in_port_t       sin_port;
    struct in_addr  sin_addr;
#define SIN_ZERO_LEN 8
    char            sin_zero[SIN_ZERO_LEN];
};
```

这个结构体的前两个字段是与 sockaddr 结构体的前两个字段一致，而剩下的字段就是 sa_data 连续的 14 字节信息里面的内容，只不过从新定义了成员变量而已，sin_port 字段是我们需要填写的端口号信息，sin_addr 字段是我们需要填写的 IP 地址信息，剩下 sin_zero 区域的 8 字节保留未用。

那么这个函数应该怎么使用呢？具体见*代码清单 16_6*，

代码清单 16-6 bind() 函数的使用方法

```
sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock < 0)
{
```

(下页继续)

(续上页)

```
    printf("Socket error\n");
}
struct sockaddr_in server_addr;
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(5001);
memset(&(server_addr.sin_zero), 0, sizeof(server_addr.sin_zero));

if(bind(sock,(struct sockaddr *)&server_addr,sizeof(struct sockaddr))==-1)
{
    printf("Unable to bind\n");
}
```

16.3.3 connect()

这个函数的作用与 netconn_connect() 函数的作用基本一致，因为就是封装了 netconn_connect() 函数。它用于客户端中，将 Socket 与远端 IP 地址、端口号进行绑定，在 TCP 客户端连接中，调用这个函数将发生握手过程（会发送一个 TCP 连接请求），并最终建立新的 TCP 连接，而对于 UDP 协议来说，调用这个函数只是在 UDP 控制块中记录远端 IP 地址与端口号，而不发送任何数据，参数信息与 bind() 函数是一样的，具体见*代码清单 16_7*。

代码清单 16-7 connect()

```
#define connect(s,name,namelen) \
                lwip_connect(s,name,namelen)

int
lwip_connect(int s,
             const struct sockaddr *name,
             socklen_t namelen);
```

16.3.4 listen()

该函数是对 netconn_listen() 函数的封装，只能在 TCP 服务器中使用，让服务器进入监听状态，等待远端的连接请求，LwIP 中可以接收多个客户端的连接，因此参数 backlog 指定了请求队列的大小，具体见*代码清单 16_8*。

代码清单 16-8 listen()

```
#define listen(s,backlog)          \
                lwip_listen(s,backlog)

int
lwip_listen(int s, int backlog);
```

16.3.5 accept()

accept() 函数与 netconn_accept() 函数作用一样，用于 TCP 服务器中，等待着远端主机的连接请求，并且建立一个新的 TCP 连接，在调用这个函数之前需要通过调用 listen() 函数让服务器进入监听状态。accept() 函数的调用会阻塞应用线程直至与远程主机建立 TCP 连接。参数 addr 是一个返回结果参数，它的值由 accept() 函数设置，其实就是远程主机的地址与端口号等信息，当新的连接已经建立后，远端主机的信息将保存在连接句柄中，它能够唯一的标识某个连接对象。同时函数返回一个 int 类型的套接字描述符，根据它能索引到连接结构，如果连接失败则返回-1，具体见*代码清单 16_9*。

代码清单 16-9 accept()

```
#define accept(s,addr,addrlen)       \
            lwip_accept(s,addr,addrlen)
int
lwip_accept(int s,
            struct sockaddr *addr,
            socklen_t *addrlen)
```

16.3.6 read()、recv()、recvfrom()

read() 与 recv() 函数的核心是调用 recvfrom() 函数，而 recvfrom() 函数是基于 netconn_recv() 函数来实现的，recv() 与 read() 函数用于从 Socket 中接收数据，它们可以是 TCP 协议和 UDP 协议，具体见*代码清单 16_10*。

代码清单 16-10 read()、recv()、recvfrom()

```
#define read(s,mem,len)          \
        lwip_read(s,mem,len)
ssize_t
lwip_read(int s, void *mem, size_t len)
{
    return lwip_recvfrom(s, mem, len, 0, NULL, NULL);
}

#define recv(s,mem,len,flags)            \
        lwip_recv(s,mem,len,flags)
ssize_t
lwip_recv(int s, void *mem, size_t len, int flags)
{
    return lwip_recvfrom(s, mem, len, flags, NULL, NULL);
}

#define recvfrom(s,mem,len,flags,from,fromlen)     \
        lwip_recvfrom(s,mem,len,flags,from,fromlen)
ssize_t
lwip_recvfrom(int s, void *mem, size_t len, int flags,
            struct sockaddr *from, socklen_t *fromlen)
```

men 参数记录了接收数据的缓存起始地址，len 用于指定接收数据的最大长度，如果函数能正确接收到数据，将会返回一个接收到数据的长度，否则将返回-1，若返回值为 0，表示连接已经终止，应用程序可以根据返回的值进行不一样的操作。recv() 函数包含一个 flags 参数，我们暂时可以直接忽略它，设置为 0 即可。注意，如果接收的数据大于用户提供的缓存区，那么多余的数据

会被直接丢弃。

16.3.7 sendto()

这个函数主要是用于 UDP 协议传输数据中，它向另一端的 UDP 主机发送一个 UDP 报文，本质上是对 netconn_send() 函数的封装，参数 data 指定了要发送数据的起始地址，而 size 则指定数据的长度，参数 flag 指定了发送时候的一些处理，比如外带数据等，此时我们不需要理会它，一般设置为 0 即可，参数 to 是一个指向 sockaddr 结构体的指针，在这里需要我们自己提供远端主机的 IP 地址与端口号，并且用 tolen 参数指定这些信息的长度，具体见代码清单 16_11。

代码清单 16-11 sendto()

```
#define sendto(s,dataptr,size,flags,to,tolen)      \
lwip_sendto(s,dataptr,size,flags,to,tolen)

ssize_t
lwip_sendto(int s, const void *data, size_t size, int flags,
            const struct sockaddr *to, socklen_t tolen)
```

16.3.8 send()

send() 函数可以用于 UDP 协议和 TCP 连接发送数据。在调用 send() 函数之前，必须使用 connect() 函数将远端主机的 IP 地址、端口号与 Socket 连接结构进行绑定。对于 UDP 协议，send() 函数将调用 lwip_sendto() 函数发送数据，而对于 TCP 协议，将调用 netconn_write_partly() 函数发送数据。相对于 sendto() 函数，参数基本是没啥区别的，但无需我们设置远端主机的信息，更加方便操作，因此这个函数在实际中使用也是很多的，具体见代码清单 16_12。

代码清单 16-12 send()

```
#define send(s,dataptr,size,flags)           \
        lwip_send(s,dataptr,size,flags)
```

(下页继续)

(续上页)

```
ssize_t
lwip_send(int s, const void *data, size_t size, int flags)
```

16.3.9 write()

这个函数一般用于处于稳定的 TCP 连接中传输数据，当然也能用于 UDP 协议中，它也是基于 lwip_send 上实现的，但是无需我们设置 flag 参数，具体见代码清单 *16_13*。

代码清单 16-13 write()

```
#define write(s,dataptr,len)                    \
            lwip_write(s,dataptr,len)

ssize_t
lwip_write(int s, const void *data, size_t size)
{
    return lwip_send(s, data, size, 0);
}
```

16.3.10 close()

close() 函数是用于关闭一个指定的套接字，在关闭套接字后，将无法使用对应的套接字描述符索引到连接结构，该函数的本质是对 netconn_delete() 函数的封装（真正处理的函数是 netconn_prepare_delete()），如果连接是 TCP 协议，将产生一个请求终止连接的报文发送到对端主机中，如果是 UDP 协议，将直接释放 UDP 控制块的内容，具体见代码清单 *16_14*。

代码清单 16-14 close()

```
#define close(s)            \
        lwip_close(s)
```

(下页继续)

(续上页)

```
int
lwip_close(int s)
```

16.3.11 ioctl()、ioctlsocket()

这两个函数很有意思（其实是一样的，本质是宏定义，都是调用 lwip_ioctl() 函数），它用于获取与设置套接字相关的操作参数，参数 cmd 指明对套接字的操作命令，在 LwIP 中只支持 FIONREAD 与 FIONBIO 命令：

1. FIONREAD 命令确定套接字 s 自动读入的数据量，这些数据已经被接收，但应用线程并未读取的，所以可以使用这个函数来获取这些数据的长度，在这个命令状态下，argp 参数指向一个无符号长整型，用于保存函数的返回值（即未读数据的长度）。如果套接字是 SOCK_STREAM 类型，则 FIONREAD 命令会返回 recv() 函数中所接收的所有数据量，这通常与在套接字接收缓存队列中排队的数据总量相同；而如果套接字是 SOCK_DGRAM 类型的，则 FIONREAD 命令将返回在套接字接收缓存队列中排队的第一个数据包大小。
2. FIONBIO 命令用于允许或禁止套接字的非阻塞模式。在这个命令下，argp 参数指向一个无符号长整型，如果该值为 0 则表示禁止非阻塞模式，而如果该值非 0 则表示允许非阻塞模式则。当创建一个套接字的时候，它就处于阻塞模式，也就是说非阻塞模式被禁止，这种情况下所有的发送、接收函数都会是阻塞的，直至发送、接收成功才得以继续运行；而如果是非阻塞模式下，所有的发送、接收函数都是不阻塞的，如果发送不出去或者接收不到数据，将直接返回错误代码给用户，这就需要用户对这些“意外”情况进行处理，保证代码的健壮性，这与 BSD Socket 是一致的。

其函数原型具体见*代码清单 16_15*。

代码清单 16-15 ioctl()、ioctlsocket()

```
#define ioctl(s,cmd,argp)       \
            lwip_ioctl(s,cmd,argp)

#define ioctlsocket(s,cmd,argp)      \
```

(下页继续)

(续上页)

```
            lwip_ioctl(s,cmd,argp)

int
lwip_ioctl(int s, long cmd, void *argp)
```

16.3.12 setsockopt()

看名字就知道，这个函数是用于设置套接字的一些选项的，参数 level 有多个常见的选项，如：

- SOL_SOCKET：表示在 Socket 层。
- IPPROTO_TCP：表示在 TCP 层。
- IPPROTO_IP：表示在 IP 层。

参数 optname 表示该层的具体选项名称，比如：

1. 对于 SOL_SOCKET 选项，可以是 SO_REUSEADDR（允许重用本地地址和端口）、SO_SNDTIMEO（设置发送数据超时时间）、SO_SNDTIMEO（设置接收数据超时时间）、SO_RCVBUF（设置发送数据缓冲区大小）等等。
2. 对于 IPPROTO_TCP 选项，可以是 TCP_NODELAY（不使用 Nagle 算法）、TCP_KEEPALIVE（设置 TCP 保活时间）等等。
3. 对于 IPPROTO_IP 选项，可以是 IP_TTL（设置生存时间）、IP_TOS（设置服务类型）等等。

代码清单 16-16 setsockopt()

```
#define setsockopt(s,level,optname,opval,optlen)   \
lwip_setsockopt(s,level,optname,opval,optlen)

int
lwip_setsockopt(int s,
                int level,
                int optname,
```

(下页继续)

(续上页)

```
                const void *optval,
                socklen_t optlen)
```

16.3.13 getsockopt()

这个函数与 setsockopt() 函数的选项参数及名称都是差不多的，只不过是作用是获得这些选项信息在这里就不过多讲解。

16.4 实验

16.4.1 TCP Client

这个实验现象与 NETCONN API 中实验的是一样的，我们直接把上次的工程拷贝过来，然后将 NETCONN API 替换成 Socket API 就基本差不多了，我们首先在 lwipopts.h 文件中将宏 LWIP_SOCKET 配置为 1，在文件中添加以下代码，注意，不要删除 LWIP_NETCONN 宏定义。

```
#define LWIP_SOCKET 1
```

在 client.c 文件中添加*代码清单 16_17* 所示代码，当然，端口号等信息根据你们自己的网络环境修改即可，然后编译工程，下载到开发板上，电脑端的操作步骤与 NETCONN API 中实验操作步骤是一样的，就不再过多赘述了。

代码清单 16-17client.c 文件内容

```
#include "client.h"

#include "lwip/opt.h"

#include "lwip/sys.h"
#include "lwip/api.h"
```

(下页继续)

(续上页)

```
#include <lwip/sockets.h>

#define PORT                5001
#define IP_ADDR         "192.168.0.181"

static void client(void *thread_param)
{
    int sock = -1;
    struct sockaddr_in client_addr;

    uint8_t send_buf[]= "This is a TCP Client test...\n";

    while (1)
    {
        sock = socket(AF_INET, SOCK_STREAM, 0);
        if (sock < 0)
        {
            printf("Socket error\n");
            vTaskDelay(10);
            continue;
        }

        client_addr.sin_family = AF_INET;
        client_addr.sin_port = htons(PORT);
        client_addr.sin_addr.s_addr = inet_addr(IP_ADDR);
        memset(&(client_addr.sin_zero), 0, sizeof(client_addr.sin_zero));

        if (connect(sock,
                    (struct sockaddr *)&client_addr,
                    sizeof(struct sockaddr)) == -1)
        {
```

(下页继续)

(续上页)

```
            printf("Connect failed!\n");
            closesocket(sock);
            vTaskDelay(10);
            continue;
        }

        printf("Connect to iperf server successful!\n");

        while (1)
        {
            if (write(sock,send_buf,sizeof(send_buf)) < 0)
                break;

            vTaskDelay(1000);
        }

        closesocket(sock);
    }

}

void
client_init(void)
{
    sys_thread_new("client", client, NULL, 512, 4);
}
```

16.4.2 TCP Server

同理，这个实验也只需把 NETCONN API 中的实验拷贝过来，然后在 lwipopts.h 文件中将宏 LWIP_SOCKET 配置为 1，再将 tcpecho.c 文件的内容替换为*代码清单 16_18* 的内容即可，操作步骤也是一样的，然后编译工程并且下载到开发板上即可看到实验现象。

代码清单 16-18tcpecho.c 文件内容

```
#include "tcpecho.h"

#include "lwip/opt.h"

#if LWIP_SOCKET
#include <lwip/sockets.h>

#include "lwip/sys.h"
#include "lwip/api.h"
/*-----------------------------------------------------------------------*/

#define PORT                5001
#define RECV_DATA           (1024)


static void
tcpecho_thread(void *arg)
{
    int sock = -1,connected;
    char *recv_data;
    struct sockaddr_in server_addr,client_addr;
    socklen_t sin_size;
    int recv_data_len;

    recv_data = (char *)pvPortMalloc(RECV_DATA);
```

(下页继续)

(续上页)

```
    if (recv_data == NULL)
    {
        printf("No memory\n");
        goto __exit;
    }

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0)
    {
        printf("Socket error\n");
        goto __exit;
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    memset(&(server_addr.sin_zero), 0, sizeof(server_addr.sin_zero));

    if (bind(sock, (struct sockaddr *)&server_addr, sizeof(struct␣
↪sockaddr)) == -1)
    {
        printf("Unable to bind\n");
        goto __exit;
    }

    if (listen(sock, 5) == -1)
    {
        printf("Listen error\n");
        goto __exit;
    }

    while (1)
```

(下页继续)

(续上页)

```
    {
        sin_size = sizeof(struct sockaddr_in);

        connected = accept(sock, (struct sockaddr *)&client_addr, &sin_
↪size);

        printf("new client connected from (%s, %d)\n",
            inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
        {
            int flag = 1;

            setsockopt(connected,
                    IPPROTO_TCP,     /* set option at TCP level */
                    TCP_NODELAY,     /* name of option */
                    (void *) &flag, /* the cast is historical cruft */
                    sizeof(int));    /* length of option value */
        }

        while (1)
        {
            recv_data_len = recv(connected, recv_data, RECV_DATA, 0);

            if (recv_data_len <= 0)
                break;

            printf("recv %d len data\n",recv_data_len);

            write(connected,recv_data,recv_data_len);

        }
        if (connected >= 0)
            closesocket(connected);
```

(下页继续)

(续上页)

```
        connected = -1;
    }
__exit:
    if (sock >= 0) closesocket(sock);
    if (recv_data) free(recv_data);
}

void
tcpecho_init(void)
{
    sys_thread_new("tcpecho_thread", tcpecho_thread, NULL, 512, 4);
}
```

16.4.3 UDP

同理，这个实验也只需把 NETCONN API 中的实验拷贝过来，然后在 lwipopts.h 文件中将宏 LWIP_SOCKET 配置为 1，再将 udpecho.c 文件的内容替换为*代码清单 16_19* 的内容即可，操作步骤也是一样的，然后编译工程并且下载到开发板上即可看到实验现象。

代码清单 16-19udpecho.c 文件内容

```
#include "udpecho.h"

#include "lwip/opt.h"

#include <lwip/sockets.h>
#include "lwip/api.h"
#include "lwip/sys.h"

#define PORT                5001
```

(下页继续)

(续上页)

```
#define RECV_DATA          (1024)

/*-----------------------------------------------------------------*/
static void
udpecho_thread(void *arg)
{
    int sock = -1;
    char *recv_data;
    struct sockaddr_in udp_addr,seraddr;
    int recv_data_len;
    socklen_t addrlen;

    while (1)
    {
        recv_data = (char *)pvPortMalloc(RECV_DATA);
        if (recv_data == NULL)
        {
            printf("No memory\n");
            goto __exit;
        }

        sock = socket(AF_INET, SOCK_DGRAM, 0);
        if (sock < 0)
        {
            printf("Socket error\n");
            goto __exit;
        }

        udp_addr.sin_family = AF_INET;
        udp_addr.sin_addr.s_addr = INADDR_ANY;
        udp_addr.sin_port = htons(PORT);
        memset(&(udp_addr.sin_zero), 0, sizeof(udp_addr.sin_zero));
```

(下页继续)

(续上页)

```
        if (bind(sock, (struct sockaddr *)&udp_addr, sizeof(struct
↪sockaddr)) == -1)
        {
            printf("Unable to bind\n");
            goto __exit;
        }
        while (1)
        {
            recv_data_len=recvfrom(sock,recv_data,
                                   RECV_DATA,0,
                                   (struct sockaddr*)&seraddr,
                                   &addrlen);

            /* 显示发送端的 IP 地址 */
            printf("receive from %s\n",inet_ntoa(seraddr.sin_addr));

            /* 显示发送端发来的字串 */
            printf("recevce:%s",recv_data);

            /* 将字串返回给发送端 */
            sendto(sock,recv_data,
                recv_data_len,0,
                (struct sockaddr*)&seraddr,
                addrlen);
        }

__exit:
        if (sock >= 0) closesocket(sock);
        if (recv_data) free(recv_data);
    }
}
```

(下页继续)

(续上页)

```
/*-----------------------------------------------------------------------*/
void
udpecho_init(void)
{
    sys_thread_new("udpecho_thread", udpecho_thread, NULL, 2048, 4);
}
```

第 17 章　使用 RAW API 接口编程

RAW API 是基于回调函数实现的 API 接口，它是很底层的 API 接口，这需要开发者对 LwIP 有较深的了解才能很好使用它，RAW API 的核心就是对控制块的处理，因为对于报文数据的处理、注册回调函数等都是需要开发者自己去实现，都是比较麻烦的，但是有一个优点，那就是处理数据效率高。

17.1 RAW API 的 UDP 编程

学习本小节之前，必须要对 UDP 控制块熟悉，如果还不熟悉，可以参考第 14.4 的内容。

17.1.1 新建控制块 udp_new()

在使用 UDP 协议进行通信之前，必须创建一个 UDP 控制块，然后将控制块与对应的端口号进行绑定，才能发送报文，而在接收 UDP 报文的时候，这个端口号就是 UDP 报文唯一识别的标志，否则 UDP 报文将无法递交到应用层去处理，即无法通过 UDP 控制块的接收回调函数递交给应用层，新建控制块的函数很简单，就是在内存池中申请一个 MEMP_UDP_PCB 类型的内存块，用于存放 UDP 控制块的相关信息，并将其初始化为 0，具体见*代码清单 17_1*。

代码清单 17-1 udp_new() 源码

```
struct udp_pcb *
udp_new(void)
{
    struct udp_pcb *pcb;

    LWIP_ASSERT_CORE_LOCKED();
```

(下页继续)

(续上页)

```
    pcb = (struct udp_pcb *)memp_malloc(MEMP_UDP_PCB);
    /*  */
    if (pcb != NULL)
    {
        memset(pcb, 0, sizeof(struct udp_pcb));
        pcb->ttl = UDP_TTL;
    }
    return pcb;
}
```

17.1.2 绑定控制块 udp_bind()

绑定控制块的作用其实就是将本机 IP 地址与端口号填写在 UDP 控制块中，以便表示唯一的应用，并且能正常与远端主机进行 UDP 通信，在这个函数中，它会将 UDP 控制块的 local_ip 与 local_port 字段进行初始化，并且把 UDP 控制块添加到 udp_pcbs 链表中，具体见*代码清单 17_2*

代码清单 17-2 udp_bind() 源码

```
err_t
udp_bind(struct udp_pcb *pcb,
         const ip_addr_t *ipaddr,
         u16_t port)
{
    struct udp_pcb *ipcb;
    u8_t rebind;

    LWIP_ASSERT_CORE_LOCKED();

    if (ipaddr == NULL)
    {
        ipaddr = IP4_ADDR_ANY;
```

(下页继续)

(续上页)

```
    }

    rebind = 0;
    /* 检查 UDP 控制块是不是存在 udp_pcbs 链表中 */
    for (ipcb = udp_pcbs; ipcb != NULL; ipcb = ipcb->next)
    {
        if (pcb == ipcb)
        {
            rebind = 1;
            break;
        }
    }

    if (port == 0)
    {
        //如果端口号为 0，则随机选择一个端口号
        port = udp_new_port();
        if (port == 0)
        {
            //没有可用端口号，返回错误
            return ERR_USE;
        }
    }
    else
    {
        for (ipcb = udp_pcbs; ipcb != NULL; ipcb = ipcb->next)
        {
            if (pcb != ipcb)
            {
                /* 判断端口号是否被占用 */
                if ((ipcb->local_port == port) &&
                        (ip_addr_cmp(&ipcb->local_ip, ipaddr) ||
```

(下页继续)

(续上页)

```
                        ip_addr_isany(ipaddr) ||
                        ip_addr_isany(&ipcb->local_ip)))
                {
                    return ERR_USE;
                }
            }
        }
    }
    //设置 IP 地址与端口号
    ip_addr_set_ipaddr(&pcb->local_ip, ipaddr);

    pcb->local_port = port;
    mib2_udp_bind(pcb);

    if (rebind == 0)
    {
        /* 如果控制块没有在 */
        pcb->next = udp_pcbs;
        udp_pcbs = pcb;
    }

    return ERR_OK;
}
```

17.1.3 建立会话 udp_connect()

说明：本来是想写建立连接的，但是对于 UDP 协议来说，建立连接的这种说法并不太准确，因为 UDP 协议本身就是一个无连接协议，因此，我们就说建立 UDP 会话好了。

其实 udp_connect() 这个函数的作用就是设置控制块中的远端 IP 地址与端口号，然后将 UDP 控制块的状态设置为会话状态 UDP_FLAGS_CONNECTED，并且将 UDP 控制块插入 udp_pcbs 链表中，这样子就是建立会话，虽然是建立会话，但是不会如 TCP 协议一样，发送请求连接、应答连

接等信息到远端主机中，因为 UDP 是无连接的协议，只将控制块的远端 IP 地址与端口号设置，表示发送数据的时候将发送到这个 IP 地址与端口号中，具体见*代码清单 17_3*

代码清单 17-3 udp_connect() 源码

```
err_t
udp_connect(struct udp_pcb *pcb,
            const ip_addr_t *ipaddr,
            u16_t port)
{
    struct udp_pcb *ipcb;

    LWIP_ASSERT_CORE_LOCKED();

    //如果没绑定本地 IP 地址与端口号，就进行绑定操作
    if (pcb->local_port == 0)
    {
        err_t err = udp_bind(pcb, &pcb->local_ip, pcb->local_port);
        if (err != ERR_OK)
        {
            return err;
        }
    }
    //设置 remote_ip 字段
    ip_addr_set_ipaddr(&pcb->remote_ip, ipaddr);

    pcb->remote_port = port;              //设置 remote_port 字段
    pcb->flags |= UDP_FLAGS_CONNECTED;

    /* 变量 udp_pcbs 链表，查找控制块是否存在链表中 */
    for (ipcb = udp_pcbs; ipcb != NULL; ipcb = ipcb->next)
    {
        if (pcb == ipcb)
        {
```

(下页继续)

(续上页)

```
            /* 已经存在就无需重复插入，返回成功 */
            return ERR_OK;
        }
    }
    /* 插入 udp_pcbs 链表首部 */
    pcb->next = udp_pcbs;
    udp_pcbs = pcb;
    return ERR_OK;
}
```

17.1.4 断开会话 udp_disconnect()

断开会话 udp_disconnect() 函数与建立会话 udp_connect() 函数相反，它主要是清除控制块的远端 IP 地址与端口号，并且将 UDP 控制块的状态清除，当然，断开会话也不会发送任何的信息到对端主机中，具体见*代码清单 17_4*。

提示：断开会话并不会删除 UDP 控制块，即不会释放 UDP 控制块的内存。

代码清单 17-4 udp_disconnect() 源码

```
void
udp_disconnect(struct udp_pcb *pcb)
{
    LWIP_ASSERT_CORE_LOCKED();

    LWIP_ERROR("udp_disconnect: invalid pcb", pcb != NULL, return);

    /* 清除 remote_ip */
    ip_addr_set_any(IP_IS_V6_VAL(pcb->remote_ip),
                    &pcb->remote_ip);

    pcb->remote_port = 0;
```

(下页继续)

(续上页)

```
    pcb->netif_idx = NETIF_NO_INDEX;
    /* 清除 UDP 控制块状态 */
    udp_clear_flags(pcb, UDP_FLAGS_CONNECTED);
}
```

17.1.5 接收数据 udp_recv()

这个函数的本质就是设置 UDP 控制块中的 recv 与 recv_arg 字段，这在 UDP 控制块就已经讲解的内容，recv 是一个函数指针，指向一个 udp_recv_fn 类型的回调函数，它非常重要，是内核与应用程序交互的桥梁，当内核接收到数据的时候，就会调用这个回调函数，进而将数据递交到应用层处理，在 recv 回调函数中，pcb、p、addr、port 等作为参数传递进去，方便用户的处理，其中 pcb 就是指向 UDP 控制块的指针，标识一个 UDP 会话，p 是指向 pbuf 的指针，里面包含着接收到的数据，而 addr 与 port 记录着发送数据段的 IP 地址与端口号，具体见*代码清单 17_5*。

代码清单 17-5 udp_recv() 源码

```
typedef void (*udp_recv_fn)(void *arg,
                            struct udp_pcb *pcb,
                            struct pbuf *p,
                            const ip_addr_t *addr,
                            u16_t port);

void
udp_recv(struct udp_pcb *pcb, udp_recv_fn recv, void *recv_arg)
{
    LWIP_ASSERT_CORE_LOCKED();

    LWIP_ERROR("udp_recv: invalid pcb", pcb != NULL, return);

    pcb->recv = recv;
    pcb->recv_arg = recv_arg;
```

(下页继续)

(续上页)

```
}
```

17.1.6 发送数据 udp_send() 与 udp_sendto()

UDP 发送数据也是依赖 IP 层，在用户使用发送数据的时候，应该为数据开辟一个 pbuf 用于存储数据，并且 pbuf 中为 UDP、IP、以太网首部预留足够的空间，然后用户调用 udp_send() 或者 udp_sendto() 函数将 pbuf 作为参数传递进去，在发送数据的时候，UDP 协议会将 UDP 首部相关的内容进行填充，形成一个完整的 UDP 报文递交到 IP 层，IP 层也会为这个数据报添加 IP 首部，形成完整的 IP 数据报递交到链路层中，然后添加以太网首部再发送出去，具体见*代码清单 17_6*。

此外，还有一个注意的地方，其实这两个函数的作用是一样的，只不过 udp_sendto() 需要指定远端远端 IP 地址与端口号而已，最终都是调用 udp_sendto_if() 函数进行发送，在 udp_sendto_if() 函数函数中将调用 udp_sendto_if_src() 函数进行发送数据，而这个函数在讲解 UDP 协议的时候就已经讲解过了，此处就不再重复赘述。

代码清单 17-6 udp_send() 与 udp_sendto()

```
err_t
udp_send(struct udp_pcb *pcb, struct pbuf *p)
{
    if (IP_IS_ANY_TYPE_VAL(pcb->remote_ip))
    {
        return ERR_VAL;
    }

    return udp_sendto(pcb, p, &pcb->remote_ip, pcb->remote_port);
}

err_t
udp_sendto(struct udp_pcb *pcb, struct pbuf *p,
           const ip_addr_t *dst_ip, u16_t dst_port)
```

(下页继续)

(续上页)

```
{
    struct netif *netif;

    if (!IP_ADDR_PCB_VERSION_MATCH(pcb, dst_ip))
    {
        return ERR_VAL;
    }

    LWIP_DEBUGF(UDP_DEBUG | LWIP_DBG_TRACE, ("udp_send\n"));

    if (pcb->netif_idx != NETIF_NO_INDEX)
    {
        netif = netif_get_by_index(pcb->netif_idx);
    }
    else
    {
        /* 找到发送报文的网卡 */
        netif = ip_route(&pcb->local_ip, dst_ip);
    }

    /* 没找到返回错误 */
    if (netif == NULL)
    {
        UDP_STATS_INC(udp.rterr);
        return ERR_RTE;
    }

    return udp_sendto_if(pcb, p, dst_ip, dst_port, netif);

}
```

17.1.7 删除 UDP 控制块 udp_remove()

这个函数的本质就是将 UDP 控制块从 udp_pcbs 链表中删除，并且释放 UDP 控制块的内存空间，这样子一个 UDP 控制块就彻底从系统中消失，想要再次使用只能通过创建一个控制块并且将它插入链表中，具体见*代码清单 17_7*。

代码清单 17-7 udp_remove() 源码

```
void
udp_remove(struct udp_pcb *pcb)
{
    struct udp_pcb *pcb2;

    LWIP_ASSERT_CORE_LOCKED();

    LWIP_ERROR("udp_remove: invalid pcb", pcb != NULL, return);

    mib2_udp_unbind(pcb);
    /* 如果 UDP 控制块在链表的首部 */
    if (udp_pcbs == pcb)
    {
        /* 删除它 */
        udp_pcbs = udp_pcbs->next;

    }
    /* 如果 UDP 控制块不在链表的首部 */
    else
    {
        for (pcb2 = udp_pcbs; pcb2 != NULL; pcb2 = pcb2->next)
        {
            /* 变量链表找到要删除的 UDP 控制块 */
            if (pcb2->next != NULL && pcb2->next == pcb)
            {
```

(下页继续)

(续上页)

```
                /* 找到了就删除它 */
                pcb2->next = pcb->next;
                break;
            }
        }
    }
    //释放内存
    memp_free(MEMP_UDP_PCB, pcb);
}
```

17.2 RAW API 的 TCP 编程

TCP 协议使用 RAW API 进行编程，本质上也是对 TCP 控制块的操作。

17.2.1 新建控制块 tcp_new()

这个函数用于分配一个 TCP 控制块，它通过 tcp_alloc() 函数分配一个 TCP 控制块结构来存储 TCP 控制块的数据信息，如果没有足够的内容分配空间，那么 tcp_alloc() 函数就会尝试释放一些不太重要的 TCP 控制块，比如就会释放处于 TIME_WAIT、CLOSING 等状态的 TCP 控制块，或者根据控制块的优先级进行释放，释放一些不重要的 TCP 控制块，以完成新 TCP 控制块的分配，分配完成后，内核会初始化 TCP 控制块的各个字段内容，具体见*代码清单 17_8*。

代码清单 17-8tcp_new() 源码

```
struct tcp_pcb *
tcp_new(void)
{
    return tcp_alloc(TCP_PRIO_NORMAL);
}
```

17.2.2 绑定控制块 tcp_bind()

对应 TCP 服务器端的程序，一般在创建一个 TCP 控制块的时候，就会调用 tcp_bind() 函数将本地的 IP 地址、端口号与一个控制块进行绑定，它的工作其实很简单，就遍历 TCP 控制块链表，在讲解 13.7 TCP 中的数据结构那一小节中，我们知道 LwIP 使用了 4 条 TCP 控制块链表来描述 TCP 控制块的各种状态，那么肯定是需要遍历所有的 TCP 控制块链表以便知道要绑定的 IP 地址与端口号是不重复的，即没有被其他 TCP 控制块使用，然后再将本地的 IP 地址、端口号与新创建的控制块进行绑定，最后再将绑定完毕的控制块插入 tcp_bound_pcbs 链表中，具体见*代码清单 17_9*

代码清单 17-9 tcp_bind() 源码

```
struct tcp_pcb *
tcp_new(void)
{
    return tcp_alloc(TCP_PRIO_NORMAL);
}

err_t
tcp_bind(struct tcp_pcb *pcb,
         const ip_addr_t *ipaddr,
         u16_t port)
{
    int i;
    int max_pcb_list = NUM_TCP_PCB_LISTS;
    struct tcp_pcb *cpcb;

    LWIP_ASSERT_CORE_LOCKED();

    if (ipaddr == NULL)
    {
        ipaddr = IP4_ADDR_ANY;
    }
```

(下页继续)

(续上页)

```
    if (port == 0)
    {
        port = tcp_new_port();
        if (port == 0)
        {
            return ERR_BUF;
        }
    }
    else
    {
        /* 遍历 TCP 控制块链表 */
        for (i = 0; i < max_pcb_list; i++)
        {
            for (cpcb = *tcp_pcb_lists[i];
                    cpcb != NULL; cpcb = cpcb->next)
            {
                if (cpcb->local_port == port)
                {
                    {
                        /* 如果已经使用了 IP 地址与端口号就返回已使用错误 */
                        if ((IP_IS_V6(ipaddr) ==
                                IP_IS_V6_VAL(cpcb->local_ip)) &&
                                (ip_addr_isany(&cpcb->local_ip) ||
                                ip_addr_isany(ipaddr) ||
                                ip_addr_cmp(&cpcb->local_ip, ipaddr)))
                        {
                            return ERR_USE;
                        }
                    }
                }
            }
        }
```

(下页继续)

(续上页)

```
    }

    //设置 IP 地址
    if (!ip_addr_isany(ipaddr))
    {
        ip_addr_set(&pcb->local_ip, ipaddr);
    }
    //设置端口号
    pcb->local_port = port;

    //插入 tcp_bound_pcbs 链表
    TCP_REG(&tcp_bound_pcbs, pcb);
    return ERR_OK;
}
```

17.2.3 控制块监听 tcp_listen()

作为 TCP 服务器端的程序，TCP 监听状态是必须要实现的，它让服务器处于监听状态，等待 TCP 客户端的连接并且去处理它，它使用的函数就是 tcp_listen()。我们也知道，一个 TCP 控制块对应着一条 TCP 连接，那么如果处于 TCP 监听状态的控制块太多，那肯定是需要消耗不少的内存资源，因此 LwIP 为了节省内存的开销，定义了不完整的 TCP 控制块——监听 TCP 控制块 tcp_pcb_listen，它是专门应用于监听状态的控制块，里面包含完整 TCP 控制块的部分字段信息，因为处于监听状态的 TCP 控制块并不需要使用其他的字段内容，这样子的 tcp_pcb_listen 结构更小，更适合与嵌入式产品使用。

LwIP 是这样子处理这两个 TCP 控制块的：首先申请一个监听 TCP 控制块 tcp_pcb_listen，将完整的 TCP 控制块的部分内容拷贝到 tcp_pcb_listen 中，设置监听 TCP 控制块 tcp_pcb_listen 的状态为监听状态 LISTEN，然后将完整的 TCP 控制块从绑定链表 tcp_bound_pcbs 中删除并且释放 TCP 控制块的内存空间，最后将监听 TCP 控制块插入监听链表 tcp_listen_pcbs 中，完成监听的操作，。

当服务器收到客户端发来的请求连接报文后，内核会遍历 TCP 监听链表 tcp_listen_pcbs，找到和报文中一致的 IP 地址、目标端口号的控制块，然后内核将新建一个完整的 TCP 控制块，将监听

TCP 控制块 tcp_pcb_listen 的字段内容拷贝到完整的 TCP 控制块中，然后填写远端 IP 地址与端口号等字段，最后再将这个完整的 TCP 控制块挂载到 tcp_active_pcbs 链表中，当然，监听 TCP 控制块 tcp_pcb_listen 并不会被删除，因为它还需等待其他客户端的连接，这正是服务器必须要实现的功能。

提示：因为 tcp_listen() 函数的本质是一个宏定义，实际调用的函数是 tcp_listen_with_backlog_and_err()，具体见*代码清单 17_10*。

代码清单 17-10 tcp_listen_with_backlog_and_err() 源码

```
struct tcp_pcb *
tcp_listen_with_backlog_and_err(struct tcp_pcb *pcb,
                                u8_t backlog,
                                err_t *err)
{
    struct tcp_pcb_listen *lpcb = NULL;
    err_t res;

    LWIP_UNUSED_ARG(backlog);

    LWIP_ASSERT_CORE_LOCKED();

    /* 如果已经处于监听状态 */
    if (pcb->state == LISTEN)
    {
        lpcb = (struct tcp_pcb_listen *)pcb;
        res = ERR_ALREADY;
        goto done;
    }
    //申请一个监听状态的 TCP 控制块 tcp_pcb_listen，为了节省内存
    lpcb = (struct tcp_pcb_listen *)memp_malloc(MEMP_TCP_PCB_LISTEN);
    if (lpcb == NULL)
    {
        res = ERR_MEM;
```

(下页继续)

(续上页)

```
        goto done;
    }
    //在监听 TCP 控制块中填写完整的 TCP 控制块的部分字段信息
    lpcb->callback_arg = pcb->callback_arg; //回调函数的传入参数
    lpcb->local_port = pcb->local_port;      //本地端口号
    lpcb->state = LISTEN;        //进入监听状态
    lpcb->prio = pcb->prio;      //优先级
    lpcb->so_options = pcb->so_options; //选项
    lpcb->netif_idx = NETIF_NO_INDEX;
    lpcb->ttl = pcb->ttl;      //生存时间
    lpcb->tos = pcb->tos;      //服务类型
    ip_addr_copy(lpcb->local_ip, pcb->local_ip);  //本地 IP 地址
    if (pcb->local_port != 0)
    {
        //将完整的 TCP 控制块从 tcp_bound_pcbs 链表中删除
        TCP_RMV(&tcp_bound_pcbs, pcb);
    }
    //释放完整的 TCP 控制块
    tcp_free(pcb);

    lpcb->accept = tcp_accept_null;
    //将监听 TCP 控制块插入监听链表 tcp_listen_pcbs 中
    TCP_REG(&tcp_listen_pcbs.pcbs, (struct tcp_pcb *)lpcb);
    res = ERR_OK;
done:
    if (err != NULL)
    {
        *err = res;
    }
    return (struct tcp_pcb *)lpcb;
}
```

17.2.4 处理连接 tcp_accept()

在服务器端，处理客户端连接的函数是 tcp_accept()，当让服务器进入监听状态后，就需要立即调用这个函数，它向监听 TCP 控制块中的 accept 字段注册一个 tcp_accept_fn 类型的函数，当检测到客户端的连接时，内核就会调用这个回调函数，以完成连接操作，而在 accept() 函数中，需要用户去处理这些连接，回调函数有 3 个参数，其中 newpcb 是新 TCP 连接对应的控制块，用户需要对这个控制块进行后续处理，具体见*代码清单 17_11*。

代码清单 17-11 tcp_accept() 源码

```
typedef err_t (*tcp_accept_fn)(void *arg,
                               struct tcp_pcb *newpcb,
                               err_t err);


void
tcp_accept(struct tcp_pcb *pcb, tcp_accept_fn accept)
{
    LWIP_ASSERT_CORE_LOCKED();
    if ((pcb != NULL) && (pcb->state == LISTEN))
    {
        struct tcp_pcb_listen *lpcb = (struct tcp_pcb_listen *)pcb;
        lpcb->accept = accept;
    }
}
```

17.2.5 建立连接 tcp_connect()

对于 TCP 客户端来说，主动建立连接是必不可少的一步，一般在客户端的实现步骤中，都是创建一个 TCP 控制块，然后绑定本地的 IP 地址与端口号，然后主动与服务器建立连接。那么建立连接的函数就是 tcp_connect() 函数，这个函数的最终目的是将 TCP 控制块从 tcp_bound_pcbs 绑定链表中取下并且放到 tcp_active_pcbs 链表中，并且发送一个连接请求报文，不过在处理这些事情之前，它会填写 TCP 控制块中发送窗口与接收窗口的相关字段，以达到最适的 TCP 连接，然后

调用 tcp_enqueue_flags() 函数构造一个连接请求报文，将 SYN 标志置 1，具体见*代码清单 17_12*。

代码清单 17-12tcp_connect() 源码

```
err_t
tcp_connect(struct tcp_pcb *pcb,
            const ip_addr_t *ipaddr,
            u16_t port,
            tcp_connected_fn connected)
{
    struct netif *netif = NULL;
    err_t ret;
    u32_t iss;
    u16_t old_local_port;

    LWIP_ASSERT_CORE_LOCKED();
    //设置远端 IP 地址（目标 IP 地址）和端口号
    ip_addr_set(&pcb->remote_ip, ipaddr);
    pcb->remote_port = port;

    if (pcb->netif_idx != NETIF_NO_INDEX)
    {
        netif = netif_get_by_index(pcb->netif_idx);
    }
    else
    {
        /* 找到要发送请求连接报文的网卡 */
        netif = ip_route(&pcb->local_ip, &pcb->remote_ip);
    }

    if (netif == NULL)
    {
        /* 找不到合适的发送网卡，返回失败 */
        return ERR_RTE;
```

（下页继续）

(续上页)

```
    }

    /* 看看本地 IP 地址是否绑定了，没绑定就进行绑定 */
    if (ip_addr_isany(&pcb->local_ip))
    {
        const ip_addr_t *local_ip =
            ip_netif_get_local_ip(netif, ipaddr);
        if (local_ip == NULL)
        {
            return ERR_RTE;
        }
        ip_addr_copy(pcb->local_ip, *local_ip);
    }

    old_local_port = pcb->local_port;
    if (pcb->local_port == 0)
    {
        //如果没绑定本地端口号，就进行绑定操作
        pcb->local_port = tcp_new_port();

        if (pcb->local_port == 0)
        {
            return ERR_BUF;
        }
    }
    //设置发送窗口的相关参数
    iss = tcp_next_iss(pcb);
    pcb->rcv_nxt = 0;
    pcb->snd_nxt = iss;
    pcb->lastack = iss - 1;
    pcb->snd_wl2 = iss - 1;
    pcb->snd_lbb = iss - 1;
```

(下页继续)

(续上页)

```
    /* 设置接收窗口的相关参数 */
    pcb->rcv_wnd = pcb->rcv_ann_wnd = TCPWND_MIN16(TCP_WND);
    pcb->rcv_ann_right_edge = pcb->rcv_nxt;
    pcb->snd_wnd = TCP_WND;
    pcb->mss = INITIAL_MSS;
    pcb->mss = tcp_eff_send_mss_netif(pcb->mss,
                                      netif,
                                      &pcb->remote_ip);
    pcb->cwnd = 1;
    pcb->connected = connected;

    /* 构造一个连接请求报文，将 SYN 标志置 1 */
    ret = tcp_enqueue_flags(pcb, TCP_SYN);
    if (ret == ERR_OK)
    {
        /* 改变 TCP 控制块的状态为 SYN_SENT */
        pcb->state = SYN_SENT;

        if (old_local_port != 0)
        {
            //将控制块从 tcp_bound_pcbs 链表移除
            TCP_RMV(&tcp_bound_pcbs, pcb);
        }

        //添加到 tcp_active_pcbs 链表中
        TCP_REG_ACTIVE(pcb);
        MIB2_STATS_INC(mib2.tcpactiveopens);

        //将控制块上的报文发送出去
        tcp_output(pcb);
    }
```

(下页继续)

(续上页)

```
    return ret;
}
```

在使用这个函数的时候，除了需要传递 IP 地址与端口号以外，还需要传入一个 tcp_connected_fn 类型的回调函数，内核会自动注册在 TCP 控制块中，当建立连接之后，就会调用 connected() 这个回调函数，具体见*代码清单 17_13*

代码清单 17-13tcp_connected_fn 类型的回调函数

```
typedef err_t (*tcp_connected_fn)(void *arg,
                                  struct tcp_pcb *tpcb,
                                  err_t err);
```

17.2.6 终止连接 tcp_close()

在任意时候，应用程序都可以主动调用 tcp_close() 函数终止一个 TCP 连接，终止连接的这个过程建议结合 TCP 状态转移图来学习，内核会根据处于不同状态的 TCP 控制块有不一样的处理，当 TCP 控制块处于关闭状态 CLOSED 的时候，会将 TCP 控制块从绑定链表 tcp_bound_pcbs 中移除，并且释放 TCP 控制块的内存空间；当 TCP 控制块处于监听状态的时候，那么会将 TCP 控制块从监听链表 tcp_listen_pcbs 中移除，并且释放控制块的内存空间；当 TCP 控制块处于 SYN_SENT 状态时，就将 TCP 控制块从 tcp_active_pcbs 链表中删除，并且释放控制块的内存空间；而对于处于其他状态的 TCP 控制块，直接通过 tcp_close_shutdown_fin() 函数来处理，主动关闭 TCP 连接，具体见*代码清单 17_14*。

代码清单 17-14 tcp_close() 源码

```
err_t
tcp_close(struct tcp_pcb *pcb)
{
    if (pcb->state != LISTEN)
    {
        tcp_set_flags(pcb, TF_RXCLOSED);
```

(下页继续)

(续上页)

```
    }

    return tcp_close_shutdown(pcb, 1);
}

static err_t
tcp_close_shutdown(struct tcp_pcb *pcb,
                u8_t rst_on_unacked_data)
{
    if (rst_on_unacked_data && ((pcb->state == ESTABLISHED)
                                || (pcb->state == CLOSE_WAIT)))
    {
        if ((pcb->refused_data != NULL)
            || (pcb->rcv_wnd != TCP_WND_MAX(pcb)))
    {
            /* 发送 TCP RESET 数据包（设置了 RST 标志） */
            tcp_rst(pcb, pcb->snd_nxt,
                    pcb->rcv_nxt,
                    &pcb->local_ip,
                    &pcb->remote_ip,
                    pcb->local_port,
                    pcb->remote_port);

            tcp_pcb_purge(pcb);
            TCP_RMV_ACTIVE(pcb);
            /* 因为已经为它发送了一个 RST，所以取消分配 pcb */
            if (tcp_input_pcb == pcb)
            {
                tcp_trigger_input_pcb_close();
            }
            else
            {
```

(下页继续)

(续上页)

```
                tcp_free(pcb);
            }
            return ERR_OK;
        }
    }

    /* 根据不同的状态进行不同的处理 */
    switch (pcb->state)
    {
    //关闭状态
    case CLOSED:
        if (pcb->local_port != 0)
        {
            TCP_RMV(&tcp_bound_pcbs, pcb);
        }
        tcp_free(pcb);
        break;
    //监听状态
    case LISTEN:
        tcp_listen_closed(pcb);
        tcp_pcb_remove(&tcp_listen_pcbs.pcbs, pcb);
        tcp_free_listen(pcb);
        break;
    //握手状态
    case SYN_SENT:
        TCP_PCB_REMOVE_ACTIVE(pcb);
        tcp_free(pcb);
        MIB2_STATS_INC(mib2.tcpattemptfails);
        break;
    //其他状态
    default:
        return tcp_close_shutdown_fin(pcb);
```

(下页继续)

(续上页)

```
    }
    return ERR_OK;
}
```

17.2.7 接收数据 tcp_recv()

这个函数的功能就是想控制块中的 recv 字段注册一个回调函数，当内核收到数据的时候就会调用这个回调函数，进而让数据递交到应用层中。回调函数的传入参数有 4 个，其中主要的是 tpcb，它是 TCP 控制块，表示了哪个 TCP 连接；p 是 pbuf 指针，指向接收到数据的 pbuf，当内核检测到对方主动终止 TCP 连接的时候，也会触发回调函数，此时的 pbuf 为空，而对于这种情况，用户就需要进行处理，也需要调用 tcp_close() 函数来终止本地到远端方向上的 TCP 连接。一般来说，这个函数在 connected() 函数中调用，具体见*代码清单 17_15*。

代码清单 17-15 tcp_recv() 源码

```
typedef err_t (*tcp_recv_fn)(void *arg,
                             struct tcp_pcb *tpcb,
                             struct pbuf *p,
                             err_t err);


void
tcp_recv(struct tcp_pcb *pcb, tcp_recv_fn recv)
{
    LWIP_ASSERT_CORE_LOCKED();
    if (pcb != NULL)
    {
        pcb->recv = recv;
    }
}
```

17.2.8 发送数据 tcp_sent()

这个函数是用于注册一个发送的回调函数，即将一个 tcp_sent_fn 类型的函数注册到 TCP 控制块的 sent 字段中，当发送的数据被对方确认接收后，内核会将发送窗口向后移动，并且调用这个注册的回调函数告诉应用，数据已经被对方接收了，那么用户就可以根据这个函数来将那些已经发送的数据删除掉或者发送新的数据，具体见*代码清单 17_16*.

代码清单 17-16tcp_sent() 源码

```
typedef err_t (*tcp_sent_fn)(void *arg,
                             struct tcp_pcb *tpcb,
                             u16_t len);


void
tcp_sent(struct tcp_pcb *pcb, tcp_sent_fn sent)
{
    LWIP_ASSERT_CORE_LOCKED();
    if (pcb != NULL)
    {
        pcb->sent = sent;
    }
}
```

17.2.9 异常处理 tcp_err()

这个函数是用于注册一个异常处理的函数，它向 TCP 控制块的 err 字段中注册一个 tcp_err_fn 类型的异常处理函数，用户需要自行编写这个函数，可以拥有完成在连接异常的一些处理，比如连接失败的时候，我们可以释放 TCP 控制块的内存空间、或者选择重连等等，具体见*代码清单 17_17*。

代码清单 17-17 tcp_err() 源码

```
typedef void  (*tcp_err_fn)(void *arg, err_t err);

void
tcp_err(struct tcp_pcb *pcb, tcp_err_fn err)
{
    LWIP_ASSERT_CORE_LOCKED();
    if (pcb != NULL)
    {
        pcb->errf = err;
    }
}
```

17.2.10 周期性回调 tcp_poll()

该函数用于在 TCP 控制块的 poll 字段注册一个类型为 tcp_poll_fn 的回调函数（函数的名字也是 poll），内核会周期性调用控制块中的 poll 回调函数，调用的周期为 interval*0.5s，因为 0.5s 是内核定时器的处理周期，用户可以适当使用 poll 回调函数完成一些周期性的事件，比如检测连接的情况、周期性发送一些数据等等，因为使用裸机编程（RAW API），确实是不太好处理这些周期性的事件，所以 LwIP 为用户考虑了很多，该函数源码具体见*代码清单 17_18*。

代码清单 17-18 tcp_poll() 源码

```
typedef err_t (*tcp_poll_fn)(void *arg, struct tcp_pcb *tpcb);

void
tcp_poll(struct tcp_pcb *pcb, tcp_poll_fn poll, u8_t interval)
{
    LWIP_ASSERT_CORE_LOCKED();

    pcb->poll = poll;

    pcb->pollinterval = interval;
```

(下页继续)

(续上页)

```
}
```

17.2.11 构建报文段 tcp_write()

在一个稳定的 TCP 连接中，我们可以调用 tcp_write() 函数来构建一个 TCP 报文段，这个函数我们在 TCP 发送讲解的时候稍作介绍过，因为如果使用 NETCONN API 与 Socket API 的话，其实不用了解这个函数，即使这两种 API 最终都是调用这个函数来构建 TCP 报文段的，但是上层 API 全部给我们封装好了，但是在使用 RAW API 的时候，我们需要调用这个函数直接构建报文段，这就需要我们对这个函数有一定了解，不过这个函数终究还是太复杂（代码多达 400 行），我就不讲解它里面是怎么实现的，只简单介绍一下整个函数的运作过程就好了，其函数原型具体见*代码清单 17_19*。

代码清单 17-19 tcp_write() 函数原型

```
err_t         tcp_write(struct tcp_pcb *pcb,
                        const void *dataptr,
                        u16_t len,
                        u8_t apiflags);
```

该函数有 4 个参数，其中 pcb 是相应的 TCP 连接，dataptr 是数据指针，len 是数据长度，以字节为单位，apiflags 是表示应用程序期望内核对报文段的额外操作，如是否拷贝数据，是否设置首部中的 PSH 标志等等。

该函数首先会调用 tcp_write_checks() 函数检查一下是否允许构建报文段，看一下数据是否能挂载到发送缓冲区中；接下来内核会将可发送的数据组成 TCP 报文段并且挂载到 TCP 控制块的 unsent 字段中，注意了，tcp_write() 函数只是构建 TCP 报文段并缓存在 unsent 字段中，真正发送 TCP 数据的函数是 tcp_output()，这个函数在讲解 TCP 发送数据的时候就讲解过了，此处不再重复赘述。那么很多同学就有疑问了，为什么这个函数不是发送数据还要调用它呢？其实这个函数在构建好一个 TCP 报文段之后，内核会在超时处理中调用 tcp_output() 函数进行发送，而后者是根据 TCP 控制块 unsent 字段的内容进行发送数据的，因此，我们只需要把数据挂载到 unsent 字段中即可，内核会处理剩下的事情，当然啦，如果想要立即发送数据，也是可以在 tcp_write() 函数后调用 tcp_output() 函数，就可以立即发送数据。

17.2.12 更新接收窗口 tcp_recved()

其实在用户接收到数据之后，应该调用一下这个函数来更新接收窗口，因为内核不知道应用层是否真正接收到数据，如果不调用这个函数，就没法进行确认，而发送的一方会认为对方没有接收到，因此会重发数据。在这个函数中，它会调用 tcp_update_rcv_ann_wnd() 函数进行更新接收窗口，以告知发送方能发送多大的数据，参数 pcb 是对应的 TCP 连接控制块，len 表示应用程序已经处理完的数据长度，那么接收窗口也会增大 len 字节的长度，具体见*代码清单 17_20*。

顺便提一下，如果发现应用程序中无法接收数据，但是能发送数据，那么很可能就是在接收到数据之后没调用 tcp_recved() 函数来更新接收窗口，导致接收窗口为 0，无法接收数据。

代码清单 17-20tcp_recved() 源码

```
void
tcp_recved(struct tcp_pcb *pcb, u16_t len)
{
    u32_t wnd_inflation;
    tcpwnd_size_t rcv_wnd;

    LWIP_ASSERT_CORE_LOCKED();

    rcv_wnd = (tcpwnd_size_t)(pcb->rcv_wnd + len);
    if ((rcv_wnd > TCP_WND_MAX(pcb)) || (rcv_wnd < pcb->rcv_wnd))
    {
        /* 窗口太大或 tcpwnd_size_t 溢出 */
        pcb->rcv_wnd = TCP_WND_MAX(pcb);
    }
    else
    {
        pcb->rcv_wnd = rcv_wnd;
    }

    wnd_inflation = tcp_update_rcv_ann_wnd(pcb);
```

(下页继续)

(续上页)

```
    /* 如果窗口编号大于 TCP_WND/4 ，立即发送一个 ack*/
    if (wnd_inflation >= TCP_WND_UPDATE_THRESHOLD)
    {
        tcp_ack_now(pcb);
        tcp_output(pcb);
    }

}
```

17.3 实验

17.3.1 TCP Client

在这个实验中，我们使用 RAW API 实现一个 TCP 客户端，我们整个流程如下：

1. 创建一个 TCP 控制块。
2. 调用 tcp_connect() 函数与服务器建立连接。
3. 注册一个异常处理函数，在连接失败的时候进行合适的处理。
4. 建立连接之后就注册一个周期性发送数据的函数与接收处理函数，接收处理函数主要是用于在服务器主动断开的时候进行重连操作。
5. 周期性发送数据到服务器中。

首先我们拿到一个移植好的 LwIP 裸机例程，然后添加两个文件，分别为 tcpclient.c 与 tcpclient.h，然后添加以下代码，具体见*代码清单 17_21* 与*代码清单 17_21*。

实验的现象是与 NETCONN API 的实验现象是一致的，此处就不重复赘述。

代码清单 17-21tcpclient.c 文件内容

```c
#include "tcpclient.h"
#include "lwip/netif.h"
#include "lwip/ip.h"
#include "lwip/tcp.h"
#include "lwip/init.h"
#include "netif/etharp.h"
#include "lwip/udp.h"
#include "lwip/pbuf.h"
#include <stdio.h>
#include <string.h>


static struct tcp_pcb *client_pcb = NULL;

static void client_err(void *arg, err_t err)
{
    printf("connect error! closed by core!!\n");
    printf("try to connect to server again!!\n");

    //连接失败的时候释放 TCP 控制块的内存
    tcp_close(client_pcb);

    //重新连接
    TCP_Client_Init();
}


static err_t client_send(void *arg, struct tcp_pcb *tpcb)
{
    uint8_t send_buf[]= "This is a TCP Client test...\n";

    //发送数据到服务器
```

(下页继续)

(续上页)

```
    tcp_write(tpcb, send_buf, sizeof(send_buf), 1);

    return ERR_OK;
}

static err_t client_recv(void *arg,
                         struct tcp_pcb *tpcb,
                         struct pbuf *p,
                         err_t err)
{
    if (p != NULL)
    {
        /* 更新窗口 */
        tcp_recved(tpcb, p->tot_len);

        /* 返回接收到的数据 */
        tcp_write(tpcb, p->payload, p->tot_len, 1);

        memset(p->payload, 0 , p->tot_len);
        pbuf_free(p);
    }
    else if (err == ERR_OK)
    {
        //服务器断开连接
        printf("server has been disconnected!\n");
        tcp_close(tpcb);

        //重新连接
        TCP_Client_Init();
    }
    return ERR_OK;
}
```

(下页继续)

(续上页)

```
static err_t client_connected(void *arg,
                              struct tcp_pcb *pcb,
                              err_t err)
{
    printf("connected ok!\n");

    //注册一个周期性回调函数
    tcp_poll(pcb,client_send,2);

    //注册一个接收函数
    tcp_recv(pcb,client_recv);

    return ERR_OK;
}


void TCP_Client_Init(void)
{
    ip4_addr_t server_ip;
    /* 创建一个 TCP 控制块  */
    client_pcb = tcp_new();

    IP4_ADDR(&server_ip, 192,168,0,181);

    printf("client start connect!\n");

    //开始连接
    tcp_connect(client_pcb,
                &server_ip,
                TCP_CLIENT_PORT,
                client_connected);
```

(下页继续)

(续上页)

```
    //注册异常处理
    tcp_err(client_pcb, client_err);
}
```

代码清单 17-22tcpclient.h 文件内容

```
#ifndef _TCPCLIENT_H_
#define _TCPCLIENT_H_

#define TCP_CLIENT_PORT 5001

void TCP_Client_Init(void);

#endif
```

然后根据自己开发板所处的网络环境，修改 main.h 的网络相关的配置信息，注意，此工程是裸机工程，因此我放置的网络配置信息所处的位置是不一样的，此处在 main.h 文件中，具体见代码清单 17_23。

代码清单 17-23main.h 的网络信息

```
/* USER CODE BEGIN 0 */
#define DEST_IP_ADDR0               192
#define DEST_IP_ADDR1               168
#define DEST_IP_ADDR2                 0
#define DEST_IP_ADDR3               102

#define DEST_PORT                  6000

#define UDP_SERVER_PORT            5002
#define UDP_CLIENT_PORT            5002
```

(下页继续)

(续上页)

```
#define LOCAL_PORT                  5001

/*Static IP ADDRESS: IP_ADDR0.IP_ADDR1.IP_ADDR2.IP_ADDR3 */
#define IP_ADDR0                    192
#define IP_ADDR1                    168
#define IP_ADDR2                      0
#define IP_ADDR3                    122

/*NETMASK*/
#define NETMASK_ADDR0               255
#define NETMASK_ADDR1               255
#define NETMASK_ADDR2               255
#define NETMASK_ADDR3                 0

/*Gateway Address*/
#define GW_ADDR0                    192
#define GW_ADDR1                    168
#define GW_ADDR2                      0
#define GW_ADDR3                      1
/* USER CODE END 0 */
```

然后在 main 函数中调用一下 TCP_Client_Init() 函数，进行开发客户端的初始化即可，main.c 文件内容具体见*代码清单 17_24*。

代码清单 17-24main.c 文件内容

```
#include "main.h"

#include <lwip/opt.h>
#include <lwip/arch.h>
#include "tcpip.h"
#include "lwip/init.h"
```

(下页继续)

(续上页)

```
#include "lwip/netif.h"
#include "ethernetif.h"
#include "netif/ethernet.h"
#include "lwip/def.h"
#include "lwip/stats.h"
#include "lwip/etharp.h"
#include "lwip/ip.h"
#include "lwip/snmp.h"
#include "lwip/timeouts.h"

#include "tcpclient.h"

struct netif gnetif;
ip4_addr_t ipaddr;
ip4_addr_t netmask;
ip4_addr_t gw;
uint8_t IP_ADDRESS[4];
uint8_t NETMASK_ADDRESS[4];
uint8_t GATEWAY_ADDRESS[4];

void LwIP_Init(void)
{
    /* IP addresses initialization */
    /* USER CODE BEGIN 0 */
#ifdef USE_DHCP
    ip_addr_set_zero_ip4(&ipaddr);
    ip_addr_set_zero_ip4(&netmask);
    ip_addr_set_zero_ip4(&gw);
#else
    IP4_ADDR(&ipaddr,IP_ADDR0,IP_ADDR1,IP_ADDR2,IP_ADDR3);
    IP4_ADDR(&netmask,NETMASK_ADDR0,NETMASK_ADDR1,
            NETMASK_ADDR2,NETMASK_ADDR3);
```

(下页继续)

(续上页)

```
    IP4_ADDR(&gw,GW_ADDR0,GW_ADDR1,GW_ADDR2,GW_ADDR3);
#endif /* USE_DHCP */
    /* USER CODE END 0 */

    /* Initilialize the LwIP stack without RTOS */
    lwip_init();

    /* add the network interface (IPv4/IPv6) without RTOS */
    netif_add(&gnetif, &ipaddr, &netmask,
              &gw, NULL, &ethernetif_init, &ethernet_input);

    /* Registers the default network interface */
    netif_set_default(&gnetif);

    if (netif_is_link_up(&gnetif))
    {
/* When the netif is fully configured this function must be called */
        netif_set_up(&gnetif);
    }
    else
    {
        /* When the netif link is down this function must be called */
        netif_set_down(&gnetif);
    }

    /* USER CODE BEGIN 3 */

    /* USER CODE END 3 */
}


int flag = 0;
```

(下页继续)

(续上页)

```
int main(void)
{
    //板级外设初始化
    BSP_Init();

    //LwIP 协议栈初始化
    LwIP_Init();
    TCP_Client_Init();       //开发板客户端初始化
    while (1)
    {
        if (flag)
        {
            flag = 0;
            //调用网卡接收函数
            ethernetif_input(&gnetif);
        }
        //处理 LwIP 中定时事件
        sys_check_timeouts();
    }
}
```

17.3.2 TCP Server

在这个实验中，我们使用开发板做一个 TCP 回显服务器，等待着上位机软件的连接，建立连接之后，将客户端发送的数据回显到客户端中，其步骤如下：

1. 创建一个 TCP 控制块。
2. 绑定本地 IP 地址与端口号。
3. 进入监听状态。
4. 处理来自客户端的连接请求。

5. 连接完成后注册一个接收数据回调函数。

6. 在回调函数中更新接收窗口，并且将接收到的数据返回到客户端中。

在 LwIP 的裸机工程中创建两个文件，分别为 tcpecho.c 与 tcpecho，并且添加以下代码，具体见*代码清单 17_25* 与*代码清单 17_26*，然后在 main 函数中调用 TCP_Echo_Init() 函数即可。

此实验的现象是与 NETCONN API 的实验现象是一致的，此处就不重复赘述。

代码清单 17-25tcpecho.c 文件内容

```
#include "tcpecho.h"
#include "lwip/netif.h"
#include "lwip/ip.h"
#include "lwip/tcp.h"
#include "lwip/init.h"
#include "netif/etharp.h"
#include "lwip/udp.h"
#include "lwip/pbuf.h"
#include <stdio.h>
#include <string.h>


static err_t tcpecho_recv(void *arg,
                          struct tcp_pcb *tpcb,
                          struct pbuf *p,
                          err_t err)
{
    if (p != NULL)
    {
        /* 更新窗口 */
        tcp_recved(tpcb, p->tot_len);

        /* 返回接收到的数据 */
        tcp_write(tpcb, p->payload, p->tot_len, 1);
```

(下页继续)

(续上页)

```
        memset(p->payload, 0 , p->tot_len);
        pbuf_free(p);

    }
    else if (err == ERR_OK)
    {
        return tcp_close(tpcb);
    }
    return ERR_OK;
}

static err_t tcpecho_accept(void *arg,
                            struct tcp_pcb *newpcb,
                            err_t err)
{
    tcp_recv(newpcb, tcpecho_recv);
    return ERR_OK;
}

void TCP_Echo_Init(void)
{
    struct tcp_pcb *pcb = NULL;

    /* 创建一个 TCP 控制块  */
    pcb = tcp_new();

    /* 绑定 TCP 控制块 */
    tcp_bind(pcb, IP_ADDR_ANY, TCP_ECHO_PORT);


    /* 进入监听状态 */
```

(下页继续)

(续上页)

```
    pcb = tcp_listen(pcb);

    /* 处理连接 */
    tcp_accept(pcb, tcpecho_accept);
}
```

代码清单 17-26tcpecho.h 文件内容

```
#ifndef _TCPECHO_H_
#define _TCPECHO_H_

#define TCP_ECHO_PORT 5001

void TCP_Echo_Init(void);

#endif
```

17.3.3 UDP

在这个实验中，我们用开发板与电脑上位机软件建立一个 UDP 会话，并且将上位机软件发送给开发板的数据返回去，做一个 UDP 回显的实验，其实验步骤如下：

1. 新建一个 UDP 控制块
2. 将控制块与本地 IP 地址、端口号进行绑定。
3. 注册一个接收数据的回调函数
4. 在接收回调函数中将接收到的数据发送到对端主机中。
5. 释放 pbuf。

首先将 LwIP 的裸机工程拷贝一份，然后添加两个文件，分别为 udpecho.c 与 udpecho.h，然后添加以下代码，然后在 main 函数中调用 UDP_Echo_Init() 函数即可，具体见*代码清单 17_27* 与*代码清单 17_28*。

这个实验现象是与 NETCONN API 的实验现象是一致的，此处就不重复赘述。

代码清单 17-27udpecho.c 文件内容

```
#include "udpecho.h"
#include "lwip/netif.h"
#include "lwip/ip.h"
#include "lwip/tcp.h"
#include "lwip/init.h"
#include "netif/etharp.h"
#include "lwip/udp.h"
#include "lwip/pbuf.h"
#include <stdio.h>


static void udp_demo_callback(void *arg,
                              struct udp_pcb *upcb,
                              struct pbuf *p,
                              const ip_addr_t *addr,
                              u16_t port)
{
    struct pbuf *q = NULL;
    const char* reply = "This is reply!\n";

    if (arg)
    {
        printf("%s",(char *)arg);
    }

    pbuf_free(p);

    q = pbuf_alloc(PBUF_TRANSPORT, strlen(reply)+1, PBUF_RAM);
    if (!q)
    {
```

(下页继续)

(续上页)

```
        printf("out of PBUF_RAM\n");
        return;
    }

    memset(q->payload, 0 , q->len);
    memcpy(q->payload, reply, strlen(reply));
    udp_sendto(upcb, q, addr, port);
    pbuf_free(q);
}

static char * st_buffer= "We get a data\n";
void UDP_Echo_Init(void)
{
    struct udp_pcb *udpecho_pcb;
    /* 新建一个控制块 */
    udpecho_pcb = udp_new();

    /* 绑定端口号 */
    udp_bind(udpecho_pcb, IP_ADDR_ANY, UDP_ECHO_PORT);

    /* 注册接收数据回调函数 */
    udp_recv(udpecho_pcb, udp_demo_callback, (void *)st_buffer);
}
```

代码清单 17-28 udpecho.h 文件内容

```
#ifndef _UDPECHO_H_
#define _UDPECHO_H_

#define UDP_ECHO_PORT 5001

void UDP_Echo_Init(void);
```

(下页继续)

(续上页)

```
#endif
```

第 18 章　使用 JPerf 工具测试网速

18.1 iPerf 与 JPerf

在讲解网络测速之前，我们先来了解一下测速的工具：iPerf 是一个跨平台的网络性能测试工具，它支持 Win/Linux/Mac/Android/iOS 等平台，iPerf 可以测试 TCP 和 UDP（我们一般不对 UDP 进行测速）带宽质量，iPerf 可以测量最大 TCP 带宽，可以具有多种参数进行测试，同时 iPerf 还可以报告带宽，延迟抖动和数据包丢失的情况，我们可以利用 iPerf 的这些特性来测试一些网络设备如路由器，防火墙，交换机等的性能。

虽然 iPerf 很好用，但是它却是命令行格式的软件，对使用测试的人员并不友好，使用者需要记下他繁琐的命令，不过它还有一个图形界面程序叫做 JPerf，使用 JPerf 程序能简化了复杂命令行参数的构造，而且它还保存测试结果，并且将测试结果实时图形化出来，更加一目了然，当然，JPerf 也肯定拥有 iPerf 的所有功能。

18.2 测试网络速度

18.2.1 获取 JPerf 网络测速工具

在测速之前，我们需要得到网络测速工具，在我们的论坛上有这个工具，然后，我们直接下载即可：http://www.firebbs.cn/forum.php?mod=viewthread&tid=26274&fromuid=37393。

下载后解压，双击 jperf.bat 运行，稍等一会就出现 JPerf 的界面，具体见图 *18_1*。

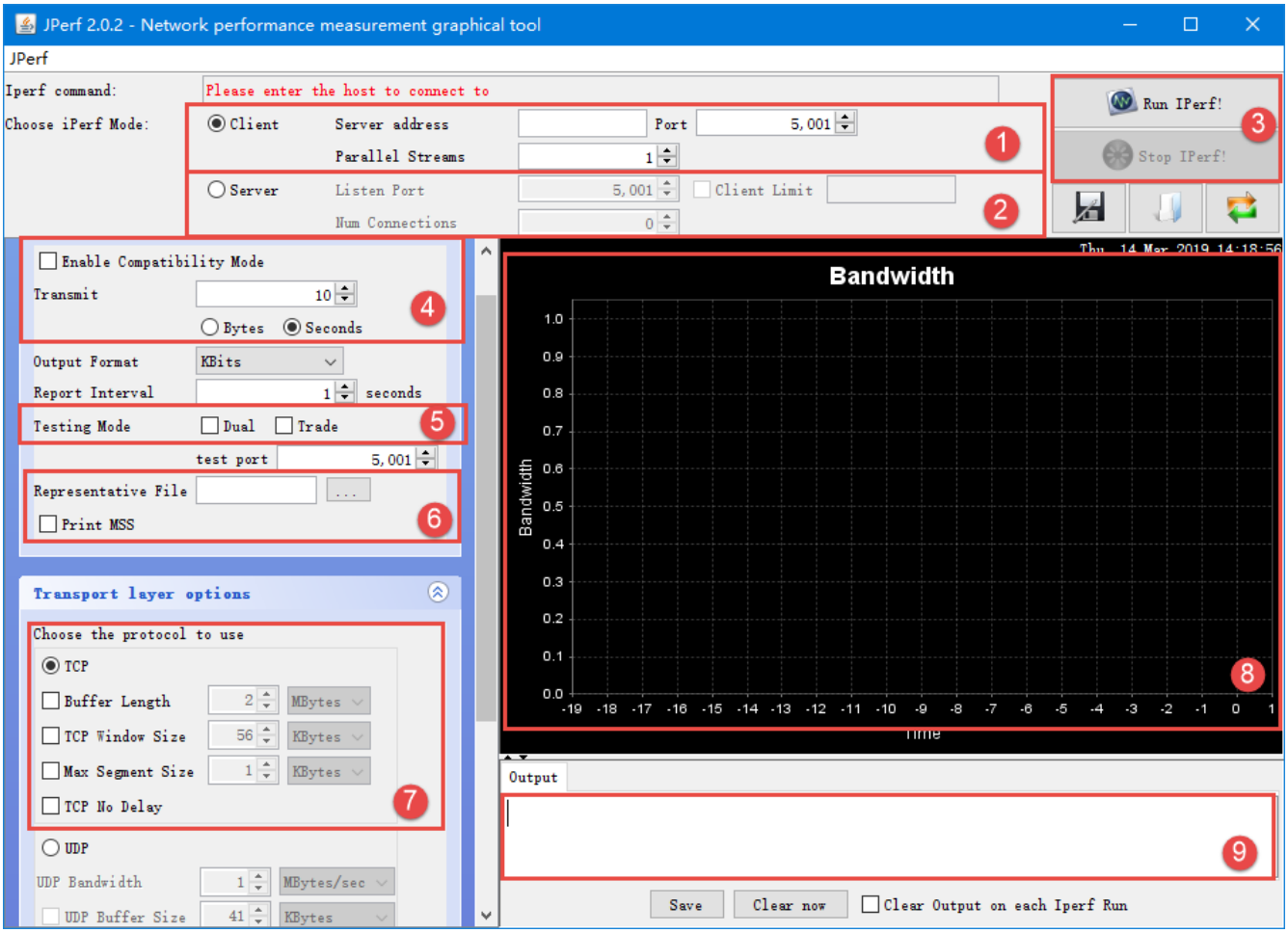


图 18-1JPerf 界面

我们来讲解一下这个界面的一些内容：

图 18-1(**1**)：客户端设置，电脑作为客户端，连接到服务器中（即我们的开发板作为服务器），服务器地址需要填写正确，端口号默认是 5001，并发流默认是 1 个。

图 18-1(**2**)：服务器设置，电脑作为服务器，我们的开发板作为客户端，client limit 选项表示仅允许指定客户端连接，Num Connections 指定最大允许连接的数量，为 0 不限制。

图 18-1(**3**)：开始和停止 JPerf 的运行。

图 18-1(**4**)：兼容旧版本（当 server 端和 client 端版本不一样时使用），默认不勾选，Transmit 设置测试模式，我们一般指定发送的时间，以秒为单位，当然也可以指定发送的数据大小，以字节为单位。

图 18-1**(5)**：如果勾选 Dual 表示同时进行双向传输测试，如果勾选 Trade 表示单独进行双向传输测试，默认不勾选。

图 18-1**(6)**：指定需要传输的文件以及显示最大 TCP 报文段。

图 18-1**(7)**：传输层设置，我们一般用来测试 TCP 连接的速度，Buffer Length 选项用于设置缓冲区大小，TCP Window Size 用于指定 TCP 窗口大小，Max Segment Size 用于设定最大 MTU 值，TCP No Delay 用于设定 TCP 不延时。

图 18-1**(8)**：网速显示窗口，以折线图的形式显示出来。

图 18-1**(9)**：网速相关数据输出窗口，以文本的形式。

18.2.2 测试开发板接收速度（NETCONN API）

首先，我们肯定需在开发板上开发程序的，那么我们就单独创建一个 iPerf 测速线程，在开发板上运行，开发板作为客户端，不断监听客户端（JPerf 上位机）的连接。

代码实现部分：我们首先拷贝一个移植好的工程，并且在工程中添加两个文件，分别为 ipref.c 和 ipref.h，然后在 ipref.c 文件下添加*代码清单 18_1* 中的代码，其实这个代码跟 TCP 服务器实验的代码都是差不多的，只不过接收到数据不进行处理而已，在 ipref.h 文件下添加*代码清单 18_2* 中的代码。

代码清单 18-1ipref.c 文件内容

```
/* FreeRTOS 头文件 */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"

#include <stdint.h>
#include <stdio.h>

#include <lwip/sockets.h>
```

(下页继续)

(续上页)

```
#include "iperf.h"

#include "lwip/opt.h"

#include "lwip/sys.h"
#include "lwip/api.h"

#define IPERF_PORT          5001
#define IPERF_BUFSZ         (4 * 1024)

void iperf_server(void *thread_param)
{
    struct netconn *conn, *newconn;
    err_t err;
    void* recv_data;

    recv_data = (void *)pvPortMalloc(IPERF_BUFSZ);
    if (recv_data == NULL)    {
        printf("No memory\n");
    }

    conn = netconn_new(NETCONN_TCP);
    netconn_bind(conn, IP_ADDR_ANY, 5001);

    LWIP_ERROR("tcpecho: invalid conn", (conn != NULL), return;);

    /* Tell connection to go into listening mode. */
    netconn_listen(conn);

    while (1)
    {
```

(下页继续)

(续上页)

```
        /* Grab new connection. */
        err = netconn_accept(conn, &newconn);
        /*printf("accepted new connection %p\n", newconn);*/
        /* Process the new connection. */
        if (err == ERR_OK)
        {
            struct netbuf *buf;
//          void *data;
            u16_t len;

            while ((err = netconn_recv(newconn, &buf)) == ERR_OK)
            {
                /*printf("Recved\n");*/
                do
                {
                    netbuf_data(buf, &recv_data, &len);
//                  err = netconn_write(newconn, data, len, NETCONN_COPY);
                }
                while (netbuf_next(buf) >= 0);
                netbuf_delete(buf);
            }
            /*printf("Got EOF, looping\n");*/
            /* Close connection and discard connection identifier. */
            netconn_close(newconn);
            netconn_delete(newconn);
        }
    }
}

void
iperf_server_init(void)
```

(下页继续)

(续上页)

```
{
    sys_thread_new("iperf_server", iperf_server, NULL, 2048, 4);
}
```

代码清单 18-2ipref.h 文件内容

```
#ifndef LWIP_IPERF_H
#define LWIP_IPERF_H


#define TCP_SERVER_THREAD_NAME              "iperf_server"
#define TCP_SERVER_THREAD_STACKSIZE          1024
#define TCP_SERVER_THREAD_PRIO               4

void iperf_server(void *thread_param);
void iperf_server_init(void);
```

在 main.c 文件中将 iperf_server_init() 调用一下即可。并且配置好开发板的 IP 地址与端口号，我们打开 JPerf 测速软件，配置好要连接的服务器 IP 地址与端口，测试时间设置得长一点，我们使用 10000 秒，然后点击开始，就得到我们需要的网速数据，速度高达 94Mbps，即 11.5M 字节/秒，已经是非常高的速度了，而且通过折线图，我们也能看到这速度是很稳定的，具体见图 *18_2*。

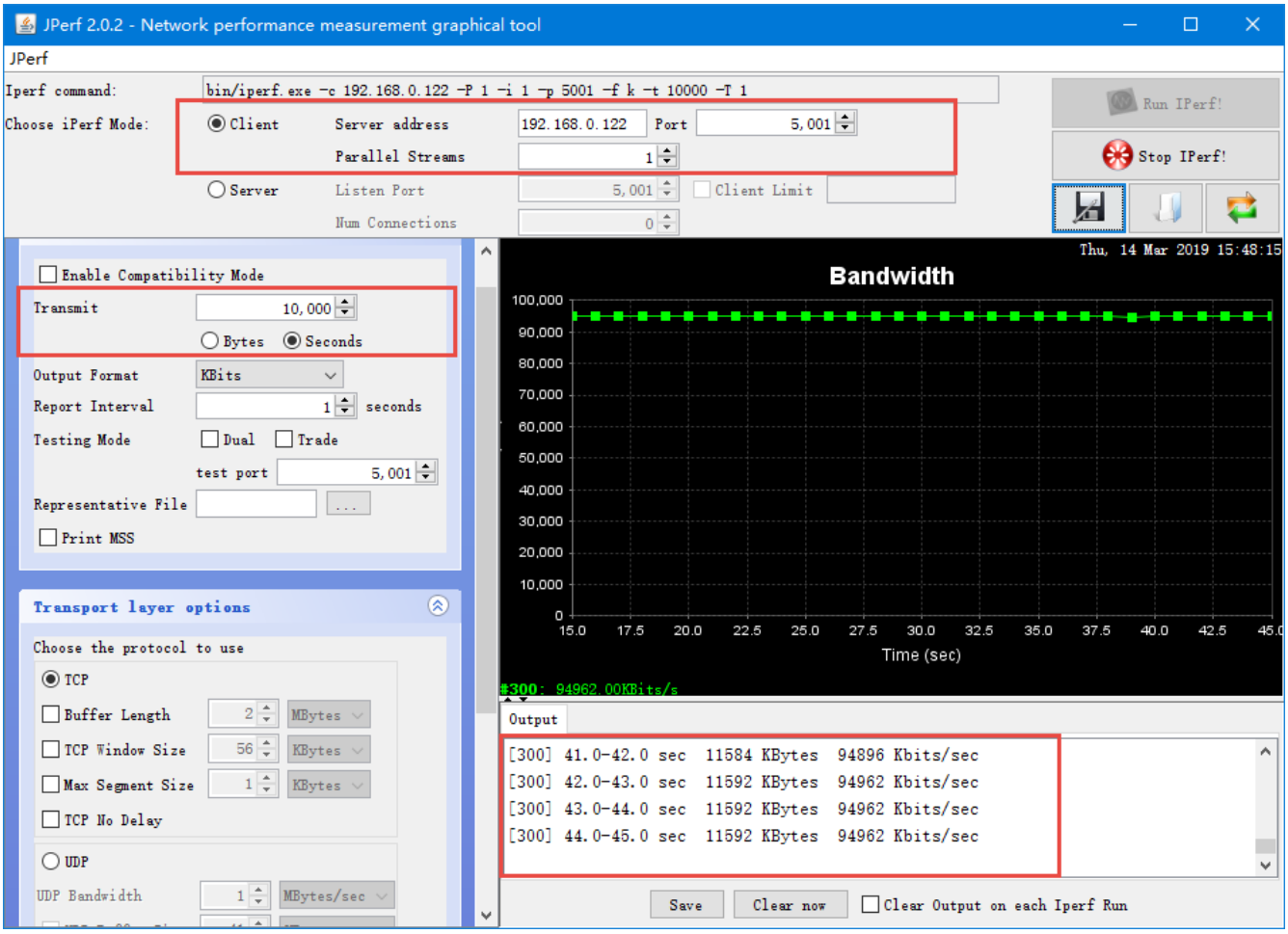


图 18-2NETCONN API 接收速度

18.2.3 测试开发板接收速度（Socket API）

这个实验我们只需要把上一个实验中的 ipref.c 文件内容替换掉就行了，具体见*代码清单 18_3*。

代码清单 18-3ipref.c 文件内容

```
/* FreeRTOS 头文件 */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
```

(下页继续)

(续上页)

```
#include "semphr.h"

#include <stdint.h>
#include <stdio.h>

//#include <lwip/time.h>
#include <lwip/sockets.h>
//#include <lwip/select.h>
//#include "netdb.h"
#include "iperf.h"

#include "lwip/opt.h"

#include "lwip/sys.h"
#include "lwip/api.h"

#define IPERF_PORT          5001
#define IPERF_BUFSZ         (4 * 1024)

void iperf_server(void *thread_param)
{
    uint8_t *recv_data;
    socklen_t sin_size;
    uint32_t tick1, tick2;
    int sock = -1, connected, bytes_received;
    uint64_t recvlen;
    struct sockaddr_in server_addr, client_addr;
    char speed[32] = { 0 };
    fd_set readset;
    struct timeval timeout;

    recv_data = (uint8_t *)pvPortMalloc(IPERF_BUFSZ);
```

(下页继续)

(续上页)

```
    if (recv_data == NULL)
    {
        printf("No memory\n");
        goto __exit;
    }

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0)
    {
        printf("Socket error\n");
        goto __exit;
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(IPERF_PORT);
    memset(&(server_addr.sin_zero), 0x0, sizeof(server_addr.sin_zero));

if (bind(sock, (struct sockaddr *)&server_addr, sizeof(struct sockaddr))␣
↪== -1)
    {
        printf("Unable to bind\n");
        goto __exit;
    }

    if (listen(sock, 5) == -1)
    {
        printf("Listen error\n");
        goto __exit;
    }

    timeout.tv_sec = 3;
```

(下页继续)

(续上页)

```
    timeout.tv_usec = 0;

    printf("iperf_server\n");
    while (1)
    {
        FD_ZERO(&readset);
        FD_SET(sock, &readset);

        if (select(sock + 1, &readset, NULL, NULL, &timeout) == 0)
            continue;

        printf("iperf_server\n");

        sin_size = sizeof(struct sockaddr_in);

        connected = accept(sock, (struct sockaddr *)&client_addr, &sin_
↪size);

        printf("new client connected from (%s, %d)\n",
            inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

        {
            int flag = 1;

            setsockopt(connected,
                    IPPROTO_TCP,     /* set option at TCP level */
                    TCP_NODELAY,     /* name of option */
                    (void *) &flag,  /* the cast is historical cruft */
                    sizeof(int));    /* length of option value */
        }

        recvlen = 0;
```

(下页继续)

(续上页)

```
        tick1 = xTaskGetTickCount();
        while (1)
        {
            bytes_received = recv(connected, recv_data, IPERF_BUFSZ, 0);
            if (bytes_received <= 0) break;

            recvlen += bytes_received;

            tick2 = xTaskGetTickCount();
            if (tick2 - tick1 >= configTICK_RATE_HZ * 5)
            {
                float f;
                f=(float)(recvlen * configTICK_RATE_HZ/125/(tick2-tick1));
                f /= 1000.0f;
//                snprintf(speed, sizeof(speed), "%.4f Mbps!\n", f);
//                printf("%s", speed);
                tick1 = tick2;
                recvlen = 0;
            }
        }

        if (connected >= 0) closesocket(connected);
        connected = -1;
    }

__exit:
    if (sock >= 0) closesocket(sock);
    if (recv_data) free(recv_data);
}

void
iperf_server_init(void)
```

(下页继续)

(续上页)

```
{
    sys_thread_new("iperf_server", iperf_server, NULL, 2048, 4);
}
```

然后得到数据，对比数据，我们发现，NETCONN API 的效率是比 Socket API 的效率更高，这是因为 Socket API 需要对数据进行拷贝，才能传递到上层应用中，不过 71Mbps（8.7M 字节/秒）的速度已经是算很高了，具体见图 *18_3*。

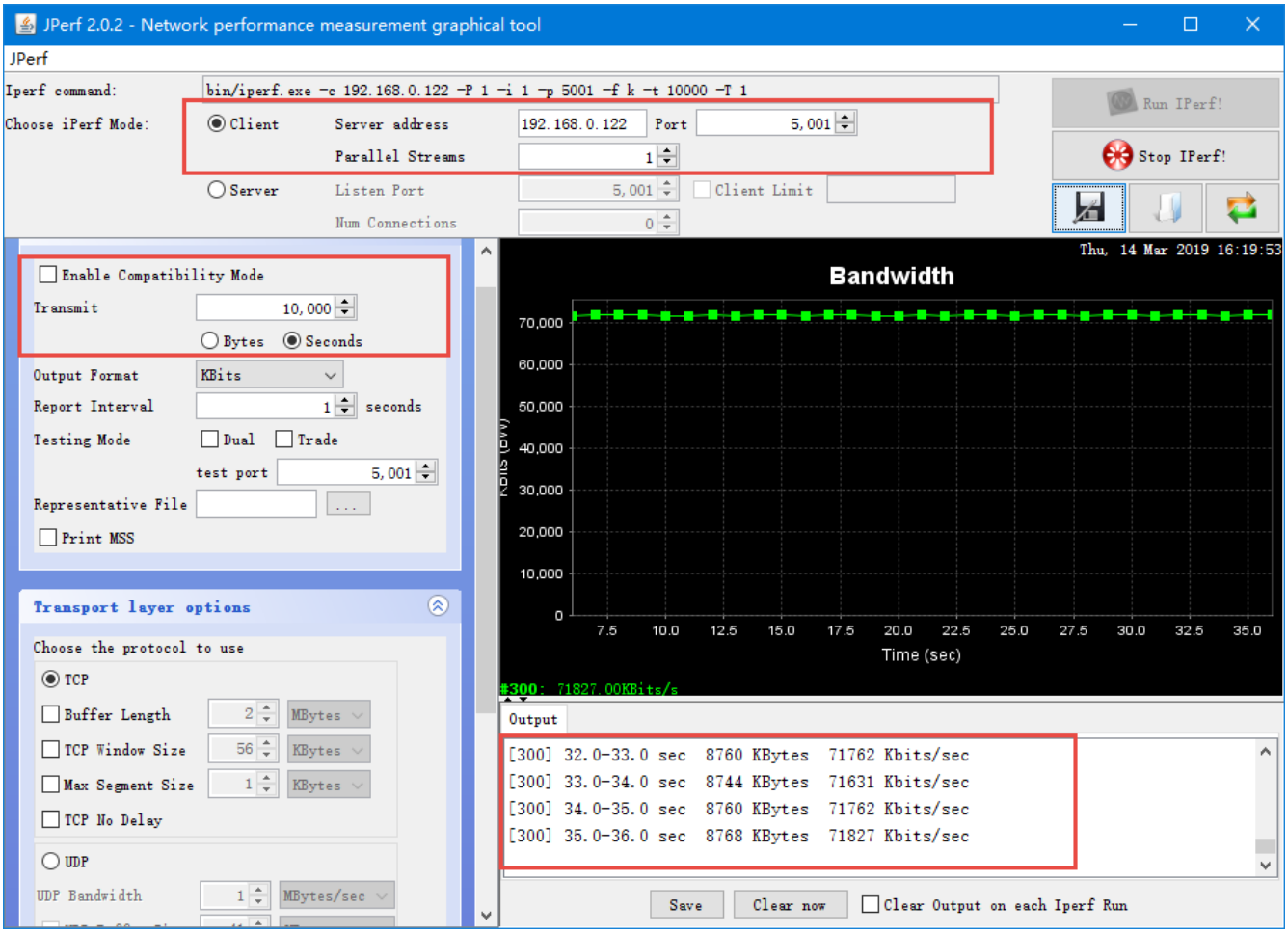


图 18-3Socket API 接收速度

18.2.4 测试开发板发送速度（NETCONN API）

测完两种 API 的接收速度，那么就来测试一下开发板的发送速度，发送速度其实是更加重要的，比如开发板采集一些图像，想要发送出去，如果发送速度跟不上的话，传输出去的图像就会卡帧，而发送速度足够快，就会很流畅，我们测试发送速度将开发板作为客户端，JPerf 软件则作为服务器，我们开发板向服务器发送数据。首先我们也是把移植好的工程拿过来，并且添加两个文件，分别为 iperf_client.c 和 iperf_client.h，然后在对应的文件中添加所示的代码

代码清单 18-4iperf_client.c 文件内容

```
#include "iperf_client.h"

#include "lwip/opt.h"

#include "lwip/sys.h"
#include "lwip/api.h"

#define IPERF_PORT          5001
#define IPERF_BUFSZ         (4 * 1024)

static void iperf_client(void *thread_param)
{
    struct netconn *conn;

    int i;

    int ret;

    uint8_t *send_buf;

    uint64_t sentlen;

    u32_t tick1, tick2;
```

(下页继续)

(续上页)

```
    ip4_addr_t ipaddr;


    send_buf = (uint8_t *) pvPortMalloc(IPERF_BUFSZ);
    if (!send_buf) return ;

    for (i = 0; i < IPERF_BUFSZ; i ++)
        send_buf[i] = i & 0xff;

    while (1)
    {
        conn = netconn_new(NETCONN_TCP);
        if (conn == NULL)
        {
            printf("create conn failed!\n");
            vTaskDelay(10);
            continue;
        }

        IP4_ADDR(&ipaddr,192,168,0,181);

        ret = netconn_connect(conn,&ipaddr,5001);
        if (ret == -1)
        {
            printf("Connect failed!\n");
            netconn_close(conn);
            vTaskDelay(10);
            continue;
        }

        printf("Connect to iperf server successful!\n");
```

(下页继续)

(续上页)

```
        tick1 = sys_now();
        while (1)
        {
            tick2 = sys_now();

            if (tick2 - tick1 >= configTICK_RATE_HZ * 5)
            {
                float f;
                f = (float)(sentlen*configTICK_RATE_HZ/125/(tick2 -␣
↪tick1));
                f /= 1000.0f;
                printf("send speed = %.4f Mbps!\n", f);

                tick1 = tick2;
                sentlen = 0;
            }
            ret = netconn_write(conn,send_buf,IPERF_BUFSZ,0);
            if (ret == ERR_OK)
            {
                sentlen += IPERF_BUFSZ;
            }

        }
//    netconn_close(conn);
//    netconn_delete(conn);
    }
}

void
iperf_client_init(void)
{
    sys_thread_new("iperf_client", iperf_client, NULL, 2048, 4);
```

(下页继续)

(续上页)

```
}
```

代码清单 18-5iperf_client.h 文件内容

```
#ifndef IPERF_CLIENT_H
#define IPERF_CLIENT_H

void iperf_client_init(void);

#endif /* IPERF_CLIENT_H */
```

在 main.c 文件中调用 iperf_client_init() 函数即可，然后配置 JPerf 软件成为客户端，让开发板进行连接，开发板连接的客户端 IP 地址与端口号根据实际情况去配置即可，具体见图 *18_4*，从实验现象可以看出，发送的速度还是很快的（94Mbps，即 11.5M 字节/秒）而且还很稳定。

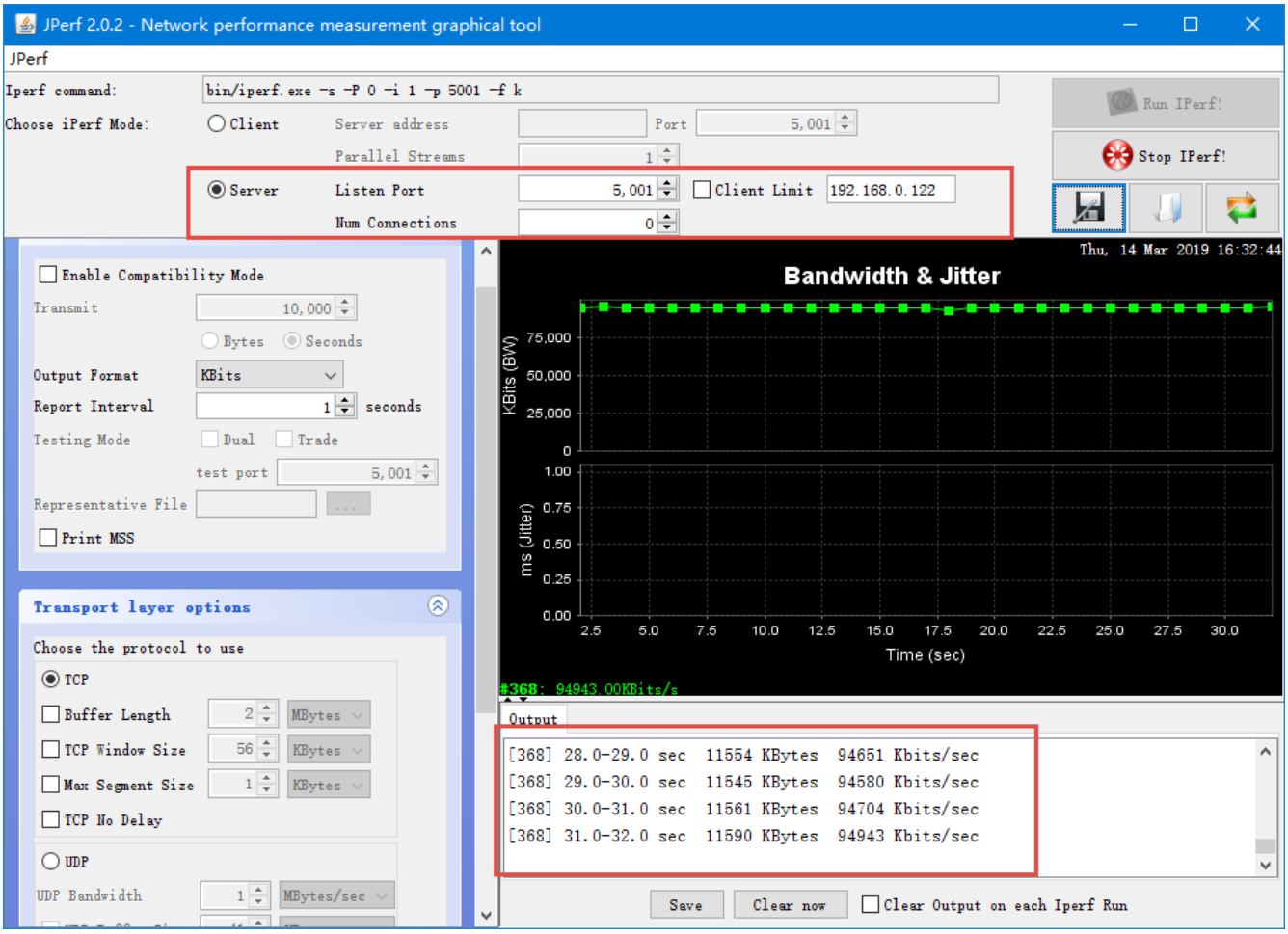


图 18-4NETCONN API 发送速度

18.2.5 测试开发板发送速度（Socket API）

本实验基于是一个实验，我们将测试开发板发送速度（NETCONN API）的工程拿过来，将 iperf_client.c 的内容替换代码清单 18-6 所示的代码即可。

代码清单 18-6iperf_client.c 文件内容

```
#include "iperf_client.h"

#include "lwip/opt.h"
```

（下页继续）

(续上页)

```
#include "lwip/sys.h"
#include "lwip/api.h"
#include <lwip/sockets.h>

#define PORT                5001
#define IP_ADDR         "192.168.0.181"

#define IPERF_BUFSZ          (4 * 1024)

static void iperf_client(void *thread_param)
{
    int sock = -1,i;
    struct sockaddr_in client_addr;
    uint8_t* send_buf;
    u32_t tick1, tick2;
    uint64_t sentlen;

    send_buf = (uint8_t *) pvPortMalloc(IPERF_BUFSZ);
    if (!send_buf)
        return ;

    for (i = 0; i < IPERF_BUFSZ; i ++)
        send_buf[i] = i & 0xff;

    while (1)
    {
        sock = socket(AF_INET, SOCK_STREAM, 0);
        if (sock < 0)
        {
            printf("Socket error\n");
            vTaskDelay(10);
```

(下页继续)

(续上页)

```
            continue;
        }

        client_addr.sin_family = AF_INET;
        client_addr.sin_port = htons(PORT);
        client_addr.sin_addr.s_addr = inet_addr(IP_ADDR);
        memset(&(client_addr.sin_zero), 0, sizeof(client_addr.sin_zero));

        if (connect(sock,
                    (struct sockaddr *)&client_addr,
                    sizeof(struct sockaddr)) == -1)
        {
            printf("Connect failed!\n");
            closesocket(sock);
            vTaskDelay(10);
            continue;
        }

        printf("Connect to iperf server successful!\n");
        tick1 = sys_now();
        while (1)
        {
            tick2 = sys_now();
            if (tick2 - tick1 >= configTICK_RATE_HZ * 5)
            {
                float f;
                f = (float)(sentlen*configTICK_RATE_HZ/125/(tick2 -␣
↪tick1));
                f /= 1000.0f;
                printf("send speed = %.4f Mbps!\n", f);

                tick1 = tick2;
```

(下页继续)

(续上页)

```
                sentlen = 0;
            }

            if (write(sock,send_buf,IPERF_BUFSZ) < 0)
                break;
            else
            {
                sentlen += IPERF_BUFSZ;
            }
        }
        closesocket(sock);
    }

}

void
iperf_client_init(void)
{
    sys_thread_new("iperf_client", iperf_client, NULL, 2048, 8);
}
```

测试结果具体见图 *18_5*，从两个实验的对比可以看出，基于 Socket API 的发送速度基本相差无几。

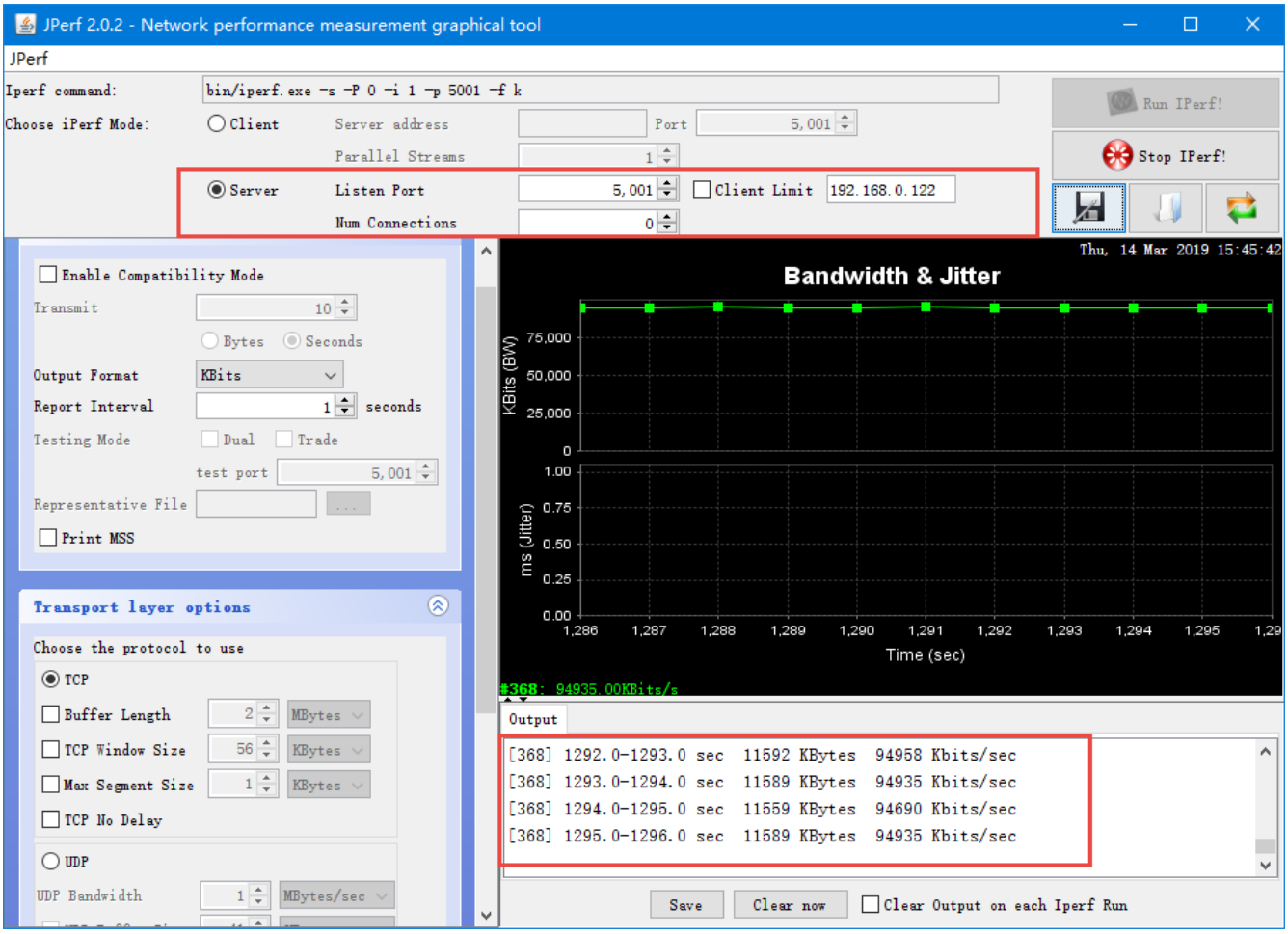


图 18-5 Socket API 发送速度

18.3 提高 LwIP 网络传输的速度

如果按照 LwIP 默认的配置，是远不可能达到我们实验所显示的速度的，因为还没优化，那肯定也是不稳定的，下面我们来看看优化的参数，首先，网速必然受限于硬件，只有硬件是很好的，那么软件才能优化的更好，网卡肯定要选择好一点的网卡，然后在工程中的 stm32f4xx_hal_config.h 文件中配置以太网发送和接收的缓冲区大小，默认是 4，我们可以稍微改大一点，具体见*代码清单 18_7*。

代码清单 18-7stm32f4xx_hal_config.h 文件配置参数

```
#define ETH_RXBUFNB    ((uint32_t)8U)        /* 接收缓冲区 */
#define ETH_TXBUFNB    ((uint32_t)8U)        /* 发送缓冲区 */
```

此外，还需在 lwipopts.h 文件中配置 LwIP 的参数，具体见代码清单 18_8，首先，我们对 LwIP 管理的内存肯定要分配的大一些，而对于发送数据是存储在 ROM 或者静态存储区的时候，还要将 MEMP_NUM_PBUF 宏定义改的大一点，当然发送缓冲区大小和发送缓冲区队列长度决定了发送速度的大小，根据不同需求进行配置，并且需要不断调试，而对于接收数据的配置，应该配置 TCP 缓冲队列中的报文段数量与 TCP 接收窗口大小，特别是接收窗口的大小，这直接可以影响数据的接收速度。

代码清单 18-8lwipopts.h 文件配置参数

```
//内存堆 heap 大小
#define MEM_SIZE                  (25*1024)

/* memp 结构的 pbuf 数量，如果应用从 ROM 或者静态存储区发送大量数据时
这个值应该设置大一点 */
#define MEMP_NUM_PBUF             25

/* 最多同时在 TCP 缓冲队列中的报文段数量 */
#define MEMP_NUM_TCP_SEG          150

/* 内存池大小 */
#define PBUF_POOL_SIZE            65

/* 每个 pbuf 内存池大小 */
#define PBUF_POOL_BUFSIZE         \

LWIP_MEM_ALIGN_SIZE(TCP_MSS+40+PBUF_LINK_ENCAPSULATION_HLEN+PBUF_LINK_HLEN)

/* 最大 TCP 报文段，TCP_MSS = (MTU - IP 报头大小 - TCP 报头大小 */
#define TCP_MSS                   (1500 - 40)
```

(下页继续)

(续上页)

```
/* TCP 发送缓冲区大小（字节） */
#define TCP_SND_BUF                 (11*TCP_MSS)

/*  TCP 发送缓冲区队列的最大长度 */
#define TCP_SND_QUEUELEN            (8* TCP_SND_BUF/TCP_MSS)

/* TCP 接收窗口大小 */
#define TCP_WND                     (11*TCP_MSS)
```

当然，除此之外，想要整个 LwIP 能高速平稳运行，只配置这些是不够的，比如我们应该使用中断的方式接收数据，这就省去了 CPU 查询数据，而且，我们应该将内核邮箱的容量增大，这样子在接收到数据之后，投递给内核就不会因为无法投递而阻塞，同时内核线程的优先级应该设置得更高一点，这样子就能及时去处理这些数据，当然，我们也可以独立使用一个新的发送线程，这样子内核就无需调用底层网卡函数，它可以专心处理数据，发送数据的事情就交由发送线程去处理，同时，在处理数据的时候，不使用串口打印信息，因为串口是一个很慢的外设，当然啦，关于提高 LwIP 网络传输的速度，还有很多东西要优化的，这也跟使用环境有关系，不能一概而论，只是给出一些方向，具体怎么实现，还需要大家亲身实践去调试。

第 19 章　HTTP 协议

19.1 什么是 HTTP 协议?

日常我们使用网络用得最多的无疑是在 Web 浏览器（下文统一使用浏览器）上查找资料、看视频、看书、看新闻等等，而在浏览器中只需要输入一些搜索就可以得到想要的信息，这归根于搜索引擎的好处，但是实际上每个网页其实由多个资源组成。我们的浏览器就是一个 HTTP 客户端，通过 HTTP 协议访问服务器，得到服务器中的 HTML 页面、文本文件、图片、音频等资源，并且将这些资源搬运到浏览器（客户端）显示给我们。

HTTP 协议是 Hyper Text Transfer Protocol（超文本传输协议）的缩写，是用于从万维网（WWW:World Wide Web ）服务器传输超文本到本地浏览器的传输协议，它是基于 TCP/IP 协议通信的，因此它也是基于 < 客户端-服务器 > 模型运作的，是一个应用层协议，可以用它来传输服务器的各种资源，如文本、图片、音频等。

HTTP 协议的特点：

1. 简单：当客户端向服务器请求服务时，只需传送请求方法和路径即可获取服务器的资源，请求方法常用的有 GET、HEAD、POST 等，每种方法规定了客户端与服务器通信的类型不同。
2. 快捷：由于 HTTP 协议简单，使得 HTTP 服务器的程序规模小，因而通信速度很快。
3. 灵活：HTTP 允许传输任意类型的数据对象，传输的类型由 Content-Type 加以标记。
4. 无连接：无连接的含义是限制每次连接只处理一个请求，服务器处理完客户的请求，并收到客户的应答后，即断开连接，简单来说就是每进行一次 HTTP 通信，都要断开一次 TCP 连接，可随着 HTTP 的普及，文档中包含大量图片的情况多了起来，每次请求完都要断开 TCP 连接，无疑增加通信量的开销，为了解决 TCP 的连接问题，HTTP1.1 提出了持久连接的方法，即任意一端只要没有明确提出断开连接，则保持 TCP 连接状态，这样子就就减少了 TCP 连接的重复建立和断开所造成的额外开销，减轻了服务端的负载。注意，在 HTTP1.1 版本之后才出现持久连接的方法。
5. 无状态：HTTP 协议是无状态协议，无状态是指协议对于事务处理没有记忆能力，即 HTTP

协议无法根据之前的状态进行本次的请求处理，这就意味着如果后续处理需要前面的信息，它必须重传数据，这样的情况可能导致 HTTP 协议传输的数据量增大，当然，凡事都有两面性，在另一方面，在服务器不需要先前信息时它的应答就较快，可以减少服务器的资源消耗，其实这种无状态对于用户来说也是不友好的，因此为了解决无状态的问题，引入了 Cookie 技术，这是一种可以让服务器知道用户上一次做了什么操作，并且记录下来，它是存储在客户端之中的，比如我们在淘宝上买东西，我们选择了几个商品，但是到了结账会跳转到另一个页面，此时如果服务器不知道我们选择了哪些商品，那怎么能结账成功呢？所以 Cookie 就是用来绕开 HTTP 的无状态性的“手段”之一，服务器可以设置或读取 Cookies 中包含信息，让服务器知道我们选择了什么商品，借此维护用户跟服务器会话中的状态，当然，Cookie 会被加密存储在客户端中，直到过期或者手动清除。

19.2 URL 与资源

我们可以把整个英特网看做是一个巨大的图书馆，里面的资源应有尽有，并且是对我们是开放的，我们想要找一本书，那么我们就需要直到他存放在哪里，然后去找到它。

网络中的资源也是应有尽有，那么怎么样才能在网络的海洋中找到我们想要的资源呢？因此 URI（Uniform Resource Identifiers）就被设计出来，用于统一管理资源，就像我们去图书馆找书一样，我们必须通过图书馆的系统，找到书所在的位置，而不是让我们自己一本一本书去找。

URL 全称是 Uniform Resource Locator，中文叫统一资源定位符，是互联网上用来标识某一处资源的绝对地址，使用它我们就必然能找到资源，除非资源已经被转移了。URI 是一个通用的概念，由两个子集组成，分别是 URL 和 URN，URL 是通过资源的位置来标识资源，而 URN 更高级一点，只需通过资源名字即可识别资源，与他们所处的位置是无关的，目前暂时还未推广 URN。

大部分 URL 都会遵循 URL 的语法，一个 URL 的组成有多个不同的组件，一个 URL 的通用格式如下：

```
<scheme>://<user>:<password>@<host>:<port>/<path>;<params>?<query>#<frag>
```

当然，绝大部分的 URL 是不会包含所有组件的内容的，关于 URL 组件具体见表格 23-1。

表格 19-1URL 组件

组件	描述
方案 scheme	指定访问服务器获取资源时使用哪种协议，有 HTTP、HTTPS、FT P、SMTP 等协议。
用户 user	某些方案访问资源时候需要指定用户名，才有权限获取资源。
密码 pass-word	用户名后面可能需要密码进行验证，用户名与密码直接使用“:”冒号分隔连接。
主机 host	资源宿主服务器的主机名或者 IP 地址（点分十进制）。
端口 port	资源宿主服务器正在监听的端口号，很多方案都有默认的端口号，而无需我们自己填写，比如 HTTP 默认使用 80 端口，HTTPS 默认使用 443 端口。端口不是一个 URL 必须的部分，如果省略端口部分，将采用默认端口。
路径 path	服务器本地资源的路径，类似于电脑中的文件路径一样，使用“/”将路径与端口隔离。从域名后的第一个“/”开始到最后一个“/”为止，虚拟目录也不是一个 URL 必须的部分，在路径之后是需要一个文件名，这就是 U RL 指定的资源。文件名部分也不是一个 URL 必须的部分，如果省略该部分，则使用默认的文件名
参数 params	某些方案会使用这个组件来输入参数，可以拥有多个参数，使用“;”符号与路径分隔开。
查询 query	某些方案会使用这个组件传递参数以激活应用程序，查询组件的内容没有通用的格式，用“?”字符与其他组件分隔开
片段 frag	一小片或者一部分资源的名字，引用对象时，不会将片段组件内容传输给服务器，这个字段是在客户端内部使用的，通过“#”字符与其他组件分隔开。

比如像我们论坛的网址：http://www.firebbs.cn/forum.php，它就是一个 URL，这是最简单的一个 URL 格式，通过它我们能访问论坛的首页资源，其中 http 就是方案，指定 HTTP 协议进行连接，域名就是www.firebbs.cn，HTTP协议默认端口是 80，在这里就无需指定端口号，路径就是 forum.php，其实这个是一个脚本文件，当客户端连接到服务器后，服务器会自动运行这个脚本文件，这样子就访问到我们论坛上的资源了。

19.3 HTTP 报文

HTTP 报文是由 3 个部分组成，分别是：对报文进行描述的“起始行”，包含属性的“首部”，以及可选的“数据主体”，对于请求报文与应答报文，只有“起始行”的格式是不一样的，具体见*代码清单 19_1* 与*代码清单 19_2*。

代码清单 19-1HTTP 请求报文

```
<method> <request-URL> <version>          //起始行
<headers>                                 //首部

<entity-body>                                       //数据主体
```

代码清单 19-2HTTP 应答报文

```
<version> <status> <reason-phrase>        //起始行
<headers>                                 //首部

<entity-body>                                     //数据主体
```

起始行和首部就是由行分隔的 ASCII 文本组成，每行都以由两个字符组成的行终止序列作为结束，其中包括一个回车符（ASCII 码 13）和一个换行符（ASCII 码 10），这个行终止序列可以写做 CRLF。

下面就对这两种 HTTP 报文的各个部分简单描述一下：

- 方法（method）：HTTP 请求报文的起始行以方法作为开始，方法用来告知服务器要做些什么，常见的方法有 GET、POST、HEAD 等，比如“GET /forum.php HTTP/1.1”使用的就是

GET 方法。

- 请求 URL（request-URL）：指定了所请求的资源。
- 版本（version）：指定报文所使用的 HTTP 协议版本，其中 <major> 指定了主要版本号，<minor> 指定了次要版本号，它们都是整数，其格式如下：

```
1 HTTP/<major>.<minor>
```

- 状态码（status）：这是在 HTTP 应答报文中使用的，状态码是在每条响应报文的起始行中返回的一个数字码，描述了请求过程中所发送的情况，比如成功、失败等，不同的状态码有不同的含义，具体见表格 19-2。

表格 19-2 状态码说明

整体范围	已定义使用范围	描述
100 ~ 199	100 ~ 101	信息提示
200~299	200 ~ 206	成功
300 ~ 399	300 ~ 305	重定向
400 ~ 499	400 ~ 415	客户端错误
500 ~ 599	500 ~ 505	服务器错误

- 原因短语（reason-phrase）：这其实是给我们看的原因短语，因为数字是不够直观，它只是状态码的一个文本形式表达而已。
- 首部（header）：HTTP 报文可以有 0 个、1 个或者多个首部，HTTP 首部字段向请求和响应报文中添加了一些附加信息，从本质上来说，它们是一个 < 名字：值 > 对，每个首部都包含一个名字，紧跟着一个冒号“:”，然后是一个可选的空格，接着是一个值，最后以 CRLF 结束，比如“Host: www.firebbs.cn”就是一个首部。
- 数据主体（entity-body）：这部分包含一个由任意数据组成的数据块，其实这与我们前面所讲的报文数据区域是一样的，用于携带数据，HTTP 报文可以承载很多类型的数字数据：图片、视频、音频、HTML 文档、软件应用程序等。

19.4 使用 Postman 获取论坛数据

既然了解了 HTTP 协议与 HTTP 报文的相关知识，我们就来使用 Postman 软件了解一下 HTTP 协议的传输过程，首先去我们的论坛上下载 Postman 软件：http://www.firebbs.cn/forum.php?mod=viewthread&tid=26274&fromuid=37393，然后安装后打开软件，就可以使用软件进行测试 HTTP 协议，可以很直观看到发送的 HTTP 报文是什么，也能很直观看到响应的数据。

首先我们测试一下获取我们论坛的资源信息，在 URL 中输入“http://www.firebbs.cn”，方法选择“GET”方法，然后点击 Send，就可以看到我们的论坛就返回了大量的数据，查看状态码、时间还有大小就立马知道这次 HTTP 请求的相关信息，而且在响应内容中，就是一个 HTML 文件内容，如果将这个内容保存下来，并且命名为 xxxx.HTML 文件的话，就可以直接在浏览器中显示我们的论坛首页了，因为这次的请求就是获取论坛首页的数据，参数一栏中并无内容，然后我们点击源码 Code，可以看到请求的 HTTP 报文内容，具体见图 *19_1*，整个 Postman 界面说明具体见图 *19_2*。

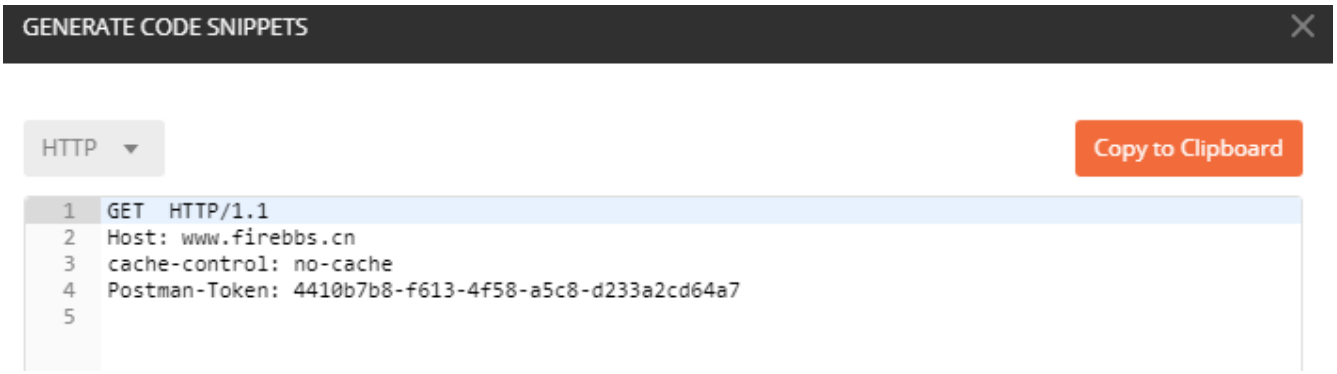


图 19-1HTTP 请求报文

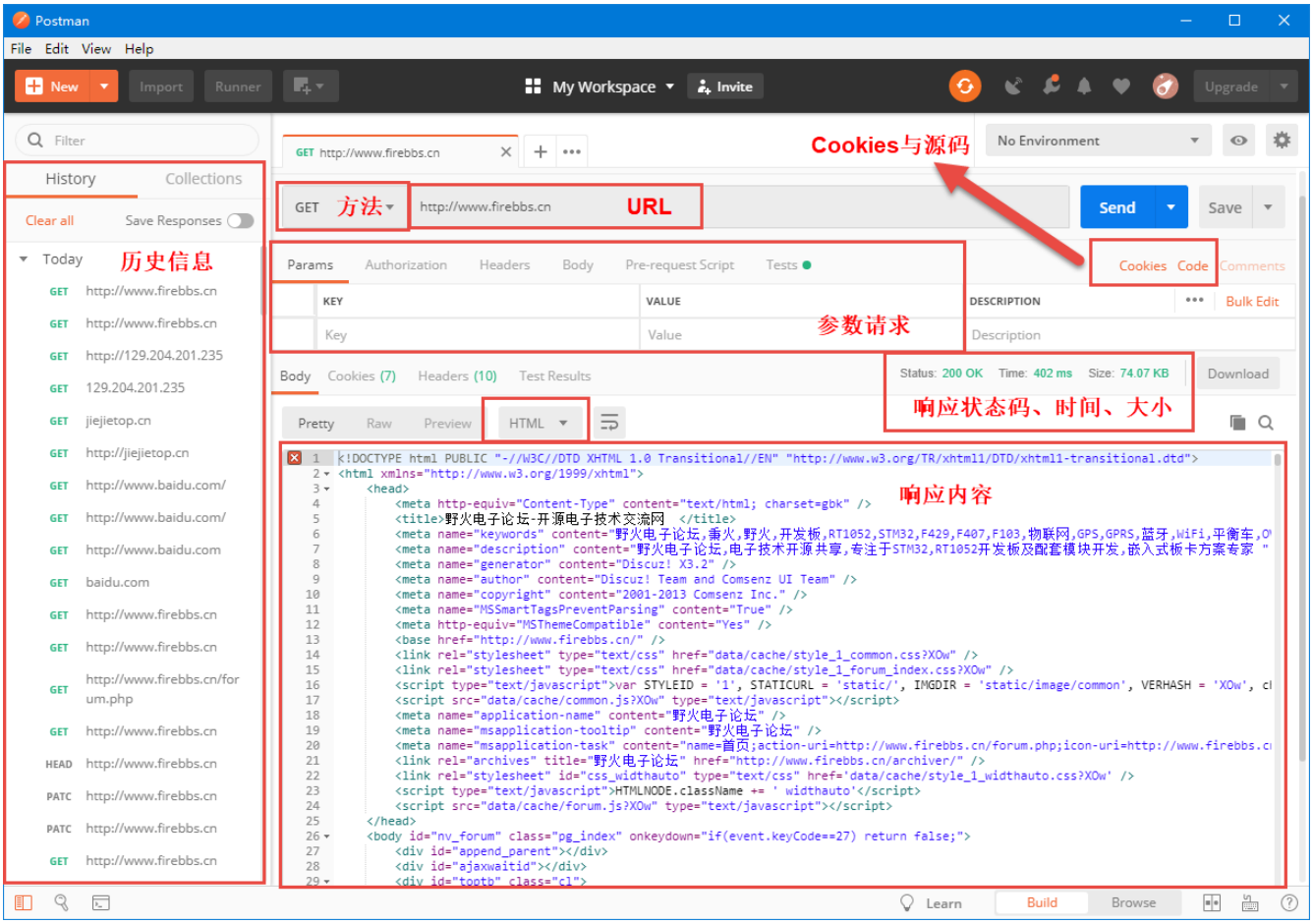


图 19-2Postman 界面

我们再访问 URL：http://www.firebbs.cn/forum.php?mod=viewthread&tid=26231#lastpost，从这个 URL 中很明显可以可看到路径与参数等，那么这个请求又是怎么样的呢，具体见图 *19_3*，很明显这个 Postman 软件也给我们写入了对应的参数，而不用我们自己设置，我们再点开 Code 可以看到 HTTP 请求包的源码，具体见图 *19_4*。除此之外，我们还可以点击 Preview 预览返回的数据，其实就是跟我们直接打开网页一样的，不过会产生中文乱码。

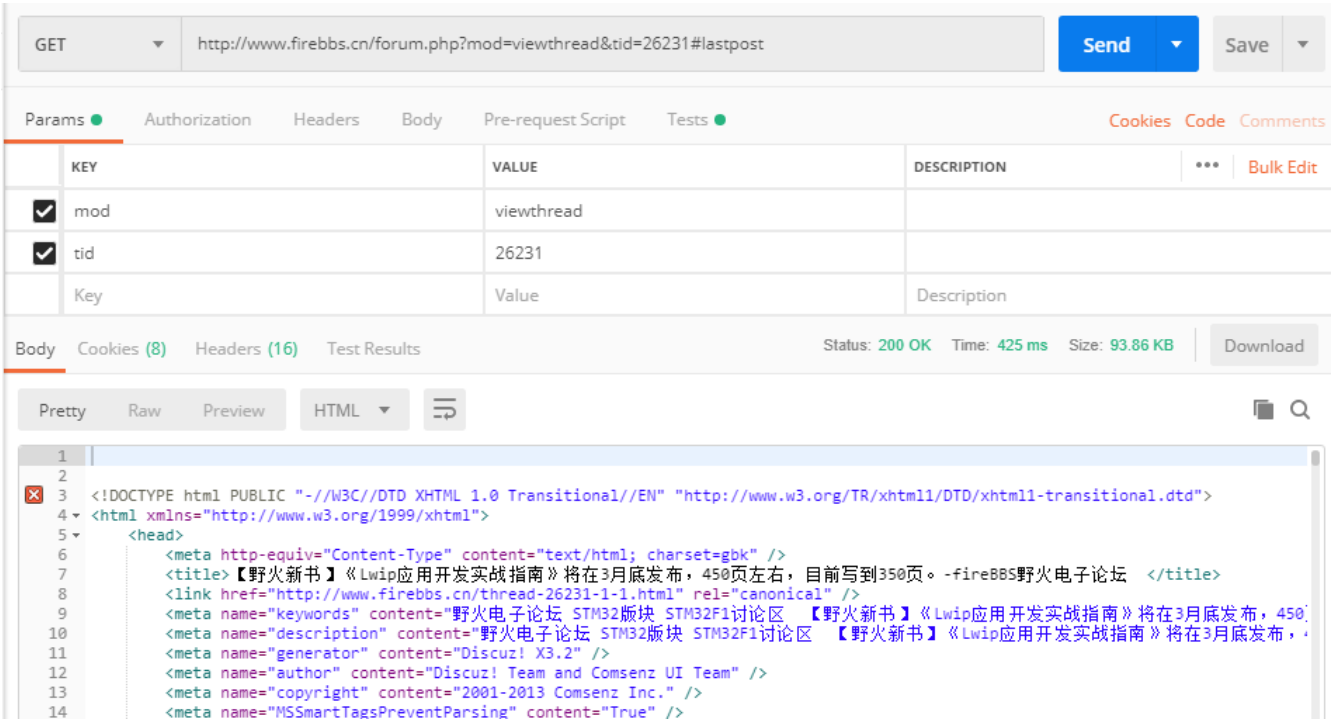


图 19-3 带参数的 URL

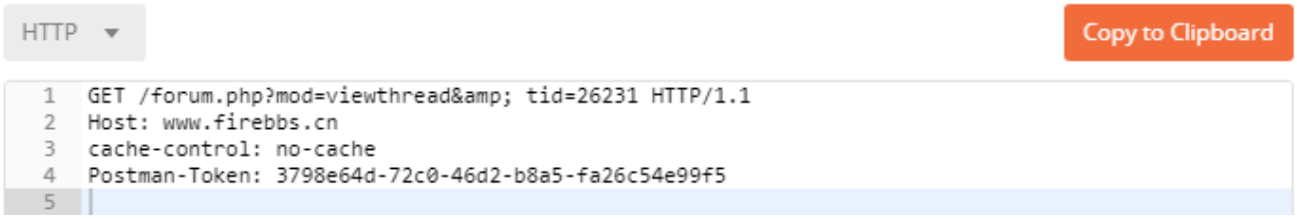


图 19-4 带参数的 HTTP 请求

我们还可以继续测试 POST 方法与其他方法，在这里我们就不再讲解太多。

19.5 使用开发板获取论坛数据

起始很多初学者，并不懂单片机是怎么访问外网的，可能就是没有一个概念，如果学到这一章的时候还不懂的话，建议回头再看看第 11.2 章的内容，然后在回来这里学习使用开发板获取外网的数据。

首先明确一下整个 HTTP 协议的流程：

1. 创建一个连接结构。
2. 连接到域名的服务器中（一般建议打开 DNS），端口号为方案对应的默认端口号。
3. 发送 HTTP 请求报文。
4. 等待服务器端的应答，并且接收数据。

此处有一个需要注意的地方，那就是网络中的数据量是非常大的，这就需要在接收数据的时候使用足够大的缓冲区，当然也可以只接收一部分数据，或者通过 FIFO 进行接收，我们本次实验是使用外部 SDRAM 进行存储数据，这样子就可以接收很大的数据量，因为 SDRAM 有 8M 的空间。

首先我们拿到一个移植好的工程，然后创建一个 client.c 文件，加入*代码清单 19_3* 所示代码，然后在 main.c 文件中调用 client_init() 函数即可。在这里要注意的是 HTTP 请求报文的处理，需要按照 HTTP 报文格式编写，如果在获取数据的时候出现错误或者乱码，那很可能就是 HTTP 请求报文编写错误。

代码清单 19-3client.c 文件内容

```
#include "client.h"

#include "lwip/opt.h"

#include "lwip/sys.h"
#include "lwip/api.h"


#include <lwip/sockets.h>
#include "./sdram/bsp_sdram.h"

#define PORT             80
#define IP_ADDR         "114.215.151.106"

#ifdef    LWIP_DNS
```

(下页继续)

(续上页)

```
#define   HOST_NAME       "www.firebbs.cn"     //
#else
#define   HOST_NAME       "114.215.151.106"     //
#endif

uint32_t *pSDRAM= (uint32_t*)SDRAM_BANK_ADDR;

uint8_t get_buf[]= "GET /forum.php HTTP/1.1\r\n \
            Host: www.firebbs.cn \r\n\r\n\r\n\r\n";

static void client(void *thread_param)
{
    int sock = -1,rece;
    struct sockaddr_in client_addr;

    uint8_t *pbufdata = pvPortMalloc(5000);

    char* host_ip;

#ifdef  LWIP_DNS
    ip4_addr_t dns_ip;
    netconn_gethostbyname(HOST_NAME, &dns_ip);
    host_ip = ip_ntoa(&dns_ip);
    PRINT_DEBUG("host name : %s , host_ip : %s\n",HOST_NAME,host_ip);
#else
    host_ip = HOST_NAME;
#endif

    while (1)
    {
        sock = socket(AF_INET, SOCK_STREAM, 0);
        if (sock < 0)
```

(下页继续)

(续上页)

```
        {
            printf("Socket error\n");
            vTaskDelay(10);
            continue;
        }

        client_addr.sin_family = AF_INET;
        client_addr.sin_port = htons(PORT);
        client_addr.sin_addr.s_addr = inet_addr(host_ip);
        memset(&(client_addr.sin_zero), 0, sizeof(client_addr.sin_zero));

        if (connect(sock,
                    (struct sockaddr *)&client_addr,
                    sizeof(struct sockaddr)) == -1)
        {
            printf("Connect failed!\n");
            closesocket(sock);
            vTaskDelay(10);
            continue;
        }

        printf("Connect to server successful!\n");
        memset(pSDRAM,0,400000);
        printf("\n*****************************************************\n");
        write(sock,get_buf,sizeof(get_buf));
        while (1)
        {
//      if(write(sock,get_buf,sizeof(get_buf)) < 0)
//        break;

            rece = recv(sock, (uint8_t*)pSDRAM, 400000, 0);
            if (rece <= 0)
```

(下页继续)

(续上页)

```
                break;

//      printf("rece = %d\n",rece);
            printf("%s\n",(uint8_t*)pSDRAM);
//      pSDRAM = (uint32_t*)(pSDRAM + rece);

        }
//    printf("%s\n",(uint8_t*)pSDRAM);
        printf("\n**************************************************\n");

        memset(pSDRAM,0,400000);
        closesocket(sock);
        vTaskDelay(10000);
    }

}

void
client_init(void)
{
    sys_thread_new("client", client, NULL, 2048, 4);
}
```

通过这个实验，我们就能获取到论坛的数据，因为论坛的数据量太大，因此获取的数据并无实际意义，仅做 HTTP 协议的使用方法而已，当然，我们可以通过 HTTP 协议获取其他类型的数据，这就需要自己去处理了，就不过多赘述，实验现象具体见图 *19_5*。

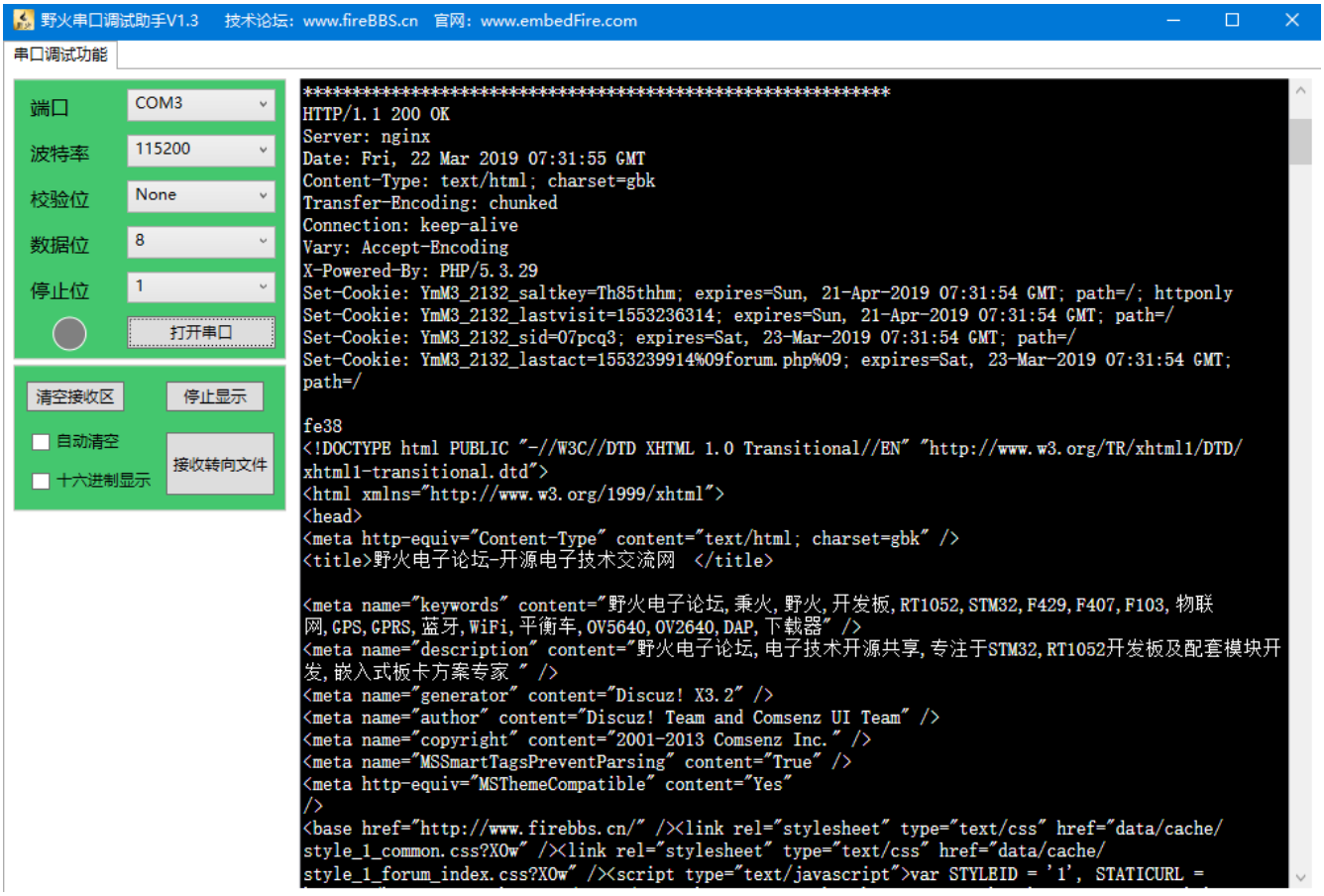

野火串口调试助手V1.3
技术论坛：www.fireBBS.cn
官网：www.embedFire.com
串口调试功能
端口
COM3
波特率
115200
校验位
None
数据位
8
停止位
1
打开串口
清空接收区
停止显示
自动清空
十六进制显示
接收转向文件
HTTP/1.1 200 OK
Server: nginx
Date: Fri, 22 Mar 2019 07:31:55 GMT
Content-Type: text/html; charset=gbk
Transfer-Encoding: chunked
Connection: keep-alive
Vary: Accept-Encoding
X-Powered-By: PHP/5.3.29
Set-Cookie: YmM3_2132_saltkey=Th85thhm; expires=Sun, 21-Apr-2019 07:31:54 GMT; path=/; httponly
Set-Cookie: YmM3_2132_lastvisit=1553236314; expires=Sun, 21-Apr-2019 07:31:54 GMT; path=/
Set-Cookie: YmM3_2132_sid=07pcq3; expires=Sat, 23-Mar-2019 07:31:54 GMT; path=/
Set-Cookie: YmM3_2132_lastact=1553239914%09forum.php%09; expires=Sat, 23-Mar-2019 07:31:54 GMT;
path=/
fe38
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/
xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=gbk" />
<title>野火电子论坛-开源电子技术交流网 </title>
<meta name="keywords" content="野火电子论坛,秉火,野火,开发板,RT1052,STM32,F429,F407,F103,物联
网,GPS,GPRS,蓝牙,WiFi,平衡车,OV5640,OV2640,DAP,下载器" />
<meta name="description" content="野火电子论坛,电子技术开源共享,专注于STM32,RT1052开发板及配套模块开
发,嵌入式板卡方案专家 " />
<meta name="generator" content="Discuz! X3.2" />
<meta name="author" content="Discuz! Team and Comsenz UI Team" />
<meta name="copyright" content="2001-2013 Comsenz Inc." />
<meta name="MSSmartTagsPreventParsing" content="True" />
<meta http-equiv="MSThemeCompatible" content="Yes"
/>
<base href="http://www.firebbs.cn/" /><link rel="stylesheet" type="text/css" href="data/cache/
style_1_common.css?XOw" /><link rel="stylesheet" type="text/css" href="data/cache/
style_1_forum_index.css?XOw" /><script type="text/javascript">var STYLEID = '1', STATICURL =

图 19-5 实验现象

第 20 章　HTTP 服务器

20.1 Hello World 网页 demo

本次实验我们利用开发板来搭建一个 HTTP 服务器，通过浏览器去访问我们的开发板，我们使用 NETCONN API 去实现，具体步骤如下：

1. 创建一个 NETCONN_TCP 连接结构
2. 将本地 IP 地址、端口号与连接结构进行绑定。
3. 监听客户端的连接。
4. 处理该连接。
5. 发送接收数据。

在这里有几个注意事项，因为浏览器访问我们的服务器，其实是通过“GET”操作命令获取我们开发板的资源，那么我们在接收到这个命令后，返回我们的资源数据给浏览器，而浏览器显示是通过 HTML 代码来标记这些资源的，那么我们在开发板中就需要定义好相关的 HTML 数据，此外，HTTP 服务器默认是打开 80 端口的。

首先我们拿到移植好的带操作系统工程，创建两个文件，分别为 httpserver_netconn.c 和 httpserver_netconn.h，在其中加入代码清单 *20_1* 与代码清单 *20_2* 所示代码。

代码清单 20-1httpserver_netconn.c 文件内容

```
#include "lwip/opt.h"
#include "lwip/arch.h"
#include "lwip/api.h"

#include "httpserver-netconn.h"
```

(下页继续)

(续上页)

```
#if LWIP_NETCONN

#ifndef HTTPD_DEBUG
#define HTTPD_DEBUG         LWIP_DBG_OFF
#endif

static const char http_html_hdr[] =
    "HTTP/1.1 200 OK\r\nContent-type: text/html\r\n\r\n";
static const char http_index_html[] =
"<html><head><title>Congrats!</title></head>\
<body><h1 align=\"center\">Hello World!</h1>\
<h2 align=\"center\">Welcome to Fire lwIP HTTP Server!</h1>\
<p align=\"center\">This is a small test page, served by httpserver-
↪netconn.</p>\
<p align=\"center\"><a href=\"http://www.firebbs.cn/forum.php/\">\
<font size=\"6\"> 野火电子论坛 </font> </a></p>\
<a href=\"http://www.firebbs.cn/forum.php/\">\
<p align=\"center\"><img src=\"http://www.firebbs.cn/data/attachment/
↪portal/\
201806/05/163015rhz7mbgbt0zfujzh.jpg\" /></a>\
</body></html>";

/** Serve one HTTP connection accepted in the http thread */
static void
http_server_netconn_serve(struct netconn *conn)
{
    struct netbuf *inbuf;
    char *buf;
    u16_t buflen;
    err_t err;

    /* 读取数据 */
```

(下页继续)

(续上页)

```
    err = netconn_recv(conn, &inbuf);

    if (err == ERR_OK)
    {
        netbuf_data(inbuf, (void**)&buf, &buflen);

        /* 判断是不是 HTTP 的 GET 命令 */
        if (buflen>=5 &&
                buf[0]=='G' &&
                buf[1]=='E' &&
                buf[2]=='T' &&
                buf[3]==' ' &&
                buf[4]=='/' )
        {

            /* 发送数据头 */
            netconn_write(conn, http_html_hdr,
            sizeof(http_html_hdr)-1, NETCONN_NOCOPY);
            /* 发送网页数据 */
            netconn_write(conn, http_index_html,
                sizeof(http_index_html)-1, NETCONN_NOCOPY);
        }
    }
    netconn_close(conn); /* 关闭连接 */

    /* 释放 inbuf */
    netbuf_delete(inbuf);
}

/** The main function, never returns! */
static void
http_server_netconn_thread(void *arg)
```

(下页继续)

(续上页)

```
{
    struct netconn *conn, *newconn;
    err_t err;
    LWIP_UNUSED_ARG(arg);

    /* 创建 netconn 连接结构 */
    /* 绑定端口号与 IP 地址，端口号默认是 80 */
    conn = netconn_new(NETCONN_TCP);
    netconn_bind(conn, IP_ADDR_ANY, 80);

    LWIP_ERROR("http_server: invalid conn", (conn != NULL), return;);

    /* 监听 */
    netconn_listen(conn);

    do
    {
        //处理连接请求
        err = netconn_accept(conn, &newconn);
        if (err == ERR_OK)
        {
            //发送网页数据
            http_server_netconn_serve(newconn);

            //删除连接结构
            netconn_delete(newconn);
        }
    }
    while (err == ERR_OK);
    //关闭
    netconn_close(conn);
    netconn_delete(conn);
```

(下页继续)

(续上页)

```
}

/** Initialize the HTTP server (start its thread) */
void
http_server_netconn_init(void)
{
sys_thread_new("http_server_netconn", http_server_netconn_thread, NULL,␣
↪2048, 4);
}

#endif
```

代码清单 20-2httpserver_netconn.h 文件内容

```
#ifndef LWIP_HTTPSERVER_NETCONN_H
#define LWIP_HTTPSERVER_NETCONN_H

void http_server_netconn_init(void);

#endif /* LWIP_HTTPSERVER_NETCONN_H */
```

根据自己开发板所处的环境配置好 IP 地址，然后在 main.c 文件中调用 http_server_netconn_init() 函数即可，打开电脑的浏览器，输入自己开发板的 IP 地址，即可得到相应的服务器信息，具体见图 *20_1*



图 20-1HTTP 服务器

20.2 提供网页控制 LED 灯开关的功能

这个实验是基于上个实验的再次开发，我们的开发板作为服务器，电脑浏览器访问服务器，然后通过网页上的控制按钮来控制我们开发板上的 LED 灯，那是什么原理呢？首先我们从上一节知道，浏览器是通过“GET”命令来获取开发板上的数据（资源），同样的，浏览器也可以通过“POST”命令来设置开发板上的资源，当然，我们在服务器端（开发板）需要对“POST”命令产生应答并且处理才可以，因此我们需要对服务器进行改写，而且将开发板上 LED 等的状态实时

显示在网页上。

首先将上一个实验的工程拷贝过来，将 httpserver_netconn.c 文件替换*代码清单 20_3* 所示的代码，在浏览器中输入我们的 IP 地址信息，然后通过网页上的控制 LED 开关按钮就可以进行控制开发板上的 LED 灯了，具体见图 *20_2*。

代码清单 20-3httpserver_netconn.c 文件内容

```
#include "lwip/opt.h"
#include "lwip/arch.h"
#include "lwip/api.h"

#include "httpserver-netconn.h"

#if LWIP_NETCONN

#ifndef HTTPD_DEBUG
#define HTTPD_DEBUG         LWIP_DBG_OFF
#endif


const static char http_html_hdr[] =
    "HTTP/1.1 200 OK\r\nContent-type: text/html\r\n\r\n";

const unsigned char Led1On_Data[] =
    "<HTML> \
<head><title>HTTP LED Control</title></head> \
<center> \
<p> \
<font size=\"6\">LED<font style = \"color:red\"> 已打开! </font> \
<form method=post action=\"off\" name=\"ledform\"> \
<input type=\"submit\" value=\"关闭\" style=\"width:80px;height:30px;\"></
↪form> \
</center> \
```

(下页继续)

(续上页)

```
</HTML> ";

//当 LED 灭时，向浏览器返回如下 html 信息，显示结果如下图 15-7 所示
const unsigned char Led1Off_Data[] =
    "<HTML> \
<head><title>HTTP LED Control</title></head> \
<center> \
<p> \
<font size=\"6\">LED<font style = \"color:red\"> 已关闭! </font> \
<form method=post action=\"on\" name=\"ledform\"> \
<input type=\"submit\" value=\"打开\" style=\"width:80px;height:30px;\"></
↪form> \
</center> \
</HTML> ";

static const char http_index_html[] =
    "<html><head><title>Congrats!</title></head>\
<body><h2 align=\"center\">Welcome to Fire lwIP HTTP Server!</h2>\
<p align=\"center\">This is a small test page : http control led.</p>\
<p align=\"center\"><a href=\"http://www.firebbs.cn/forum.php/\">\
<font size=\"6\"> 野火电子论坛 </font> </a></p>\
<a href=\"http://www.firebbs.cn/forum.php/\">\
<img src=\"http://www.firebbs.cn/data/attachment/portal/201806/\
05/163015rhz7mbgbt0zfujzh.jpg\"/></a>\
</body></html>";

static bool led_on = FALSE;

/* 发送网页数据 */
void httpserver_send_html(struct netconn *conn, bool led_status)
{
    //发送数据头
```

(下页继续)

(续上页)

```
    netconn_write(conn, http_html_hdr,
                sizeof(http_html_hdr)-1, NETCONN_NOCOPY);

    /* 根据 LED 状态，发送不同的 LED 数据 */
    if (led_status == TRUE)
        netconn_write(conn, Led1On_Data,
                    sizeof(Led1On_Data)-1, NETCONN_NOCOPY);
    else
        netconn_write(conn, Led1Off_Data,
                    sizeof(Led1Off_Data)-1, NETCONN_NOCOPY);

    netconn_write(conn, http_index_html,
                sizeof(http_index_html)-1, NETCONN_NOCOPY);

}

static void httpserver_serve(struct netconn *conn)
{
    struct netbuf *inbuf;
    char *buf;
    u16_t buflen;
    err_t err;

    /* 等待客户端的命令数据 */
    err = netconn_recv(conn, &inbuf);

    if (err == ERR_OK)
    {
        netbuf_data(inbuf, (void**)&buf, &buflen);
        /* “GET”命令 */
        if (buflen>=5 &&
                buf[0]=='G' &&
```

(下页继续)

(续上页)

```
                buf[1]=='E' &&
                buf[2]=='T' &&
                buf[3]==' ' &&
                buf[4]=='/' )
        {

            /* 发送数据 */
            httpserver_send_html(conn, led_on);
        }
        // “POST” 命令
        else if (buflen>=8&&buf[0]=='P'&&buf[1]=='O'
                &&buf[2]=='S'&&buf[3]=='T')
        {
            if (buf[6]=='o'&&buf[7]=='n')
            {
                //请求打开 LED
                led_on = TRUE;
                LED1_ON;
            }
            else if (buf[6]=='o'&&buf[7]=='f'&&buf[8]=='f')
            {
                //请求关闭 LED
                led_on = FALSE;
                LED1_OFF;
            }
            //发送数据
            httpserver_send_html(conn, led_on);
        }

        netbuf_delete(inbuf);
    }
    /* 关闭 */
```

(下页继续)

(续上页)

```
    netconn_close(conn);

}

/** The main function, never returns! */
static void
httpserver_thread(void *arg)
{
    struct netconn *conn, *newconn;
    err_t err;
    LWIP_UNUSED_ARG(arg);

    /* 创建连接结构 */
    conn = netconn_new(NETCONN_TCP);
    LWIP_ERROR("http_server: invalid conn", (conn != NULL), return;);

    led_on = TRUE;
    LED1_ON;

    /* 绑定 IP 地址与端口号 */
    netconn_bind(conn, NULL, 80);

    /* 监听 */
    netconn_listen(conn);

    do
    {
        err = netconn_accept(conn, &newconn);
        if (err == ERR_OK)
        {
            httpserver_serve(newconn);
            netconn_delete(newconn);
```

(下页继续)

(续上页)

```
        }
    }
    while (err == ERR_OK);

    netconn_close(conn);

    netconn_delete(conn);
}

/** Initialize the HTTP server (start its thread) */
void
httpserver_init()
{
sys_thread_new("http_server_netconn",httpserver_thread, NULL, 1024, 4);
}

#endif
```

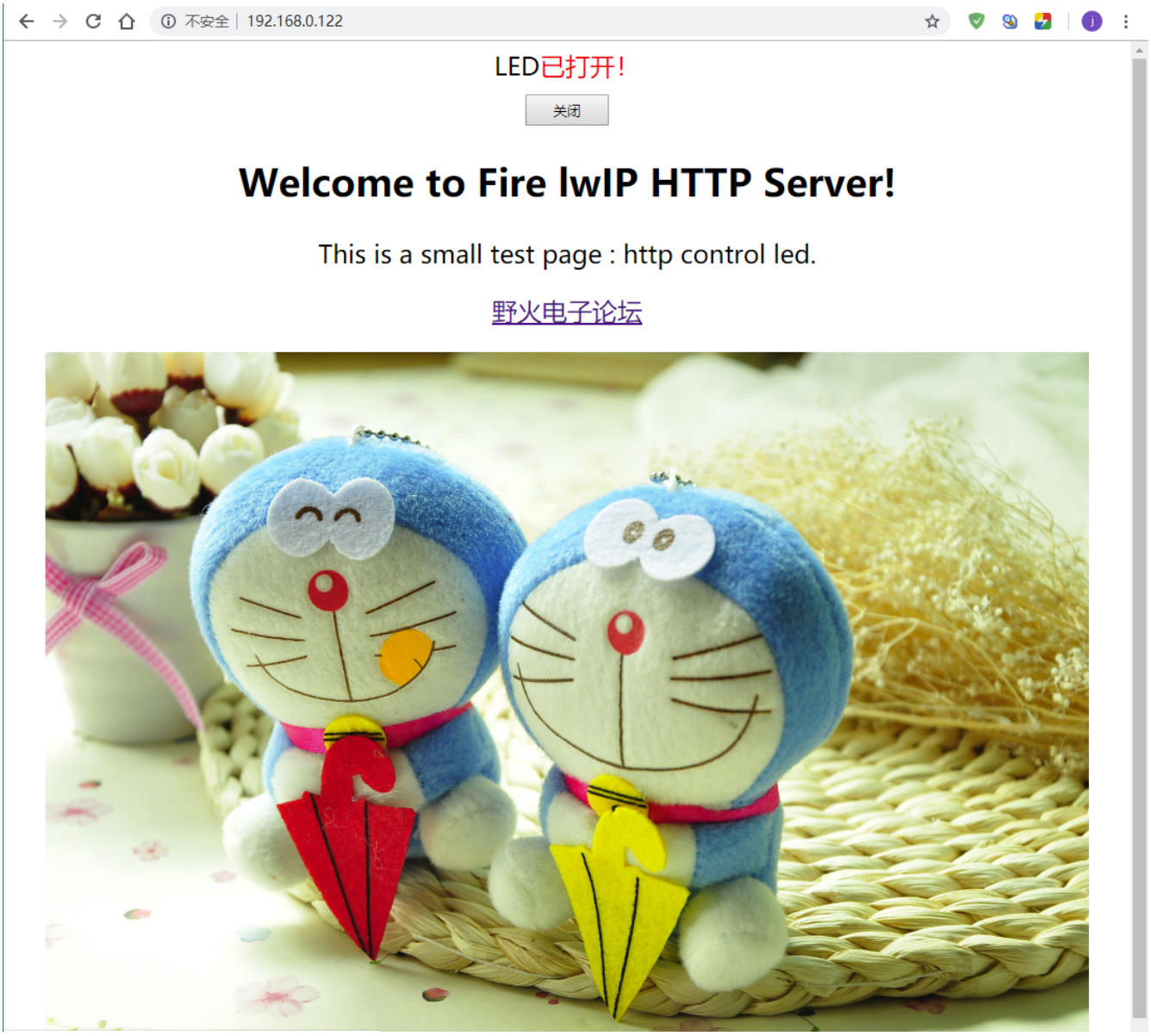


图 20-2 网页控制 LED 灯

第 21 章　MQTT 协议

21.1 MQTT 协议简介

MQTT 协议全称是 Message Queuing Telemetry Transport，翻译过来就是消息队列遥测传输协议，它是物联网常用的应用层协议，运行在 TCP/IP 中的应用层中，依赖 TCP 协议，因此它具有非常高的可靠性，同时它是基于 TCP 协议的 < 客户端-服务器 > 模型发布/订阅主题消息的轻量级协议，也是我们常说的发送与接收数据，下面我们来初步了解一下 mqtt 相关的名称与功能。

21.2 MQTT 通信模型

MQTT 协议提供一对多的消息发布，可以降低应用程序的耦合性，用户只需要编写极少量的应用代码就能完成一对多的消息发布与订阅，该协议是基于 < 客户端-服务器 > 模型，在协议中主要有三种身份：发布者（Publisher）、服务器（Broker）以及订阅者（Subscriber）。其中，MQTT 消息的发布者和订阅者都是客户端，服务器只是作为一个中转的存在，将发布者发布的消息进行转发给所有订阅该主题的订阅者；发布者可以发布在其权限之内的所有主题，并且消息发布者可以同时是订阅者，实现了生产者与消费者的脱耦，发布的消息可以同时被多个订阅者订阅。



图 21-1MQTT 通信模型

MQTT 客户端的功能：

1. 发布消息给其它相关的客户端。
2. 订阅主题请求接收相关的应用消息。
3. 取消订阅主题请求移除接收应用消息。
4. 从服务端终止连接。

MQTT 服务器常被称为 Broker（消息代理），以是一个应用程序或一台设备，它一般为云服务器，比如 BTA 三巨头的一些物联网平台就是常使用 MQTT 协议，它是位于消息发布者和订阅者之间，以便用于接收消息并发送到订阅者之中，它的功能有：

1. 接受来自客户端的网络连接请求。
2. 接受客户端发布的应用消息。
3. 处理客户端的订阅和取消订阅请求。
4. 转发应用消息给符合条件的已订阅客户端（包括发布者自身）。

21.3 消息主题与服务质量

什么是主题？MQTT 服务器为每个连接的客户端（订阅者）添加一个标签，该标签与服务器中的所有订阅相匹配，服务器会将消息转发给与标签相匹配的每个客户端（订阅者），当然订阅者也是需要有权限才能订阅对应的主题，比如像阿里云中的，订阅者只能订阅同一个产品下的主题，而不能跨产品订阅，这样子的处理就能达到信息的安全性以及多个订阅者能及时收到消息。一个主题可以有多个级别，各个级别之间用斜杠字符分隔，例如/test 和 /test/test1/test2 都是有效的主题。

发布者与订阅者可以通过主题名字，一般为 UTF-8 编码（反正用英文字符串就不会错）的形式发布和订阅主题，比如我们可以直接定义一个名字为“test”的主题，绝大多数的 MQTT 服务器支持动态发布/定阅主题，即当前服务器中没有某个主题，但是客户端直接可以向该主题发布/订阅

消息，这样子服务器就会创建对应的主题，当然，服务器中一般也会默认提供多个系统主题，所有连接的客户端均可订阅。

每个客户端与服务器建立连接后就是一个会话，客户端和服务器之间会有状态交互，订阅是基于会话之上，每个订阅中，都会包含一个主题过滤器，它是一个表达式，用于标识订阅相关的一个或多个主题，主题过滤器可以使用通配符，因此订阅者需要指定订阅的主题名字与服务质量（QoS），订阅者能订阅多个主题，也就能接收到多个发布者发布的消息。同理，发布者也需要首先与服务器建立会话，并且指定发送的主题名字与服务质量，同时它也能向多个不同的主题发送消息。

那么什么是服务质量呢？MQTT 的服务质量提供 3 个等级：

1. QoS0：最多发送一次消息，在消息发送出去后，接收者不会发送回应，发送者也不会重发消息，消息可能送达一次也可能根本没送达，这个服务质量常用在不重要的消息传递中，因为即使消息丢了也没有太大关系。
2. QoS1：最少发送一次消息（消息最少需要送达一次，也有可送达多次），QoS1 的 PUBLISH 报文的可变报头中包含一个报文标识符，需要 PUBACK 报文确认。即需要接收者返回 PUBACK 应答报文。
3. QoS2：这是最高等级的服务质量，消息丢失和重复都是不可接受的，只不过使用这个服务质量等级会有额外的开销，这个等级常用于支付中，因为支付是必须有且仅有一次成功，总不能没给钱或者给了多次钱吧。

21.4 MQTT 控制报文

21.4.1 固定报头

MQTT 协议工作在 TCP 协议之上，因为客户端和服务器都是应用层，那么必然需要一种协议在两者之间进行通信，那么随之而来的就是 MQTT 控制报文，MQTT 控制报文有 3 个部分组成，分别是固定报头（fixed header）、可变报头（variable header）、有效荷载（数据区域 payload）。固定报头，所有的 MQTT 控制报文都包含，可变报头与有效载荷是部分 MQTT 控制报文包含。

固定报头占据两字节的空间，具体见图 *21_2*。



图 21-2 固定报头

固定报头的第一个字节分为控制报文的类型（4bit），以及控制报文类型的标志位，控制类型共有 14 种，其中 0 与 15 被系统保留出来，其他的类型具体见表格 21-1。

表格 21-1 固定报头类型

类型	值	说明
Reserved	0	系统保留
CONNECT	1	客户端请求连接服务端
CONNACK	2	连接报文确认
PUBLISH	3	发布消息
PUBACK	4	消息发布收到确认（QoS 1）
PUBREC	5	发布收到（QoS2）
PUBREL	6	发布释放（QoS2）
PUBCOMP	7	消息发布完成（QoS2）
SUBSCRIBE	8	客户端订阅请求
SUBACK	9	订阅请求报文确认
UNSUBSCRIBE	10	客户端取消订阅请求
UNSUBACK	11	取消订阅报文确认
PINGREQ	12	心跳请求
PINGRESP	13	心跳响应
DISCONNECT	14	客户端断开连接
Reserved	15	系统保留

固定报头的 bit0-bit3 为标志位，依照报文类型有不同的含义，事实上，除了 PUBLISH 类型报文以外，其他报文的标志位均为系统保留，PUBLISH 报文的第一字节 bit3 是控制报文的重复分发标志（DUP），bit1-bit2 是服务质量等级，bit0 是 PUBLISH 报文的保留标志，用于标识 PUBLISH 是否保留，当客户端发送一个 PUBLISH 消息到服务器，如果保留标识位置 1，那么服务器应该保留这条消息，当一个新的订阅者订阅这个主题的时候，最后保留的主题消息应被发送到新订阅的用户。

固定报头的第二个字节开始是剩余长度字段，是用于记录剩余报文长度的，表示当前的消息剩余的字节数，包括可变报头和有效载荷区域（如果存在），但剩余长度不包括用于编码剩余长度字段本身的字节数。

剩余长度字段使用一个变长度编码方案，对小于 128 的值它使用单字节编码，而对于更大的数值则按下面的方式处理：每个字节的低 7 位用于编码数据长度，最高位（bit7）用于标识剩余长度字段是否有更多的字节，且按照大端模式进行编码，因此每个字节可以编码 128 个数值和一个延

续位，剩余长度字段最大可拥有 4 个字节。

当剩余长度使用 1 个字节存储时，其取值范围为 0(0x00)~127(0x7f)。

当使用 2 个字节时，其取值范围为 128(0x80,0x01)~16383(0Xff,0x7f)。

当使用 3 个字节时，其取值范围为 16384(0x80,0x80,0x01)~2097151(0xFF,0xFF,0x7F)。

当使用 4 个字节时,其取值范围为 2097152(0x80,0x80,0x80,0x01)~268435455(0xFF,0xFF,0xFF,0x7F)。

总的来说，MQTT 报文理论上可以发送最大 256M 的报文，当然，这种情况是非常少的。

21.4.2 可变报头

可变报头并不是所有的 MQTT 报文都带有的（比如 PINGREQ 心跳请求与 PINGRESP 心跳响应报文就没有可变报头），只有某些报文才拥有可变报头，它在固定报头和有效负载之间，可变报头的内容会根据报文类型的不同而有所不同，但可变报头的报文标识符（Packet Identifier）字段存在于在多个类型的报文里，而有一些报文又没有报文标识符字段，具体见表格 21-2，报文标识符结构具体见图 *21_3*。

表格 21-2 需要报文标识符字段的报文类型

报文类型	是否需要报文标识符字段
CONNECT	不需要
CONNACK	不需要
PUBLISH	需要（如果 QoS > 0）
PUBACK	需要
PUBREC	需要
PUBREL	需要
PUBCOMP	需要
SUBSCRIBE	需要
SUBACK	需要
UNSUBSCRIBE	需要
UNSUBACK	需要
PINGREQ	不需要
PINGRESP	不需要
DISCONNECT	不需要

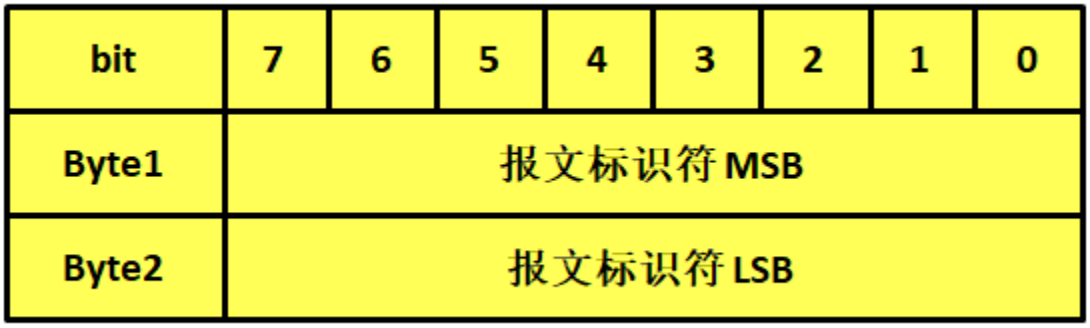


图 21-3 报文标识符

因为对于不同的报文，可变报头是不一样的，下面就简单讲解几个报文的可变报头。

21.4.2.1 CONNECT

在一个会话中，客户端只能发送一次 CONNECT 报文，它是客户端用于请求连接服务器的报文，常称之为连接报文，如果客户端发送多次连接报文，那么服务端必须将客户端发送的第二个 CONNECT 报文当作协议违规处理并断开客户端的连接。

CONNECT 报文的可变报头包含四个字段：协议名（Protocol Name）、协议级别（Protocol Level）、连接标志（Connect Flags）以及保持连接（Keep Alive）字段。

协议名是 MQTT 的 UTF-8 编码的字符串，其中还包含用于记录协议名长度的两字节字段 MSB 与 LSB。

在协议名之后的是协议级别，MQTT 协议使用 8 位的无符号值表示协议的修订版本，对于 MQTT3.1 版的协议，协议级别字段的值是 3(0x03)，而对于 MQTT3.1.1 版的协议，协议级别字段的值是 4(0x04)。如果服务器发现连接报文中的协议级别字段是不支持的协议级别，服务端必须给发送一个返回码为 0x01（不支持的协议级别）的 CONNACK 响应连接报文，然后终止客户端的连接请求。

连接标志字段涉及的内容比较多，它在协议级别之后使用一个字节表示，但分成很多个标志位，具体见图 *21_4*。

bit	7	6	5	4	3	2	1	0
Byte	User Name Flag	Password Flag	Will Retain	Will QoS		Will Flag	Clean Session	reserved

图 21-4 连接标志字段

bit0 是 MQTT 保留的标志位，在连接过程中，服务器会检测连接标志的 bit0 是否为 0，如果不为 0 则服务器任务这个连接报文是不合法的，会终止连接请求。

bit1 是清除会话标志 Clean Session，一般来说，客户端在请求连接服务器时总是将清除会话标志设置为 0 或 1，在建立会话连接后，这个值就固定了，当然这个值的选择取决于具体的应用，如果清除会话标志设置为 1，那么客户端不会收到旧的应用消息，而且在每次连接成功后都需要重新订阅相关的主题。清除会话标志设置为 0 的客户端在重新连接后会收到所有在它连接断开期

间（其他发布者）发布的 QoS1 和 QoS2 级别的消息。因此，要确保不丢失连接断开期间的消息，需要使用 QoS1 或 QoS2 级别，同时将清除会话标志设置为 0。

bit2 是遗嘱标志 Will Flag，如果该位被设置为 1，表示如果客户端与服务器建立了会话，遗嘱消息（Will Message）将必须被存储在服务器中，当这个客户端断开连接的时候，遗嘱消息将被发送到订阅这个会话主题的所有订阅者，这个消息是很有用的，我们可以知道这个设备的状况，它是否已经掉线了，以备启动备用方案，当然，想要不发送遗嘱消息也是可以的，只需要让服务器端收到 DISCONNECT 报文时删除这个遗嘱消息即可。

bit3-bit4 用于指定发布遗嘱消息时使用的服务质量等级，与其他消息的服务质量是一样的，遗嘱 QoS 的值可以等于 0(0x00)，1(0x01)，2(0x02)，当然，使用遗嘱消息的前提是遗嘱标志位为 1。

bit5 表示遗嘱保留标志位，当客户端意外断开连接时，如果 Will Retain 置一，那么服务器必须将遗嘱消息当作保留消息发布，反之则无需保留。

bit6 是密码标志位 Password Flag，如果密码标志被设置为 0，有效载荷中不能包含密码字段，反之则必须包含密码字段。

bit6 是用户名标志位 User Name Flag，如果用户名标志被设置为 0，有效载荷中不能包含用户名字段，反之则必须包含用户名字段。

保持连接字段是一个以秒为单位的时间间隔，它使用了两个字节来记录允许客户端最大空闲时间间隔，简单来说就是，客户端必须在这段时间中与服务器进行通信，让服务器知道客户端还处于连接状态而不是断开了，当然，如果没有任何其它的控制报文可以发送，客户端也必须要发送一个 PINGREQ 报文，以告知服务器还是处于连接状态的。

总的来说，整个 CONNECT 报文可变报头的内容如下，具体见图 *21_5*。

bit	说明	7	6	5	4	3	2	1	0
Byte1	长度 MSB (0)	0	0	0	0	0	0	0	0
Byte2	长度 LSB (4)	0	0	0	0	0	1	0	0
Byte3	'M'	0	1	0	0	1	1	0	1
Byte4	'Q'	0	1	0	1	0	0	0	1
Byte5	'T'	0	1	0	1	0	1	0	0
Byte6	'T'	0	1	0	1	0	1	0	0
Byte7	协议版本(4)	0	0	0	0	0	1	0	0
Byte8	连接标志	X	X	X	X	X	X	X	0
Byte9	Keep Alive MSB	X	X	X	X	X	X	X	X
Byte10	Keep Alive LSB	X	X	X	X	X	X	X	X

图 21-5CONNECT 报文可变报头

21.4.2.2 CONNACK

我们再来讲解一下 CONNACK 报文的可变报头部分，其实有了上一个的经验，这部分对大家来说是很简单的，它是由连接确认标志字段（Connect Acknowledge Flags）与连接返回码字段（Connect Return code）组成，各占用 1 个字节。

它的第 1 个字节是连接确认标志字段，bit1-bit7 是保留位且必须设置为 0，bit0 是当前会话（Session Present）标志位。

它的第 2 个字节是返回码字段，如果服务器收到一个 CONNECT 报文，但出于某些原因无法处理它，服务器会返回一个包含返回码的 CONNACK 报文。如果服务器返回了一个返回码字段是非 0

的 CONNACK 报文，那么它必须关闭网络连接，返回码描述具体见表格 21-3。

表格 21-3 返回码值与对应的描述

返回码值	描述
0x00	连接已被服务端接受
0x01	连接已拒绝，服务端不支持客户端请求的 MQTT 协议级别
0x02	连接已拒绝，服务器标识符是正确的 UTF-8 编码，但不允许使用
0x03	连接已拒绝，网络连接已建立，但 MQTT 服务不可用
0x04	连接已拒绝，用户名或密码的数据格式无效
0x05	连接已拒绝，客户端未被授权连接到此服务器
0x06~0xFF	保留未使用

提示：如果服务端收到清理会话（CleanSession）标志为 1 的连接，除了将 CONNACK 报文中的返回码设置为 0 之外，还必须将 CONNACK 报文中的当前会话设置（Session Present）标志为 0。

那么总的来说，CONNACK 报文的可变报头部分内容具体见图 *21_6*。

bit	说明	7	6	5	4	3	2	1	0
Byte1	连接确认标志	0	0	0	0	0	0	0	X
Byte2	返回码	X	X	X	X	X	X	X	X

图 21-6CONNACK 可变报头

在此，就不再对 MQTT 报文的可变报头部分过多赘述，大家可以参考 MQTT 协议手册，里面有很详细的描述。

21.4.3 有效载荷

有效载荷也是存在与某些报文中，不同的报文有效载荷也是不一样的，比如：

CONNECT 报文的有效载荷（payload）包含一个或多个以长度为前缀的字段，可变报头中的标志决定是否包含这些字段。如果包含的话，必须按这个顺序出现：客户端标识符，遗嘱主题，遗嘱消息，用户名，密码 。

SUBSCRIBE 报文的有效载荷包含了一个主题过滤器列表，它们标识着客户端想要订阅的主题，每一个过滤器后面跟着一个字节，这个字节被叫做服务质量要求（Requested QoS），它给出了服务端向客户端发送应用消息所允许的最大 QoS 等级。

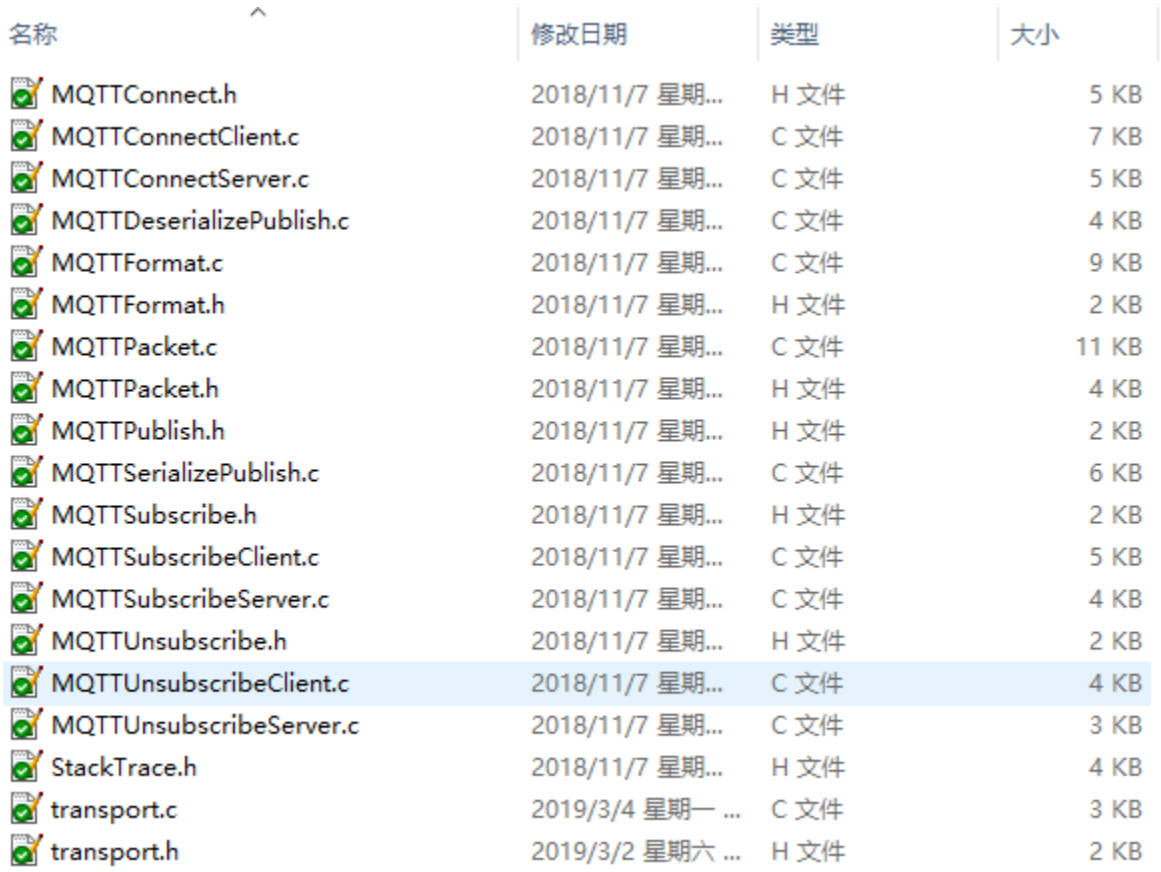
这里只是讲述了一小部分内容，关于具体的有效载荷部分也可以去看 MQTT 手册，此处就不再赘述。

21.5 移植 MQTT 协议

初步了解了 MQTT 协议，那么我们怎么在开发板上运行它呢？首先，LwIP 协议肯定是要的，因为 MQTT 是应用层协议，基于 TCP 协议至少，首先我们就需要把 LwIP 协议跑通，我们就使用 Socket API 来进行移植。

首先下载 MQTT 的库：https://github.com/eclipse/paho.mqtt.embedded-c。

然后创建一个 MQTT 文件夹，再将 MQTTPacketsrc 目录下的文件添加到工程目录 MQTT 文件夹，再将 MQTTPacketsamples 目录下的 transport.c、transport.h 添加到这个文件夹下，添加完成后文件夹内容具体见图 *21_7*。



名称	修改日期	类型	大小
MQTTConnect.h	2018/11/7 星期...	H 文件	5 KB
MQTTConnectClient.c	2018/11/7 星期...	C 文件	7 KB
MQTTConnectServer.c	2018/11/7 星期...	C 文件	5 KB
MQTTDeserializePublish.c	2018/11/7 星期...	C 文件	4 KB
MQTTFormat.c	2018/11/7 星期...	C 文件	9 KB
MQTTFormat.h	2018/11/7 星期...	H 文件	2 KB
MQTTPacket.c	2018/11/7 星期...	C 文件	11 KB
MQTTPacket.h	2018/11/7 星期...	H 文件	4 KB
MQTTPublish.h	2018/11/7 星期...	H 文件	2 KB
MQTTSerializePublish.c	2018/11/7 星期...	C 文件	6 KB
MQTTSubscribe.h	2018/11/7 星期...	H 文件	2 KB
MQTTSubscribeClient.c	2018/11/7 星期...	C 文件	5 KB
MQTTSubscribeServer.c	2018/11/7 星期...	C 文件	4 KB
MQTTUnsubscribe.h	2018/11/7 星期...	H 文件	2 KB
MQTTUnsubscribeClient.c	2018/11/7 星期...	C 文件	4 KB
MQTTUnsubscribeServer.c	2018/11/7 星期...	C 文件	3 KB
StackTrace.h	2018/11/7 星期...	H 文件	4 KB
transport.c	2019/3/4 星期一 ...	C 文件	3 KB
transport.h	2019/3/2 星期六 ...	H 文件	2 KB

图 21-7MQTT 文件夹下的内容

我们把这些文件加入我们的工程之中，并且指定头文件路径，然后实现 transport.c 文件的移植层接口，其内容具体见*代码清单 21_1*

代码清单 21-1transport.c 文件内容

```
#include "transport.h"
#include "lwip/opt.h"
#include "lwip/arch.h"
#include "lwip/api.h"
#include "lwip/inet.h"
#include "lwip/sockets.h"                    (1)
#include "string.h"
```

(下页继续)

(续上页)

```
static int mysock;

/*********************************************************************************
** 函数名称: transport_sendPacketBuffer
** 函数功能: 以 TCP 方式发送数据
** 入口参数: unsigned char* buf: 数据缓冲区
**           int buflen: 数据长度
** 出口参数: <0 发送数据失败
*********************************************************************************/
int32_t transport_sendPacketBuffer( uint8_t* buf, int32_t buflen)
{
    int32_t rc;
    rc = write(mysock, buf, buflen);                (2)
    return rc;
}

/*********************************************************************************
** 函数名称: transport_getdata
** 函数功能: 接收 TCP 数据
** 入口参数: unsigned char* buf: 数据缓冲区
**           int count: 数据长度
** 出口参数: <=0 接收数据失败
*********************************************************************************/
int32_t transport_getdata(uint8_t* buf, int32_t count)
{
    int32_t rc;

    rc = recv(mysock, buf, count, 0);               (3)
    return rc;
}
```

(下页继续)

(续上页)

```
/*********************************************************************************
** 函数名称: transport_open
** 函数功能: 打开一个接口，并且和服务器 建立连接
** 入口参数: char* servip: 服务器域名
**           int   port: 端口号
** 出口参数: <0 打开连接失败
*********************************************************************************/
int32_t transport_open(int8_t* servip, int32_t port)
{
    int32_t *sock = &mysock;
    int32_t ret;
//  int32_t opt;
    struct sockaddr_in addr;

    //初始化服务器信息
    memset(&addr,0,sizeof(addr));
    addr.sin_len = sizeof(addr);
    addr.sin_family = AF_INET;
    //填写服务器端口号
    addr.sin_port = PP_HTONS(port);
    //填写服务器 IP 地址
    addr.sin_addr.s_addr = inet_addr((const char*)servip);

    //创建 SOCK
    *sock = socket(AF_INET,SOCK_STREAM,0);   (4)
    //连接服务器
    ret = connect(*sock,(struct sockaddr*)&addr,sizeof(addr));       (5)
    if (ret != 0)
    {
        //关闭链接
        close(*sock);
        //连接失败
```

(下页继续)

(续上页)

```
        return -1;
    }
    //连接成功, 设置超时时间 1000ms
//  opt = 1000;
//  setsockopt(*sock,SOL_SOCKET,SO_RCVTIMEO,&opt,sizeof(int));

    //返回套接字
    return *sock;
}

/*************************************************************************
** 函数名称: transport_close
** 函数功能: 关闭套接字
** 入口参数: unsigned char* buf: 数据缓冲区
**           int buflen: 数据长度
** 出口参数: <0 发送数据失败
*************************************************************************/
int transport_close(void)
{

    int rc;
//  rc = close(mysock);
    rc = shutdown(mysock, SHUT_WR);
    rc = recv(mysock, NULL, (size_t)0, 0);
    rc = close(mysock);                          (6)
    return rc;
}
```

代码清单 21-1**(1)**：添加头文件，我们使用 Socket API 就添加 LwIP 中对应的头文件。

代码清单 21-1**(2)**：transport_sendPacketBuffer() 函数是 MQTT 发送数据函数，这个函数必须以 TCP 协议发送数据，参数 buf 指定数据缓冲区，buflen 指定了数据长度，调用 write() 函数进行发送数据，并且返回发送状态。

代码清单 21-1**(3)**：transport_getdata() 函数是 MQTT 接收数据的函数，需要我们用 Socket API 获取接收到的数据，参数 buf 指定数据缓冲区，count 指定了获取数据长度，我们只要调用 recv() 将数据获取回来即可。

代码清单 21-1**(4)(5)**：transport_open() 函数用于打开一个连接接口，并且让客户端和服务器建立连接，这个函数是实现 MQTT 的前提，必须产生 TCP 连接才能进入下一步操作，因此我们在函数中需要根据配置信息连接到服务器中，socket() 用于创建一个套接字，并且调用 connect() 函数连接到服务器上，如果连接失败则关闭套接字，返回-1。

代码清单 21-1**(6)**：transport_close() 是 MQTT 与服务器断开的时候会调用的函数，它用来关闭一个套接字的。

然后我们在工程中实现两个线程，一个是 MQTT 发送线程，另一个是 MQTT 接收线程，这样子的话，我们的 MQTT 协议就在开发板中跑起来了，我们提供了完整的 MQTT 客户端连接到服务器 demo，下面简单实现两个线程的处理，更多的代码请参考我们的工程，我们首先在 USER 目录下创建一个 mqttclient.c 文件，然后加入*代码清单 21_2* 所示代码。

代码清单 21-2mqttclient.c 文件内容（部分）

```
void mqtt_recv_thread(void *pvParameters)
{
    uint32_t curtick;
    uint8_t no_mqtt_msg_exchange = 1;
    uint8_t buf[MSG_MAX_LEN];
    int32_t buflen = sizeof(buf);
    int32_t type;
    fd_set readfd;
    struct timeval tv;       //等待时间
    tv.tv_sec = 0;
    tv.tv_usec = 10;


MQTT_START:
    //开始连接
    Client_Connect();
```

(下页继续)

(续上页)

```
    //获取当前滴答，作为心跳包起始时间
    curtick = xTaskGetTickCount();
    while (1)
    {
        //表明无数据交换
        no_mqtt_msg_exchange = 1;

        FD_ZERO(&readfd);
        FD_SET(MQTT_Socket,&readfd);

        //等待可读事件
        select(MQTT_Socket+1,&readfd,NULL,NULL,&tv);

        //判断 MQTT 服务器是否有数据
        if (FD_ISSET(MQTT_Socket,&readfd) != 0)
        {
            //读取数据包--注意这里参数为 0，不阻塞
            type = ReadPacketTimeout(MQTT_Socket,buf,buflen,0);
            if (type != -1)
            {
                mqtt_pktype_ctl(type,buf,buflen);
                //表明有数据交换
                no_mqtt_msg_exchange = 0;
                //获取当前滴答，作为心跳包起始时间
                curtick = xTaskGetTickCount();
            }
        }

        //这里主要目的是定时向服务器发送 PING 保活命令
        if ((xTaskGetTickCount() - curtick) >(KEEPLIVE_TIME/2*1000))
        {
            curtick = xTaskGetTickCount();
```

(下页继续)

(续上页)

```
            //判断是否有数据交换
            if (no_mqtt_msg_exchange == 0)
            {
                //如果有数据交换，这次就不需要发送 PING 消息
                continue;
            }

            if (MQTT_PingReq(MQTT_Socket) < 0)
            {
                //重连服务器
                PRINT_DEBUG("发送保持活性 ping 失败....\n");
                goto CLOSE;
            }

            //心跳成功
            PRINT_DEBUG("发送保持活性 ping 作为心跳成功....\n");
            //表明有数据交换
            no_mqtt_msg_exchange = 0;
        }
    }

CLOSE:
    //关闭链接
    transport_close();
    //重新链接服务器
    goto MQTT_START;
}

void mqtt_send_thread(void *pvParameters)
{
    int32_t ret;
    uint8_t no_mqtt_msg_exchange = 1;
```

(下页继续)

(续上页)

```
    uint32_t curtick;
    uint8_t res;
    /* 定义一个创建信息返回值，默认为 pdTRUE */
    BaseType_t xReturn = pdTRUE;
    /* 定义一个接收消息的变量 */
//    uint32_t* r_data;
    DHT11_Data_TypeDef* recv_data;
    //初始化 json 数据
    cJSON* cJSON_Data = NULL;
    cJSON_Data = cJSON_Data_Init();
    double a,b;
MQTT_SEND_START:

    while (1)
    {

        xReturn = xQueueReceive( MQTT_Data_Queue,     /* 消息队列的句柄 */
                                 &recv_data,       /* 发送的消息内容 */
                                 3000); /* 等待时间 3000ms */
        if (xReturn == pdTRUE)
        {
            a = recv_data->temperature;
            b = recv_data->humidity;
            printf("a = %f,b = %f\n",a,b);
            //更新数据
            res = cJSON_Update(cJSON_Data,TEMP_NUM,&a);
            res = cJSON_Update(cJSON_Data,HUM_NUM,&b);

            if (UPDATE_SUCCESS == res)
            {
                //更新数据成功，
                char* p = cJSON_Print(cJSON_Data);
```

(下页继续)

(续上页)

```
                //发布消息
                ret = MQTTMsgPublish(MQTT_Socket,(char*)TOPIC,QOS0,(uint8_
↪t*)p);
                if (ret >= 0)
                {
                    //表明有数据交换
                    no_mqtt_msg_exchange = 0;
                    //获取当前滴答，作为心跳包起始时间
                    curtick = xTaskGetTickCount();
                }
                vPortFree(p);
                p = NULL;
            }
            else
                PRINT_DEBUG("update fail\n");
        }
        //这里主要目的是定时向服务器发送 PING 保活命令
        if ((xTaskGetTickCount() - curtick) >(KEEPLIVE_TIME/2*1000))
        {
            curtick = xTaskGetTickCount();
            //判断是否有数据交换
            if (no_mqtt_msg_exchange == 0)
            {
                //如果有数据交换，这次就不需要发送 PING 消息
                continue;
            }

            if (MQTT_PingReq(MQTT_Socket) < 0)
            {
                //重连服务器
                PRINT_DEBUG("发送保持活性 ping 失败....\n");
                goto MQTT_SEND_CLOSE;
```

(下页继续)

(续上页)

```
            }

            //心跳成功
            PRINT_DEBUG("发送保持活性 ping 作为心跳成功....\n");
            //表明有数据交换
            no_mqtt_msg_exchange = 0;
        }
    }
MQTT_SEND_CLOSE:
    //关闭链接
    transport_close();
    //开始连接
    Client_Connect();
    goto MQTT_SEND_START;
}

void
mqtt_thread_init(void)
{
    sys_thread_new("mqtt_recv_thread", mqtt_recv_thread, NULL, 2048, 6);
    sys_thread_new("mqtt_send_thread", mqtt_send_thread, NULL, 2048, 7);
}
```

21.6 cJSON 移植

其实在 mqttclient.c 文件中我们就使用了 cJSON 的内容，而且，在连接到各大云平台中经常使用 JSON 格式发送和接收数据，因此我们必须将 cJSON 也移植到我们的工程中。cJSON 是一个用于解析 JSON 包的 C 语言库，库文件为 cJSON.c 和 cJSON.h，所有的实现都在这两个文件中。

cJSON 的移植很简单，首先我们首先下载到 cJSON 的源码文件：https://github.com/DaveGamble/cJSON。

然后在文件目录下找到 cJSON.c 和 cJSON.h，将它们拷贝到我们的工程目录下的 cJSON 文件夹下（如果没有就创建它），然后添加到工程中，并且指定头文件路径即可，因为我们使用的是 FreeRTOS 操作系统，那么 cJSON 中的动态内存分配、释放函数就需要配合操作系统的动态内存分配函数与释放函数，在 cJSON.c 文件中修改*代码清单 21_3* 所示的代码即可，当然还需要注意包含 FreeRTOS 相关的头文件。

代码清单 21-3cJSON.c 文件修改的内容

```
static void * CJSON_CDECL internal_malloc(size_t size)
{
//    return malloc(size);
    return pvPortMalloc(size);
}
static void CJSON_CDECL internal_free(void *pointer)
{
//    free(pointer);
    vPortFree(pointer);
}
static void * CJSON_CDECL internal_realloc(void *pointer,
        size_t size)
{
//    return realloc(pointer, size);
    return NULL;
}
#else
#define internal_malloc pvPortMalloc
#define internal_free vPortFree
#define internal_realloc
#endif
```

为了更好利用 cJSON 提供的函数来处理我们的程序，我们简单对 cJSON 进行了封装，包含 cJSON 格式数据的初始化、更新、解析等，当然大家也可以自行封装使用，我们创建一个 cJSON_Process.c 文件，并添加以下代码，具体见*代码清单 21_4*。

代码清单 21-4 cJSON_Process.c 文件内容

```
#include "cJSON_Process.h"
#include "main.h"

cJSON* cJSON_Data_Init(void)
{
    cJSON* cJSON_Root = NULL;     //json 根节点

    cJSON_Root = cJSON_CreateObject();    /* 创建项目 */
    if (NULL == cJSON_Root)
    {
        return NULL;
    }/* 添加元素  键值对 */
    cJSON_AddStringToObject(cJSON_Root, NAME, DEFAULT_NAME);
    cJSON_AddNumberToObject(cJSON_Root, TEMP_NUM, DEFAULT_TEMP_NUM);
    cJSON_AddNumberToObject(cJSON_Root, HUM_NUM, DEFAULT_HUM_NUM);

    char* p = cJSON_Print(cJSON_Root);  /*p 指向的字符串是 json 格式的 */

//  PRINT_DEBUG("%s\n",p);

    vPortFree(p);
    p = NULL;

    return cJSON_Root;

}
uint8_t cJSON_Update(const cJSON * const object,const char * const string,
↪void *d)
{
    cJSON* node = NULL;     //json 根节点
    node = cJSON_GetObjectItem(object,string);
```

(下页继续)

(续上页)

```
    if (node == NULL)
        return NULL;
    if (cJSON_IsBool(node))
    {
        int *b = (int*)d;
//    printf ("d = %d",*b);
        cJSON_GetObjectItem(object,string)->type = *b ? cJSON_True : cJSON_
↪False;
//    char* p = cJSON_Print(object);    /*p 指向的字符串是 json 格式的 */
        return 1;
    }
    else if (cJSON_IsString(node))
    {
        cJSON_GetObjectItem(object,string)->valuestring = (char*)d;
//    char* p = cJSON_Print(object);    /*p 指向的字符串是 json 格式的 */
        return 1;
    }
    else if (cJSON_IsNumber(node))
    {
        double *num = (double*)d;
//    printf ("num = %f",*num);
//    cJSON_GetObjectItem(object,string)->valueint = (double)*num;
        cJSON_GetObjectItem(object,string)->valuedouble = (double)*num;
//    char* p = cJSON_Print(object);    /*p 指向的字符串是 json 格式的 */
        return 1;
    }
    else
        return 1;
}

void Proscess(void* data)
{
```

(下页继续)

(续上页)

```
    PRINT_DEBUG("开始解析 JSON 数据");
    cJSON *root,*json_name,*json_temp_num,*json_hum_num;
    root = cJSON_Parse((char*)data); //解析成 json 形式

    json_name = cJSON_GetObjectItem( root , NAME);  //获取键值内容
    json_temp_num = cJSON_GetObjectItem( root , TEMP_NUM );
    json_hum_num = cJSON_GetObjectItem( root , HUM_NUM );

    PRINT_DEBUG("name:%s\n temp_num:%f\n hum_num:%f\n",
                json_name->valuestring,
                json_temp_num->valuedouble,
                json_hum_num->valuedouble);

    cJSON_Delete(root);  //释放内存
}
```

到了这里，我们的整个 MQTT 的程序框架基本完成了，下面我们就开始使用 MQTT 程序。

第 22 章　连接到百度天工物接入

22.1 物接入简介

物接入（IoT Hub）是面向物联网领域开发者的全托管云服务，通过主流的物联网协议（如 MQTT）进行通信，可以在智能设备与云端之间建立安全的双向连接，快速实现物联网项目。支持亿级并发连接和消息数，支持海量设备与云端安全可靠的双向连接，无缝对接天工平台和百度云的各项产品和服务。

物接入分为设备型和数据型两种项目类型，我们在这里使用数据型项目类型，传输一些温湿度数据，同时可以无缝对接时序数据库 TSDB、物可视等，将数据实时显示出来，同时我们采用 MQTT 协议与云端进行通讯。

22.2 使用 IoT Hub

22.2.1 创建计费套餐

在使用物接入服务前，必须要创建一个百度云账号用于管理我们的设备，所有的设备信息都是在云端后台进行管理的，地址：https://console.bce.baidu.com/iot2/hub/。

在使用 IoT Hub 之前应先创建计费套餐并设定每个月收发消息的流量额度，每个用户只能创建一个计费套餐，所有项目将共享该套餐的额度，目前 IoT Hub 的计费是很便宜的，每个月的前 1000000 条消息是免费的，更多的消息数量请自行参考 IoT Hub 的收费说明，因此我们用它来做实验早已足够，计算一下，假设我们的开发板在一个月内不间断以 3 秒的频率发送一次消息到 IoT Hub，那么一个月只能发送 30*24*60*(60/3) = 864000，还不到 1000000 条呢，因此这个流量用来做实验早就绰绰有余，即使这个月用完了流量，下个月还有 1000000 条，只要当月没超出这个流量都是可以免费使用，具体见图 *22_1*。

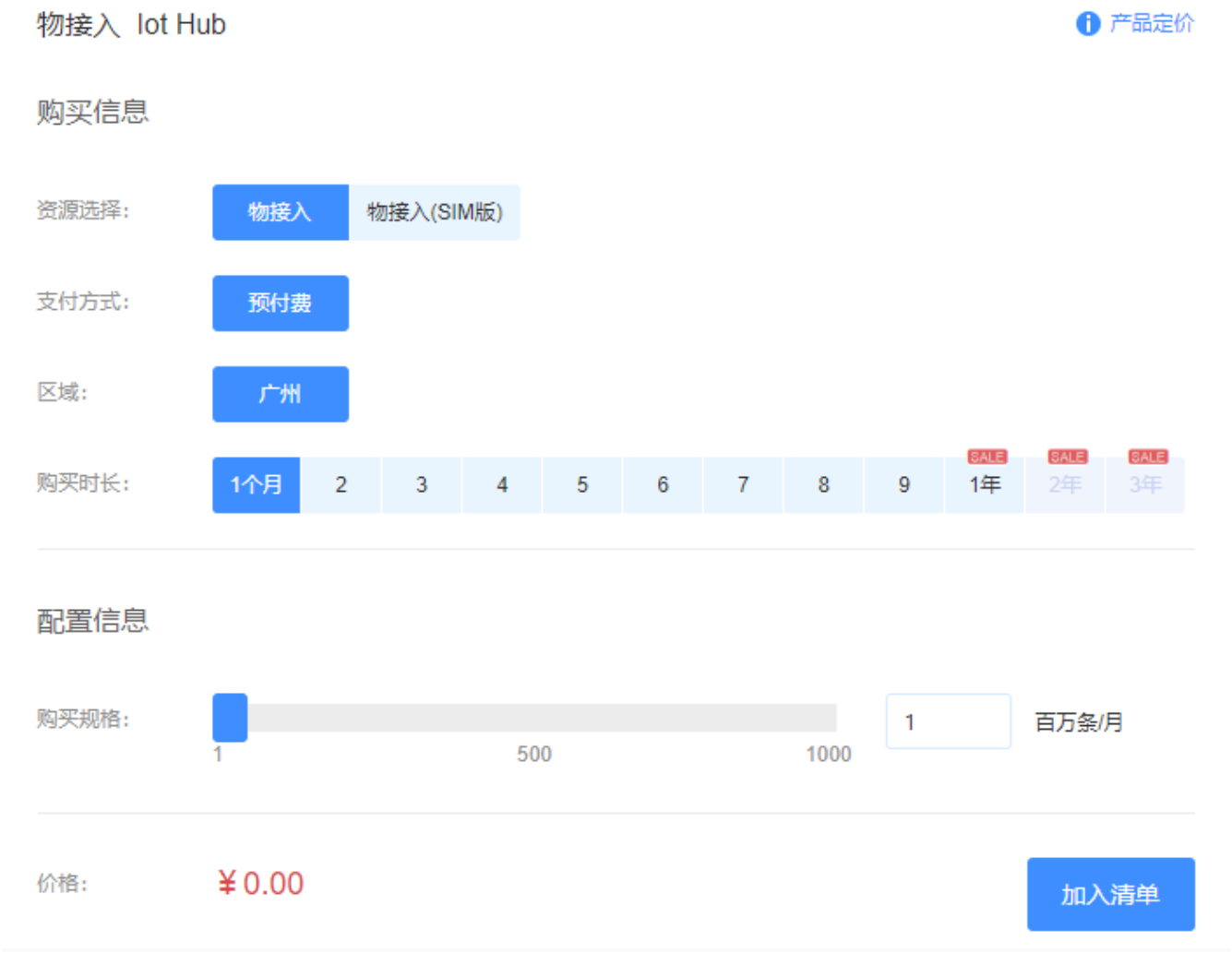


图 22-1 创建计费套

22.2.2 创建项目

IoT Hub 是用于物联网设备的后台管理的，将设备接入 IoT Hub 的第二步就是创建一个项目，每个项目会对应一个接入点（endpoint），一个项目就表示一个完整的物接入服务，首先登录物接入控制台页面：https://console.bce.baidu.com/iot2/hub/project/list，点击“创建项目”，填写需要创建 IoT Hub 服务的项目名称、选择项目类型为数据型，并提交即可，创建的项目，默认具有多种通信方式，我们暂时只关注 TCP 连接方式即可，因为开发板是采用 MQTT 协议与云端交互的，端

口号是 1883，具体见图 *22_1* 与图 *22_3*。

此处补充说明一点：通过项目可以将不同项目的设备进行隔离和管理，一个项目下允许有多个用户（设备），用户与身份进行绑定，而每个身份需要绑定一个策略，而策略下可以创建多个不同的主题，因此一个设备可以订阅多个不同的主题，并且同一个项目下的主题是共享的，所有的设备均可进行订阅。



图 22-2 创建项目

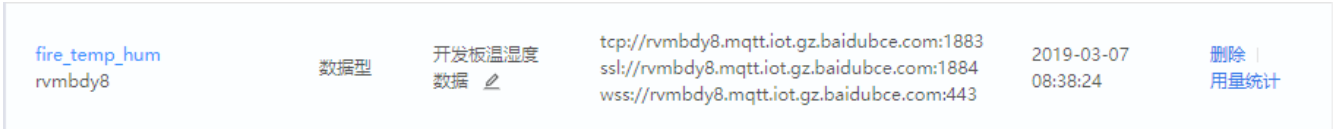


图 22-3 创建完成

22.2.3 创建策略

首先我们点击“fire_temp_hum”项目，进入项目里面，首先创建一个策略（简单来说就是主题），输入对应的名称与主题，选择发布与订阅权限，当创建完成后，项目下的设备就可以订阅这个主题，具体见图 *22_4*。

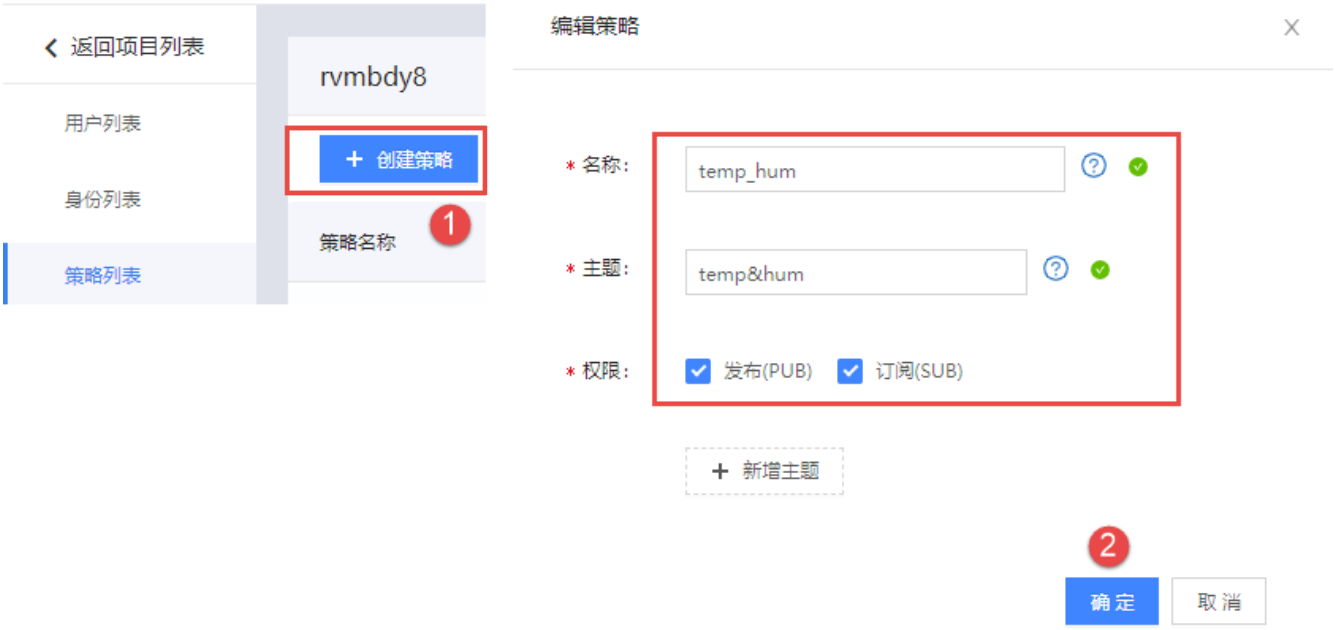


图 22-4 创建策略

22.2.4 创建身份与创建用户

身份是开发板连接 IoT Hub 的重要途径，这里面包含了秘钥，标识着开发板能否通过 IoT Hub 的验证，这也是安全保障的主要操作，首先点击“身份列表”，再点击“创建身份”，然后根据自己的信息进行填写，在这里要注意的是需要勾选密码验证，再选择之前我们创建的策略进行绑定（如果没有则必须创建），当身份创建完成时候生成的秘钥是用于开发板连接 IoT Hub 的必要条件，记住不能丢失，如果丢失了就找不回来了，在创建的时候就把它保存好，具体见图 *22_6*。

在创建用户的时候，只需要把用户名设置好，然后再绑定身份即可，这样子一个设备就对应到云端了，并且可以向策略中设置的主题发布消息和订阅主题，具体见图 *22_5*。



图 22-5 创建用户并绑定身份


创建身份
1 创建身份
2 设置策略
3 配置确认
* 名称：
stm32f4
* 认证方式：
证书认证
密码认证
下一步
取 消
创建身份
创建身份
2 设置策略
3 配置确认
* 策略：
temp_hum
为设备选择策略（包括主题和权限），若没有须创建
上一步
下一步
取 消
创建身份
创建身份
设置策略
3 配置确认
身份
名称：
stm32f4
密钥：
点击复制
请合理保管以上密钥，密钥丢失无法找回，只能重新生成
策略
名称：
temp_hum
确 认

图 22-6 创建身份

当所有的东西都创建完成，我们点击身份操作中的“测试连接”，在网页上测试一下我们能不能正常进行 MQTT 通信、发布与定阅主题，在页面中输入刚刚保存的秘钥，点击“connect”进行连接测试，如果前面的步骤全部做完，并且秘钥是正确的，就可以发现我们的连接是正常的，然后我们订阅一下之前创建策略时候的主题名字“temp&hum”，再向这个主题发送一个消息，消息的内容由自己定义，具体见图 *22_7*。

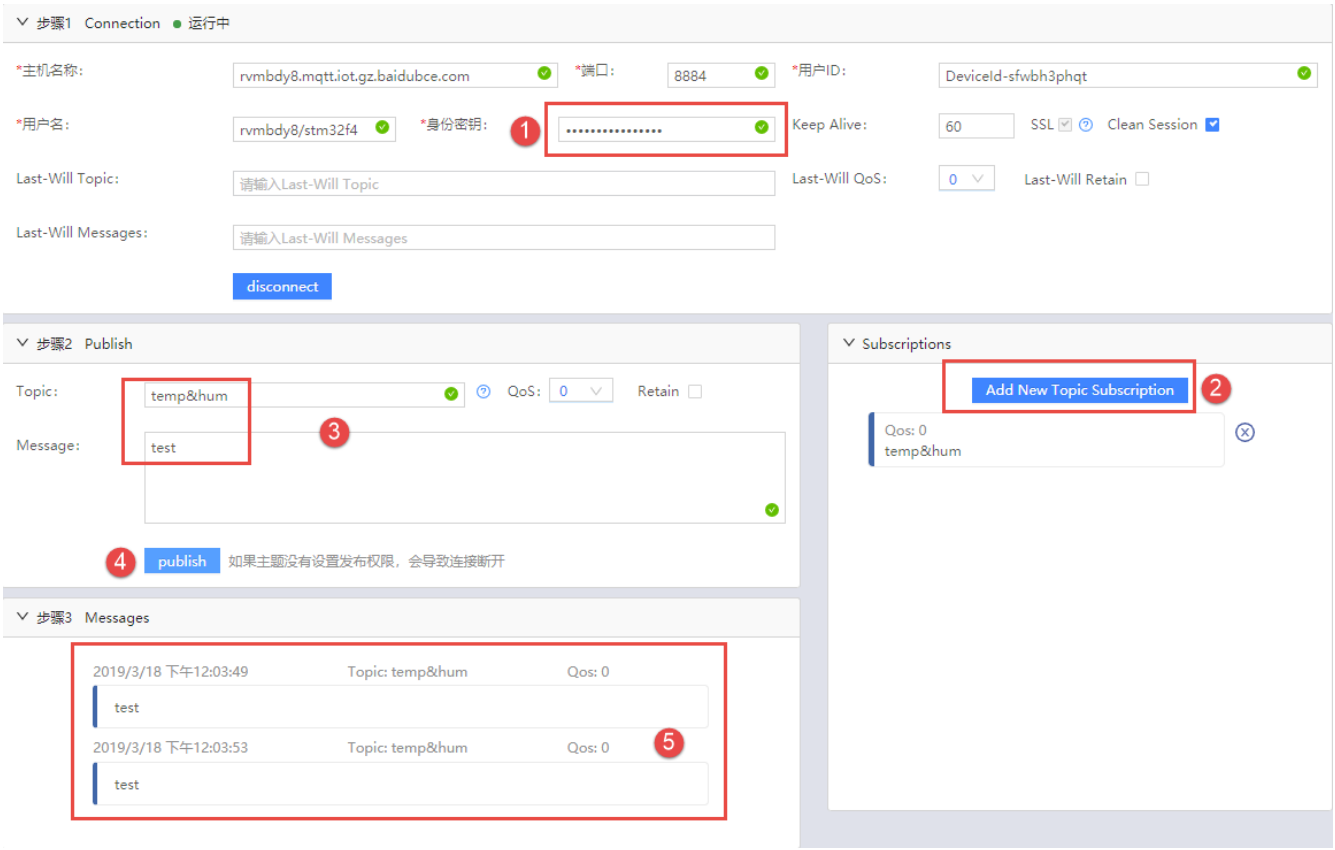


图 22-7 测试连接

22.2.5 MQTT 软件测试连接

仔细观察的同学可能会发现，这个连接的端口是 8884，因为这是在网页上测试连接的，所以端口不一样是正常的，那么我们也可以使用 MQTT 客户端软件进行连接测试，首先在我们论坛中提供的资料下载客户端软件，http://www.firebbs.cn/forum.php?mod=viewthread&tid=26274&fromuid=37393，然后进行安装，就可以利用软件当做客户端去测试连接天工物接入平台，MQTT.fx 是一款基于 Eclipse Paho，使用 Java 语言编写的 MQTT 客户端工具，支持通过 Topic 订阅和发布消息。打开软件，点击设置，来配置客户端的基本信息，填写用户名与密码，进行连接，具体见图 *22_8*

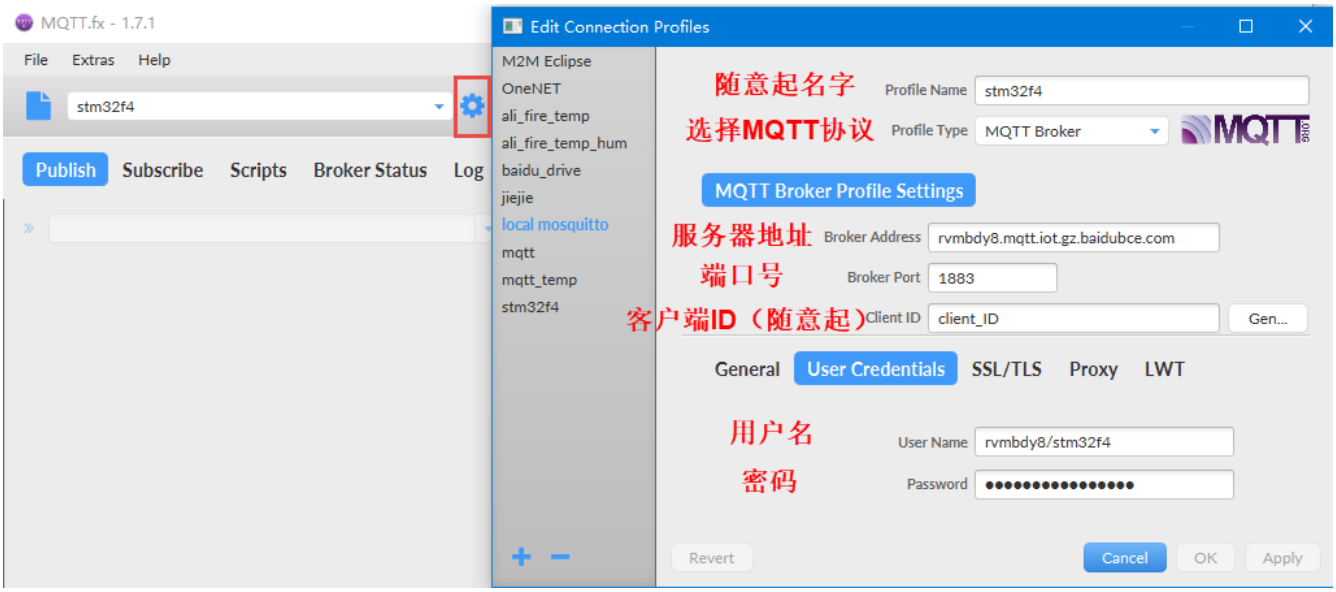


图 22-8 配置 MQTT 客户端软件

配置完成之后，可以尝试连接，点击 Connetc，当连接成功后，下面的 Publish 与 Subscribe 等选项是可以选择的，那么我们点击订阅选项 Subscribe，输入订阅的主题，然后点击 subscribe，可以发现已经成功订阅，具体见图 *22_9*，但是此时还未有消息，因此我们需要去 Publish 选项中向这个主题发布消息，当发布成功后，订阅的主题就会回显出服务器发送给客户端的消息，具体见图 *22_10*。

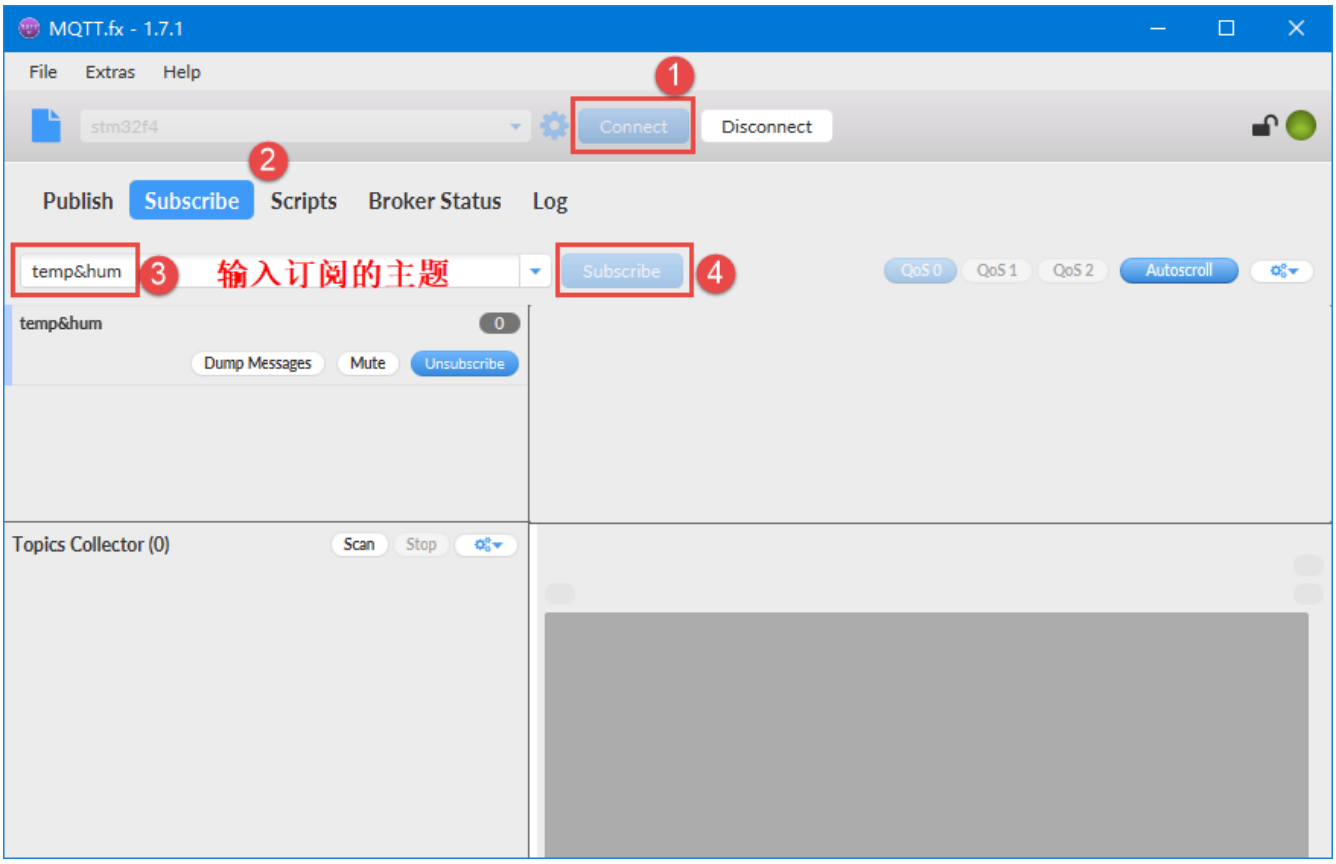

MQTT.fx - 1.7.1
File
Extras
Help
1
stm32f4
Connect
Disconnect
2
Publish
Subscribe
Scripts
Broker Status
Log
temp&hum
3
输入订阅的主题
Subscribe
4
QoS 0
QoS 1
QoS 2
Autoscroll
temp&hum
0
Dump Messages
Mute
Unsubscribe
Topics Collector (0)
Scan
Stop

图 22-9 订阅主题

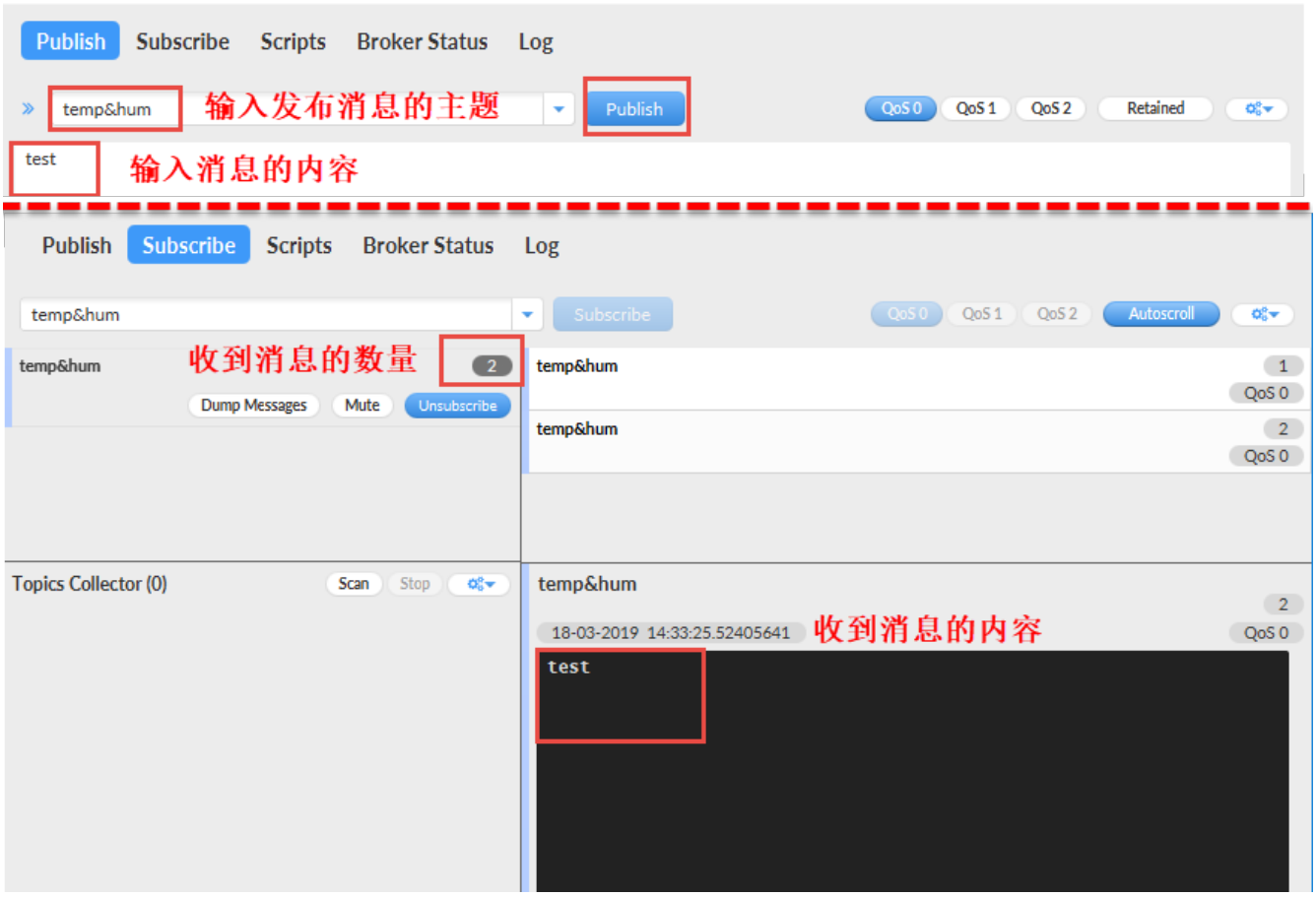


图 22-10 发布消息与收到消息

这样子说明我们已经能正常连接到百度天工物了，下一步就是使用开发板进行连接测试。

22.3 开发板连接 IoT Hub

首先拿到上一章移植好的例程，打开 mqttclient.h 文件，配置连接的参数，如服务器域名（此处需要开启 DNS 域名解析功能），然后配置端口号、用户名与秘钥（这里的秘钥是自己的，根据实际情况进行配置），配置订阅的主题等信息，具体见*代码清单 22_1*。

代码清单 22-1mqttclient.h 的宏定义配置

```
#ifdef    LWIP_DNS
#define   HOST_NAME       "rvmbdy8.mqtt.iot.gz.baidubce.com"   //服务器域名
#else
#define   HOST_NAME       "14.215.190.20"     //服务器 IP 地址
#endif


//#define   HOST_IP       "129.204.201.235"
#define   HOST_PORT     1883    //由于是 TCP 连接，端口必须是 1883

#define   CLIENT_ID     "client_ID"          //随机的 id
#define   USER_NAME     "rvmbdy8/stm32f4"     //用户名
#define   PASSWORD      "9EeVCRrCN5i6XXXX"  //秘钥

#define   TOPIC         "temp&hum"       //订阅的主题

#define   TEST_MESSAGE  "test_message"  //发送测试消息
```

然后开始连接到云平台，首先我们介绍一下我们提供的封装代码，看看 MQTT 客户端是怎么连接到云平台的，具体见代码清单 22_2。

代码清单 22-2 连接到云平台

```
void Client_Connect(void)
{
    char* host_ip;

#ifdef  LWIP_DNS
    ip4_addr_t dns_ip;
    netconn_gethostbyname(HOST_NAME, &dns_ip);                (1)
    host_ip = ip_ntoa(&dns_ip);
    PRINT_DEBUG("host name : %s , host_ip : %s\n",HOST_NAME,host_ip);
#else
```

(下页继续)

(续上页)

```
    host_ip = HOST_NAME;
#endif
MQTT_START:

    //创建网络连接
    PRINT_DEBUG("1. 开始连接对应云平台的服务器...\n");
    PRINT_DEBUG("服务器 IP 地址: %s, 端口号: %0d! \n",host_ip,HOST_PORT);
    while (1)
    {
        //连接服务器
        MQTT_Socket = transport_open((int8_t*)host_ip,HOST_PORT); (2)
        //如果连接服务器成功
        if (MQTT_Socket >= 0)
        {
            PRINT_DEBUG("连接云平台服务器成功! \n");
            break;
        }
        PRINT_DEBUG("连接云平台服务器失败, 等待 3 秒再尝试重新连接! \n");
        //等待 3 秒
        vTaskDelay(3000);
    }

    PRINT_DEBUG("2.MQTT 用户名与秘钥验证登陆...\n");
    //MQTT 用户名与秘钥验证登陆
    if (MQTT_Connect() != Connect_OK)                                    (3)
    {
        //重连服务器
        PRINT_DEBUG("MQTT 用户名与秘钥验证登陆失败...\n");
        //关闭链接
        transport_close();
        goto MQTT_START;
    }
```

(下页继续)

(续上页)

```
    //订阅消息
    PRINT_DEBUG("3. 开始订阅消息...\n");
    //订阅消息
    if (MQTTSubscribe(MQTT_Socket,(char *)TOPIC,QOS1) < 0)   (4)
    {
        //重连服务器
        PRINT_DEBUG("客户端订阅消息失败...\n");
        //关闭链接
        transport_close();
        goto MQTT_START;
    }

    //无限循环
    PRINT_DEBUG("4. 开始循环接收订阅的消息...\n");

}
```

代码清单 22-2**(1)**： 如果打开了 DNS 域名解析功能，就将天工物的域名进行解析，转成 IP 地址。

代码清单 22-2**(2)**：根据 IP 地址与端口号建立 TCP 连接，这个函数的功能在一开始的时候就已经讲解过了，如果没能连接成功就等待 3 秒重新尝试连接直到连接成功。

代码清单 22-2**(3)**：调用 MQTT_Connect() 函数进行验证操作，验证用户名与密码是否准确，如果无法通过验证，就终止 TCP 连接，然后 goto MQTT_START 尝试重新连接，该函数具体见代码清单 22-3。

代码清单 22-2**(4)**：调用 MQTTSubscribe() 函数订阅主题，指定主题的名字，与消息服务质量，该函数的实现具体见*代码清单 22_3*。

代码清单 22-3 连接相关函数

```
uint8_t MQTT_Connect(void)
{
```

(下页继续)

(续上页)

```
    MQTTPacket_connectData data = MQTTPacket_connectData_initializer; (1)
    uint8_t buf[200];
    int buflen = sizeof(buf);
    int len = 0;
    data.clientID.cstring = CLIENT_ID;       //可以是随机的
    data.keepAliveInterval = KEEPLIVE_TIME; //保持活跃
    data.username.cstring = USER_NAME;       //用户名
    data.password.cstring = PASSWORD;        //秘钥
    data.MQTTVersion = MQTT_VERSION;         //3 表示 3.1 版本, 4 表示 3.11 版
本
    data.cleansession = 1;
    //组装消息
    len = MQTTSerialize_connect((unsigned char *)buf, buflen, &data); (2)
    //发送消息
    transport_sendPacketBuffer(buf, len);                    (3)

    /* 等待连接响应 */
    if (MQTTPacket_read(buf, buflen, transport_getdata) == CONNACK) (4)
    {
        unsigned char sessionPresent, connack_rc;
        if (MQTTDeserialize_connack(&sessionPresent, &connack_rc,
                        buf, buflen) != 1 || connack_rc != 0)
        {
            PRINT_DEBUG("无法连接, 错误代码是: %d! \n", connack_rc);
            return Connect_NOK;
        }
        else
        {
            PRINT_DEBUG("用户名与秘钥验证成功, MQTT 连接成功! \n");
            return Connect_OK;
        }
    }
```

(下页继续)

(续上页)

```
    else
        PRINT_DEBUG("MQTT 连接无响应! \n");
    return Connect_NOTACK;
}


/*********************************************************************
** 函数名称: MQTTSubscribe
** 函数功能: 订阅消息
** 入口参数: int32_t sock: 套接字
**           int8_t *topic: 主题
**           enum QoS pos: 消息质量
** 出口参数: >=0: 发送成功 <0: 发送失败
** 备    注:
*********************************************************************/
int32_t MQTTSubscribe(int32_t sock,char *topic,enum QoS pos)
{
    static uint32_t PacketID = 0;
    uint16_t packetidbk = 0;
    int32_t conutbk = 0;
    uint8_t buf[100];
    int32_t buflen = sizeof(buf);
    MQTTString topicString = MQTTString_initializer;
    int32_t len;
    int32_t req_qos,qosbk;

    fd_set readfd;
    struct timeval tv;
    tv.tv_sec = 2;
    tv.tv_usec = 0;

    FD_ZERO(&readfd);
```

(下页继续)

(续上页)

```
    FD_SET(sock,&readfd);

    //复制主题
    topicString.cstring = (char *)topic;
    //订阅质量
    req_qos = pos;

    //串行化订阅消息
    len = MQTTSerialize_subscribe(buf, buflen, 0,
                                  PacketID++, 1, &topicString, &req_qos);
    //发送 TCP 数据
    if (transport_sendPacketBuffer(buf, len) < 0)            (5)
        return -1;

    //等待可读事件--等待超时
    if (select(sock+1,&readfd,NULL,NULL,&tv) == 0)           (6)
        return -2;
    //有可读事件--没有可读事件
    if (FD_ISSET(sock,&readfd) == 0)
        return -3;

    //等待订阅返回--未收到订阅返回
    if (MQTTPacket_read(buf, buflen, transport_getdata) != SUBACK)
        return -4;

    //拆订阅回应报文
    if (MQTTDeserialize_suback(&packetidbk,1,
                               &conutbk, &qosbk, buf, buflen) != 1)
        return -5;

    //检测返回数据的正确性
    if ((qosbk == 0x80)||(packetidbk != (PacketID-1)))
```

(下页继续)

(续上页)

```
        return -6;

    //订阅成功
    return 0;
}
```

代码清单 22-3(**1**)：首先定义一个 MQTTPacket_connectData 类型的结构体，它是 MQTT 库中提供的结构体，用于 MQTT 建立会话时候的字段填充，我们从 MQTT 连接报文也知道，它有很多字段，因此我们需要填写对应的字段，如客户端 ID、保存活跃的时间、用户名、密码、MQTT 版本等信息，当然也可以选择遗嘱消息等。

代码清单 22-3(**2**)：调用 MQTT 库提供的 MQTTSerialize_connect() 函数，将我们定义的数据组装好，也可以称之为串行格式化数据，然后我们就可以直接将组装好的数据发送出去，这样子能满足服务器的处理数据格式要求。

代码清单 22-3(**3**)：调用底层 transport_sendPacketBuffer() 函数将数据发送出去。

代码清单 22-3(**4**)：调用 MQTTPacket_read() 函数等待着服务器的连接应答报文，如果等不到就表示没建立会话。

代码清单 22-3(**5**)：对于订阅主题，也是差不多的操作，调用 MQTTSerialize_connect() 函数将数据封装好，再发送出去。

代码清单 22-3(**6**)：等待着服务器的回应，如果有消息回应，就将订阅回应报文拆解，得到相关的信息。

在建立会话之后，我们就能发布与订阅消息了，当然，如果需要订阅多个主题，在合适的地方将订阅主题的函数再调用一次即可，注意传递的参数是主题的名称。

我们使用 DHT11 传感器采集温湿度数据，然后将这些数据实时上报到云端，打开我们的配套例程，将程序编译，下载到开发板，打开串口调试助手，复位就可以看到实验的现象，具体见图 *22_11*。



图 22-11 发布与订阅主题实验现象

22.4 IoT Hub 的规则引擎

22.4.1 什么是规则引擎

什么是规则引擎？简单来说就是一个中间层，它在软件上的实现能省去很多 if/else 等判断的嵌套。但是我们现在说的 IoT Hub 规则引擎是为了让业务逻辑更加清晰，从海量的物联网数据筛选出适合的数据，并将其进行储存、转发、汇报等操作，让业务规则也变得更加简单，因为在物联网中，数据量是非常巨大的，业务规则更是多种多样，IoT Hub 规则引擎就是需要将海量的数据与千奇

百怪的业务规则变得简单，以适应业务规则的多样性，我们只需要在后台设置一些规则，就能让数据产生不同的作用。而物联网产品会产生大量的数据上报到云端，这些数据往往对应着不同的应用分析场景，如监控厂区的温度湿度监控点，每分钟都会有温度和湿度数据传往云端，对于这些数据，我们往往希望它们发挥不同的作用，并且实时性要非常好，以便在出现问题的时候第一时间能得到告警作用，例如以下应用场景：

1. 实时告警异常的数据，如温度湿度过高或者过低；
2. 分析数据，统计两个小时内的温度最大最小和均值等；
3. 将全部的数据做冷备份以便查询；
4. 对去除异常数据之后的正常数据做数据分析和预测等等。

而规则引擎就是通过灵活的设定规则，将设备传上云端的数据，送往不同的数据目的地（如监控告警设备、时序数据库 TSDB、Kafka、对象存储 BOS 等）以达到不同的业务目标，其示意图具体见图 *22_12*。

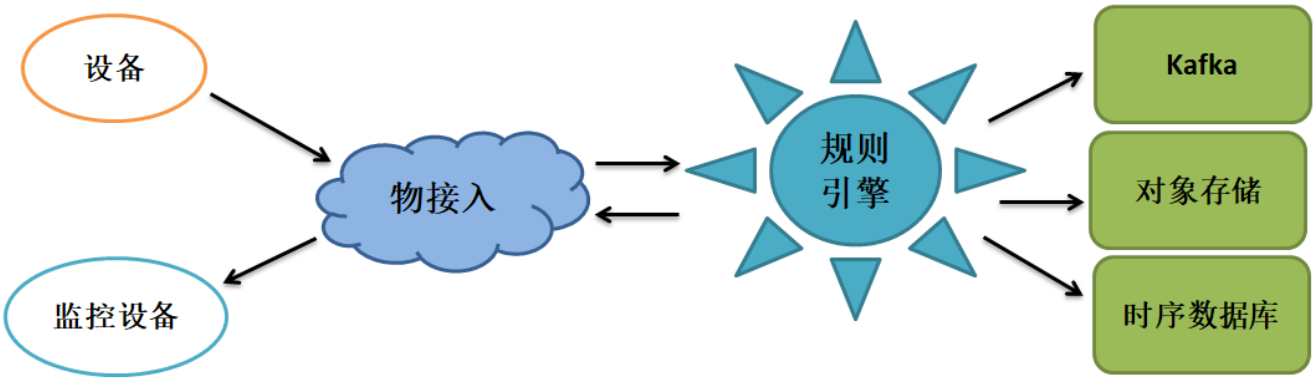


图 22-12 规则引擎示意图

22.4.2 使用规则引擎

在使用规则引擎之前，我们需要指定，规则引擎处理的数据是 JSON 数据（查询字段为 * 且约束条件为空的规则除外），因此我们需要将数据以 JSON 格式上报云端，这样子才能让规则引擎识别到我们的数据，然后明确使用规则引擎的目的，比如我们要监控温度数据，只有当温度高于 30℃（此处是为了方便测试而选取的温度值）的时候，就告警一下温度数据，那么问题来了，这

些数据应该告警到哪里呢？所以我们需要创建一个监控设备，按照之前创建开发板设备一样，创建一个监控温度的设备，此处就不再赘述过程，我们创建一个 MQTT 上位机软件作为监控设备的客户端，监控开发板上报的温湿度数据。

首先打开设置规则引擎的网页：https://console.bce.baidu.com/iot2/re/rule/list，点击创建一个规则引擎，填写要处理数据的实例（项目）和主题“temp&hum”，在输入完成的时候，如果我们开发板运行程序，那么测速输入就会有数据，当然，也能自己将数据输入来测试规则引擎是否可用，然后在查询字段中填写“temp”，表示查询数据中的 temp 字段内容，而约束条件表示对数据的处理，比如我们输入“temp>30”就表示只对“temp”值大于 30 的数据进行处理，通过测试我们也知道，当前“temp”的值是 25.06，还不满足规则引擎的约束条件，因此是不会有数据输出的。此外我们还需设置数据目的地，即规则满足引擎运作之后的数据要往哪里转发，是存储还是告警到监控设备中，我们在设置数据目的地的时候，选择物接入主题，选择实例和主题，完成之后就提交，其操作过程具体见图 *22_13*，按编号顺序进行操作即可。

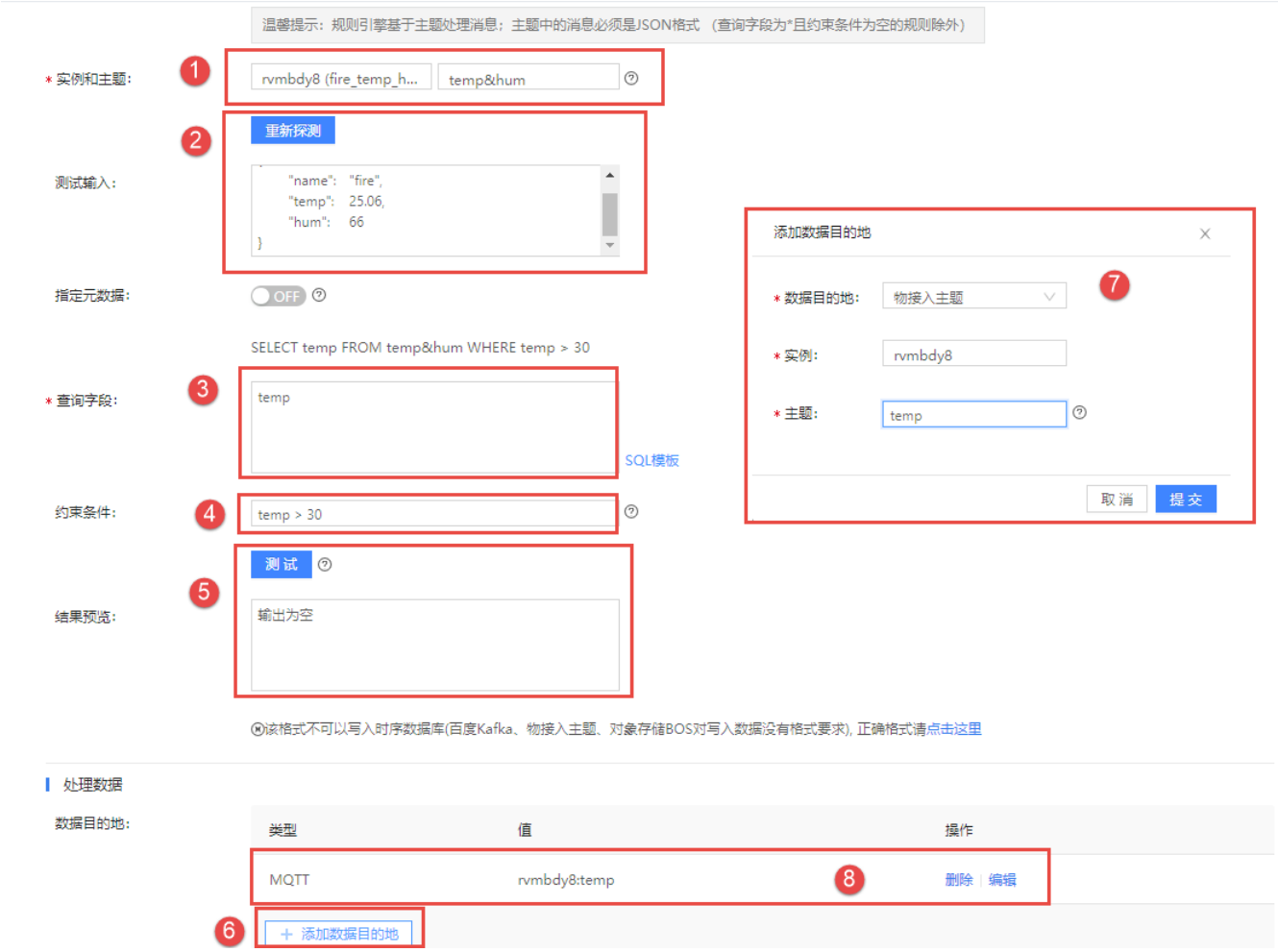


图 22-13 添加规则引擎

提示：这里设置的规则是无法写入时序数据库的。

然后在 MQTT 客户端软件上连到天工物，订阅 temp 主题，然后打开开发板，当 DHT11 温湿度传感器测得的温度大于 30℃，就会触发规则引擎，并且将数据转发到指定的数据目的地——temp 主题，在 MQTT 软件上我们就能看到对应的数据，具体见图 *22_14*。

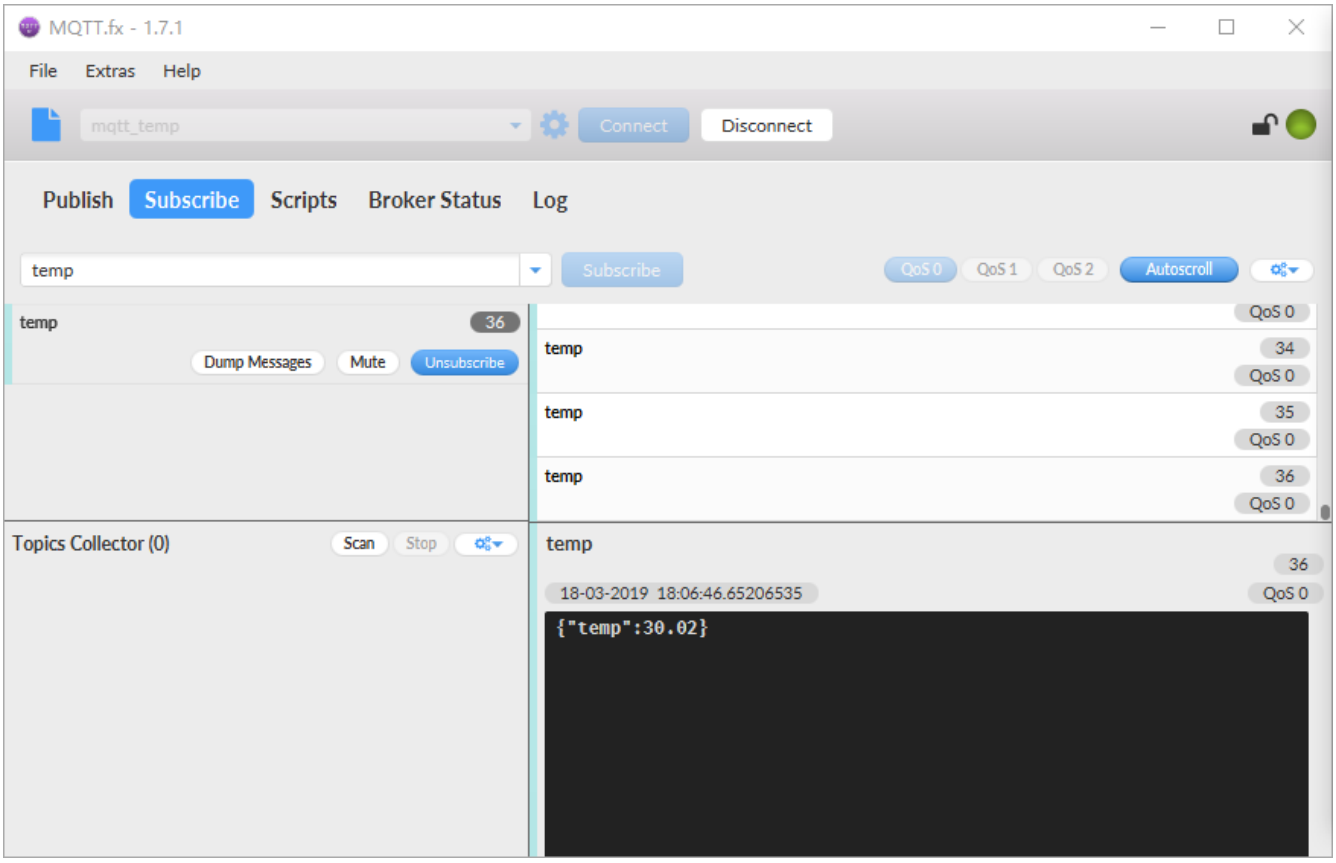


图 22-14 触发规则引擎

22.5 数据可视化

22.5.1 IoT Hub 的时序数据库

时序数据库（Time Series Database）是用于存储和管理时间序列数据的专业化数据库，为时间序列数据提供高性能读写和强计算能力的分布式云端数据库服务。时序数据库特别适用于物联网设备监控和互联网业务监控场景。

在物联网场景下，时序数据库有非常广泛的应用。如工业生产环境下，每个厂区有大量的监测点，如果以 10 秒的频率发送监测点的数据。1 万个监测点每年会产生 310 亿左右的数据点，时序数据库不仅可以轻松存储海量数据点，还可以对这些数据进行快速查询，而且 IoT Hub 提供的物可

视能让大量的数据通过图形化（仪表、折线图、柱形图等）实时显示，更加直观，由于规则引擎的设置，使得管理人员不用盯着海量数据找到异常数据；同时还能利用大数据工具分析数据，一般时序数据库的运作流程如下，具体见图 *22_15*。

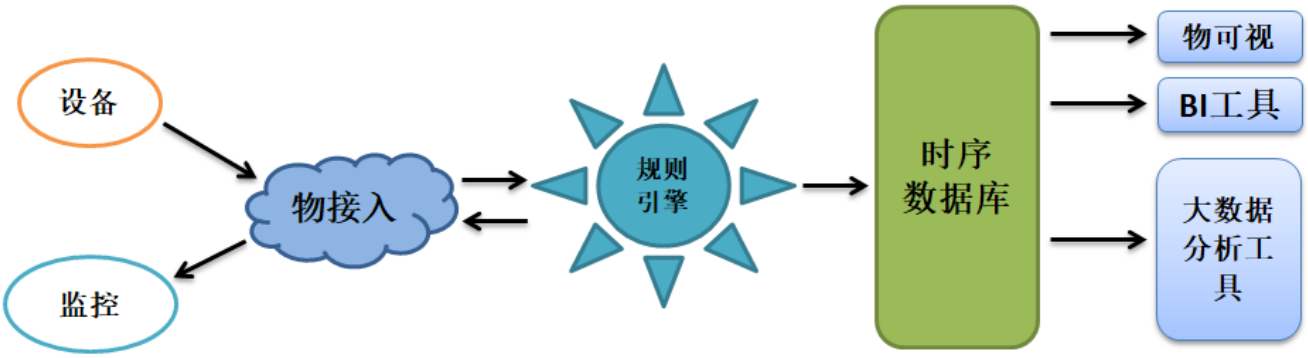


图 22-15 时序数据库

在使用顺序数据库之前，我们先创建一个时序数据库，打开连接：https://console.bce.baidu.com/iot2/tsdb/list，然后点击“创建数据库”，根据实际需求进行创建操作，注意，时序数据库是需要钱的，不过不贵，每个月 100 万点写入只需要 2 块钱，我们用来测试的早就足以。

从图 *22_15* 的流程我们也知道，开发板采集的温湿度数据想要写入时序数据库，必须通过规则引擎，而且还必须满足时序数据库的写入要求，因此我们需要先设置规则引擎，具体见图 *22_16*。

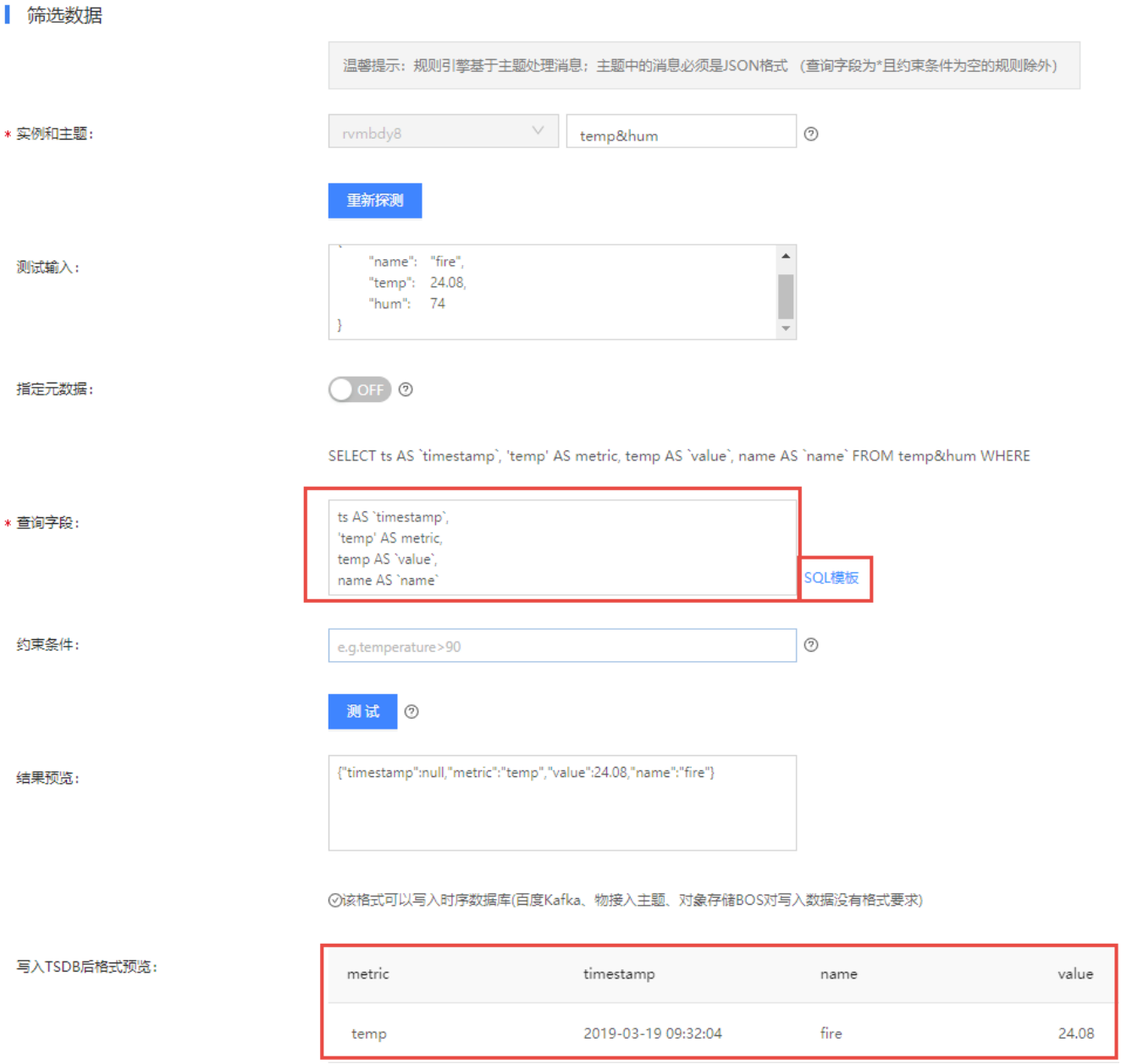


图 22-16 设置写入时序数据库的规则引擎

首先我们想要筛选温度数据，那么我们在查询字段中写入：

代码清单 22-4 时序数据库的规则引擎设置

```
ts AS `timestamp`,
'temp' AS metric,
temp AS `value`,
name AS `name`
```

这段代码表示将设备的温度数据写入到名为 temp 的 metric 里的默认域 value 里面去，时间戳为消息自带的 ts 字段（如果没有就是系统时间），把 name 的内容写入到 name 字段中，可以查看一下写入时序数据库后的格式预览，如果符合需求就可以写入，当然，这些规则是由我们自己定义的，大家可以随意设置，此外点击查询字段的 “SQL 模板”，会弹出提示，有很多规则的说明，大家自行参考。

同理对湿度的数据也是这样子写入时序数据库，在此就不再重复赘述。

22.5.2 IoT Hub 的物可视

操作这一步之前，必须将数据写入了时序数据库，然后通过物可视页面进行创建，打开：https://console.bce.baidu.com/iot2/viz2/#/workspaces，创建一个工作区，然后点击 “新建仪表盘”，拖动仪表盘到画布上，具体见图 *22_17*

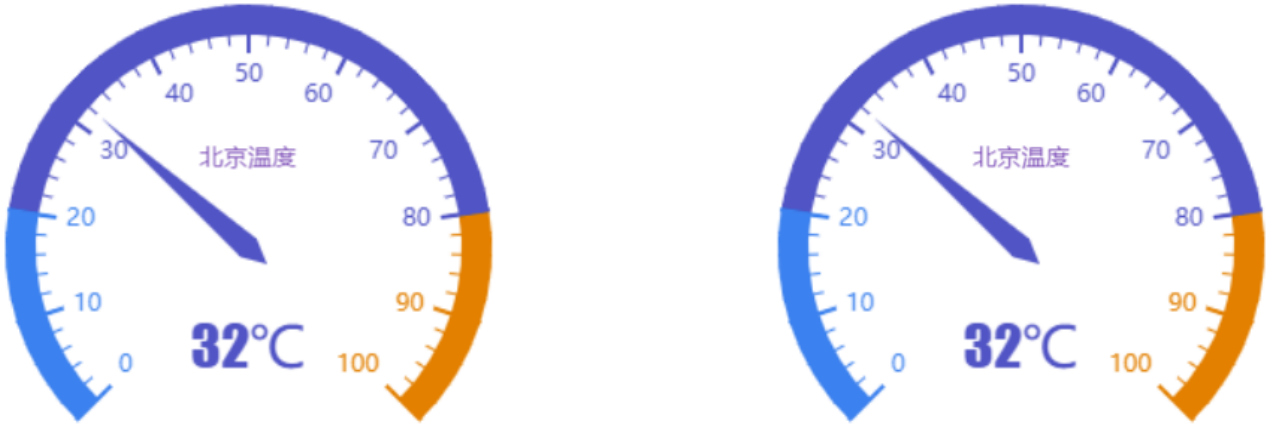


图 22-17 仪表盘

然后设置自定义的格式，关于设置显示的内容就不过多讲解，此处主要是讲解从时序数据库得到数据并且显示的过程。

首先在数据区域新建一个数据表，数据表的数据来源是时序数据库，具体见图 *22_18*。



图 22-18 新建数据表

然后在数据表中填写各项信息，度量 Metric 就是我们之前在写入时序数据库时候设置的度量，域 Field 就是我们写入数据库时候设置的温度值,，标签就是 name，然后勾选自动更新，更新时间为 5 秒即可，具体见图 *22_19*。



图 22-19 数据表信息完善

设置好数据表之后，就可以给仪表盘绑定数据源，首先在设计中点击仪表盘，选择数据设置，点击指标值的“绑定按钮”，选择其中一个数据表，然后进行绑定，当绑定成功，仪表盘的数据就会跟随数据表中的值动态变化，当然，数据表的数据更新是 5 秒钟，会稍有延迟，不过影响不大，具体见图 *22_20*。

同理对于湿度数据也是这样子的操作流程，设置规则引擎，将湿度数据写入时序数据库，创建数据表并且绑定，就不再赘述。

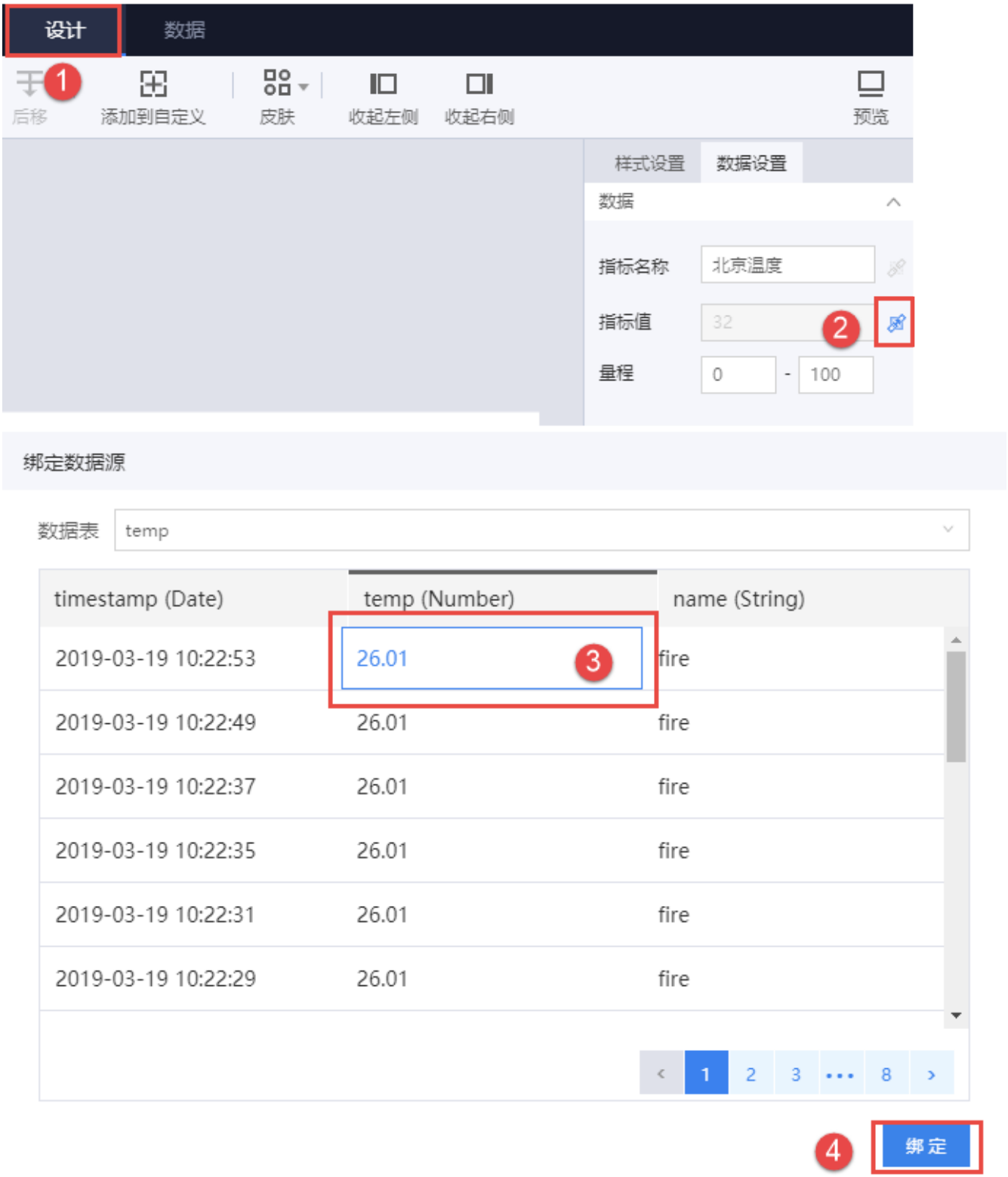

设计
数据
1
后移
添加到自定义
皮肤
收起左侧
收起右侧
预览
样式设置
数据设置
数据
指标名称
北京温度
指标值
32
2
量程
0
-
100
绑定数据源
数据表
temp
timestamp (Date)
temp (Number)
name (String)
2019-03-19 10:22:53
26.01
3
fire
2019-03-19 10:22:49
26.01
fire
2019-03-19 10:22:37
26.01
fire
2019-03-19 10:22:35
26.01
fire
2019-03-19 10:22:31
26.01
fire
2019-03-19 10:22:29
26.01
fire
‹
1
2
3
•••
8
›
4
绑定

图 22-20 绑定数据源

本次实验完成的效果图具体见图 22-21，当然，大家也可以通过连接来查看一下这个效果：https://viz.baidubce.com/prod/sharing/dashboard/b8dc632168679b05fe79c9a16f48942b。它是动态更新的，只要开发板将数据上报到云端，那么这些东西就会一直工作，只要通过浏览器都能随时随地查看数据信息。

提示：此处修改了仪表盘的名称为“野火办公室温/湿度”。

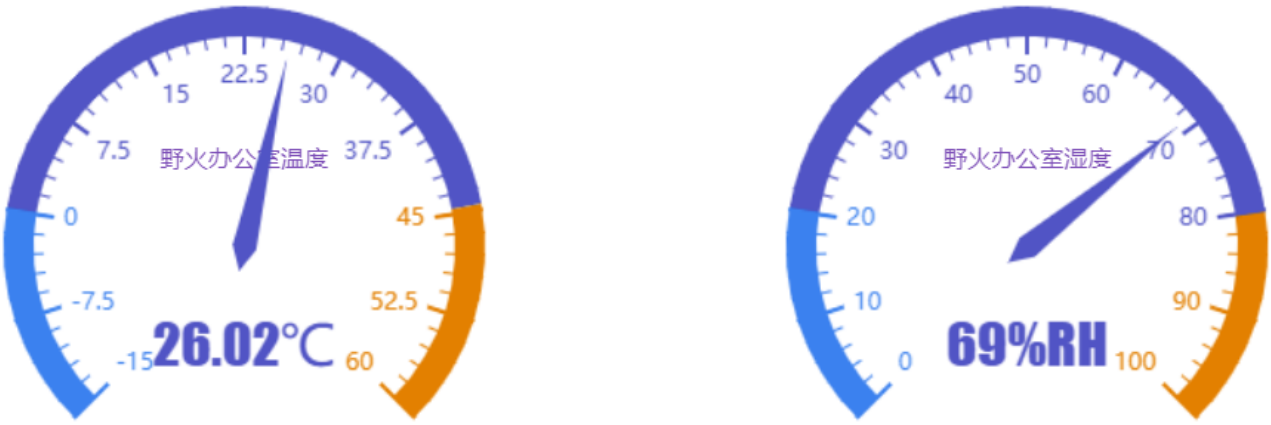


图 22-21 实验效果图

第 23 章　连接到阿里云物联

既然懂得专门连接百度天工物接入，那么连接阿里云物联其实也是一样的，因为都是基于 MQTT 协议进行通信的，首先打开阿里云物联：https://iot.console.aliyun.com/product，创建账号并登陆，而且阿里云物联是需要通过实名认证才能使用的。

相对于百度天工物接入，阿里云物联的安全性更好，因为即使是客户端连接服务器都需要各种验证，还使用哈希加密算法进行加密。

23.1 使用阿里云物联

下面开始使用阿里云物联，首先在“产品”信息中创建一个产品，每个产品下允许有多个设备，产品就是设备的集合，通常是一组具有相同功能定义的设备集合。例如：产品指同一个型号的产品，设备就是该型号下的某个设备。操作步骤“创建产品”-> “高级产品”-> 填写产品信息，联网方式选择“以太网”，这样子一个产品就创建完成。

然后在“设备”选项中添加一个设备，比如我们添加一个“fire_temp_hum”设备，是开发板的温湿度数据采集设备，在创建完成后，会有设备证书，这个一定要保存好，丢失了就找不回来了，具体见图 *23_1*。



图 23-1 设备证书

最后我们定义一个主题（Topic 类列表），让设备能对这个主题进行订阅或者发布操作，回到“产品”选项，选择“Topic 类列表”，定义“Topic 类列表”，具体见图 *23_2*，再填写“Topic 类列表”的信息即可，注意选择设备的操作权限“发布和订阅”，这点很重要，如果没有权限，设备是无法对这个主题进行操作的，具体见图 *23_3*，至此，一个产品与设备就创建完成了，当然，为了更好调试，我们可以尝试创建多个设备，以备在后续进行调试。



图 23-2 Topic 类列表



图 23-3 Topic 类列表信息

23.2 MQTT 软件测试连接

在创建完成后，可以通过 MQTT 软件来测试一下能否正常连接，在这一步之前必须已在物联网平台控制台中创建产品和设备，并获取设备证书信息（ProductKey、DeviceName 和 DeviceSerect）。

打开软件，点击设置图标，配置连接的信息，具体见图 *23_4*，只不过这里的配置比百度云天工物接入麻烦很多，因为考虑了安全的问题，需要进行加密验证。

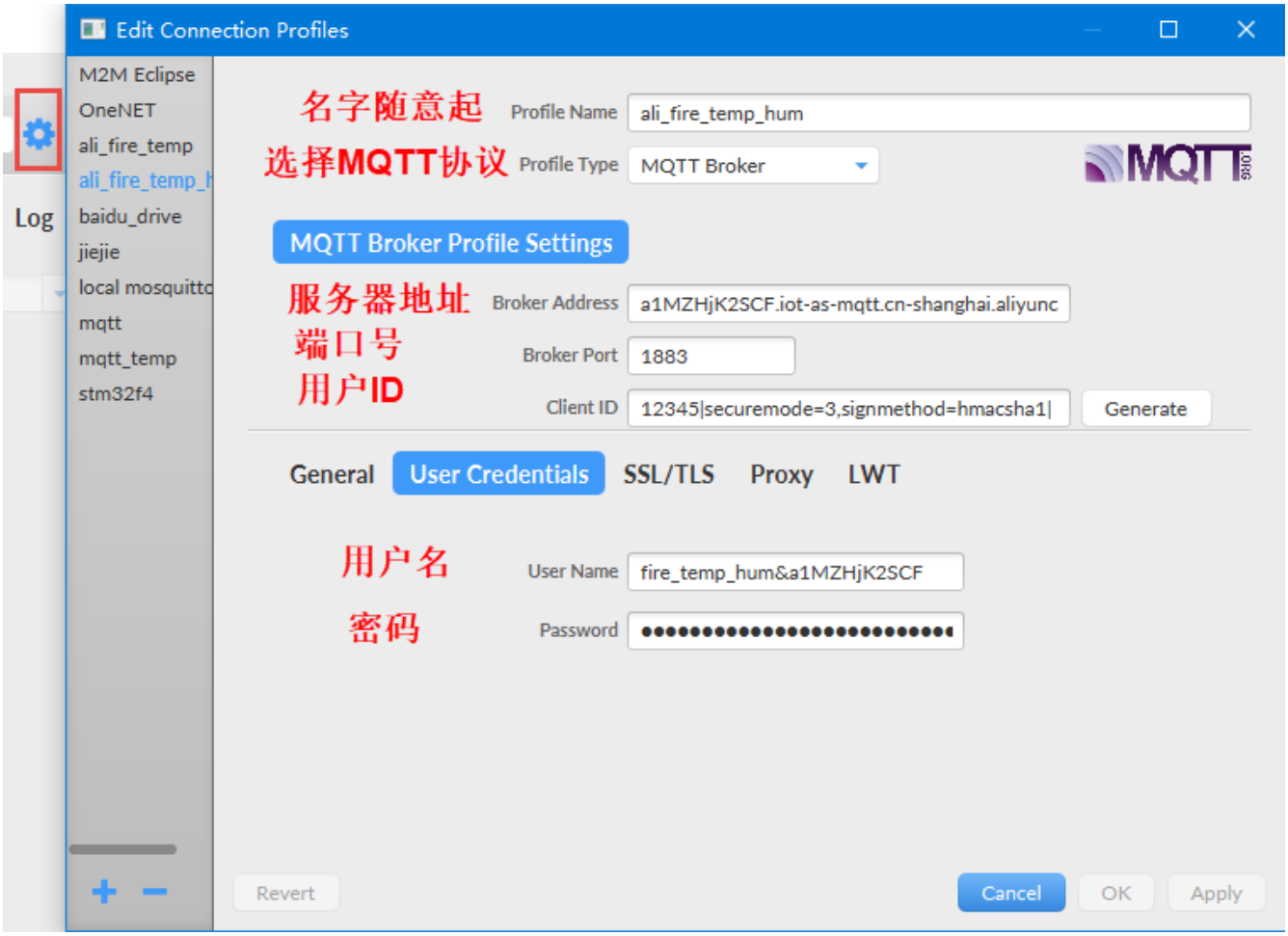


图 23-4MQTT 软件配置

这个参数看起来很简单，但是它的组成却不简单，下面具体介绍一下参数是怎么来的，具体见表格 23-1。

表格 23-1 连接的参数设置

参数	说明
Profile Name	自定义名称
Profile Type	选择 MQTT Broker
Broker Address	连接域名。 格式：${YourProductKey}.iot-as-mqtt. ${region}.aliyuncs.com。 其中： ${YourProductKey} 是在创建设备时候保存的 YourP roductKey ${region} 是你物联网平台服务所在地域的代码，一般在控制台左上角就有显示，根据 https://help.aliyun.com/ document_detail/40654.html? spm=a2c4g.11186623.2.21.1ce29ff5q TtXmW 替换物联网平台服务所在地域的代码即可
Broker Port	因为是 TCP 连接，端口号为 1883
Client ID	格式：${clientId}securemode=3,signm ethod=hmacsha1。 ${clientId} 为设备的 ID 信息。可取任意值，长度在 64 字符以内即可， securemode 为安全模式，TCP 直连模式设置为 securemode=3 ，TLS 直连为 securemode=2， signmethod 为算法类型，支持 hmacmd5 和 hmacsha 1，常用的是哈希加密。
User Name	格式：${YourDeviceName}&${YourPrduct Key} 由设备名 DeviceName、符号（&）和产品 ProductKey 组成，这些信息在创建设备的时候都已经保存的。
Password	密码由参数值拼接加密而成，可以使用 Password 生成小工具生成。 小工具可以从我们提供的资料中下载： http://www.firebbs.cn/forum.php? mod=viewthread&tid=26274&fromuid= 37393，小工具的使用具体见图 *23_5* ，填写对应的信息，然后生成 password，就可以用来连接了，注意的是 clientId 是填写前面自己定义的 clientId 即可。

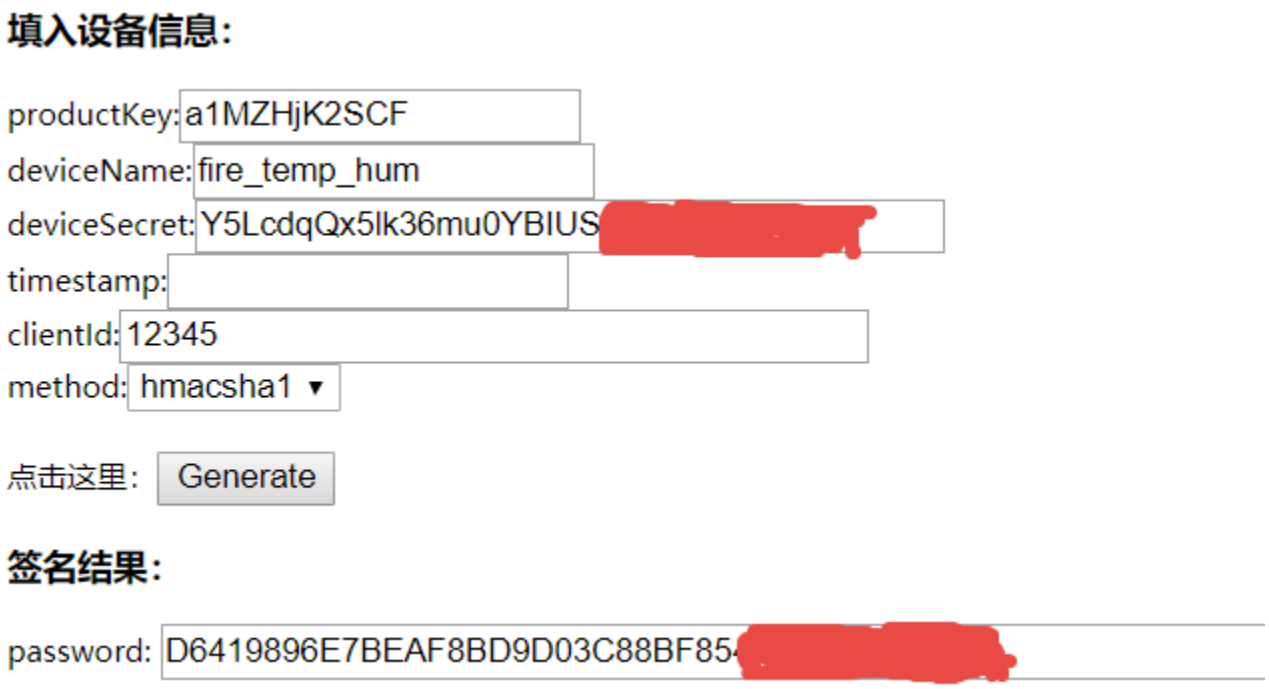


图 23-5Password 生成小工具

把信息全部填写完毕，可以测试连接，点击 Connect，然后在订阅选项 Subscribe 中，输入订阅的主题“/a1MZHjK2SCF/fire_temp_hum/user/temp_hum”，这个主题可以在“设备”选项中的“Topic 列表”得到，然后点击 subscribe，可以发现已经成功订阅，然后对这个主题进行发布操作，我们就能看到订阅已经收到发布的消息，表明 MQTT 通信成功，具体见图 *23_6*。

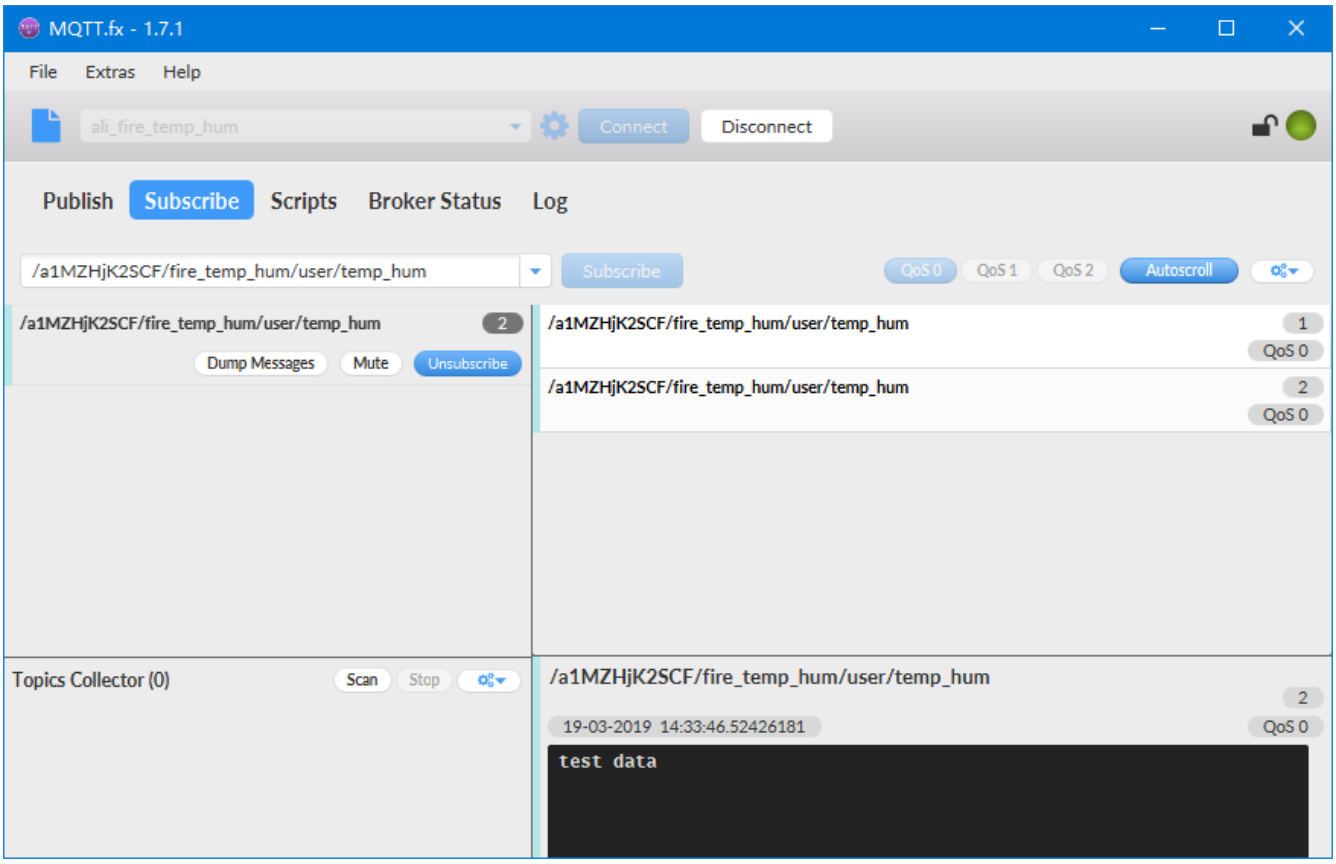


图 23-6 测试成功

与此同时，在阿里云物联的控制台中可以看到设备是处于在线状态的，这就说明我们的连接是正常的，具体见图 *23_7*。

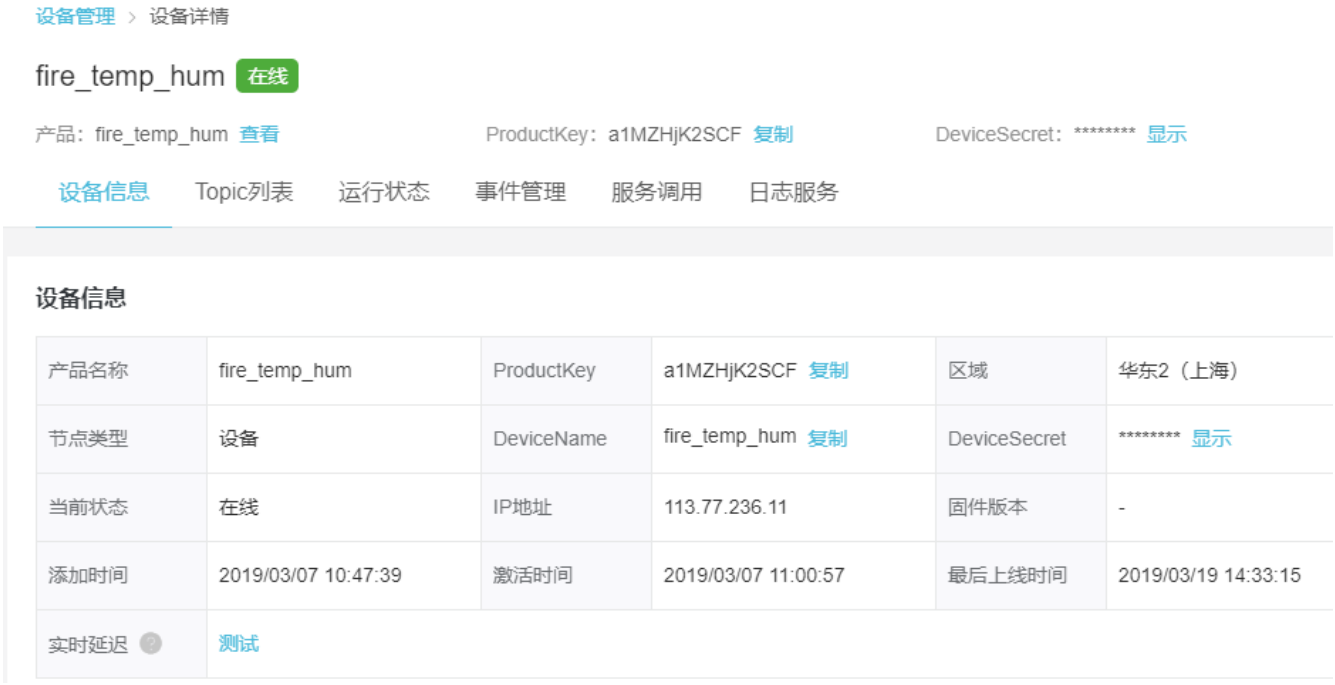


图 23-7 设备状态

23.3 开发板连接阿里云物联

接下来就使用开发板进行连接阿里云物联，其实在连接百度天工物接入的时候，我们就已经封装好一整套 MQTT 连接云平台的函数接口，现在只需要修改一下 mqttclient.h 头文件的宏定义即可，具体见*代码清单 23_1*。

代码清单 23-1mqttclient.h 宏定义

```
#if    LWIP_DNS
#define   HOST_NAME      \
    "a1MZHjK2SCF.iot-as-mqtt.cn-shanghai.aliyuncs.com"  //服务器域名
#else
#define   HOST_NAME       "139.196.135.135"      //服务器 IP 地址
#endif
```

(下页继续)

(续上页)

```
#define   HOST_PORT      1883    //由于是 TCP 连接，端口必须是 1883

#define   CLIENT_ID      "12345|securemode=3,signmethod=hmacsha1|"          /
↪/
#define   USER_NAME      "fire_temp_hum&a1MZHjK2SCF"      //用户名
#define   PASSWORD       "D6419896E7BEAF8BD9D03C88BF854EA0D3XXXXX"  //密码

#define   TOPIC          "/a1MZHjK2SCF/fire_temp_hum/user/temp_hum" //主题
#define   TEST_MESSAGE   "test_message"  //发送测试消息
```

将程序编译并且下载到开发板上，就能看到连接的实验现象，整个代码的运作过程也是一样的，因为都是 MQTT 协议通信，首先与服务器建立连接，然后通过 MQTT 报文请求建立会话，然后进行订阅主题，再发布消息，当收到消息的时候使用 cJSON 将消息解析出来，具体见图 *23_8*。

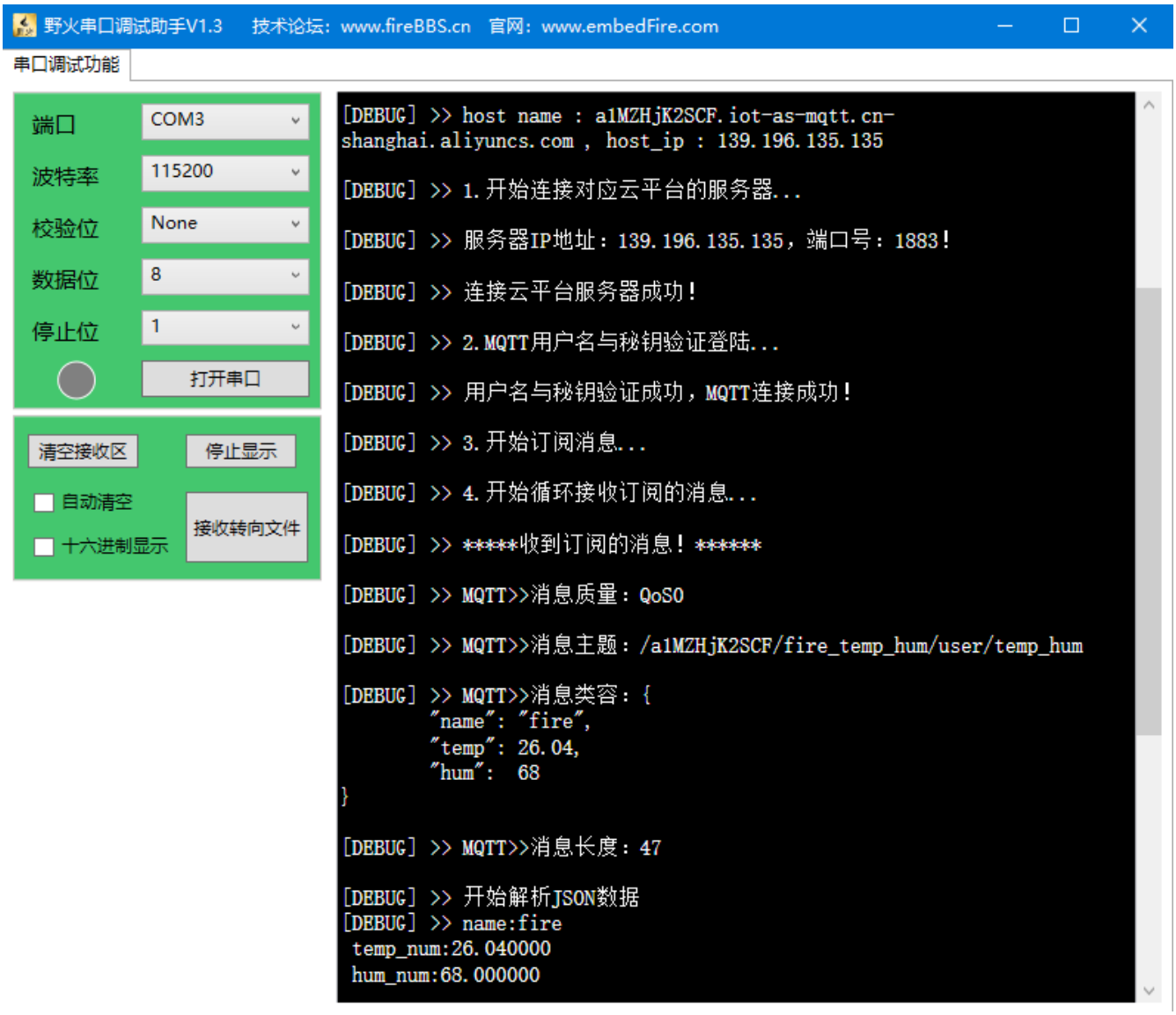


图 23-8 实验现象

23.4 阿里云物联的规则引擎

与百度的规则引擎作用是一样的，可以将收到的消息进行不同的业务处理，比如我们也利用规则引擎监控开发板的稳定数据。

首先，打开规则引擎设置页面：https://iot.console.aliyun.com/rule/list，点击创建一个规则，填写基

本的规则信息，并且选择数据的格式是 JSON 格式，具体见图 *23_9*。

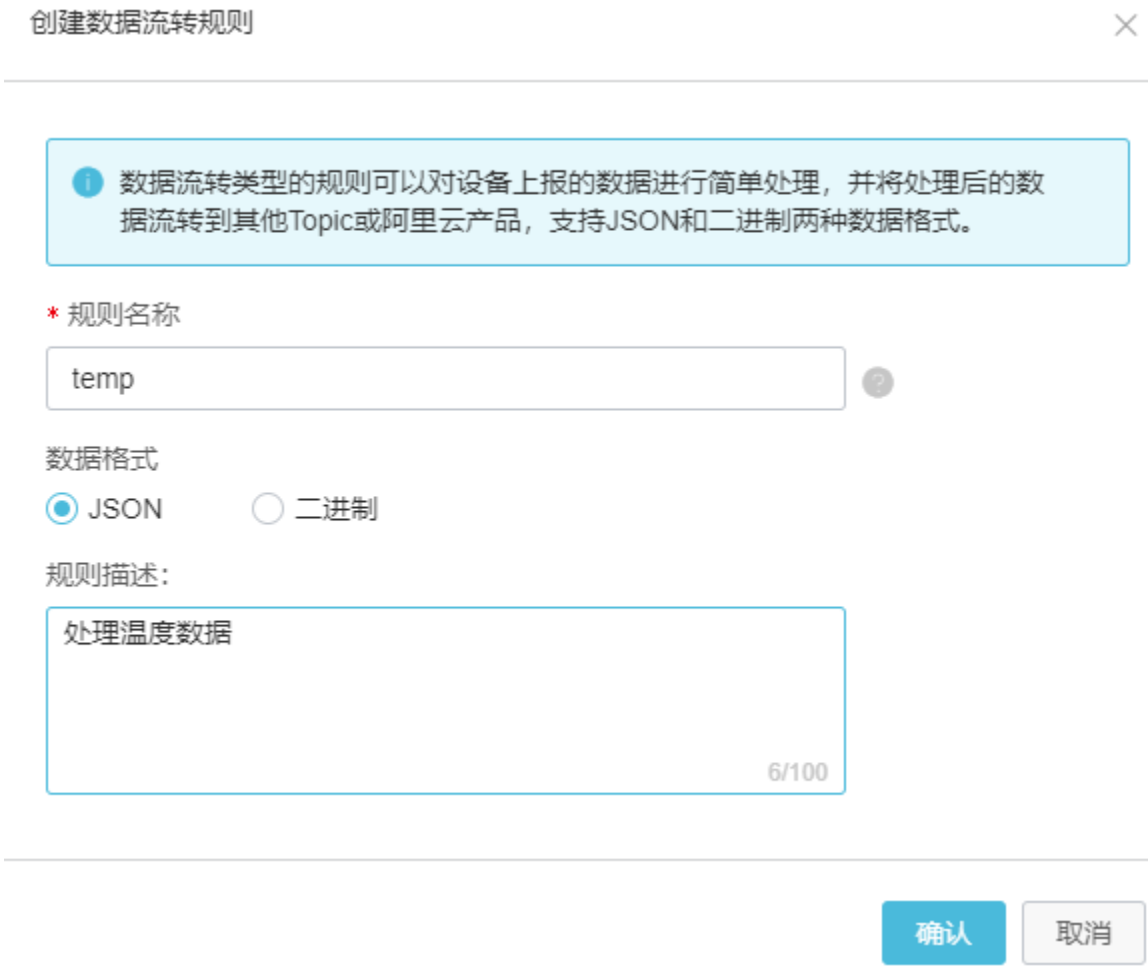


图 23-9 创建规则

然后编写 SQL 语句处理数据，关于 SQL 语句的语法可以参考网页：https://help.aliyun.com/document_detail/30554.html?spm=5176.11485173.0.0.5af859afrAuwd0。此处就不做过多赘述，当然，阿里云物联这个平台还是很不错的，只需要我们填写少量的规则语句就能筛选出需要的数据，具体见图 *23_10*。



图 23-10 设置规则

配置好规则我们就先进行测试一下规则是否可用，点击“SQL 调试”，输入我们的 JSON 数据，然后通过其输出验证，具体见图 *23_11*，表明我们的规则是能正常运作的。

代码清单 23-2 验证数据

```
{
    "name": "fire",
    "temp": 32.04,
    "hum":  68
}
```



图 23-11SQL 测试

接下来“转发数据”选项中添加数据目的地，数据目的地可以是另一个主题、时序数据库、存储到云数据库 (RDS) 中、发送数据到消息队列 (Message Queue) 中等等，我们就将数据转发到同产品下的另一个主题即可，这就需要我们创建多一个设备（监控设备），具体见图 *23_12*。

注意，在创建规则完成后一定要记得启动规则，因为阿里云物联的规则引擎默认是不启动的，需要我们自己去启动它。

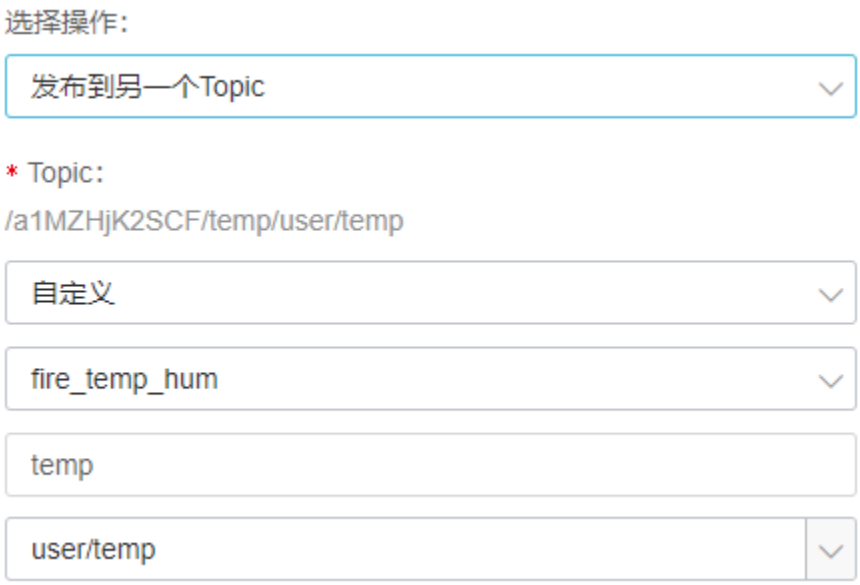


图 23-12 数据目的地

打开我们的 MQTT 客户端软件，并且使用新的设备进行连接（注意：不要是与开发板是同一个设备），然后订阅规则引擎转发的目的地主题；打开开发板电源，开发板连接到阿里云物平台后就会发送数据，如果满足规则就会将数据转发到目的地主题中，具体见图 *23_13*。

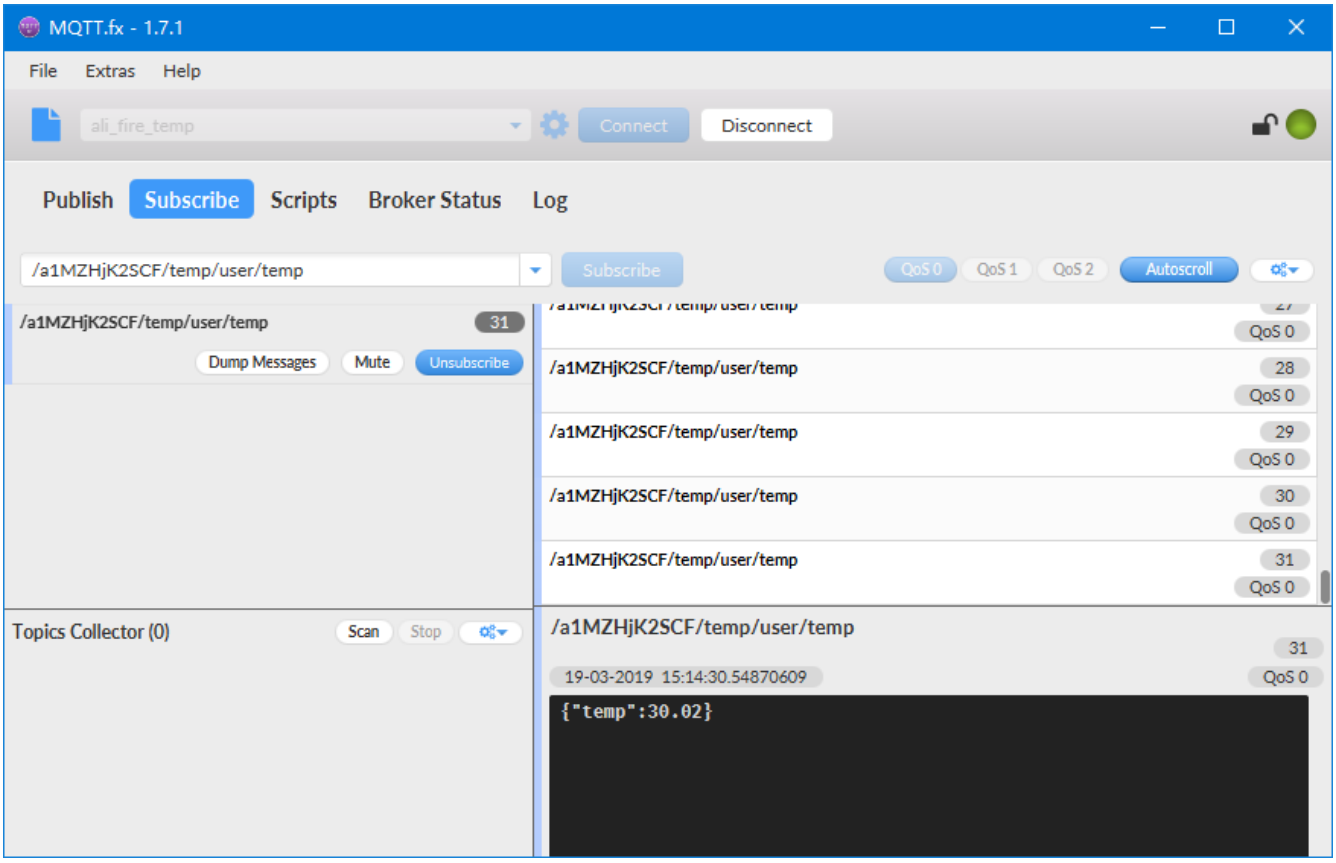

MQTT.fx - 1.7.1
File
Extras
Help
ali_fire_temp
Connect
Disconnect
Publish
Subscribe
Scripts
Broker Status
Log
/a1MZHjK2SCF/temp/user/temp
Subscribe
QoS 0
QoS 1
QoS 2
Autoscroll
/a1MZHjK2SCF/temp/user/temp
31
Dump Messages
Mute
Unsubscribe
/a1MZHjK2SCF/temp/user/temp
28
/a1MZHjK2SCF/temp/user/temp
29
/a1MZHjK2SCF/temp/user/temp
30
/a1MZHjK2SCF/temp/user/temp
31
Topics Collector (0)
Scan
Stop
/a1MZHjK2SCF/temp/user/temp
19-03-2019 15:14:30.54870609
{"temp":30.02}

图 23-13 实验现象

第 24 章　连接到 OneNET

有了前面两个实验的经验，废话不多说，直接使用 OneNET。

24.1 使用 OneNET

首先注册与登陆 OneNET，然后进入开发者中心：https://open.iot.10086.cn/develop/global/product/#/public，选择公有协议产品，点击“添加产品”，填写产品相关的信息，联网方式选择 wifi（其实我们是以太网，但是没有这个选项，那就选择 wifi，没啥影响的），接入协议必须选择 MQTT，操作系统选择无，运营商这个随意选择，具体见图 *24_1*。


添加产品
产品信息
* 产品名称:
fire_temp_hum
* 产品行业:
智能家居
* 产品类别:
家用电器
生活电器
空气监测仪
产品简介:
温湿度
技术参数
* 联网方式:
wifi
移动蜂窝网络
NB-IoT
* 设备接入协议:
MQTT
* 操作系统:
Linux
Android
VxWorks
μC/OS
无
* 网络运营商:
移动
电信
联通
其他

图 24-1 添加产品

在添加产品完毕，继续添加设备，填写设备的相关信息，鉴权信息就是登陆密码，不过需要注意的是，这个鉴权信息在产品内是唯一的，一般推荐使用产品序列号，可作为设备登录参数之一，不同协议设备可能鉴权信息的参数不一致，不过现在是做实验，选择一个好记的即可，数据保密性要选择私有，除此之外还剩下一些设备相关的信息，就不过多赘述，具体见图 *24_2*，为了测试，我们在这里就创建 2 个设备，大家可以自行创建更多的设备用于测试也是可以的。

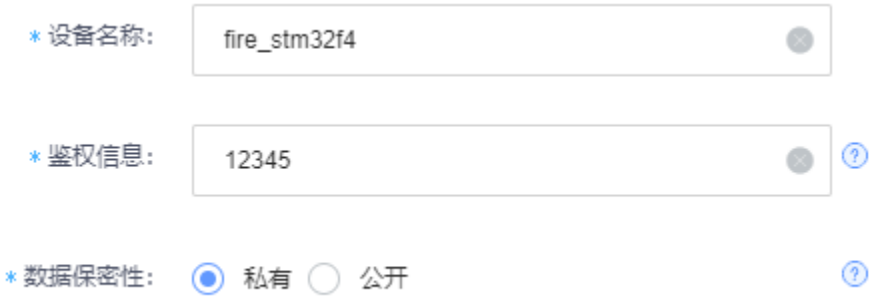


图 24-2 创建设备信息

24.2 测试连接

OneNET 的数据交互做的很好，它支持动态创建主题（除系统主题外），即不用我们在平台上创建任何的主题，只需要随意订阅某个主题即可，同一个产品下，即使是不同设备之间的主题之间的消息是共享的，简单来说，我们在开发板上可以随意向某个主题发起订阅请求，也可以向这个主题发布消息，而同一产品的其他设备如果订阅了这个主题，那么将收到开发板发布的消息数据，这样子更加方便嵌入式的开发者，只不过这样子的信息安全就没有阿里云物联那么好。

现在我们打开 MQTT 软件，进行连接测试，与前面的实验操作是一样的，配置好相关信息，即可，这些信息都可以在平台上找到，需要注意的是服务器地址是 183.230.40.39；端口号是 6002，这与我们常见的 1883 是不一样的；Client ID 是设备 ID，在设备列表中是可以找到的；用户名是产品 ID，在产品列表中可以找到；密码就是创建设备时候的鉴权信息，具体见图 *24_3*。

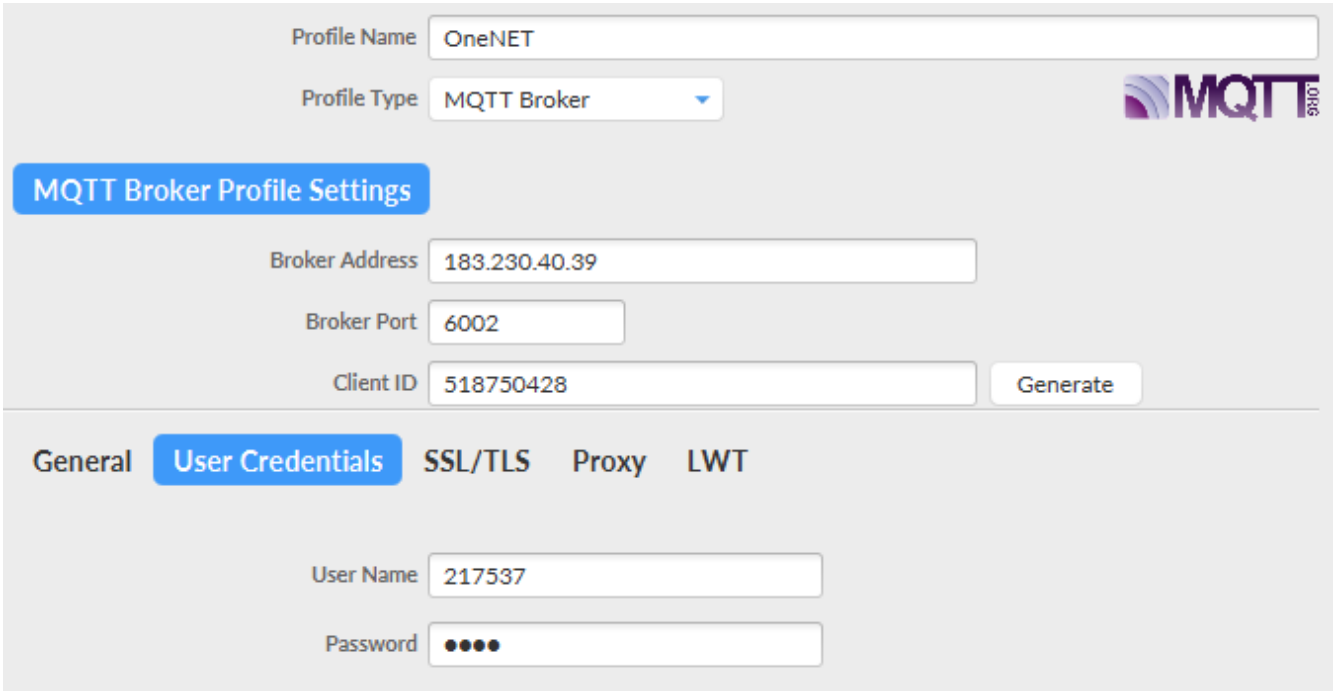


图 24-3MQTT 软件配置

配置好就可以连接，然后随便订阅一个主题，再用客户端进行发布消息，如果收到消息，表明通信成功，具体见图 *24_4*。

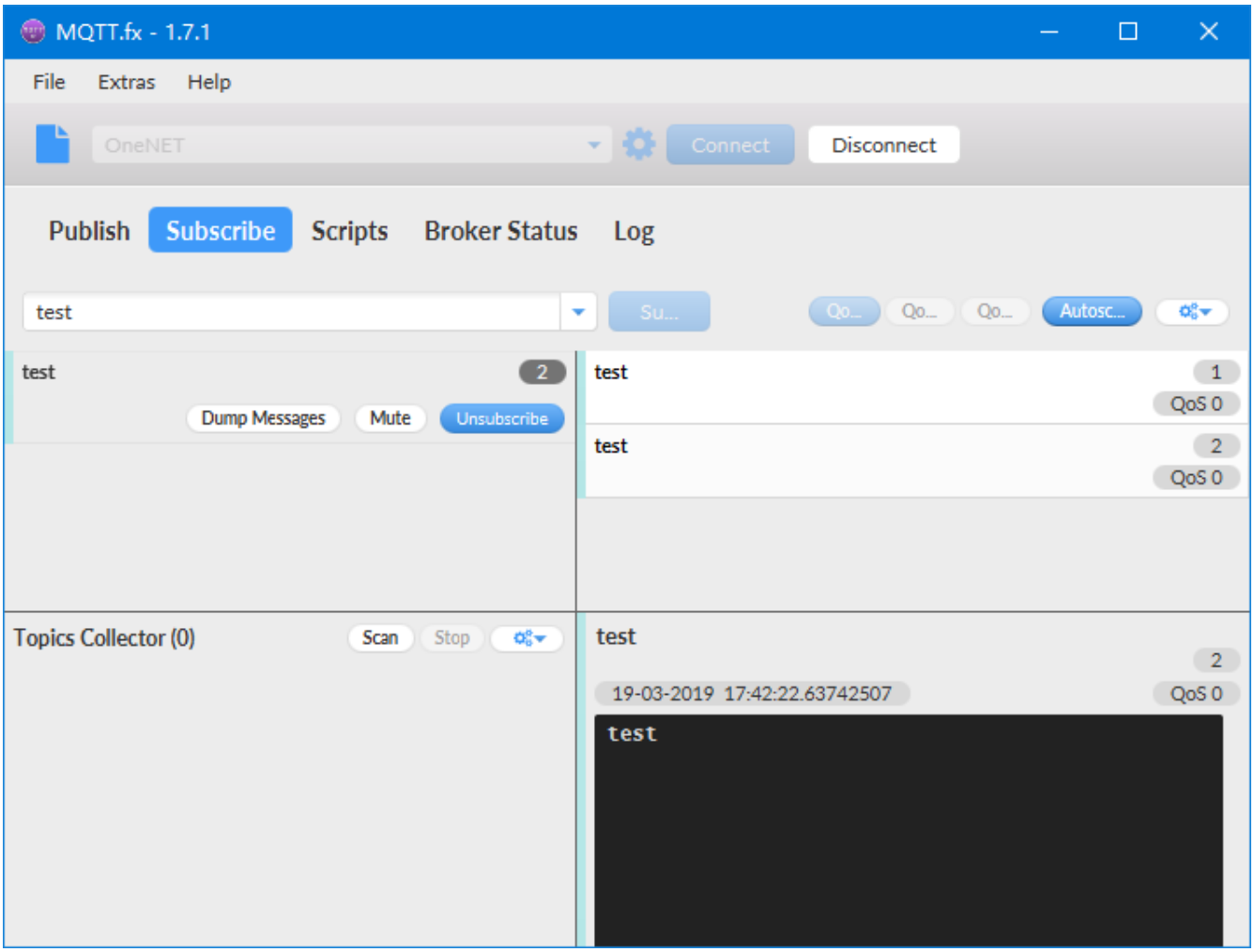


图 24-4 测试连接结果

24.3 开发板连接 OneNET

在开发板中，我们将连接阿里云物联的代码拷贝一份，然后修改一下 mqttclient.h 头文件的宏定义即可，具体见代码清单 24-1。

代码清单 24-1mqttclient.h 头文件的宏定义

```
#if    LWIP_DNS
#define   HOST_NAME       "mqtt.heclouds.com"     //服务器域名
#else
#define   HOST_NAME       "183.230.40.39"     //服务器 IP 地址
#endif


#define   HOST_PORT     6002

#define   CLIENT_ID     "518725049"          //
#define   USER_NAME     "217537"     //用户名
#define   PASSWORD      "12345"  //秘钥

#define   TOPIC         "temp_hum"      //订阅的主题
#define   TEST_MESSAGE  "test_message"  //发送测试消息
```

下载到开发板上，就能连接到 OneNET 了，因为都是使用 MQTT 协议，因此连接的过程都是一样的，而且数据的订阅与发布也是一样的，编译好程序，然后下载到开发板，就能看到发布数据与接收数据了，具体见图 *24_5*。

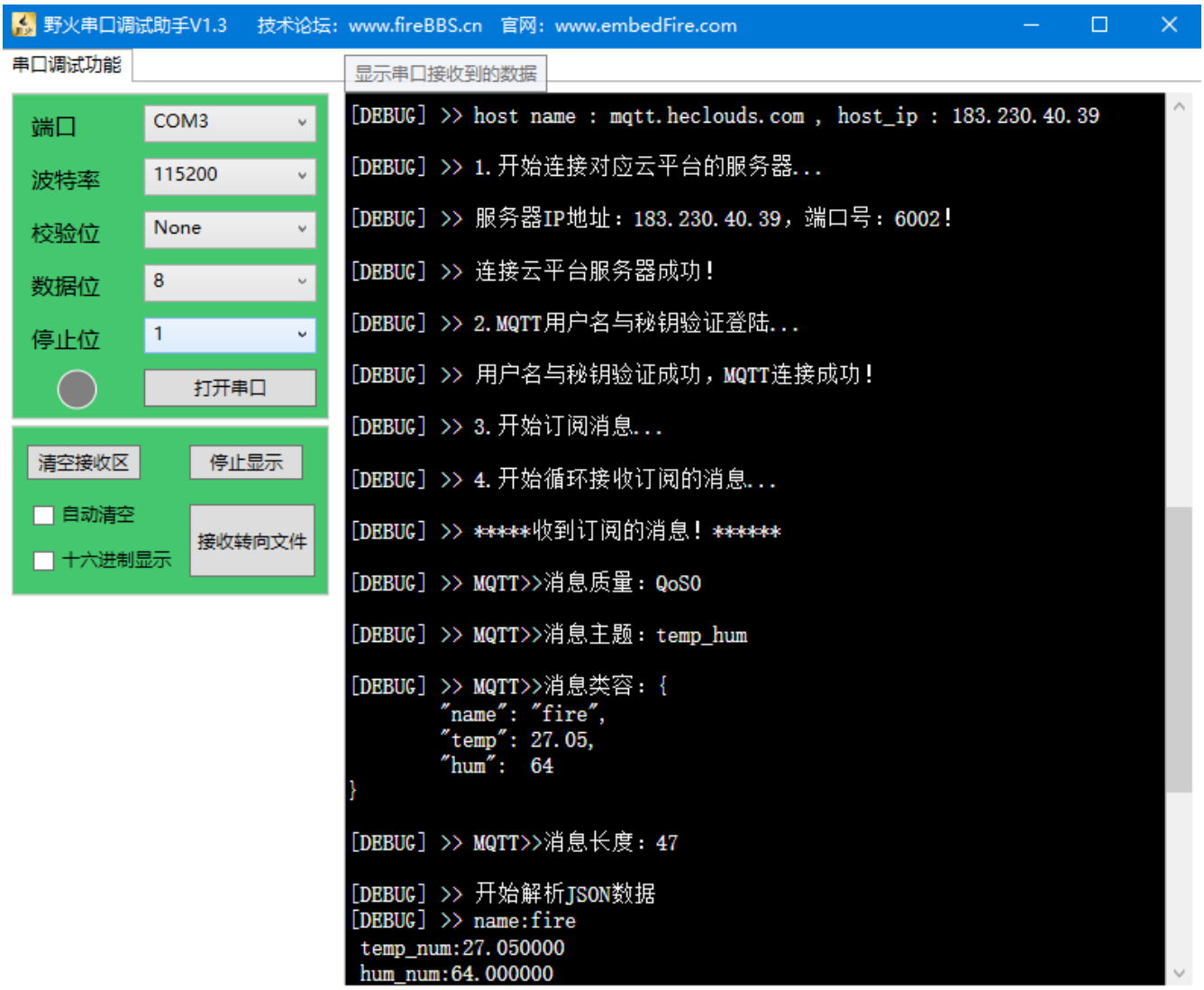


图 24-5 实验现象

此外，我们再使用 MQTT 软件来订阅一下开发板发布的主题“temp_hum”，很显然，软件收到了开发板发布的消息，说明正如我们所了解一样，通信正常，具体见图 *24_6*。

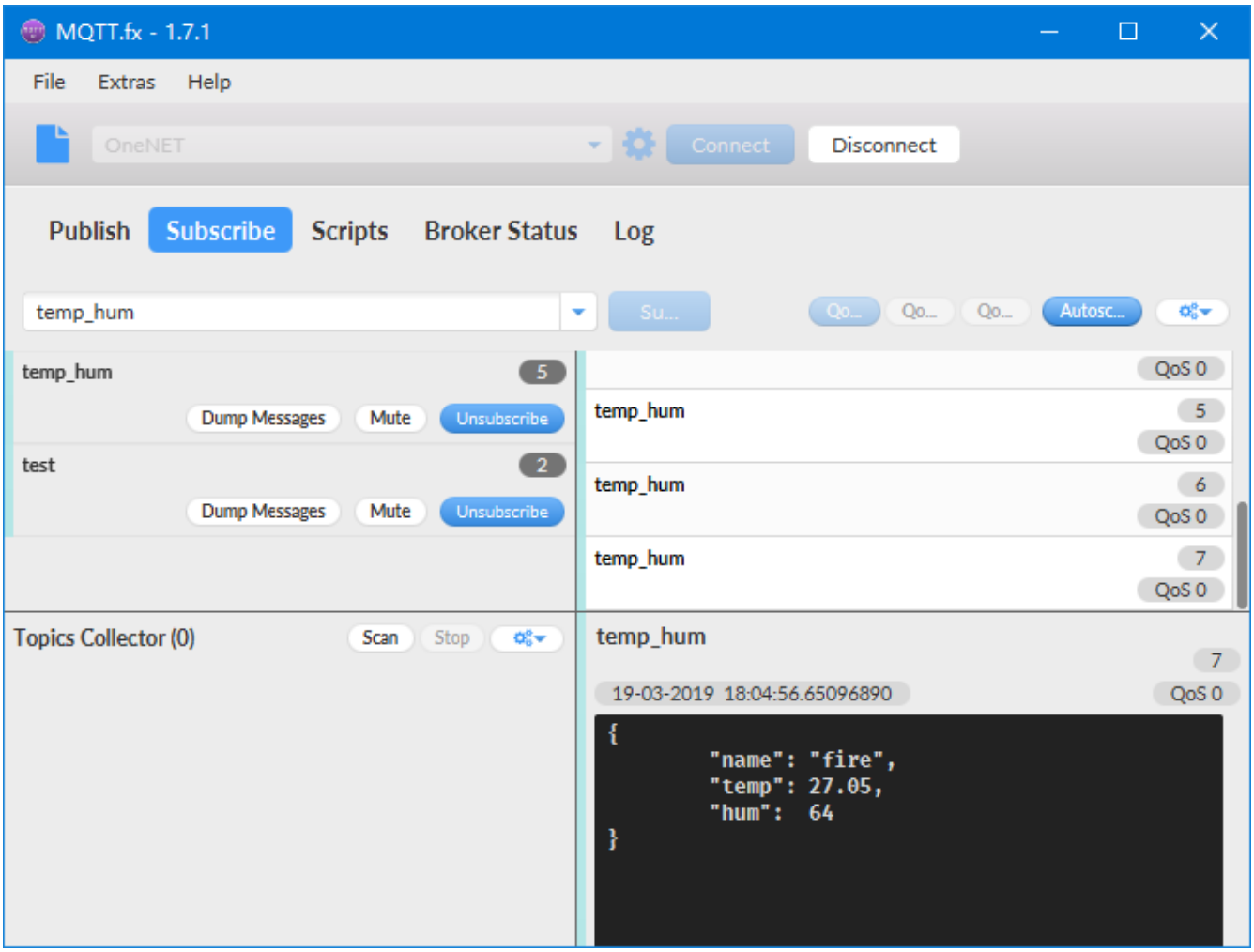


图 24-6MQTT 软件订阅 temp_hum 主题

24.4 添加数据流

OneNET 平台通过数据流与数据点来组织设备上行数据，设备上传并存储数据时，必须以 key-value 的格式上传数据，其中 key 即为数据流（stream）名称，value 为实际存储的数据点（point），value 格式可以为 int、float、string、json 等多种自定义格式，我们本书就教大家向 OneNET 平台上报数据点，而平台会将所有的数据点按时序进行存储，这就形成了数据流，数据流中的数据在存储的同时可以“流向”后续服务，数据流是平台后续数据服务（规则、触发器、消息队列等）的服务对象，用户可以通过选择数据流的方式选择服务的数据来源，这与我们之前的规则引擎差

不多，只不过换了个说法而已。

首先在 OneNET 控制台中添加两个数据流，分别为 temp 与 hum，表示温度与湿度的数据流，打开：https://open.iot.10086.cn/develop/global/product/#/datasm?pid=217537，点击“添加数据流模板”，填写相关信息，数据流名称填写你发布数据的 JSON 格式的字符串，类似于规则引擎的筛选，具体见图 *24_7*，添加 temp 完成后再添加一个 hum 数据流模板。



图 24-7 添加数据流模板

设备是使用 publish 报文来上传数据点的，而且上报数据点的格式也是有要求的，对于普通的主题，发布消息数据根本没有任何格式要求，随意发布数据都可以，但是对于发布数据点等与平台数据相关的操作，则必须通过系统主题进行发布，发布数据点的主题是“$dp”，“$”符号表示系统的主题，所有的数据都要往这个主题发布，否则系统是不会存储数据的，大家在创建数据流之后，打开开发板，向 OneNET 平台上报数据（非“$dp”主题），然后在后台可以看到并无数据记录，具体见图 *24_8*。

如果想要发送数据并让平台存储，形成数据流，将数据应用到可视化上或者其他服务，那么就必须向“$dp”主题发送数据，而向系统主题发送数据就会有格式要求，目前 OneNET 支持 7 种格式的发布，我在这里只讲解第三种格式的数据上报，因为这与我们的前面实验的数据最为相似，而其他的数据格式大家可以参考一下官方的接入资料，可以到论坛里面获取：http://www.firebbs.cn/forum.php?mod=viewthread&tid=26274&fromuid=37393。



图 24-8 无数据

24.5 系统主题的发布格式

从 MQTT 发布报文（PUBLISH）的格式我们可以知道，主题名必须是 PUBLISH 报文可变报头的第一个字段，而系统主题名就是 “$dp”，只有当 QoS 等级是 1 或 2 时，报文标识符（Packet Identifier）字段才能出现在 PUBLISH 报文中，那么可变报头的结构具体见图 *24_9*。

bit	说明	7	6	5	4	3	2	1	0
Byte1	长度 MSB (0)	0	0	0	0	0	0	0	0
Byte2	长度 LSB (3)	0	0	0	0	0	0	1	1
Byte3	'$'	0	0	1	0	0	1	0	0
Byte4	'd'	0	1	1	0	0	1	0	0
Byte5	'p'	0	1	1	1	0	0	0	0
Byte6	报文标识符	xxxx							
Byte7	报文标识符	xxxx							

图 24-9PUBLISH 报文可变报头

有效载荷（Payload）则包含真正的数据点内容，本书只讲解第三种类型的数据格式，具体见图 *24_10*，第 1 个字节指明了发布报文的数据类型，OneNET 平台支持 7 种数据类型，第三种是 JSON 格式，因此数据字段就需要是 JSON 格式的数据，否则就会发生错误；第 2 第 3 字节指明了后面携带的数据长度，第二字节是数据长度的高字节位，第 3 字节是数据长度的低字节位；而第 4 字节之后就是真正的数据区域，使用 JSON 数据即可。

bit	说明	7	6	5	4	3	2	1	0
Byte1	数据类型(3)	0	0	0	0	0	0	1	1
Byte2	长度 MSB	0	0	0	0	0	0	0	0
Byte3	长度 LSB (0x33)	0	0	1	1	0	0	1	1

Byte4	通用JSON格式：
	{
	"name": "fire",
	"temp": 25.09,
	"hum": 74
	}
		XXXX							

....		....							
Byte n		XXXX							

图 24-10 Payload 内容

24.6 使用开发板发布数据点

因为是有格式的要求，我们就稍微改一下代码的接口，在数据之前添加类型、长度等字段，因为我们的代码也是自己封装好的，所以想修改也是比较简单，而 MQTT 层的代码也无需改动，因为是发布数据点，因此我们实现一个 MQTTMsgPublish2dp() 函数用于发布到“$dp”主题中，具体见*代码清单 24_2*。

代码清单 24-2 MQTTMsgPublish2dp() 源码

```
/**********************************************************************************
** 函数名称: MQTTMsgPublish2dp
** 函数功能: 用户推送消息到'$dp'系统主题
** 入口参数: MQTT_USER_MSG  *msg: 消息结构体指针
** 出口参数: >=0: 发送成功 <0: 发送失败
** 备    注:
```

(下页继续)

(续上页)

```
*********************************************************************/
int32_t MQTTMsgPublish2dp(int32_t sock, int8_t qos, int8_t type,uint8_t*␣
↪msg)
{
    int32_t ret;
    uint16_t msg_len = 0;
    msg_len = strlen((char *)msg);
    uint8_t* q = pvPortMalloc(msg_len+3); //目前只支持 1、3、4 类型的 json 数据
    switch (type)
    {
    case TopicType1:
        *(uint8_t*)&q[0] = 0x01;
        break;
    case TopicType3:
        *(uint8_t*)&q[0] = 0x03;
        break;
    case TopicType5:
        *(uint8_t*)&q[0] = 0x05;
        break;
    default:
        goto publish2dpfail;
    }
    *(uint8_t*)&q[0] = 0x03;
    *(uint8_t*)&q[1] = ((msg_len)&0xff00)>>8;
    *(uint8_t*)&q[2] = (msg_len)&0xff;
    memcpy((uint8_t*)(&q[3]),(uint8_t*)msg,msg_len);

    //发布消息
    ret = MQTTMsgPublish(MQTT_Socket,(char*)"$dp",qos,(uint8_t*)q,msg_
↪len+3);

publish2dpfail:
```

(下页继续)

(续上页)

```
    vPortFree(q);
    q = NULL;
    return ret;
}
```

而为了配合 MQTTMsgPublish2dp() 函数，我们还需要把 MQTTMsgPublish() 函数也稍作修改，让其传入指定的数据长度，因为 strlen() 函数会在遇到 ASCII 码‘00’的时候认为数据已经没有了，而有效载荷的数据长度高字节位很有可能是 0，因此不使用自动统计数据，只传入指定数据长度，具体见*代码清单 24_3*。

代码清单 24-3 MQTTMsgPublish() 源码

```
/*********************************************************
** 函数名称: mqtt_msg_publish
** 函数功能: 用户推送消息
** 入口参数: MQTT_USER_MSG  *msg: 消息结构体指针
** 出口参数: >=0: 发送成功 <0: 发送失败
** 备    注:
*********************************************************/
int32_t MQTTMsgPublish(int32_t sock,
                       char *topic,
                       int8_t qos,
                       uint8_t* msg,
                       uint16_t msg_len)
{
    int8_t retained = 0;       //保留标志位
    // uint32_t msg_len;          //数据长度
    uint8_t buf[MSG_MAX_LEN];
    int32_t buflen = sizeof(buf),len;
    MQTTString topicString = MQTTString_initializer;
    uint16_t packid = 0,packetidbk;
```

(下页继续)

(续上页)

```
    //填充主题
    topicString.cstring = (char *)topic;

    //填充数据包 ID
    if ((qos == QOS1)||(qos == QOS2))
    {
        packid = GetNextPackID();
    }
    else
    {
        qos = QOS0;
        retained = 0;
        packid = 0;
    }

    // msg_len = strlen((char *)msg);
    //推送消息
    len = MQTTSerialize_publish(buf, buflen, 0, qos, retained,
                                packid, topicString,
                                (unsigned char*)msg, msg_len);
    if (len <= 0)
        return -1;
    if (transport_sendPacketBuffer(buf, len) < 0)
        return -2;

    //质量等级 0,不需要返回
    if (qos == QOS0)
    {
        return 0;
    }

    //等级 1
```

(下页继续)

(续上页)

```
    if (qos == QOS1)
    {
        //等待 PUBACK
        if (WaitForPacket(sock,PUBACK,5) < 0)
            return -3;
        return 1;

    }
    //等级 2
    if (qos == QOS2)
    {
        //等待 PUBREC
        if (WaitForPacket(sock,PUBREC,5) < 0)
            return -3;
        //发送 PUBREL
        len = MQTTSerialize_pubrel(buf, buflen,0, packetidbk);
        if (len == 0)
            return -4;
        if (transport_sendPacketBuffer(buf, len) < 0)
            return -6;
        //等待 PUBCOMP
        if (WaitForPacket(sock,PUBREC,5) < 0)
            return -7;
        return 2;
    }
    //等级错误
    return -8;
}
```

然后在 mqttclient.h 头文件中修改相关的宏定义即可，在这里多添加了一个枚举类型的 TopicType，表示选择发布到“$dp”主题的类型，具体见代码清单 *24_4*。

代码清单 24-4mqttclient.h 头文件宏定义

```
#if    LWIP_DNS
#define   HOST_NAME       "mqtt.heclouds.com"     //服务器域名
#else
#define   HOST_NAME       "183.230.40.39"     //服务器 IP 地址
#endif


#define   HOST_PORT     6002

#define   CLIENT_ID     "518725049"          //
#define   USER_NAME     "217537"     //用户名
#define   PASSWORD      "12345"  //秘钥

#define   TOPIC         "temp_hum"      //订阅的主题
#define   TEST_MESSAGE  "test_message"  //发送测试消息

enum TopicType
{
    TopicType1 = 1,
    TopicType3 = 3,
    TopicType5 = 5
};
```

编译好程序后，就下载到开发板中，然后在 OneNET 的控制台中可以看到数据点已经被存储了，而且能看到数据流信息，具体见图 *24_11*，然后我们就能用这些数据流作为可视化的数据源。

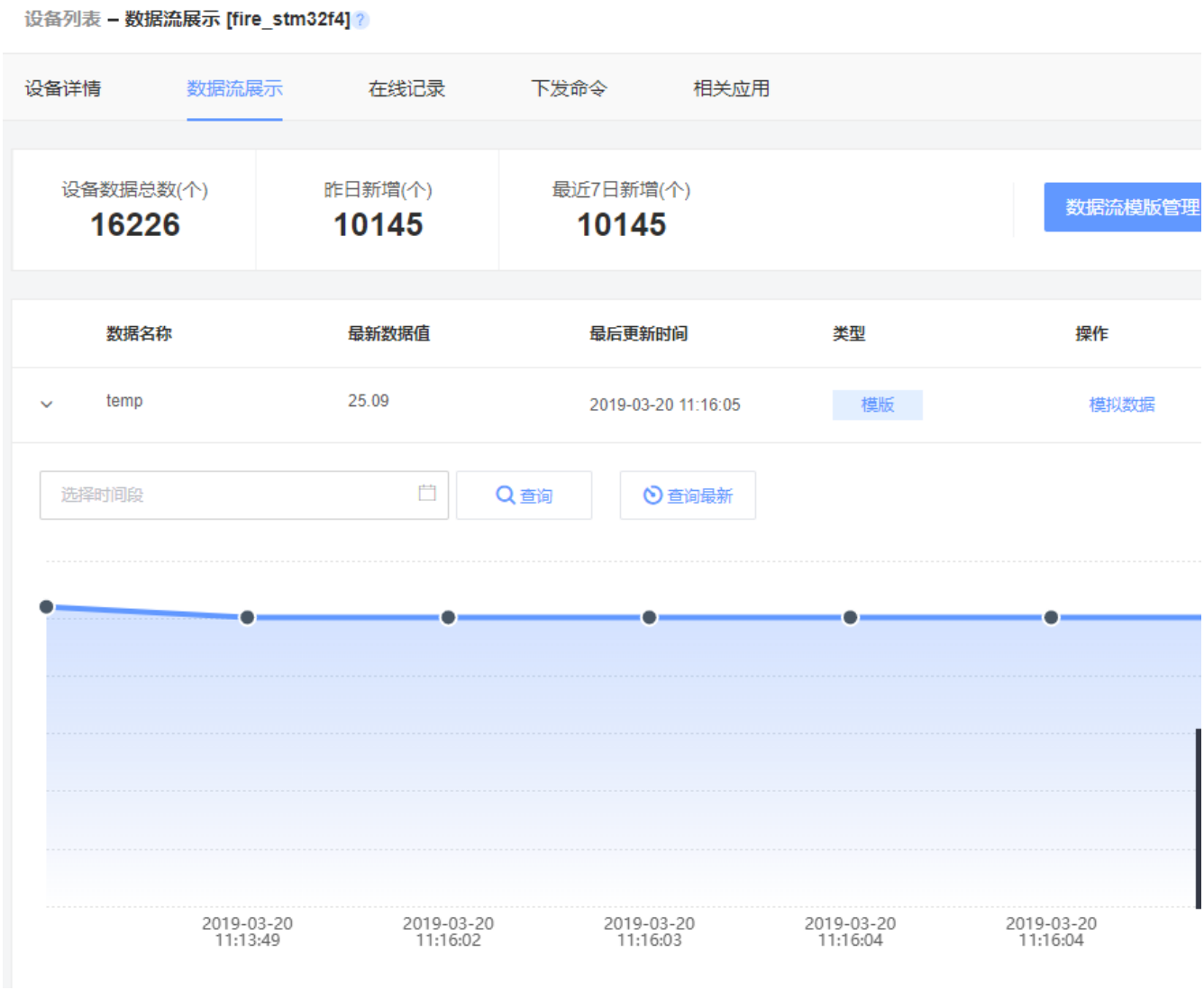


图 24-11 数据流

24.7 数据可视化

与百度云一样，数据可视化需要依赖数据的来源，而上一章我们得到数据的来源，那么就利用平台自身的可视化技术来让数据实时显示，首先进入“应用管理”页面：https://open.iot.10086.cn/develop/global/product/#/app/independent?pid=217537，点击添加应用，然后填写应用的相关信息，因为是测试应用，所以选择“私有”选项即可，具体见图 *24_12*。



图 24-12 添加应用

在应用编辑页面，选中两个仪表盘，放到画布中，然后点击仪表盘，选择“属性”，选择设备为开发板设备“fire_stm32f4”，数据流选择我们在一开始添加的数据流“temp”，选择自动刷新的频率为 3 秒，同样的另一个湿度的仪表盘也是这样子操作，此外还可以添加折线图到画布中，也是一样的选择设备与数据流，至此可视化应用的设计部署就已经完成，具体见图 24-13。然后我们就能看到数据会随着开发板发布的数据更新，演示链接：https://open.iot.10086.cn/iotbox/appsquare/appview?openid=c673cc3ee3e436d298494aba2e5c08b8。

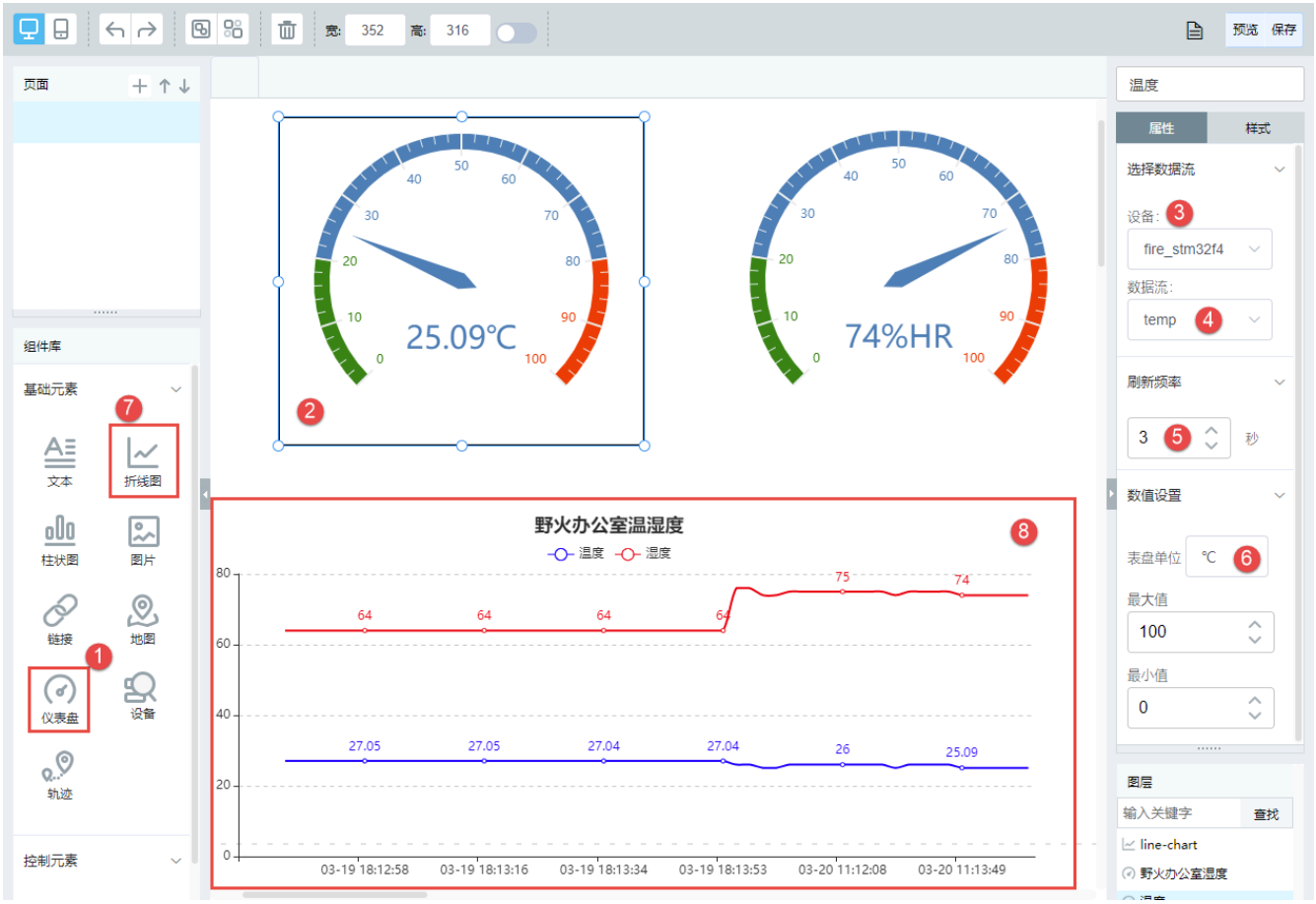

宽: 352
高: 316
预览
保存
页面
组件库
基础元素
文本
折线图
柱状图
图片
链接
地图
仪表盘
设备
轨迹
控制元素
25.09℃
74%HR
野火办公室温湿度
温度
湿度
64
64
64
75
74
27.05
27.05
27.04
27.04
26
25.09
03-19 18:12:58
03-19 18:13:16
03-19 18:13:34
03-19 18:13:53
03-20 11:12:08
03-20 11:13:49
温度
属性
样式
选择数据流
设备:
fire_stm32f4
数据流:
temp
刷新频率
3
秒
数值设置
表盘单位
℃
最大值
100
最小值
0
图层
输入关键字
查找
line-chart
野火办公室温湿度

图 24-13 应用编辑

联系我们

东莞野火电子技术有限公司

官网：www.embedfire.com

论坛：www.firebbs.cn

淘宝：fire-stm32.taobao.com

邮箱：http://firege@embedfire.com

电话：0769-33894118

关注野火公众号，可下载野火全部产品的资料（书籍、视频、程序）



野火 STM32 系列图书，关注公众号即可下载电子版

如何参与项目

本项目从代码至文档全部开源，期待大家的参与！

无论是小到发现错别字、发现代码写得不规范、有 bug 抑或是想参与到文档的编辑、投稿完整的文章，我们都热烈欢迎。

具体参与方式请阅读如下说明：如何参与野火开源项目

项目源码

本项目代码及文档源码托管在 github 和 gitee，维护时主要以 github 开发，gitee 主要是方便国内用户下载，可在这两个平台提交 **pull request** 参与到本项目。

- github 仓库地址：https://github.com/Embdefire/embed_linux_tutorial
- gitee 仓库地址：https://gitee.com/wildfireteam/embed_linux_tutorial

文档在线阅读与 pdf

本项目的文档是托管在 **readthedocs** 网站的，可在该网站在线浏览文档或者生成 **pdf** 文档离线阅读。

常见问题

版权说明

野火电子保留本项目的所有版权。

公司组织有生存的压力，因为开源而导致生存不下去，是有 GEEK 精神的人不愿看到的事情。由于我们还不熟悉各种版权条例，所以关于版权的问题还在选择中。

目前我们保留本项目的所有版权，但我们秉承开源的心是不变的。如果你使用本项目不是用于商业目的，基本不需要考虑版权问题。

我们主要担忧的商业目的如下，列出来的意思是如果用于以下目的，我们极有可能会追究版权。

- 同业竞争，目前开发板是我们主要的盈利来源，禁止把本项目用于其它开发板项目。
- 文档出版，项目中包含的文档我们随时会提交至出版社出版，所以我们保留文档出版的权利。如果你只是在自己的文章中使用了本项目文档中的内容，需要注明来源。